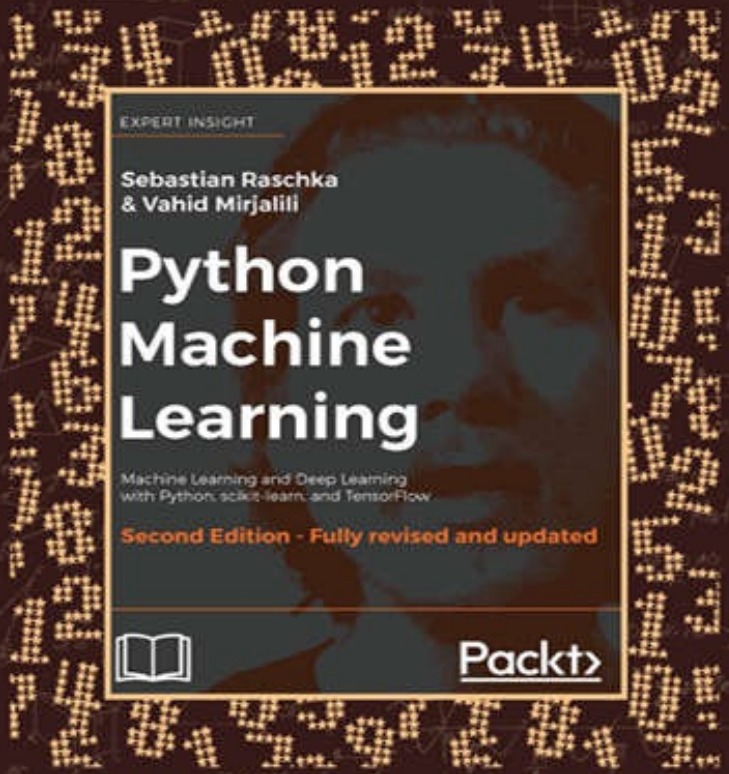
HZ BOOKS 华章IT
[PACKT] PUBLISHING
数据科学与工程技术丛书

# Python机器学习

（原书第2版）

[美] 塞巴斯蒂安·拉施卡（Sebastian Raschka）
瓦希德·米尔贾利利（Vahid Mirjalili） 著
陈斌 译




PYTHON MACHINE LEARNING
SECOND EDITION

机械工业出版社
China Machine Press

数据科学与工程技术丛书

# Python机器学习（原书第2版）

# Python Machine Learning，Second Edition

（美）塞巴斯蒂安·拉施卡（Sebastian Raschka）　（美）瓦希德·米尔贾利利（Vahid Mirjalili）　著

陈斌　译

ISBN：978-7-111-61150-9

本书纸版由机械工业出版社于2018年出版，电子版由华章分社（北京华章图文信息有限公司，北京奥维博世图书发行有限公司）在中华人民共和国境内（不包括中国香港、澳门特别行政区及中国台湾地区）制作与发行。

版权所有，侵权必究

客服热线：+ 86-10-68995265

客服信箱：service@bbbvip.com

官方网址：www.hzmedia.com.cn

新浪微博 @华章数媒

微信公众号 华章电子书（微信号：hzebook）

# 目录


译者序
关于作者
关于审校人员
前言
第1章　赋予计算机从数据中学习的能力
1.1　构建把数据转换为知识的智能机器
1.2　三种不同类型的机器学习
1.2.1　用有监督学习预测未来
1.2.2　用强化学习解决交互问题
1.2.3　用无监督学习发现隐藏结构
1.3　基本术语与符号
1.4　构建机器学习系统的路线图
1.4.1　预处理——整理数据
1.4.2　训练和选择预测模型
1.4.3　评估模型和预测新样本数据
1.5　用Python进行机器学习
1.5.1　从Python包索引安装Python和其他包
1.5.2　采用Anaconda　Python和软件包管理器
1.5.3　科学计算、数据科学和机器学习软件包
1.6　小结
第2章　训练简单的机器学习分类算法
2.1　人工神经元——机器学习早期历史一瞥
2.1.1　人工神经元的正式定义
2.1.2　感知器学习规则
2.2　在Python中实现感知器学习算法
2.2.1　面向对象的感知器API
2.2.2　在鸢尾花数据集上训练感知器模型
2.3　自适应神经元和学习收敛
2.3.1　梯度下降为最小代价函数
2.3.2　用Python实现Adaline
2.3.3　通过调整特征大小改善梯度下降
2.3.4　大规模机器学习与随机梯度下降
2.4　小结
第3章　scikit-learn机器学习分类器一览
3.1　选择分类算法
3.2　了解scikit-learn软件库的第一步——训练感知器
3.3　基于逻辑回归的分类概率建模
3.3.1　逻辑回归的直觉与条件概率
3.3.2　学习逻辑代价函数的权重
3.3.3　把转换的Adaline用于逻辑回归算法
3.3.4　用scikit-learn训练逻辑回归模型

3.3.5 通过正则化解决过拟合问题
3.4 支持向量机的最大余量分类
3.4.1 最大边际的直觉
3.4.2 用松弛变量处理非线性可分
3.4.3 其他的scikit-learn实现
3.5 用核支持向量机求解非线性问题
3.5.1 处理线性不可分数据的核方法
3.5.2 利用核技巧，发现高维空间的分离超平面
3.6 决策树学习
3.6.1 最大限度地获取信息——获得最大收益
3.6.2 构建决策树
3.6.3 通过随机森林组合多个决策树
3.7 K-近邻——一种懒惰的学习算法
3.8 小结
第4章 构建良好的训练集——预处理
4.1 处理缺失数据
4.1.1 识别数据中的缺失数值
4.1.2 删除缺失的数据
4.1.3 填补缺失的数据
4.1.4 了解scikit-learn评估器API
4.2 处理分类数据
4.2.1 名词特征和序数特征
4.2.2 映射序数特征
4.2.3 分类标签编码
4.2.4 为名词特征做热编码
4.3 分裂数据集为独立的训练集和测试集
4.4 把特征保持在同一尺度上
4.5 选择有意义的特征
4.5.1 L1和L2正则化对模型复杂度的惩罚
4.5.2 L2正则化的几何解释
4.5.3 L1正则化的稀疏解决方案
4.5.4 为序数特征选择算法
4.6 用随机森林评估特征的重要性
4.7 小结
第5章 通过降维压缩数据
5.1 用主成分分析实现无监督降维
5.1.1 主成分分析的主要步骤
5.1.2 逐步提取主成分
5.1.3 总方差和解释方差
5.1.4 特征变换
5.1.5 scikit-learn的主成分分析
5.2 基于线性判别分析的有监督数据压缩
5.2.1 主成分分析与线性判别分析

5.2.2 线性判别分析的内部逻辑
5.2.3 计算散布矩阵
5.2.4 在新的特征子空间选择线性判别式
5.2.5 将样本投影到新的特征空间
5.2.6 用scikit-learn实现的LDA
5.3 非线性映射的核主成分分析
5.3.1 核函数与核技巧
5.3.2 用Python实现核主成分分析
5.3.3 投影新的数据点
5.3.4 scikit-learn的核主成分分析
5.4 小结
第6章 模型评估和超参数调优的最佳实践
6.1 用管道方法简化工作流
6.1.1 加载威斯康星乳腺癌数据集
6.1.2 集成管道中的转换器和评估器
6.2 使用k折交叉验证评估模型的性能
6.2.1 抵抗方法
6.2.2 k折交叉验证
6.3 用学习和验证曲线调试算法
6.3.1 用学习曲线诊断偏差和方差问题
6.3.2 用验证曲线解决过拟合和欠拟合问题
6.4 通过网格搜索为机器学习模型调优
6.4.1 通过网格搜索为超参数调优
6.4.2 以嵌套式交叉验证来选择算法
6.5 比较不同的性能评估指标
6.5.1 含混矩阵分析
6.5.2 优化分类模型的准确度和召回率
6.5.3 绘制受试者操作特性图
6.5.4 多元分类评分指标
6.6 处理类的不平衡问题
6.7 小结
第7章 综合不同模型的组合学习
7.1 集成学习
7.2 采用多数票机制的集成分类器
7.2.1 实现基于多数票的简单分类器
7.2.2 用多数票原则进行预测
7.2.3 评估和优化集成分类器
7.3 套袋——基于导引样本构建分类器集成
7.3.1 套袋简介
7.3.2 应用套袋技术对葡萄酒数据集中的样本分类
7.4 通过自适应增强来利用弱学习者
7.4.1 增强是如何实现的
7.4.2 用scikit-learn实现AdaBoost

7.5 小结
第8章 应用机器学习于情感分析
8.1 为文本处理预备好IMDb电影评论数据
8.1.1 获取电影评论数据集
8.1.2 把电影评论数据预处理成更方便格式的数据
8.2 词袋模型介绍
8.2.1 把词转换成特征向量
8.2.2 通过词频逆反文档频率评估单词相关性
8.2.3 清洗文本数据
8.2.4 把文档处理为令牌
8.3 训练文档分类的逻辑回归模型
8.4 处理更大的数据集——在线算法和核心学习
8.5 具有潜在狄氏分配的主题建模
8.5.1 使用LDA分解文本文档
8.5.2 LDA与scikit-learn
8.6 小结
第9章 将机器学习模型嵌入网络应用
9.1 序列化拟合scikit-learn评估器
9.2 搭建SQLite数据库存储数据
9.3 用Flask开发网络应用
9.3.1 第一个Flask网络应用
9.3.2 表单验证与渲染
9.4 将电影评论分类器转换为网络应用
9.4.1 文件与文件夹——研究目录树
9.4.2 实现主应用app.py
9.4.3 建立评论表单
9.4.4 创建一个结果页面的模板
9.5 在面向公众的服务器上部署网络应用
9.5.1 创建PythonAnywhere账户
9.5.2 上传电影分类应用
9.5.3 更新电影分类器
9.6 小结
第10章 用回归分析预测连续目标变量
10.1 线性回归简介
10.1.1 简单线性回归
10.1.2 多元线性回归
10.2 探索住房数据集
10.2.1 加载住房数据
10.2.2 可视化数据集的重要特点
10.2.3 用关联矩阵查看关系
10.3 普通最小二乘线性回归模型的实现
10.3.1 用梯度下降方法求解回归参数
10.3.2 通过scikit-learn估计回归模型的系数

10.4 利用RANSAC拟合稳健的回归模型
10.5 评估线性回归模型的性能
10.6 用正则化方法进行回归
10.7 将线性回归模型转换为曲线——多项式回归
10.7.1 用scikit-learn增加多项式的项
10.7.2 为住房数据集中的非线性关系建模
10.8 用随机森林处理非线性关系
10.8.1 决策树回归
10.8.2 随机森林回归
10.9 小结
第11章 用聚类分析处理无标签数据
11.1 用k-均值进行相似性分组
11.1.1 scikit-learn的k-均值聚类
11.1.2 k-均值++——更聪明地设置初始聚类中心的方法
11.1.3 硬聚类与软聚类
11.1.4 用肘法求解最佳聚类数
11.1.5 通过轮廓图量化聚类质量
11.2 把集群组织成有层次的树
11.2.1 以自下而上的方式聚类
11.2.2 在距离矩阵上进行层次聚类
11.2.3 热度图附加树状图
11.2.4 scikit-learn凝聚聚类方法
11.3 通过DBSCAN定位高密度区域
11.4 小结
第12章 从零开始实现多层人工神经网络
12.1 用人工神经网络为复杂函数建模
12.1.1 单层神经网络扼要重述
12.1.2 介绍多层神经网络体系
12.1.3 利用正向传播激活神经网络
12.2 识别手写数字
12.2.1 获取MNIST数据集
12.2.2 实现一个多层感知器
12.3 训练人工神经网络
12.3.1 逻辑成本函数的计算
12.3.2 开发反向传播的直觉
12.3.3 通过反向传播训练神经网络
12.4 关于神经网络的收敛性
12.5 关于神经网络实现的最后几句话
12.6 小结
第13章 用TensorFlow并行训练神经网络
13.1 TensorFlow与模型训练的性能
13.1.1 什么是TensorFlow
13.1.2 如何学习TensorFlow

13.1.3 学习TensorFlow的第一步
13.1.4 使用阵列结构
13.1.5 用TensorFlow的底层API开发简单的模型
13.2 用TensorFlow的高级API高效率地训练神经网络
13.2.1 用TensorFlow的Layers API构建多层神经网络
13.2.2 用Keras研发多层神经网络
13.3 多层网络激活函数的选择
13.3.1 逻辑函数回顾
13.3.2 在多元分类中调用softmax函数评估类别概率
13.3.3 利用双曲正切拓宽输出范围
13.3.4 修正线性单元激活函数
13.4 小结
第14章 深入探讨TensorFlow的工作原理
14.1 TensorFlow的主要功能
14.2 TensorFlow的排序与张量
14.3 了解TensorFlow的计算图
14.4 TensorFlow中的占位符
14.4.1 定义占位符
14.4.2 为占位符提供数据
14.4.3 用batchsizes为数据阵列定义占位符
14.5 TensorFlow中的变量
14.5.1 定义变量
14.5.2 初始化变量
14.5.3 变量范围
14.5.4 变量复用
14.6 建立回归模型
14.7 在TensorFlow计算图中用张量名执行对象
14.8 在TensorFlow中存储和恢复模型
14.9 把张量转换成多维数据阵列
14.10 利用控制流构图
14.11 用TensorBoard可视化图
14.12 小结
第15章 深度卷积神经网络图像识别
15.1 构建卷积神经网络的模块
15.1.1 理解CNN与学习特征的层次
15.1.2 执行离散卷积
15.1.3 子采样
15.2 拼装构建CNN
15.2.1 处理多个输入或者彩色频道
15.2.2 通过淘汰正则化神经网络
15.3 用TensorFlow实现深度卷积神经网络
15.3.1 多层CNN体系结构
15.3.2 加载和预处理数据

15.3.3 用TensorFlow的低级API实现CNN模型
15.3.4 用TensorFlow的Layers API实现CNN
15.4 小结
第16章 用递归神经网络为序列数据建模
16.1 序列数据
16.1.1 序列数据建模——顺序很重要
16.1.2 表示序列
16.1.3 不同类别的序列建模
16.2 用于序列建模的RNN
16.2.1 理解RNN的结构和数据流
16.2.2 在RNN中计算激活值
16.2.3 长期交互学习的挑战
16.2.4 LSTM单元
16.3 用TensorFlow实现多层RNN序列建模
16.4 项目一：利用多层RNN对IMDb电影评论进行情感分析
16.4.1 准备数据
16.4.2 嵌入式
16.4.3 构建一个RNN模型
16.4.4 情感RNN类构造器
16.4.5 build方法
16.4.6 train方法
16.4.7 predict方法
16.4.8 创建SentimentRNN类的实例
16.4.9 训练与优化情感分析RNN模型
16.5 项目二：用TensorFlow实现字符级RNN语言建模
16.5.1 准备数据
16.5.2 构建字符级RNN语言模型
16.5.3 构造器
16.5.4 build方法
16.5.5 train方法
16.5.6 sample方法
16.5.7 创建和训练CharRNN模型
16.5.8 处于取样状态的CharRNN模型
16.6 总结

# 译者序

人工智能的研究从20世纪40年代已经开始，在近80年的发展中经历了数次大起大落。自从2016年AlphaGo战胜顶尖的人类围棋选手之后，人工智能再一次进入了人们的视野，成为当今的热门话题。各大互联网公司都投入了大量的资源研究和开发自动驾驶、人脸识别、语音识别和机器翻译等技术。人类已经开始担忧人工智能可能带来的各种影响。人工智能的最新发展可以说是“古树发新枝”，到底是什么原因使沉寂多年的人工智能技术焕发了青春的活力呢？

首先，移动互联网的飞速发展产生了海量的数据，使我们有机会更加深入地认识社会、探索世界、掌握规律。其次，大数据技术为我们提供了有力的技术手段，使我们可以面对瞬息万变的市场，有效地存储和处理海量数据。再次，计算技术特别是GPU的广泛应用使算力有了大幅度的提升，以前需要几天的运算如今只需要几分钟或几秒钟，这也为人工智能和机器学习的普及与应用提供了计算基础。在这几项技术发展的基础之上，深度学习技术终于破茧而出，成为引领人工智能发展的重要力量。

本书英文版在美国出版后备受欢迎，究其原因，除了机器学习是所有技术人员关注的焦点以外，还在于本书系统性地梳理和分析了机器学习的各种经典算法，最为重要的是作者通过Python语言以具体代码示例深入浅出地介绍了各种算法的应用方法。如果你想了解机器学习并掌握机器学习的具体技术，那就请翻开此书，通过一个又一个案例领略机器学习的风采。所以这本书既是一本初步了解机器学习的启蒙读物，也是一本让你从初学者变成AI专家的教练示范材料。

毋庸置疑，人工智能（AI）、区块链（BlockChain）、云计算（Cloud）、大数据（Big Data）、物联网（IoE）这五项技术（简写为ABCDE）已经成为计算机和互联网技术未来发展的五大核心动力。特别是人工智能技术，它将是继蒸汽机、电力、计算机、互联网之后的又一股重要的革命性力量。之前的几次革命解放的是我们的四肢，而人工智能解放的将是我们的头脑。

# 关于作者

塞巴斯蒂安·拉施卡是畅销书《Python机器学习》的作者，他在Python编程方面拥有多年经验，曾经就如何实际应用数据科学、机器学习和深度学习做过数次讲座，包括在SciPy（重要的Python科学计算会议）上做的机器学习教学。

虽然塞巴斯蒂安的学术研究项目主要集中在解决计算生物学的问题方面，但他一般喜欢在数据科学、机器学习和Python方面进行写作和讨论，而且他总是乐于帮助别人在不需要机器学习的背景下开发数据驱动的解决方案。

他的工作和贡献使他得到了2016~2017学年系杰出研究生奖，以及《ACM计算评论》2016年度最佳奖。

在闲暇时间里，塞巴斯蒂安喜欢为开源项目做出一些贡献，而他所实现的方法现已成功地用于像Kaggle这样的机器学习竞赛。

我想借此机会感谢伟大的Python社区和开源软件包的开发人员，他们为我从事科学研究和数据科学创造了完美的环境。

另外，我要感谢我的父母，他们始终鼓励和支持我追求我热爱的道路和事业。

特别感谢scikit-learn的核心开发人员。作为这个项目的贡献者，我很高兴能够与伟大的人士合作，他们不仅在机器学习方面非常博学，而且也是优秀的程序员。

瓦希德·米尔贾利利拥有机械工程博士学位，从事大规模分子结构计算模拟新方法的研究。目前他在密歇根州立大学计算机科学与工程系工作，致力于把机器学习应用到各种计算机视觉研究项目中。

瓦希德以Python作为编程语言的首选，在学术和研究生涯中积累了丰富的Python编码经验。他为密歇根州立大学的工程专业教授Python编程，这让他有机会帮助学生理解不同的数据结构并在Python中开发高效的代码。

虽然瓦希德的广泛研究兴趣集中在深度学习和计算机视觉应用方面，但他对利用深度学习技术来扩展生物识别数据（如人脸图像）中的隐私保护尤其感兴趣，目的是确保信息不会超出用户想要披露的范围。

此外，他还与一群从事自动驾驶汽车工作的工程师合作，设计了用于检测行人的多光谱图像融合神经网络模型。

我想感谢博士导师阿伦·罗斯博士，他为我提供了在其实验室研究新问题的机会。我还要感谢毗湿奴·伯德利博士，他激发了我对深度学习的兴趣，并让我揭开了核心概念的神秘面纱。

# 关于审校人员

贾里德·霍夫曼是一位企业家、游戏玩家、讲故事的人以及机器学习的狂热分子和数据库迷。在过去的10年中，他致力于开发软件和分析数据。之前的工作涉及网络安全、金融系统、商业智能、Web服务、开发者工具和业务战略等不同的领域。作为Minecraft公司数据科学团队的创始人，他近几年主要关注大数据和机器学习。工作以外，他会玩游戏或者与朋友和家人一起享受美丽的太平洋西北地区的美好生活。

感谢Packt给我机会参与编纂如此伟大的著作，感谢太太的长期鼓励，

感谢女儿当我在深夜里审校此书和调试代码时能安静地酣睡。

孙怀恩于台湾交通大学取得统计学硕士学位。目前，他作为数据科学家在PEGATRON公司分析生产线的数据。机器学习和深度学习是他主要的研究领域。

# 前言

通过新闻媒体的报道，你可能已经了解到机器学习已经成为当代最激动人心的技术。像谷歌、Facebook、苹果、Amazon和IBM这样的大公司基于各自的考虑，已经在机器学习的研究和应用方面投入了巨资。机器学习似乎已经成为流行词，但这绝不是昙花一现。这个激动人心的领域开启了许多新的可能性，已经在日常生活中不可或缺。智能手机的语音助手、为客户推荐合适的产品、防止信用卡欺诈、过滤垃圾邮件、检测和诊断疾病等都是明证。

如果有志于从事深度学习，想更好地解决问题或开展深度学习方面的研究，那么这本书就是为你而写。然而，深度学习背后的理论概念可能艰深难懂。但近几年已经出版了许多机器学习方面的著作，阅读它们有助于通过研发强大的机器学习算法走上机器学习之路。

熟悉机器学习的示例代码及应用是深入该领域的捷径。通过具体的示例学以致用有助于阐明宽泛的概念。请记住，能力越大责任越大！除了用Python和基于Python的机器学习软件库掌握实践经验外，本书还介绍了机器学习算法背后的数学概念，这对于成功地使用机器学习必不可少。这使得本书有别于其他的纯实战书籍。本书将对机器学习概念的必要细节进行讨论，同时对机器学习算法的工作原理、使用方法以及最为重要的如何避免掉入最常见的陷阱，提供直观且翔实的解释。

如果在谷歌专业网站以“机器学习”作为关键词进行搜索，结果会找到180万个出版物。当然我们无法对过去60年来所出现的各种不同算法和应用逐一进行考证。然而，本书将开始一个激动人心的旅程，涵盖所有重要的主题和概念，让你在该领域捷足先登。如果你发现所提供的知识还不能解渴，没关系，本书还引用了许多其他有用的资源，供你追踪该领域的精要突破。

如果已经详细研究了机器学习理论，那么本书可以教你如何把知识付诸实践。如果以前用过机器学习技术，想更深入地了解其工作原理，那么本书就是为你而备。如果机器学习对你是全新的领域，那么不必担心，你更有理由为此感到兴奋。我保证机器学习将会改变你解决问题的思路，并让你看到如何通过释放数据的力量来解决问题。

在深入机器学习领域之前，先回答一个最重要的问题：“为什么要用Python？”答案很简单：Python功能强大且易于取得。Python已成为数据科学最常用的编程语言，因为它可以让我们忘记编程的冗长乏味，同时提供了可以把想法落地、概念直接付诸行动的环境。

我们认为，对机器学习的研究使我们成为更好的科学家、思想家和问题解决者。本书将与你分享这些知识。知识是要靠学习获得的。学习的关键在于热情，而要真正掌握技能只能通过实践。前面的路或许崎岖不平，有些话

题可能颇具挑战性，但我们希望你能抓住这个机会，更多地考虑本书所带来的回报。请记住，我们共同踏上这个旅程，本书将为你的军火库添加许多强大的武器，让你以数据驱动的方式来解决最棘手的问题。

## 本书内容

第1章介绍了机器学习在解决不同问题时的主要应用领域。另外，还讨论了构建典型的机器学习模型所需要的基本步骤，从而形成一条导引后续各章节的管道。

第2章追溯了机器学习的起源，介绍了二元感知器、分类器和自适应线性神经元。对模式分类的基本原理作了简单介绍，同时关注算法优化和机器学习的交互。

第3章描述了基本的机器学习分类算法，并用最流行和全面的开源机器学习软件库scikit-learn提供了实际案例。

第4章讨论了如何解决未处理数据集中最常见的问题，如数据缺失。也讨论了用来识别数据集中信息量最大特性的几种方法，并教你如何将不同类型的变量作为机器学习算法的适当输入。

第5章描述了减少数据集中的特征数，同时保留大部分有用和识别性信息的基本技术。讨论了基于主成分分析的标准降维方法，并将其与有监督学习和非线性变换技术进行了比较。

第6章讨论了在预测模型的性能评价中该做和不该做什么。此外，还讨论了模型性能评估的不同度量以及优化机器学习算法的技术。

第7章介绍了有效结合多种学习算法的不同概念，讲解了如何建立专家小组来克服个别学习者的弱点，从而产生更准确更可靠的预测。

第8章讨论了将文本数据转换为有意义的机器学习算法，以根据文本内容预测人们意见的基本步骤。

第9章继续使用前一章中的预测模型，并介绍了使用嵌入式机器学习模型开发网络应用的基本步骤。

第10章讨论根据目标和响应变量之间的线性关系建模，从而进行连续预测的基本技术。在介绍了不同的线性模型之后，还讨论了多项式回归和基于树的建模方法。

第11章将焦点转移到机器学习的其他子领域，即无监督学习。用来自于三个基本聚类家族的算法来寻找一组拥有一定程度相似性的对象。

第12章扩展了基于梯度的优化概念，该概念在第2章中介绍过，用来在Python中构建基于常见的强大的多层神经网络的反向传播算法。

第13章基于前一章的知识，为更有效地训练神经网络提供实用指南。该章的重点是TensorFlow，这是一个开源的Python软件库，允许我们充分利用现代的多核GPU。

第14章更详细地介绍了TensorFlow的计算图和会话的核心概念。另外，该章还介绍了如何保存会话以及可视化神经网络图等主题，这对本书其他章节的学习会非常有用。

第15章讨论了深度神经网络的结构体系，这些结构体系已成为计算机视觉和图像识别领域（卷积神经网络）的新标准。本章讨论了作为特征提取器的卷积层之间的主要概念，并将卷积神经网络体系结构应用于图像识别，以获得近乎完美的识别准确度。

第16章介绍了深度学习的另外一种常用的神经网络结构体系，它特别适合于处理序列数据和时间序列数据。在该章中，我们应用不同的递归神经网络体系结构来处理文本数据。作为热身练习，我们将从一个情感分析开始，并学习如何生成全新的文本。

## 阅读本书需要的材料

要执行本书的示例代码，需要在MacOS、Linux或者Microsoft Windows操作系统上安装Python 3.6.0或更新的版本。本书将持续使用包括SciPy、NumPy、scikit-learn、Matplotlib和pandas在内的Python的科学计算软件库。

第1章将为建立Python环境及其核心库提供指令和有用的提示。我们将逐渐添加更多的软件库。另外，会分别在不同的章节提供安装指令：用于自然语言处理的NLTK库（第8章），Flask网络框架库（第9章），Seaborn统计数据可视化库（第10章）和有关图像处理单元的有效神经元网络训练的TensorFlow（第13～16章）。

## 本书的目标读者

如果你想知道如何开始用Python回答数据方面的关键问题，那就开始学习本书吧！不论是从头学起，还是要扩展数据科学方面的知识，本书都是不可或缺的重要资源。

## 下载示例代码及彩色图像

本书的示例源码及所有截图和样图，可以从http://www.packtpub.com通过个人账号下载，也可以访问华章公司官网http://www.hzbook.com，通过注册并登录个人账号下载。

本书的代码包也托管在GitHub上，地址如下：

https://github.com/PacktPublishing/Python-Machine-Learning-Second-Edition。书中用到的彩色图像截图或者图表的PDF文件也可以从http://www.packtpub.com/sites/default/files/downloads/PythonMachineLearningSe下载。

# 第1章　赋予计算机从数据中学习的能力

机器学习是使数据具有意义的算法的应用和科学，也是计算机科学中最令人兴奋的领域！在数据丰沛的时代，计算机可以通过自我学习获得算法把数据转化为知识。近年来涌现出了许多强大的机器学习开源软件库，现在是进入该领域的最佳时机，掌握强大的算法可以从数据中发现模式并预测未来。

本章将讨论机器学习的主要概念、不同类型及相关术语，为利用机器学习技术成功地解决实际问题奠定基础。

本章将主要涵盖下述几个方面：

·机器学习的基本概念

·三种类型的机器学习及基本术语

·成功设计机器学习系统的基石

·为数据分析和机器学习安装和配置Python

## 1.1 构建把数据转换为知识的智能机器

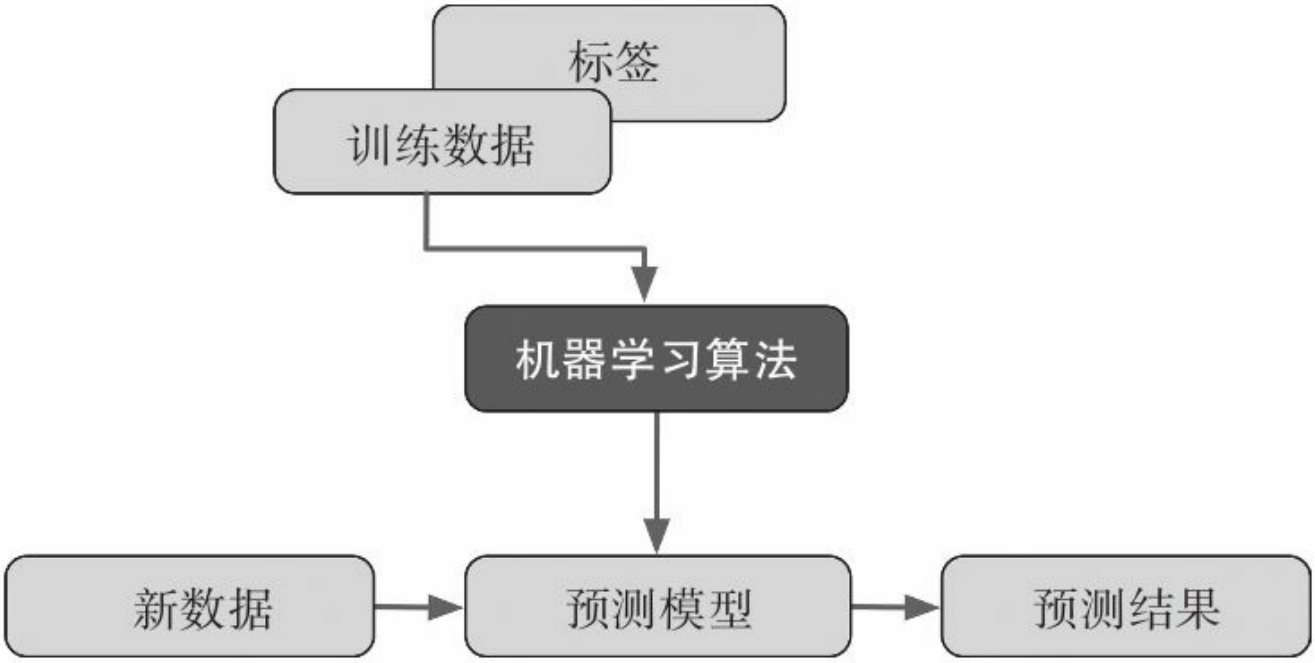
在当今的科技时代，大量结构化和非结构化数据是我们的丰富资源。机器学习在二十世纪下半叶演变为人工智能（AI）的一个分支，它涉及从数据中通过自我学习获得算法以进行预测。机器学习并不需要先在大量的数据中进行人工分析，然后提取规则并建立模型，而是提供了一种更有效的方法来捕获数据中的知识，逐步提高预测模型的性能，以完成数据驱动的决策。机器学习不仅在计算机科学研究中越来越重要，在日常生活中也发挥出越来越大的作用。归功于机器学习，今天才会有强大的垃圾邮件过滤、方便的文本和语音识别、可靠的网络搜索引擎、具有挑战性的下棋程序，并有希望在不久的将来可以享受安全和高效的自动驾驶。

## 1.2 三种不同类型的机器学习

本节将讨论有监督、无监督和强化三种不同类型的机器学习，分析它们之间的根本差别，并用概念性的例子开发一个可以解决实际问题的应用：

### 1.2.1 用有监督学习预测未来

有监督学习的主要目标是从有标签的训练数据中学习模型，以便对未知或未来的数据做出预测。“监督”一词指的是已经知道样本所需要的输出信号或标签。

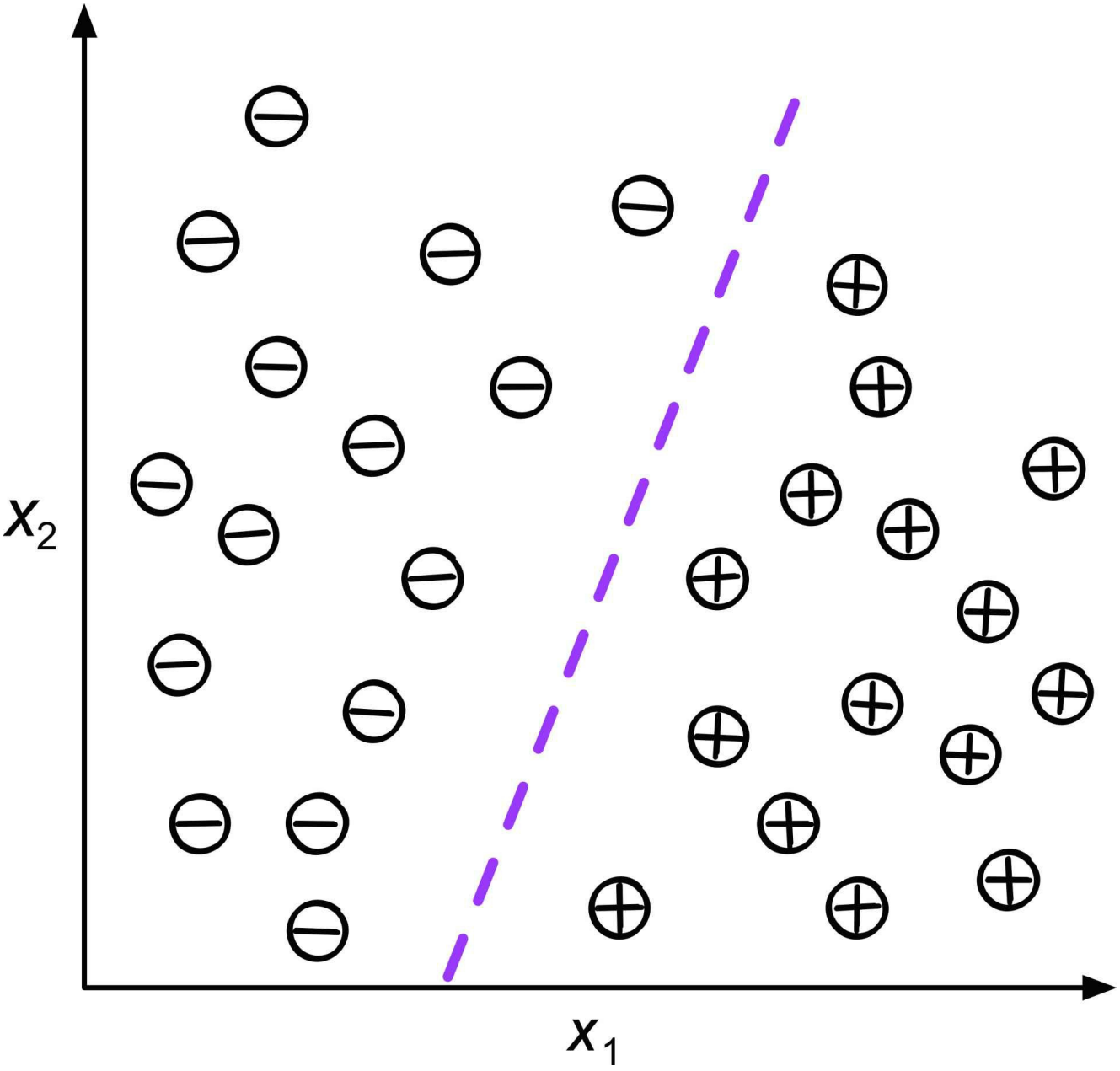



以垃圾邮件过滤为例，可以采用有监督的机器学习算法，基于打过标签的电子邮件语料库来训练模型，然后用模型来预测新邮件是否属于垃圾邮件。带有离散分类标签的有监督学习也被称为分类任务，例如上述的垃圾邮件过滤。有监督学习的另一个子类被称为回归，其结果信号是连续的数值。

#### 预测标签的分类

分类是有监督学习的一个分支，其目的是根据过去的观测结果来预测新样本的分类标签。这些分类标签是离散的无序值，可以理解为样本组成员的关系。前面提到的邮件垃圾检测就是典型的二元分类任务，机器学习算法学习规则以区分垃圾和非垃圾邮件。

但是，数据集的分类并非都是二元的。有监督学习算法经过学习得到的预测模型可以将训练集中出现过的标签分配给尚未标记的新样本。多元分类任务的典型例子是识别手写字符。首先，收集包含字母表中所有字母的多个手写示例形成训练集。然后，当用户通过输入设备提供一个新的手写字符时，预测模型能够准确地将其识别为字母表中的正确字母。然而，如果0～9之间的数字不是训练集的一部分，那么机器学习系统将无法正确地识别。




下图将通过30个训练样本阐述二元分类任务的概念，其中15个标签为阴性（-），另外15个标签为阳性（+）。该数据集为二元，意味着每个样本都与$x_1$或$x_2$的值相关。现在，可以通过机器学习算法来形成一组规则，用一条断线来代表决策边界以区分两类数据，并根据$x_1$和$x_2$的值为新数据分类。

### 预测连续结果的回归

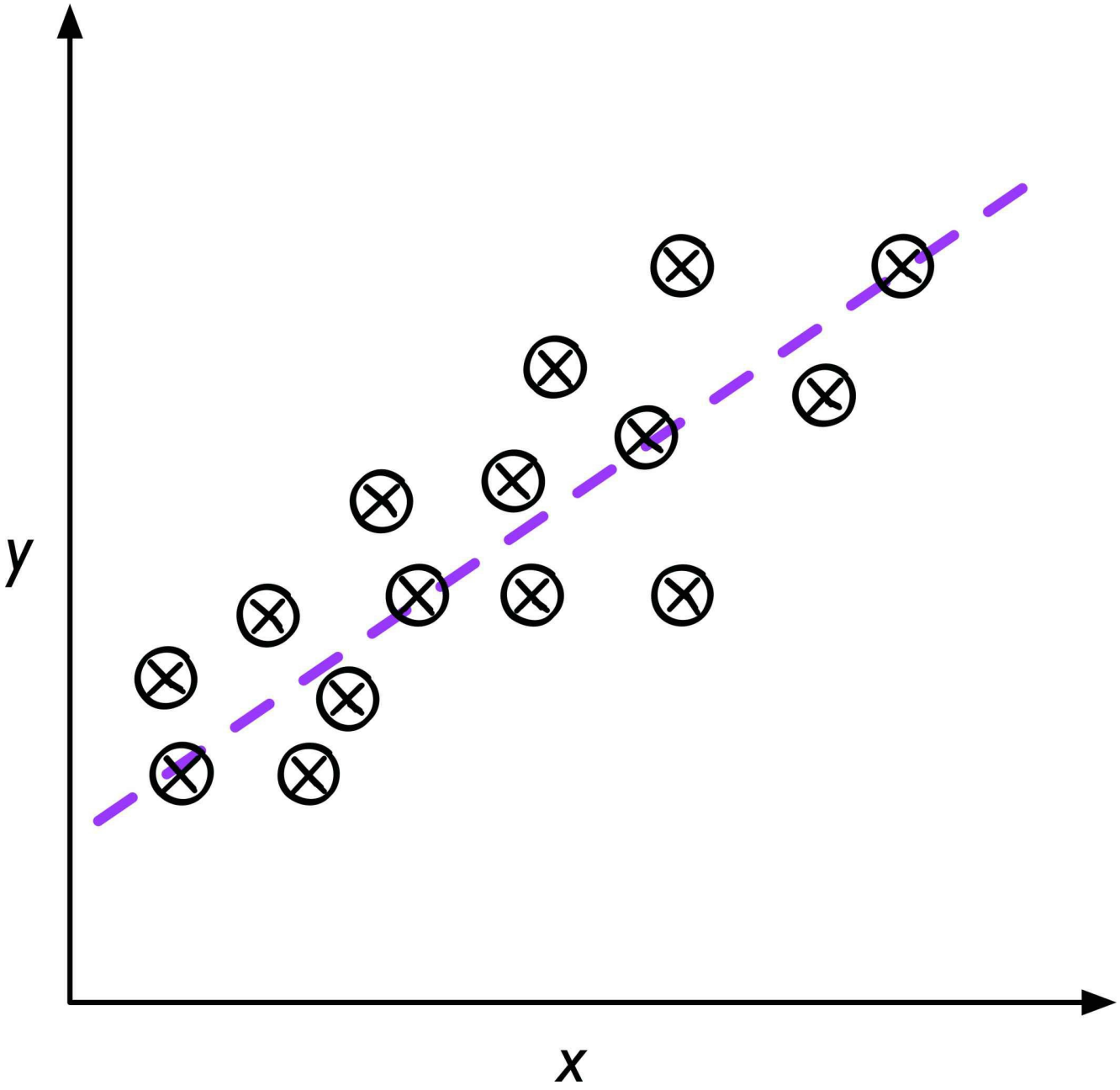
上一节学习到分类任务是为样本分配无序的分类标签。第二类有监督学习是对连续结果的预测，也称为回归分析。回归分析包括一些预测（解释）变量和一个连续的响应变量（结果或目标），试图寻找那些能够预测结果的变量之间的关系。

以预测学生SAT数学成绩为例。假设学习时间与考试成绩相关，可以用该关系训练数据学习建模，用将来打算参加该项考试学生的学习时间来预测

其考试成绩。

1886年，弗朗西斯·高尔顿在其论文《回归平均的遗传身高》中首次提到回归一词。高尔顿描述了一种生物学现象，即种群身高的变化不会随时间的推移而增加。他观察到父母的身高不会遗传给自己的孩子，相反，孩子的身高会回归种群的均值。

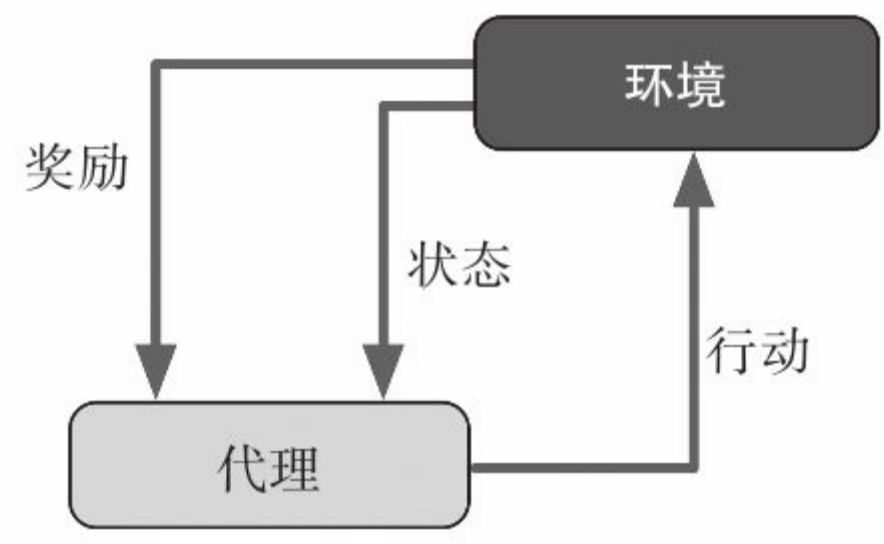
下图说明了线性回归的概念。给定预测变量x和响应变量y，对数据进行线性拟合，谋求样本点和拟合线之间的平均距离最小（距离方差）。现在可以用从该数据中学习到的截距和斜率来预测新数据的结果变量：


y
x

## 1.2.2 用强化学习解决交互问题

另一种机器学习是强化学习。强化学习的目标是开发系统或代理，通过它们与环境的交互来提高其预测性能。当前环境状态的信息通常包含所谓的奖励信号，可以把强化学习看作是与有监督学习相关的领域。然而强化学习的反馈并非标定过的正确标签或数值，而是奖励函数对行动的度量。代理可以与环境交互完成强化学习，通过探索性的试错或深思熟虑的规划来最大化这种奖励。

强化学习的常见例子是国际象棋。代理根据棋盘的状态或环境来决定一系列的行动，奖励为比赛结果的输赢：




强化学习有许多不同的子类。然而，大逻辑是强化学习代理试图通过一系列与环境的交互来最大化奖励。每种状态都可以与正面或负面的奖励相关联，奖励可以定义为完成一个总目标，如赢棋或输棋。例如，国际象棋每步的结果都可以认为是一种不同的环境状态。为进一步探索国际象棋的案例，观察一下棋盘上与正面事件相关联的某些位置，比如吃掉对手或威胁皇后的棋子。棋盘上的其他位置与负面事件相关联，例如在接下来的回合中输给对手一个棋子。实际上并不是每个回合都会有棋子被吃掉，强化学习涉及根据即时或延迟反馈来最大化奖励，从而学习一系列的走法。

本节对强化学习做了基本概述，请注意强化学习应用已超出了本书的范围，我们主要关注分类、回归分析和聚类。

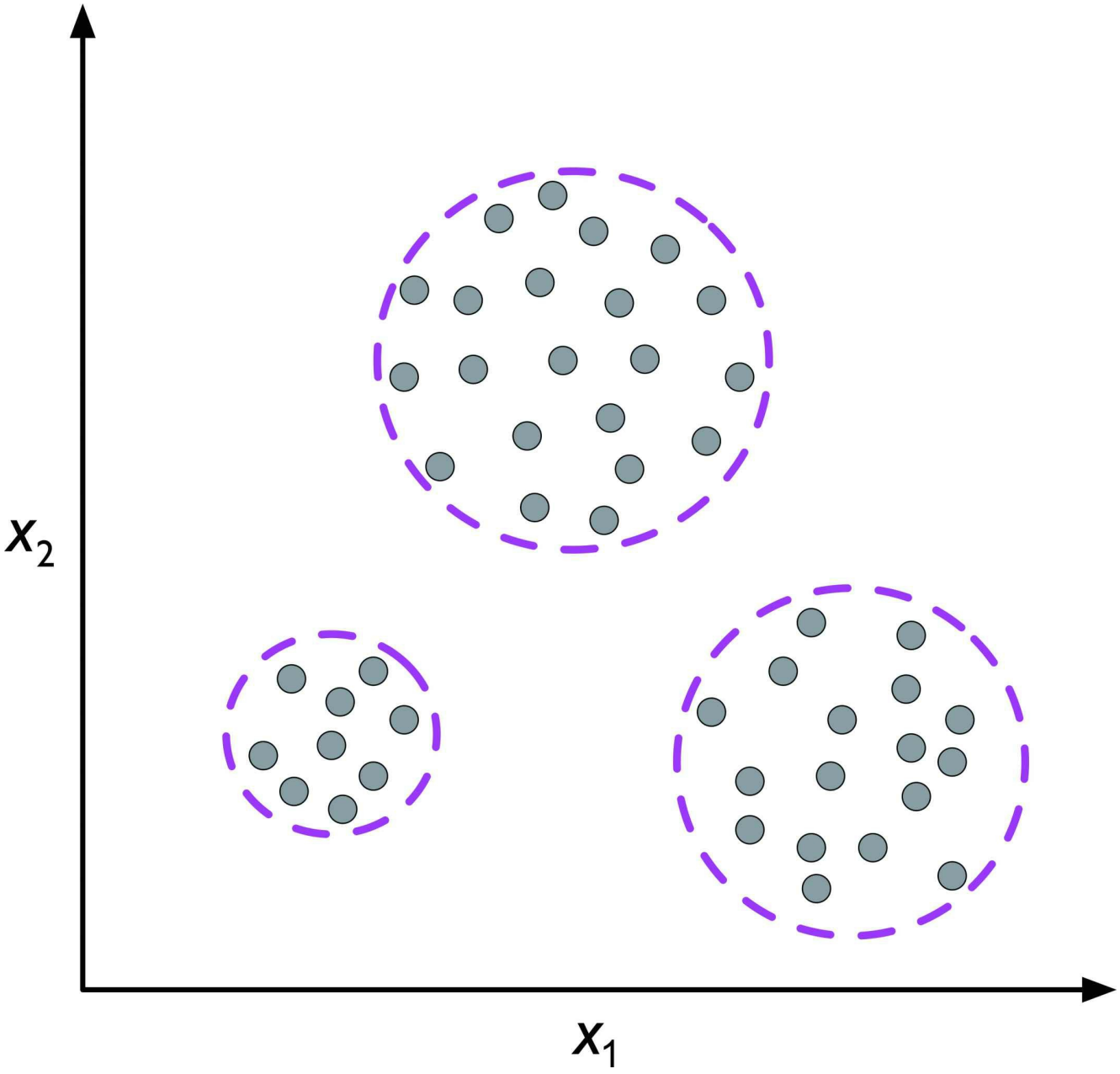
### 1.2.3 用无监督学习发现隐藏结构

在有监督学习中训练模型时，事先知道正确的答案；在强化学习过程中，定义了代理对特定动作的奖励。然而，无监督学习处理的是无标签或结构未知的数据。使用无监督学习技术，可以在没有已知结果变量或奖励函数的指导下，探索数据结构以提取有意义的信息。

#### 1.2.3.1 寻找聚类的子集

聚类是探索性的数据分析技术，可以在事先不了解组员的情况下，将信息分成有意义的组群。为在分析过程中出现的每个群定义一组对象，它们之间都具有一定程度的相似性，但与其他群中对象的差异性更大，这就是为什么聚类有时也被称为无监督分类。聚类是构造信息和从数据中导出有意义关系的一种有用的技术。例如，它允许营销人员根据自己的兴趣发现客户群，以便制订不同的市场营销计划。

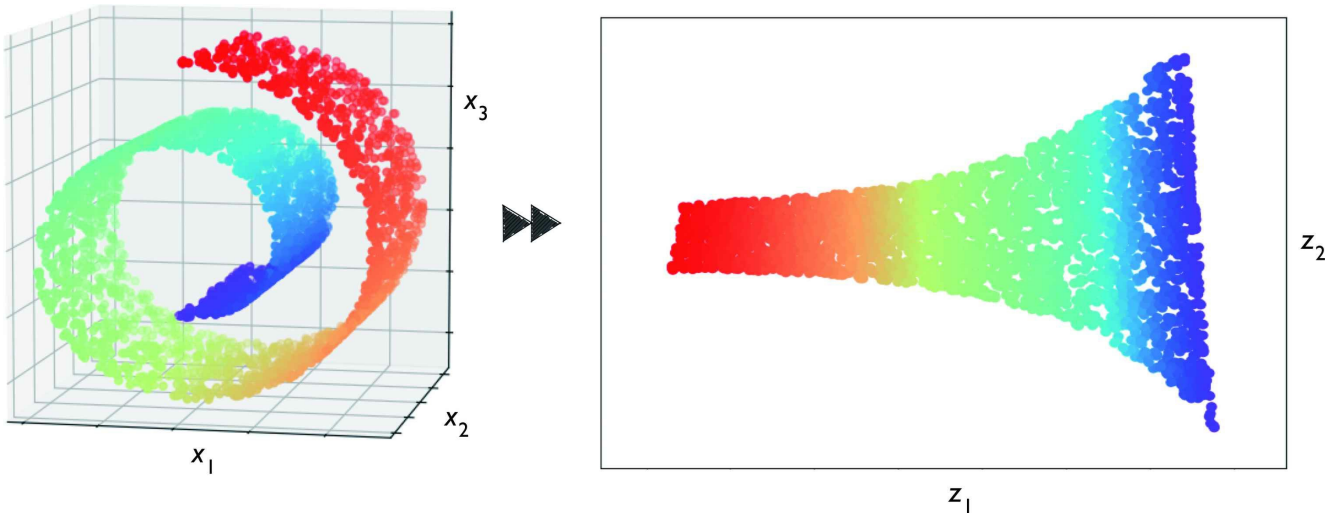
下图解释了如何应用聚类把无标签数据根据$x_1$和$x_2$的相似性分成三组：




### 1.2.3.2 通过降维压缩数据

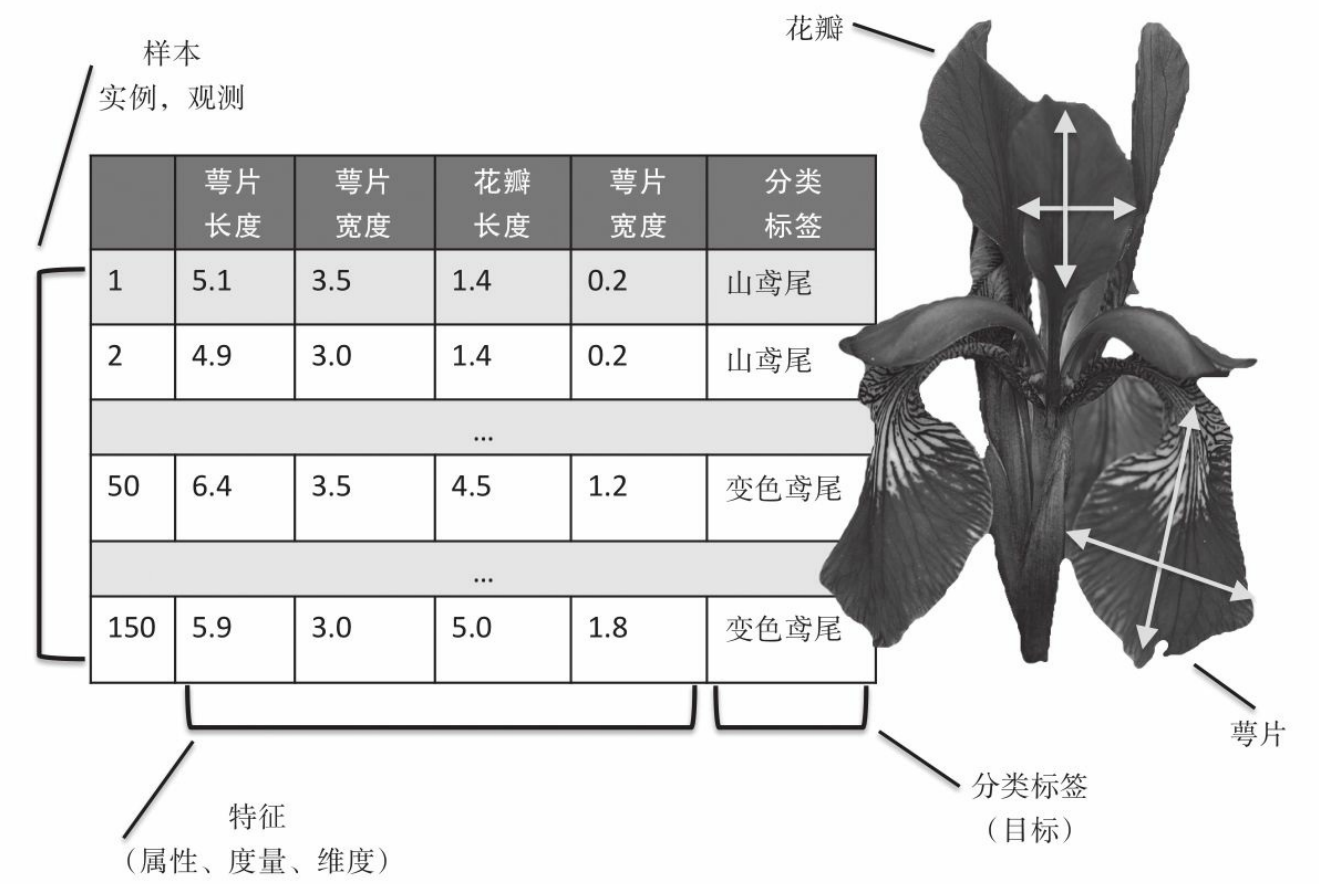
无监督学习的另一个子类是降维。高维数据的每个观察通常都伴随着大量测量数据，这对有限的存储空间和机器学习算法的计算性能提出了挑战。无监督降维是特征预处理中数据去噪的一种常用方法，它也降低了某些算法对预测性能的要求，并在保留大部分相关信息的同时将数据压缩到较小维数的子空间上。

降维有时有利于数据的可视化。例如，为了通过二维或三维散点图或直方图实现数据的可视化，可以把高维特征数据集投影到一、二或三维特征空间。下图展示了一个采用非线性降维将三维瑞士卷压缩成新的二维特征子空间的实例：


$x_3$
$x_2$
$x_1$
$z_2$
$z_1$

## 1.3 基本术语与符号

本章已经讨论了有监督、无监督和强化三大类机器学习，下面将介绍本书中常用的基本术语。下表是描述鸢尾属植物数据集的摘要，这是机器学习领域的典型案例。该数据集包含了对Setosa、Versicolor和Virginica三种不同鸢尾属植物150多朵鸢尾花的测量结果。数据集中每行代表一朵花的样本数据，每种花的数据以厘米为单位按列存储，称为特征数据集：




| | 萼片长度 | 萼片宽度 | 花瓣长度 | 萼片宽度 | 分类标签 |
|---|---|---|---|---|---|
| 1 | 5.1 | 3.5 | 1.4 | 0.2 | 山鸢尾 |
| 2 | 4.9 | 3.0 | 1.4 | 0.2 | 山鸢尾 |
| ... | | | | | |
| 50 | 6.4 | 3.5 | 4.5 | 1.2 | 变色鸢尾 |
| ... | | | | | |
| 150 | 5.9 | 3.0 | 5.0 | 1.8 | 变色鸢尾 |

为了简单且高效地实现，将会用到线性代数的一些基础知识。下面的章节中将会用矩阵和向量符号来表示数据。按照约定将每个样本表示为特征矩阵X的一行，每个特征表示为一列。

鸢尾属数据集包含150个样本的4种特征，可以用150×4矩阵（$X \in R^{150 \times 4}$）表示：

$$\begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & x_4^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & x_4^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(150)} & x_2^{(150)} & x_3^{(150)} & x_4^{(150)} \end{bmatrix}$$

除非特别注明，本书的其余部分将用上标i指第i个训练样本，下标j表示训练数据集的维度。

用小写和黑斜体字符表示向量（$x \in R^{n\times 1}$），用大写和黑斜体字符表示矩阵（$X \in R^{n\times m}$）。分别采用斜体字符$x^{(n)}$或者$x_{(m)}^{(n)}$表示向量或者矩阵中的某个元素。

例如$x_1^{150}$表示第150个鸢尾花样本的第一个维度，即萼片长度。因此该矩阵中每行代表一朵花的数据，可以写成4维行向量$x^{(i)} \in R^{1\times 4}$

$$\boldsymbol{x}^{(i)} = \begin{bmatrix} x_1^{(i)} & x_2^{(i)} & x_3^{(i)} & x_4^{(i)} \end{bmatrix}$$

每个特征维度是一个150个元素的列向量$x_j \in R^{150\times 1}$，例如：

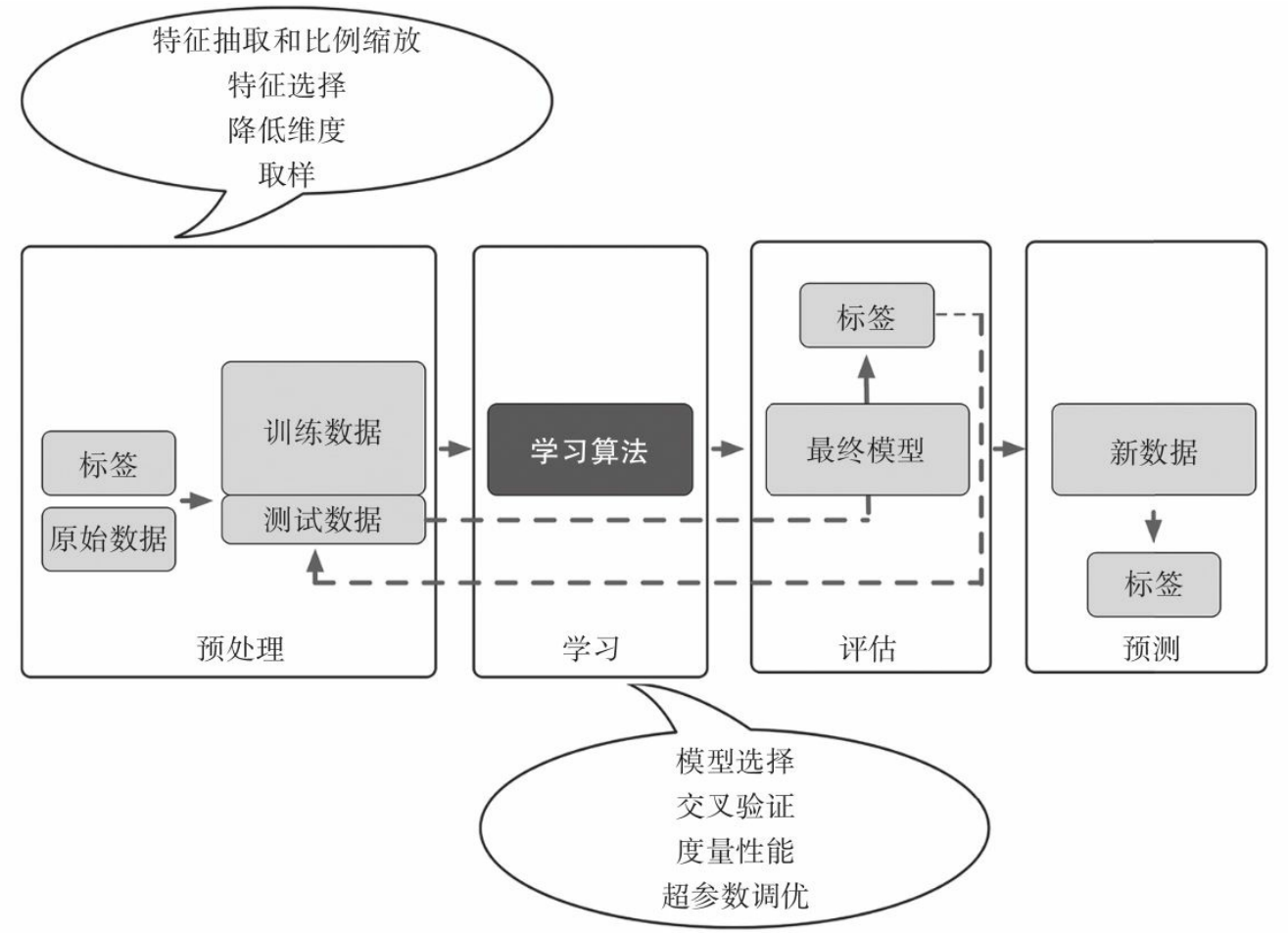
$$\boldsymbol{x}_j = \begin{bmatrix} x_j^{(1)} \\ x_j^{(2)} \\ \vdots \\ x_j^{(150)} \end{bmatrix}$$

类似，可以把目标变量（分类标签）存储为150个元素的列向量：

$$\boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ \cdots \\ y^{(150)} \end{bmatrix} \left( y \in \{\text{Setosa, Versicolor, Virginica}\} \right)$$

## 1.4 构建机器学习系统的路线图

前面的章节讨论了机器学习的基本概念及其三种不同类型。本节将讨论伴随算法的机器学习系统的其他重要部分。下图展示了在预测建模中使用机器学习的典型工作流程，将在以下的几个小节中详细讨论：



### 1.4.1 预处理——整理数据

让我们从讨论构建机器学习系统的路线图开始。原始数据很少以能满足学习算法最佳性能所需要的理想形式出现。因此，数据的预处理是任何机器学习应用中最关键的步骤之一。以前一节的鸢尾花数据集为例，可以把原始数据看成是一系列的花朵图像，要从中提取有意义的特征。有意义的特征可能是颜色、色调、强度、高度、长度和宽度。许多机器学习算法也要求所选择特征的测量结果具有相同的单位，以获得最佳性能，通常通过把特征数据变换为[0，1]的取值范围或者均值和单位方差为0的标准正态分布来实现，后面的章节将会介绍。

某些选定的特征可能是高度相关的，因此在某种程度上是多余的。在这种情况下，降维技术对于将特征压缩到低维子空间非常有价值。降低特征空间维数的优点是减少存储空间，提高算法运行的速度。在某些情况下，如果数据集包含大量不相关的特征或噪声，即数据集具有较低的信噪比，那么降维也可以提高模型预测的性能。

为了确定机器学习算法不仅能在训练集上表现良好，对新数据也有很好的适应性，我们希望将数据集随机分成单独的训练集和测试集。用训练集来训练和优化机器学习模型，同时把测试集保留到最后用以评估最终的模型。

### 1.4.2 训练和选择预测模型

后面的章节中可以看到已经开发了许多不同的机器学习算法来解决不同的问题。从戴维·沃尔珀特著名的“天下没有免费的午餐定理”，可以得出的重要结论是学习不是“免费”的（戴维·沃尔珀特1996年的论文《在学习算法之间没有先验差别》以及戴维·沃尔珀特和W.G.麦克里迪1997年的论文《算法优化没有免费的午餐定理》）。直观地说，可以把这个概念与亚伯拉罕·马斯洛的说法联系起来：“我想这是一种诱惑，如果你拥有的唯一工具就是一把锤子，那会把所有的东西都当作钉子来对待。”例如，每个分类算法都有其固有的偏差，如果不对任务做任何假设，没有哪个分类模型更优越。在实践中，至少要比较几种不同的算法，以便训练和选择性能最好的模型。但在比较不同模型之前，首先必须确定性能度量的指标。一个常用的度量标准是分类准确度，其定义为正确分类样本占所有分类样本的百分比。

有人可能会问：如果不用测试集进行模型选择，而将其留做最终的模型评估，那么如何知道哪个模型在最终测试集和真实数据上表现良好？为了解决嵌套在这个问题中的问题，可以采用不同的交叉检验技术，将训练集进一步分裂为训练集和验证集，以评估模型的泛化性能。最后，也不能期望软件库所提供的不同学习算法的参数默认值对特定问题是最优的。因此，后面的章节经常会使用超参数优化技术来进行模型性能的调优。直观地说，可以把那些超参数看作是从数据中学习不到的，更像模型的旋钮那样，可以来回旋转以改善模型的性能。后面章节中的实际案例会有更清楚的说明。

### 1.4.3 评估模型和预测新样本数据

在选择了适合训练集的模型之后，可以用测试集来评估它在新数据上的性能，以评估泛化误差。如果对模型的性能感到满意，那么就可以用它来预测未来的新数据。需要注意的是前面提到的诸如特征尺度和降维这样的性能测量参数，仅是从训练集获得的，而相同的参数会被进一步转换成测试集，以及任何新的数据样本。否则，对测试数据的性能评估可能会过于乐观。

## 1.5 用Python进行机器学习

Python是数据科学中最常用的编程语言，其优秀的开发人员和开源社区为其提供了大量有价值的附加软件。

像Python这样的解释型语言，尽管对计算密集型任务而言，其性能不如低级的编程语言，但是已经在Fortran和C基础上研发出像NumPy和SciPy这样的扩展软件库，可以实现快速矢量化的多维阵列操作。

机器学习编程主要用scikit-learn，这是目前最为常用和可访问的开源机器学习库。

### 1.5.1 从Python包索引安装Python和其他包

Python可用于微软Windows、苹果MacOS和开源Linux所有三大操作系统，可以从Python官网下载安装程序及文档：https://www.python.org。

本书基于Python 3.5.2或更新版，建议使用最新的Python 3，尽管大部分的代码示例也可以与Python 2.7.13或更新版兼容。如果决定使用Python 2.7来执行这些代码示例，那么请确保了解这两个版本之间的主要差异。从下述网站可以找到专门比较Python 3.5和Python 2.7之间差异的优秀总结文章。https://wiki.python.org/moin/python2orpython3。

本书所用的附加软件包可以通过pip程序安装，该程序从Python 3.3起就一直是标准库的一部分。可以在https://docs.python.org/3/installing/index.html上发现更多关于pip的信息。

在成功地安装Python后，可以在终端上执行pip命令来安装Python的附加包：

```
pip install SomePackage
```

对已经安装的包可以通过--upgrade选项完成升级：

```
pip install SomePackage --upgrade
```

### 1.5.2 采用Anaconda Python和软件包管理器

本书高度推荐由Continuum Analytics发行的Anaconda作为Python的科学计算软件包。免费的Anaconda既可用于商业，也可供企业使用。该软件包括数据科学、数学、工程在内的所有基本包，并把它们捆绑在用户友好的跨平台版本中。可以从http://continuum.io/downloads下载Anaconda的安装程序，从https://conda.io/docs/test-drive.html下载Anaconda的快速启动指南。

在成功地安装了Anaconda之后，可以执行下述命令安装Python包：

```
conda install SomePackage
```

已经安装过的包可以通过执行下述命令升级：

```
conda update SomePackage
```

### 1.5.3 科学计算、数据科学和机器学习软件包

本书将主要使用NumPy的多维阵列来存储和操作数据。偶尔也会用pandas库，该工具建立在NumPy之上，可以提供额外的更高级的数据操作，可以使表格数据操作更加方便。为了加强学习经验和定量数据的可视化，我们将使用定制化程度非常高的Matplotlib软件库，这对直观理解往往极为有用。

本书所使用的主要的Python软件包的版本号在下面列出。为了确保代码示例能够正确运行。请保证你安装的软件包的版本号等于或大于所列出的版本号：

- NumPy 1.12.1
- SciPy 0.19.0
- scikit-learn 0.18.1
- Matplotlib 2.0.2
- pandas 0.20.1

## 1.6 小结

本章从宏观角度探讨了机器学习，让你对全局和主要概念有所了解，后续章节将会探讨更多的细节。我们了解到有监督学习有两个重要分支：分类与回归。分类模型为对象分配已知的类标签，回归分析可以预测目标变量的连续结果。无监督学习不仅为发现未标记数据中的结构提供了有用的技术，对特征预处理过程中的数据压缩也很有用。本章简要地讨论了应用机器学习技术解决问题的典型路线图，为后续章节深入讨论和动手实践奠定了基础。最后，搭建了Python环境，安装和更新了所需要的软件包，为执行机器学习示例代码做好了准备。

除了机器学习本身，本书后续还将引入不同的数据预处理技术，这将有助于从不同的机器学习算法中获得最佳性能。本书除了广泛讨论分类算法以外，还将探索回归分析和聚类的不同技术。

本书是一段激动人心的旅程，它将涵盖机器学习领域的许多强大技术。通过阅读各章，逐步建立知识基础，逐步接近机器学习。下一章将通过实现最早的机器学习分类算法开启这个旅程，这也将为第3章做好准备，该章将覆盖更多使用开源scikit-learn机器学习库的高级机器学习算法。

# 第2章　训练简单的机器学习分类算法

本章将用感知器和自适应线性神经元两个早期的算法来描述机器学习的分类算法。将从用Python逐步实现感知器开始，训练模型对鸢尾属植物数据集的不同花朵分类。这有助于理解机器学习分类算法的概念，以及如何在Python中有效地实现。

讨论自适应线性神经元优化的基础知识将为采用基于scikit-learn机器学习软件库（见第3章）的更强大分类器奠定基础。

本章将主要涵盖下述几个方面：

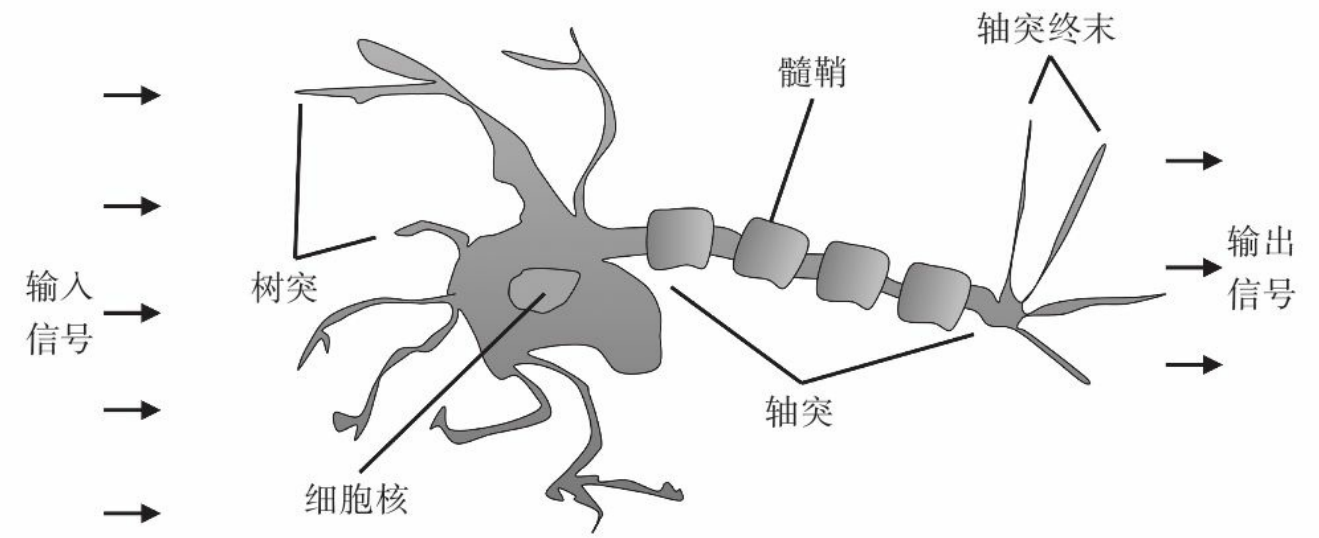
·建立对机器学习算法的直觉

·在数据读入、处理和可视化中使用pandas、NumPy和Matplotlib

·用Python实现线性分类算法

## 2.1 人工神经元——机器学习早期历史一瞥

在更详细地讨论感知器及其相关算法之前，先简要地回顾机器学习的开端。为了设计人工智能，人们尝试了解生物大脑的工作原理。沃伦·麦库洛和沃尔特·皮兹首次提出了简化脑细胞的概念，即所谓的麦库洛-皮兹（MCP）神经元（沃伦·麦库洛，沃尔特·皮兹.《神经活动内在思想的逻辑演算》.数学生物物理学通报，1943，5（4）：115-133）。神经元是大脑中连接起来参与化学和电信号处理与传输的神经细胞，见下图：




麦库洛和皮兹把神经细胞描述为带有二进制输出的简单逻辑门：多个信号到达树突，然后整合到细胞体，并当累计信号量超过一定的阈值时，输出信号将通过轴突。

论文发表之后仅几年，弗兰克·罗森布拉特就首先提出了基于MCP神经元模型的感知器学习规则概念（F.罗森布拉特.《感知：感知和识别自动机》.康奈尔航空实验室，1957）。根据其感知规则，罗森布拉特提出了一个算法，它能自动学习最优权重系数，乘以输入特征，继而做出神经元触发与否的决定。在有监督学习和分类情况下，这样的算法可以用来预测样本是属于某类还是另一类。

### 2.1.1 人工神经元的正式定义

更准确地说，可以把人工神经元逻辑放在二元分类场景，将两个类分别命名为1（正类）和-1（负类）以简化操作。定义决策函数（（z）），接受输入值x及其相应的权重w，z为所谓的净输入$z=w_1x_1+\ldots+w_mx_m$：

$$\boldsymbol{w}=\begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \boldsymbol{x}=\begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

如果某个特定样本的净输入值$x^{(i)}$比定义的阈值θ大，则预测结果为1，否则为-1。在感知算法中，决策函数φ（·）是单位阶跃函数的变体：

$$\phi(z)=\begin{cases} 1, & \text{if } z \geqslant \theta \\ -1, & \text{otherwise} \end{cases}$$

为了简化，把阈值θ放到等式的左边，权重零定义为$w_0=-\theta$，$x_0=1$，这样就可以用更紧凑的方式来表示z：

$$z=w_0x_0+w_1x_1+\cdots+w_mx_m=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}$$

和

$$\phi(z)=\begin{cases} 1, & \text{if } z \geqslant \theta \\ -1, & \text{otherwise} \end{cases}$$

机器学习文献通常把负的阈值或权重$w_0=-\theta$称为偏置（bias unit）。

后面将常用线性代数的基本符号。例如，用矢量点积的方法表示x和w的值相乘后累加的结果，上标T表示转置，该操作将列向量转换为行向量，反之亦然：

$$z = w_0 x_0 + w_1 x_1 + \cdots w_j x_j = \sum_{j=0}^{m} \boldsymbol{x}_j \boldsymbol{w}_j = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x}$$

例如：

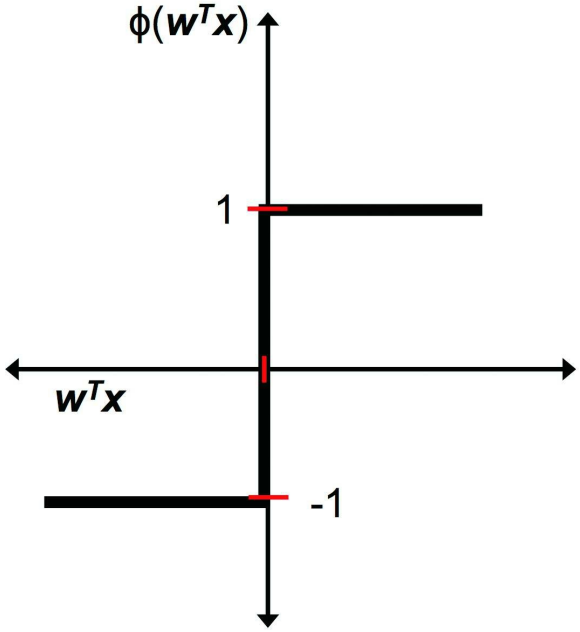
$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \times \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32$$

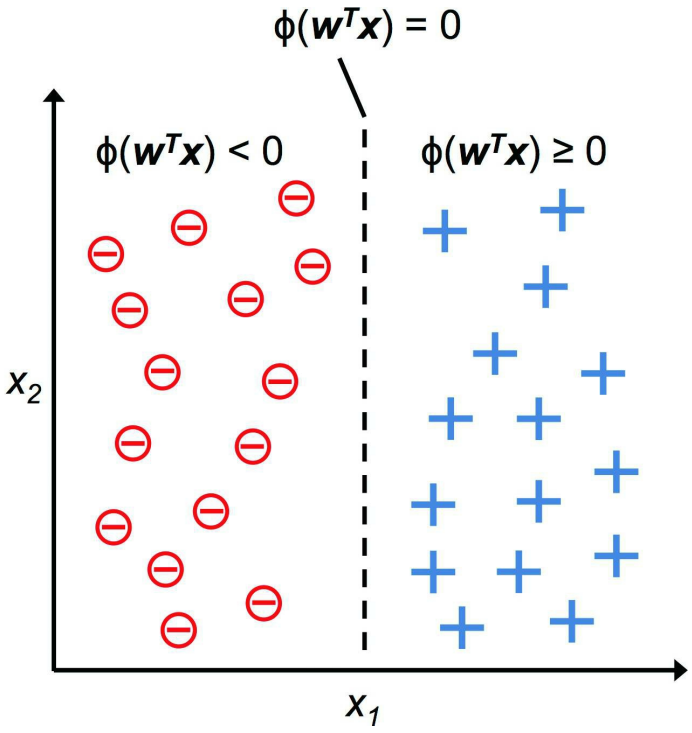
转置操作也可以从矩阵的对角线上反映出来，例如：

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

本书仅涉及非常基本的线性代数概念，然而，如果你需要做个快速回顾，可以去看济科·科特勒的《线性代数回顾与参考》，该书可以从下述网站免费获得：http://www.cs.cmu.edu/~zkolter/course/linalg/linalg_notes.pdf

下图解释了如何通过感知决策函数把净输入$z=w^{\mathrm{T}}x$（图左）转换为二元输出（-1或者1），以及如何区分两个可分隔的线性类（图右）：


ϕ(w^T x)
1
w^T x
-1


ϕ(w^T x) = 0
ϕ(w^T x) < 0
ϕ(w^T x) ≥ 0
x_2
x_1

## 2.1.2 感知器学习规则

MCP神经元和罗森布拉特的阈值感知器模型背后的逻辑是用还原论方法来模拟大脑神经元的工作情况：要么触发，要么不触发。因此，罗森布拉特的初始感知规则相当简单，可以总结为以下几步：

1.把权重初始化为0或者小的随机数。

2.对每个训练样本$x^{(i)}$：

a.计算输出值$\hat{y}$。

b.更新权重。

输出值为单位阶跃函数预测的预先定义好的类标签，同时更新权重向量w的每个值wj，更准确的表达式为：

$$w_j := w_j + \Delta w_j$$

$\triangle w_j$是用来更新$w_j$的值，该值根据感知器的学习规则计算：

$$\Delta w_j = \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$$

η为学习率（一般是0.0到1.0之间的常数），$y^{(i)}$为第i个训练样本的正确类标签，$\hat{y}^{(i)}$为预测的类标签。需要注意的是权重向量中的所有权重值同时被更新，这意味着在所有的权重wj更新之前，不会重新计算$\hat{y}^{(i)}$。具体来说，二维数据集的更新可以表示为：

$$\Delta w_0 = \eta(y^{(i)} - output^{(i)})$$

$$\Delta w_1 = \eta(y^{(i)} - output^{(i)})x_1^{(i)}$$

$$\Delta w_2 = \eta(y^{(i)} - output^{(i)})x_2^{(i)}$$

在实现Python感知器规则之前，先做个简单的思考实验来说明该学习规则到底有多么简单。在感知器正确预测分类标签的两种情况下，权重保持不变：

$$\Delta w_j=\eta(-1-(-1))x_j^{(i)}=0$$

$$\Delta w_j=\eta(-1)x_j^{(i)}=0$$

然而，如果预测有错误，权重偏向正或负的目标类：

$$\Delta w_j=\eta(-1--1)x_j^{(i)}=\eta(2)x_j^{(i)}$$

$$\Delta w_j=\eta(-1--1)x_j^{(i)}=\eta(-2)x_j^{(i)}$$

为了更好地理解乘积因子$x_j^{(i)}$，让我们看下另外一个简单的例子，其中：

$$\hat{y}^{(i)}=-1,\ y^{(i)}=+1,\ \eta=1$$

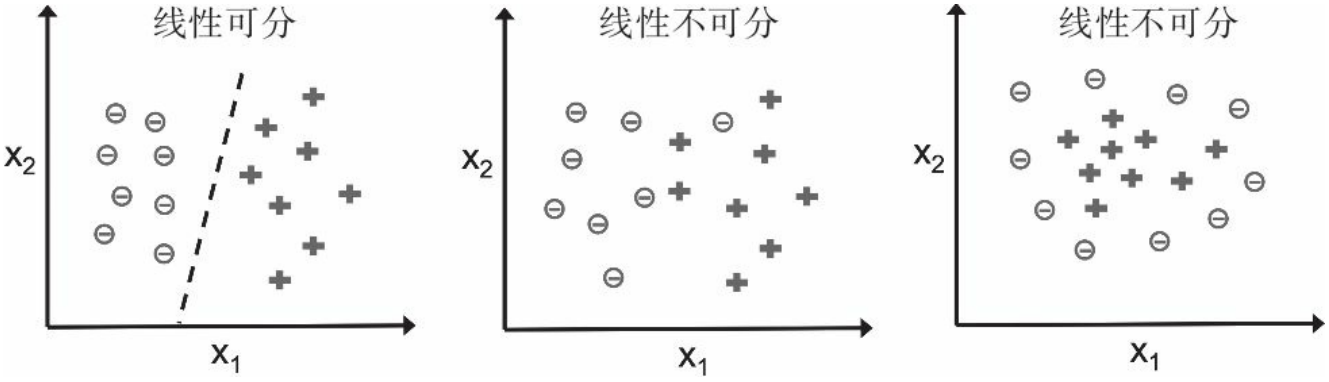
假设$x_j^{(i)}$=0.5，模型错把该样本判断为-1。在这种情况下，把相应的权重增加1，这样当下次再遇到该样本时净输入$x_j^{(i)}$×$w_j$就会呈现为正数，因此，极有可能超过单位阶跃函数的阈值，从而把该样本判断为+1：

$$\Delta w_j=(1--1)0.5=(2)0.5=1$$

权重更新与$x_j^{(i)}$成正比。例如，假设有另外一个样本$x_j^{(i)}=2$被错误地分类为-1，可以将决策边界推到更大，以确保下一次分类正确：

$$\Delta w_j=(1--1)2=(2)2=4$$

重要的是要注意，只有两个类线性可分，并且学习速率足够小，这样感知器的收敛性才能得到保证。如果两个类不能用线性决策边界分离，可以为训练集设置最大通过数（迭代次数）及容忍错误的阈值，否则分类感知器将永远都不会停止更新权重：



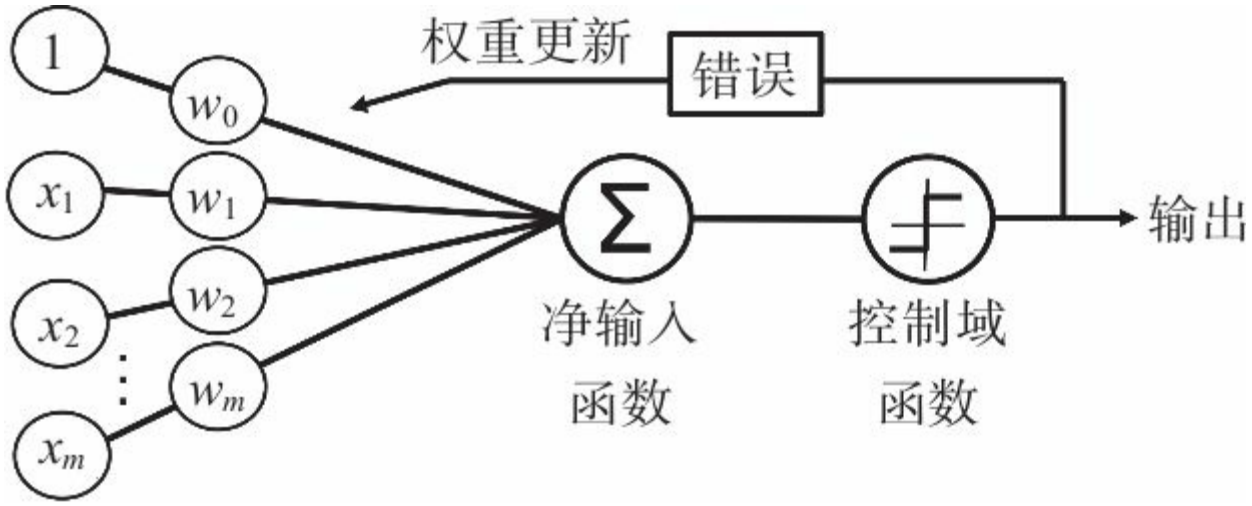

下载示例代码

直接从Packt购买的书，可以从下列网站直接下载示例代码：http://www.packtpub.com

从其他地方购买的书，可以直接从下列网站下载示例代码：

https://github.com/rasbt/python-machine-learning-book-2nd-edition

在进行下一节代码实现之前，让我们把所学的知识总结到一个简单的图中，来说明感知器的一般概念：




上图说明了感知器如何接收样本x的输入，并将其与权重w结合起来计算净输入。然后，净输入传递到阈值函数，产生一个二元输出-1或+1，即预测样本的分类标签。在学习阶段，该输出用于计算预测结果的错误并更新权重。

## 2.2 在Python中实现感知器学习算法

上一节学会了罗森布拉特感知规则的工作机制。现在继续在Python中实现它，并将其应用于第1章中介绍的鸢尾花数据集。

### 2.2.1 面向对象的感知器API

我们将用面向对象的方法把感知器接口定义为一个Python类，它允许初始化新的Perceptron对象，这些对象可以通过fit方法从数据中学习，并通过单独的predict方法进行预测。作为约定，我们在对象初始化时未创建但通过对象的其他方法创建的属性的后面添加下划线（_），例如self.w_。

如果对Python科学库不熟或需要回顾，请参考下面的资源：

·NumPy：https://sebastianraschka.com/pdf/books/dlb/appendix_f_NumPy-intro.pdf

·pandas：https://pandas.pydata.org/pandas-docs/stable/10min.html

·Matplotlib：http://matplotlib.org/users/beginner.html

下面的代码实现了感知器：

```
import numpy as np

class Perceptron(object):
    """Perceptron classifier.

    Parameters
    ------------
    eta : float
      Learning rate (between 0.0 and 1.0)
    n_iter : int
      Passes over the training dataset.
    random_state : int
      Random number generator seed for random weight
      initialization.

    Attributes
    -----------
    w_ : 1d-array
      Weights after fitting.
    errors_ : list
      Number of misclassifications (updates) in each epoch.

    """
    def __init__(self, eta=0.01, n_iter=50, random_state=1):
        self.eta = eta
        self.n_iter = n_iter
        self.random_state = random_state

    def fit(self, X, y):
        """Fit training data.
```

```
        Parameters
    ----------
    X : {array-like}, shape = [n_samples, n_features]
      Training vectors, where n_samples is the number of
      samples and
      n_features is the number of features.
    y : array-like, shape = [n_samples]
      Target values.

    Returns
    -------
    self : object

    """
    rgen = np.random.RandomState(self.random_state)
    self.w_ = rgen.normal(loc=0.0, scale=0.01,
                          size=1 + X.shape[1])
    self.errors_ = []

    for _ in range(self.n_iter):
        errors = 0
        for xi, target in zip(X, y):
            update = self.eta * (target - self.predict(xi))
            self.w_[1:] += update * xi
            self.w_[0] += update
            errors += int(update != 0.0)
        self.errors_.append(errors)
    return self

def net_input(self, X):
    """Calculate net input"""
    return np.dot(X, self.w_[1:]) + self.w_[0]

def predict(self, X):
    """Return class label after unit step"""
    return np.where(self.net_input(X) >= 0.0, 1, -1)
```

这段实现感知器的代码用给定的学习率η（eta）和训练次数n_iter初始化新的Perceptron对象。通过fit方法，初始化self.w_的权重，并把数据存入向量$R^{m+1}$，m代表数据集的维数或特征数，+1为偏差单位向量的第一个分量。请记住，该向量的第一个分量self.w_[0]代表前面讨论过的偏差单位。

另外，该向量包含来源于正态分布的小随机数，通过调用rgen.normal（loc=0.0，scale=0.01，size=1+X.shape[1]）产生标准差为0.01的正态分布，其中rgen为NumPy随机数生成器，随机种子由用户指定，因此可以保证在需要时可以重现以前的结果。

不把权重初始化为零的原因是，只有当权重初始化为非零的值，学习率η（eta）才影响分类结果。如果把所有的权重都初始化为零，学习率参数

η（eta）只影响权重向量的大小，而无法影响其方向。如果熟悉三角函数，考虑向量v1=[1 2 3]，v1和向量v2=0.5×v1之间的角度将会是0，参见下面的代码片段：

```
>>> v1 = np.array([1, 2, 3])
>>> v2 = 0.5 * v1
>>> np.arccos(v1.dot(v2) / (np.linalg.norm(v1) *
...                         np.linalg.norm(v2)))
0.0
```

这里np.arccos为三角反余弦，np.linalg.norm是计算向量长度的函数（从随机正态分布而不是均匀分布中抽取随机数，以及选择标准偏差为0.01，这些决定是任意的，记住，只对小随机值感兴趣的目的是要避免前面讨论过的所有向量为零的情况）。

对单维阵列，NumPy索引与用（[]）表示的Python列表类似。对二维阵列，第一个索引为行，第二个索引为列。例如用X[2，3]来指二维阵列X的第三行第四列。

初始化权重后，用fit方法遍历训练集的所有样本，并根据前一节讨论过的感知器学习规则更新权重。fit方法调用predict方法来预测分类标签并更新权重，但predict也可以用来在模型拟合后预测新数据的标签。另外，也把在每个迭代中收集到的分类错误记入self.errors_列表，这样就可以在后期分析训练阶段感知器的表现。用net_input方法中的np.dot函数来计算向量点积wTx。

对阵列a和b的向量点积计算，在纯Python中用sum（[i*j for i，j in zip（a，b）]）来实现，在NumPy中用a.dot（b）或者np.dot（a，b）来完成。然而，NumPy与传统Python相比的好处是算术运算向量化。向量化意味着基本的算术运算自动应用在阵列的所有元素上。把算术运算形成一连串的阵列指令，而不是对每个元素完成一套操作，这样可以更好地使用现代CPU的单指令多数据支持（SIMD）架构。另外，NumPy采用以C或Fortran语言编写的高度优化的线性代数库，诸如基本线性代数子程序（BLAS）和线性代数包（LAPACK）。最后，NumPy也允许用线性代数的基本知识以更加紧凑和自然的方式编写像向量和矩阵点积这样的代码。

### 2.2.2 在鸢尾花数据集上训练感知器模型

为了实现感知器，将从鸢尾花数据集加载Setosa和Versicolor两种花的数据。虽然感知器规则并不局限于两个维度，但暂时只考虑以可视化为目的的萼片长度、花瓣长度两个特征。另外，出于实际考虑，只选择Setosa和Versicolor两种花。然而，感知器算法可以扩展到多元分类，例如一对全部（OvA）的技术。

OvA，有时也被称为一对其余（OvR），是可以把分类器从二元扩展到多元的一种技术。OvA可以为每个类训练一个分类器，所训练的类被视为正类，所有其他类的样本都被视为负类。假设要对新的数据样本进行分类，就可以用n个分类器，其中n为分类标签的数量，并以最高的置信度为特定样本分配分类标签。在感知器的情况下，将用OvA来选择与最大净输入值相关的分类标签。

首先可以用pandas库，从UCI机器学习库把鸢尾花数据集直接加载到DataFrame对象，然后用tail方法把最后5行数据列出来以确保数据加载的正确性。

```
>>> import pandas as pd
>>> df = pd.read_csv('https://archive.ics.uci.edu/ml/'
...                  'machine-learning-databases/iris/iris.data',
...                  header=None)
>>> df.tail()
```

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 145 | 6.7 | 3.0 | 5.2 | 2.3 | Iris-virginica |
| 146 | 6.3 | 2.5 | 5.0 | 1.9 | Iris-virginica |
| 147 | 6.5 | 3.0 | 5.2 | 2.0 | Iris-virginica |
| 148 | 6.2 | 3.4 | 5.4 | 2.3 | Iris-virginica |
| 149 | 5.9 | 3.0 | 5.1 | 1.8 | Iris-virginica |

如果无法上网或UCI的服务器宕机（https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data），可以直接从本书的代码集找到鸢尾花数据集（也包括本书所有其他的数据集）。要从本地文件目录加载鸢尾花数据，可不用下面的命令：

```
df = pd.read_csv('https://archive.ics.uci.edu/ml/'
        'machine-learning-databases/iris/iris.data',
        header=None)
```

而是用下面的命令：

```
df = pd.read_csv('your/local/path/to/iris.data',
                 header=None)
```

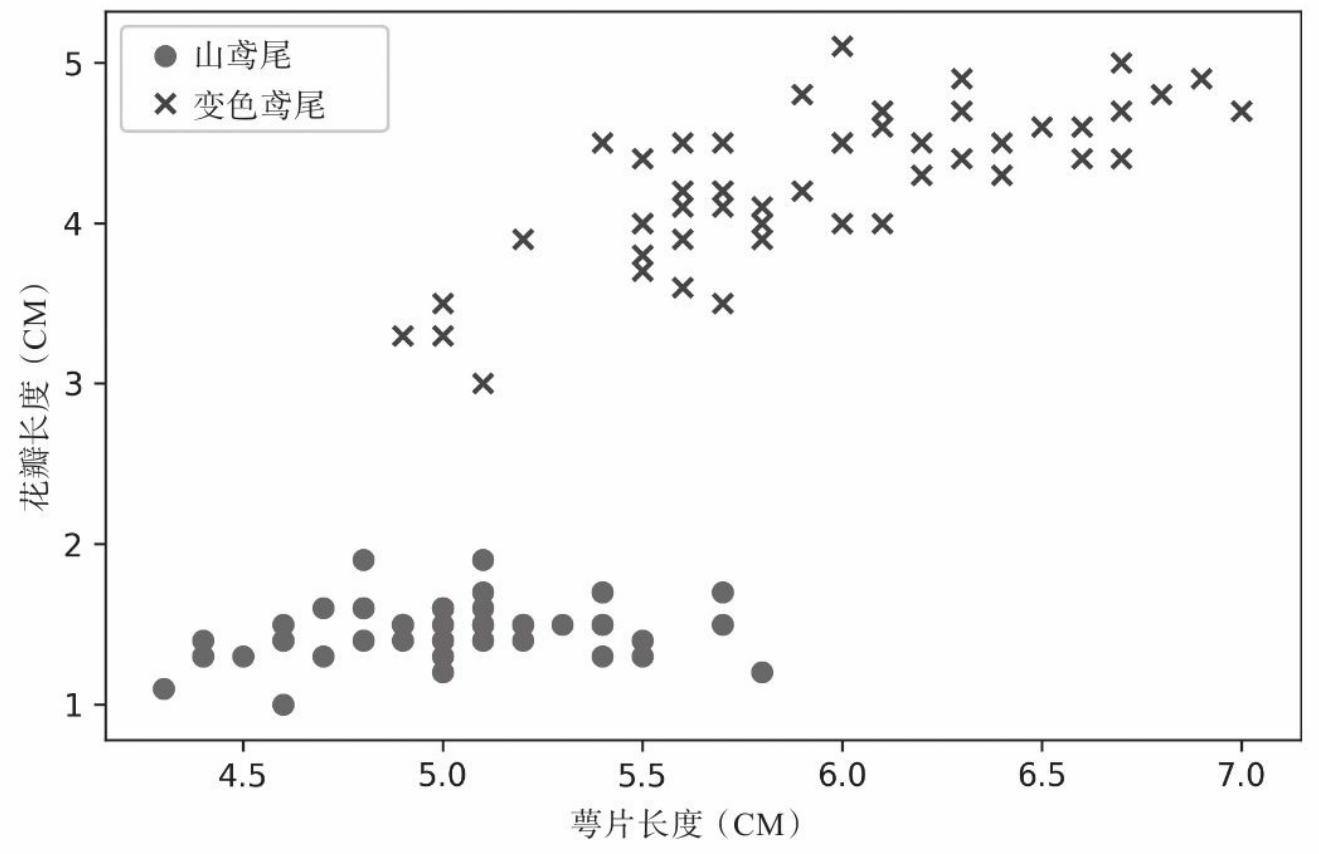
接下来，提取与50朵Iris-setosa和50朵Iris-versicolor鸢尾花相对应的前100个类标签，然后转换为整数型的类标签1（versicolor）和-1（setosa），并存入向量y，再调用pandas的DataFrame的方法获得到相应的NumPy表示。

同样，可以从100个训练样本中提取特征的第一列（萼片长度）和第三列（花瓣长度），并将它们存入特征矩阵X，然后经过可视化处理形成二维

散点图：

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np

>>> # select setosa and versicolor
>>> y = df.iloc[0:100, 4].values
>>> y = np.where(y == 'Iris-setosa', -1, 1)

>>> # extract sepal length and petal length
>>> X = df.iloc[0:100, [0, 2]].values

>>> # plot data
>>> plt.scatter(X[:50, 0], X[:50, 1],
...             color='red', marker='o', label='setosa')
>>> plt.scatter(X[50:100, 0], X[50:100, 1],
...             color='blue', marker='x', label='versicolor')
>>> plt.xlabel('sepal length [cm]')
>>> plt.ylabel('petal length [cm]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

执行前面的代码示例可以看到下面的二维散点图：



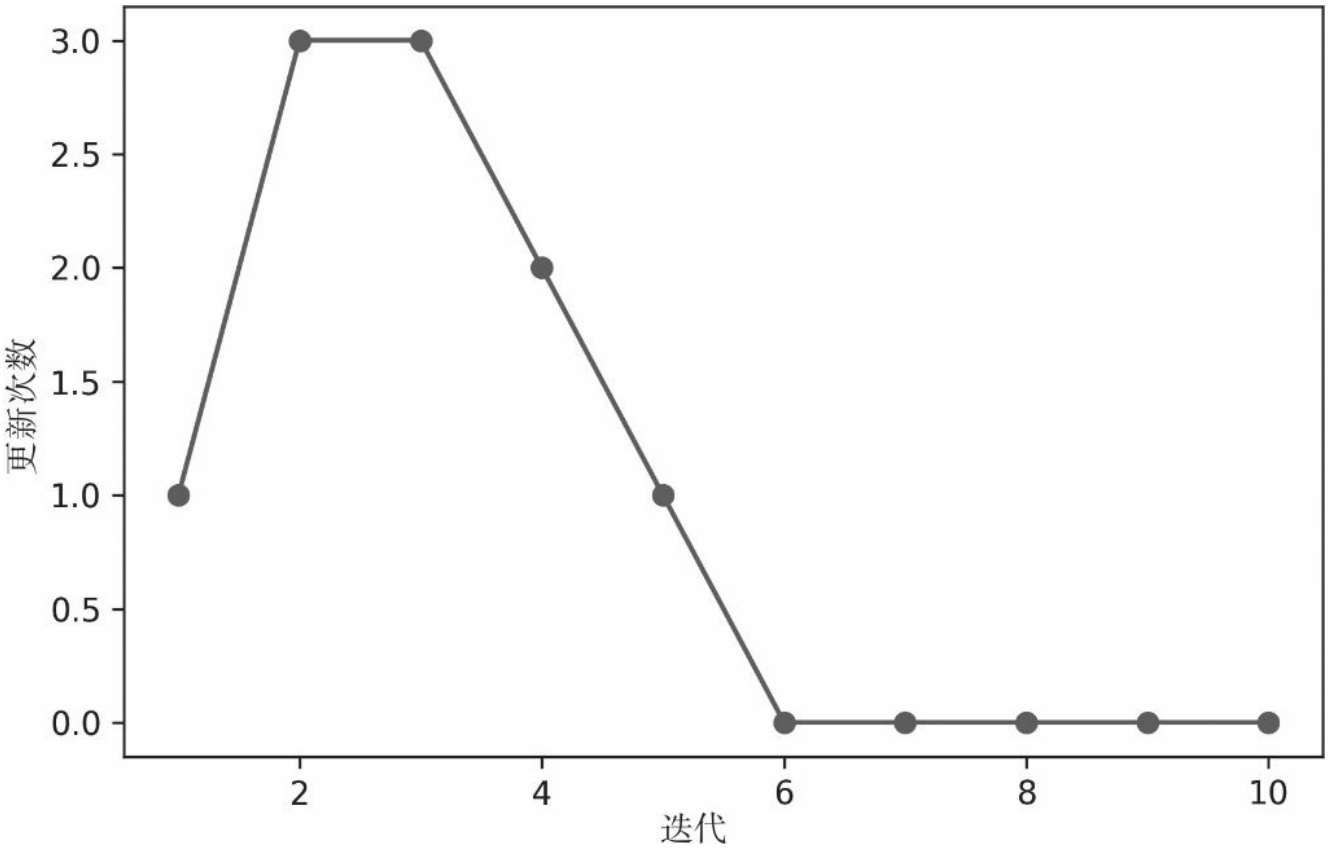

前面的散点图显示了鸢尾花数据集的样本在花瓣长度和萼片长度两个特征轴之间的分布情况。从这个二维特征子空间中可以看到，一个线性的决策

边界足以把Setosa花与Versicolor花区分开。因此，像感知器这样的线性分类器应该能够完美地对数据集中的花朵进行分类。

现在是在鸢尾花数据子集上训练感知器算法的时候了。此外，也将绘制每个迭代的分类错误，以检查算法是否收敛，并找到分隔两类鸢尾花的决策边界：

```
>>> ppn = Perceptron(eta=0.1, n_iter=10)
>>> ppn.fit(X, y)
>>> plt.plot(range(1, len(ppn.errors_) + 1),
...          ppn.errors_, marker='o')
>>> plt.xlabel('Epochs')
>>> plt.ylabel('Number of updates')
>>> plt.show()
```

执行前面的代码，可以看到分类错误与迭代之间的关系，图示如下：




正如上图可以看到的，感知器在第6次迭代后开始收敛，现在应该能够完美地对训练样本进行分类。下面通过实现一个短小精干的函数来完成二维数据决策边界的可视化：

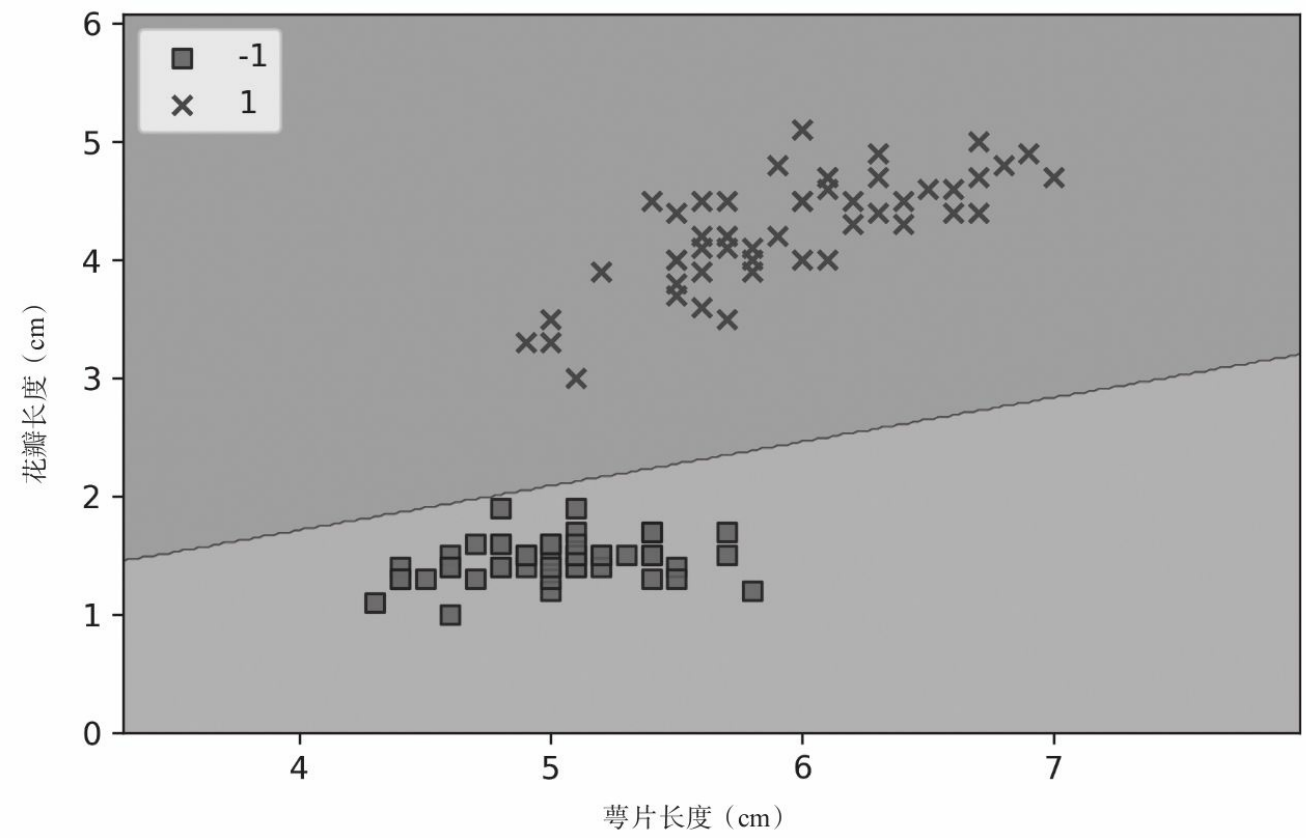
```
from matplotlib.colors import ListedColormap

def plot_decision_regions(X, y, classifier, resolution=0.02):

    # setup marker generator and color map
    markers = ('s', 'x', 'o', '^', 'v')
    colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
    cmap = ListedColormap(colors[:len(np.unique(y))])

    # plot the decision surface
    x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
                           np.arange(x2_min, x2_max, resolution))
    Z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
    Z = Z.reshape(xx1.shape)
    plt.contourf(xx1, xx2, Z, alpha=0.3, cmap=cmap)
    plt.xlim(xx1.min(), xx1.max())
    plt.ylim(xx2.min(), xx2.max())

    # plot class samples
    for idx, cl in enumerate(np.unique(y)):
        plt.scatter(x=X[y == cl, 0],
                    y=X[y == cl, 1],
                    alpha=0.8,
                    c=colors[idx],
                    marker=markers[idx],
                    label=cl,
                    edgecolor='black')
```

首先定义颜色和标记并通过ListedColormap来从颜色列表创建色度图。然后，通过NumPy的meshgrid函数创建网格阵列xx1和xx2，利用特征向量确定特征的最小和最大值。由于在两个特征维度上训练感知器分类器，所以需要对网格阵列进行扁平化，并创建一个与鸢尾花训练子集相同列数的矩阵，这样就可以调用predict方法来预测相应网格点的分类标签z。

在把预测获得的分类标签z改造成与xx1和xx2相同维数的网格后，现在可以通过调用MatPlotlib的contourf函数画出轮廓图，并把网格阵列中每个预测的分类结果标在不同颜色的决策区域：

```
>>> plot_decision_regions(X, y, classifier=ppn)
>>> plt.xlabel('sepal length [cm]')
>>> plt.ylabel('petal length [cm]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

执行示例代码后可以看到下图所示的决策区域：




如图所示，感知器通过学习掌握了决策边界，能够把鸢尾花训练集的样本完美分开。

虽然感知器可以完美地对鸢尾花的两个类进行分类，但收敛是感知器的最大问题之一。弗兰克·罗森布拉特从数学上证明了如果两个类可以通过一个线性超平面分离，则感知器学习规则收敛。然而，如果不能被这样的线性决策边界完全分离，那么除非设定最大的迭代数，否则永远不会停止更新权重。

## 2.3 自适应神经元和学习收敛

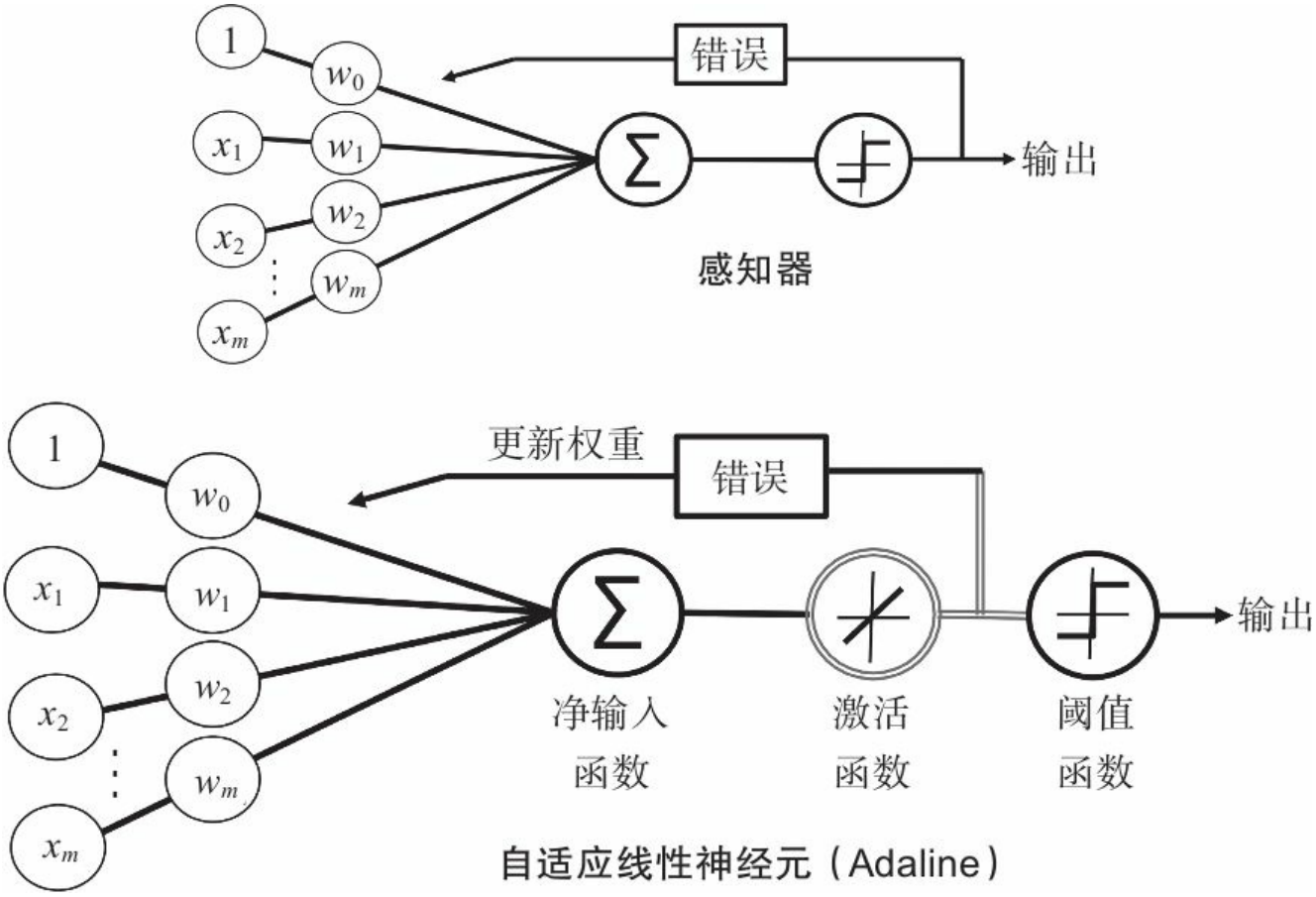
本节将讨论另外一种单层神经网络：自适应线性神经元（Adaline）。Adaline仅仅在弗兰克·罗森布拉特提出感知器算法几年后，由伯纳德·威德罗及其博士生特德·霍夫联合提出，可以被视为对前者的优化和改进。（参见伯纳德·威德罗等的《用化学“薄膜”的自适应Adaline神经元》，斯坦福电子实验室第1553-2号技术报告，1960年10月，加州。）

Adaline算法特别有趣，因为它说明了定义最小化连续性代价函数的关键概念。这为理解如逻辑回归、支持向量机和回归模型等更高级的机器学习算法奠定了基础，以后的章节将讨论这些问题。

Adaline规则（也称为威德罗–霍夫规则）和弗兰克·罗森布拉特的感知器之间的关键差异在于Adaline规则的权重更新是基于线性激活函数，而感知器是基于单位阶跃函数。Adaline的线性激活函数φ（z）是净输入的等同函数，即：

$$\phi(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x})=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}$$

虽然线性激活函数可用于学习权重，但仍然使用阈值函数做最终的预测，这类似于先前看到的单位阶跃函数。感知器与自适应算法的主要区别图示如下：




上图表明Adaline算法把正确的分类标签与线性激活函数连续评估后，输出计算的模型误差并与更新的权重进行比较。相反，感知器比较正确分类标签与预测分类标签。

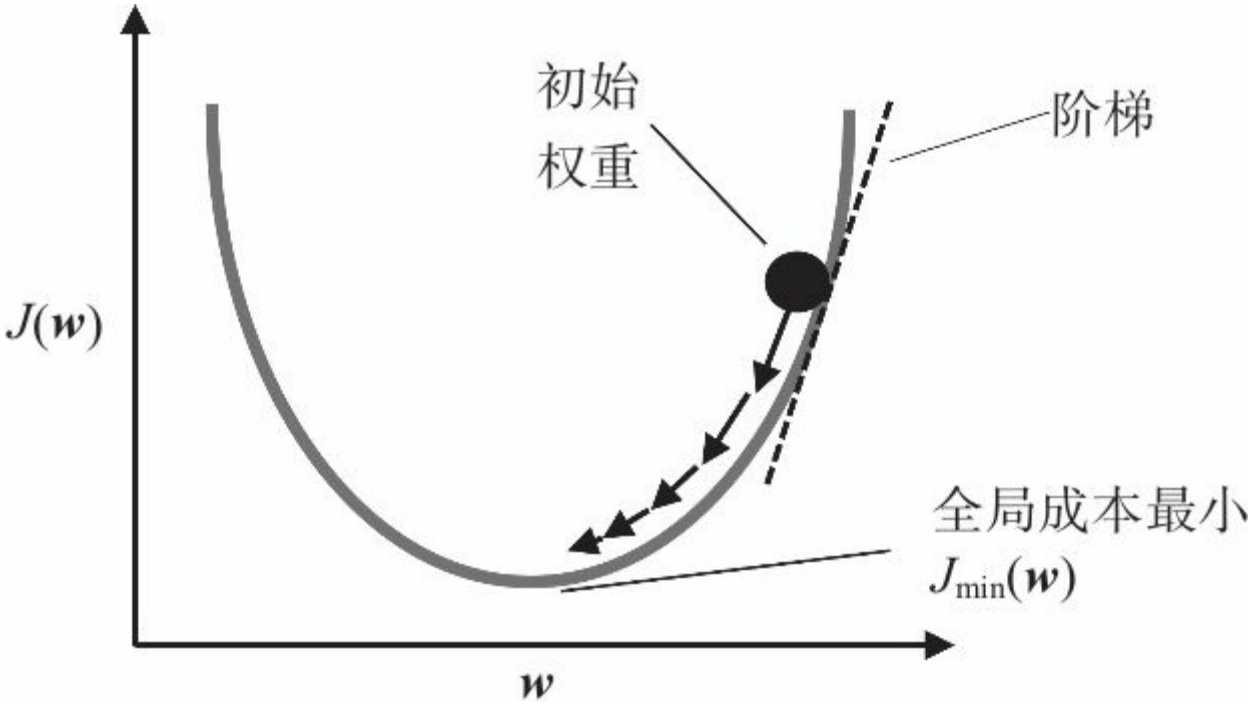
## 2.3.1 梯度下降为最小代价函数

有监督机器学习算法的一个关键是在学习过程中优化目标函数。该目标函数通常是要最小化的代价函数。对自适应线性神经元Adaline而言，可以把学习权重的代价函数J定义为在计算结果和真正的分类标签之间的误差平方和（SSE）：

$$J(\boldsymbol{w})=\frac{1}{2}\sum_i\left(y^{(i)}-\phi\left(z^{(i)}\right)\right)^2$$

添加$\frac{1}{2}$只是为了方便，下面的段落可以看到如何更容易地推导梯度。与单位阶跃函数相反，这种连续线性激活函数的主要优点是代价函数变得可分。代价函数的另外一个优点是其凸起，因此，可以用被称为梯度下降的简单而强大的优化算法来寻找权重，从而在鸢尾花数据集样本分类过程中最大限度地减小代价函数。

如图所示，可以把梯度下降背后的主要逻辑描述为走下坡路直到局部或全局最小点为止。每次迭代都向梯度相反的方向上迈出一步，步幅由学习率以及梯度斜率来决定：




现在可以通过梯度下降代价函数J（w）的梯度ΔJ（w）朝反方向上迈出一步来更新权重：

$$\boldsymbol{w}:=\boldsymbol{w}+\Delta\boldsymbol{w}$$

其中，把权重变化△w定义为负的梯度乘以学习率η：

$$\Delta \boldsymbol{w} = -\eta \nabla J(\boldsymbol{w})$$

要计算代价函数的梯度需要分别用每个权重$w_j$来计算代价函数的偏导数：

$$\frac{\partial J}{\partial w_j} = -\sum_i \left( y^{(i)} - \phi\left( z^{(i)} \right) \right) x_j^{(i)}$$

这样就可以把权重$w_j$的更新写为：

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_i \left( y^{(i)} - \phi\left( z^{(i)} \right) \right) x_j^{(i)}$$

因为同时更新所有的权重，所以Adaline学习规则就成为：

$$\boldsymbol{w} := \boldsymbol{w} + \Delta \boldsymbol{w}$$

对熟悉微积分的人，与第j个权重相对应的SSE代价函数的偏导数可以计算如下：

$$\frac{\partial J}{\partial w_j} = \frac{\partial}{\partial w_j}\frac{1}{2}\sum_i\left(y^{(i)} - \phi\left(z^{(i)}\right)\right)$$

$$= \frac{1}{2}\frac{\partial}{\partial w_j}\sum_i\left(y^{(i)} - \phi\left(z^{(i)}\right)\right)^2$$

$$= \frac{1}{2}\sum_i 2\left(y^{(i)} - \phi\left(z^{(i)}\right)\right)\frac{\partial}{\partial w_j}\left(y^{(i)} - \phi\left(z^{(i)}\right)\right)$$

$$= \sum_i\left(y^{(i)} - \phi\left(z^{(i)}\right)\right)\frac{\partial}{\partial w_j}\left(y^{(i)} - \sum_i\left(w_j x_j^{(i)}\right)\right)$$

$$= \sum_i\left(y^{(i)} - \phi\left(z^{(i)}\right)\right)\left(-x_j^{(i)}\right)$$

$$= -\sum_i\left(y^{(i)} - \phi\left(z^{(i)}\right)\right)x_j^{(i)}$$

尽管Adaline学习规则看起来与感知器一样，但应该注意的是当时$z^{(i)}=w^Tx^{(i)}$时，$\varphi(z^{(i)})$为实数而不是整数分类标签。此外，权重更新是基于训练集中的所有样本计算，而不是在每个样本之后逐步更新权重，这也就是这种方法被称为批量梯度下降的原因。

### 2.3.2 用Python实现Adaline

因为感知器规则和Adaline非常相近，本章将在前面感知器实现的基础上改变fit方法，通过梯度下降最小化代价函数来更新权重。

```
class AdalineGD(object):
    """ADAptive LInear NEuron classifier.

    Parameters
    ------------
    eta : float
      Learning rate (between 0.0 and 1.0)
    n_iter : int
      Passes over the training dataset.
    random_state : int
      Random number generator seed for random weight
      initialization.


    Attributes
    -----------
```

```
    w_ : 1d-array
      Weights after fitting.
    cost_ : list
      Sum-of-squares cost function value in each epoch.

    """
    def __init__(self, eta=0.01, n_iter=50, random_state=1):
        self.eta = eta
        self.n_iter = n_iter
        self.random_state = random_state

    def fit(self, X, y):
        """ Fit training data.

        Parameters
        ----------
        X : {array-like}, shape = [n_samples, n_features]
          Training vectors, where n_samples is the number of
          samples and
          n_features is the number of features.
        y : array-like, shape = [n_samples]
      Target values.

    Returns
    -------
    self : object

    """
    rgen = np.random.RandomState(self.random_state)
    self.w_ = rgen.normal(loc=0.0, scale=0.01,
                          size=1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        net_input = self.net_input(X)
        output = self.activation(net_input)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self

def net_input(self, X):
    """Calculate net input"""
    return np.dot(X, self.w_[1:]) + self.w_[0]

def activation(self, X):
    """Compute linear activation"""
    return X

def predict(self, X):
    """Return class label after unit step"""
    return np.where(self.activation(self.net_input(X))
                    >= 0.0, 1, -1)
```

不像感知器那样在每次训练模型后更新权重，我们根据整个训练集计算梯度，对偏差单元（零权重）调用self.eta*errors.sum（）来计算，对从1到m的权重调用self.eta*X.T.dot（errors），这里X.T.dot（errors）是特征矩阵与误差向量的矩阵向量相乘。

请注意，activation方法对代码没有影响，因为它只是一个标识函数。在这里，我们添加了激活函数（通过调用activation方法计算）来说明信息是如何通过单层神经网络流动的：来自输入数据、净输入、激活和输出的特征。下一章中将了解采用非同一性、非线性激活函数的逻辑回归分类器。将会看到逻辑回归模型与Adaline的唯一区别是它的激活和代价函数密切相关。

与感知器类似，把所收集的代价存储在self.cost_列表以检验训练后的算法是否收敛。

矩阵向量乘法与向量点积非常相似，都把矩阵中的每行当成单一的行向量来计算。这种向量化的方法代表了更紧凑的表示方法，可以用NumPy做更为有效的计算。例如：

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 \\ 8 \\ 9 \end{bmatrix} = \begin{bmatrix} 1\times 7 + 2\times 8 + 3\times 9 \\ 4\times 7 + 5\times 8 + 6\times 9 \end{bmatrix} = \begin{bmatrix} 50 \\ 122 \end{bmatrix}$$

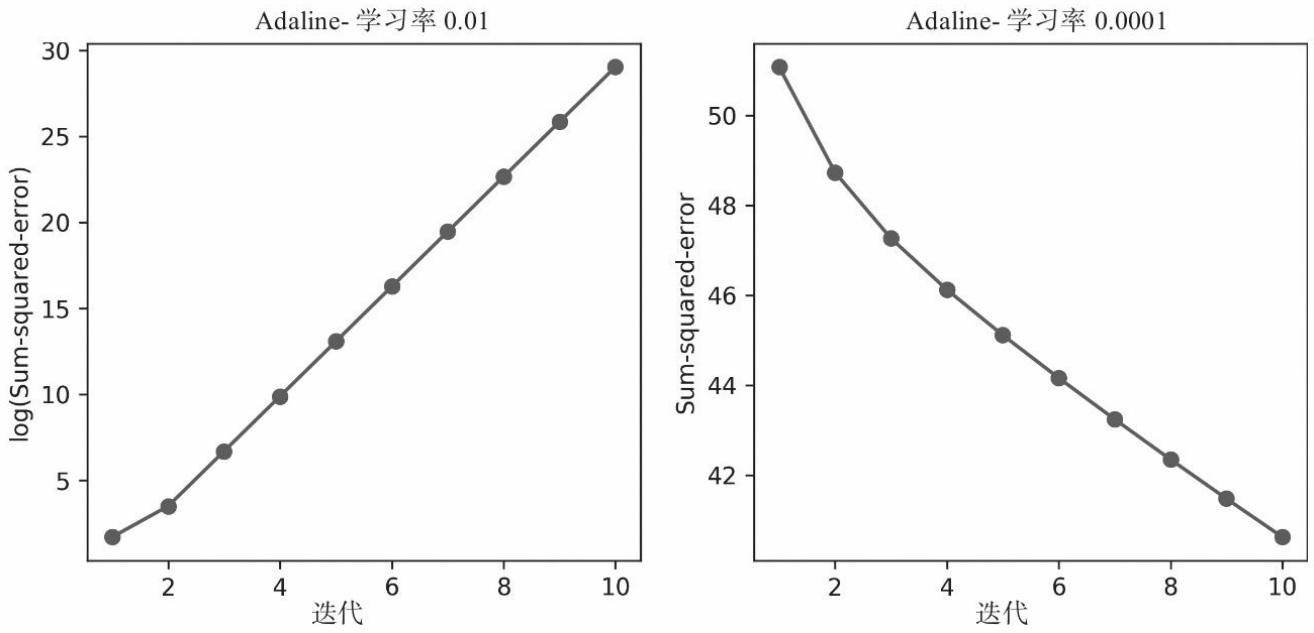
在实践中，常常需要一些实验来找到一个好的学习率η以达到最优收敛。所以选择了η=0.1和η=0.000 1两个不同的学习率，把代价函数与迭代次数在图中画出以观察Adaline如何实现从训练数据中学习。

学习率η（eta）和迭代次数n_tier是感知器和Adaline学习算法的超参数。第6章会分析不同的技术，自动寻找以确保分类模型获得最佳性能所需要的不同超参数值。

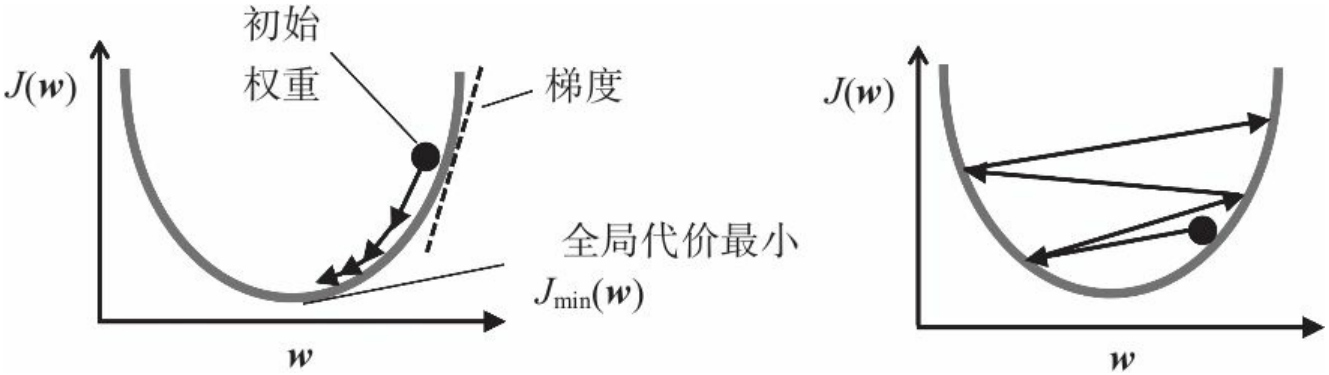
通过下述代码根据两种不同的学习率画出代价与迭代次数之间的关系图：

```
>>> fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))

>>> ada1 = AdalineGD(n_iter=10, eta=0.01).fit(X, y)
>>> ax[0].plot(range(1, len(ada1.cost_) + 1),
...            np.log10(ada1.cost_), marker='o')
>>> ax[0].set_xlabel('Epochs')
>>> ax[0].set_ylabel('log(Sum-squared-error)')
>>> ax[0].set_title('Adaline - Learning rate 0.01')

>>> ada2 = AdalineGD(n_iter=10, eta=0.0001).fit(X, y)
>>> ax[1].plot(range(1, len(ada2.cost_) + 1),
...            ada2.cost_, marker='o')
>>> ax[1].set_xlabel('Epochs')
>>> ax[1].set_ylabel('Sum-squared-error')
>>> ax[1].set_title('Adaline - Learning rate 0.0001')
>>> plt.show()
```

正如从下面绘制的代价函数图所看到的，有两种不同类型的问题。左图显示选择太大的学习率所出现的情况。因为选择的全局最小值过低，无法最小化代价函数，误差经过每次迭代变得越来越大。另一方面，可以看到代价在右图降低，但所选择的学习率η=0.000 1是如此之小，以至于算法需要经过多次迭代才能收敛到全局最低代价：




下图说明如果改变某个特定权重参数值来最小化代价函数J会发生什么情况。左图显示选择一个好的学习率，代价会逐渐降低，向全局最小的方向发展。然而，右图显示如果选择的学习率太大，就会错过全局的最小值。


初始
权重
梯度
J(w)
全局代价最小
$J_{min}(w)$
w
J(w)
w

### 2.3.3 通过调整特征大小改善梯度下降

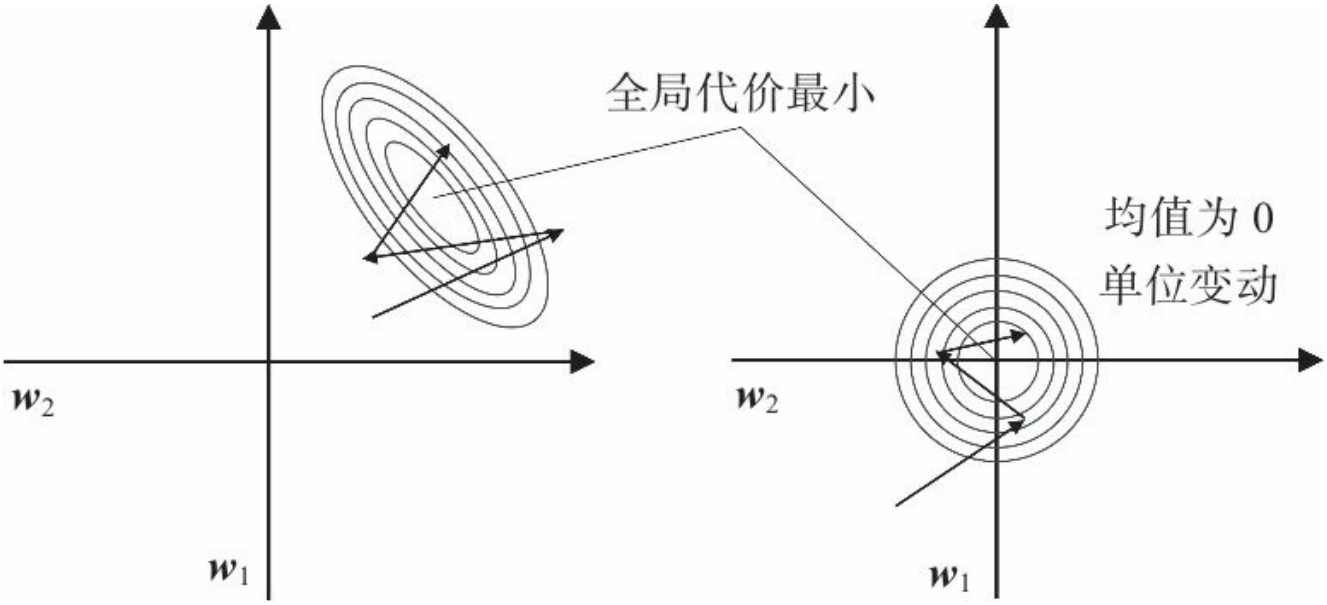
本书遇到的许多机器学习算法需要通过对某种特征进行调整以优化性能，第3章和第4章将详细讨论。

梯度下降是许多可以从特征调整中受益的算法之一。本节将用一种被称为标准化的特征尺度调整方法来加快收敛，它可以使数据具有标准正态分布的特性，有助于梯度下降学习。标准化可以改变每个特征的平均值以使其居中为零，而且每个特征的标准偏差为1。例如，标准化第j个特征，可以简单地从每个训练样本减去平均值，然后除以标准偏差$\sigma_j$：

$$x_j = \frac{x_j - \mu_j}{\sigma_j}$$

这里$x_j$是包含所有n个训练样本的第j个特征值，而该标准化技术应用于数据集的每个特征j。

标准化有助于梯度下降学习的原因之一是优化器必须遍历几个步骤才能发现好的或者最优的解决方案（全局最小值），如下图所示，两个图形代表了代价的表面，是二元分类问题中两个模型权重的函数：



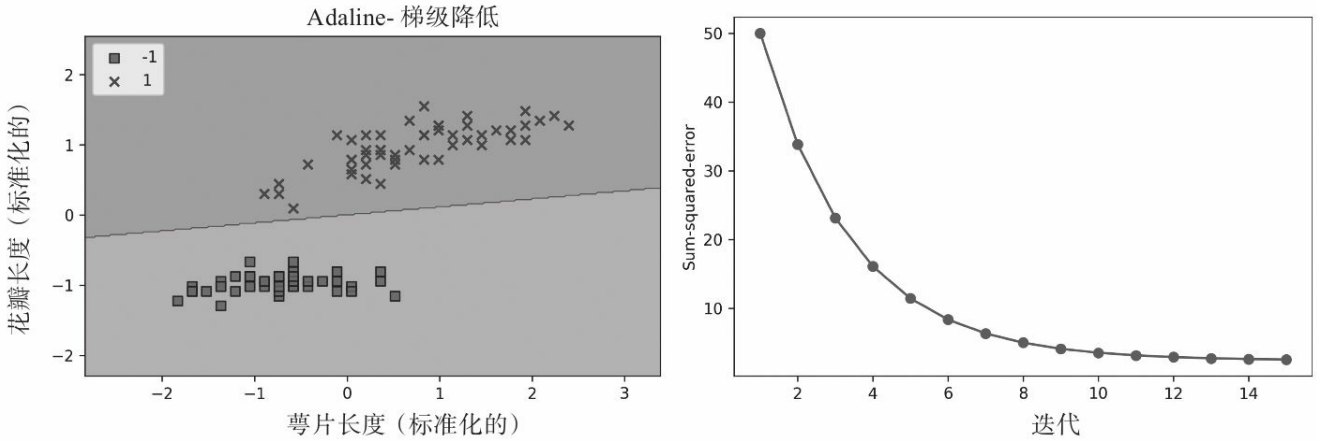

用内置的NumPy方法mean和std可以很容易实现标准化：

```
>>> X_std = np.copy(X)
>>> X_std[:,0] = (X[:,0] - X[:,0].mean()) / X[:,0].std()
>>> X_std[:,1] = (X[:,1] - X[:,1].mean()) / X[:,1].std()
```

标准化完成之后，将再次训练Adaline，然后在学习率η=0.01的条件下，经过几轮迭代后观察其现在的收敛情况：

```
>>> ada = AdalineGD(n_iter=15, eta=0.01)
>>> ada.fit(X_std, y)

>>> plot_decision_regions(X_std, y, classifier=ada)
>>> plt.title('Adaline - Gradient Descent')
>>> plt.xlabel('sepal length [standardized]')
>>> plt.ylabel('petal length [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.tight_layout()
>>> plt.show()

>>> plt.plot(range(1, len(ada.cost_) + 1), ada.cost_, marker='o')
>>> plt.xlabel('Epochs')
>>> plt.ylabel('Sum-squared-error')
>>> plt.show()
```

执行代码后应该可以看到下面所示的决策区域图以及代价下降图：




从图中可以看到，Adaline在学习率η=0.01的情况下，经过训练已经开始收敛。然而，请注意SSE保持非零，即使所有的样本都可以分类正确。

### 2.3.4 大规模机器学习与随机梯度下降

上一节学习了如何基于整个训练集计算代价梯度，从相反方向来最小化代价函数，这就是为什么这种方法有时也被称为批量梯度下降。假设现在有一个拥有数百万个数据点的非常大的数据集，这在许多机器学习应用中并不少见。在这种情况下，运行批量梯度下降在计算上相当昂贵，因为每向全局最小值走一步都需要重新评估整个训练集。

批量梯度下降算法的一种常用替代方法是随机梯度下降法，有时也被称为迭代或在线梯度下降法。该方法并不是基于所有样本$x^{(i)}$的累积误差之和来更新权重：

$$\Delta \boldsymbol{w} = \eta \sum_i \left( y^{(i)} - \phi\left(z^{(i)}\right)\right) \boldsymbol{x}^{(i)}$$

为每个训练样本逐渐更新权重：

$$\eta \left( y^{(i)} - \phi\left(z^{(i)}\right)\right) \boldsymbol{x}^{(i)}$$

虽然随机梯度下降可以被看作是梯度下降的近似，但因为权重更新更加频繁，所以通常收敛得更快。由于每个梯度都是基于单个训练实例计算出来的，误差表面比梯度下降更大，这也有优势，因为如果采用非线性代价函数，随机梯度下降更容易逃脱浅度局部极小值，从本书后面的第12章可以看到。要通过随机梯度下降获得满意的结果，很重要的一点是将训练数据以随机的顺序呈现出来，同时要把训练集重新洗牌以防止迭代循环。

在随机梯度下降实现过程中，固定的学习率η经常被逐步下降的适应性学习率所取代，例如：

$$\frac{c_1}{[\text{迭代数}] + c_2}$$

$c_1$和$c_2$为常数，要注意随机梯度下降没有到达全局的最小值，但是在一个非常靠近这个点的区域。用适应性学习率可以把代价降到最低。

随机梯度下降的另外一个优点是可以用它来做在线学习。对在线学习，

模型可以在数据到达时实时完成训练。这对累积大量数据的情况特别有用。采用在线学习的方法，系统可以立即适应变化，而且在存储空间有限的情况下，可以在更新模型后丢弃训练数据。

批量梯度下降和随机梯度下降之间的折中就是所谓的小批量学习。小批量学习可以理解为对训练数据的较小子集采用批量梯度下降，例如，每次32个样本。小批量梯度下降的优点是可以通过更频繁的权重更新，实现快速收敛。此外，小批量的学习在随机梯度下降过程中用向量化操作取代训练样本的for循环，进一步提高学习算法的计算效率。

因为已经采用梯度下降实现了Adaline自适应学习规则，所以只需要通过随机梯度下降对学习算法更新权重做出些调整。在调用fit方法的过程中，将在每个训练样本之后更新权重。此外，对在线学习，将在实现时调用额外的partial_fit方法，不再重新初始化权重。为了检验算法在训练后是否收敛，将在每次迭代计算训练样本的平均代价。此外，将增加一个选项，在每次迭代开始之前，对训练数据重新洗牌以避免优化代价函数时重复循环。通过random_state参数，允许为反复训练定义随机种子：

```
class AdalineSGD(object):
    """ADAptive LInear NEuron classifier.

    Parameters
    ------------
    eta : float
      Learning rate (between 0.0 and 1.0)
    n_iter : int
      Passes over the training dataset.
    shuffle : bool (default: True)
      Shuffles training data every epoch if True
      to prevent cycles.
    random_state : int
      Random number generator seed for random weight
      initialization.


    Attributes
    -----------
    w_ : 1d-array
      Weights after fitting.
    cost_ : list
  Sum-of-squares cost function value averaged over all
  training samples in each epoch.


"""
def __init__(self, eta=0.01, n_iter=10,
             shuffle=True, random_state=None):
    self.eta = eta
    self.n_iter = n_iter
    self.w_initialized = False
    self.shuffle = shuffle
    self.random_state = random_state

def fit(self, X, y):
    """ Fit training data.

    Parameters
    ----------
    X : {array-like}, shape = [n_samples, n_features]
      Training vectors, where n_samples is the number
      of samples and
      n_features is the number of features.
    y : array-like, shape = [n_samples]
      Target values.

    Returns
    -------
    self : object
```

```
        """
        self._initialize_weights(X.shape[1])
        self.cost_ = []
        for i in range(self.n_iter):
            if self.shuffle:
                X, y = self._shuffle(X, y)
            cost = []
            for xi, target in zip(X, y):
                cost.append(self._update_weights(xi, target))
            avg_cost = sum(cost) / len(y)
            self.cost_.append(avg_cost)
        return self

    def partial_fit(self, X, y):
        """Fit training data without reinitializing the weights"""
        if not self.w_initialized:
            self._initialize_weights(X.shape[1])
        if y.ravel().shape[0] > 1:
            for xi, target in zip(X, y):
                self._update_weights(xi, target)
        else:
            self._update_weights(X, y)
        return self

    def _shuffle(self, X, y):
        """Shuffle training data"""
        r = self.rgen.permutation(len(y))
        return X[r], y[r]

    def _initialize_weights(self, m):
        """Initialize weights to small random numbers"""
        self.rgen = np.random.RandomState(self.random_state)
        self.w_ = self.rgen.normal(loc=0.0, scale=0.01,
                                   size=1 + m)
        self.w_initialized = True

    def _update_weights(self, xi, target):
        """Apply Adaline learning rule to update the weights"""
        output = self.activation(self.net_input(xi))
        error = (target - output)
        self.w_[1:] += self.eta * xi.dot(error)
        self.w_[0] += self.eta * error
        cost = 0.5 * error**2
        return cost

    def net_input(self, X):
        """Calculate net input"""
        return np.dot(X, self.w_[1:]) + self.w_[0]

    def activation(self, X):
        """Compute linear activation"""
        return X

    def predict(self, X):
        """Return class label after unit step"""
```

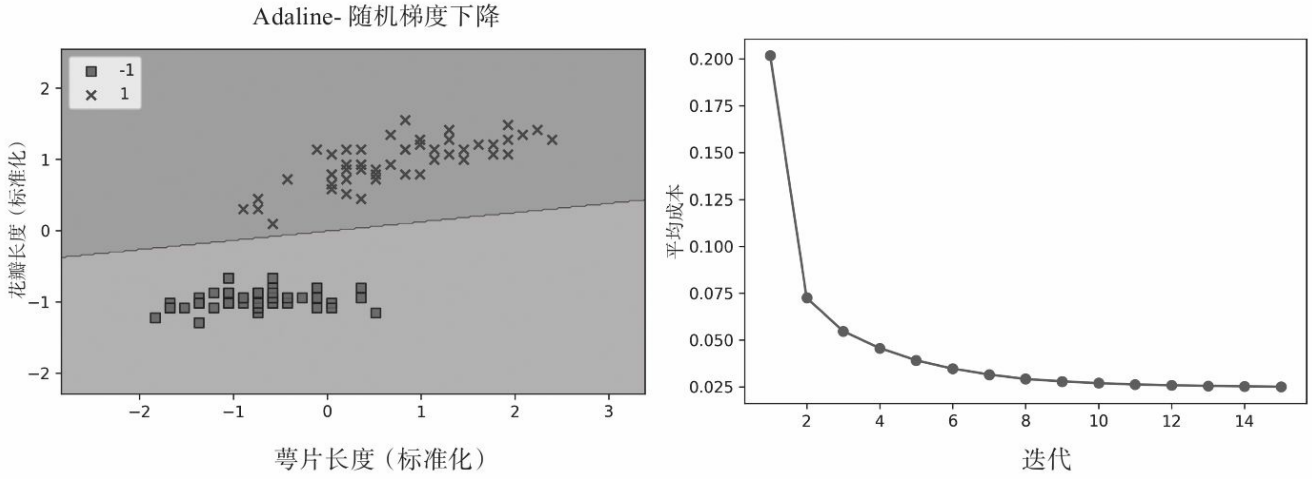
```
        return np.where(self.activation(self.net_input(X))
                        >= 0.0, 1, -1)
```

AdalineSGD分类器调用的_shuffle方法的实现过程如下：通过调用np.random中permutation函数产生范围从0到100的独立数字的随机序列。然后以这些数字作为索引来对特征矩阵和分类标签向量洗牌。

可以调用fit方法来训练AdalineSGD分类器，用plot_decision_regions把训练结果用图形表示出来。

```
>>> ada = AdalineSGD(n_iter=15, eta=0.01, random_state=1)
>>> ada.fit(X_std, y)

>>> plot_decision_regions(X_std, y, classifier=ada)
>>> plt.title('Adaline - Stochastic Gradient Descent')
>>> plt.xlabel('sepal length [standardized]')
>>> plt.ylabel('petal length [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.show()
>>> plt.plot(range(1, len(ada.cost_) + 1), ada.cost_, marker='o')
>>> plt.xlabel('Epochs')
>>> plt.ylabel('Average Cost')
>>> plt.show()
```

通过执行前面的示例代码可以得到下面两张图：




正如上图可以看到的，平均代价降低得非常快，在15次迭代后，最终的决策边界看起来与批量梯度Adaline的下降结果类似。如果要更新模型，例如，流式数据的在线学习，可以对单个样本直接调用partial_fit方法——例如，ada.partial_fit（X_std[0，：]，y[0]）。

## 2.4 小结

本章我们对有监督学习的线性分类器的基本概念有了很好的理解。通过实现感知器了解了如何通过向量化实现梯度下降和基于随机梯度下降的在线学习，有效地训练自适应线性神经元。

现在我们已经看到了如何用Python实现简单的分类器，这为下一章的学习做好了准备，我们将用Python语言的scikit-learn机器学习软件库获得更先进和更强大、在学术和工业界中常用的机器学习分类器。用面向对象的方法来实现感知器和Adaline自适应算法，这将有助于了解scikit-learn API，它是基于本章的相同核心理念通过调用fit和predict方法实现的。我们将基于这些核心理念学习关于逻辑回归的知识，包括用来处理非线性决策边界问题的模拟分类概率和支持向量机。此外，将介绍一种不同类型的基于树的有监督学习算法，这些算法通常被合并进入强大的集成分类器中。

# 第3章　scikit-learn机器学习分类器一览

本章将浏览学术界和工业界常用且强大的一系列机器学习算法。了解几个有监督学习分类算法之间的差异，培养鉴别算法优劣的直觉。此外，scikit-learn软件库为有效率且富有成果地使用这些算法提供了友好的用户界面，本章将继续使用。

本章将主要涵盖下述几个方面：

·介绍强大且常用的分类算法，如逻辑回归、支持向量机和决策树。

·scikit-learn机器学习库通过对用户友好的Python API提供各种机器学习算法，本章将介绍实例并对其进行解释。

·讨论线性和非线性决策边界分类器的优劣。

## 3.1 选择分类算法

每种算法都是基于某些假设且有其自身的特点，为特定问题选择合适的分类算法需要实践经验。戴维·沃尔珀特提出的“没有免费午餐的定理”，明确说明不存在适合所有可能场景的分类算法（戴维·沃尔珀特.《无明显差别的学习算法》.神经元计算8.7（1996）：1341-1390）。在实践中，因为样本特征的数量、数据中的噪声以及是否线性可分等方面有所不同，所以建议比较至少几种不同学习算法的性能，以选择适合特定问题的最佳模型。

分类器最终的计算性能以及预测能力在很大程度上取决于可供学习的基础数据。训练机器学习算法的主要步骤可以概括如下：

1.选择特征并收集训练样本。

2.选择度量性能的指标。

3.选择分类器并优化算法。

4.评估模型的性能。

5.调整算法。

本书的方法是逐步构建机器学习的知识，本章将主要聚焦在不同算法的主要概念上，并针对诸如特征选择、预处理、性能指标和超参数调整等主题进行回顾，本书的后半部分将对此进行更详细的讨论。

## 3.2 了解scikit-learn软件库的第一步——训练感知器

你已经在第2章学习了两个机器学习分类算法，亲手实现了感知规则和Adaline。现在来看一下scikit-learn的API，它结合了对用户友好的界面和几种高度优化的分类算法。scikit-learn软件库不仅提供了大量的学习算法，同时也包含了预处理数据、微调和评估模型等许多方便的功能。第4章和第5章将对此及其他基本概念进行更详细的讨论。

作为理解scikit-learn软件库的起点，本章将训练类似在第2章中实现的感知器。为了简单起见，本书将在以后章节中继续用已经熟悉的鸢尾花数据集。这么做很方便，因为该数据集简单且常见，经常被用来测试和检验算法，况且已经在前面使用scikit-learn软件库过程中获得了该数据集。下面将用鸢尾花数据的两个特征来实现可视化。

把150个鸢尾花样本的花瓣长度和宽度存入特征矩阵X，把相应的品种分类标签存入向量y：

```
>>> from sklearn import datasets
>>> import numpy as np

>>> iris = datasets.load_iris()
>>> X = iris.data[:, [2, 3]]
>>> y = iris.target
>>> print('Class labels:', np.unique(y))
Class labels: [0 1 2]
```

np.unique（y）函数把返回的三个独立分类标签存储在iris.target，把鸢尾花iris-setosa，Iris-versicolor和iris-virginica以整数（0，1，2）存储。虽然许多scikit-learn函数和分类方法也能处理字符串型的分类标签，但采用整数作为分类标签是值得推荐的方法，这样不但可以避免技术故障，并且由于内存占用更小可以提高计算性能。此外，分类标签编码为整数是大多数机器学习库常见的约定。为了评估经过训练的模型对未知数据处理的效果，再进一步将数据集分裂成单独的训练集和测试集。本书将在第6章中，围绕模型评估的最佳实践进行更详细的讨论：

```
>>> from sklearn.model_selection import train_test_split
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.3, random_state=1, stratify=y)
```

利用scikit-learn库model_selection模块的train_test_split函数，把X和y阵列随机分为30%的测试数据（45个样本）和70%的训练数据（105个样本）。

请注意，train_test_split函数在分割数据之前已经在内部将训练洗牌；否则，所有分类标签为0和1的样本都被分去了训练集，所有分类标签为2的45份样本数据都被分去了测试集。通过设置random_state参数，为内部的伪随机数发生器提供一个固定的随机种子（random_state=1），该发生器用于在分割数据集之前洗牌。采用这样固定的random_state可以确保结果可重复。

最后，通过定义stratify=y获得内置的分层支持。这种分层意味着调用train_test_split方法可以返回与输入数据集的分类标签相同比例的训练和测试数据子集。可以调用NumPy的bincount函数来对阵列中的每个值进行统计，以验证数据：

```
>>> print('Labels counts in y:', np.bincount(y))
Labels counts in y: [50 50 50]
>>> print('Labels counts in y_train:', np.bincount(y_train))
Labels counts in y_train: [35 35 35]
>>> print('Labels counts in y_test:', np.bincount(y_test))
Labels counts in y_test: [15 15 15]
```

许多机器学习和优化算法也需要对特征进行调整以获得最佳性能，正如在第2章的梯度下降示例中所看到的那样。在这里，将通过调用scikit-learn软件库中的预处理模块preprocessing中的类StandardScaler来对特征进行标准化：

```
>>> from sklearn.preprocessing import StandardScaler
>>> sc = StandardScaler()
>>> sc.fit(X_train)
>>> X_train_std = sc.transform(X_train)
>>> X_test_std = sc.transform(X_test)
```

用前面的代码加载预处理模块preprocessing中的StandardScaler类，初始化一个新的StandardScaler对象，然后分配给变量sc。调用StandardScaler的fit方法对训练数据的每个特征维度参数μ（样本均值）和σ（标准偏差）进行估算。然后调用transform方法，利用估计的参数μ和σ对训练数据进行标准化。在标准化测试集时，要注意使用相同的特征调整参数以确保训练集与测试集的数值具有可比性。

标准化训练数据后，可以训练感知器模型。scikit-learn软件库的大多数算法通过调用一对其他（OvR）方法，默认支持多元分类，允许把三种花的数据同时提交给感知器。具体代码实现如下：

```
>>> from sklearn.linear_model import Perceptron

>>> ppn = Perceptron(n_iter=40, eta0=0.1, random_state=1)
>>> ppn.fit(X_train_std, y_train)
```

scikit-learn界面让我们想起在第2章中实现感知器的过程：从linear_model模块加载Perceptron类之后，初始化新的Perceptron对象，然后调用fit方法对模型进行训练。在这里，模型参数eta0相当于前面动手实现的感知器中用到的学习率，n_iter参数定义了遍历训练集的迭代次数。

记得在第2章中讨论过，要找到合适的学习率需要一些试验。学习率过大，算法会错过全局的最小成本点。学习率过小，算法需要经过更多的迭代才会收敛，降低学习速度，这对大型数据集尤为明显。同时，用random_state参数确保在每次迭代后训练集初始洗牌的结果可以重现。

用scikit-learn训练完模型后，可以调用predict方法做预测，就像在第2章中动手实现的感知器那样，具体代码如下：

```
>>> y_pred = ppn.predict(X_test_std)
>>> print('Misclassified samples: %d' % (y_test != y_pred).sum())
Misclassified samples: 3
```

执行代码后，可以看到感知器在处理45个花朵样本时出现过3次错误。因此，测试集上的分类错误率大约为0.067或6.7%（6/45≈0.067）。

许多机器学习实践者报告了模型的分类准确度，而不是错误分类错误率，简单计算如下：

1-分类错误率=0.933或者93.3%

scikit-learn也实现了大量各种不同的性能指标，可以调用metrics模块来使用。例如，可以计算测试集上感知器的分类准确度，如下所示：

```
>>> from sklearn.metrics import accuracy_score
>>> print('Accuracy: %.2f' % accuracy_score(y_test, y_pred))
Accuracy: 0.93
```

这里y_test是正确分类标签，而y_pred是先前预测的分类标签。另外，每个scikit-learn分类器都有一个评分方法，可以通过综合调用predict和accuracy_score计算出分类器的预测准确度。

```
>>> print('Accuracy: %.2f' % ppn.score(X_test_std, y_test))
Accuracy: 0.93
```

注意，我们根据本章测试集评估模型的性能。第5章将学到包括图形分析在内的有用技术，如用来检测和防止过拟合的学习曲线。过拟合意味着虽然模型可以捕捉训练数据中的模式，但却不能概括未见过的新数据。

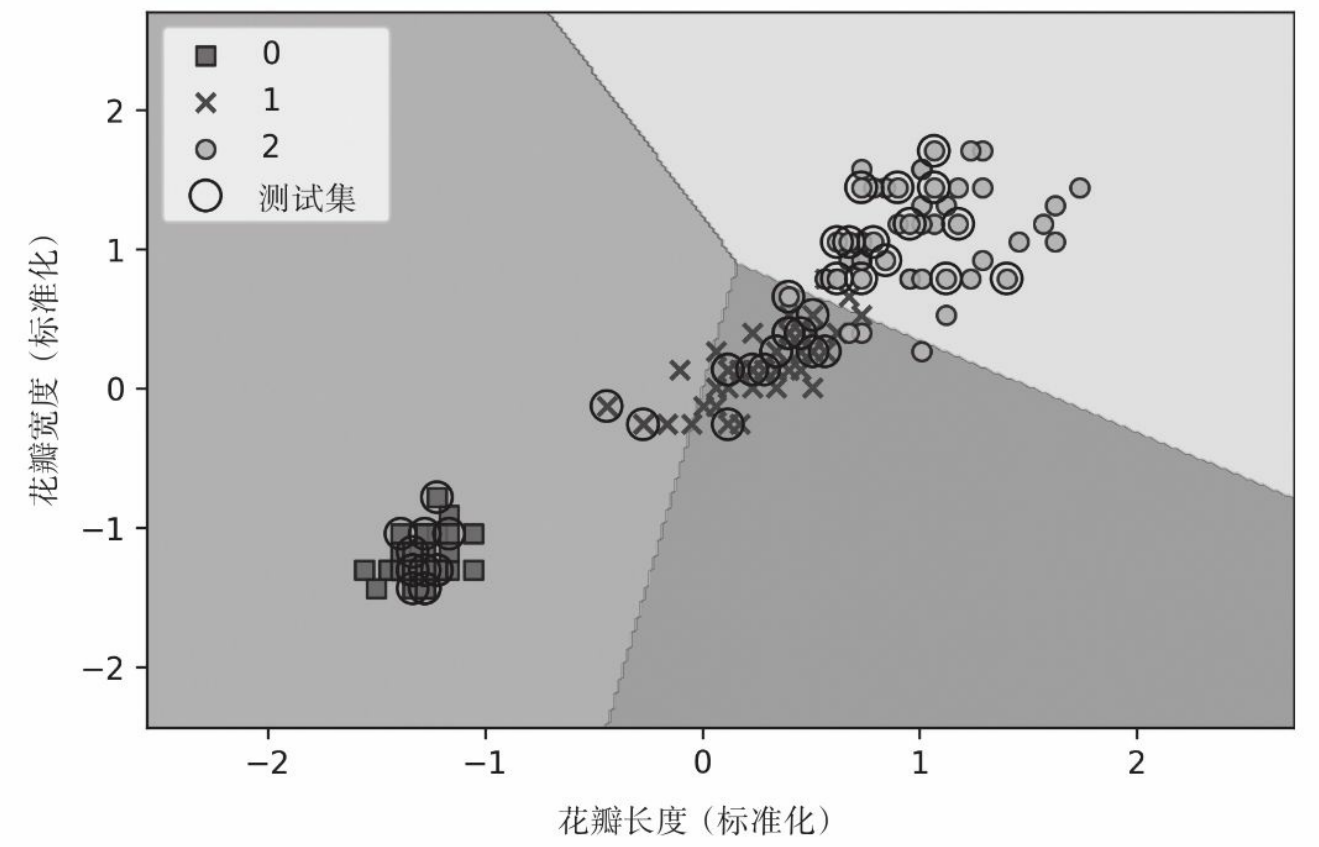
最后，可以利用第2章中的plot_decision_regions函数绘制新训练感知器的模型决策区，并以可视化的方式展示区分不同花朵样本的效果。但是，略加修改以通过圆圈来突出显示来自测试集的样本：

```
from matplotlib.colors import ListedColormap
import matplotlib.pyplot as plt


def plot_decision_regions(X, y, classifier, test_idx=None,
                          resolution=0.02):

    # setup marker generator and color map
    markers = ('s', 'x', 'o', '^', 'v')
    colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
    cmap = ListedColormap(colors[:len(np.unique(y))])

    # plot the decision surface
    x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
                       np.arange(x2_min, x2_max, resolution))
Z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
Z = Z.reshape(xx1.shape)
plt.contourf(xx1, xx2, Z, alpha=0.3, cmap=cmap)
plt.xlim(xx1.min(), xx1.max())
plt.ylim(xx2.min(), xx2.max())

for idx, cl in enumerate(np.unique(y)):
    plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1],
                alpha=0.8, c=colors[idx],
                marker=markers[idx], label=cl,
                edgecolor='black')

# highlight test samples

if test_idx:
    # plot all samples
    X_test, y_test = X[test_idx, :], y[test_idx]

    plt.scatter(X_test[:, 0], X_test[:, 1],
                c='', edgecolor='black', alpha=1.0,
                linewidth=1, marker='o',
                s=100, label='test set')
```

通过对plot_decision_regions函数的小幅修改，可以定义在结果图上标记的样本索引。代码如下：

```
>>> X_combined_std = np.vstack((X_train_std, X_test_std))
>>> y_combined = np.hstack((y_train, y_test))
>>> plot_decision_regions(X=X_combined_std,
...                       y=y_combined,
...                       classifier=ppn,
...                       test_idx=range(105, 150))
>>> plt.xlabel('petal length [standardized]')
>>> plt.ylabel('petal width [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

正如结果图所看到的，三种花不能被线性决策边界完全分离：




第2章曾讨论过感知器算法从不收敛于不完全线性可分离的数据集，这就是为什么在实践中通常不推荐使用感知器算法。下面的章节将研究更强大的可以收敛到成本最小值的线性分类器，即使这些类并不是完全线性可分的。

Perceptron以及其他scikit-learn函数和类经常会有我们不清楚的额外参数。可以通过Python的帮助功能阅读更多关于这些参数的描述。或者通过杰出的scikit-learn在线文档http://scikit-learn.org/stable/上网学习。

## 3.3 基于逻辑回归的分类概率建模

虽然感知器规则提供了良好且易用的入门级机器学习分类算法，但其最大缺点是如果类不是完全线性可分的，那么它永远都不收敛。前一节的分类任务就是该场景的一个例子。直观地说，可以把原因想成权重在不断更新，因为每个迭代至少会有一个错误分类样本存在。当然，也可以改变学习率，增加迭代次数，但要小心感知器永远不会在该数据集上收敛。为了提高效率，现在来看另一种简单且强大的解决线性二元分类问题的算法，即逻辑回归。不要望文生义，逻辑回归是一种分类模型，但不是回归模型。

### 3.3.1 逻辑回归的直觉与条件概率

逻辑回归是一种很容易实现的分类模型，但仅在线性可分类上表现不错。它是行业中应用最广泛的分类算法之一。与感知器和Adaline类似，本章所介绍的逻辑回归模型也是二元分类的线性模型，而且可以利用OvR技术扩展到多元分类。

要解释作为概率模型的逻辑回归原理，首先要介绍让步比，即有利于某一特定事件的概率。让步比可以定义为$\frac{p}{(1-p)}$，p代表阳性事件的概率。阳性事件并不一定意味着好，它指的是要预测的事件，例如，病人有某种疾病的可能性。可以认为阳性事件的分类标签y=1。可以进一步定义logit函数，这仅仅是让步比的对数形式（log-odds）：

$$logit(p)=\log\frac{p}{(1-p)}$$

注意log是自然对数，与计算机科学中的通用惯例一致。logit函数输入值的取值范围在0到1之间，转换或计算的结果值为整个实数范围，可以用它来表示特征值和对数概率（log-odds）之间的线性关系：

$$logit\left(p\left(y=1|\boldsymbol{x}\right)\right)=w_0x_0+w_1x_1+\cdots+w_mx_m=\sum_{i=0}^{m}w_ix_i=\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}$$

这里p（y=1|x）是某个特定样本属于x类给定特征标签为1的条件概率。

实际上感兴趣的是预测某个样本属于某个特定类的概率，它是logit函数的逆形式。也被称为逻辑sigmoid函数，由于其特别的S形，有时干脆简称为sigmoid函数：

$$\phi\ (z)=\frac{1}{1+\mathrm{e}^{-z}}$$

这里z为净输入，是权重和样本特征的线性组合，$z=w^{\mathrm{T}}x=w_0x_0+w_1x_1+\ldots+w_mx_m$。

注意，与第2章中用过的约定类似，$w_0$代表偏置单位，是为$x_0$提供

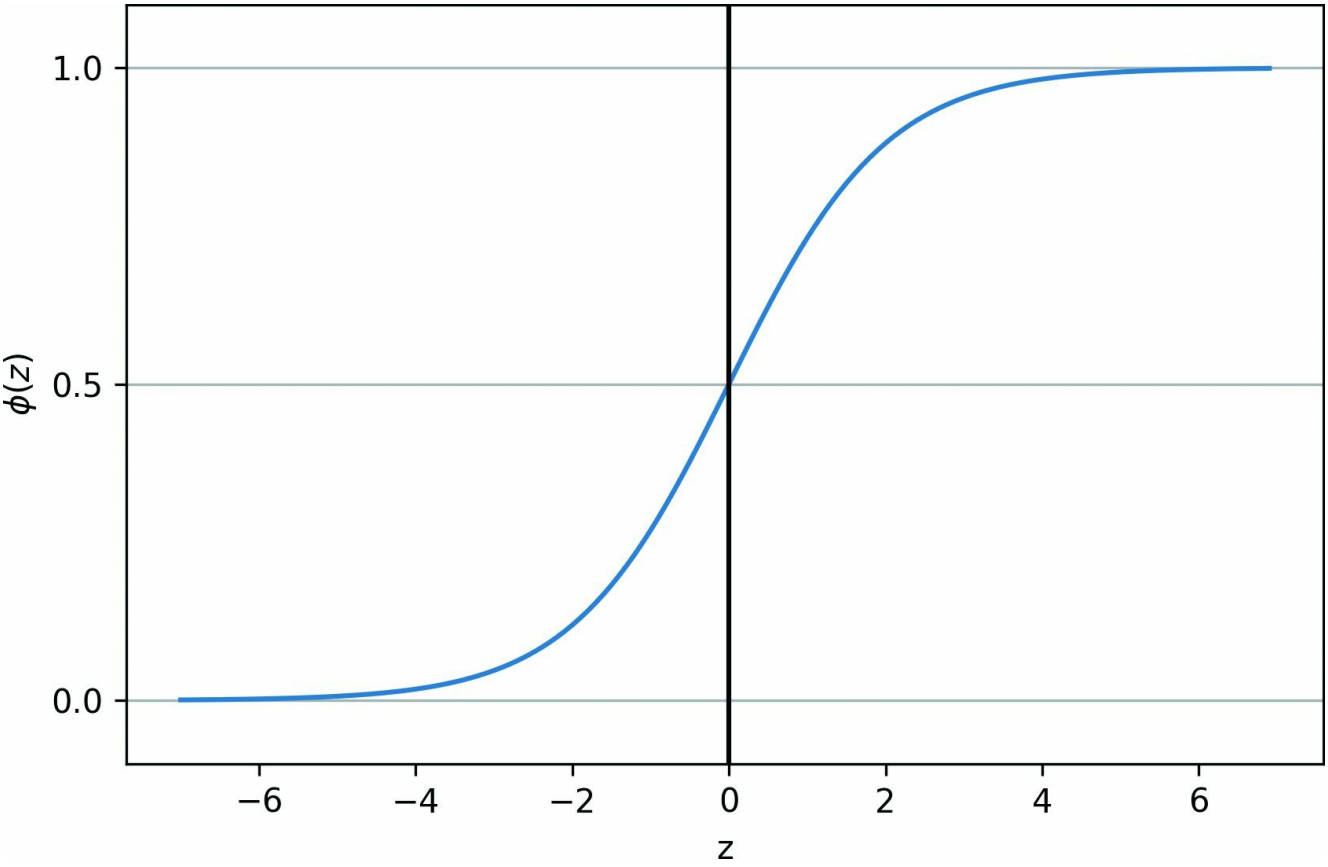
的额外输入值，其值为1。

现在简单地用-7到7之间的一些值绘出sigmoid函数来观察具体的情况：

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> def sigmoid(z):
...     return 1.0 / (1.0 + np.exp(-z))
```

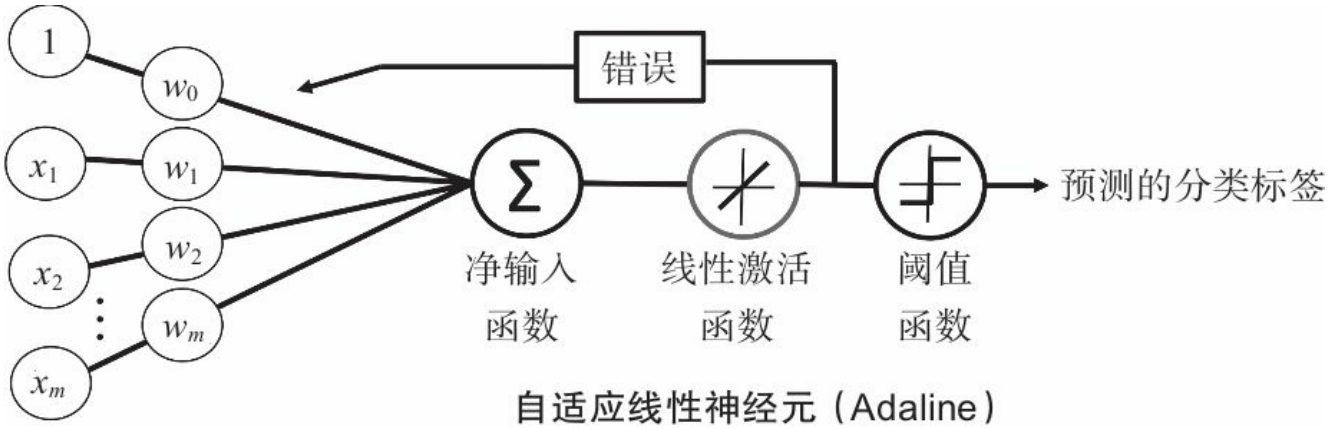
```
>>> z = np.arange(-7, 7, 0.1)
>>> phi_z = sigmoid(z)
>>> plt.plot(z, phi_z)
>>> plt.axvline(0.0, color='k')
>>> plt.ylim(-0.1, 1.1)
>>> plt.xlabel('z')
>>> plt.ylabel('$\phi (z)$')
>>> # y axis ticks and gridline
>>> plt.yticks([0.0, 0.5, 1.0])
>>> ax = plt.gca()
>>> ax.yaxis.grid(True)
>>> plt.show()
```

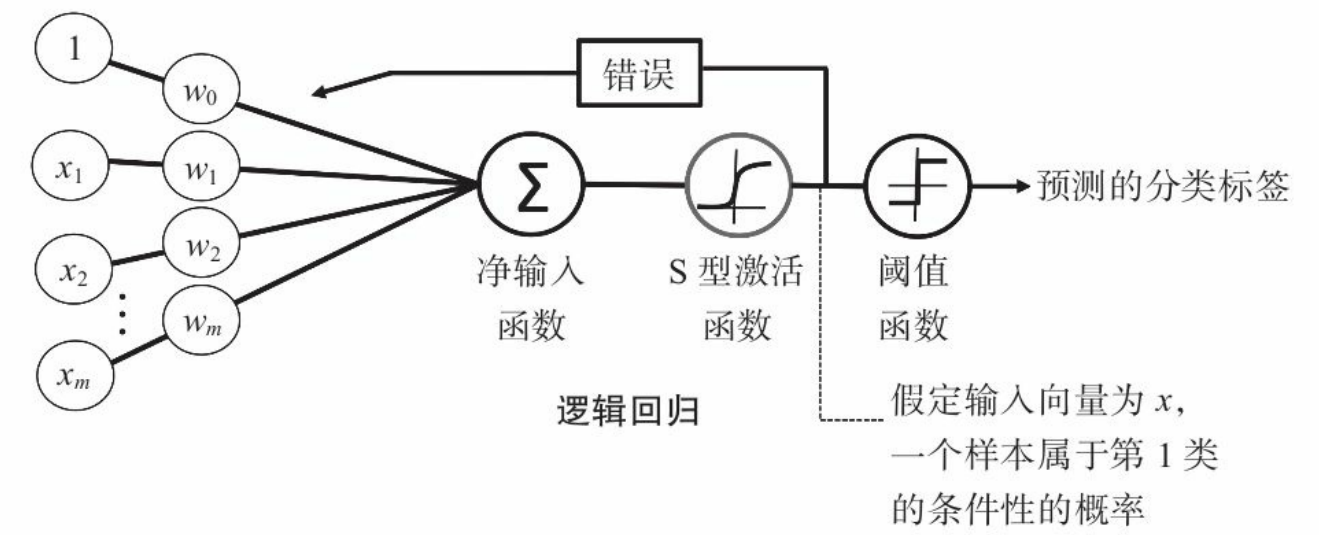
执行前面的代码示例，现在应该能看到S形曲线：




从上图可以看到，当z趋向无限大时（z→∞），φ（z）的值接近于1，因为当z值很大时，$e^{-z}$的值会变得非常小。类似，作为越来越大的分母，当z→∞时，（z）的值趋于0。因此得出这样的结论：S函数将以实数值作为输入，并在截距为（z）=0.5时转换为范围在[0，1]之间的值。

为了获得对逻辑回归模型的一些直觉概念，可以把它与第2章联系起来。Adaline用恒等函数φ（z）=z作为激活函数。在逻辑回归中，这个激活函数简单地变成前面定义的S函数。Adaline和逻辑回归的区别如下图所示：






S函数的输出接着被翻译成样本属于第1类的概率，（z）=P（y=1|x；w），假设特征x被权重w参数化。例如对某种花的样本，计算出其（z）=0.8，该样本属于Iris-versicolor花的机会为80%。因此，该样本属于Iris-setosa的概率可以计算为P（y=0|x；w）=1-P（y=1|x；w）=0.2或者20%。预测概率可以通过阈函数简单地转换为二元输出：

$$\hat{y} = \begin{cases} 1 & \text{if} \phi(z) \geqslant 0.5 \\ 0 & \text{otherwise} \end{cases}$$

前面的S函数图等同于下述结果：

$$\hat{y} = \begin{cases} 1 & \text{if } z \geqslant 0.0 \\ 0 & \text{otherwise} \end{cases}$$

事实上，有许多应用程序不仅对预测分类标签感兴趣，而且也对评估类中成员概率也特别有兴趣（在应用阈值函数之前S函数的输出）。例如，天气预报用逻辑回归不仅能预测某天是否会下雨，而且还能预报下雨的可能性。同样，可以用逻辑回归来预测病人在某些症状下有特定疾病的机会，这就是为什么逻辑回归在医学领域备受欢迎。

## 3.3.2 学习逻辑代价函数的权重

学习了如何使用逻辑回归模型来预测概率和分类标签，现在简要讨论一下如何拟合模型的参数，例如前一章的权重w。上一章定义平方和误差代价函数为：

$$J(\boldsymbol{w})=\sum_{i}\frac{1}{2}\left(\phi\left(z^{(i)}\right)-y^{(i)}\right)^{2}$$

为了在Adaline分类模型中学习权重w，我们简化了函数。要解释清楚如何得到逻辑回归的代价函数，需要首先定义在建立逻辑回归模型时想要最大化的可能性L，假设数据集中的样本都是相互独立的个体。公式如下：

$$L(\boldsymbol{w})=P(y\mid\boldsymbol{x};\boldsymbol{w})=\prod_{i=1}^{n}P\left(y^{(i)}\mid\boldsymbol{x}^{(i)};\boldsymbol{w}\right)=\prod_{i=1}^{n}\left(\phi\left(z^{(i)}\right)\right)^{y^{(i)}}\left(1-\phi\left(z^{(i)}\right)\right)^{1-y^{(i)}}$$

在实践中，最大化该方程的自然对数，也被称为对数似然函数：

$$l(\boldsymbol{w})=\log\left(L(\boldsymbol{w})\right)=\sum_{i=1}^{n}\left[y^{(i)}\log\left(\phi\left(z^{(i)}\right)\right)+\left(1-y^{(i)}\right)\log\left(1-\phi\left(z^{(i)}\right)\right)\right]$$

首先，应用对数log函数降低数值下溢的可能性，这种情况在似然率非常小的情况可能发生。其次，假如你还记得微积分的话，可以把因子乘积转换成因子求和，这样就可以通过加法技巧更容易地得到该函数的导数。

现在可以用诸如梯度上升等优化算法来最大化这个对数似然函数。另外一个选择是重写对数log似然函数作为代价函数J，像在第2章中那样，用梯度下降方法最小化代价函数J：

$$J(\boldsymbol{w})=\sum_{i=1}^{n}\left[-y^{(i)}\log\left(\phi\left(z^{(i)}\right)\right)-\left(1-y^{(i)}\right)\log\left(1-\phi\left(z^{(i)}\right)\right)\right]$$

为了更好地理解这个代价函数，让我们计算一个样本训练实例的代价：

$$J(\phi(z),y;\boldsymbol{w})=-y\log(\phi(z))-(1-y)\log(1-\phi(z))$$

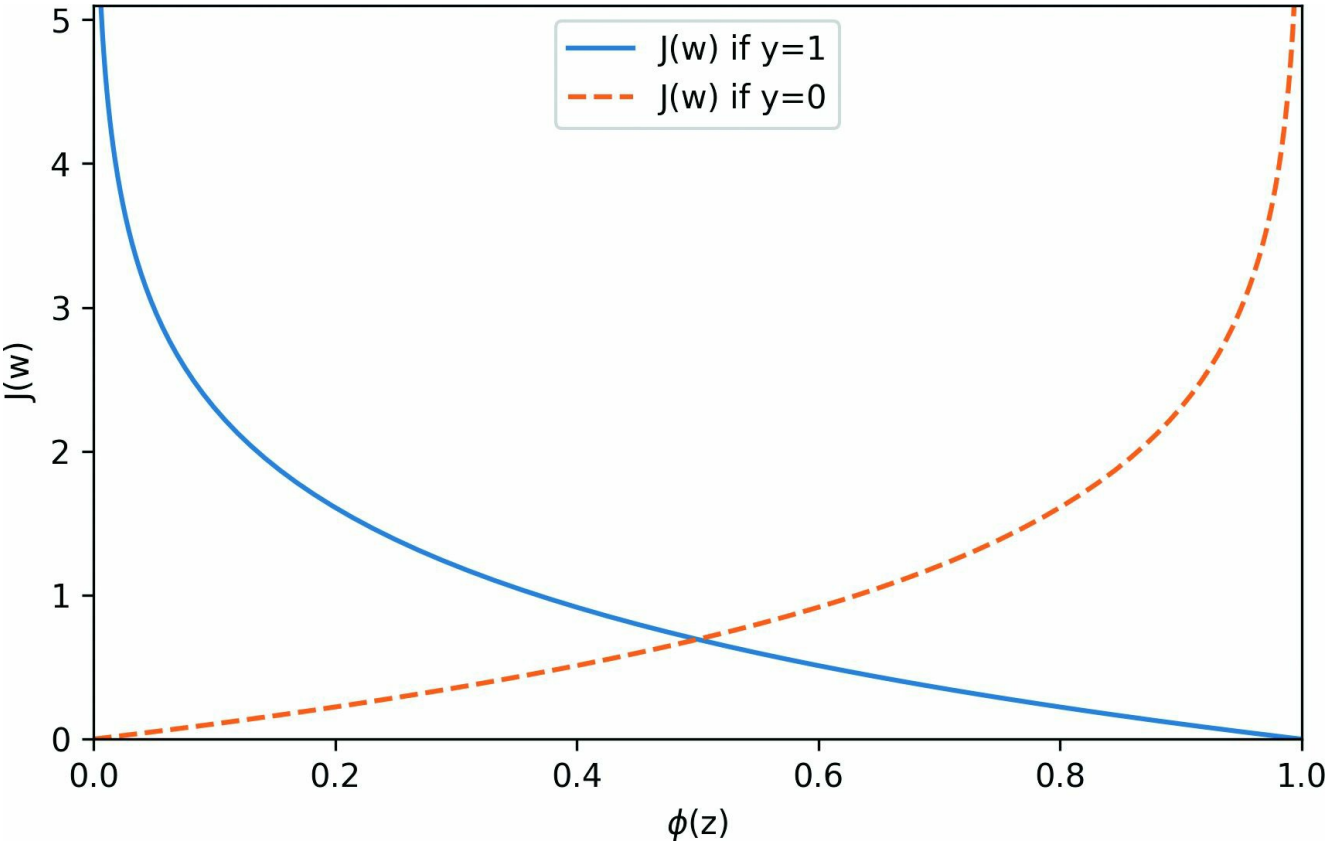
从方程中可以看到，如果y=0，第一项为零，如果y=1，第二项为零：

$$J\left(\phi\left(z\right), y; \boldsymbol{w}\right) = \begin{cases} -\log\left(\phi\left(z\right)\right) & \text{if } y = 1 \\ -\log\left(1 - \phi\left(z\right)\right) & \text{if } y = 0 \end{cases}$$

通过下述简短的代码来绘制一张图来说明（z）不同样本实例分类的代价：

```
>>> def cost_1(z):
...     return - np.log(sigmoid(z))
>>> def cost_0(z):
...     return - np.log(1 - sigmoid(z))
>>> z = np.arange(-10, 10, 0.1)
>>> phi_z = sigmoid(z)
>>> c1 = [cost_1(x) for x in z]
>>> plt.plot(phi_z, c1, label='J(w) if y=1')
>>> c0 = [cost_0(x) for x in z]
>>> plt.plot(phi_z, c0, linestyle='--', label='J(w) if y=0')
>>> plt.ylim(0.0, 5.1)
>>> plt.xlim([0, 1])
>>> plt.xlabel('$\phi$(z)')
>>> plt.ylabel('J(w)')
>>> plt.legend(loc='best')
>>> plt.show()
```

结果图中的x轴显示了S函数的激活情况。范围在0到1之间（S函数的输入值z的范围在-10到10之间），y轴是相关联的逻辑代价。




从上图可以看到，如果正确地预测样本属于第1类，代价就会接近0（实线）。类似地，可以在y轴上看到，如果正确地预测y=0（虚线），那么代价也接近0。然而，如果预测错误，代价就会趋于无穷大。关键在于用越来越大的代价惩罚错误的预测。

### 3.3.3 把转换的Adaline用于逻辑回归算法

如果要自己动手实现逻辑回归，可以直接用新的代价函数取代第2章中实现的Adaline代价函数J：

$$J(\boldsymbol{w}) = -\sum_i y^{(i)} \log\left(\phi\left(z^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log\left(1 - \phi\left(z^{(i)}\right)\right)$$

在对训练样本进行分类的过程中，用该公式来计算每次迭代的代价。另外，需要用S激活函数替换线性激活函数，同时把阈值函数的返回类标签从0变为1，不再返回-1和1。如果能在Adaline编码中完成这三步，即可获得下述逻辑回归的代码实现：

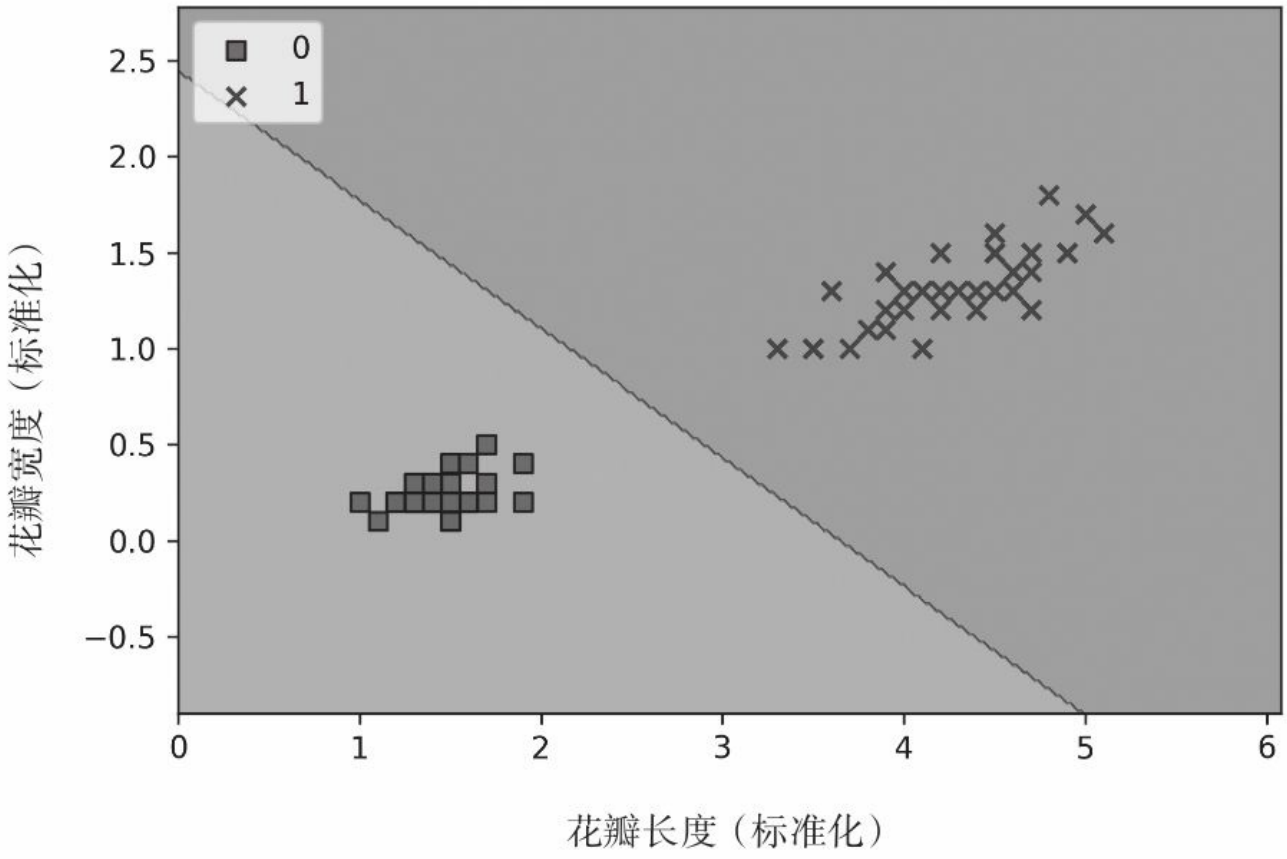
```
class LogisticRegressionGD(object):
    """Logistic Regression Classifier using gradient descent.

    Parameters
    ------------
    eta : float
      Learning rate (between 0.0 and 1.0)
    n_iter : int
      Passes over the training dataset.
    random_state : int
      Random number generator seed for random weight
      initialization.


    Attributes
    -----------
    w_ : 1d-array
      Weights after fitting.
    cost_ : list
  Sum-of-squares cost function value in each epoch.

"""
def __init__(self, eta=0.05, n_iter=100, random_state=1):
    self.eta = eta
    self.n_iter = n_iter
    self.random_state = random_state

def fit(self, X, y):
    """ Fit training data.

    Parameters
    ----------
    X : {array-like}, shape = [n_samples, n_features]
      Training vectors, where n_samples is the number of
      samples and
      n_features is the number of features.
    y : array-like, shape = [n_samples]
      Target values.
```

```
        Returns
        -------
        self : object

        """
        rgen = np.random.RandomState(self.random_state)
        self.w_ = rgen.normal(loc=0.0, scale=0.01,
                              size=1 + X.shape[1])
        self.cost_ = []

        for i in range(self.n_iter):
            net_input = self.net_input(X)
            output = self.activation(net_input)
            errors = (y - output)
            self.w_[1:] += self.eta * X.T.dot(errors)
            self.w_[0] += self.eta * errors.sum()

            # note that we compute the logistic `cost` now
            # instead of the sum of squared errors cost
            cost = (-y.dot(np.log(output)) -
                    ((1 - y).dot(np.log(1 - output))))
            self.cost_.append(cost)
        return self

    def net_input(self, X):
        """Calculate net input"""
        return np.dot(X, self.w_[1:]) + self.w_[0]

    def activation(self, z):
        """Compute logistic sigmoid activation"""
        return 1. / (1. + np.exp(-np.clip(z, -250, 250)))

    def predict(self, X):
        """Return class label after unit step"""
        return np.where(self.net_input(X) >= 0.0, 1, 0)
        # equivalent to:
        # return np.where(self.activation(self.net_input(X))
        #                 >= 0.5, 1, 0)
```

当拟合逻辑回归模型时，必须记住该模型只适用于二元分类。所以，只考虑Iris-setosa和Iris-versicolor两种花（类0和类1），并验证逻辑回归的有效性：

```
>>> X_train_01_subset = X_train[(y_train == 0) | (y_train == 1)]
>>> y_train_01_subset = y_train[(y_train == 0) | (y_train == 1)]
>>> lrgd = LogisticRegressionGD(eta=0.05,
...                             n_iter=1000,
...                             random_state=1)
>>> lrgd.fit(X_train_01_subset,
...          y_train_01_subset)The
>>> plot_decision_regions(X=X_train_01_subset,
...                       y=y_train_01_subset,
...                       classifier=lrgd)
>>> plt.xlabel('petal length [standardized]')
>>> plt.ylabel('petal width [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

运行上述代码可以绘制决策区域图如下：




## 逻辑回归的梯度下降学习算法

应用微积分可以发现，梯度下降的逻辑回归过程中的权重更新相当于第2章讨论Adaline时用过的方程。但是要注意的是下面的梯度下降学习规则推导是为对逻辑回归的梯度下降学习规则背后的数学概念感兴趣的读者准备的。这并不是学习本章其余部分的必要条件。首先从计算对数似然函数的偏导数开始：

$$\frac{\partial}{\partial w_j} l(\boldsymbol{w}) = \left( y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \frac{\partial}{\partial w_j} \phi(z)$$

在继续讨论之前，先计算S函数的偏导数：

$$\frac{\partial}{\partial z} \phi(z) = \frac{\partial}{\partial z} \frac{1}{1+\mathrm{e}^{-z}} = \frac{1}{\left(1+\mathrm{e}^{-z}\right)^2} \mathrm{e}^{-z} \frac{1}{1+\mathrm{e}^{-z}} \left( 1 - \frac{1}{1+\mathrm{e}^{-z}} \right)$$

$$= \phi(z)\left(1-\phi(z)\right)$$

可以在第一个方程中替换$\frac{\partial}{\partial_z}\phi(z)=\phi(z)(1-\phi(z))$：

$$\left( y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \frac{\partial}{\partial w_j} \phi(z)$$

$$= \left( y \frac{1}{\phi(z)} - (1-y) \frac{1}{1-\phi(z)} \right) \phi(z)\left(1-\phi(z)\right) \frac{\partial}{\partial w_j} z$$

$$= \left( y\left(1-\phi(z)\right) - (1-y)\phi(z) \right) x_j$$

$$= \left( y - \phi(z) \right) x_j$$

记住目标是找到可以最大化对数似然率的权重，这样就可以更新每个权重，如下所示：

$$w_j := w_j + \eta \sum_{i=1}^{n} \left( y^{(i)} - \phi\left( z^{(i)} \right) \right) x_j^{(i)}$$

因为要更新所有的权重，因此把通用的更新规则表示如下：

$$\boldsymbol{w} := \boldsymbol{w} + \Delta \boldsymbol{w}$$

Δw定义为：

$$\Delta \boldsymbol{w}=\eta \nabla J(\boldsymbol{w})$$

由于最大化对数似然相当于最小化前面定义的代价函数J，因此，可以得到下述梯度下降更新规则：

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_{i=1}^{n} \left( y^{(i)} - \phi\left(z^{(i)}\right) \right) x_j^{(i)}$$

$$\boldsymbol{w} := \boldsymbol{w}+\Delta \boldsymbol{w},\ \Delta \boldsymbol{w}=-\eta \nabla J(\boldsymbol{w})$$

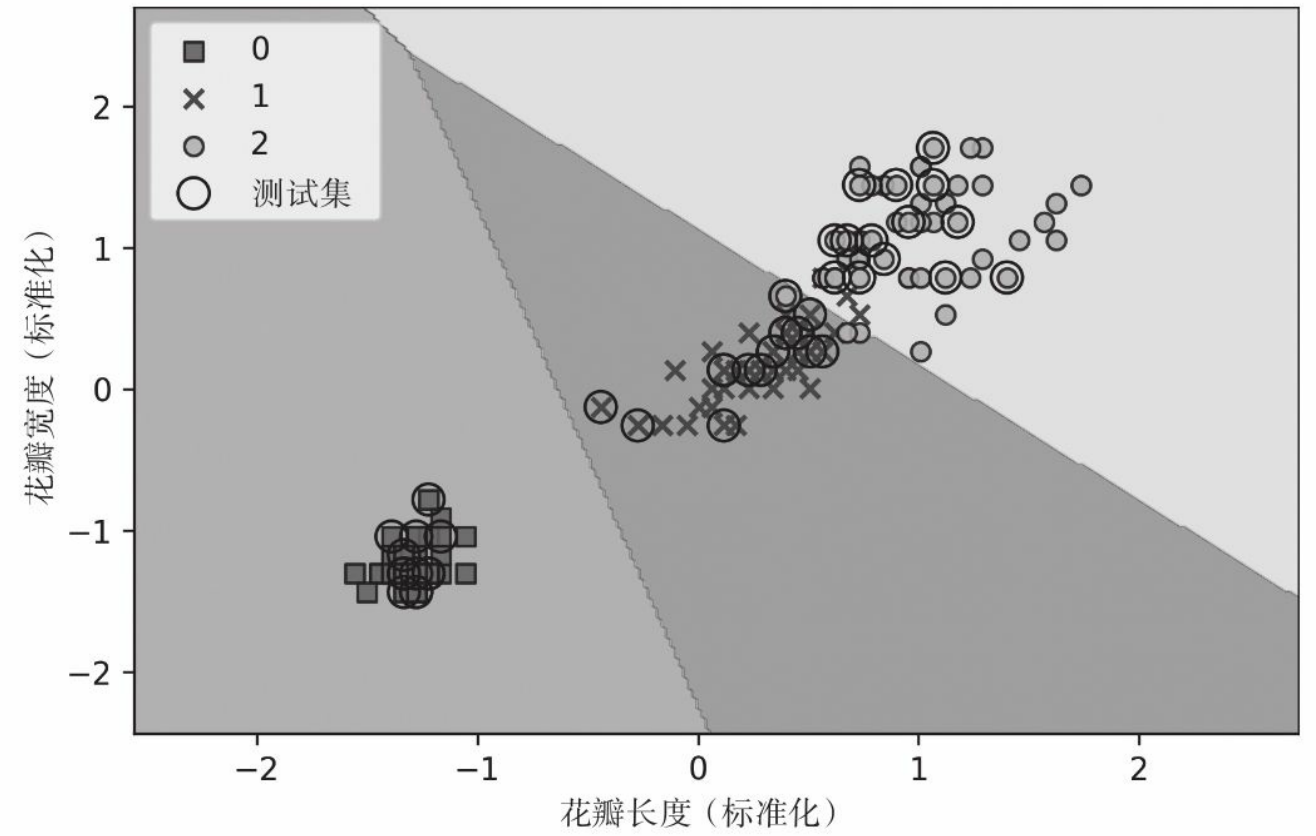
这与第2章中的Adaline梯度下降规则相同。

## 3.3.4 用scikit-learn训练逻辑回归模型

前一小节刚讨论了实用编码和有关数学计算，这有助于说明Adaline与逻辑回归概念之间的差异。现在来学习如何使用更优化的scikit-learn的逻辑回归实现，该实现也支持多元分类场景（默认OvR）。下面的代码示例将用sklearn.linear_model.LogisticRegression类以及熟悉的fit方法在三种花的标准化训练集上训练模型：

```
>>> from sklearn.linear_model import LogisticRegression
>>> lr = LogisticRegression(C=100.0, random_state=1)
>>> lr.fit(X_train_std, y_train)
>>> plot_decision_regions(X_combined_std,
...                       y_combined,
...                       classifier=lr,
...                       test_idx=range(105, 150))
>>> plt.xlabel('petal length [standardized]')
>>> plt.ylabel('petal width [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

在训练数据上拟合模型后，把决策区域、训练样本和测试样本绘制出来：



看着前面用来训练的逻辑回归模型的代码，你现在可能会想，“这个神秘的参数C到底是什么？”下一节将先介绍过拟合和正则化的概念，然后再讨论该参数。在此之前先讨论类成员的概率问题。

属于训练集中某个特定类的概率可以用predict_proba计算。例如，可以预测测试集中前三类的概率如下：

```
>>> lr.predict_proba(X_test_std[:3, :])
```

执行这段代码会返回如下阵列：

```
array([[  3.20136878e-08,   1.46953648e-01,   8.53046320e-01],
       [  8.34428069e-01,   1.65571931e-01,   4.57896429e-12],
       [  8.49182775e-01,   1.50817225e-01,   4.65678779e-13]])
```

第一行对应第一种花类成员的概率，第二行对应第三种花类成员的概率，以此类推。注意，如所预见的那样，列数据之和为1（可以执行lr.predict_proba（X_test_std[：3，：]）.sum（axis=1）来确认）。

第一行的最大值约为0.853，这意味着预测第一个样本属于第三类（Iris-virginica）的概率为85.7%。所以，你可能已经注意到，可以识别每行中最大列值得到预测的分类标签，例如，可以用NumPy的argmax函数实现：

```
>>> lr.predict_proba(X_test_std[:3, :]).argmax(axis=1)
```

执行该调用返回分别对应于Iris-virginica，Iris-setosa，Iris-setosa的分类结果：

```
array([2, 0, 0])
```

手工直接调用predict方法可以获得前面的条件概率分类标签，并快速验证如下：

```
>>> lr.predict(X_test_std[:3, :])
array([2, 0, 0])
```

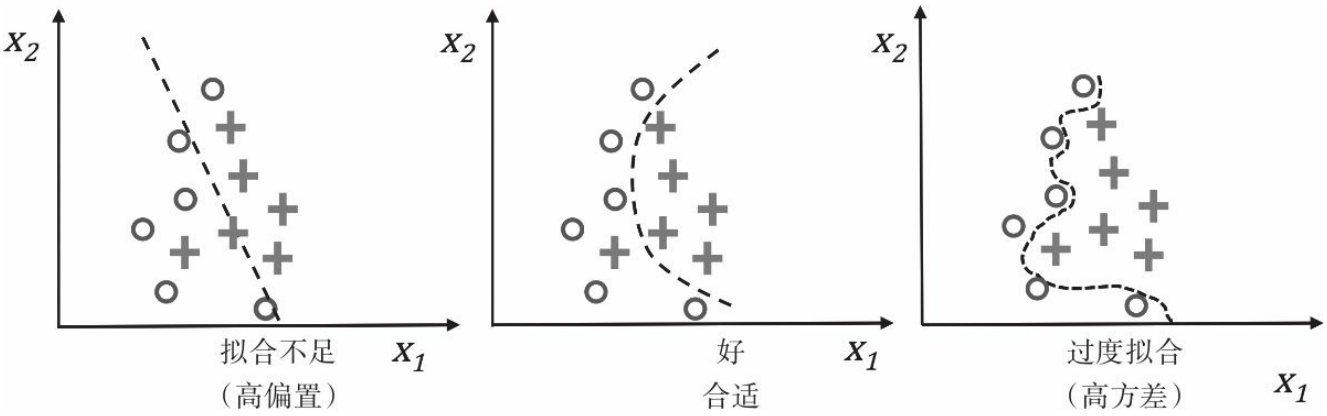
最后提醒一句，如果想单独预测花样本的分类标签，scikit-learn期望输入一个二维阵列。因此，必须先把单行转换成这种格式。调用NumPy的reshape方法增加一个新维度可以将一行数据转换成为二维阵列，代码如下：

```
>>> lr.predict(X_test_std[0, :].reshape(1, -1))
array([2])
```

## 3.3.5 通过正则化解决过拟合问题

过拟合是机器学习中的常见问题，虽然模型在训练数据上表现良好，但无法概括未见过的新数据或测试数据。如果某个模型出现过拟合的问题，我们也会说该模型具有较高的方差，可能是因为相对于给定的数据，参数太多，模型过于复杂。同样，模型也可能会出现偏置高欠拟合的情况，这意味着模型不足以捕捉训练数据中的复杂模式，因此对未见过的数据表现不良。

虽然迄今只遇到过线性分类模型，但是比较线性决策边界和更复杂的非线性决策边界是阐明过拟合与欠拟合问题的最好方法，如下图所示：



方差用来度量模型对样本预测结果的一致性，例如对训练集的不同子集，模型是否能保持不变。模型对训练数据的随机性很敏感。相反，针对不同的训练集多次重新建模，用偏置来度量预测值与真值之间的差异。偏置度量非随机所带来的系统误差。

找到好的偏差——方差平衡的方法之一是通过正则化来调整模型的复杂性。正则化是处理共线性（特征之间的高相关性），滤除数据中的噪声，并最终避免过拟合的非常有效的方法。正则化的逻辑是引入额外的信息（偏置）来惩罚极端的参数值（权重）。最常见的正则化是所谓的L2正则化（有时也称为L2收缩或权重衰减），具体如下：

$$\frac{\lambda}{2}\|\boldsymbol{w}\|^2 = \frac{\lambda}{2}\sum_{j=1}^{m} w_j^2$$

在这里λ为所谓的正则化参数。

正则化是样本特征缩放（例如标准化）极为重要的另一个原因。

要正常进行正则化必须确保所有样本特征的比例都适当。

逻辑回归的代价函数可以通过增加一个简单的正则项来调整，这将在模型训练的过程中缩小权重：

$$J(\boldsymbol{w})=\sum_{i=1}^{n}\left[-y^{(i)}\log\left(\phi\left(z^{(i)}\right)\right)-\left(1-y^{(i)}\right)\log\left(1-\phi\left(z^{(i)}\right)\right)\right]+\frac{\lambda}{2}\|w\|^2$$

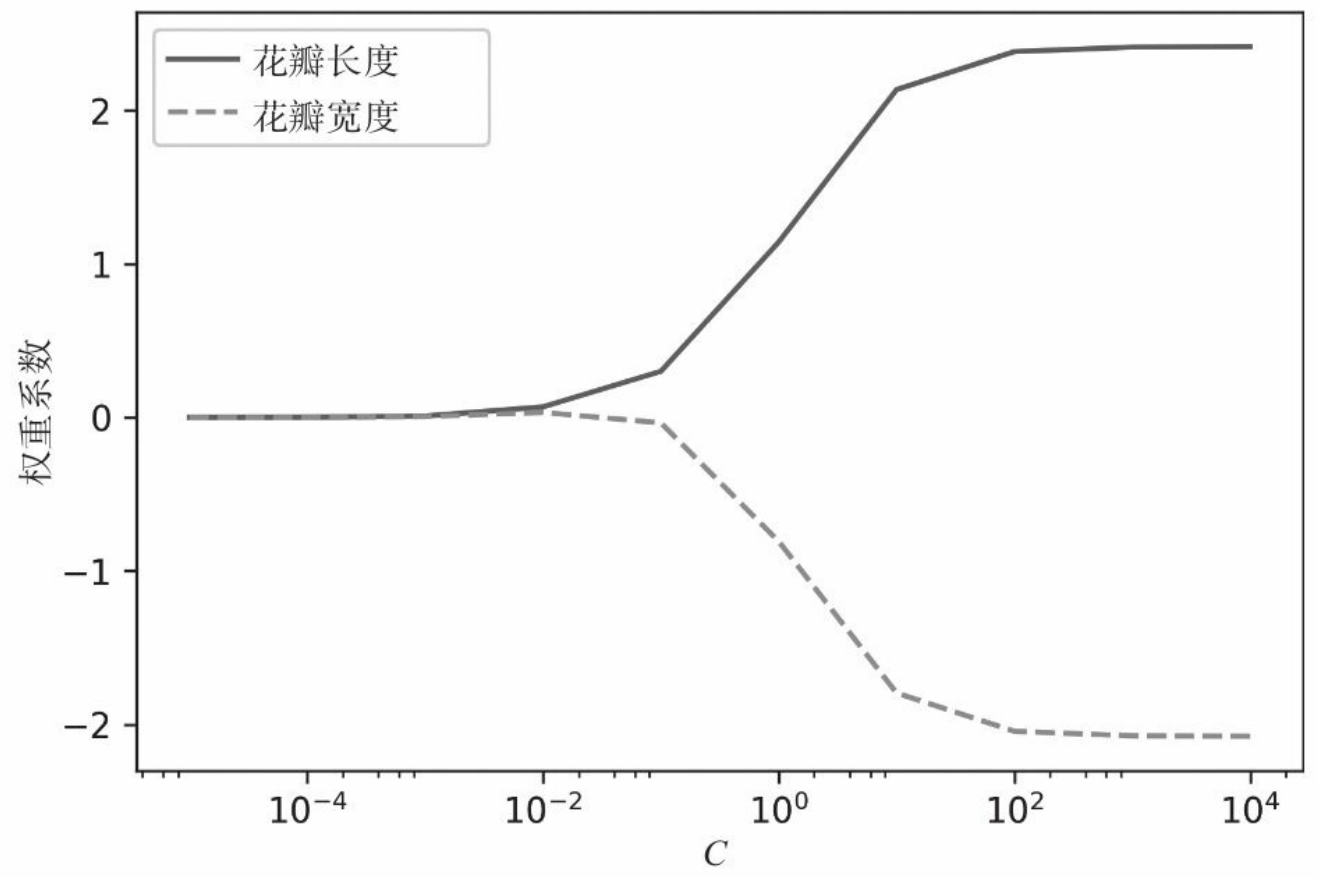
λ正则化参数可以控制如何在更好地拟合训练数据的同时保持较小的权重。可以通过加大λ值增加正则化的强度。

scikit-learn实现LogisticRegression类的参数C来自于支持向量机的约定，下一节将讨论该主题。C与λ直接相关联，与正则化参数λ成反比。因此，降低逆正则化参数C意味着增加正则化的强度，这可以通过绘制两个权重系数的L2正则路径实现可视化：

```
>>> weights, params = [], []
>>> for c in np.arange(-5, 5):
...     lr = LogisticRegression(C=10.**c, random_state=1)
...     lr.fit(X_train_std, y_train)
...     weights.append(lr.coef_[1])
...     params.append(10.**c)
>>> weights = np.array(weights)
>>> plt.plot(params, weights[:, 0],
...          label='petal length')
>>> plt.plot(params, weights[:, 1], linestyle='--',
...          label='petal width')
>>> plt.ylabel('weight coefficient')
>>> plt.xlabel('C')
>>> plt.legend(loc='upper left')
>>> plt.xscale('log')
>>> plt.show()
```

执行上面的代码用10个不同的逆正则化参数C值拟合逻辑回归模型，我们只收集分类标签为1的权重系数（这里对应数据集中的第二类即Iris-versicolor）而不是所有类，记住采用的是一对多（OvR）的多元分类技术。

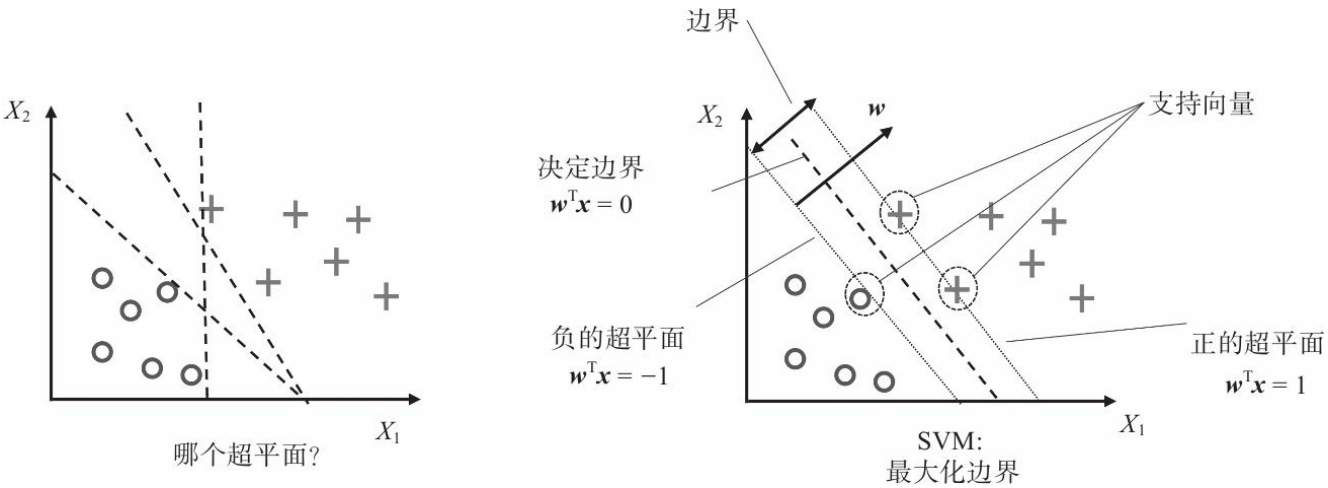
如下图所示，减小参数C增大正则化的强度，权重系数变小：



深入讨论各种分类算法超出了本书的范围，我强烈向读者推荐由Sage出版社出版，斯科特·梅纳德博士2009年撰写的《逻辑回归：从入门到高级概念及应用》一书，以了解更多关于逻辑回归方面的知识。

## 3.4 支持向量机的最大余量分类

另外一种强大而且广泛使用的机器学习算法是支持向量机（SVM），可以把它当成是感知器的延伸。感知器算法的目标是把分类误差减少到最小。而支持向量机算法优化的目标是寻找最大化的边界。边界定义为分离超平面（决策边界）与其最近的训练样本之间的距离，即所谓的支持向量。下图对此做了展示：



### 3.4.1 最大边际的直觉

决策边界间隔大的模型背后的理由是它们往往有较低的泛化误差，而间隔较小的模型则更容易过拟合。为了更好地解释间隔最大化的概念，让我们仔细看看那些与决策边界平行的正超平面和负超平面，表示如下：

$$w_0+\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_{pos}=1 \quad (1)$$

$$w_0+\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_{neg}=-1 \quad (2)$$

把两个线性等式（1）和（2）相减可以得到：

$$\Rightarrow \boldsymbol{w}^{\mathrm{T}}(\boldsymbol{x}_{pos}-\boldsymbol{x}_{neg})=2$$

可以通过向量w的长度归一化该方程，定义如下：

$$\|\boldsymbol{w}\|=\sqrt{\sum_{j=1}^{m} w_j^2}$$

因此得到下述方程：

$$\frac{\boldsymbol{w}^{\mathrm{T}}(\boldsymbol{x}_{pos}-\boldsymbol{x}_{neg})}{\|\boldsymbol{w}\|}=\frac{2}{\|\boldsymbol{w}\|}$$

方程的左边可以被解释为超平面正负之间的距离，即想要最大化的所谓间隔。现在，支持向量机的目标函数变成通过最大化$\frac{2}{\|\boldsymbol{w}\|}$来最大化间隔，在样本分类正确的条件约束下，可以表示为：

$$w_0+\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(i)} \geqslant 1 \text{ if } y^{(i)}=1$$

$$w_0+\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(i)} \leqslant -1 \text{ if } y^{(i)}=-1$$

$$\text{for } i=1\cdots N$$

这里N为数据集中的样本数量。

基本上，这两个方程所有的负样本应该都落在超平面的负面，而所有的

正样本应该都落在超平面的正面，可以以更紧凑的方式表示：

$$y^{(i)}(w_0+\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(i)}) \geqslant 1 \ \forall_i$$

实际上，最小化$\frac{1}{2}\|\boldsymbol{w}\|^2$的倒数更为容易，这可以通过二次规划的方法实现。然而，对二次规划细节的讨论已经超出了本书的范围。可以阅读《统计学习理论的本质》一书学习更多关于支持向量机方面的知识。该书作者为弗拉基米尔·万普尼克，由施普林格科学和商业媒体出版社在2000年发行。也可以阅读克里斯对支持向量机的优秀解释（克里斯·J.C.布尔赫斯，《支持向量机模式识别教程优秀解释汇编》.数据挖掘与知识发现，1998，2（2）：121-167）。

## 3.4.2 用松弛变量处理非线性可分

尽管不想对最大间隔分类背后更复杂的数学概念进行讨论，但还是要简要地提一下由弗拉基米尔·万普尼克于1995年提出的松弛变量ξ，它引出了所谓的软间隔分类。引入松弛变量ζ的目的是要处理非线性可分数据，放松线性约束需要，允许在分类错误存在的情况下通过适当代价的惩罚来确保优化可以收敛。

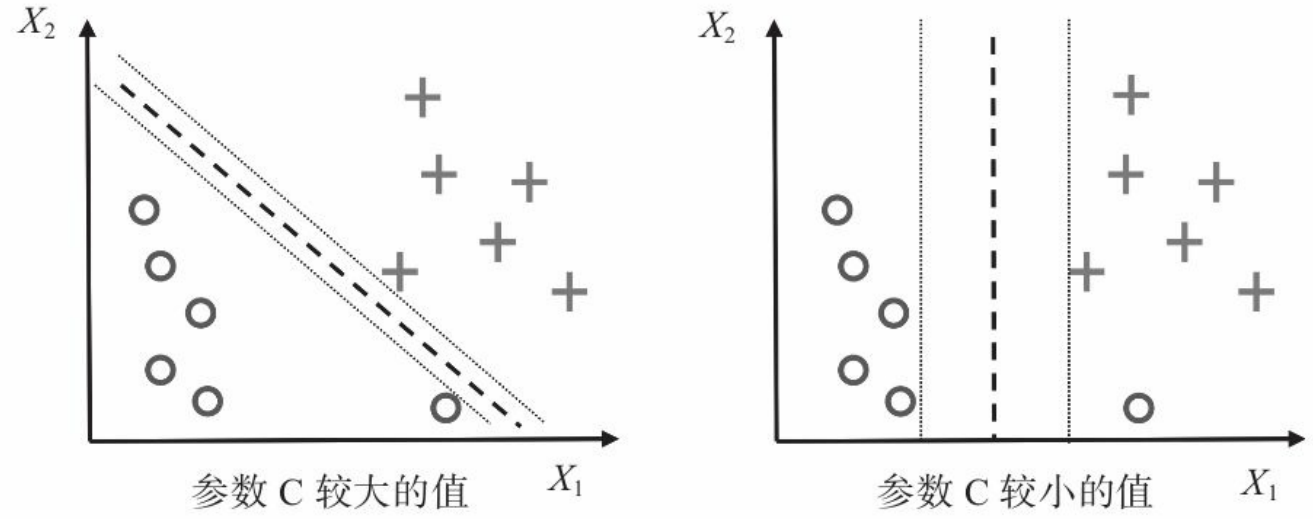
可以直接把阳性松弛变量加入线性约束：

$$
\begin{aligned}
w_0+\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(i)} &\geqslant 1-\xi^{(i)} \quad \text{if } y^{(i)}=1 \\
w_0+\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}^{(i)} &\leqslant -1+\xi^{(i)} \quad \text{if } y^{(i)}=-1 \\
&\text{for } i = 1 \cdots N
\end{aligned}
$$

这里N为数据集中的样本数量。因此新的最小化目标（约束主体）就表示为：

$$
\frac{1}{2}\|\boldsymbol{w}\|^2 + C\left(\sum_i \xi^{(i)}\right)
$$

可以通过变量C来控制对分类错误的惩罚。C值越大相应的错误惩罚就越大，如果选择较小的C值，则对分类错误的要求不那么严。因此，可以用参数C来控制间隔的宽度，从而权衡偏置方差，如下图所示：

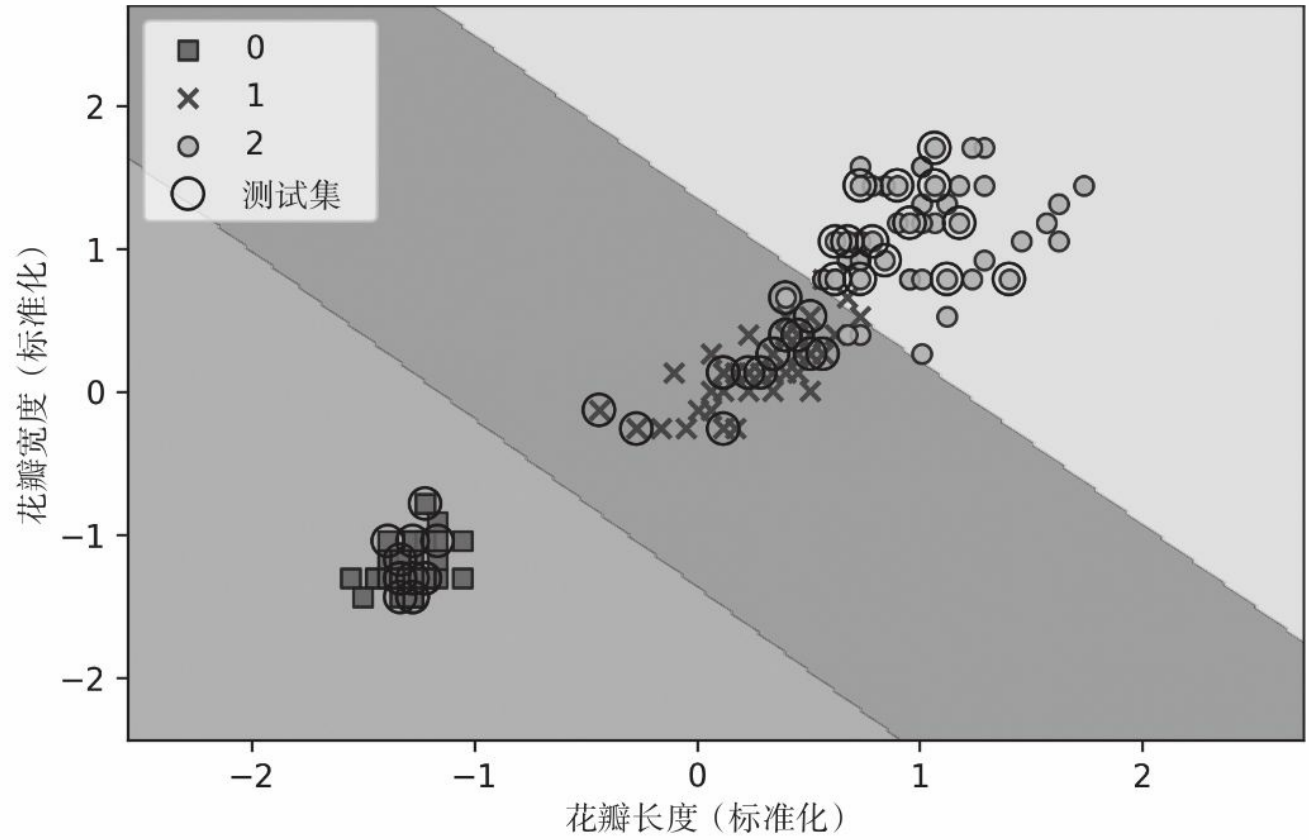


该概念与上节讨论的正则化回归相关，即减小C值增加偏置，但降低模型的方差。

已经了解了线性支持向量机背后的基本概念，现在可以训练一个支持向量机模型来对鸢尾花数据集中的不同种花进行分类：

```
>>> from sklearn.svm import SVC
>>> svm = SVC(kernel='linear', C=1.0, random_state=1)
>>> svm.fit(X_train_std, y_train)
>>> plot_decision_regions(X_combined_std,
...                       y_combined,
...                       classifier=svm,
...                       test_idx=range(105, 150))
>>> plt.xlabel('petal length [standardized]')
>>> plt.ylabel('petal width [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

执行前面的代码示例，在鸢尾花数据集上训练分类器之后得到SVM的三个决策区域如下：




## 逻辑回归与支持向量机：

在实际的分类任务中，线性逻辑回归和线性支持向量机通常会产

生非常相似的结果。逻辑回归试图最大化训练数据的条件似然性，使其比SVM更容易离群，支持向量机主要关心的是最接近决策边界的点。另一方面，逻辑回归也有优点，其模型更简单更容易实现。此外，逻辑回归模型更容易更新，这在处理流式数据时很有吸引力。

### 3.4.3 其他的scikit-learn实现

前几章中用过的scikit-learn软件库中的Perceptron，LogisticRegression类和LIBLINEAR（http://www.csie.ntu.edu.tw/~cjlin/liblinear/）是由台湾大学基于C/C++开发和高度优化的软件。用来训练SVM的SVC类所用的LIBSVM，相当于专门为SVM准备的C/C++库，同样也是他们研发的（http://www.csie.ntu.edu.TW/~cjlin/libsvm/）。

与原生Python相比，用LIBLINEAR和LIBSVM的好处是它们允许快速训练大量线性分类器。然而，有时候数据太多无法加载到内存。因此，scikit-learn也提供了通过SGDClassifier类实现的不同函数，同时通过调用partial_fit支持在线学习。SGDClassifier类的逻辑与第2章为Adaline实现的随机梯度算法类似。初始化随机梯度下降感知器、逻辑回归和带有默认参数的支持向量机如下：

```
>>> from sklearn.linear_model import SGDClassifier
>>> ppn = SGDClassifier(loss='perceptron')
>>> lr = SGDClassifier(loss='log')
>>> svm = SGDClassifier(loss='hinge')
```

## 3.5 用核支持向量机求解非线性问题

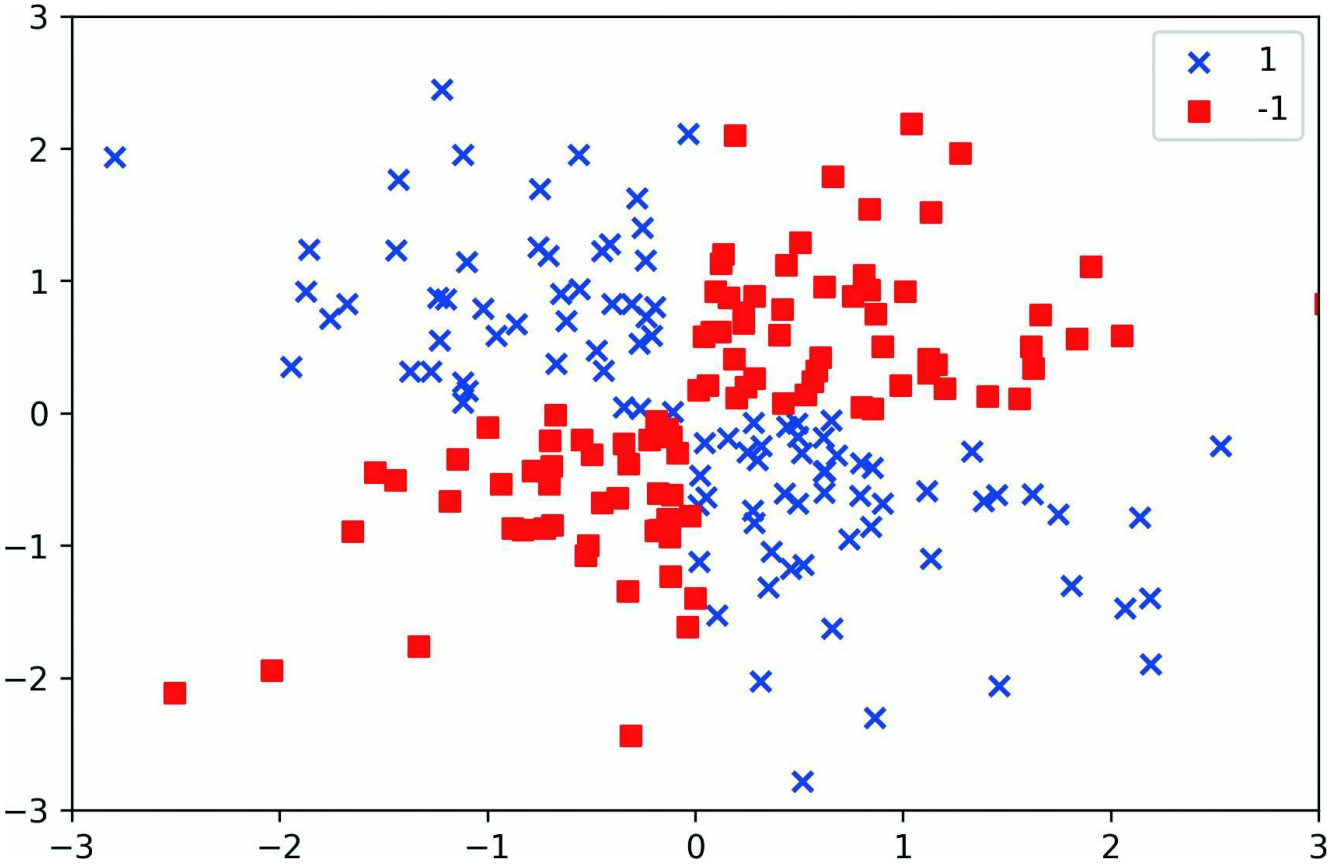
支持向量机在机器学习领域享有较高知名度的另一个原因是容易通过核化来解决非线性分类问题。在讨论核支持向量机的逻辑之前，先创建一个样本数据集来观察这种非线性分类问题的具体情形。

### 3.5.1 处理线性不可分数据的核方法

执行下述代码创建一个简单的数据集，调用NumPy的logical_or函数形成一个异或门，其中有100个样本的分类标签为1，100个样本的分类标签为-1：

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> np.random.seed(1)
>>> X_xor = np.random.randn(200, 2)
>>> y_xor = np.logical_xor(X_xor[:, 0] > 0,
...                        X_xor[:, 1] > 0)
>>> y_xor = np.where(y_xor, 1, -1)
>>> plt.scatter(X_xor[y_xor == 1, 0],
...             X_xor[y_xor == 1, 1],
...             c='b', marker='x',
...             label='1')
>>> plt.scatter(X_xor[y_xor == -1, 0],
...             X_xor[y_xor == -1, 1],
...             c='r',
...             marker='s',
...             label='-1')
>>> plt.xlim([-3, 3])
>>> plt.ylim([-3, 3])
>>> plt.legend(loc='best')
>>> plt.show()
```

执行上述代码会产生具有随机噪声的XOR数据集，图示如下：

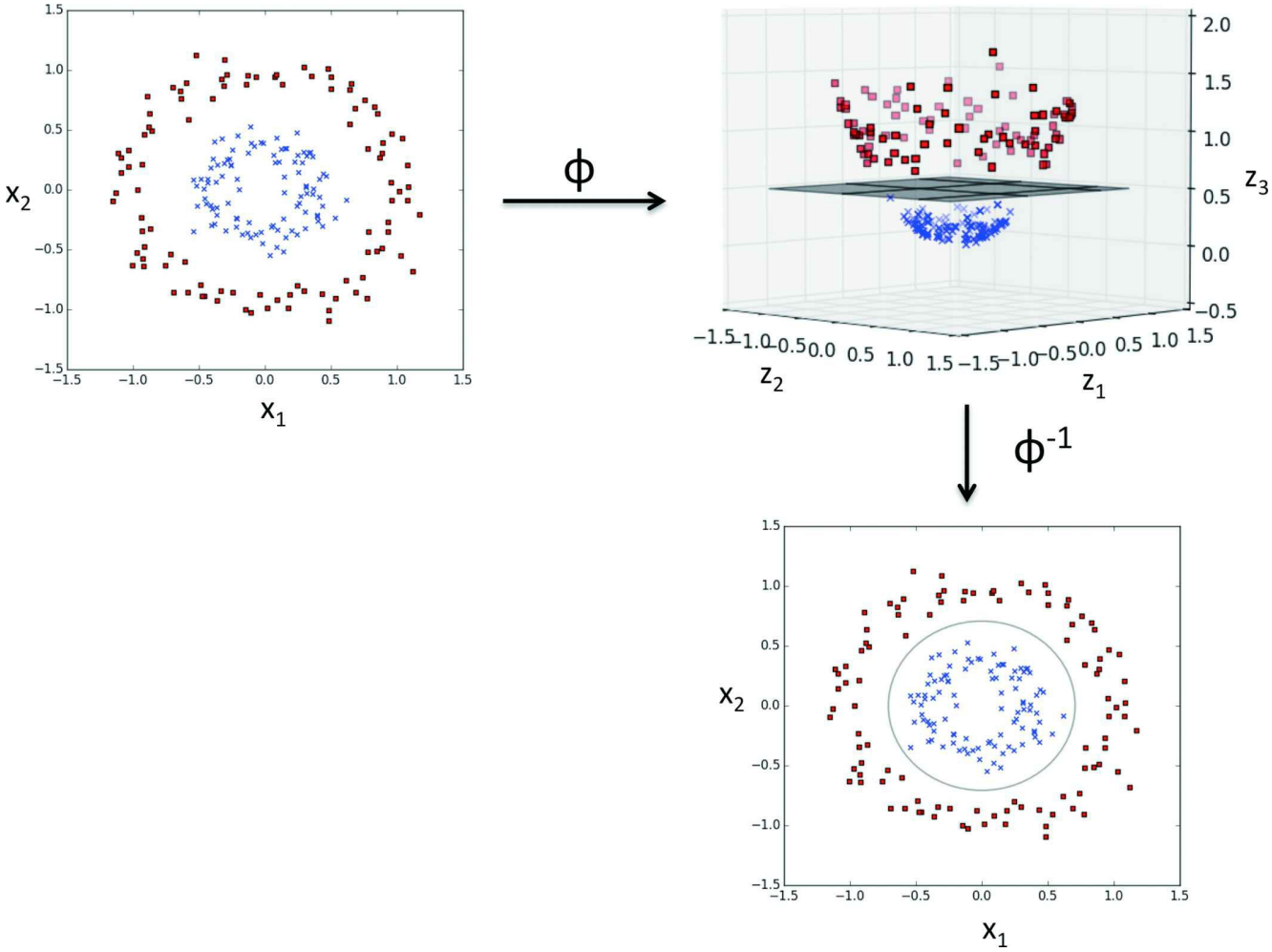



显然不能用前面章节讨论过的线性逻辑回归或线性支持向量机模型所产生的线性超平面作为决策边界来分隔样本的正类和负类。

核方法的逻辑是针对线性不可分数据，建立非线性组合，通过映射函数把原始特征投影到一个高维空间，特征在该空间变得线性可分。如下图所示，可以将一个二维数据集转换为一个新的在三维空间上表示的特征，这样就可以通过下述投影完成分类：

$$\phi(x_1, x_2)=(z_1, z_2, z_3)=(x_1, x_2, x_1^2+x_2^2)$$

通过线性超平面可以将图中所示的两类分开，如果把它投射到原来的特征空间上，将形成非线性决策边界：


x2
x1
ϕ
z3
z2
z1
ϕ-1
x2
x1

## 3.5.2 利用核技巧，发现高维空间的分离超平面

为了使用SVM解决非线性问题，需要调用映射函数φ将训练数据变换成在高维度空间上表示的特征，然后训练线性SVM模型对新特征空间里的数据进行分类。可以用相同的映射函数φ对新的、未见过的数据进行变换，用线性支持向量机模型进行分类。

然而，这种映射方法的问题是构建新特征的计算成本非常高，特别是在处理高维度数据时。这就是所谓的核技巧可以发挥作用的地方。虽然没有详细研究如何通过解决二次规划任务来训练支持向量机，但实际上只需要用$(x^{(i)})^T(x^{(j)})$点乘替换$x^{(i)T}x^{(j)}$。为显著降低计算两点间点乘的昂贵计算成本，定义所谓的核函数如下：

$$\mathcal{K}(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}) = \phi(\boldsymbol{x}^{(i)})^T \phi(\boldsymbol{x}^{(j)})$$

其中最为广泛使用的是径向基核函数（RBF）或简称为高斯核：

$$\mathcal{K}\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = \exp\left(-\frac{\left\|\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(j)}\right\|^2}{2\sigma^2}\right)$$

该公式常被简化为：

$$\mathcal{K}(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}) = \exp(-\gamma||\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(j)}||^2)$$

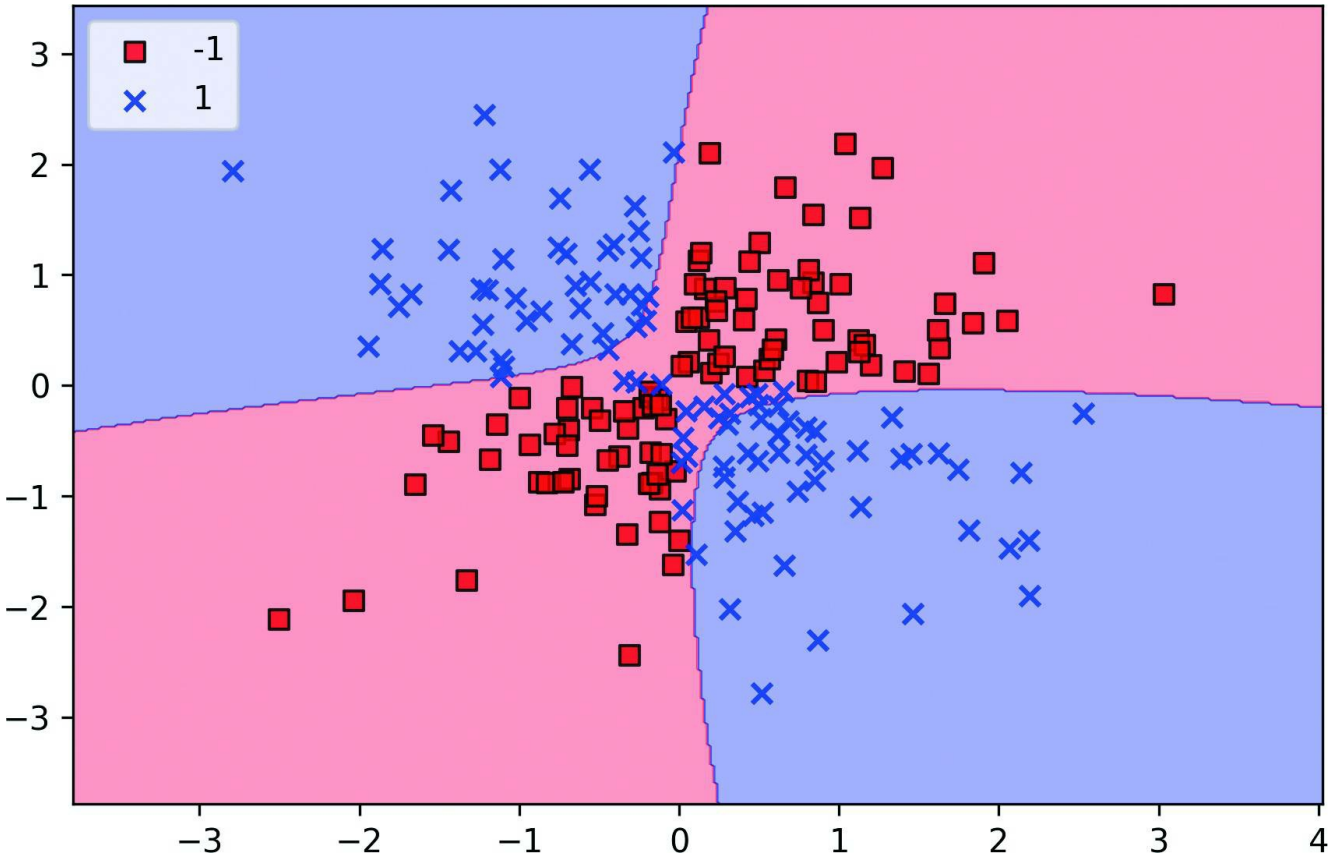
这里，$\gamma = \frac{1}{2\sigma^2}$是要优化的自由参数。

简而言之，术语“核”可以理解为两个样本之间的相似性函数。公式中的负号把距离反转为相似性得分值，而指数运算把由此而产生的相似性得分值控制在1（完全相似）和0（非常不同）之间。

描绘了核技巧背后的宏观概念，现在看能否训练核支持向量机来划出非线性决策边界，以区分XOR数据。这里只需用先前导入的scikit-learn库的SVC类，以参数kernel='rbf'替换kernel='linear'：

```
>>> svm = SVC(kernel='rbf', random_state=1, gamma=0.10, C=10.0)
>>> svm.fit(X_xor, y_xor)
>>> plot_decision_regions(X_xor, y_xor, classifier=svm)
>>> plt.legend(loc='upper left')
>>> plt.show()
```

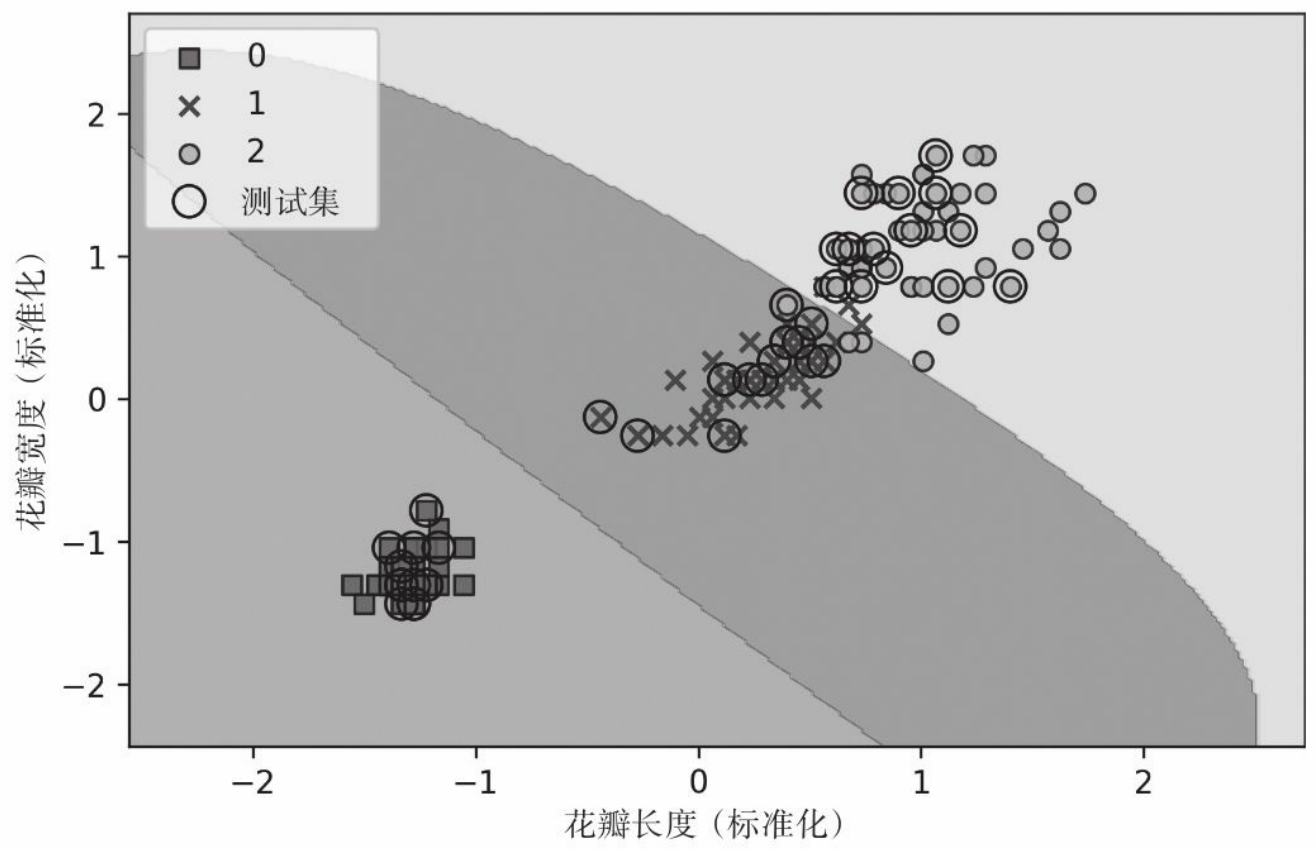
下面的结果图可以看到核SVM对XOR数据的区分效果还算不错：




参数γ的值设置为0.1，可以理解为高斯球的截止参数。增大γ值，将加大训练样本的影响范围，导致决策边界紧缩和波动。为了更好地理解γ，把RBF核支持向量机应用于鸢尾花数据集：

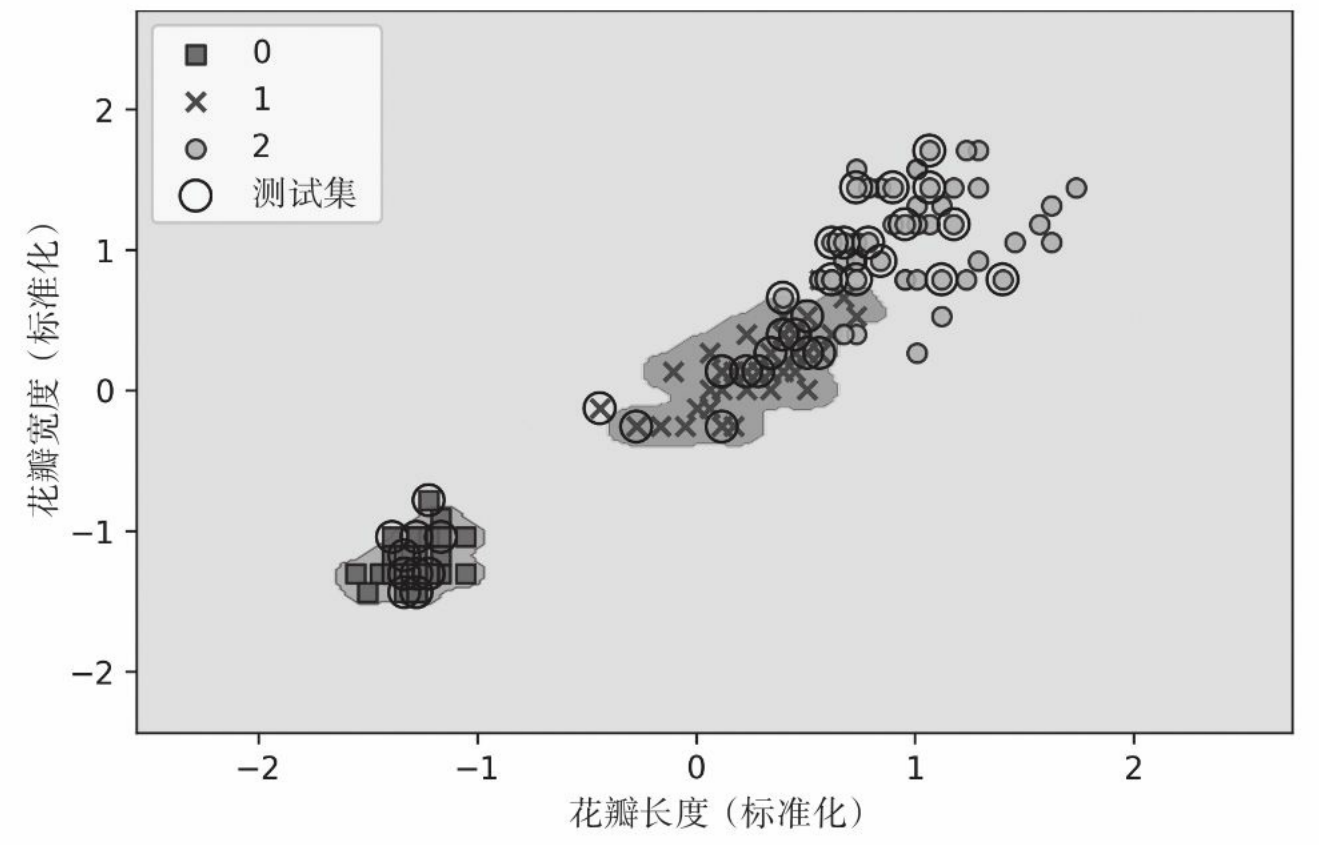
```
>>> svm = SVC(kernel='rbf', random_state=1, gamma=0.2, C=1.0)
>>> svm.fit(X_train_std, y_train)
>>> plot_decision_regions(X_combined_std,
...                       y_combined, classifier=svm,
...                       test_idx=range(105,150))
>>> plt.xlabel('petal length [standardized]')
>>> plt.ylabel('petal width [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

由于选择了相对较小的γ值，得到的径向基核支持向量机模型的决策边界相对偏松，如下图所示：




现在来观察加大γ值对决策边界的影响：

```
>>> svm = SVC(kernel='rbf', random_state=1, gamma=100.0, C=1.0)
>>> svm.fit(X_train_std, y_train)
>>> plot_decision_regions(X_combined_std,
...                       y_combined, classifier=svm,
...                       test_idx=range(105,150))
>>> plt.xlabel('petal length [standardized]')
>>> plt.ylabel('petal width [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```



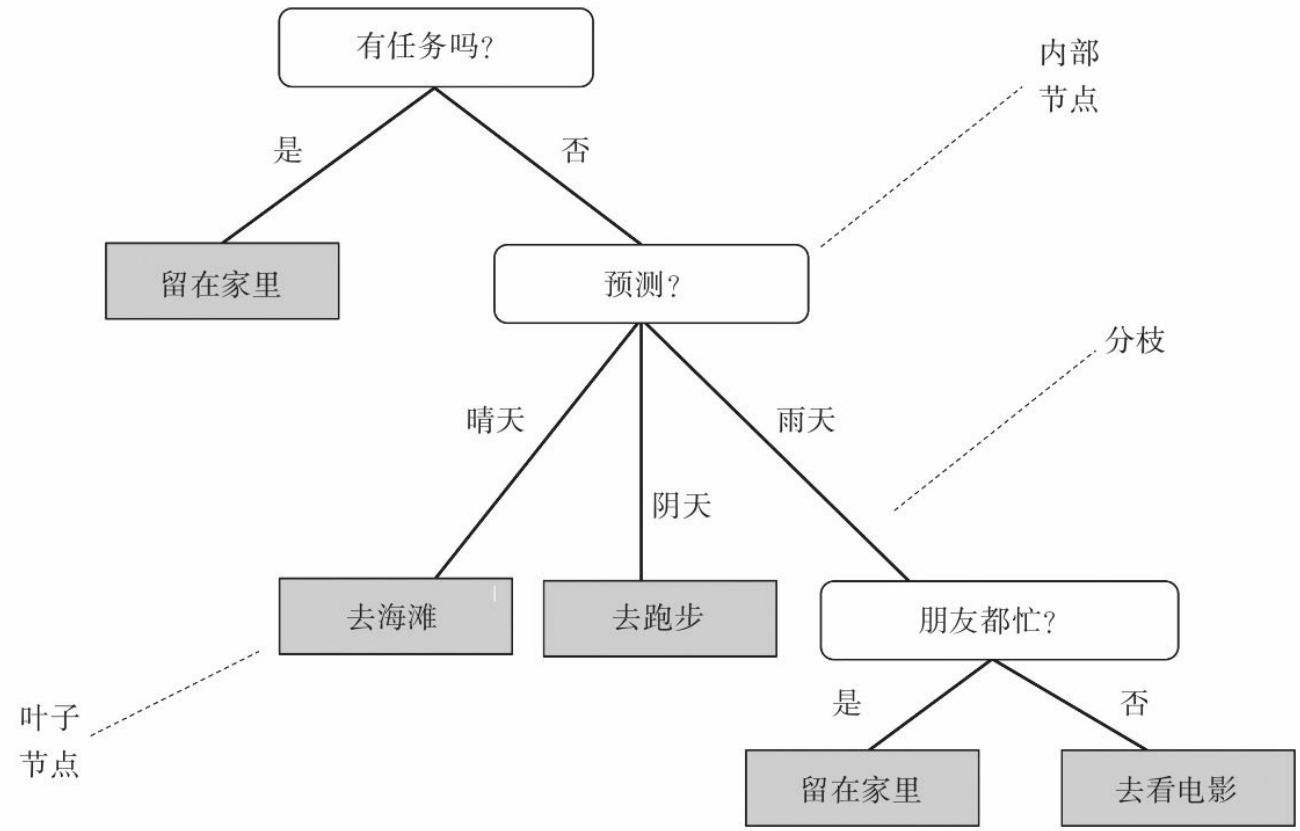

从结果图中可以看到，采用较大的γ值，类0和1周围的决策边界更为紧密：

虽然非常适合在数据集上训练模型，但这种分类器在未见过的数据上可能有很高的泛化误差。这说明参数γ在控制过拟合问题上也起着重要的作用。

## 3.6 决策树学习

如果关心可解释性，那么决策树分类器是有吸引力的模型。正如名称所暗示的那样，可以把该模型看作是根据要回答的一系列问题做出决定来完成数据分类。

考虑下面的例子用决策树来决定某一天活动：




基于训练集的特征，决策树模型通过学习一系列问题来推断样本的分类标签。虽然前面的图说明了基于分类变量的决策树概念，但如果特征是像鸢尾花数据这样的实数，概念照样适用。例如，可以简单地定义萼片宽度特征轴的临界值并且问一个二选一问题："萼片的宽度≥2.8厘米？"。

决策树算法从树根开始，在信息增益（IG）最大的节点上分裂数据，后续章节会更详细地解释。各子节点在迭代过程中重复该分裂过程，直到只剩下叶子为止。这意味着所有节点上的样本都属于同一类。在实践中，出现根深叶茂的树容易导致过拟合。因此，通常希望通过限制树的最大深度对树修剪。

### 3.6.1 最大限度地获取信息——获得最大收益

为保证在特征信息量最大的情况下分裂节点，需要先定义目标函数，然后通过决策树学习和优化算法。该目标函数可以最大化每次分裂的信息增益，表示如下：

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^{m} \frac{N_j}{N_p} I(D_j)$$

这里f是据以分裂数据的特征，$D_P$和$D_j$为父节点和第j个子节点，I为杂质含量，$N_P$为父节点的样本数，$N_J$是第j个子节点的样本数。正如所看到的，父节点和子节点之间信息增益仅在杂质含量方面存在着差异，即子节点杂质含量越低，信息增益越大。然而，为简单起见，同时考虑减少搜索空间的组合，大多数软件库（包括scikit-learn）只实现二元决策树。这意味着每个父节点分有$D_{left}$和$D_{right}$两个子节点：

$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$

在二元决策树中常用的三个杂质度量或者分隔标准分别为基尼杂质度（$I_G$），熵（$I_H$）和分类错误（$I_E$）。让我们从为所有的非空类定义熵开始（p（i|t）≠0）：

$$I_H(t) = -\sum_{i=1}^{c} p(i|t) \log_2 p(i|t)$$

这里，p（i|t）为某节点t属于c类样本的概率。如果节点上的所有样本都属于同一类，则熵为0，如果类的分布均匀，则熵值最大。例如，对二元分类，如果p（i=1|t）=1或p（i=0|t）=0，则熵为0。如果类均匀地分布p（i=1|T）=0.5，p（i=0|T）=0.5，则熵为1。因此，可以说以熵为判断标准试图最大化树的信息之间的相互关系。

直观地说，可以把基尼杂质理解为尽量减少错误分类概率的判断标准：

$$I_G(t)=\sum_{i=1}^{c}p(i|t)\big(1-p(i|t)\big)=1-\sum_{i=1}^{c}p(i|t)^2$$

与熵类似，如果类是完全混合的，基尼杂质最大，例如，在二元分类中（c=2）：

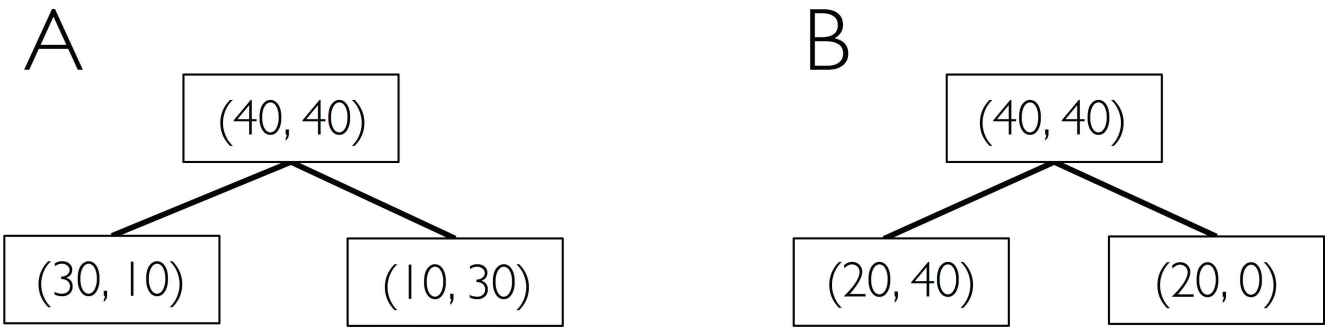
$$I_G(t)=1-\sum_{i=1}^{c}0.5^2=0.5$$

然而，基尼杂质和熵在实践中通常会产生非常相似的结果，而且通常不值得花很多时间用不同的杂质标准来评估，更好的选择是试验不同的修剪方法。

另外一种杂质度量方法是分类误差：

$$I_E=1-\max\{p(i|t)\}$$

这对修剪树枝是个有用的判断标准，但我们并不推荐把它用于决策树，因为它对节点的分类概率变化不太敏感。可以通过下图显示的两种可能的分裂场景来说明：




从数据集的父节点$D_P$开始，它包含40个分类标签为1的样本和40个分类标签为2的样本，要分裂成$D_{left}$和$D_{right}$两个数据集。在A和B两种场景下，用分类误差作为分裂标准的信息增益（$IG_E=0.25$）相同：

$$I_E(D_P)=1-0.5=0.5$$

$$\text{A: } I_E(D_{left})=1-\frac{3}{4}=0.25$$

$$\text{A: } I_E(D_{right})=1-\frac{3}{4}=0.25$$

$$\text{A: } IG_E=0.5-\frac{4}{8}\times 0.25-\frac{4}{8}\times 0.25=0.25$$

$$\text{B: } I_E(D_{left})=1-\frac{4}{6}=\frac{1}{3}$$

$$\text{B: } I_E(D_{right})=1-1=0$$

$$\text{B: } IG_E=0.5-\frac{6}{8}\times\frac{1}{3}-0=0.25$$

然而，与场景A（$IG_G$=0.125）相比，基尼杂质有利于分割场景B（$IG_G$=0.16），该场景确实是更纯粹：

$$I_G(D_P)=1-(0.5^2+0.5^2)=0.5$$

$$\mathrm{A}: I_G\left(D_{left}\right)=1-\left(\left(\frac{3}{4}\right)^2+\left(\frac{1}{4}\right)^2\right)=\frac{3}{8}=0.375$$

$$\mathrm{A}: I_G\left(D_{right}\right)=1-\left(\left(\frac{1}{4}\right)^2+\left(\frac{3}{4}\right)^2\right)=\frac{3}{8}=0.375$$

$$\mathrm{A}: IG_G=0.5-\frac{4}{8}\times 0.375-\frac{4}{8}\times 0.375=0.125$$

$$\mathrm{B}: I_G\left(D_{left}\right)=1-\left(\left(\frac{2}{6}\right)^2+\left(\frac{4}{6}\right)^2\right)=\frac{4}{9}=0.\overline{4}$$

$$\mathrm{B}: I_G(D_{right})=1-(1^2+0^2)=0$$

$$\mathrm{B}: IG_G=0.5-\frac{6}{8}\times 0.\overline{4}-0=0.1\overline{6}$$

同样，熵准则对场景B（$IG_H$=0.31）比场景A（$IG_H$=0.19）更有利：

$$I_H(D_p)=-(0.5\log_2(0.5)+0.5\log_2(0.5))=1$$

$$\mathrm{A}: I_H\left(D_{left}\right)=-\left(\frac{3}{4}\log_2\left(\frac{3}{4}\right)+\frac{1}{4}\log_2\left(\frac{1}{4}\right)\right)=0.81$$

$$\mathrm{A}: I_H\left(D_{right}\right)=-\left(\frac{1}{4}\log_2\left(\frac{1}{4}\right)+\frac{3}{4}\log_2\left(\frac{14}{4}\right)\right)=0.81$$

$$\mathrm{A}: IG_H=1-\frac{4}{8}\times 0.81-\frac{4}{8}\times 0.81=0.19$$

$$\mathrm{B}: I_H\left(D_{left}\right)=-\left(\frac{2}{6}\log_2\left(\frac{2}{6}\right)+\frac{4}{6}\log_2\left(\frac{4}{6}\right)\right)=0.92$$

$$\mathrm{B}: I_H(D_{right})=0$$

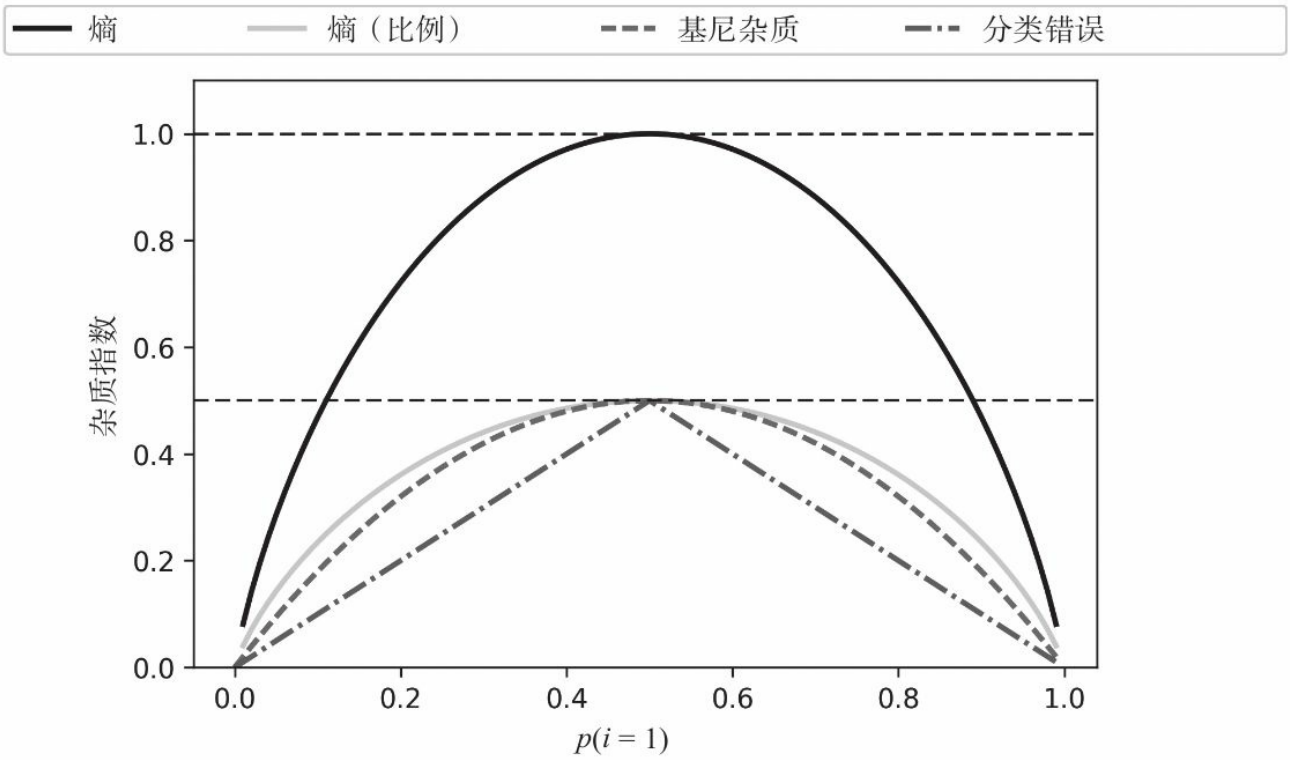
$$\mathrm{B}: IG_H=0.5-\frac{6}{8}\times 0.92-0=0.31$$

为了能更直观地比较前面讨论过的三种不同的杂质标准，把分类标签为1，概率范围在[0，1]之间的杂质情况画在图上。注意，还将添加一个小比例样本的熵（熵/2）来观察基尼杂质介于熵和分类误差中间的度量标准，代码如下：

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
```

```
>>> def gini(p):
...     return (p)*(1 - (p)) + (1 - p)*(1 - (1-p))
>>> def entropy(p):
...     return - p*np.log2(p) - (1 - p)*np.log2((1 - p))
>>> def error(p):
...     return 1 - np.max([p, 1 - p])
>>> x = np.arange(0.0, 1.0, 0.01)
>>> ent = [entropy(p) if p != 0 else None for p in x]
>>> sc_ent = [e*0.5 if e else None for e in ent]
>>> err = [error(i) for i in x]
>>> fig = plt.figure()
>>> ax = plt.subplot(111)
>>> for i, lab, ls, c, in zip([ent, sc_ent, gini(x), err],
...                     ['Entropy', 'Entropy (scaled)',
...                         'Gini Impurity',
...                         'Misclassification Error'],
...                     ['-', '-', '--', '-.'],
...                     ['black', 'lightgray',
...                        'red', 'green', 'cyan']):
...     line = ax.plot(x, i, label=lab,
...                      linestyle=ls, lw=2, color=c)
>>> ax.legend(loc='upper center', bbox_to_anchor=(0.5, 1.15),
...           ncol=5, fancybox=True, shadow=False)
>>> ax.axhline(y=0.5, linewidth=1, color='k', linestyle='--')
>>> ax.axhline(y=1.0, linewidth=1, color='k', linestyle='--')
>>> plt.ylim([0, 1.1])
>>> plt.xlabel('p(i=1)')
>>> plt.ylabel('Impurity Index')
>>> plt.show()
```

执行前面的示例代码得到下面的图：

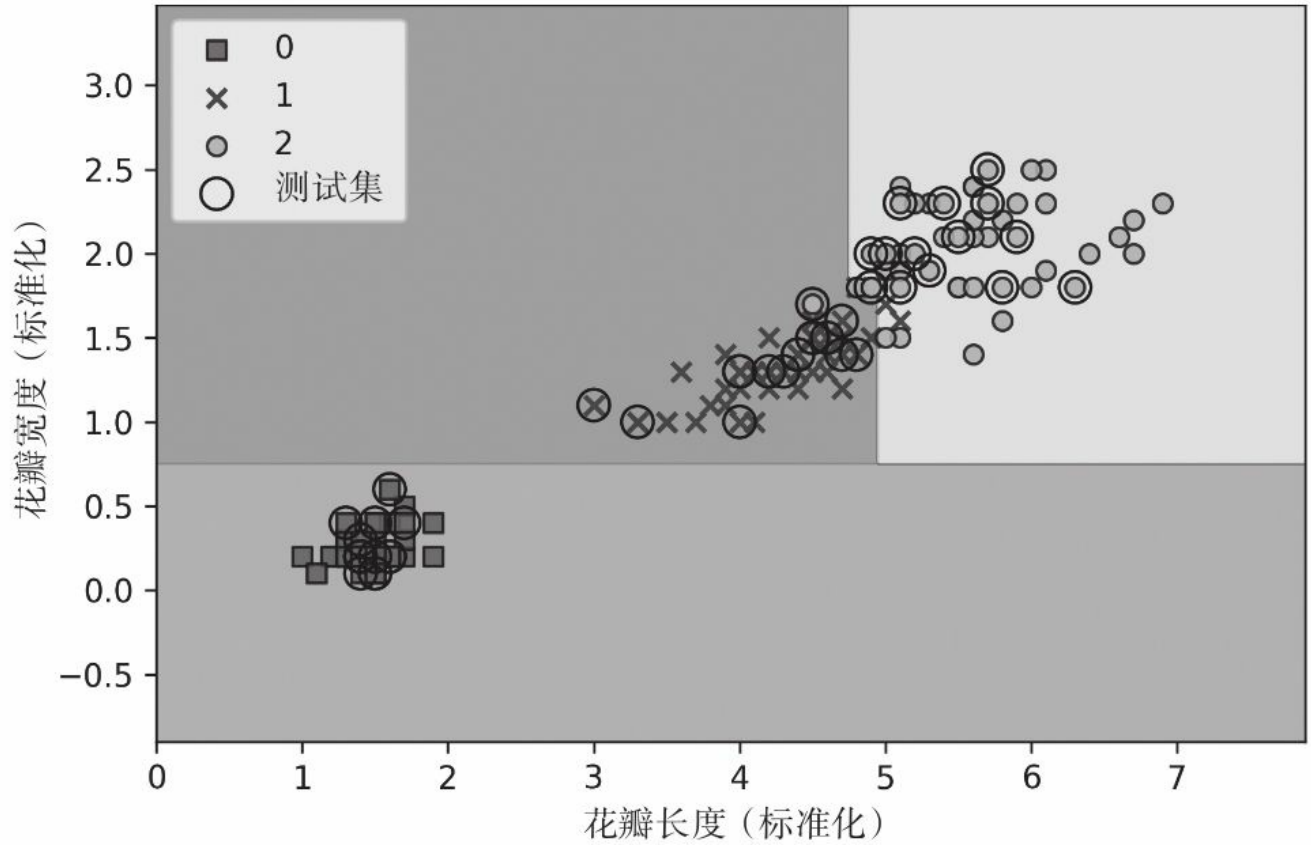

熵
熵（比例）
基尼杂质
分类错误
杂质指数
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
p(i = 1)

### 3.6.2 构建决策树

决策树可以通过将特征空间划分成不同的矩形来构建复杂的决策边界。然而，必须小心，决策树越深，决策边界就越复杂，越容易导致过拟合。假设最大深度为3，以熵作为杂质度量的标准，用scikit-learn来训练决策树模型。虽然为了可视化的目的可能需要调整样本特征数据的比例，但请注意该调整并非决策树算法的要求。代码如下：

```
>>> from sklearn.tree import DecisionTreeClassifier
>>> tree = DecisionTreeClassifier(criterion='gini',
...                               max_depth=4,
...                               random_state=1)
>>> tree.fit(X_train, y_train)
>>> X_combined = np.vstack((X_train, X_test))
>>> y_combined = np.hstack((y_train, y_test))
>>> plot_decision_regions(X_combined,
...                       y_combined,
...                       classifier=tree,
...                       test_idx=range(105, 150))
>>> plt.xlabel('petal length [cm]')
>>> plt.ylabel('petal width [cm]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

执行代码后，通常我们会得到决策树与坐标轴平行的决策边界：




scikit-learn有一个不错的功能，允许在模型训练后，把决策树以.dot文件的格式导出。然后调用Graphviz程序完成可视化。

可以从http://www.graphviz.org免费下载该程序，它支持Linux、Windows和MacOS。除了GraphViz以外，还将用被称为pydotplus的Python库，其功能与GraphViz类似，允许把.dot数据文件转换成决策树的图像。在安装GraphViz后，可以直接通过pip程序安装pydotplus（http://www.graphviz.org/Download.php上有详细的指令），例如，在你自己的计算机上执行下面的命令：

```
> pip3 install pydotplus
```

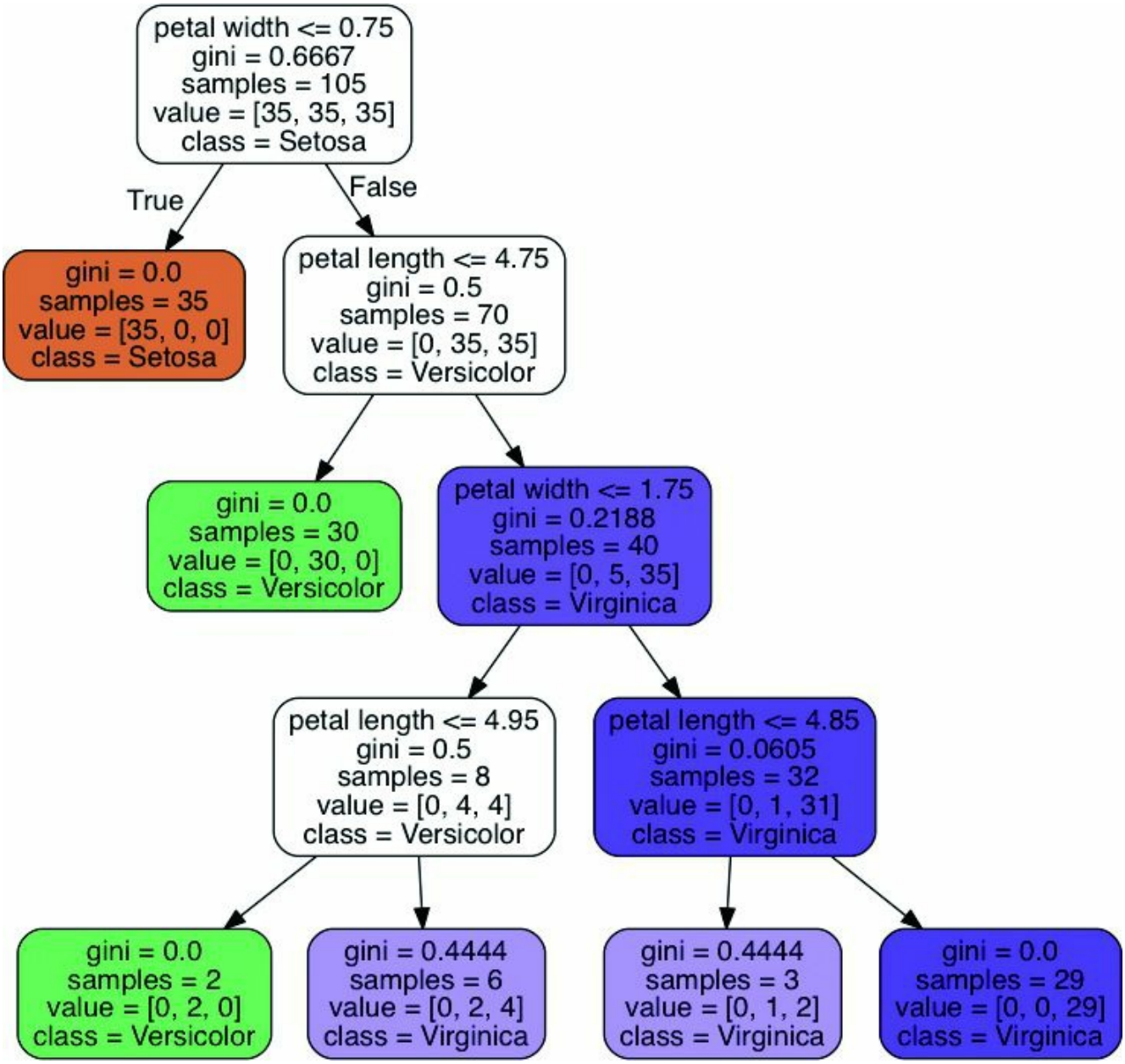
请注意，在某些系统中，可能要执行下述命令先安装pydotplus所依赖的软件包：

```
pip3 install graphviz
pip3 install pyparsing
```

执行下述代码将在本地目录以PNG格式创建决策树图像文件：

```
>>> from pydotplus import graph_from_dot_data
>>> from sklearn.tree import export_graphviz
>>> dot_data = export_graphviz(tree,
...                            filled=True,
...                            rounded=True,
...                            class_names=['Setosa',
...                                         'Versicolor',
...                                         'Virginica'],
...                            feature_names=['petal length',
...                                           'petal width'],
...                            out_file=None)
>>> graph = graph_from_dot_data(dot_data)
>>> graph.write_png('tree.png')
```

通过设置out_file=None，可以直接把数据赋予dot_data变量，而不是在磁盘上产生中间文件tree.dot。参数filled、rounded、class_names和feature_names为可选项，但要使图像的视觉效果更好，就需要添加颜色、框的边缘圆角，并在每个节点上显示大多数分类标签，以及分裂标准的特征。这些设置完成后会得到以下的决策树图像：




现在可以在决策树图上很好地追溯训练集的分裂过程。从105个样本的根节点开始，以花瓣宽度0.75厘米作为截止条件，先分割成35个和70个样本的两个子节点。可以看到左面子节点的纯度已经很高，只包含Iris-setosa样本（基尼杂质率=0）。而右面的子节点则进一步分裂成Iris-versicolor和Iris-virginica两类。

从树以及决策区域图上可以看到决策树在花朵分类上表现不错。不幸的是scikit-learn目前还不提供手工决策树修剪功能。然而，可以返回到前面的代码示例，把决策树的参数max_depth修改为3，然后与当前的模型进行比较，虽然这里并不会去实现，但是把这作为练习留给感兴趣的读者。

### 3.6.3 通过随机森林组合多个决策树

随机森林由于其良好的分类性能、可扩展性和易用性，在过去十年在机器学习的应用中广受欢迎。直观地说，可以把随机森林看成是决策树的集合。随机森林背后的逻辑是对分别受较大方差影响的多个决策树取平均值，以建立一个具有更好的泛化性能和不易过拟合的强大模型。随机森林算法可以概括为四个简单步骤：

1.随机提取规模为n的引导样本（从训练集中随机选择n个可替换样本）。

2.基于引导样本的数据生成决策树。在每个节点完成以下任务：

a.随机选择d个特征，无须替换。

b.根据目标函数提供的最佳分裂特征来分裂节点，例如，最大化信息增益。

3.把步骤1和2重复k次。

聚合每棵树的预测结果，以多数票机制确定标签的分类。第7章将结合不同的集成学习模式更详细地讨论多数票机制。

如果对取样时有无替换的概念不熟悉，那可以通过一个简单的思考实验来阐明。假设玩抽奖游戏，从一个罐中随机抽取数字。罐子中有0、1、2、3和4五个唯一的数字，每次抽取一个数字。第一轮，从罐中抽取某个数的概率是1/5。在无替换采样的场景下，每轮抽奖后不把数字放回罐中。因此，下一轮从剩余的数字中抽取某个数的概率取决于前一轮。例如，如果剩下的数字是0、1、2和4，那么在下一轮中抽到0的概率将变成1/4。

然而，有替换的随机抽样总是把抽到的数放回罐中，以便确保在下一轮抽取时到某个数字的概率保持不变。换句话说，有替换取样的样本（数字）是独立的，并且协方差为零。例如，五轮随机数的抽样结果可以是这样的：

无替换随机取样：2、1、3、4、0

有替换随机取样：1、3、3、4、1

应该注意，在步骤2中训练单个决策树时，并不评估所有的特征来确定节点的最佳分裂方案，而是仅仅考虑其中的一个随机子集。

尽管随机森林的解释性不如决策树，但是随机森林有一个大的优势，那就是不必为选择好的超参数值担心。通常不需要修剪随机森林，因为集合的模型对来自于单个决策树的噪声抵抗力强。实际上，唯一需要关心的参数是在步骤3中选择随机森林中树的多寡k。通常，树越多，随机森林分类器的性能就越好，当然计算成本的增加也就越大。

虽然在实践中并不常见，但是随机森林分类器的其他参数也可以被优化，可以用第5章将要讨论的技术进行优化。这些超参数分别包括步骤1中的导引样本规模n和步骤2.1中的每次分裂的随机选择的特征数d。通过导引样本规模n来平衡随机森林的偏置与方差。

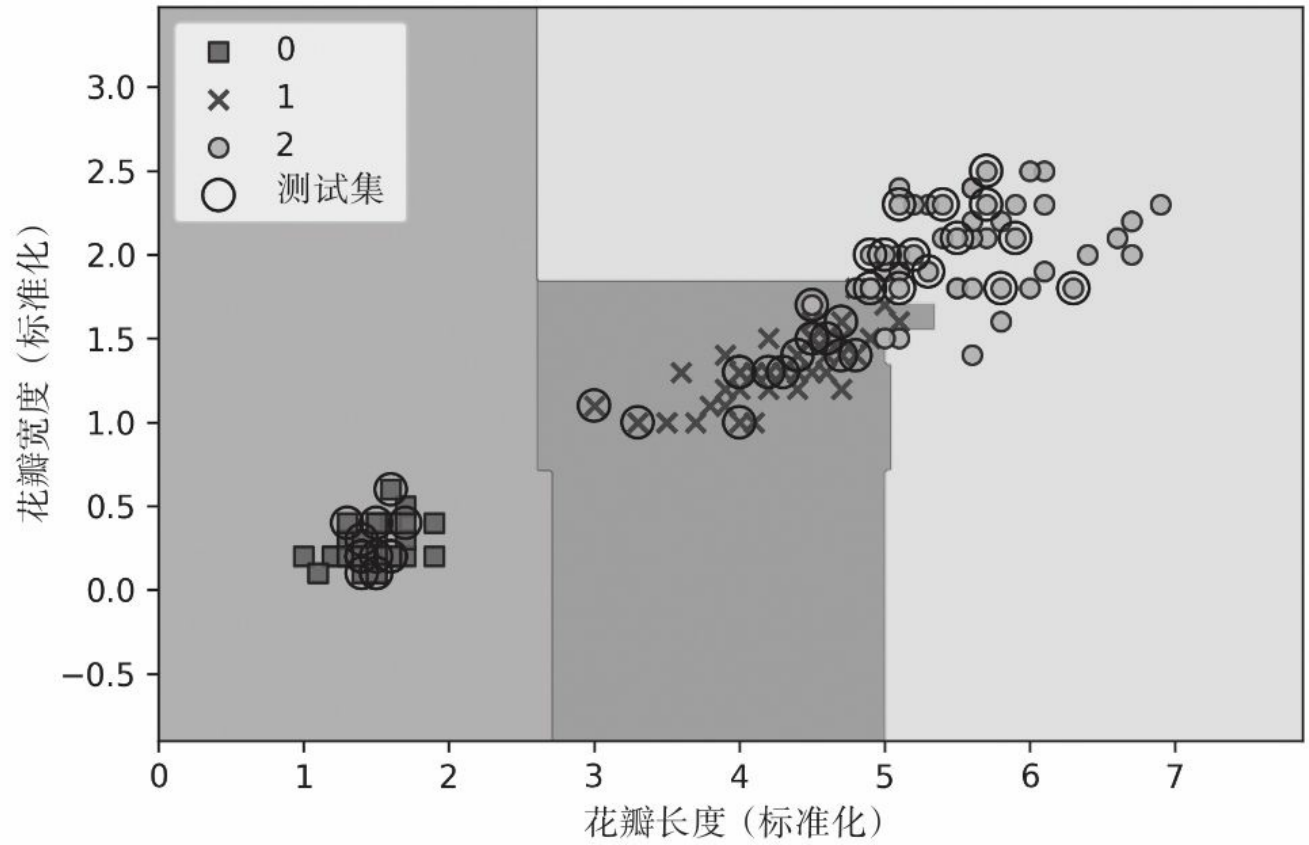
因为特定训练数据包含在导引样本中的概率较低，所以减小导引样本的规模可以增加单棵树之间的多样性。因此，缩小导引样本的规模可能会增加随机森林的随机性，有助于减少过拟合的影响。然而，较小规模的导引样本通常会导致随机森林的总体性能较差，训练和测试样本之间的性能差别很小，总体上测试性能较低。相反，加大导引样本的规模可能会增加过拟合的程度。由于导引样本的存在，各个决策树相互之间变得更相似，它们能通过学习更紧密地拟合原始的训练集。

在大多数随机森林的实现中，包括scikit-learn中的RandomForestClassifier，选择导引样本的规模与原始训练集的样本规模一致，这通常会有一个良好的偏置方差权衡。对于每轮分裂中特征d的数量，希望选择的值小于训练集的特征总数。在scikit-learn以及其他实现中，合理的默认值为$d = \sqrt{m}$，这里m为训练集的特征总数。

为方便起见，并未从各决策树本身开始构建随机森林分类器，因为scikit-learn已经实现了一个可供使用：

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> forest = RandomForestClassifier(criterion='gini',
...                                 n_estimators=25,
...                                 random_state=1,
...                                 n_jobs=2)
>>> forest.fit(X_train, y_train)
>>> plot_decision_regions(X_combined, y_combined,
...                       classifier=forest, test_idx=range(105,150))
>>> plt.xlabel('petal length')
>>> plt.ylabel('petal width')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

执行前面的代码可以看到由随机森林中树的组合产生的决策区域，如下图所示：




通过设置参数n_estimators并用熵作为杂质度的判断准则来分裂节点，利用前面的代码训练了由25棵决策树组成的随机森林。虽然这是在非常小的训练集上，为了演示的目的而生长出来的小规模随机森林，但通过设置参数n_jobs，我们可以用多核计算机（这里是两核）来并行训练模型。

## 3.7 K-近邻——一种懒惰的学习算法

本章要讨论的最后一个有监督学习算法是K-近邻（KNN）分类器，它特别有趣，因为它与迄今为止讨论过的学习算法在本质上不同。

参数与非参数模型

机器学习算法可以分为参数与非参数模型。参数模型指我们估计参数，然后从训练集中学习出一个函数，用来对新数据点分类，而不再需要原来的训练集。参数模型的典型例子是感知器、逻辑回归和线性支持向量机。相比之下，非参数模型不能用一组固定的参数来描述，参数的个数随着训练数据的增加而增长。前面已经看到两个非参数模型的例子，决策树分类器（随机森林）和核支持向量机。

KNN算法是基于实例的学习，属于非参数模型。基于实例学习的模型以记忆训练集为特征，懒惰学习是基于实例学习的一种特殊情况，这与它在学习过程中付出零代价有关。

KNN是懒惰学习的典型例子。所谓的懒惰，并不是说它看上去很简单，而在于它不是从训练数据中学习判别函数，而是靠记忆训练过的数据集来完成任务。

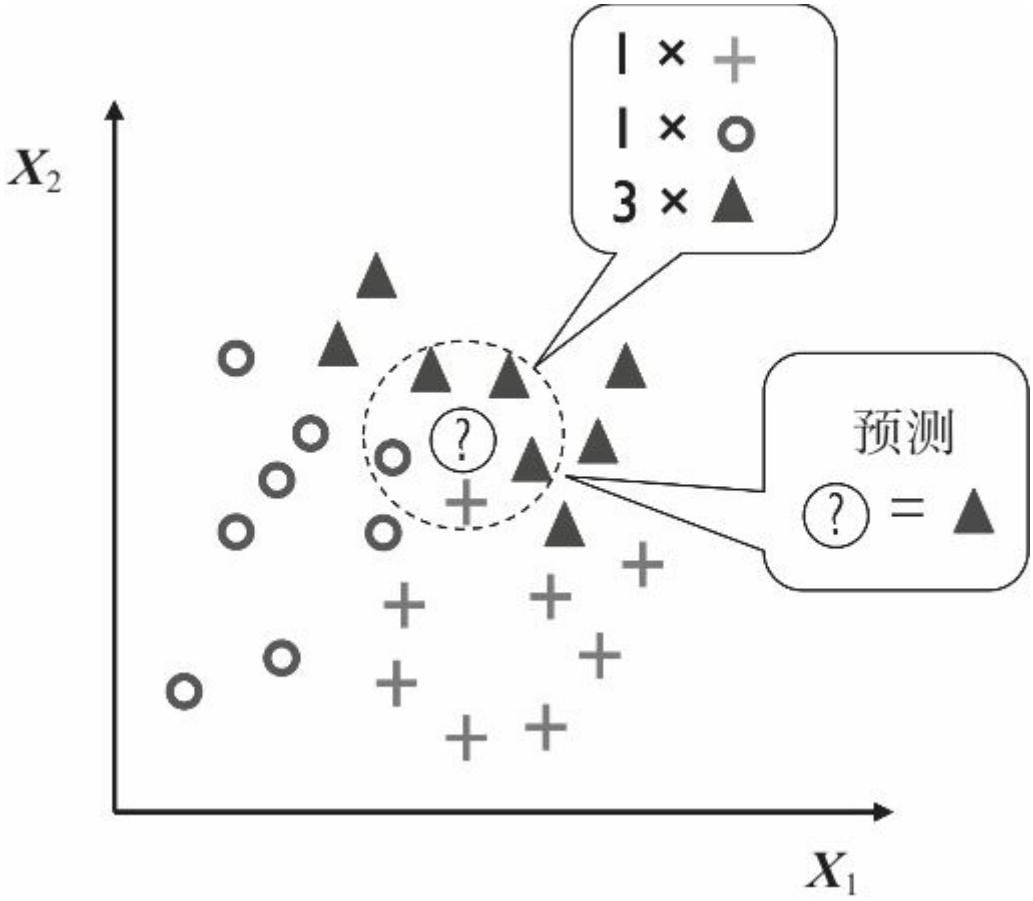
KNN算法本身相当简单，可以总结为以下的几步：

1.选择k个数和一个距离度量。

2.找到要分类样本的k-近邻。

3.以多数票机制确定分类标签。

下图显示了新的数据点（？）是如何基于多数表决的原则为五个最近的相邻节点分配三角形分类标签的。



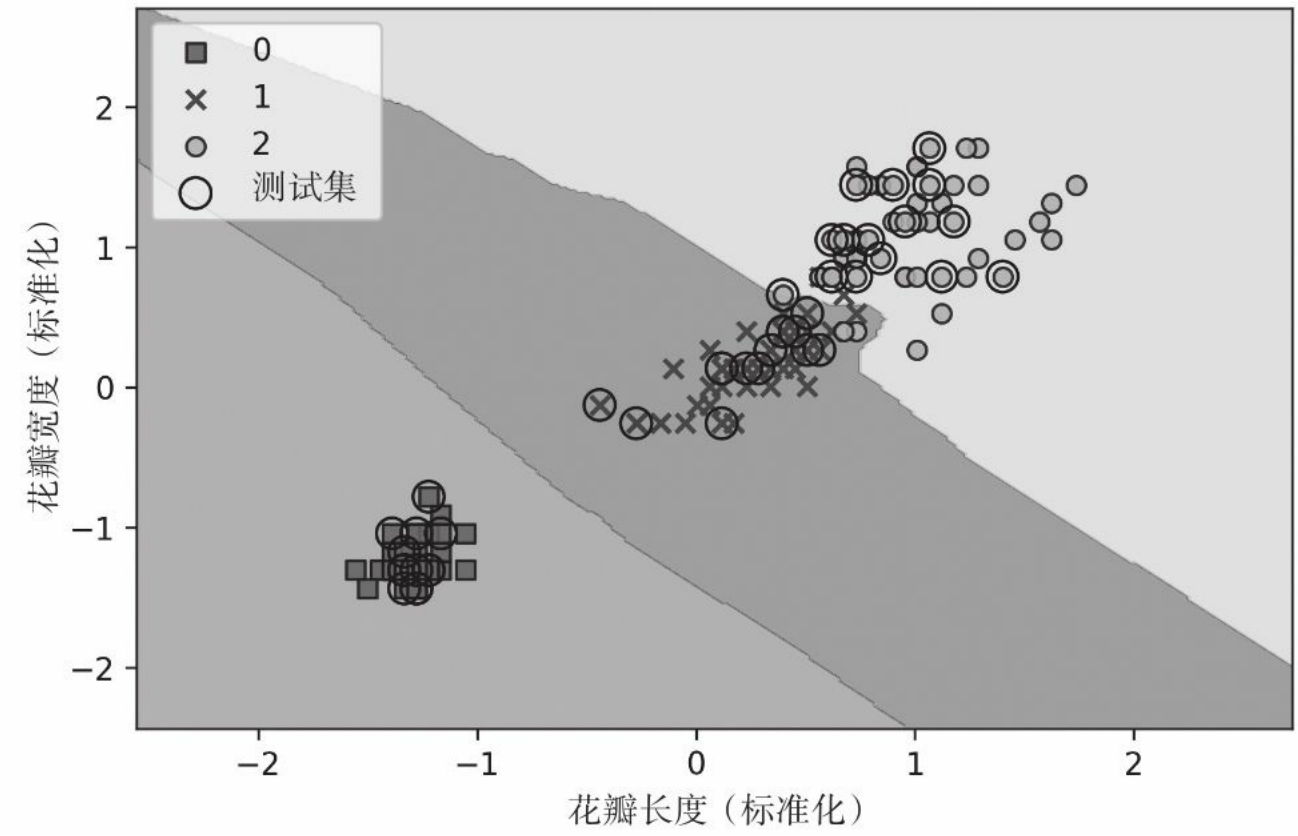

基于所选择的距离度量，KNN算法从训练样本中发现k个最接近要分类的数据点的样本。新数据点的分类标签由最靠近该点的k个数据点的多数票决定。

这种基于内存方法的主要优点是当新的训练数据出现时，分类器可以立即适应。缺点是新样本分类的计算复杂度与训练样本在最坏情况下的规模呈线性关系，除非数据集只有很少的维度（特征），而且算法实现采用有效的数据结构如KD树。（J.H.弗里德曼，J.L.本特利，R.A.芬克尔.《一种以对数预期时间来寻找最佳匹配的算法》.数学软件汇刊（TOMS），1977，3（3）：209–226）此外，由于没有训练步骤，所以不能丢弃训练样本。因此，如果要处理大型数据集，存储空间将面临挑战。

执行下面的代码可以用scikit-learn的欧氏距离度量实现KNN模型：

```
>>> from sklearn.neighbors import KNeighborsClassifier
>>> knn = KNeighborsClassifier(n_neighbors=5, p=2,
...                            metric='minkowski')
>>> knn.fit(X_train_std, y_train)
>>> plot_decision_regions(X_combined_std, y_combined,
...                       classifier=knn, test_idx=range(105,150))
>>> plt.xlabel('petal length [standardized]')
>>> plt.ylabel('petal width [standardized]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

该数据集用KNN模型指定五个邻居，得到下图所示的相对平滑决策边界：



在争执不下的情况下，用scikit-learn实现的KNN算法将更喜欢近距离样本的邻居。如果邻居有相似的距离，该算法将选择在训练集中最先出现的分类标签。

合适地选择k值对在过拟合与欠拟合之间找到恰当的平衡至关重要。必须确保选择适合于数据集中特征的距离度量。通常用简单的欧几里得距离来度量，例如，鸢尾花数据集中的花样本，其特征度量以厘米为单位。然而，如果用欧几里得距离度量，那么对数据进行标准化也是很重要的，确保每个特征都对距离起着同样的作用。在前面代码中使用的闵可夫斯基距离只是欧几里得和曼哈顿距离之外的一种选择，可以表示如下：

$$d\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = \sqrt[p]{\sum_k \left|x_k^{(i)} - x_k^{(j)}\right|^p}$$

如果参数p=2，就是欧几里得距离，如果p=1，就是曼哈顿距离。在scikit-learn中可以通过设置度量参数采用不同的方法来度量距离。可以从下述网站找到列表：

http://scikit-

learn.org/stable/modules/generated/sklearn.neighbors.DistanceMetric.html

### 维数诅咒

因为维数诅咒，KNN易于过拟合，了解这一点非常重要。当固定规模的训练集的维数越来越大时，特征空间变得越来越稀疏，这种现象被称为维数诅咒。直观地说，可以认为即使是最近的邻居在高维空间的距离也很远，以至于无法合适地估计。

在逻辑回归一节已经讨论了正则化的概念，并以此作为避免过拟合的一种方法。然而，在不适用正则化的模型中，如决策树和KNN，可以利用特征选择和降维技术来避免维数诅咒。下一章将对此进行更详细的讨论。

## 3.8 小结

本章学习了许多不同的解决线性和非线性问题的机器学习算法。如果关心可解释性，决策树特别有吸引力。逻辑回归不仅是一种有用的在线随机梯度下降模型，而且还可以预测特定事件的概率。虽然支持向量机有强大的线性模型，也可以通过核技巧扩展到非线性问题，但必须调整许多参数才能做好预测。相比之下，像随机森林这样的组合方法不需要调整太多参数，而且不易过拟合，像决策树一样容易，这使得其成为许多实际问题领域具有吸引力的模型。KNN分类器通过懒惰学习提供了另一种分类方法，允许在没有任何模型训练的情况下进行预测，但预测所涉及的计算成本昂贵。

然而，比选择适当的学习算法更重要的是训练集中的可用数据。如果没有丰富和差异化的特征，任何算法都不能做出好的预测。

下一章将讨论数据预处理、特征选择和降维几个重要主题，这些是建立强大的机器学习模型所必需的。第6章将会看到如何评价和比较模型的性能以及学习有用的技巧来调整算法。

# 第4章　构建良好的训练集——预处理

数据的质量及其所包含的有价值信息是决定机器学习算法优劣的关键。因此，在将数据集提供给机器学习算法之前，确保对数据集的检查和预处理非常关键。本章将讨论必要的数据预处理技术，以帮助建立良好的机器学习模型。

本章将主要涵盖下述几个方面：

·去除和填补数据集的缺失数值

·将分类数据转换为适合机器学习算法的格式

·为构造模型选择相关的特征

## 4.1 处理缺失数据

在实际应用中，由于各种原因缺少样本的一个或多个数值的现象并不少见。可能在数据收集的过程中出现了错误，某些测量不适当，也可能是某个字段在调查时为空白。常见的缺失是数据表中的空白或占位符，如NaN，它表示该位置不是一个数字，或者是NULL（在关系型数据库中常用的未知值指示符）。

不幸的是大多数的计算工具都无法处理这些缺失，但如果简单地忽略它们甚至可能会产生不可预知的结果。因此，在对数据进行进一步分析之前，必须先处理好缺失数值。本节将介绍几种处理缺失数值的实用技术，包括删除或用其他样本或特征填充。

### 4.1.1 识别数据中的缺失数值

在讨论处理缺失数值技术之前，先用以逗号分隔的（CSV）文件创建一个简单的示例性数据帧，以便更好地理解问题：

```
>>> import pandas as pd
>>> from io import StringIO

>>> csv_data = \
... '''A,B,C,D
... 1.0,2.0,3.0,4.0
... 5.0,6.0,,8.0
... 10.0,11.0,12.0,'''
>>> # If you are using Python 2.7, you need
>>> # to convert the string to unicode:

>>> # csv_data = unicode(csv_data)
>>> df = pd.read_csv(StringIO(csv_data))
>>> df
     A    B    C    D
0 1.0  2.0  3.0  4.0
1 5.0  6.0  NaN  8.0
2 10.0 11.0 12.0 NaN
```

前面的代码调用read_csv函数把CSV格式的数据读入pandas数据帧（DataFrame），结果发现有两个失踪的表单元被NaN所取代。前面代码示例中的StringIO函数只用于说明。从csv_data读入数据到pandas的数据帧（DataFrame）就像用硬盘上的普通CSV文件一样方便。

对比较大的数据帧，手工查找丢失的数值可能很烦琐。在这种情况下，可以调用isnull方法返回包含布尔值的数据帧，指示一个表单元是包含了数字型的数值（False）还是数据缺失（True）。

调用sum方法，可以得到每列缺失数值的统计，如下所示：

```
>>> df.isnull().sum()
A    0
B    0
C    1
D    1
dtype: int64
```

这样就可以统计表中每列缺失数值的情况。下一小节将介绍处理这些缺失数据的策略。

尽管scikit-learn是为NumPy阵列开发的，有时可以用pandas的DataFrame来更方便地预处理数据。在为scikit-learn评估器提供数据之前，随时可以通过values属性来存取DataFrame底层NumPy阵列中的数据：

```
>>> df.values
array([[  1.,   2.,   3.,   4.],
       [  5.,   6.,  nan,   8.],
       [ 10.,  11.,  12.,  nan]])
```

### 4.1.2 删除缺失的数据

处理缺失数据最简单的方法是从数据集中彻底删除相应的特征（列）或样本（行），调用dropna方法可以很容易地删除缺失的数据行：

```
>>> df.dropna(axis=0)
     A    B    C    D
0  1.0  2.0  3.0  4.0
```

类似，也可以通过设置axis参数为1来删除其中至少一行包含NaN的列：

```
>>> df.dropna(axis=1)
     A    B
0  1.0  2.0
1  5.0  6.0
2  10.0 11.0
```

dropna方法还支持一些其他的参数，有时候也可以派上用场：

```
# only drop rows where all columns are NaN
# (returns the whole array here since we don't
# have a row with where all values are NaN
>>> df.dropna(how='all')
     A    B    C    D
0  1.0  2.0  3.0  4.0
1  5.0  6.0  NaN  8.0
2 10.0 11.0 12.0  NaN


# drop rows that have less than 4 real values
>>> df.dropna(thresh=4)
     A    B    C    D
0  1.0  2.0  3.0  4.0


# only drop rows where NaN appear in specific columns (here: 'C')
>>> df.dropna(subset=['C'])
     A    B    C   D
0  1.0  2.0  3.0 4.0
2 10.0 11.0 12.0 NaN
```

虽然删除缺失数据似乎很方便，但也有一定的缺点。例如，可能最终会因为删除太多样本而使分析变得不可靠。也可能因为删除了太多特征列使分类器无法获得有价值的信息。下一节将研究处理缺失数据的最常用方法之一，即插值技术。

### 4.1.3 填补缺失的数据

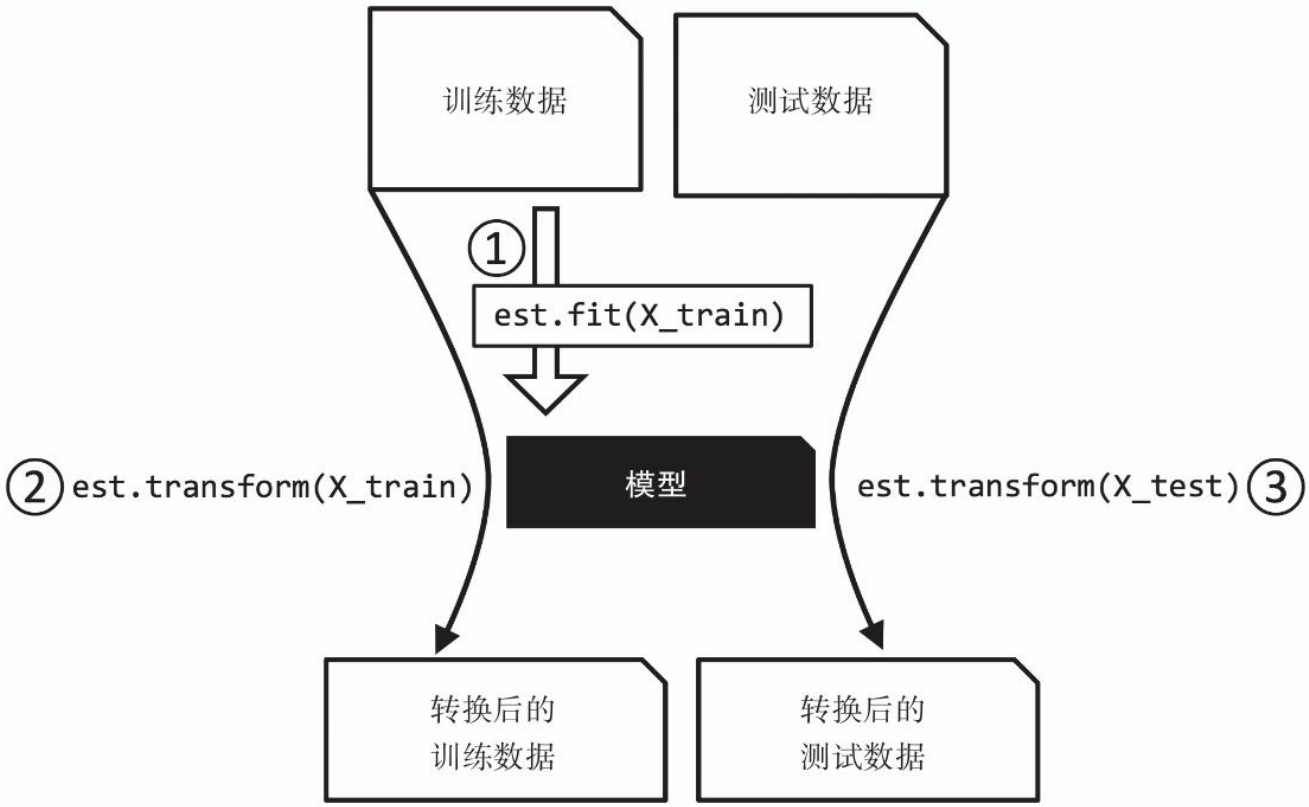
通常，删除样本或整列特征不可行，因为这会损失太多有价值的数据。在这种情况下，可以用不同的插值技术，根据其他的训练样本来估计缺失数据。最常见的插值技术是均值插补，只需用整个特征列的平均值替换缺失数据。一个方便的实现方式是调用scikit-learn的Imputer，如下面的代码所示：

```
>>> from sklearn.preprocessing import Imputer
>>> imr = Imputer(missing_values='NaN', strategy='mean', axis=0)
>>> imr = imr.fit(df.values)
>>> imputed_data = imr.transform(df.values)
>>> imputed_data
array([[  1.,   2.,   3.,   4.],
       [  5.,   6.,  7.5,   8.],
       [ 10.,  11.,  12.,   6.]])
```

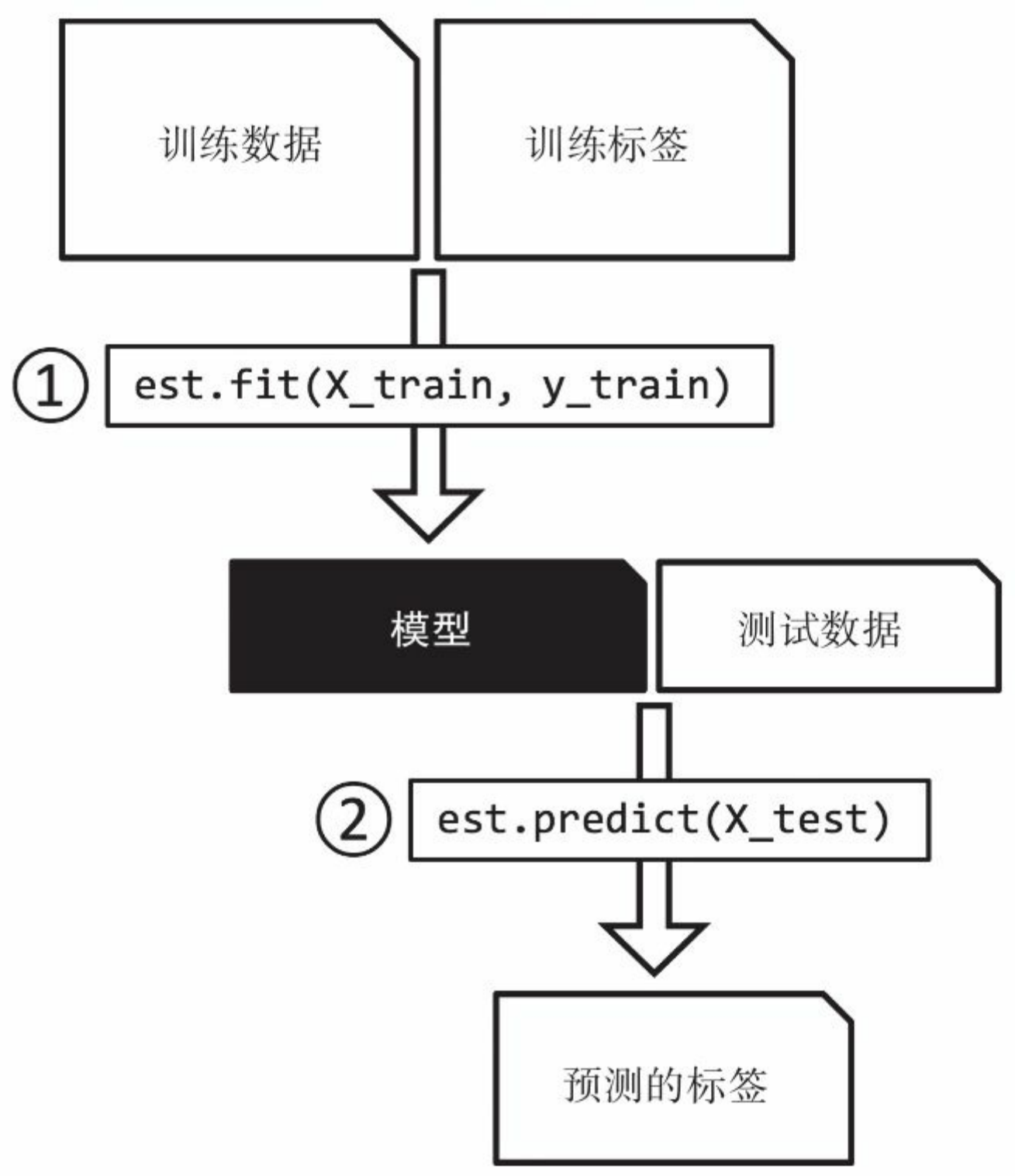
这里用相应的平均值替换NaN，特征列分别计算。如果把参数axis的值从0换成1，就可以计算行均值。strategy的其他参数包括median或者most_frequent，后者用最频繁的值来替代缺失数据。这对填补分类的特征值非常有价值，例如存储如红、绿、蓝颜色编码的特征列，本章后面会遇到这样的数据实例。

## 4.1.4 了解scikit-learn评估器API

上一节用scikit-learn库的Imputer类填补数据集中缺失的数据。Imputer类属于scikit-learn库中用来进行数据转换的转换器类。评估工具的两个基本方法是fit和transform。采用fit方法从训练数据学习参数，采用transform方法利用参数来对数据进行变换。任何要转换的数据阵列都必须要有与拟合模型的数据阵列相同数量的特征。下图展示了如何用在训练集上测试过的转换器转换训练数据和新的测试数据：




第3章所涉及的分类器属于scikit-learn的评估器，该API与转换器类在概念上非常像。本章后面的评估器有predict和transform方法。可能还记得，当训练评估器分类时，也曾调用fit方法学习模型的参数。然而有监督学习为拟合模型额外提供了分类标签，并且调用predict方法对新数据样本进行预测，如下图所示：


训练数据
训练标签
①
est.fit(X_train, y_train)
模型
测试数据
②
est.predict(X_test)
预测的标签

## 4.2 处理分类数据

到目前为止我们只研究了数值特征。然而，现实世界中含一个或多个分类特征的数据集比比皆是。本节将以简单而有效的例子来讨论如何用数值计算库处理这类数据。

### 4.2.1 名词特征和序数特征

在谈论分类数据时，必须进一步区分名词特征和序数特征。序数特征可以理解为可排序的分类值。举个例子，T恤的尺寸是一个序数特征，因为可以定义顺序XL＞L＞M。相反，名词特征并不意味着任何顺序，T恤颜色就是名词特征，如果套用前面的例子把它当成序数特征就不对了，因为说红的比蓝的大通常不符合逻辑。

#### 创建示例数据集

在探讨用不同的技术来处理这样的分类数据之前，先创建一个新的DataFrame：

```
>>> import pandas as pd
>>> df = pd.DataFrame([
...             ['green', 'M', 10.1, 'class1'],
...             ['red', 'L', 13.5, 'class2'],
...             ['blue', 'XL', 15.3, 'class1']])
>>> df.columns = ['color', 'size', 'price', 'classlabel']
>>> df
   color size  price classlabel
0  green    M   10.1     class1
1    red    L   13.5     class2
2   blue   XL   15.3     class1
```

从前面的输出可以看到，新创建的DataFrame包含一个名词（color）特征列、一个序数（size）特征列和一个数值（price）特征列。分类标签存储在最后一列。本书所讨论的分类学习算法并不包括分类标签中的序数。

## 4.2.2 映射序数特征

为了确保机器学习算法能够正确地解读序数特征，需要将分类字符串转换成整数。不幸的是，找不到可以自动导出特征尺寸正确顺序的方便函数，因此要人工定义映射关系。在下面的简单例子中，假设知道特征尺寸在数量上的关系，例如XL=L+1=M+2：

```
>>> size_mapping = {
...                 'XL': 3,
...                 'L': 2,
...                 'M': 1}
>>> df['size'] = df['size'].map(size_mapping)
>>> df
   color  size  price classlabel
0  green     1   10.1     class1
1    red     2   13.5     class2
2   blue     3   15.3     class1
```

如果想以后再把整数型数值转换回原来字符串的形式，可以简单地定义一个反向映射字典inv_size_mapping={v：k for k，v in size_mapping.items（）}，然后在变换后的特征列上调用pandas的map方法来应用。如下所示：

```
>>> inv_size_mapping = {v: k for k, v in size_mapping.items()}
>>> df['size'].map(inv_size_mapping)
0     M
1     L
2    XL
Name: size, dtype: object
```

### 4.2.3 分类标签编码

许多机器学习库要求分类标签的编码为整数值。虽然大多数scikit-learn的分类评估器可以在内部进行整数分类标签的转换，但提供整数阵列作为分类标签以避免技术障碍被视为最佳实践。可以采用类似于先前讨论的序数特征映射方法为分类标签编码。需要记住，分类标签并不是序数，具体哪个整数匹配特定的字符串标签无关紧要。因此，可以从0开始简单地枚举：

```
>>> import numpy as np
>>> class_mapping = {label:idx for idx,label in
...                  enumerate(np.unique(df['classlabel']))}
>>> class_mapping
{'class1': 0, 'class2': 1}
```

接下来，可以用映射字典将分类标签转换为整数：

```
>>> df['classlabel'] = df['classlabel'].map(class_mapping)
>>> df
   color  size  price  classlabel
0  green     1   10.1           0
1    red     2   13.5           1
2   blue     3   15.3           0
```

可以在字典中反向映射键值对，将转换后的分类标记匹配到原来的字符串，如下所示：

```
>>> inv_class_mapping = {v: k for k, v in class_mapping.items()}
>>> df['classlabel'] = df['classlabel'].map(inv_class_mapping)
>>> df
   color  size  price classlabel
0  green     1   10.1     class1
1    red     2   13.5     class2
2   blue     3   15.3     class1
```

另外，也可以在scikit-learn中直接调用方便的LabelEncoder类实现：

```
>>> from sklearn.preprocessing import LabelEncoder
>>> class_le = LabelEncoder()
>>> y = class_le.fit_transform(df['classlabel'].values)
>>> y
array([0, 1, 0])
```

请注意，fit_transform方法只是分别调用fit和transform的一种快捷方式，可以调用inverse_transform方法将分类的整数型标签转换回原来的字符串形

式：

```
>>> class_le.inverse_transform(y)
array(['class1', 'class2', 'class1'], dtype=object)
```

### 4.2.4 为名词特征做热编码

上一节用简单的字典映射方法将序数特征size转换为整数。由于scikit-learn分类评估器把分类标签当成无序的名词特征数据进行分类，用方便的LabelEncoder通过编码把字符串标签转换为整数。因此可以用类似方法转换数据集中的名词特征列color，代码如下：

```
>>> X = df[['color', 'size', 'price']].values
>>> color_le = LabelEncoder()
>>> X[:, 0] = color_le.fit_transform(X[:, 0])
>>> X
array([[1, 1, 10.1],
       [2, 2, 13.5],
       [0, 3, 15.3]], dtype=object)
```

执行上面的代码后，NumPy阵列x的第一列现在拥有了新的颜色值，其编码格式如下：

- blue = 0
- green = 1
- red = 2

如果就此打住，并把阵列提供给分类器，那就会犯处理分类数据最常见的错误。知道问题的所在吗？虽然颜色值并没有任何特定的顺序，但机器学习算法会假设green大于blue，red大于green。尽管该假设并不正确，但是算法仍然可以产生有用的结果。然而，这些并不是最优的结果。

解决这个问题的常见方案是使用一种被称为独热编码的技巧。该方法背后的逻辑是为名词特征列的每个唯一值创建一个新的虚拟特征。于是将把color特征转换为三个新特征：blue、green和red。然后用二进制值表示样本的特定颜色。例如，blue样本可以编码为blue=1，green=0，red=0。可以调用scikit-learn.preprocessing模块中的OneHotEncoder来实现这种转换：

```
>>> from sklearn.preprocessing import OneHotEncoder

>>> ohe = OneHotEncoder(categorical_features=[0])
>>> ohe.fit_transform(X).toarray()
array([[  0. ,   1. ,   0. ,   1. ,  10.1],
       [  0. ,   0. ,   1. ,   2. ,  13.5],
       [  1. ,   0. ,   0. ,   3. ,  15.3]])
```

在初始化OneHotEncoder时，用参数categorical_features定义想要变换的变量在列中的位置（注意color位于特征矩阵X的第一列）。默认情况下，当调用transform方法时，OneHotEncode返回一个稀疏矩阵，然后将稀疏矩阵转换为常规密度的NumPy阵列，通过调用toarray方法达到可视化的目的。稀疏矩阵是存储大数据集更有效的方式，得到许多scikit-learn函数的支持，该方法对阵列中包含很多零的情况特别有效。省略toarray步骤，也可以把编码器初始化为OneHotEncoder（...，sparse=False），返回常规的NumPy阵列。

有一个更方便的通过独热编码创建虚拟特征的方法是在pandas中调用get_dummies方法。把get_dummies方法应用到数据帧，只转换字符串列，而保持所有其他的列不变：

```
>>> pd.get_dummies(df[['price', 'color', 'size']])
   price  size  color_blue  color_green  color_red
0   10.1     1           0            1          0
1   13.5     2           0            0          1
2   15.3     3           1            0          0
```

当使用独热编码为数据集编码时，必须小心它会带来多重共线性，对某些方法这可能是个问题（例如，那些需要矩阵求逆的方法）。高度相关的特征很难计算反转矩阵，因此可能会导致数值估计不稳定。为了减少变量之间的相关性，可以直接从独热编码阵列删除一个特征列。请注意，尽管删除了一个特征列，但并没有失去任何重要的信息。例如，如果删除color_blue列，特征信息仍然得以保留，因为如果观察到color_green=0和color_red=0，这意味着余下的观察结果必然是蓝色。

在调用get_dummies函数时，可以通过传递True参数给drop_first删除第一列，如下面的代码示例所示：

```
>>> pd.get_dummies(df[['price', 'color', 'size']],
...                drop_first=True)
   price  size  color_green  color_red
0   10.1     1            1          0
1   13.5     2            0          1
2   15.3     3            0          0
```

OneHotEncoder没有去除列参数，但可以简单地分割独热编码的NumPy

阵列，如下述代码片段所示：

```
ohe = OneHotEncoder(categorical_features=[0])
ohe.fit_transform(X).toarray()[:, 1:]
array([[  1. ,   0. ,   1. ,  10.1],
       [  0. ,   1. ,   2. ,  13.5],
       [  0. ,   0. ,   3. ,  15.3]])
```

## 4.3 分裂数据集为独立的训练集和测试集

第1章和第3章简要地介绍了把数据集分裂成独立训练集和测试集的概念。记住，比较预测值与测试集的真值可以理解为评估模型所表现的无偏见性能，然后再放松它在现实世界的要求。本节将创建一个新的数据集，即葡萄酒数据集。在对数据集进行预处理后，再探索不同的特征选择技术，以减少数据集的维数。

葡萄酒数据集是另一个开源数据集，可以从UCI的机器学习库（https://archive.ics.uci.edu/ml/datasets/Wine）获得。该数据集包含了178个样本的13个特征，从不同角度对不同的化学特性进行了描述。

可以在本书的代码包https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data中找到葡萄酒数据集（以及本书用到的其他数据集），以备脱机工作或者UCI服务器暂时不可用时使用，想要从本地目录加载葡萄酒数据集，可以用下面这行

```
df = pd.read_csv('your/local/path/to/wine.data',
                 header=None)
```

替换此行：

```
df = pd.read_csv('https://archive.ics.uci.edu/ml/'
        'machine-learning-databases/wine/wine.data',
         header=None)
```

可以直接用pandas从UCI的机器学习库读入开源的葡萄酒数据集：

```
>>> df_wine = pd.read_csv('https://archive.ics.uci.edu/'
                          'ml/machine-learning-databases/'
                          'wine/wine.data', header=None)
>>> df_wine.columns = ['Class label', 'Alcohol',
...                    'Malic acid', 'Ash',
```

```
...                         'Alcalinity of ash', 'Magnesium',
...                         'Total phenols', 'Flavanoids',
...                         'Nonflavanoid phenols',
...                         'Proanthocyanins',
...                         'Color intensity', 'Hue',
...                         'OD280/OD315 of diluted wines',
...                         'Proline']
>>> print('Class labels', np.unique(df_wine['Class label']))
Class labels [1 2 3]
>>> df_wine.head()
```

下表中列出了葡萄酒数据集的13种化学特征，描述了178个葡萄酒样本的化学特性：

| | 分类标签 | 酒精 | 苹果酸 | 灰 | 灰的碱度 | 镁 | 总酚 | 黄酮类化合物 | 非黄烷类酚类 | 原花青素 | 色彩强度 | 色调 | 稀释酒OD280 OD315 | 脯氨酸 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 14.23 | 1.71 | 2.43 | 15.6 | 127 | 2.80 | 3.06 | 0.28 | 2.29 | 5.64 | 1.04 | 3.92 | 1 065 |
| 1 | 1 | 13.20 | 1.78 | 2.14 | 11.2 | 100 | 2.65 | 2.76 | 0.26 | 1.28 | 4.38 | 1.05 | 3.40 | 1 050 |
| 2 | 1 | 13.16 | 2.36 | 2.67 | 18.6 | 101 | 2.80 | 3.24 | 0.30 | 2.81 | 5.68 | 1.03 | 3.17 | 1 185 |
| 3 | 1 | 14.37 | 1.95 | 2.50 | 16.8 | 113 | 3.85 | 3.49 | 0.24 | 2.18 | 7.80 | 0.86 | 3.45 | 1 480 |
| 4 | 1 | 13.24 | 2.59 | 2.87 | 21.0 | 118 | 2.80 | 2.69 | 0.39 | 1.82 | 4.32 | 1.04 | 2.93 | 735 |

这些样本来自于1、2、3三种不同类别的葡萄酒，分别是意大利同一地区种植的三种不同品种的葡萄，但来自于不同的葡萄酒，如数据集摘要中所描述的那样（https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.names）。

把数据集随机分裂成独立的训练集和测试集的一个方便的方法是，从scikit-learn的model_selection子模块中调用train_test_split函数：

```
>>> from sklearn.model_selection import train_test_split
>>> X, y = df_wine.iloc[:, 1:].values, df_wine.iloc[:, 0].values
>>> X_train, X_test, y_train, y_test =\
...     train_test_split(X, y,
...                      test_size=0.3,
...                      random_state=0,
...                      stratify=y)
```

首先把NumPy阵列的特征列1-13赋予变量X，把第一列的分类标签赋予变量y。调用train_test_split函数把数据集随机分裂成x和y，分别对应独立的训练集和测试集。通过设置test_size=0.3，把30%的葡萄酒样本分配给X_test和y_test，把余下70%的样本分配给X_train和y_train。把分类标签阵列y作为参数提供给stratify，确保训练集和测试集具有与原始数据集相同的分类比例。

把数据集分裂为训练集和测试集的目的是要确保机器学习算法可以从中获得有价值的信息。因此没必要将太多信息分配给测试集。然而，测试集越小，泛化误差的估计就越不准确。将数据集分裂为训练集和测试集就是对两者的平衡。在实践中，最常用的分裂比例为60：40，70：30或80：20，具体取决于初始数据集的规模。然而，对大规模数据集，训练集和测试集的分裂比例为90：10或99：1也是常见和适当的做法。普遍的做法是在模型训练和评估后保留测试数据，然后在整个数据集上再对分类器进行训练，以提高模型的预测性能。虽然通常推荐这种方法，但它可能会导致较差的泛化性能。例如当数据集规模很小而且测试集包含异常值时。此外，在对整个数据集进行模型再度拟合后，我们没有任何剩余的独立数据来评估性能。

## 4.4 把特征保持在同一尺度上

特征比例调整是预处理环节中很容易被遗忘的关键步骤。决策树和随机森林是两种少有的不必担心特征比例调整的机器学习算法。这两种算法不随特征比例的影响。然而，大多数其他的机器学习和优化算法，在相同特征比例的情况下表现优异，正如在第2章中实现梯度下降优化算法时所见到的那样。

可以用一个简单的例子说明特征比例的重要性。假设有两个特征，一个是在1到10范围内测量的，而另一个是在1到100 000范围内测量的。回顾第2章中Adaline的平方误差函数，你很自然会说算法主要是忙于优化误差较大的第二个特征的权重。另一个例子是用欧氏距离度量的k-近邻（KNN）算法，样本之间的计算距离将由第二个特征轴支配。

归一化和标准化两种常见方法可以将不同的特征统一到同一比例。这些术语不严格而且经常在不同领域使用，具体含义要根据场景来判断。归一化通常指的是把特征的比例调整到[0，1]区间，这是最小最大比例调整的一种特殊情况。为了使数据正常化，可以简单地对每个特征列应用最小最大比例调整，其中样本$x^{(i)}$的新值$x^{(i)}_{norm}$可以计算如下：

$$x^{(i)}_{norm} = \frac{x^{(i)} - x_{min}}{x_{max} - x_{min}}$$

这里，$x^{(i)}$某个特定样本，$x_{min}$为特征列的最小值，$x_{max}$为特征列的最大值。

scikit-learn实现的最大最小比例调整程序可以应用如下：

```
>>> from sklearn.preprocessing import MinMaxScaler
>>> mms = MinMaxScaler()
>>> X_train_norm = mms.fit_transform(X_train)
>>> X_test_norm = mms.transform(X_test)
```

通过最大最小比例调整实现归一化是一种常用的技术，这对需要有界区间值的问题很有用。标准化对于许多机器学习算法来说更为实用，特别是梯度下降等优化算法。因为许多线性模型，如第3章中的逻辑回归和支持向量机，把权重值初始化为0或接近0的随机值。使用标准化，可以把特征列的中心设在均值为0且标准偏差为1的位置，这样特征列呈正态分布，可以使学习权重更容易。此外，标准化保持了关于离群值的有用信息，使算法对离群值

不敏感，这与最小最大比例调整刚好相反，它把数据调整到有限的值域。标准化的过程可以用下述等式来表示：

$$x_{\text{std}}^{(i)} = \frac{x^{(i)} - \mu_{\text{x}}}{\sigma_{\text{x}}}$$

这里$\mu_x$是某个特定特征列的样本均值，$\sigma_x$为对应的标准方差。

下表说明了标准化和归一化两个常用的特征比例调整技术之间的区别，该表是由0到5所组成的简单示例数据集：

| 输入 | 标准化 | Min-max 归一化 | 输入 | 标准化 | Min-max 归一化 |
|---|---|---|---|---|---|
| 0.0 | −1.46 385 | 0.0 | 3.0 | 0.29 277 | 0.6 |
| 1.0 | −0.87 831 | 0.2 | 4.0 | 0.87 831 | 0.8 |
| 2.0 | −0.29 277 | 0.4 | 5.0 | 1.46 385 | 1.0 |

执行下面的代码示例可以完成表中数据的标准化和归一化：

```
>>> ex = np.array([0, 1, 2, 3, 4, 5])
>>> print('standardized:', (ex - ex.mean()) / ex.std())
standardized: [-1.46385011 -0.87831007 -0.29277002  0.29277002
0.87831007  1.46385011]
>>> print('normalized:', (ex - ex.min()) / (ex.max() - ex.min()))
normalized: [ 0.   0.2  0.4  0.6  0.8  1. ]
```

与MinMaxScaler类相似，scikit-learn也有标准化类：

```
>>> from sklearn.preprocessing import StandardScaler
>>> stdsc = StandardScaler()
>>> X_train_std = stdsc.fit_transform(X_train)
>>> X_test_std = stdsc.transform(X_test)
```

再次强调，在训练数据上用StandardScaler类拟合一次，然后再用这些参数来转换测试集或任何新的数据点。

## 4.5 选择有意义的特征

如果模型在训练集上表现得远比在测试集要好，那很有可能发生了过拟合。正如第3章中讨论的那样，过拟合意味着模型在拟合参数过程中对训练集某些特征的适应性过强，但却不能很好地概括新数据，因此模型的方差较大。

过拟合的原因是，与给定的训练数据相比，我们的模型太过复杂。减少泛化误差的常见解决方案如下：

·收集更多的训练数据

·通过正则化引入对复杂性的惩罚

·选择参数较少的简单模型

·减少数据的维数

收集更多训练数据的方法通常不适用。第6章将介绍一种有用的技术，来判断更多的训练数据是否有所帮助。下面的章节将学习通过特征选择来正则化和降维，进而减少过拟合的常用方法，采用需要较少参数的简单模型来拟合数据。

### 4.5.1 L1和L2正则化对模型复杂度的惩罚

回顾第3章，L2正则化是通过惩罚权重大的个体来降低模型复杂度的一种方法，在那里，权重向量w的L2范数定义如下：

$$L2:\|\boldsymbol{w}\|_2^2=\sum_{j=1}^{m}w_j^2$$

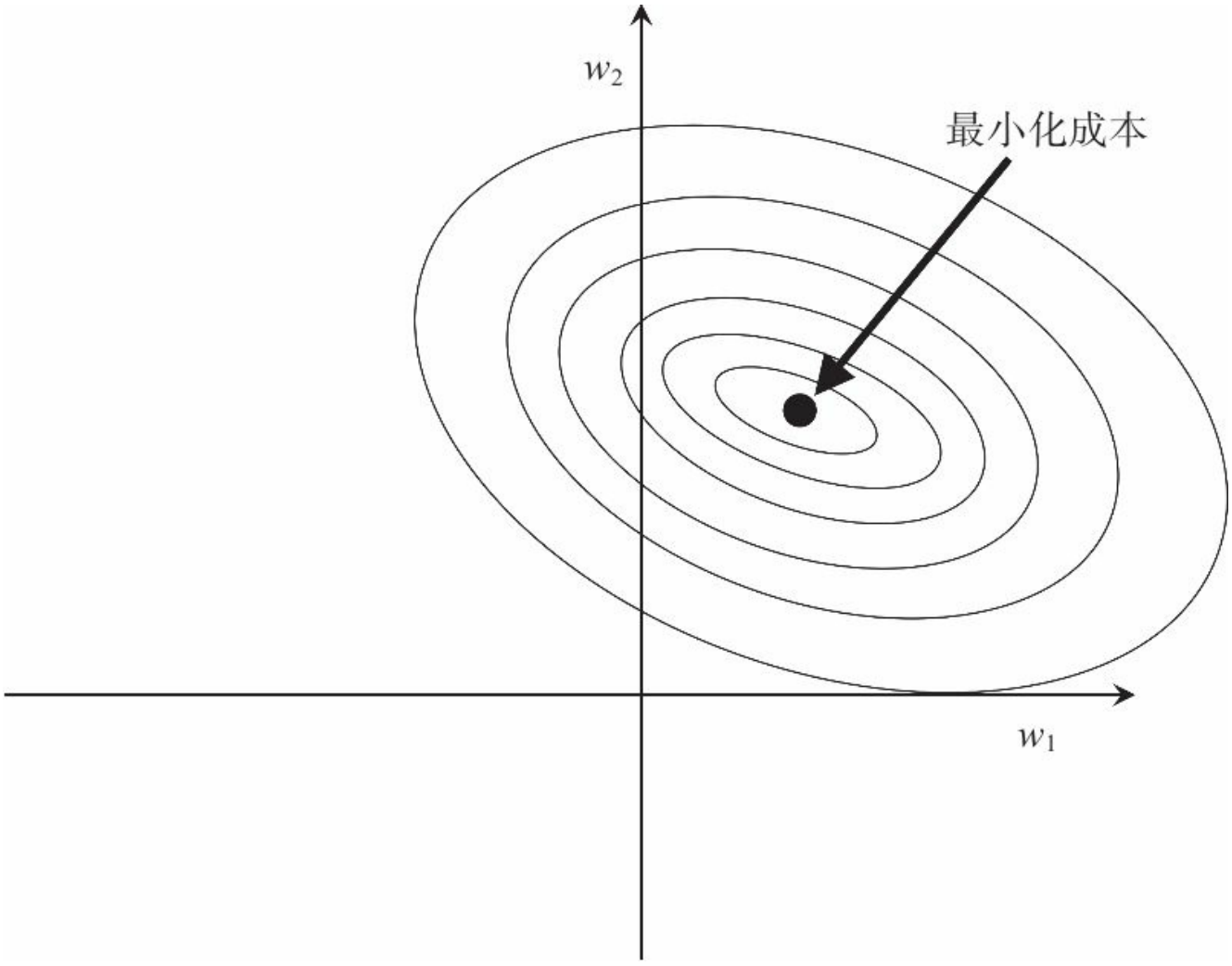
另一种降低模型复杂度的方法是L1正则化：

$$L1:\|\boldsymbol{w}\|_1=\sum_{j=1}^{m}\left|w_j\right|$$

这里只是用权重绝对值之和替代了权重平方之和。与L2正则化相比，L1正则化通常会产生大部分特征权重为0的稀疏特征向量。如果高维度数据集的样本包含许多不相关的特征，特别是在有更多不相关维度的情况下，稀疏性很有实用价值。从这个意义上说，L1正则化可以理解为一种特征选择技术。

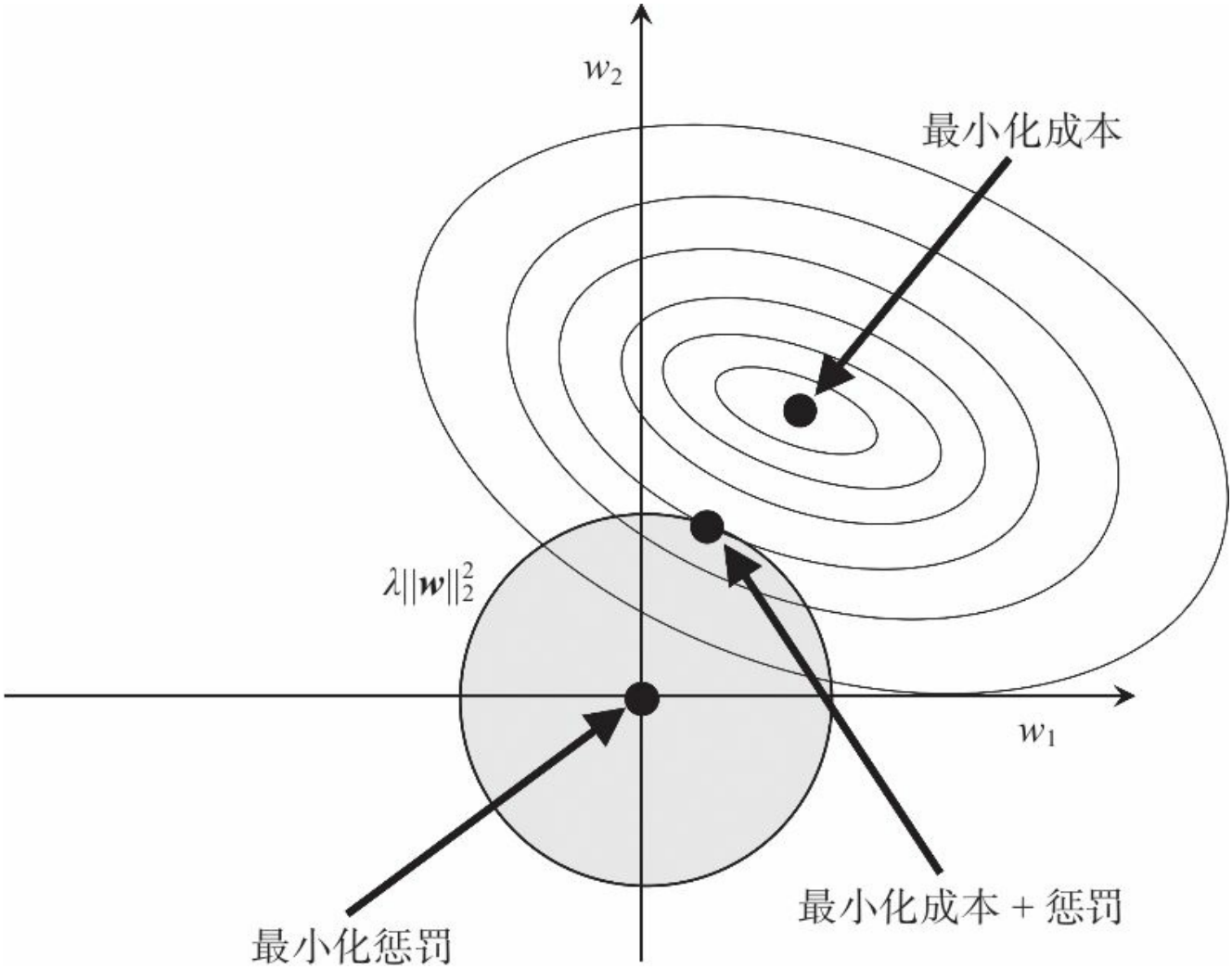
## 4.5.2 L2正则化的几何解释

如上节所述，L2正则化为代价函数增加了惩罚项，与未正则化的代价函数所训练的模型相比，L2还可以有效地抑制权重值的极端化。为了更好地理解L1正则化对稀疏性的促进作用，先理解正则化的几何解释。绘制权重为w1和w2的凸代价函数轮廓。这里用《训练简单的机器学习分类算法》中的Adaline，把误差平方和（SSE）作为代价函数，与逻辑回归相比，圆形的代价函数更容易绘制，同样的概念也适用于后者。记住，目标是寻找权重系数的组合，以最小化训练代价函数，如下图所示（椭圆中心点）：




现在，可以把正则化看作是在代价函数中加入惩罚项来促进较小的权重；换句话说，惩罚较大的权重。

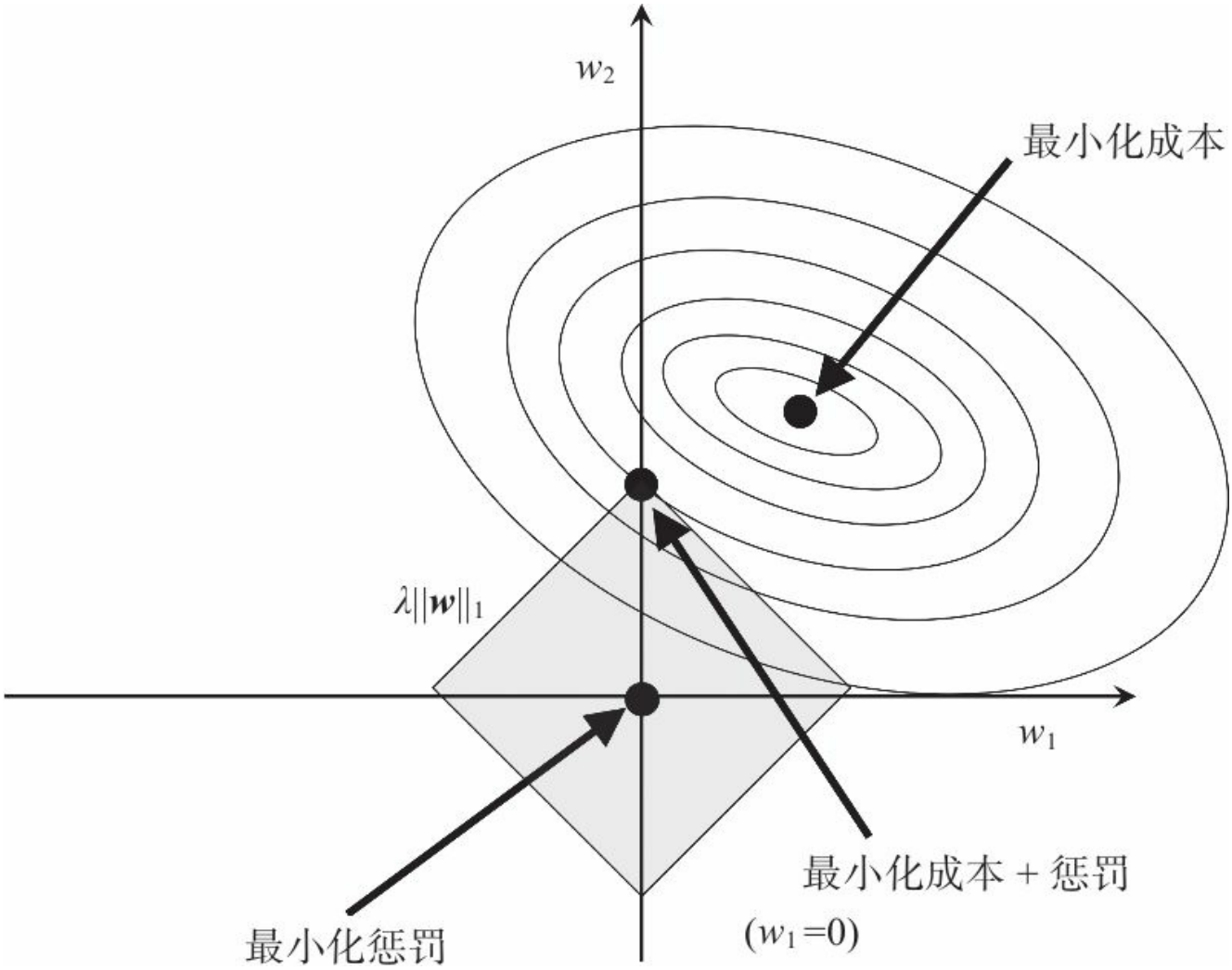
因此，通过正则化参数λ增大正则化的强度，我们将权重最小化使其趋向于零，降低了模型对训练数据的依赖性。下图说明了这个概念：




用圆形阴影来表示二次L2的正则项。权重系数不能超过正则化的预算——权重系数的组合不能落在阴影区域外。另外，仍然希望最小化代价函数。在惩罚的约束下，尽可能确定L2圆形与无惩罚代价函数轮廓的交叉点。λ正则化参数越大，惩罚成本的增速越快，导致L2圆形变窄。例如，正则参数趋于无穷大，则权重系数将快速变为0，即L2圆形的中心。总之，目标是最小化无惩罚成本与惩罚项的总和，可以理解为增加偏置和偏好简单模型，以最小化模型在缺乏足够训练数据拟合情况下的方差。

### 4.5.3 L1正则化的稀疏解决方案

现在讨论L1正则化和稀疏性。L1正则化的逻辑与前面讨论的类似。然而，由于L1惩罚是绝对权重系数的总和（记住L2是二次项），因此可以用菱形区域来表示，如下图所示：




上图可以看到代价函数的轮廓与L1的菱形在$w_1=0$处相交。由于L1正则化系统的轮廓是尖锐的，所以更可能的是代价函数的椭圆与L1菱形边界的交点位于轴上，从而促进了稀疏性。

关于L1正则化可能导致稀疏性的数学细节超出了本书的范围。如果有兴趣，可以在《统计学习的要素》的3.4节中发现对L2和L1正则化的极好解释，该书由特来沃尔·黑斯蒂，罗伯特·蒂施莱尼和杰罗姆·弗里德曼编纂，施普林格科学与商业媒体于2009年出版。

scikit-learn的正则化模型支持L1正则化，因此可以简单的设置参数

penalty为'l1'获得一个稀疏解决方案：

```
>>> from sklearn.linear_model import LogisticRegression
>>> LogisticRegression(penalty='l1')
```

应用到标准的葡萄酒数据，L1正则化的逻辑回归将产生以下的稀疏解决方案：

```
>>> lr = LogisticRegression(penalty='l1', C=1.0)
>>> lr.fit(X_train_std, y_train)
```

```
>>> print('Training accuracy:', lr.score(X_train_std, y_train))
Training accuracy: 1.0
>>> print('Test accuracy:', lr.score(X_test_std, y_test))
Test accuracy: 1.0
```

训练和测试准确度（100%）表明，如果模型在这两个数据集上表现得都很好，那么通过lr.intercept_访问截距项，可以看到阵列返回了三个值：

```
>>> lr.intercept_
array([-1.26338637, -1.21582071, -2.3701035 ])
```

在多元分类的数据集上拟合LogisticRegression对象，在默认情况下，采用一对多的方法，第一个值是可以分出类别1而不是类别2、3的拟合模型的截距，第二个值是可以分出类别2而不是类别1、3的拟合模型的截距，第三个值是可以分出类别3而不是类别1、2的拟合模型的截距：

```
>>> lr.coef_
array([[ 1.24559337, 0.18041967, 0.74328894, -1.16046277, 0. ,
         0., 1.1678711, 0., 0., 0., 0., 0.54941931, 2.51017406],
       [-1.53720749, -0.38727002, -0.99539203, 0.3651479,
         -0.0596352 , 0., 0.66833149, 0., 0., -1.9346134,
         1.23297955, 0., -2.23135027],
       [ 0.13579227, 0.16837686, 0.35723831, 0., 0., 0.,
         -2.43809275, 0., 0., 1.56391408, -0.81933286,
         -0.49187817, 0.]])
```

访问lr.coef_的属性所获得的权重阵列包含三行权重系数，每行为一组权重向量。每行由13个权重组成，每个权重乘以葡萄酒数据集13个维度中的每个特征来计算净输入：

$$z = w_0 x_0 + \cdots + w_m x_m = \sum_{j=0}^{m} x_j w_j = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x}$$

在scikit-learn中，$w_0$对应intercept_，当j＞0时，$w_j$对应coef_中的值。

正则化是特征选择的一种方法，L1正则化的结果训练了对数据集中潜在的不相关特征处理能力强的模型。

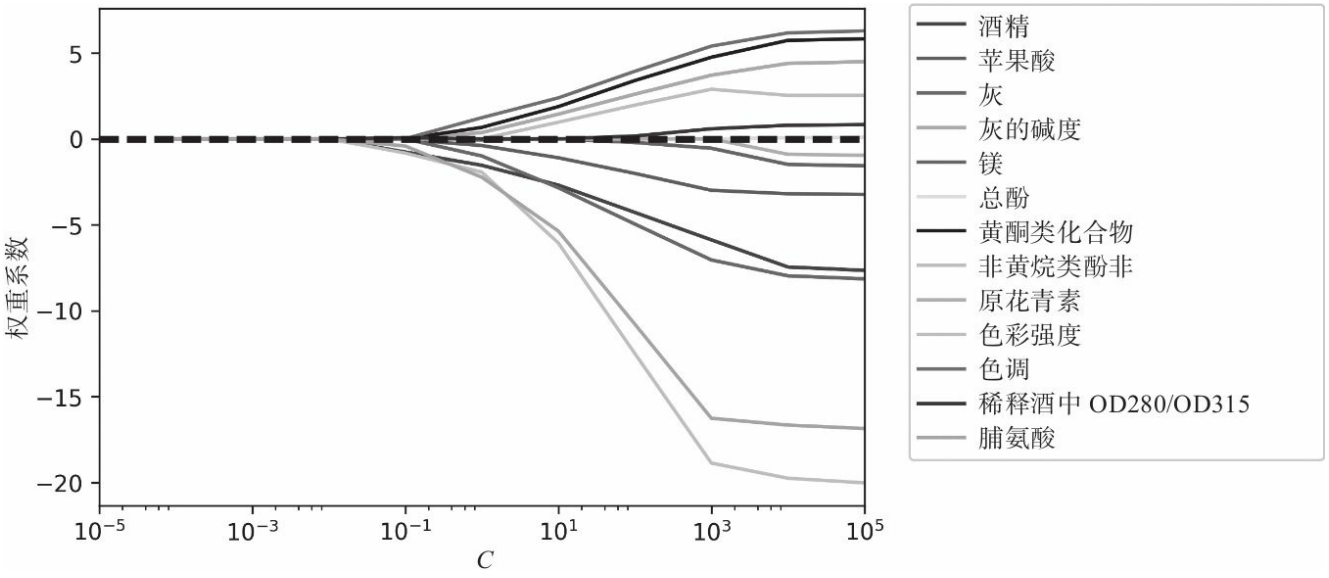
严格地说，前面例子的权重向量不一定是稀疏的，因为它们所包含的非零项更多。然而，可以通过进一步加强正则化来减少稀疏性（更多的零项），即选择较低的参数C值。

本章的最后一个正则化示例将改变正则化强度并测算正则化路径，即不同特征在不同正则化强度下的权重系数：

```
>>> import matplotlib.pyplot as plt

>>> fig = plt.figure()
>>> ax = plt.subplot(111)

>>> colors = ['blue', 'green', 'red', 'cyan',
...           'magenta', 'yellow', 'black',
...           'pink', 'lightgreen', 'lightblue',
...           'gray', 'indigo', 'orange']
>>> weights, params = [], []
>>> for c in np.arange(-4., 6.):

...     lr = LogisticRegression(penalty='l1',
...                             C=10.**c,
...                             random_state=0)
...     lr.fit(X_train_std, y_train)
...     weights.append(lr.coef_[1])
...     params.append(10**c)

>>> weights = np.array(weights)

>>> for column, color in zip(range(weights.shape[1]), colors):
...     plt.plot(params, weights[:, column],
...              label=df_wine.columns[column + 1],
...              color=color)
>>> plt.axhline(0, color='black', linestyle='--', linewidth=3)
>>> plt.xlim([10**(-5), 10**5])
>>> plt.ylabel('weight coefficient')
>>> plt.xlabel('C')
>>> plt.xscale('log')
>>> plt.legend(loc='upper left')
>>> ax.legend(loc='upper center',
...           bbox_to_anchor=(1.38, 1.03),
...           ncol=1, fancybox=True)
>>> plt.show()
```

由此而产生的路径图为了解L1正则化的行为提供了进一步的认识。从中可以看到，如果定义一个正则化参数（C＜0.1）强大的惩罚模型，那么所

有的特征权重将为零。C是正则化参数λ的逆：



## 4.5.4 为序数特征选择算法

另外一种降低模型复杂度以及避免过拟合的方法是通过特征选择降维，这对未正则化的模型特别有用。主要有两类降维技术：特征选择和特征提取。通过特征选择可以从原始特征中选择子集，特征提取则是从特征集中提取信息以构造新的特征子空间。

这部分将看到经典的特征选择算法系列。第5章将讨论不同的特征提取技术以把数据集压缩到低维度的特征子空间。

顺序特征选择算法属于贪婪搜索算法，用于把初始的d维特征空间降低到k维特征子空间（k＜d）。特征选择算法的逻辑是自动选择与问题最相关的特征子集，提高模型的计算效率或者通过去除无关特征消除噪声降低模型的泛化误差，这对不支持正则化的算法有效。

经典的序数特征选择算法是逆顺序选择（SBS），其目的是应对分类器性能最小的衰减来降低初始特征子空间的维数，从而提高计算效率。在某些情况下，SBS甚至可以提高模型的预测能力，即使出现了过拟合的情况。

贪婪算法在组合搜索问题的每个阶段都做出局部最优选择，通常会产生次优解决方案，与穷举搜索算法相比，该算法评估所有可能的组合，并保证找到最优解。然而，在实践中，穷举搜索往往在计算上不可行，而贪婪算法允许不太复杂，计算更有效的解决方案。

SBS算法的逻辑非常简单：它顺序地从完整的特征子集中移除特征，直到新特征子空间包含需要的特征数量。为了确定每个阶段要删除哪个特征，需要定义期待最小化的标准函数J。标准函数计算的标准值可以简单地定义为分类器在去除特定特征前后的性能差异。每个阶段要删除的特征可以定义为该标准值最大的特征。或者更直观地说，每个阶段去除因特征去除性能损失最小的那个特征。基于前面SBS的定义，可以总结出四个简单的步骤来描述该算法：

1.用k=d初始化算法，d为特征空间$X_d$的维数；

2.确定使标准值最大的特征$x^-$，即$x^-=\text{argmax}\, J(X_k-x)$），其中$x\in X_k$；

3.从特征集中去除特征$x^-$：$X_{k-1}=X_k-x^-$；k=k-1。

4.如果k等于期望的特征数则停止；否则转往步骤2。

参考F.费里、P.普迪尔、M.哈代夫、J.克特拉1994年发表在《对于大尺度特征选择技术的比较研究》403-413页上的《几种顺序特征算法的详细评价》。

不幸的是scikit-learn尚未实现SBS算法。但它很简单，下面将用Python从头实现：

```
from sklearn.base import clone
from itertools import combinations
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split


class SBS():
    def __init__(self, estimator, k_features,
                 scoring=accuracy_score,
                 test_size=0.25, random_state=1):
        self.scoring = scoring
        self.estimator = clone(estimator)
        self.k_features = k_features
        self.test size = test size
    self.random_state = random_state

def fit(self, X, y):

    X_train, X_test, y_train, y_test = \
```

```
            train_test_split(X, y, test_size=self.test_size,
                             random_state=self.random_state)

        dim = X_train.shape[1]
        self.indices_ = tuple(range(dim))
        self.subsets_ = [self.indices_]
        score = self._calc_score(X_train, y_train,
                                 X_test, y_test, self.indices_)
        self.scores_ = [score]

        while dim > self.k_features:
            scores = []
            subsets = []

            for p in combinations(self.indices_, r=dim - 1):
                score = self._calc_score(X_train, y_train,
                                         X_test, y_test, p)
                scores.append(score)
                subsets.append(p)

            best = np.argmax(scores)
            self.indices_ = subsets[best]
            self.subsets_.append(self.indices_)
            dim -= 1

            self.scores_.append(scores[best])
        self.k_score_ = self.scores_[-1]

        return self

    def transform(self, X):
        return X[:, self.indices_]

    def _calc_score(self, X_train, y_train, X_test, y_test,
                    indices):
        self.estimator.fit(X_train[:, indices], y_train)
        y_pred = self.estimator.predict(X_test[:, indices])
        score = self.scoring(y_test, y_pred)
        return score
```

前面的实现过程定义了参数k_features，指定了想要返回的理想特征数。在默认情况下，调用scikit-learn的accuracy_score对模型在特征子空间的性能（分类评估器）进行评估。在fit方法的while循环中，对由itertools.combination函数创建的特征子集进行评估，并逐渐减少特征直至特征子集的维度满足需要。在每次迭代中，基于内部创建的测试数据X_test，收集每个子集中最好的准确性得分并存储在列表self.scores_。稍后将用这些分数来评估结果。最终把特征子集的列号存储在self.indices_，可以用它调用transform方法返回包括选定特征列在内的新数据阵列。注意，fit方法并未具体计算判断标准，而是简单地去除了在最好子集以外的特征。

现在，看看使用scikit-learn的KNN分类器实现的SBS的实例：

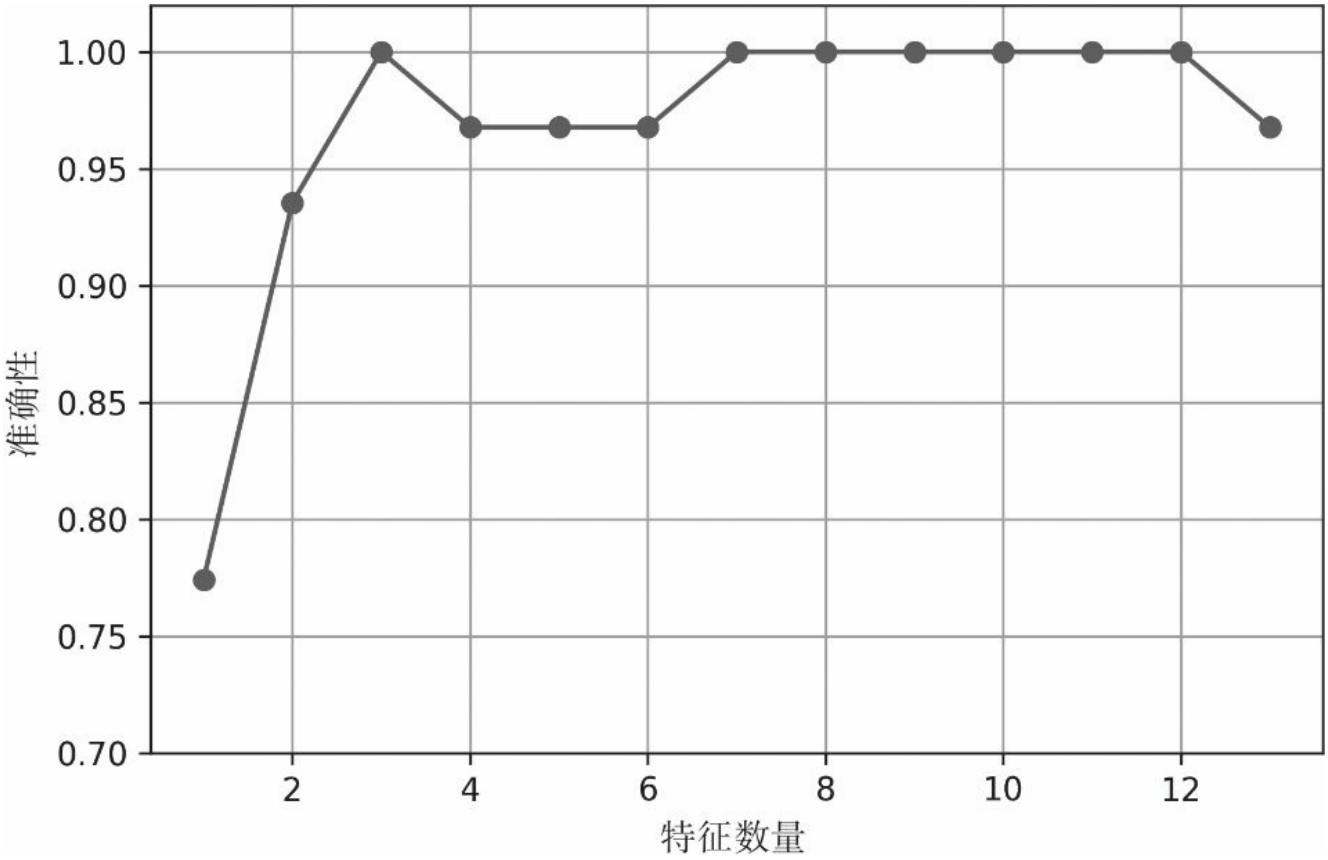
```
>>> import matplotlib.pyplot as plt
>>> from sklearn.neighbors import KNeighborsClassifier

 >>> knn = KNeighborsClassifier(n_neighbors=5)

 >>> sbs = SBS(knn, k_features=1)
 >>> sbs.fit(X_train_std, y_train)
```

虽然SBS代码已经用fit函数将数据集分裂成测试集和训练集，但是仍然为训练集X_train的算法提供数据。SBS的fit方法将为测试（验证）和训练创建新子集，这就是为什么该测试集也被称为验证集。这种方法对防止原来的测试集成为训练数据的一部分很有用。

请记住，SBS算法收集每个阶段最佳子集的特征评分。现在开始更令人兴奋的部分，绘制在验证数据集上计算的KNN分类器的分类准确度。代码如下：

```
>>> k_feat = [len(k) for k in sbs.subsets_]

>>> plt.plot(k_feat, sbs.scores_, marker='o')
>>> plt.ylim([0.7, 1.02])
>>> plt.ylabel('Accuracy')
>>> plt.xlabel('Number of features')
>>> plt.grid()
>>> plt.show()
```

从下图可以看到，可能是由于降低了维数诅咒，KNN分类器通过减少特征数提高了在测试集上的预测准确度，这在第3章介绍KNN算法时曾经讨论过。此外，还可以看到当分类器的准确度达到100%时，特征数分别为k={3，7，8，9，10，11，12}：




为了满足好奇心，让我们来看看在验证数据集上产生如此良好性能的最小特征子集（k=3）是究竟是个什么样子：

```
>>> k3 = list(sbs.subsets_[10])
>>> print(df_wine.columns[1:][k3])
Index(['Alcohol', 'Malic acid', 'OD280/OD315 of diluted wines'],
dtype='object')
```

用前面的代码可以得到存储在sbs.subsets_属性第10位的三个特征子集的列号，以及从葡萄酒数据pandas的DataFrame的列索引返回的相应特征名。

接着评估该KNN分类器在原始测试集上的性能：

```
>>> knn.fit(X_train_std, y_train)
>>> print('Training accuracy:', knn.score(X_train_std, y_train))
Training accuracy: 0.967741935484
>>> print('Test accuracy:', knn.score(X_test_std, y_test))
Test accuracy: 0.962962962963
```

前面的代码用完整的特征集在训练集上取得大约97%的准确率，在测试集上获得大约96%的准确率，这表明模型已经可以很好地泛化到新数据。现在用选定的三个特征子集来看下KNN的效果：

```
>>> knn.fit(X_train_std[:, k3], y_train)
>>> print('Training accuracy:',
...       knn.score(X_train_std[:, k3], y_train))
Training accuracy: 0.951612903226
>>> print('Test accuracy:',
...       knn.score(X_test_std[:, k3], y_test))
Test accuracy: 0.925925925926
```

如果采用葡萄酒数据集少于四分之一的原始特征，测试集的预测准确度略有下降。这可能表明三个特征所提供的差异信息并不比原始数据集少。然而，必须记住，葡萄酒数据集是一个小数据集，非常容易受到随机性的影响，即如何将数据集分成训练集和测试集，以及如何进一步将训练集分裂为训练集和验证集。

虽然减少特征数量并没有提高KNN模型的性能，但缩小了数据集的规模，在实际应用中可能会在涉及昂贵的数据收集的情况下非常有用。此外，通过大量减少特征数量可以得到更简单的模型，这些模型也更容易解释。

用scikit-learn实现特征选择算法

scikit-learn有更多的特征选择算法。包括基于特征权重的递归逆向消除法，基于树根据重要性选择特征的方法以及单变量统计检验方法。对不同特征选择方法的全面讨论超出了本书的范围，但可以从下述网站找到得优秀的总结与例证

http://scikit-learn.org/stable/modules/feature_selection.html

此外，我实现了几种不同的序数特征选择方法，这与前面实现的简单SBS相关。可以从下述网站找到这些Python实现的mlxtend软件包。

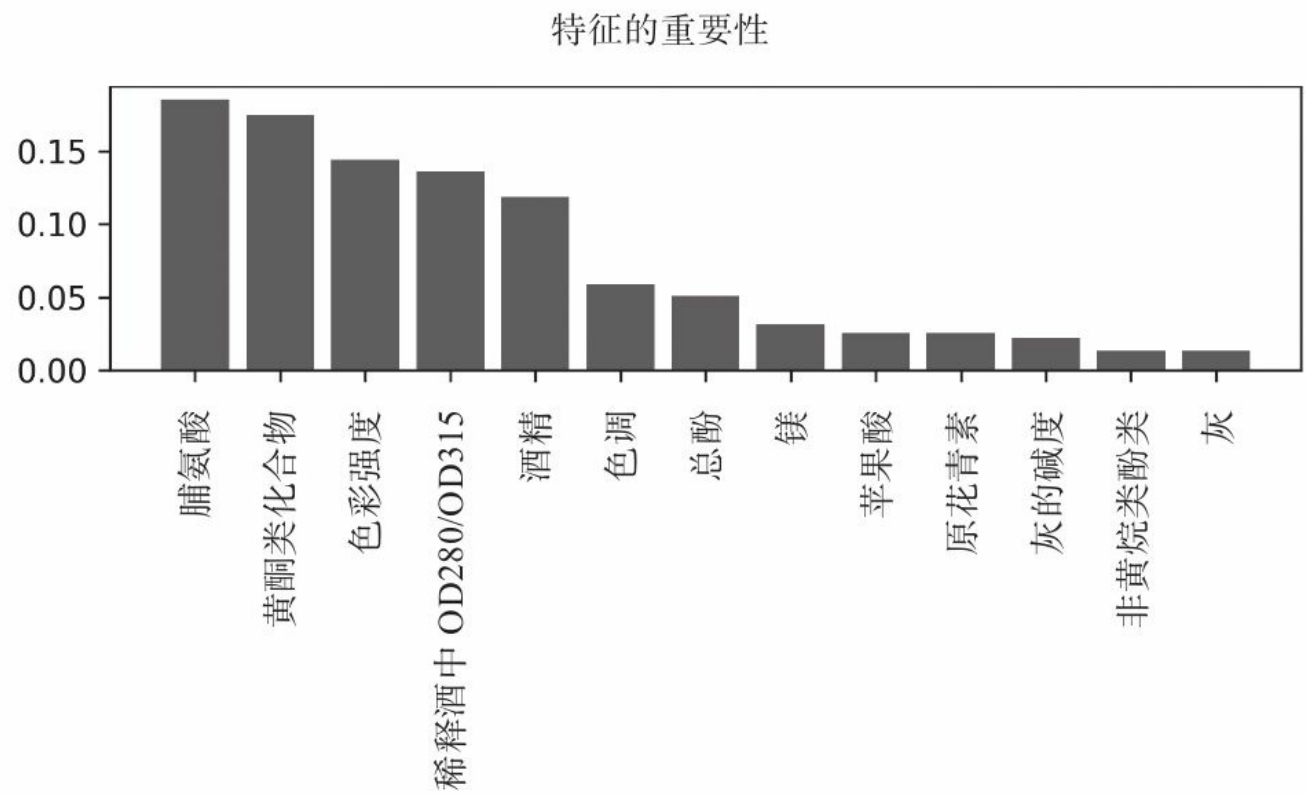
http://rasbt.github.io/mlxtend/user_guide/feature_selection/SequentialFeatureS

## 4.6 用随机森林评估特征的重要性

前面的章节学习了如何通过逻辑回归用L1正则化来消除不相关的特征，用SBS算法进行特征选择，并将其应用到KNN算法。另一个用来从数据集中选择相关特征的有用方法是随机森林，即在第3章《scikit-learn机器学习分类器一览》中介绍的集成技术。可以用随机森林根据森林中所有决策树计算平均不纯度的减少来测量特征的重要性，而不作任何数据是线性可分或不可分的假设。用scikit-learn实现随机森林非常方便，它已经收集好了特征的重要性值，可以在完成RandomForestClassifier拟合后，通过访问feature_importances_的属性取得。下面的代码将在葡萄酒数据集上训练拥有10 000棵树的森林，并根据13个特征各自的重要性为其排序，在第3章中讨论过，基于树的模型并不需要使用标准或归一化的特征：

```
>>> from sklearn.ensemble import RandomForestClassifier

>>> feat_labels = df_wine.columns[1:]

>>> forest = RandomForestClassifier(n_estimators=500,
...                                 random_state=1)
>>> forest.fit(X_train, y_train)
>>> importances = forest.feature_importances_

>>> indices = np.argsort(importances)[::-1]

>>> for f in range(X_train.shape[1]):
...     print("%2d) %-*s %f" % (f + 1, 30,
...                             feat_labels[indices[f]],
...                             importances[indices[f]]))
>>> plt.title('Feature Importance')
>>> plt.bar(range(X_train.shape[1]),
...         importances[indices],
...         align='center')

>>> plt.xticks(range(X_train.shape[1]),
...            feat_labels, rotation=90)
>>> plt.xlim([-1, X_train.shape[1]])
>>> plt.tight_layout()
>>> plt.show()

 1) Proline                        0.185453
 2) Flavanoids                     0.174751
 3) Color intensity                0.143920
 4) OD280/OD315 of diluted wines   0.136162
 5) Alcohol                        0.118529
 6) Hue                            0.058739
 7) Total phenols                  0.050872
 8) Magnesium                      0.031357
 9) Malic acid                     0.025648
10) Proanthocyanins                0.025570
11) Alcalinity of ash              0.022366
12) Nonflavanoid phenols           0.013354
13) Ash                            0.013279
```

执行代码后可以画出一张图，把葡萄酒数据集中不同的特征按其相对重要性进行排序。请注意，特征重要性值被正常化所以总和为1：




可以得出这样的结论：脯氨酸和黄酮的含量、颜色强度、OD280/OD315衍射和酒精浓度是数据集中基于500棵决策树根据平均不纯度减少而确定的最具差异性的特征。有趣的是，结果中排在前两位的特征也包括根据前一节实现的SBS算法得出的三个特征子集（乙醇浓度和稀释葡萄酒的OD280/OD315）。然而，就可解释性而言，随机森林是值得一提的重要技术。如果两个或多个特征高度相关，一个特征可能排得很靠前，而另一个可能无法完全捕获。另一方面，如果所关心的只是模型的预测性能，而不是对特征重要值的解释，那就不需要关心这个问题了。

为了总结特征的重要值和随机森林，值得一提的是scikit-learn也实现了Select-FromModel对象，可以在模型拟合后，根据用户指定的阈值选择特征，如果想用RandomForestClassifier作为特征选择器以及scikit-learn的Pipeline对象的中间步骤，这将会很有用，它允许连接不同的有评估的预处理步骤，第6章将详细介绍。例如，可以在以下代码中，通过将threshold设置为0.1把数据集减少到只包含五个最重要的特征：

```
>>> from sklearn.feature_selection import SelectFromModel

>>> sfm = SelectFromModel(forest, threshold=0.1, prefit=True)
>>> X_selected = sfm.transform(X_train)
>>> print('Number of samples that meet this criterion:',
...       X_selected.shape[0])
Number of samples that meet this criterion: 124

>>> for f in range(X_selected.shape[1]):
...     print("%2d) %-*s %f" % (f + 1, 30,
...                             feat_labels[indices[f]],
...                             importances[indices[f]]))
1) Proline                        0.185453
2) Flavanoids                     0.174751
3) Color intensity                0.143920
4) OD280/OD315 of diluted wines   0.136162
5) Alcohol                        0.118529
```

## 4.7 小结

本章从寻找能确保正确地处理缺失数据的有用技术开始。在将数据输入机器学习算法之前，必须确保对分类变量进行正确的编码，并且已经看到了如何将序数特征和名词特征值映射成整数来表示。

此外，简要地介绍了L1正则化，它可以通过降低模型的复杂性来避免过拟合。用序数特征选择算法从数据集中选择有意义的特征以去除不相关的特征。

下一章将了解另一种有用的降维方法：特征提取。它可以将特征压缩到较低维度的子空间，而不像特征选择那样要完全去除特征。

# 第5章　通过降维压缩数据

第4章介绍了采用不同的特征选择技术降低数据集维度的各种方法。特征选择降维的另一种方法是特征提取。本章将学习三种基本技术，有助于将数据集的信息内容概括为新的低维特征子空间，而不是原始数据。压缩数据是机器学习的一个重要课题，有助于存储和分析在现代科技时代所产生和收集的大量数据。

本章将主要涵盖下述几个方面：

·无监督数据压缩的主成分分析（PCA）

·以线性判别分析（LDA）为最大化可分性的有监督降维技术

·利用核主成分分析（KPCA）进行非线性降维

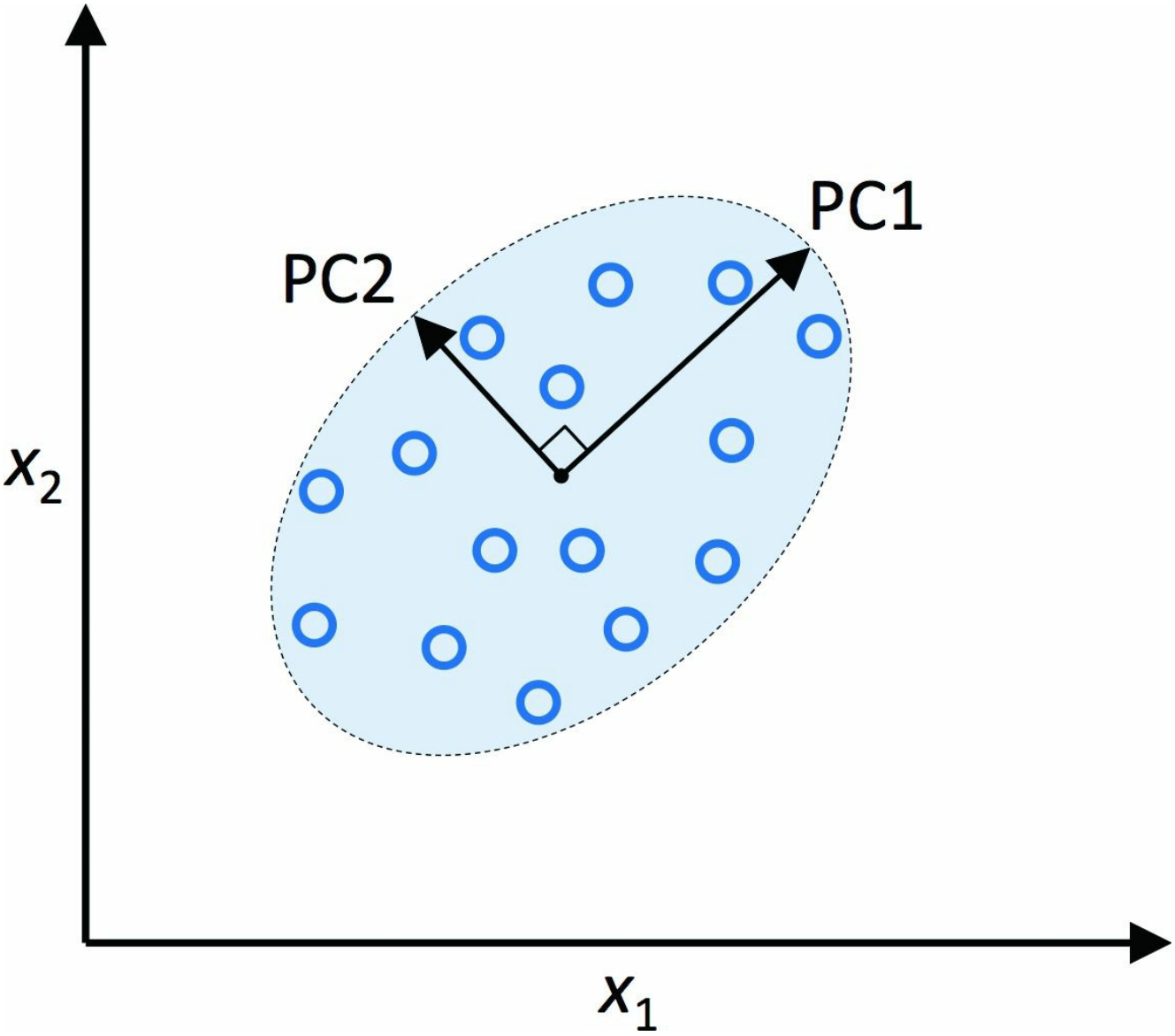
## 5.1 用主成分分析实现无监督降维

类似于特征选择，特征提取可以用不同的技术来减少数据集特征的数量。特征选择和特征提取的区别在于，特征选择算法（如序数逆选择）保持原始特征，特征提取将数据转换或投影到新的特征空间。在降维的背景下，可以把特征提取理解为数据压缩方法，其目的是维护大部分的相关信息。在实际应用中，特征提取不仅可以优化机器学习算法的存储空间或计算效率，而且还可以通过减少维数提高预测性能，尤其是对非正则模型。

### 5.1.1 主成分分析的主要步骤

这部分将讨论PCA，一种无监督的线性变换技术，该技术广泛地应用于不同领域，特别是特征提取和降维。PCA的其他应用包括股票市场交易探索性数据分析和去噪，以及生物信息学基因组数据和基因表达水平分析。

PCA根据特征之间的相关性来识别数据中的模式。简单地说，PCA旨在寻找高维数据中存在最大方差的方向，并将其投影到维数等于或小于原始数据的新子空间。假设新特征轴彼此正交，该空间的正交轴（主成分）可以解释为最大方差的方向，其中$x_1$和$x_2$为原始特征轴，而PC1和PC2为主成分方向，如右图所示。




如果用PCA降维，可以构建d×k维变换矩阵W，把样本向量x映射到新的k维特征子空间，该空间的维数比原来的d维特征空间少：

$$\boldsymbol{x} = \left[x_1, x_2, \cdots x_d\right], \boldsymbol{x} \in \mathbb{R}^d$$
$$\downarrow \boldsymbol{xW}, \boldsymbol{W} \in \mathbb{R}^{d \times k}$$
$$\mathbf{z} = \left[z_1, z_2, \cdots z_k\right], \mathbf{z} \in \mathbb{R}^k$$

在高维原数据转换到k维新子空间（通常是k<<d）的结果中，第一主成分有最大的方差，所有随之而来的主成分都有最大的方差，由此而产生的主成分相互正交（相关），约束条件是这些成分与主成分之间互不相关（即使输入的特征是相关的结果也一样）。注意，PCA方向对数据的比例尺度非常敏感，需要在PCA之前对特征进行标准化，如果特征值以不同的比例尺度测量，则需要确保所有特征的重要性保持均衡。

在深入讨论PCA降维算法之前，先用几个简单的步骤来概括该方法：

1.标准化d维数据集。

2.构建协方差矩阵。

3.将协方差矩阵分解为特征向量和特征值。

4.通过降阶对特征值排序，对相应的特征向量排序。

5.选择对应k个最大特征值的k个特征向量，其中k为新子空间的维数（k≤d）。

6.从最上面的k特征向量开始构造投影矩阵W。

7.用投影矩阵W变换d维输入数据集X获得新的k维特征子空间。

下面将用Python逐步完成一个PCA实例。然后展示如何方便地用scikit-learn实现PCA。

### 5.1.2 逐步提取主成分

本节将讨论PCA的前四个步骤：

1.标准化数据集。

2.构建协方差矩阵。

3.获取协方差矩阵特征值和特征向量。

4.以降序对特征值排序，从而对特征向量排序。

首先从加载一直在用的葡萄酒数据集开始（见第4章）：

```
>>> import pandas as pd
df_wine = pd.read_csv('https://archive.ics.uci.edu/ml/'
                      'machine-learning-databases/wine/wine.data',
                      header=None)
```

可以从本书的代码包找到葡萄酒数据集（和本书用到的其他全部数据集），当脱机工作或UCI的服务器暂时宕机时，可以从下面的网站获得：

https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data

要想从本地目录加载葡萄酒数据集，可以从执行下面的命令：

```
df = pd.read_csv('https://archive.ics.uci.edu/ml/'
        'machine-learning-databases/wine/wine.data',
         header=None)
```

换成执行下述命令：

```
df = pd.read_csv('your/local/path/to/wine.data',
                 header=None)
```

下一步将葡萄酒数据按7：3分裂为独立的训练集和测试集，并标准化单位方差：

```
>>> from sklearn.model_selection import train_test_split
>>> X, y = df_wine.iloc[:, 1:].values, df_wine.iloc[:, 0].values
>>> X_train, X_test, y_train, y_test = \
>>>     train_test_split(X, y, test_size=0.3,
...                      stratify=y,
...                      random_state=0)
>>> # standardize the features
>>> from sklearn.preprocessing import StandardScaler
>>> sc = StandardScaler()
>>> X_train_std = sc.fit_transform(X_train)
>>> X_test_std = sc.transform(X_test)
```

执行完前面的代码完成必要的预处理后，进行第二步：构造协方差矩阵。d×d维协方差对称矩阵，其中d为数据集维数，把不同特征之间的协方差对应存储起来。例如：

可以通过以下的方程计算特征$x_j$和$x_k$的整体协方差：

$$\sigma_{jk} = \frac{1}{n}\sum_{i=1}^{n}\left(x_j^{(i)} - \mu_j\right)\left(x_k^{(i)} - \mu_k\right)$$

其中$\mu_j$和$\mu_k$分别为特征j和k的样本均值。注意，如果把数据集标准化，那么样本的均值为零。两个样本之间的正协方差表示特征值以相同方向增加或者减少，而负协方差表示特征在相反的方向上变化。例如，三个特征的协方差矩阵可以写成（请注意∑是希腊字母sigma的大写形式，不要和求和的符号混淆）：

$$\sum = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_2^2 & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_3^2 \end{bmatrix}$$

协方差矩阵的特征向量表示主成分（最大方差的方向），而相应的特征值将定义它们的大小。我们将从葡萄酒数据集13×13维协方差矩阵中获得13个特征向量和特征值。

第三步，得到协方差矩阵的特征值和特征向量。正如前面介绍的线性代数类，特征向量v满足以下条件：

$$\sum \boldsymbol{v} = \lambda \boldsymbol{v}$$

在这里，λ是标量，即特征值。由于特征向量和特征值的手工计算很烦琐，将调用NumPy的linalg.eig函数获得葡萄酒协方差矩阵的特征向量和特征值：

```
>>> import numpy as np
>>> cov_mat = np.cov(X_train_std.T)
>>> eigen_vals, eigen_vecs = np.linalg.eig(cov_mat)
>>> print('\nEigenvalues \n%s' % eigen_vals)
Eigenvalues
[ 4.84274532  2.41602459  1.54845825  0.96120438  0.84166161
0.6620634     0.51828472  0.34650377  0.3131368   0.10754642
0.21357215  0.15362835    0.1808613 ]
```

用numpy.cov函数计算标准化训练数据的协方差矩阵。用linalg.eig函数完成特征值分解，产生含有13个特征值的向量（eigen_vals），对应的特征向量组成存储在13×13维矩阵的列（eigen_vecs）中。

numpy.linalg.eig函数能处理对称和非对称的方阵操作。但是，可能会发现它在某些情况下返回复特征值。与之相关的一个函数numpy.linalg.eigh可以分解赫米特矩阵，从数值角度来说，这是解决对称矩阵更稳定的方法，例如协方差矩阵。该方法可以确保numpy.linalg.eigh始终能返回真值。

### 5.1.3 总方差和解释方差

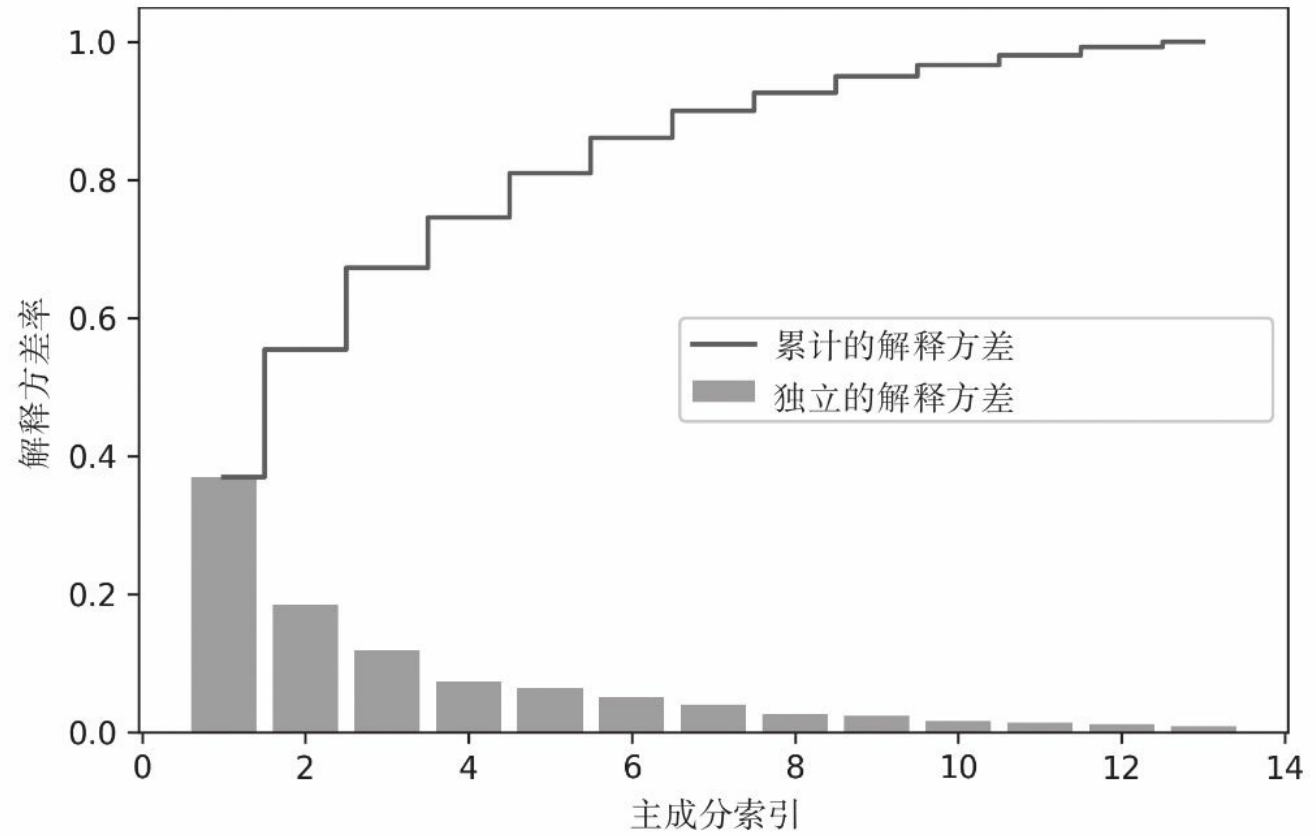
因为想通过压缩数据集到新的特征子空间降低维数，所以只选择包含主成分信息（方差）和特征向量（主成分）的子集。特征值代表特征向量的大小，通过对特征值降序排列找出前k个最重要的特征向量。但是在收集k个信息最丰富的特征向量之前，先把特征值的方差解释比画出来。特征值$\lambda_j$的方差解释比就是特征值$\lambda_j$与特征值总和之比：

$$\frac{\lambda_j}{\sum_{j=1}^{d}\lambda_j}$$

调用NumPy的cumsum函数可以计算出解释方差和，然后用Matplotlib的step函数绘图：

```
>>> tot = sum(eigen_vals)
>>> var_exp = [(i / tot) for i in
...            sorted(eigen_vals, reverse=True)]
>>> cum_var_exp = np.cumsum(var_exp)
>>> import matplotlib.pyplot as plt
>>> plt.bar(range(1,14), var_exp, alpha=0.5, align='center',
...         label='individual explained variance')
>>> plt.step(range(1,14), cum_var_exp, where='mid',
...         label='cumulative explained variance')
>>> plt.ylabel('Explained variance ratio')
>>> plt.xlabel('Principal component index')
>>> plt.legend(loc='best')
>>> plt.show()
```

结果表明第一主成分仅占方差的40%左右。此外，还可以看到前两个主成分结合起来可以解释数据集中几乎60%的方差（见下图）：




虽然解释方差图让我们回想起在第4章中通过随机森林计算的重要特征值，但是要注意，PCA是一种无监督学习方法，这意味着有关的分类标签信息会被忽略。随机森林用类成员信息计算节点的不纯度，方差测量沿特征轴传播的值。

### 5.1.4 特征变换

在成功地把协方差矩阵分解为特征对之后，现在接着完成最后的三个步骤，将葡萄酒数据集变换到新的主成分轴。本节将讨论其余的步骤：

·选择与前k个特征值对应的特征向量，其中k为新特征子空间的维数（k≤d）。

·用前k个特征向量构造投影矩阵W。

·用投影矩阵W变换d维输入数据集X以获得新的k维特征子空间。

通俗地说，把特征对按特征值降序排列，投影矩阵特征值，利用所选的特征向量构建投影矩阵，用投影矩阵把数据变换到低维子空间。

从把特征向量按特征值降序排列开始：

```
>>> # Make a list of (eigenvalue, eigenvector) tuples
>>> eigen_pairs = [(np.abs(eigen_vals[i]), eigen_vecs[:, i])
...                for i in range(len(eigen_vals))]
>>> # Sort the (eigenvalue, eigenvector) tuples from high to low
>>> eigen_pairs.sort(key=lambda k: k[0], reverse=True)
```

接着收集对应于前两个最大特征值的特征向量，捕获数据集中约60%的方差。注意，这里只选择了两个特征向量来说明问题，而且将在本节后的二维散点图中绘制数据。在实践中，主成分的数量必须通过在计算效率和分类器性能之间的平衡来确定：

```
>>> w = np.hstack((eigen_pairs[0][1][:, np.newaxis],
...                eigen_pairs[1][1][:, np.newaxis]))
>>> print('Matrix W:\n', w)
Matrix W:
 [[-0.13724218  0.50303478]
 [ 0.24724326  0.16487119]
 [-0.02545159  0.24456476]
 [ 0.20694508 -0.11352904]
 [-0.15436582  0.28974518]
 [-0.39376952  0.05080104]
 [-0.41735106 -0.02287338]
 [ 0.30572896  0.09048885]
 [-0.30668347  0.00835233]
 [ 0.07554066  0.54977581]
 [-0.32613263 -0.20716433]
 [-0.36861022 -0.24902536]
 [-0.29669651  0.38022942]]
```

执行代码依据前两个特征向量创建了一个13×2维的投影矩阵W。

取决于NumPy和LAPACK的具体版本，得到的矩阵W，其符号可能相反。请注意这并不是个问题。如果v是一个矩阵向量∑，那么：

$$\sum v=\lambda v$$

在这里λ为特征值，而-λ为具有同样特征值的特征向量：

$$\sum \cdot (-v)=-v\sum =-\lambda v=\lambda \cdot (-v)$$

现在可以用投影矩阵将样本x（表示为1×13维的行向量）转换到PCA子空间（主成分1和2）从而获得x′，即由两个新特征组成的二维样本向量：

$$x'=xW$$

```
>>> X_train_std[0].dot(w)
array([ 2.38299011,  0.45458499])
```

同样，可以通过计算矩阵点积将整个124×13维的训练集转换成两个主成分：

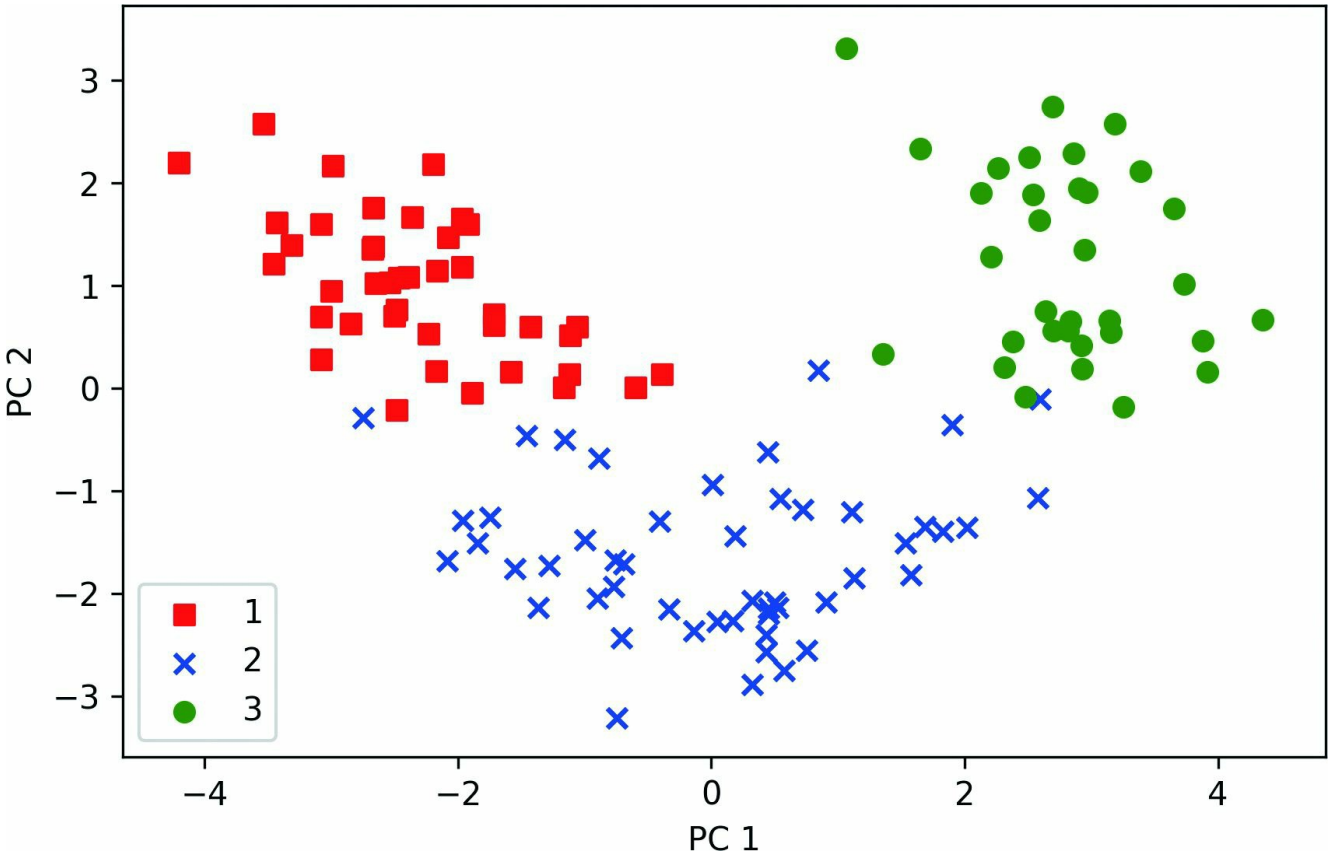
$$X'=XW$$

```
>>> X_train_pca = X_train_std.dot(w)
```

最后在二维散点图上可视化存储在124×2维矩阵的葡萄酒训练集：

```
>>> colors = ['r', 'b', 'g']
>>> markers = ['s', 'x', 'o']
>>> for l, c, m in zip(np.unique(y_train), colors, markers):
...     plt.scatter(X_train_pca[y_train==l, 0],
...                 X_train_pca[y_train==l, 1],
...                 c=c, label=l, marker=m)
>>> plt.xlabel('PC 1')
>>> plt.ylabel('PC 2')
>>> plt.legend(loc='lower left')
>>> plt.show()
```

从结果图中可以看到，与第二主成分（y轴）相比，数据更多的是沿着x轴（第一主成分）分布，这与前面得出的解释方差比结论一致。但是更直观，线性分类器能够很好地区分不同的类别：



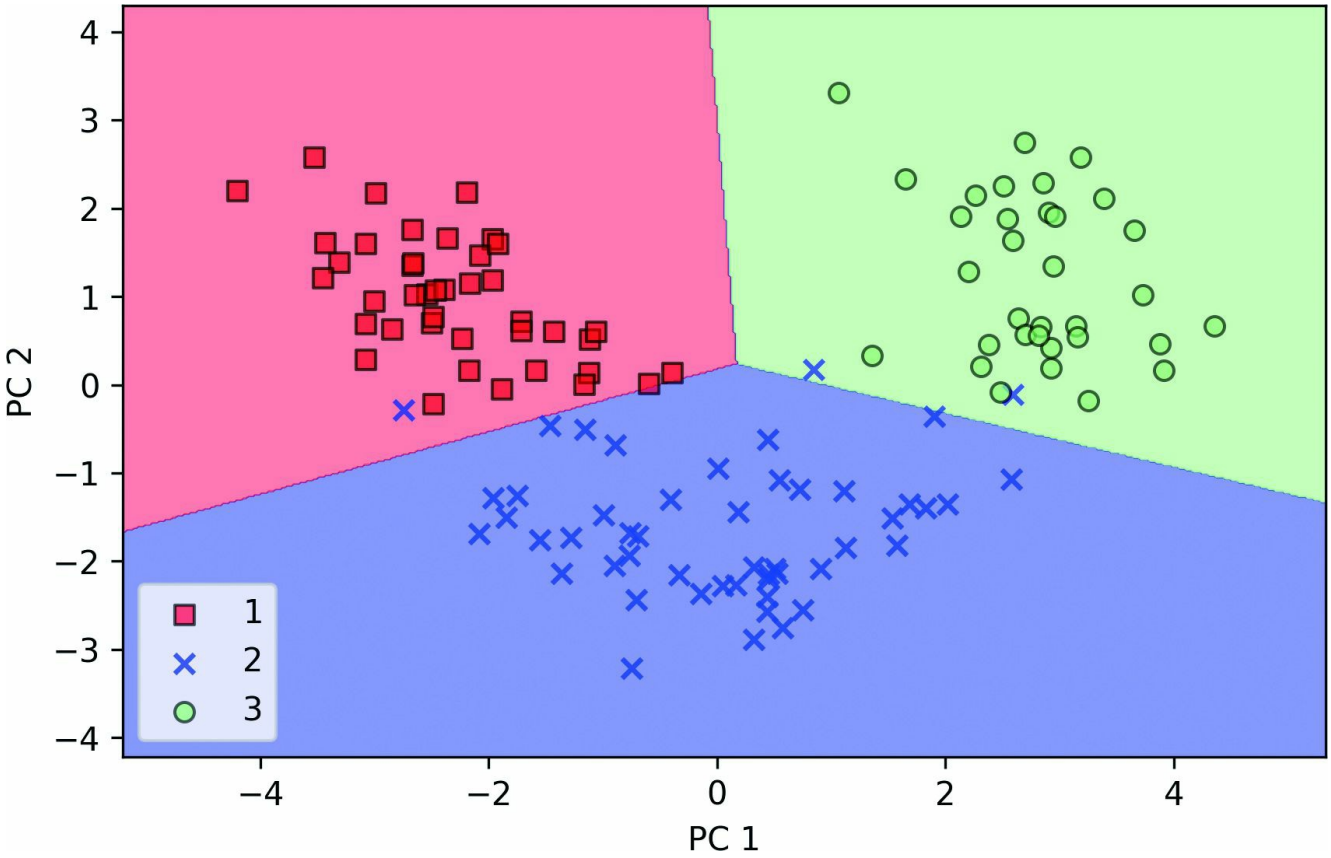

虽然前面的散点图对分类标签信息进行了编码，但是必须记住，PCA是一种不使用任何分类标签信息的无监督学习技术。

### 5.1.5 scikit-learn的主成分分析

尽管前一节的详细解释对掌握PCA的内部运作很有帮助，现在将讨论如何运用scikit-learn的PCA类。PCA是scikit-learn的另一个转换器类，在用相同模型参数转换训练数据和测试数据之前，首先用训练数据来拟合模型。现在，把scikit-learn中的PCA类应用在葡萄酒训练集上，通过逻辑回归转换样本，调用plot_decision_region函数实现决策区域的可视化。第2章定义了该函数：

```
from matplotlib.colors import ListedColormap

def plot_decision_regions(X, y, classifier, resolution=0.02):

    # setup marker generator and color map

    markers = ('s', 'x', 'o', '^', 'v')
    colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
    cmap = ListedColormap(colors[:len(np.unique(y))])

    # plot the decision surface
    x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
                           np.arange(x2_min, x2_max, resolution))
    Z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
    Z = Z.reshape(xx1.shape)
    plt.contourf(xx1, xx2, Z, alpha=0.4, cmap=cmap)
    plt.xlim(xx1.min(), xx1.max())
    plt.ylim(xx2.min(), xx2.max())

    # plot class samples
    for idx, cl in enumerate(np.unique(y)):
        plt.scatter(x=X[y == cl, 0],
                    y=X[y == cl, 1],
                    alpha=0.6,
                    c=cmap(idx),
                    edgecolor='black',
                    marker=markers[idx],
                    label=cl)

>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.decomposition import PCA
>>> pca = PCA(n_components=2)
>>> lr = LogisticRegression()
>>> X_train_pca = pca.fit_transform(X_train_std)
>>> X_test_pca = pca.transform(X_test_std)
>>> lr.fit(X_train_pca, y_train)
>>> plot_decision_regions(X_train_pca, y_train, classifier=lr)
>>> plt.xlabel('PC 1')
>>> plt.ylabel('PC 2')
>>> plt.legend(loc='lower left')
>>> plt.show()
```

通过执行前面的代码，现在应该看到训练数据的决策区域减少为两个主成分轴：

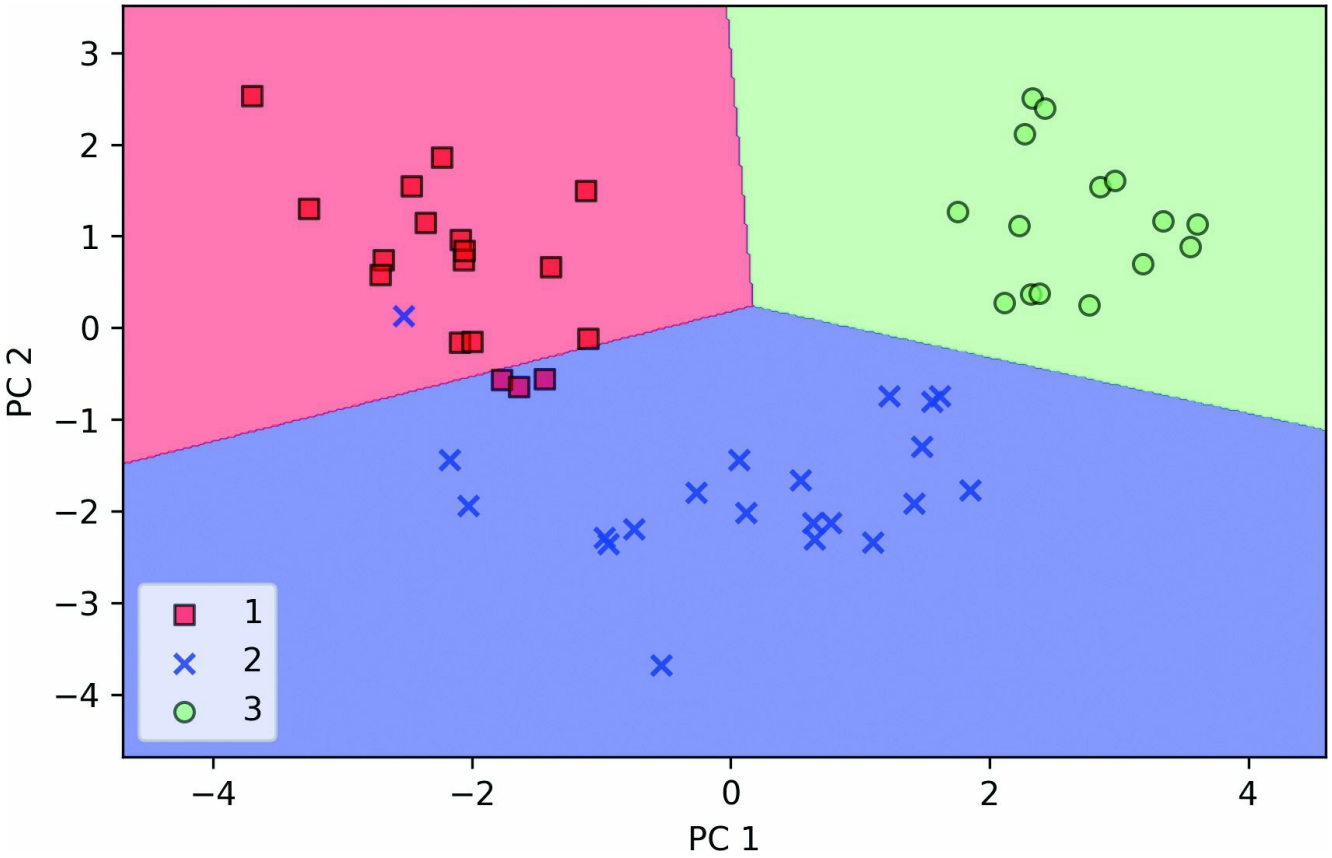



当比较scikit-learn实现的PCA与自己实现的PCA在预测方面的差异时，两个结果的图可能会如同镜子里的图像，一正一反。注意，这并不是哪个实现出了差错，造成差异的原因在于特征求解器，有些特征向量可能有正负号的问题。

这没什么大惊小怪的，如果需要可以把数据乘上-1直接反转镜像。要注意的是特征向量通常被调整为长度为1的单位。为完整起见，在转换后的测试集上用逻辑回归绘制出决策区域，看它是否能很好地完成数据分类任务：

```
>>> plot_decision_regions(X_test_pca, y_test, classifier=lr)
>>> plt.xlabel('PC1')
>>> plt.ylabel('PC2')
>>> plt.legend(loc='lower left')
>>> plt.show()
```

在测试集上执行上述代码得出决策区域之后，可以看到逻辑回归在该二维特征子空间上表现得相当不错，在测试数据集中只有很少的样本被错误归类：




如果对不同主成分的解释方差比较感兴趣，可以简单地把参数n_components设置为None，然后初始化PCA类，这样可以保持所有的主成分，然后可以通过调用explained_variance_ratio_属性访问解释方差比：

```
>>> pca = PCA(n_components=None)
>>> X_train_pca = pca.fit_transform(X_train_std)
>>> pca.explained_variance_ratio_
array([ 0.36951469,  0.18434927,  0.11815159,  0.07334252,
0.06422108, 0.05051724,  0.03954654,  0.02643918,  0.02389319,
0.01629614, 0.01380021,  0.01172226,  0.00820609])
```

请注意，设置n_components=None，当初始化PCA类时，将返回排序后的所有主成分并降维。

## 5.2 基于线性判别分析的有监督数据压缩

线性判别分析（LDA）可用于特征提取以提高计算效率和减少在非正则化过程中因维数过高而造成的过拟合。

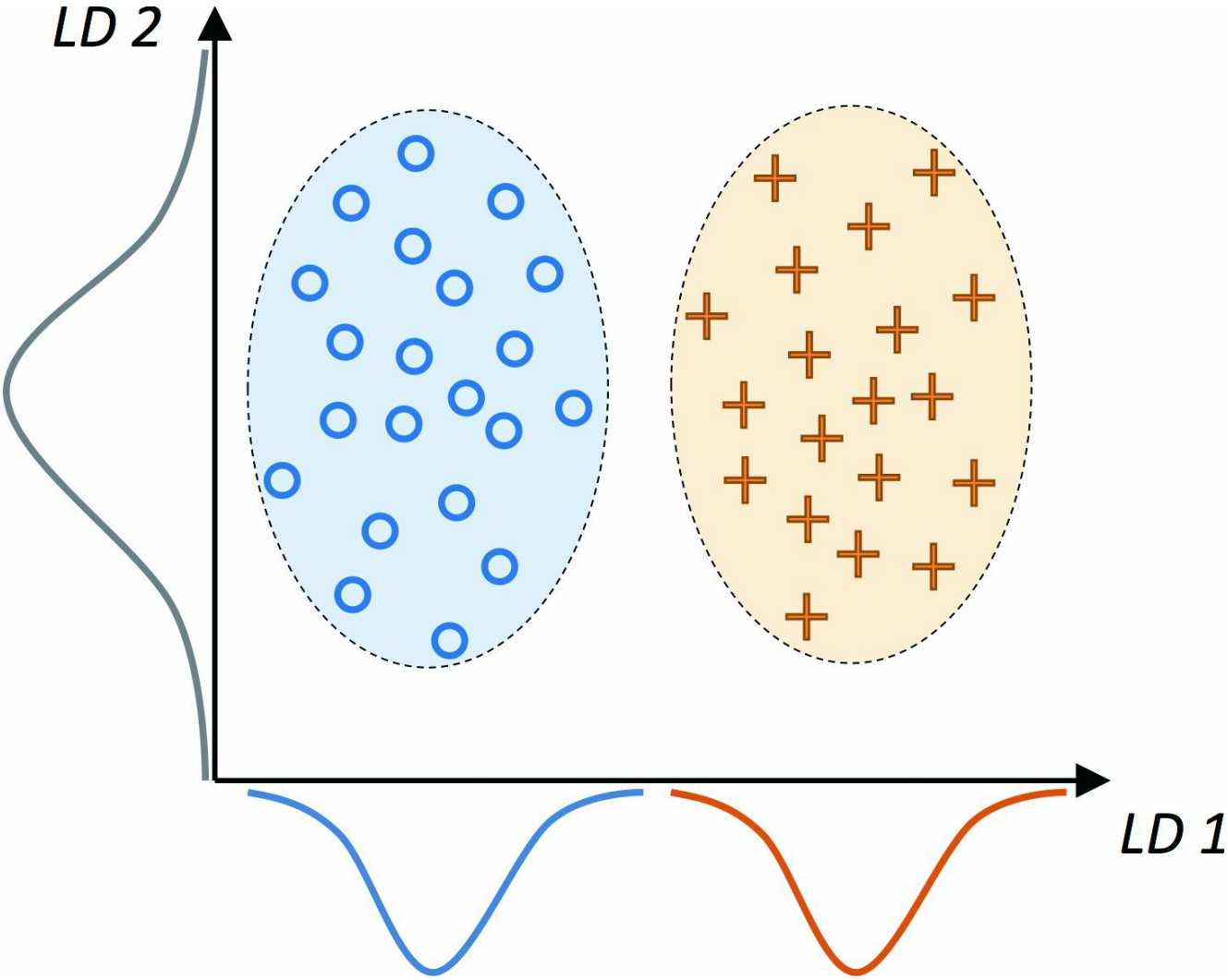
LDA背后的基本概念与PCA非常类似。PCA试图找到数据集中最大方差的正交成分轴，而LDA的目标是寻找和优化具有可分性的特征子空间。后续章节将更详细地讨论LDA和PCA之间的相似性，并逐步讨论LDA方法。

### 5.2.1 主成分分析与线性判别分析

LDA和PCA都是可以用来减少数据集维数的线性变换技术。前者是无监督学习算法，后者是有监督学习方法。因此，有人可能直观地认为，LDA是比PCA更优越的用于分类任务的特征提取技术。然而，A.M.马丁内兹在报告中说，在某些情况下，PCA的预处理在图像识别任务中往往能有更好的分类结果。例如，如果每个类只包含少量的样本（《PCA与LDA》，A.M.马丁内兹，A.C.凯克，IEEE《模式分析与机器智能》，2001年，23（2）：228-233）。

有时候LDA也被称为菲舍尔LDA。罗纳德·A.菲舍尔最初为解决二元分类问题于1936年提出了菲舍尔线性判别。（多重度量在分类问题中的应用，罗纳德·A.菲舍尔，优生学纪事，7（2）：179-188，1936）.菲舍尔线性判别后来被C.R.拉奥在1948年假设协方差相同且类呈正态分布的情况下延伸至多元分类问题，现在称之为LDA（C.R.拉奥，《重度度量在生物分类问题中的应用》，《皇家统计学会》杂志，B系列（方法论），1948年，10（2）：159-203）。

右图概括了LDA对二元问题的概念。第1类样本标记为圆形，第2类样本标记为叉。




如x轴所示的线性判别式（LD 1）能很好地把两个正态分布类分离。虽然在y轴上显示的示例性线性判别式（LD 2）捕获了数据集中的大量方差，但由于它不捕获任何类差异信息，所以无法成为好的线性判别方法。

LDA假设数据呈正态分布。此外，假设类具有相同的协方差矩阵，并且这些特征在统计上彼此独立。然而，即使其中的一个或多个假设被（略微）违反，降维的LDA仍然可以很好地发挥作用（R.O.杜达，P.E.哈特和D.G.斯朵尔克，《模式分类》第2版，纽约，2001）。

### 5.2.2 线性判别分析的内部逻辑

在深入了解代码实现之前，先简要概述执行LDA所需的主要步骤：

1.标准化d维数据集（d是特征数量）。

2.计算每个类的d维均值向量。

3.构建跨类的散布矩阵$S_B$和类内部的散布矩阵$S_w$。

4.矩阵$S_w^{-1}S_B$计算特征向量和对应的特征值。

5.按特征值降序排列，并对相应的特征向量排序。

6.选择对应于k个最大特征值的特征向量，构建d×k维变换矩阵W，特征向量为该矩阵的列。

7.把变换矩阵W投射到新的特征子空间。

可以看到LDA与PCA非常相似，它们都将矩阵分解为特征值和特征向量，从而形成新的低维特征空间。然而，正如前面提到的，LDA考虑分类的标记信息，以步骤2中计算均值向量形式表示。在后面的部分中，我们将更详细地讨论这七个步骤，并附有说明性的代码实现。

## 5.2.3 计算散布矩阵

本章已经对PCA中的葡萄酒数据集进行了标准化，可以跳过第一步直接计算均值向量，分别用来构造类内散布矩阵和类间散布矩阵。每个均值向量$m_i$存储对应分类样本i的特征平均值$\mu_m$：

$$\boldsymbol{m}_i = \frac{1}{n_i}\sum_{\boldsymbol{x}\in D_i}^{c}\boldsymbol{x}_m$$

由此产生三个均值向量：

$$\boldsymbol{m}_i = \begin{bmatrix} \mu_{i,\,alcohol} \\ \mu_{i,\,malic\ acid} \\ \vdots \\ \mu_{i,\,proline} \end{bmatrix} \qquad i \in \{1,2,3\}$$

```
>>> np.set_printoptions(precision=4)
>>> mean_vecs = []
>>> for label in range(1,4):
...     mean_vecs.append(np.mean(
...                 X_train_std[y_train==label], axis=0))
...     print('MV %s: %s\n' %(label, mean_vecs[label-1]))
MV 1: [ 0.9066 -0.3497  0.3201 -0.7189  0.5056  0.8807  0.9589 -0.5516
0.5416  0.2338  0.5897  0.6563  1.2075]

MV 2: [-0.8749 -0.2848 -0.3735  0.3157 -0.3848 -0.0433  0.0635 -0.0946

0.0703 -0.8286  0.3144  0.3608 -0.7253]

MV 3: [ 0.1992  0.866   0.1682  0.4148 -0.0451 -1.0286 -1.2876  0.8287
-0.7795  0.9649 -1.209  -1.3622 -0.4013]
```

现在可以用均值向量计算类内散布矩阵$S_W$：

$$\boldsymbol{S}_W = \sum_{i=1}^{c}\boldsymbol{S}_i$$

累加每个样本的散布矩阵$S_i$：

$$\boldsymbol{S}_i = \sum_{\boldsymbol{x} \in D_i}^{c} (\boldsymbol{x} - \boldsymbol{m}_i)(\boldsymbol{x} - \boldsymbol{m}_i)^{\mathrm{T}}$$

```
>>> d = 13 # number of features
>>> S_W = np.zeros((d, d))
>>> for label, mv in zip(range(1, 4), mean_vecs):
...     class_scatter = np.zeros((d, d))
>>>     for row in X_train_std[y_train == label]:
...         row, mv = row.reshape(d, 1), mv.reshape(d, 1)
...         class_scatter += (row - mv).dot((row - mv).T)
...     S_W += class_scatter
>>> print('Within-class scatter matrix: %sx%s' % (
...       S_W.shape[0], S_W.shape[1]))
Within-class scatter matrix: 13x13
```

在计算散布矩阵时所做的假设是训练集中的分类标签均匀分布。但是，如果显示分类标签的数量，就会发现这个假设被违反了：

```
>>> print('Class label distribution: %s'
...       % np.bincount(y_train)[1:])
Class label distribution: [41 50 33]
```

因此，在把散布矩阵$S_W$累加之前，需要调整每个散布矩阵$S_i$的比例尺寸。当用分类样本$n_i$的数量来划分散布矩阵时，可以看到计算散布矩阵实际上与计算散布矩阵的协方差矩阵$\sum_i$是一样的（协方差矩阵已经归一化）：

$$\sum_i = \frac{1}{n_i} \boldsymbol{S}_W = \frac{1}{n_i} \sum_{\boldsymbol{x} \in D_i}^{c} (\boldsymbol{x} - \boldsymbol{m}_i)(\boldsymbol{x} - \boldsymbol{m}_i)^{\mathrm{T}}$$

```
>>> d = 13 # number of features
>>> S_W = np.zeros((d, d))
>>> for label,mv in zip(range(1, 4), mean_vecs):
...     class_scatter = np.cov(X_train_std[y_train==label].T)
...     S_W += class_scatter
>>> print('Scaled within-class scatter matrix: %sx%s'
...       % (S_W.shape[0], S_W.shape[1]))
Scaled within-class scatter matrix: 13x13
```

计算类内散布矩阵（或协方差矩阵）后，可以进一步计算类间散布矩阵$S_B$：

$$\boldsymbol{S}_B = \sum_{i=1}^{c} n_i \left(\boldsymbol{m}_i - \boldsymbol{m}\right)\left(\boldsymbol{m}_i - \boldsymbol{m}\right)^{\mathrm{T}}$$

m为计算出的包括所有类样本的总均值：

```
>>> mean_overall = np.mean(X_train_std, axis=0)
>>> d = 13  # number of features
>>> S_B = np.zeros((d, d))
>>> for i, mean_vec in enumerate(mean_vecs):
...     n = X_train[y_train == i + 1, :].shape[0]
...     mean_vec = mean_vec.reshape(d, 1)  # make column vector
...     mean_overall = mean_overall.reshape(d, 1)
...     S_B += n * (mean_vec - mean_overall).dot(
...                 (mean_vec - mean_overall).T)
>>> print('Between-class scatter matrix: %sx%s' % (
...                S_B.shape[0], S_B.shape[1]))
```

### 5.2.4 在新的特征子空间选择线性判别式

LDA的其余步骤与PCA类似。然而，不是分解协方差矩阵的特征值，而是求解矩阵$S_w^{-1}S_B$的广义特征值：

```
>>> eigen_vals, eigen_vecs =\
...             np.linalg.eig(np.linalg.inv(S_W).dot(S_B))
```

在计算了特征对之后，可以按降序对特征值排列：

```
>>> eigen_pairs = [(np.abs(eigen_vals[i]), eigen_vecs[:,i])
...               for i in range(len(eigen_vals))]
>>> eigen_pairs = sorted(eigen_pairs,
...                key=lambda k: k[0], reverse=True)
>>> print('Eigenvalues in descending order:\n')
>>> for eigen_val in eigen_pairs:
...     print(eigen_val[0])


Eigenvalues in descending order:

349.617808906
172.76152219
3.78531345125e-14
2.11739844822e-14
1.51646188942e-14
1.51646188942e-14
1.35795671405e-14
1.35795671405e-14
7.58776037165e-15
5.90603998447e-15
5.90603998447e-15
2.25644197857e-15
0.0
```

LDA的线性判别数量最多为c-1，c为分类标签的数量，因为在散布矩阵$S_B$之间是秩为1或更少的c个矩阵的总和。确实可以看到，只有两个非零的特征值（由于NumPy的浮点运算，特征值3-13不完全为零）。

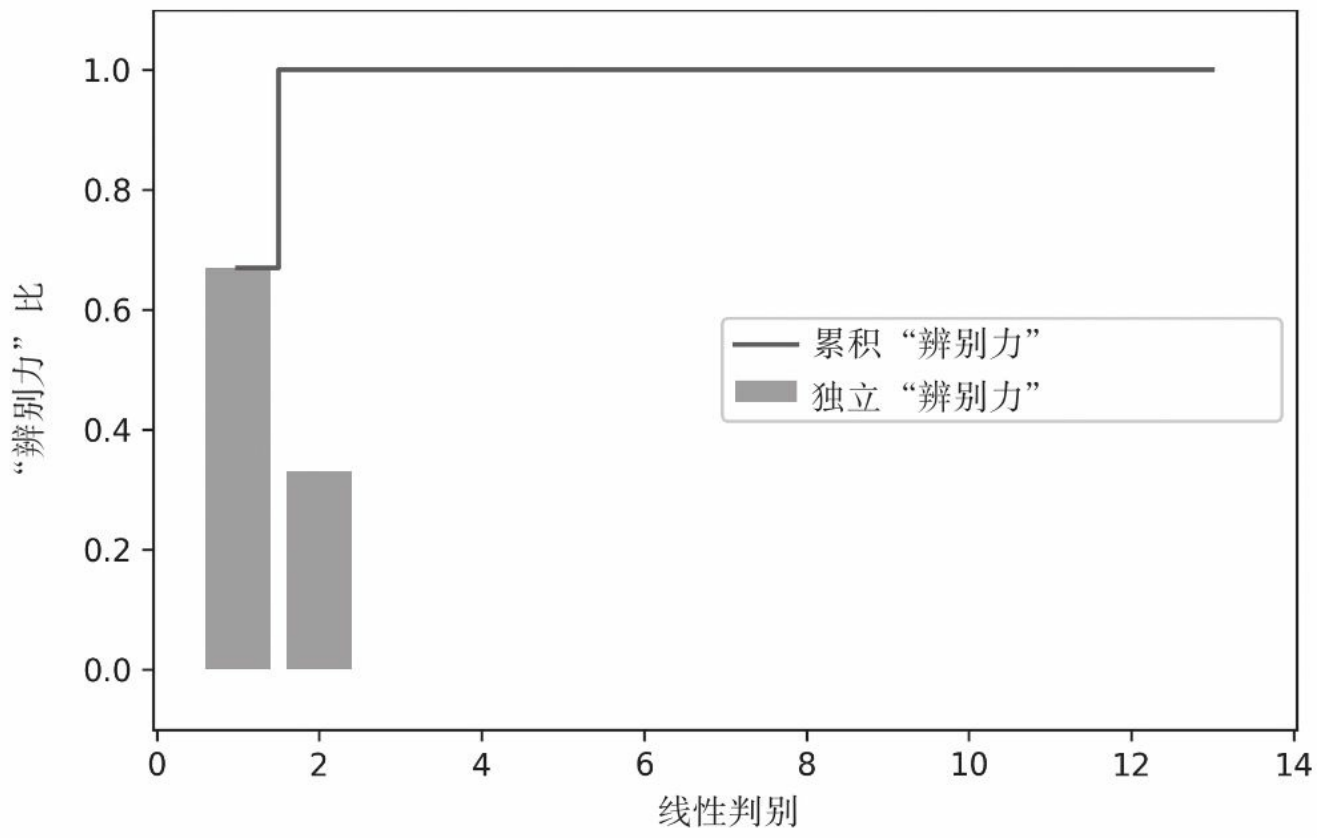
注意，在完全共线性的罕见情况下（所有同类的样本点都在一条直线上），协方差矩阵有秩，这将导致只有一个非零特征值的特征向量。

要度量有多少类之间的差异信息是由线性判别所捕获的（特征向量），

可以通过降低特征值画出线性判别图来展示，这与在PCA章节创建解释方差图类似。为简单起见，将类的判别信息的内容称为discriminability（辨别力）：

```
>>> tot = sum(eigen_vals.real)
>>> discr = [(i / tot) for i in sorted(eigen_vals.real, reverse=True)]
>>> cum_discr = np.cumsum(discr)
>>> plt.bar(range(1, 14), discr, alpha=0.5, align='center',
...         label='individual "discriminability"')
>>> plt.step(range(1, 14), cum_discr, where='mid',
...          label='cumulative "discriminability"')
>>> plt.ylabel('"discriminability" ratio')
>>> plt.xlabel('Linear Discriminants')
>>> plt.ylim([-0.1, 1.1])
>>> plt.legend(loc='best')
>>> plt.show()
```

从图中可以看到，仅前两个线性判别就捕获了葡萄酒训练集100%的有用信息：




现在把两个最具判别性的特征向量列叠加起来，创建转换矩阵W：

```
>>> w = np.hstack((eigen_pairs[0][1][:, np.newaxis].real,
...                eigen_pairs[1][1][:, np.newaxis].real))
>>> print('Matrix W:\n', w)
Matrix W:
 [[-0.1481 -0.4092]
 [ 0.0908 -0.1577]
 [-0.0168 -0.3537]
 [ 0.1484  0.3223]
 [-0.0163 -0.0817]
 [ 0.1913  0.0842]
 [-0.7338  0.2823]
 [-0.075  -0.0102]
 [ 0.0018  0.0907]
 [ 0.294  -0.2152]
 [-0.0328  0.2747]
 [-0.3547 -0.0124]
 [-0.3915 -0.5958]]
```

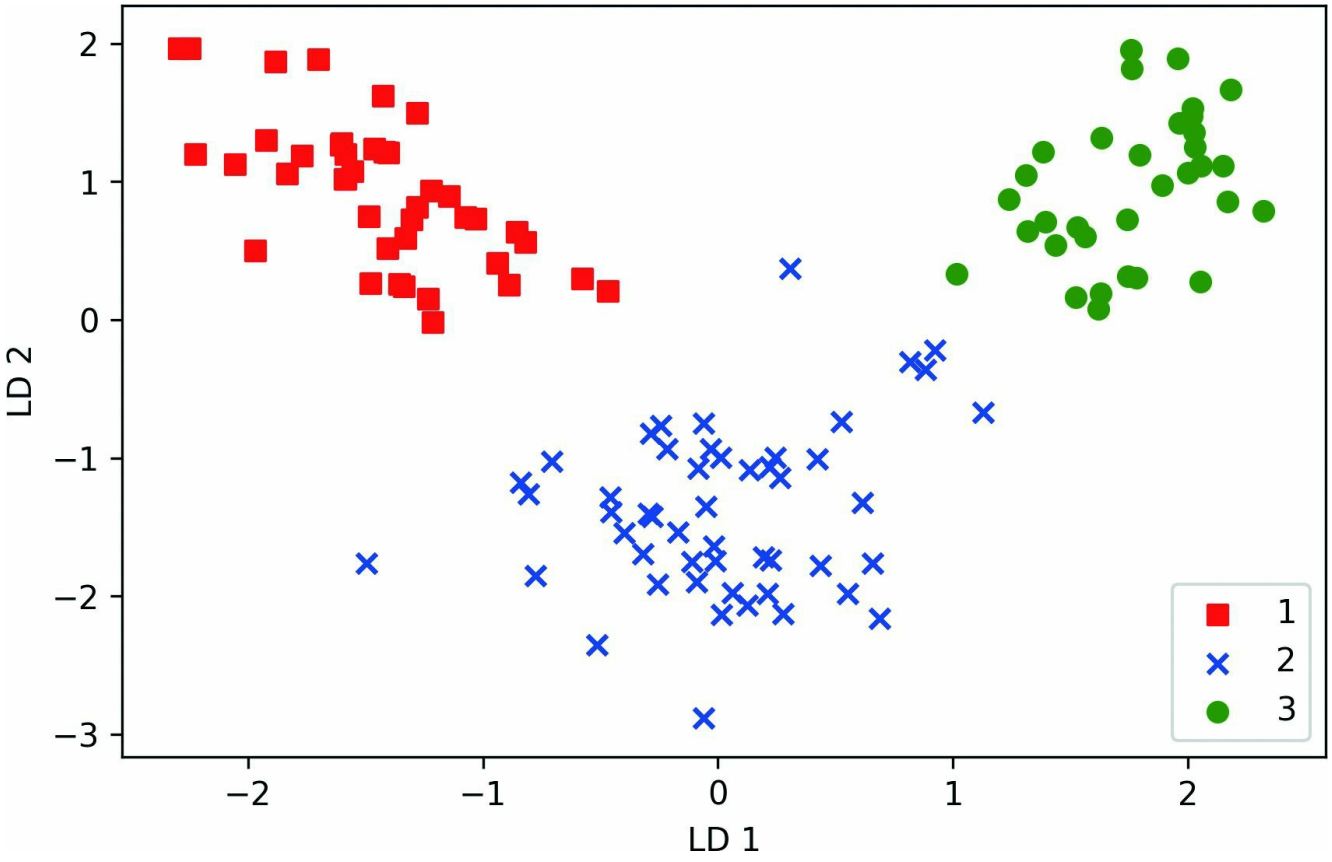
### 5.2.5 将样本投影到新的特征空间

可以用前面小节中创建的转换矩阵W，通过矩阵相乘来转换训练集：

$$X'=XW$$

```
>>> X_train_lda = X_train_std.dot(w)
>>> colors = ['r', 'b', 'g']
>>> markers = ['s', 'x', 'o']
>>> for l, c, m in zip(np.unique(y_train), colors, markers):
...     plt.scatter(X_train_lda[y_train==l, 0],
...                 X_train_lda[y_train==l, 1] * (-1),
...                 c=c, label=l, marker=m)
>>> plt.xlabel('LD 1')
>>> plt.ylabel('LD 2')
>>> plt.legend(loc='lower right')
>>> plt.show()
```

从结果图中可以看到三类葡萄酒在新的特征子空间完全线性可分：


LD 2
LD 1
2
1
0
−1
−2
−3
−2
−1
0
1
2
1
2
3

### 5.2.6 用scikit-learn实现的LDA

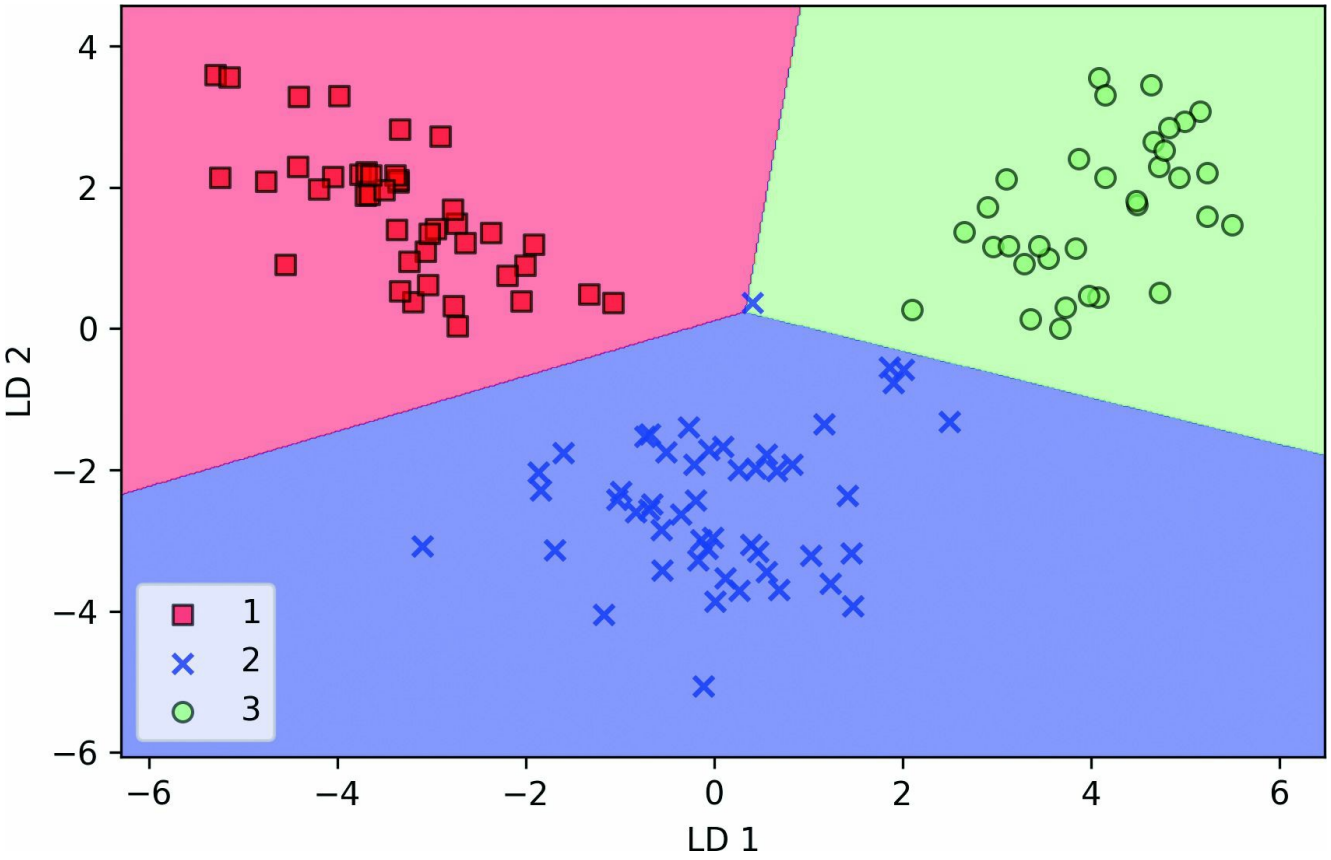
逐步实现是理解LDA内部工作机制以及LDA和PCA差异的好方法。现在来看看如何用scikit-learn实现LDA类：

```
>>> from sklearn.discriminant_analysis import
...    LinearDiscriminantAnalysis as LDA
>>> lda = LDA(n_components=2)
>>> X_train_lda = lda.fit_transform(X_train_std, y_train)
```

下一步，看看逻辑回归分类器在LDA变换后如何处理低维训练集：

```
>>> lr = LogisticRegression()
>>> lr = lr.fit(X_train_lda, y_train)
>>> plot_decision_regions(X_train_lda, y_train, classifier=lr)
>>> plt.xlabel('LD 1')
>>> plt.ylabel('LD 2')
>>> plt.legend(loc='lower left')
>>> plt.show()
```

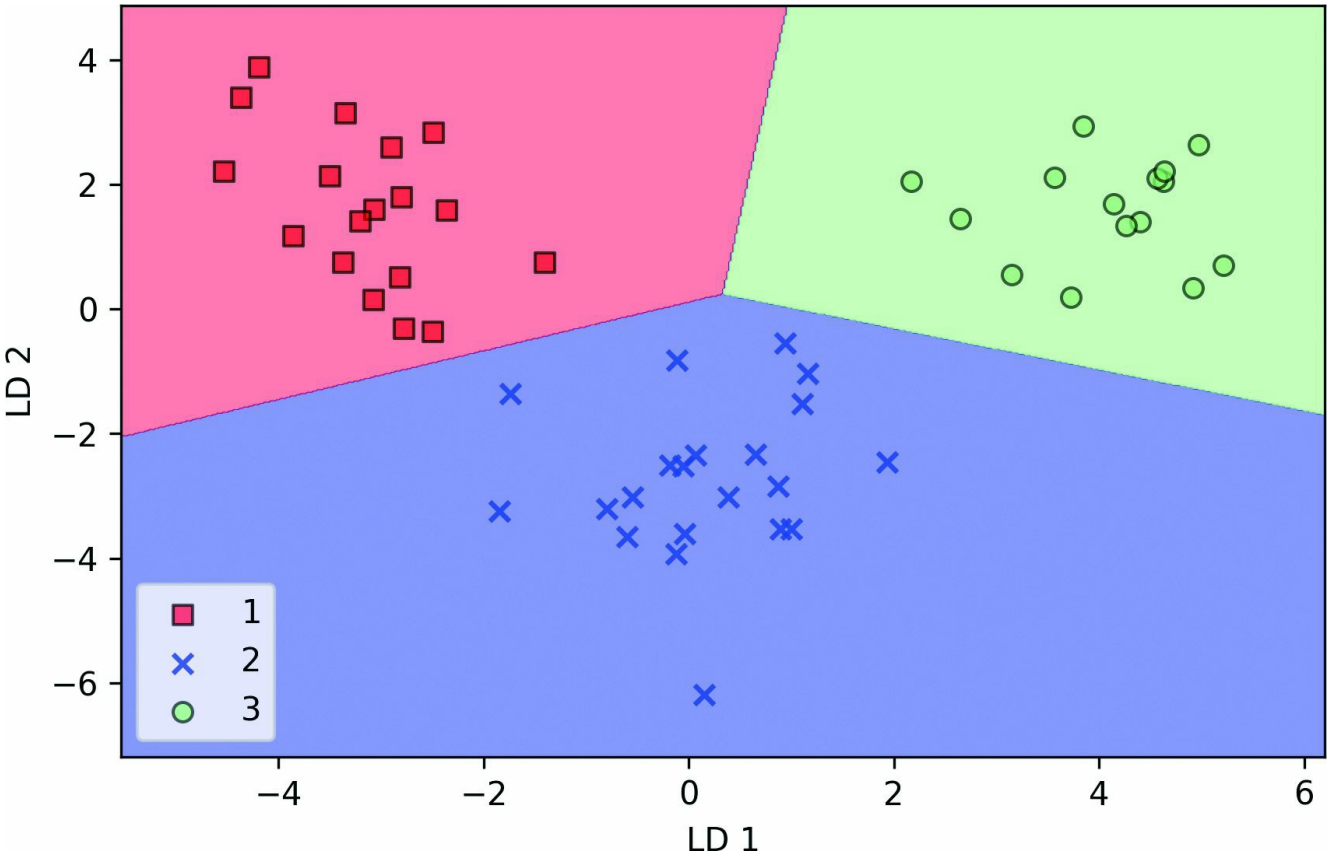
从结果图可以看到逻辑回归模型把一个2类样本分错了类：




可以改变决策边界使逻辑回归模型能够正确地对训练样本中的所有样本进行分类。但是更重要的是观察测试集上的结果：

```
>>> X_test_lda = lda.transform(X_test_std)
>>> plot_decision_regions(X_test_lda, y_test, classifier=lr)
>>> plt.xlabel('LD 1')
>>> plt.ylabel('LD 2')
>>> plt.legend(loc='lower left')
>>> plt.show()
```

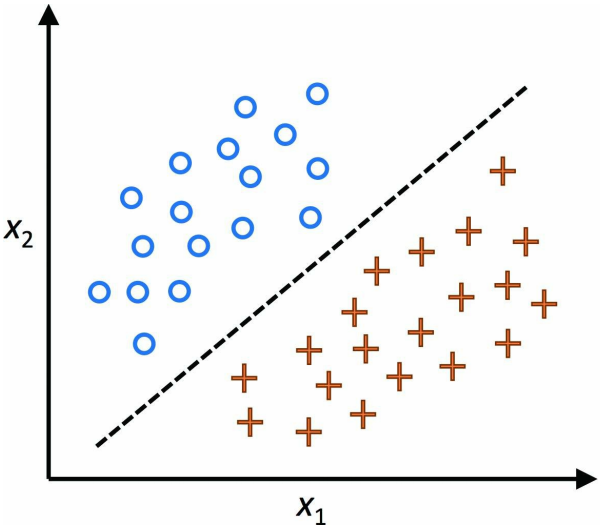
如下图所示，逻辑回归分类器能够用一个二维特征子空间代替原来的13个葡萄酒特征，从而在测试集中对样本进行精确分类：

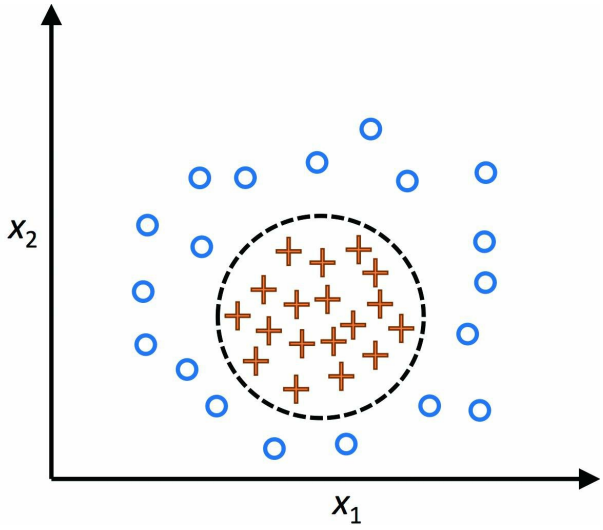

LD 2
LD 1
1
2
3
4
2
0
−2
−4
−6
−4
−2
0
2
4
6

## 5.3 非线性映射的核主成分分析

许多机器学习算法对输入数据的线性可分性做出假设。感知器甚至需要完全线性可分离的训练数据才能收敛。至今为止讨论过的其他算法，Adaline、逻辑回归和（标准的）支持向量机等，认为缺乏完美线性可分性的原因是噪声。

然而，如果面对的是非线性问题（在现实世界中可能会遇到相当多这样的问题），像PCA和LDA这样的线性变换降维技术可能并不是最好的选择。回顾第3章，本节将介绍涉及核支持向量机的概念的核版PCA，或叫作KPCA。讨论如何利用核PCA把不可线性分离的数据转换至适合线性分类器的新的低维子空间。


$x_2$
$x_1$


$x_2$
$x_1$

### 5.3.1 核函数与核技巧

记得在第3章中讨论过的核支持向量机，可以把数据投射到一个新的更高维度，在那里类变成线性可分，从而解决了非线性问题。要将样本的$x \in R^d$变换到更高的k维子空间，定义了一个非线性映射函数φ：

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^k \left(k >> d\right)$$

可以认为φ是一个用来创建原始特征非线性组合的函数，把原来的d维数据集映射到较大的k维特征空间。例如，如果特征向量$x \in R^d$（x是一个包含d个特征的列向量）有两个维度（d=2），可以通过下面的计算把它潜在地映射到三维空间：

$$\boldsymbol{x} = \left[x_1, x_2\right]^{\mathrm{T}}$$

$$\downarrow \phi$$

$$\mathbf{z} = \left[x_1^2, \sqrt{2x_1x_2}, x_2^2\right]^{\mathrm{T}}$$

换言之，通过核PCA的非线性映射将数据转换到高维空间。然后在这个高维空间用标准的PCA将数据投影回到低维空间，在该低维空间线性分类器可以分开样本（前提是输入空间的密度可以分离）。然而，该方法的缺点是计算成本非常昂贵，而这正是核技巧发挥作用的地方。可以利用核技巧在原始特征空间计算两个高维特征向量之间的相似性。

在继续讨论利用核技巧来解决计算成本昂贵问题之前，先回顾一下在本章开头实现的标准PCA方法。当时用下述公式计算了k和j两个特征之间的协方差，如下所示：

$$\sigma_{jk} = \frac{1}{n}\sum_{i=1}^{n}\left(x_j^{(i)} - \mu_j\right)\left(x_k^{(i)} - \mu_k\right)$$

标准化使特征处在均值为零的中心，例如$\mu_j=0$且$\mu_k=0$，简化方程式如下：

$$\sigma_{jk} = \frac{1}{n}\sum_{i=1}^{n} x_j^{(i)} x_k^{(i)}$$

注意前面的方程计算两特征之间的协方差。计算协方差矩阵Σ的一般方程式如下：

$$\sum = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{x}^{(i)} \boldsymbol{x}^{(i)^{\mathrm{T}}}$$

伯恩哈德·史科夫阐述了该方法（B.史科夫，A.斯毛拉，K.R.穆勒，《核主成分分析》，1997，P583-588），可以用非线性特征组合通过φ取代原始特征空间样本之间的点积：

$$\sum = \frac{1}{n}\sum_{i=1}^{n} \phi\left(\boldsymbol{x}^{(i)}\right) \phi\left(\boldsymbol{x}^{(i)}\right)^{\mathrm{T}}$$

为了得到特征向量这个协方差矩阵中的主成分，必须求解下面的方程：

$$\sum \boldsymbol{v} = \lambda \boldsymbol{v}$$

$$\Rightarrow \frac{1}{n}\sum_{i=1}^{n} \phi\left(\boldsymbol{x}^{(i)}\right) \phi\left(\boldsymbol{x}^{(i)}\right)^{\mathrm{T}} \boldsymbol{v} = \lambda \boldsymbol{v}$$

$$\Rightarrow v = \frac{1}{n\lambda}\sum_{i=1}^{n} \phi\left(\boldsymbol{x}^{(i)}\right) \phi\left(\boldsymbol{x}^{(i)}\right)^{\mathrm{T}} \boldsymbol{v} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{a}^{(i)} \phi\left(\boldsymbol{x}^{(i)}\right)$$

这里的λ和v分别为协方差矩阵∑的特征值和特征向量，a可以通过提取的核（相似）矩阵κ的特征向量获得，正如在接下来的段落中将要看到的。

展示核矩阵的推导过程如下。首先以矩阵的表示方式写出协方差矩阵，其中φ（X）是一个n×k维的矩阵：

$$\sum = \frac{1}{n}\sum_{i=1}^{n} \phi\left(\boldsymbol{x}^{(i)}\right) \phi\left(\boldsymbol{x}^{(i)}\right)^{\mathrm{T}} = \frac{1}{n} \phi(\boldsymbol{X})^{\mathrm{T}} \phi(\boldsymbol{X})$$

现在可以写出以下的特征向量方程：

$$v=\frac{1}{n}\sum_{i=1}^{n}a^{(i)}\phi\left(\boldsymbol{x}^{(i)}\right)=\lambda\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{a}$$

因为∑v=λv，所以得到：

$$\frac{1}{n}\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{a}=\lambda\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{a}$$

把两边都与φ（X）相乘产生以下的结果：

$$\frac{1}{n}\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{a}=\lambda\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{a}$$

$$\Rightarrow\frac{1}{n}\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{a}=\lambda\boldsymbol{a}$$

$$\Rightarrow\frac{1}{n}\boldsymbol{K}\boldsymbol{a}=\lambda\boldsymbol{a}$$

K为类似的（核）矩阵：

$$\boldsymbol{K}=\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}$$

在3.5节中，明确用核函数κ利用核技巧来避免φ中样本x的成对点乘计算，因此避免了对特征向量做具体计算：

$$\kappa\left(\boldsymbol{x}^{(i)},\boldsymbol{x}^{(j)}\right)=\phi\left(\boldsymbol{x}^{(i)}\right)^{\mathrm{T}}\phi\left(\boldsymbol{x}^{(j)}\right)$$

换句话说，在核PCA之后得到的是已经投射到各成分的样本，而不是像标准PCA方法那样构造变换矩阵。核函数（或简称核）基本上可以理解为计算两个向量之间点积的函数，即相似性的度量。

描述最常用的核如下：

·多项式核：

$$\kappa\left(\boldsymbol{x}^{(i)},\boldsymbol{x}^{(j)}\right)=\left(\boldsymbol{x}^{(i)\mathrm{T}}\boldsymbol{x}^{(j)}+\theta\right)^{p}$$

在这里，θ是阈值，p是由用户指定的幂次。

·双曲正切（S）的核：

$$\kappa\left(\boldsymbol{x}^{(i)},\boldsymbol{x}^{(j)}\right)=\tanh\left(\eta\boldsymbol{x}^{(i)\mathrm{T}}\boldsymbol{x}^{(j)}+\theta\right)$$

·径向基函数（RBF）或高斯核函数，我们将在下一节中使用下面的例子：

$$\kappa\left(\boldsymbol{x}^{(i)},\boldsymbol{x}^{(j)}\right)=\exp\left(\frac{\|\boldsymbol{x}^{(i)}-\boldsymbol{x}^{(j)}\|^{2}}{2\sigma^{2}}\right)$$

往往把它写成下面的形式，并引入变量$\gamma=\frac{1}{2\sigma}$：

$$\kappa\left(\boldsymbol{x}^{(i)},\boldsymbol{x}^{(j)}\right)=\exp\left(-\gamma\|\boldsymbol{x}^{(i)}-\boldsymbol{x}^{(j)}\|^{2}\right)$$

为了总结迄今所学的知识，定义以下三个步骤来实现RBF的核主成分分析：

1.计算核（相似）矩阵κ，在这里需要做一下的计算：

$$\kappa\left(\boldsymbol{x}^{(i)},\boldsymbol{x}^{(j)}\right)=\exp\left(-\gamma\|\boldsymbol{x}^{(i)}-\boldsymbol{x}^{(j)}\|^{2}\right)$$

对每对样本进行计算：

$$K=\begin{bmatrix}\kappa\left(x^{(1)},x^{(1)}\right) & \kappa\left(x^{(1)},x^{(2)}\right) & \cdots & \kappa\left(x^{(1)},x^{(n)}\right)\\ \kappa\left(x^{(2)},x^{(1)}\right) & \kappa\left(x^{(2)},x^{(2)}\right) & \cdots & \kappa\left(x^{(2)},x^{(n)}\right)\\ \vdots & \vdots & \ddots & \vdots\\ \kappa\left(x^{(n)},x^{(1)}\right) & \kappa\left(x^{(n)},x^{(2)}\right) & \cdots & \kappa\left(x^{(n)},x^{(n)}\right)\end{bmatrix}$$

例如，如果数据集包含100个训练样本，成对相似性的对称核矩阵为100×100维。

2.用下面的公式处理核矩阵K：

$$K'=K-1_nK-K1_n+1_nK1_n$$

这里，$1_n$是一个n×n维的矩阵（与核矩阵相同），所有的值都是$\frac{1}{n}$。

3.根据按降序排列的特征值，收集排在中心核矩阵前k个相应特征向量。与标准PCA相反，特征矩阵不是主成分轴，但样本已经投影到这些轴。

现在，你可能会想知道为什么需要在第二步集中核矩阵。以前假设所处理的是标准化数据，当通过构建协方差矩阵和用非线性特征组合取代点乘时，所有特征的均值都是零。因此，在第二步集中核心矩阵是必要的，因为没有具体计算新的特征空间，所以不能保证新的特征空间也集中在零。

下一节将通过Python实现核PCA将上述三个步骤付诸实践。

### 5.3.2 用Python实现核主成分分析

前一节讨论了核主成分分析的核心概念。现在将用Python按照前面总结好的三个步骤实现RBF核PCA。将看到用SciPy和NumPy的一些辅助功能实现核PCA其实很简单：

```
  from scipy.spatial.distance import pdist, squareform
  from scipy import exp
  from scipy.linalg import eigh
  import numpy as np

  def rbf_kernel_pca(X, gamma, n_components):
      """
      RBF kernel PCA implementation.

      Parameters
      ------------
      X: {NumPy ndarray}, shape = [n_samples, n_features]

      gamma: float
        Tuning parameter of the RBF kernel

      n_components: int
        Number of principal components to return

      Returns
      ------------
       X_pc: {NumPy ndarray}, shape = [n_samples, k_features]
         Projected dataset

      """
      # Calculate pairwise squared Euclidean distances
      # in the MxN dimensional dataset.
      sq_dists = pdist(X, 'sqeuclidean')

      # Convert pairwise distances into a square matrix.
    mat_sq_dists = squareform(sq_dists)

    # Compute the symmetric kernel matrix.
    K = exp(-gamma * mat_sq_dists)

    # Center the kernel matrix.
    N = K.shape[0]
    one_n = np.ones((N,N)) / N
    K = K - one_n.dot(K) - K.dot(one_n) + one_n.dot(K).dot(one_n)

    # Obtaining eigenpairs from the centered kernel matrix
    # scipy.linalg.eigh returns them in ascending order
    eigvals, eigvecs = eigh(K)

eigvals, eigvecs = eigvals[::-1], eigvecs[:, ::-1]

# Collect the top k eigenvectors (projected samples)
X_pc = np.column_stack((eigvecs[:, i]
                        for i in range(n_components)))

return X_pc
```

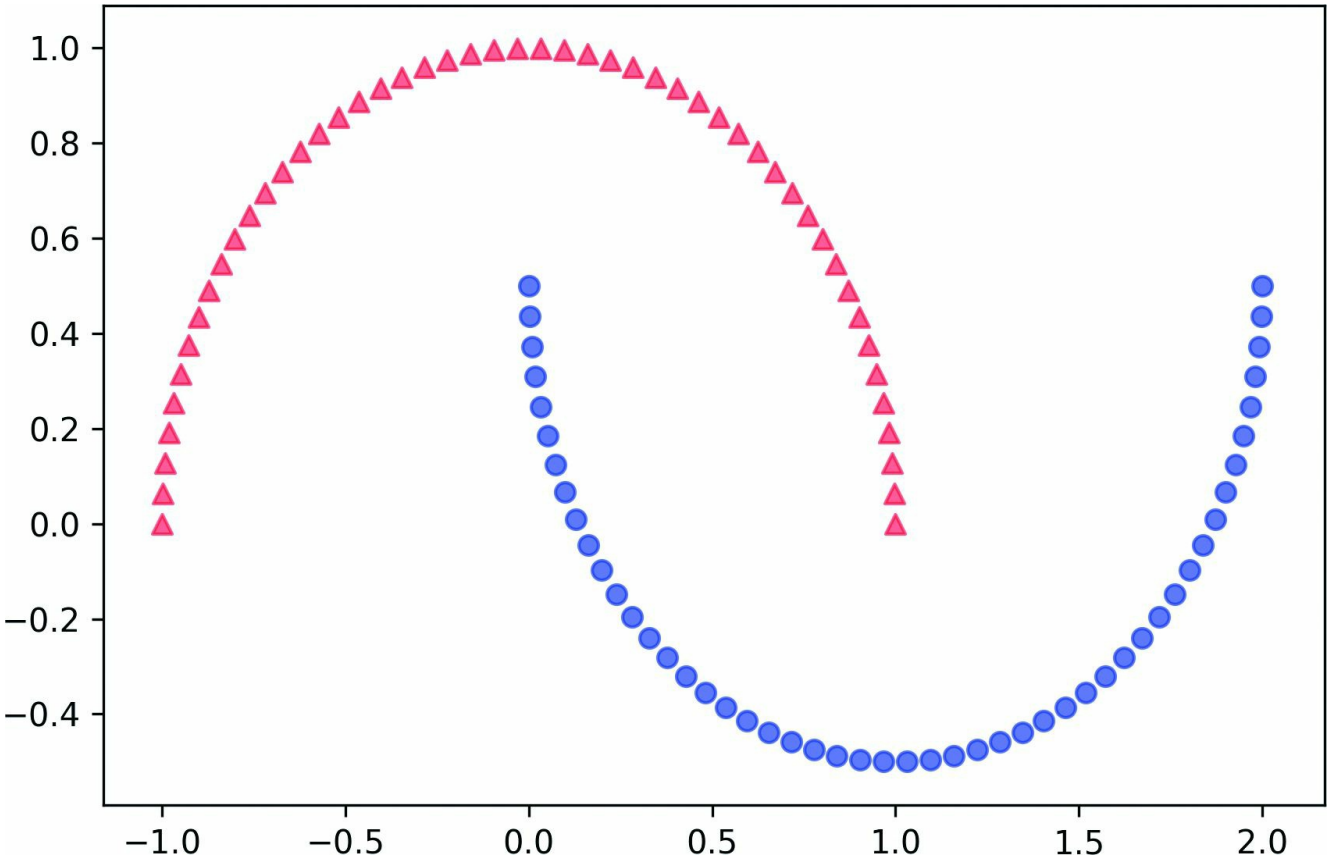
使用RBF核主成分分析降维的缺点是必须事先为参数γ指定优先级。需要通过试验找到合适的γ值，最好的做法是用算法调优参数，例如执行网格搜索，第6章将更详细地讨论。

#### 5.3.2.1 案例1–分离半月形

现在用rbf_kernel_pca来处理一些非线性示例数据集。首先创建二维数据集，其中100个样本组成两个半月形：

```
>>> from sklearn.datasets import make_moons
>>> X, y = make_moons(n_samples=100, random_state=123)
>>> plt.scatter(X[y==0, 0], X[y==0, 1],
...             color='red', marker='^', alpha=0.5)
>>> plt.scatter(X[y==1, 0], X[y==1, 1],
...             color='blue', marker='o', alpha=0.5)
>>> plt.show()
```

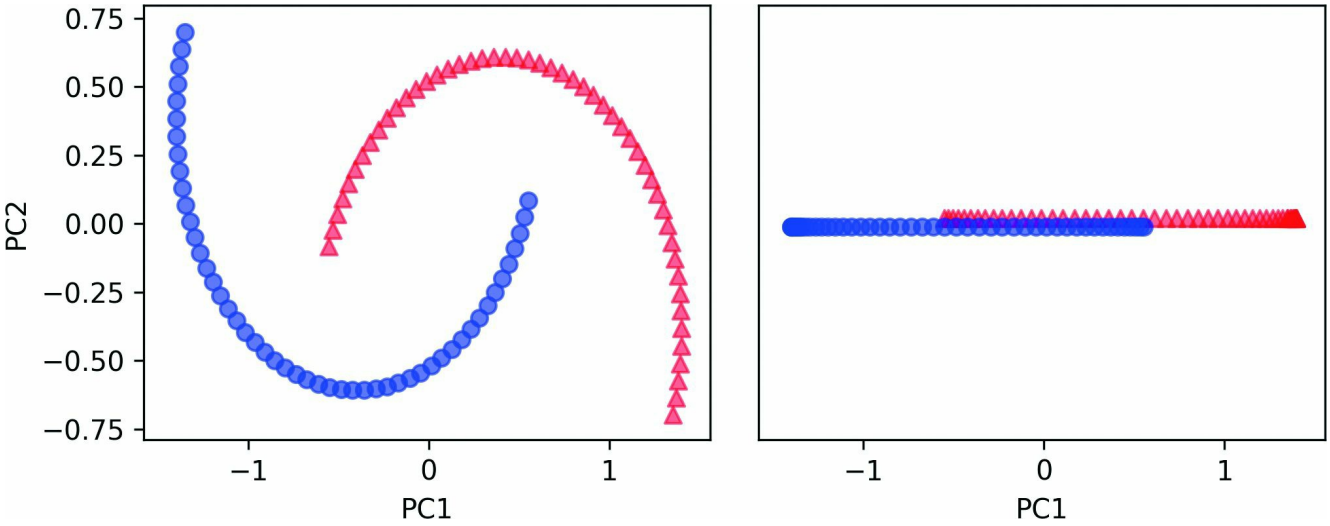
图中三角形符号的半月代表一类样本，而圆符号的半月代表另一类：




这两个半月形显然不能线性可分，要通过核主成分分析来表示半月形，以便把数据集变成线性分类器的合适输入。但是先观察如何通过标准的PCA把数据集投射到主成分上：

```
>>> from sklearn.decomposition import PCA
>>> scikit_pca = PCA(n_components=2)
>>> X_spca = scikit_pca.fit_transform(X)
>>> fig, ax = plt.subplots(nrows=1,ncols=2, figsize=(7,3))
>>> ax[0].scatter(X_spca[y==0, 0], X_spca[y==0, 1],
...               color='red', marker='^', alpha=0.5)
>>> ax[0].scatter(X_spca[y==1, 0], X_spca[y==1, 1],
...               color='blue', marker='o', alpha=0.5)

>>> ax[1].scatter(X_spca[y==0, 0], np.zeros((50,1))+0.02,
...               color='red', marker='^', alpha=0.5)
>>> ax[1].scatter(X_spca[y==1, 0], np.zeros((50,1))-0.02,
...               color='blue', marker='o', alpha=0.5)
>>> ax[0].set_xlabel('PC1')
>>> ax[0].set_ylabel('PC2')
>>> ax[1].set_ylim([-1, 1])
>>> ax[1].set_yticks([])
>>> ax[1].set_xlabel('PC1')
>>> plt.show()
```

从结果图明显可以看出线性分类器在标准PCA转换的数据集上表现不佳：




注意，在绘制第一主成分图（右子图）时，三角形样本只是略微向上偏移，而圆形样本也只是略微向下偏移，这进一步展示了两个类的重叠情况。左子图显示，原来的半月形在垂直中心上只有微弱的剪切和翻转，这种转变无助于线性分类器区分圆形和三角形。同样，如果我们把数据集投射到一个一维特征轴上，所对应的两个半月形状的三角形也是线性不可分的，如右子图所示。

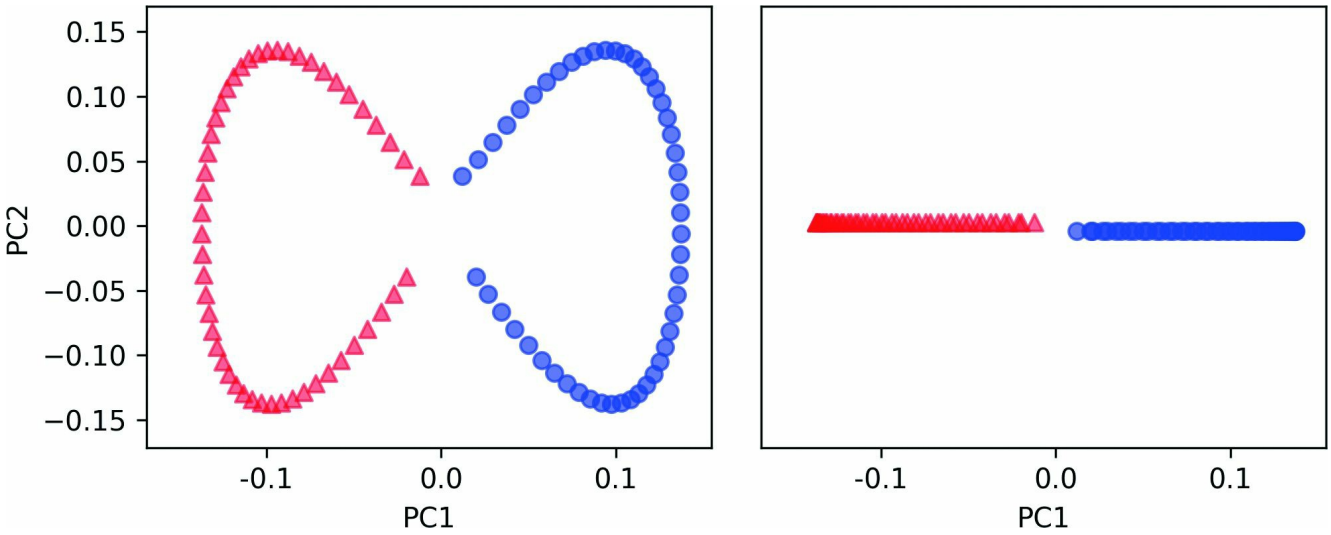
请记住PCA是一种无监督学习方法，与LDA相反，它不用分类标签的分类信息来最大化差别。加入三角形和圆形只是为了以图来说明分离的程度。

现在尝试使用上一节实现的核主成分分析函数rbf_kernel_pca：

```
>>> X_kpca = rbf_kernel_pca(X, gamma=15, n_components=2)
>>> fig, ax = plt.subplots(nrows=1,ncols=2, figsize=(7,3))
>>> ax[0].scatter(X_kpca[y==0, 0], X_kpca[y==0, 1],
...               color='red', marker='^', alpha=0.5)
>>> ax[0].scatter(X_kpca[y==1, 0], X_kpca[y==1, 1],
...               color='blue', marker='o', alpha=0.5)
>>> ax[1].scatter(X_kpca[y==0, 0], np.zeros((50,1))+0.02,
...               color='red', marker='^', alpha=0.5)
>>> ax[1].scatter(X_kpca[y==1, 0], np.zeros((50,1))-0.02,
...               color='blue', marker='o', alpha=0.5)
>>> ax[0].set_xlabel('PC1')
>>> ax[0].set_ylabel('PC2')
>>> ax[1].set_ylim([-1, 1])
>>> ax[1].set_yticks([])

>>> ax[1].set_xlabel('PC1')
>>> plt.show()
```

可以看到circles和triangles的类线性分离良好，因此成为适合线性分类

器的训练集：



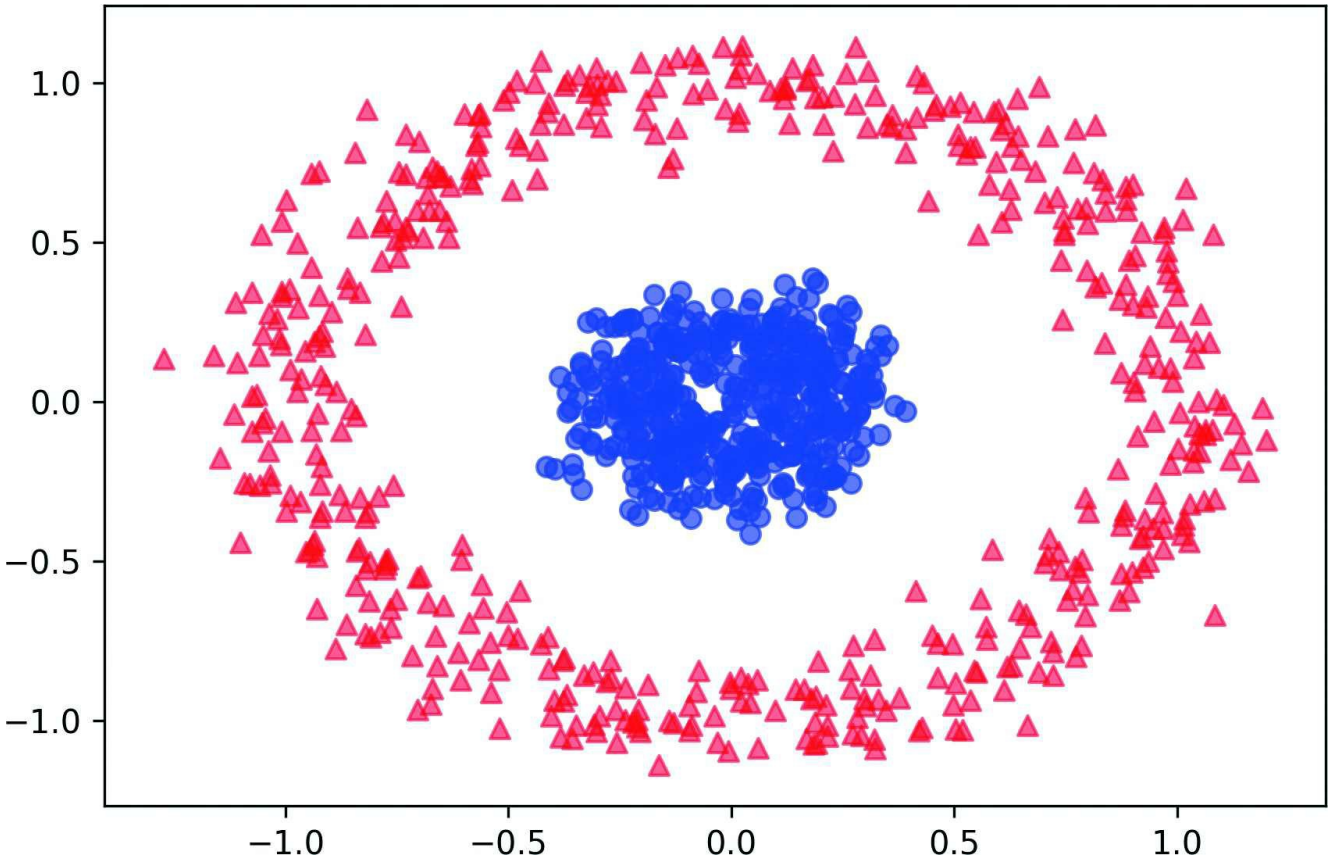

不幸的是，调优参数γ对于不同的数据集没有普遍价值。需要实验来寻找适合给定问题的γ值。第6章将讨论那些有助于完成自动化调优参数任务的技术。这里将用作者发现的γ值来产生好结果。

#### 5.3.2.2 案例2–分离同心圆

前一节展示了如何通过核主成分分析分离两个半月形数据集。由于对理解核主成分分析的概念做了大量的工作，来看一下另一个非线性问题的有趣例子，即同心圆：

```
>>> from sklearn.datasets import make_circles
>>> X, y = make_circles(n_samples=1000,
...                     random_state=123, noise=0.1, factor=0.2)
>>> plt.scatter(X[y==0, 0], X[y==0, 1],
...                     color='red', marker='^', alpha=0.5)
>>> plt.scatter(X[y==1, 0], X[y==1, 1],
...                     color='blue', marker='o', alpha=0.5)
>>> plt.show()
```

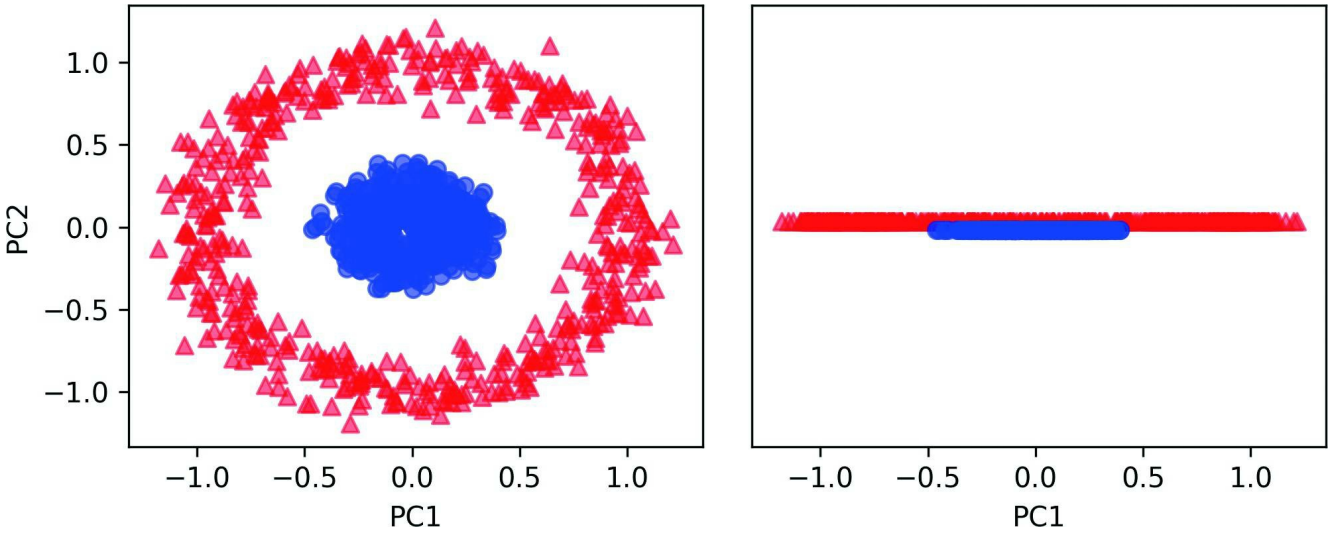
同样假设一个二元分类问题，三角形代表一类，而圆形代表另一类：




从标准PCA方法开始，与RBF核主成分的分析结果进行比较：

```
>>> scikit_pca = PCA(n_components=2)
>>> X_spca = scikit_pca.fit_transform(X)
>>> fig, ax = plt.subplots(nrows=1,ncols=2, figsize=(7,3))
>>> ax[0].scatter(X_spca[y==0, 0], X_spca[y==0, 1],
...               color='red', marker='^', alpha=0.5)
>>> ax[0].scatter(X_spca[y==1, 0], X_spca[y==1, 1],
...               color='blue', marker='o', alpha=0.5)
>>> ax[1].scatter(X_spca[y==0, 0], np.zeros((500,1))+0.02,
...              color='red', marker='^', alpha=0.5)
>>> ax[1].scatter(X_spca[y==1, 0], np.zeros((500,1))-0.02,
...               color='blue', marker='o', alpha=0.5)
>>> ax[0].set_xlabel('PC1')
>>> ax[0].set_ylabel('PC2')
>>> ax[1].set_ylim([-1, 1])
>>> ax[1].set_yticks([])
>>> ax[1].set_xlabel('PC1')
>>> plt.show()
```

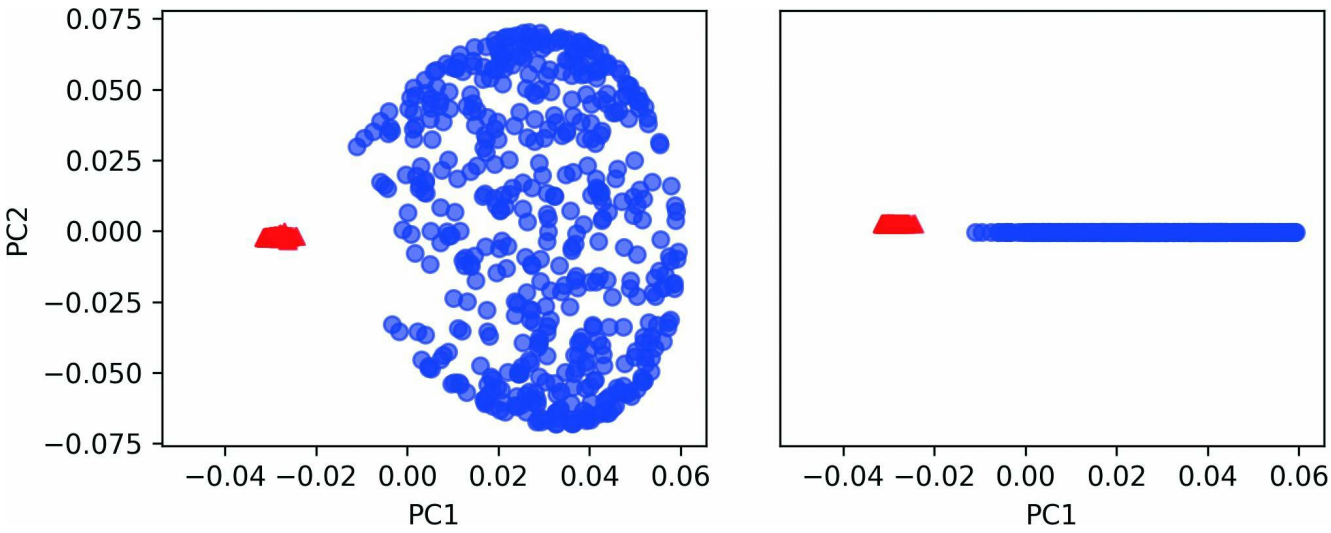
从结果中再一次看到标准PCA不能产生适合训练的线性分类器：




用一个适当的γ值看是否能幸运地用RBF核函数PCA完成分类：

```
>>> X_kpca = rbf_kernel_pca(X, gamma=15, n_components=2)
>>> fig, ax = plt.subplots(nrows=1,ncols=2, figsize=(7,3))
>>> ax[0].scatter(X_kpca[y==0, 0], X_kpca[y==0, 1],
...               color='red', marker='^', alpha=0.5)
>>> ax[0].scatter(X_kpca[y==1, 0], X_kpca[y==1, 1],
...               color='blue', marker='o', alpha=0.5)
>>> ax[1].scatter(X_kpca[y==0, 0], np.zeros((500,1))+0.02,
...               color='red', marker='^', alpha=0.5)
>>> ax[1].scatter(X_kpca[y==1, 0], np.zeros((500,1))-0.02,
...               color='blue', marker='o', alpha=0.5)
>>> ax[0].set_xlabel('PC1')
>>> ax[0].set_ylabel('PC2')
>>> ax[1].set_ylim([-1, 1])
>>> ax[1].set_yticks([])
>>> ax[1].set_xlabel('PC1')
>>> plt.show()
```

RBF核PCA再次将数据投影到新的子空间，这两个类在该空间线性可分：



### 5.3.3 投影新的数据点

在前面核PCA应用的两个半月形和同心圆例子中，将单个数据集投射到新的特征。然而，实际上要转换的数据集可能不止一个，还有训练和测试数据，以及在建模和评估之后收集到的新样本。本节将学习如何用非训练集的数据点投影。

记得本章开始时提到的标准PCA方法，通过计算变换矩阵和输入样本之间的点积来投射数据。投射矩阵的列是从协方差矩阵中得到的前k个特征向量（v）。

现在的问题是如何将这个概念转移到核主成分分析。想一下核主成分分析背后的概念，记得我们得到了中心核矩阵（不是协方差矩阵）的特征向量（a），这意味着它们已经被投射到主成分轴上。因此，如果想要投射新样本x'到主成分轴，需要做如下的计算：

$$\phi(x')^{T}v$$

幸运的是，可以使用核技巧避免具体计算投影数据（x'）$^{T}$v。然而，值得注意的是，与标准PCA相比，核PCA是基于内存的方法，这意味着每次都必须重新使用原始的训练集来创建新的样本。必须计算每个训练样本i与新样本x'之间的成对RBF核（相似度）：

$$\phi(x')^{T}v=\sum_{i}a^{(i)}\phi(x')^{T}\phi\left(x^{(i)}\right)$$

$$=\sum_{i}a^{(i)}\kappa\left(x',x^{(i)}\right)$$

在这里，核矩阵K的特征向量a和特征值λ满足以下条件的方程：

$Ka=\lambda a$

在计算新样本与训练样本之间的相似度后，必须对特征向量a进行归一化处理。因此，修改之前实现的rbf_kernel_pca函数，让它返回核矩阵的特征值：

```
from scipy.spatial.distance import pdist, squareform
from scipy import exp
from scipy.linalg import eigh
import numpy as np

def rbf_kernel_pca(X, gamma, n_components):
    """
    RBF kernel PCA implementation.

    Parameters
    ------------
    X: {NumPy ndarray}, shape = [n_samples, n_features]

    gamma: float
```

```
      Tuning parameter of the RBF kernel

    n_components: int
  Number of principal components to return

Returns
------------
 X_pc: {NumPy ndarray}, shape = [n_samples, k_features]
   Projected dataset

 lambdas: list
   Eigenvalues

"""
# Calculate pairwise squared Euclidean distances
# in the MxN dimensional dataset.
sq_dists = pdist(X, 'sqeuclidean')

# Convert pairwise distances into a square matrix.
mat_sq_dists = squareform(sq_dists)

# Compute the symmetric kernel matrix.
K = exp(-gamma * mat_sq_dists)

# Center the kernel matrix.
N = K.shape[0]
one_n = np.ones((N,N)) / N
K = K - one_n.dot(K) - K.dot(one_n) + one_n.dot(K).dot(one_n)

# Obtaining eigenpairs from the centered kernel matrix
# scipy.linalg.eigh returns them in ascending order
eigvals, eigvecs = eigh(K)
eigvals, eigvecs = eigvals[::-1], eigvecs[:, ::-1]

# Collect the top k eigenvectors (projected samples)
alphas = np.column_stack((eigvecs[:, i]
                          for i in range(n_components)))

# Collect the corresponding eigenvalues
lambdas = [eigvals[i] for i in range(n_components)]

return alphas, lambdas
```

现在创建一个新的半月数据集，用更新后的RBF核PCA将其投影到一维子空间：

```
>>> X, y = make_moons(n_samples=100, random_state=123)
>>> alphas, lambdas = rbf_kernel_pca(X, gamma=15, n_components=1)
```

为了确保实现代码投影新样本，假设半月形数据集的第26个样本是一个新数据点x'，任务是把该数据点投射到新的子空间。

```
>>> x_new = X[25]
>>> x_new
array([ 1.8713187 ,  0.00928245])
>>> x_proj = alphas[25] # original projection
>>> x_proj
array([ 0.07877284])
>>> def project_x(x_new, X, gamma, alphas, lambdas):
...     pair_dist = np.array([np.sum(
...                     (x_new-row)**2) for row in X])
...     k = np.exp(-gamma * pair_dist)
...     return k.dot(alphas / lambdas)
```

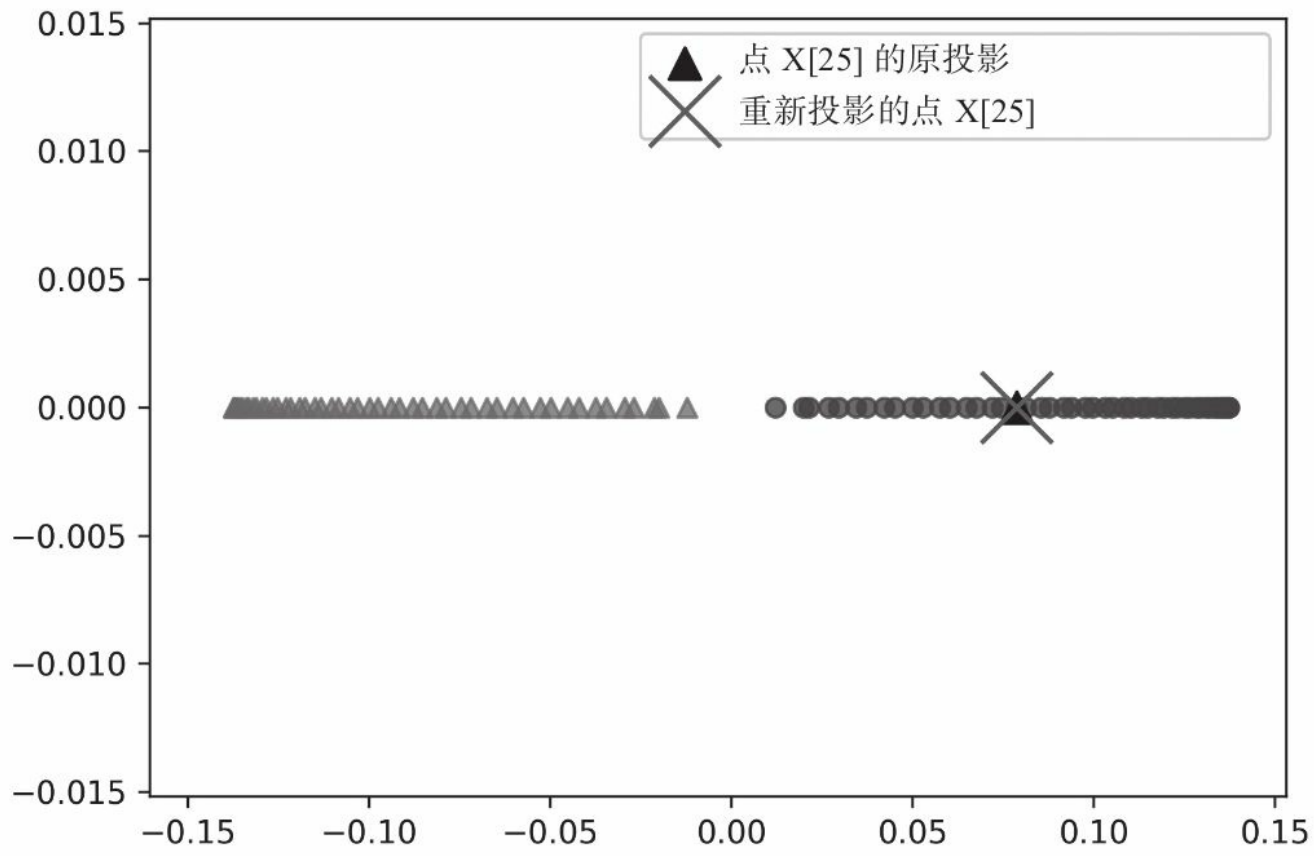
执行下面的代码能够重现原始投影。调用project_x函数，可以投射任何新的数据样本：

```
>>> x_reproj = project_x(x_new, X,
...       gamma=15, alphas=alphas, lambdas=lambdas)
>>> x_reproj
array([ 0.07877284])
```

最后实现在第一个主成分上投影的可视化：

```
>>> plt.scatter(alphas[y==0, 0], np.zeros((50)),
...             color='red', marker='^',alpha=0.5)
>>> plt.scatter(alphas[y==1, 0], np.zeros((50)),
...             color='blue', marker='o', alpha=0.5)
>>> plt.scatter(x_proj, 0, color='black',
...             label='original projection of point X[25]',
...             marker='^', s=100)
>>> plt.scatter(x_reproj, 0, color='green',
...             label='remapped point X[25]',
...             marker='x', s=500)
>>> plt.legend(scatterpoints=1)
>>> plt.show()
```

现在可以看到下面的散点图将样本x'正确地映射到第一主成分：


点 X[25] 的原投影
重新投影的点 X[25]
0.015
0.010
0.005
0.000
−0.005
−0.010
−0.015
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15

### 5.3.4 scikit-learn的核主成分分析

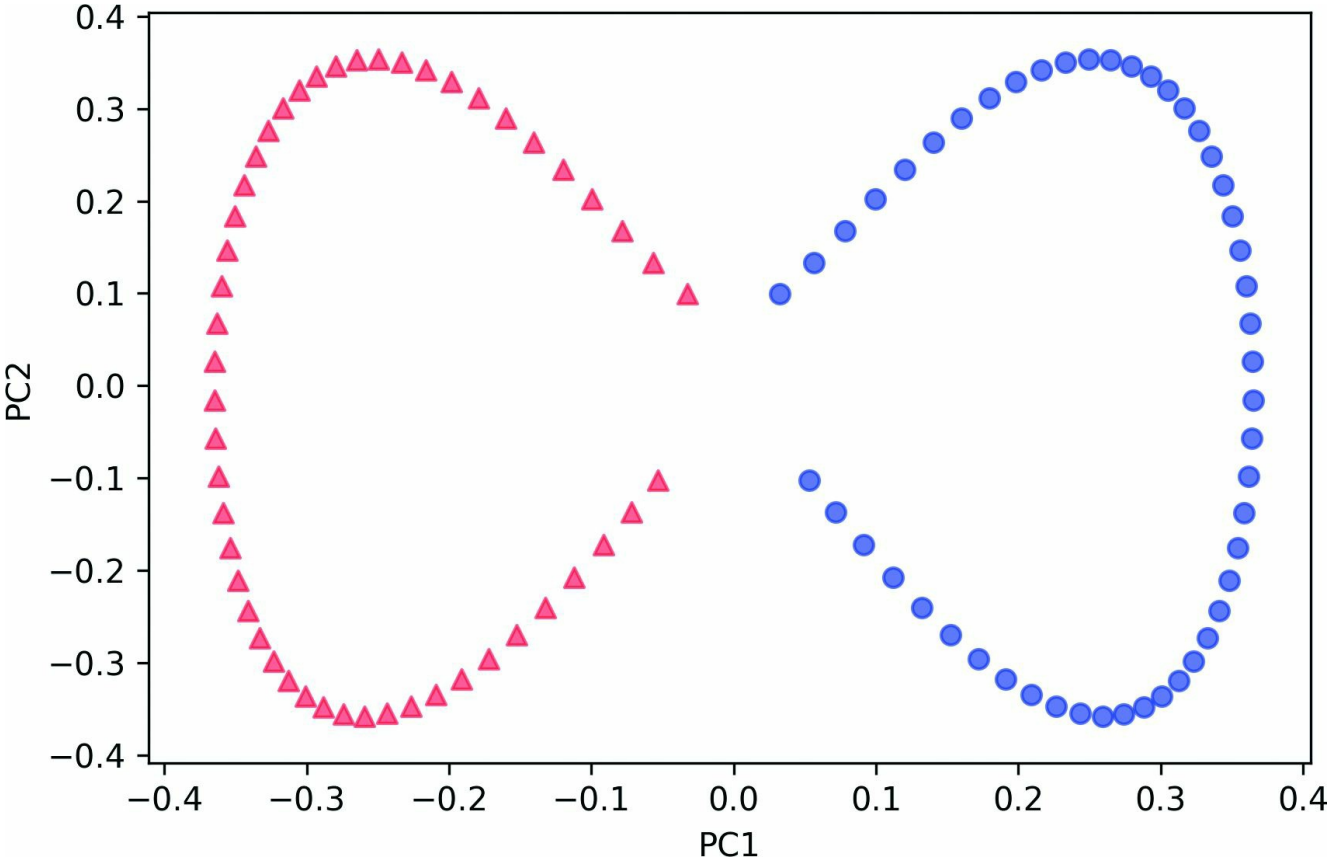
为方便起见，scikit-learn在sklearn.decomposition子模块中实现了核PCA类。用法类似于标准PCA类，可以通过kernel参数来指定：

```
>>> from sklearn.decomposition import KernelPCA
>>> X, y = make_moons(n_samples=100, random_state=123)
>>> scikit_kpca = KernelPCA(n_components=2,
...                         kernel='rbf', gamma=15)
>>> X_skernpca = scikit_kpca.fit_transform(X)
```

为了验证所得的结果与自己实现的核PCA一致，将转换后的半月形数据绘制到前两个主成分上：

```
>>> plt.scatter(X_skernpca[y==0, 0], X_skernpca[y==0, 1],
...             color='red', marker='^', alpha=0.5)
>>> plt.scatter(X_skernpca[y==1, 0], X_skernpca[y==1, 1],
...             color='blue', marker='o', alpha=0.5)
>>> plt.xlabel('PC1')
>>> plt.ylabel('PC2')
>>> plt.show()
```

可以看到scikit-learn的KernelPCA的结果与我们自己动手实现的一致：




scikit-learn还实现了非线性降维的高级技术，但是这部分内容超出了本书的范围。感兴趣的读者可以从下述网站了解更多关于目前scikit-learn实现的精彩概括，也包括案例讲解：

http://scikit-learn.org/stable/modules/manifold.html

## 5.4 小结

本章学习了用于特征提取降维的三种不同的基本技术：标准PCA、LDA和核PCA。PCA将数据投影到低维子空间，忽略分类标签，沿着正交特征轴最大化方差。与PCA相反，LDA是一种有监督的降维技术，这意味着它考虑了训练集中的分类信息，试图在线性特征空间中最大化类的可分性。

最后，了解了非线性特征提取器核PCA。用核技巧和临时投射到更高维度特征空间的方法，最终能够把非线性特征组成的数据集压缩到低维子空间，这些类在这里线性可分。

掌握了这些必要的数据预处理技术，现在你已经为在下一章了解有效地结合不同的预处理技术并评估不同模型性能的最佳实践做好了准备。

# 第6章　模型评估和超参数调优的最佳实践

前几章学习了基本的机器学习分类算法，以及在把数据输入到算法之前如何预处理。现在学习通过调优算法和评估模型性能来构建良好的机器学习模型的最佳实践。

本章将主要涵盖下述几个方面：

·获得对模型性能的无偏差评估

·诊断机器学习算法的常见问题

·微调机器学习模型

·使用不同的指标评估预测模型的性能

## 6.1 用管道方法简化工作流

在前面的章节中，如第4章的特征比例标准化，或者第5章通过主成分分析数据压缩，曾经讨论过当采用不同的预处理技术时，需要复用拟合训练数据获得的参数来调整和压缩新数据，如不同测试集的样本。这一部分将介绍一个非常方便的工具，scikit-learn的Pipeline类。可以拟合任意多个转换步骤的模型，并以此对新数据进行预测。

### 6.1.1 加载威斯康星乳腺癌数据集

本章将研究威斯康星乳腺癌数据集，其中包含569个恶性和良性肿瘤细胞的样本。数据集的前两列分别存储样本的唯一ID和相应的诊断结果（M=恶性，B=良性）。列3-32包含30个根据细胞核的数字化图像计算出的特征值，可用来建立模型预测肿瘤是良性还是恶性。威斯康星乳腺癌数据集保存在UCI机器学习存储库，可以从下述网址获得有关该数据集的更详细信息：

https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic)

可以从本书的代码包中找到乳腺癌数据集（和本书用到的所有其他数据集），当离线工作或UCI服务器暂时宕机时，可从下述网址获得数据集：

https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/wdbc.data

可以执行下面的代码从本地目录加载葡萄酒数据集：

```
df = pd.read_csv('https://archive.ics.uci.edu/ml/'
                 'machine-learning-databases'
                 '/breast-cancer-wisconsin/wdbc.data',
                 header=None)
```

可以用下述代码替代前面的代码片段：

```
df = pd.read_csv('your/local/path/to/wdbc.data',
                 header=None)
```

本节将用pandas直接从UCI网站读入数据集，然后用三个简单的步骤将其分裂成训练集和测试集：

1.从UCI网站直接读入数据集：

```
>>> import pandas as pd
>>> df = pd.read_csv('https://archive.ics.uci.edu/ml/'
...                  'machine-learning-databases'
...                  '/breast-cancer-wisconsin/wdbc.data',
                     header=None)
```

2.接着把30个特征分配给NumPy阵列x。利用LabelEncoder对象将分类标签从原来的字符串型（'M'和'B'）转换为整数型：

```
>>> from sklearn.preprocessing import LabelEncoder

>>> X = df.loc[:, 2:].values
>>> y = df.loc[:, 1].values
>>> le = LabelEncoder()
>>> y = le.fit_transform(y)
>>> le.classes_
array(['B', 'M'], dtype=object)
```

在阵列y中对分类标签（诊断）编码之后，恶性肿瘤用1类代表，良性肿瘤用0类代表。通过调用LabelEncoder的transform方法对两个虚拟分类标签的映射再次进行检查：

```
>>> le.transform(['M', 'B'])
array([1, 0])
```

3.在下一小节构建第一个模型管道之前，先按照8：2的比例把数据分成独立的训练集和测试集：

```
>>> from sklearn.model_selection import train_test_split

>>> X_train, X_test, y_train, y_test = \
>>>     train_test_split(X, y,
...                      test_size=0.20,
...                      stratify=y,
...                      random_state=1)
```

### 6.1.2 集成管道中的转换器和评估器

前一章了解到许多机器学习算法要求输入的特征数据有相同的比例以获得最佳性能。因此，需要标准化威斯康星乳腺癌数据集中的列，然后才能将其输入到像逻辑回归这样的线性分类器。此外，假设希望通过主成分分析（PCA）把初始30维的数据压缩到较低的二维子空间，PCA是第5章已经介绍了的降维特征提取技术。

与其在训练集和测试集分别完成拟合和转换的步骤，不如把StandardScaler、PCA和LogisticRegression在管道中链接起来：

```
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.decomposition import PCA
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.pipeline import make_pipeline
>>> pipe_lr = make_pipeline(StandardScaler(),
...                         PCA(n_components=2),
...                         LogisticRegression(random_state=1))
>>> pipe_lr.fit(X_train, y_train)
>>> y_pred = pipe_lr.predict(X_test)
>> print('Test Accuracy: %.3f' % pipe_lr.score(X_test, y_test))
Test Accuracy: 0.956
```

make_pipeline函数可以包括任意多个scikit-learn转换器（支持fit和transform方法作为输入的对象），后接实现fit及predict方法的scikit-learn评估器。前面的代码示例提供了StandardScaler和PCA两个转换器以及逻辑回归评估器作为make_pipeline函数的输入，然后以这些对象为基础构建scikit-learn的Pipeline对象。

可以把scikit-learn的Pipeline想象成一种元评估器，或者是独立转换器和评估器的封装。如果调用Pipeline的fit方法，数据将通过在中间步骤调用fit和transform方法完成一系列转换器的传递，直至到达评估对象（管道中的最后一个元素）为止。然后用评估器来拟合转换后的训练数据。

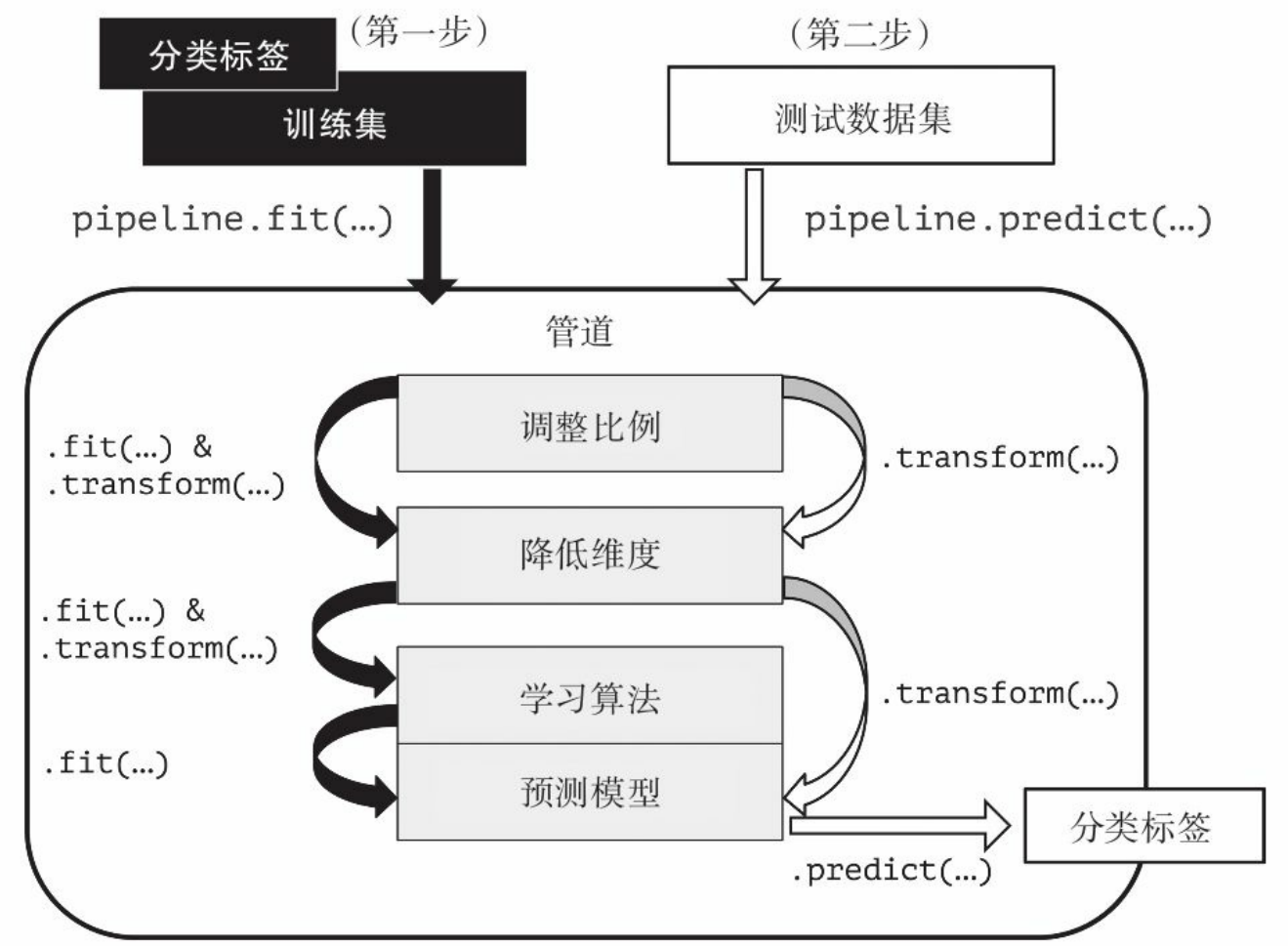
在执行前面示例代码中的pipe_lr管道的fit方法时，StandardScaler首先在训练集上调用fit和transform方法。然后将转换后的训练数据传递给管道的下一个对象，即PCA。与前面的步骤类似，PCA也在调整后的输入数据基础上调用fit和transform，并将其传递给管道中的最后一个环节，即评估器。

最后，训练数据通过调用StandardScaler和PCA完成转换后，逻辑回归评估器完成拟合。应该再次注意到管道的中间步骤没有数量限制，但是，管道的最后一个元素必须是评估器。

类似于在管道上调用fit，管道也实现predict方法。如果将数据集输入到

管道对象实例调用predict，数据将通过中间步骤调用transform完成转换。评估对象在最后一步将返回针对转换后数据的预测结果。

scikit-learn的管道是非常有用的封装工具，本书的其余部分将经常用到。为了确保掌握管道对象的工作机制，请仔细观察下图，它总结了前面段落讨论过的内容：



## 6.2 使用k折交叉验证评估模型的性能

建立机器学习模型的关键步骤之一是在模型未见过的数据上评估其性能。假设模型适合于某个训练集，并用相同的数据来评估它在新数据上的表现。还记得在第3章通过正则化解决过拟合的讨论吗？如果模型过于简单，可能是欠拟合（高偏差），如果模型过于复杂，可能是过拟合（高方差）。

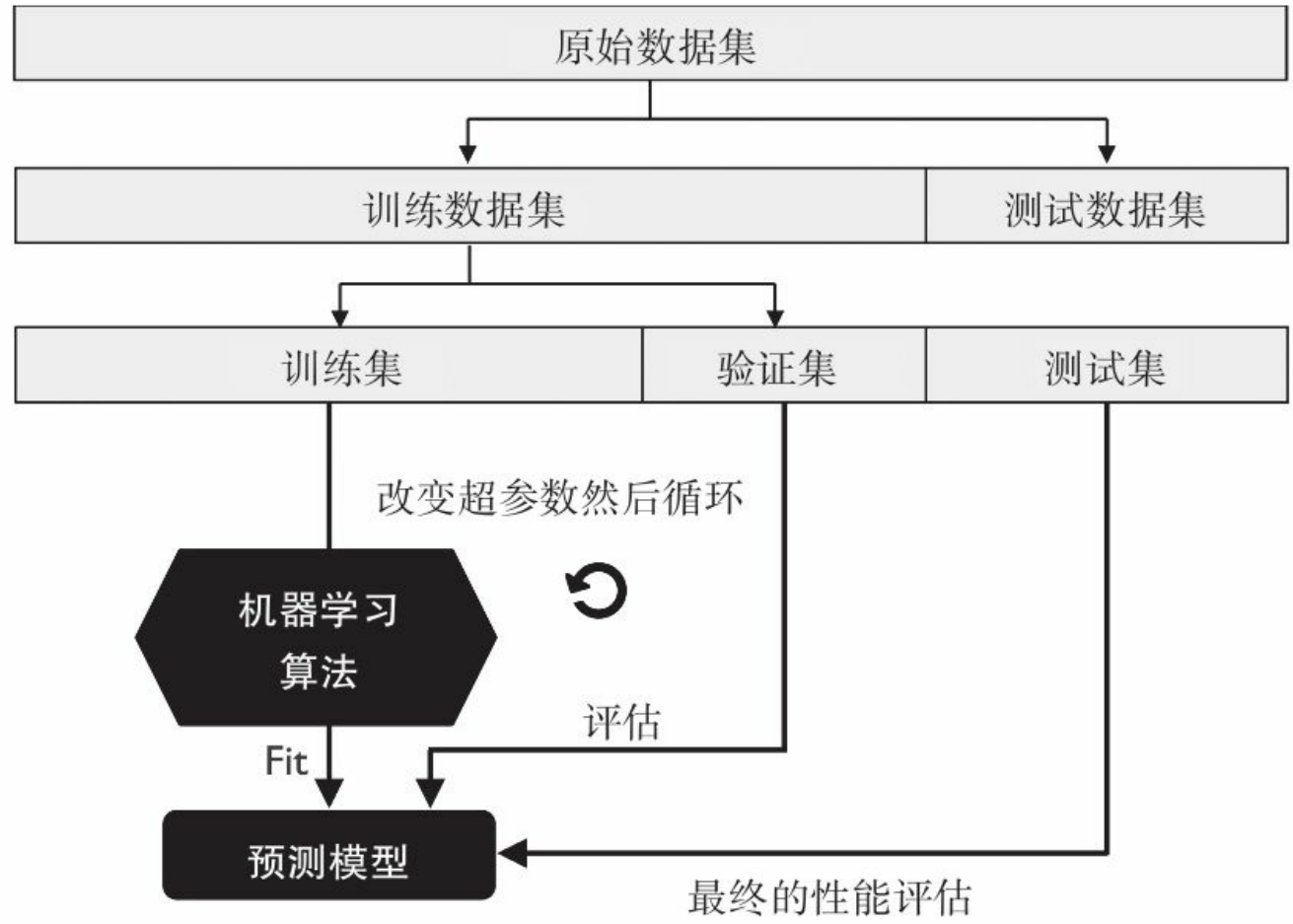
需要小心地评估模型以找到一个可接受的偏差与方差的平衡点。本节将介绍常见的两种验证技术，即抵抗交叉验证和k折交叉验证，它有助于获得对模型泛化性能的可靠评估，即模型对未见过的数据的表现。

### 6.2.1 抵抗方法

抵抗交叉验证是评估机器学习模型泛化性能的经典和常用方法。抵抗验证方法把初始数据集分裂成独立的训练集和测试集，前者用于训练模型，后者用来评估模型的泛化性能。然而，典型的机器学习应用也对调整和比较不同的参数设置感兴趣，目的是进一步提高对未见过数据的预测性能。该过程也被称为模型选择，模型选择指的是对给定的分类问题，选择最优的参数值（也称为超参数）。然而，如果在模型选择时反复使用相同的测试集，它将成为训练数据的一部分，这样更容易引起模型的过拟合。尽管存在着这个问题，许多人仍然使用测试集进行模型选择，这是机器学习的一个不良实践。

模型选择更好的方式是将数据分裂为训练集、验证集和测试集。训练集用于拟合不同的模型，验证集用于在模型选择过程中验证性能。让模型使用在训练和选择步骤中没见过的测试集的好处是，可以对该模型面对新数据的泛化能力有不太偏颇的评估。下图说明了抵抗交叉验证的概念，用不同的参数对模型进行训练之后，在验证集上反复评估模型的性能。一旦对调优的超参数值感到满意，就开始评估模型在测试集上的泛化性能：

抵抗方法的缺点是性能评估可能会对如何把训练集分裂成训练子集和验证子集非常敏感，评估结果会随不同数据样本而变化。下一节将讨论更强大的性能评估技术，即k折交叉验证，即在训练数据的k个子集上反复使用k次抵抗方法。


原始数据集
训练数据集
测试数据集
训练集
验证集
测试集
改变超参数然后循环
机器学习
算法
评估
Fit
预测模型
最终的性能评估

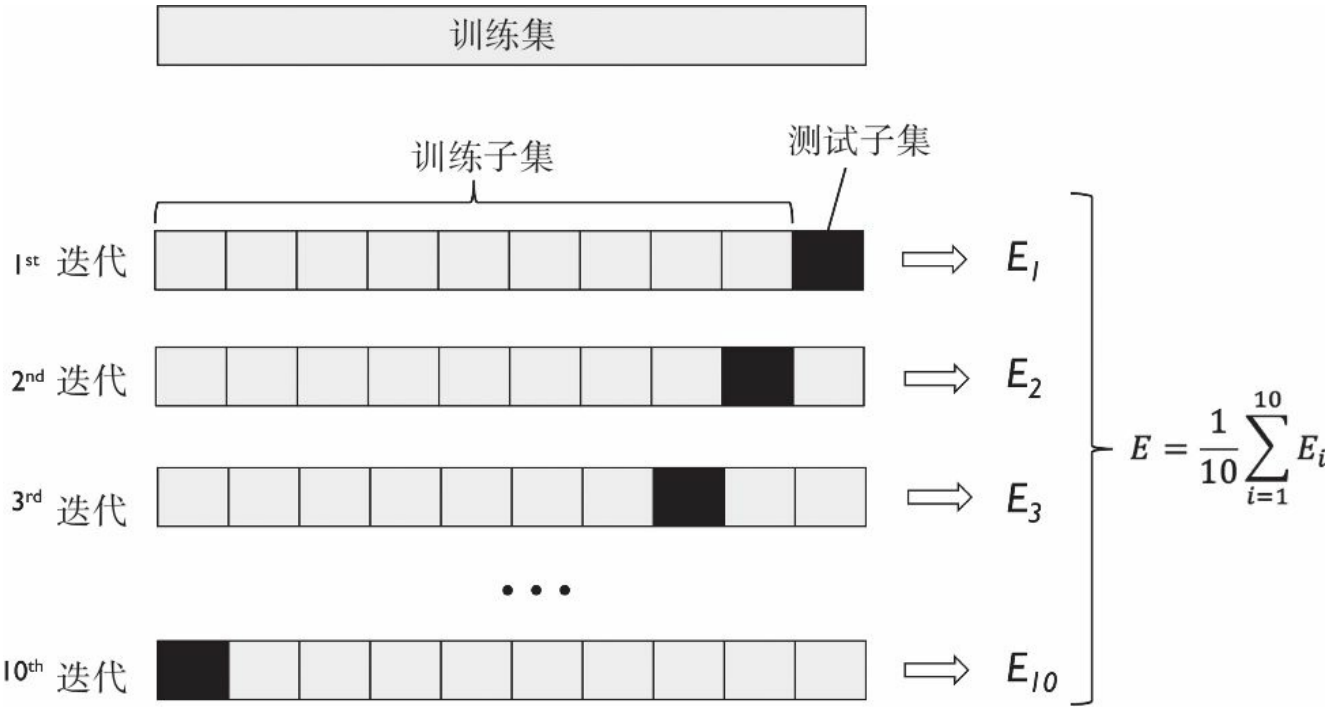
## 6.2.2 k折交叉验证

k折交叉验证将训练集随机分裂成k个无更换子集，其中k–1个子集用于模型训练，一个子集用于性能评估。重复该过程k次，得到k个模型和k次性能估计。

第3章通过实例解释了什么叫作有替换，什么叫作无替换。如果没有读到该章或想要回顾，请参考第3章通过随机森林结合多种决策树中的信息栏。

接着计算基于不同且独立的数据子集模型的平均性能，以获得与抵抗方法相比对训练子集不太敏感的性能评估。通常用k折交叉验证为模型调优，即寻找最优的超参数值，以获得令人满意的综合性能。

由于k折交叉验证属于无替换的重新采样技术，其优点在于每个采样点仅用于训练和验证（如测试子集部分）一次，而对模型性能的评估将产生比抵抗方法更低的方差。下图总结了k折交叉验证（k=10）的逻辑。把训练集分裂为10个子集，9个子集用于训练模型，一个子集用于评估模型。此外，用模型在每个子集上的性能评估结果Ei（例如，分类准确度或错误）来计算模型的平均性能评估结果E：




实践证明k折交叉验证的参数k的最优标准值为10。例如，让·高海威通过对各种现实世界的数据集的实验表明，10折交叉验证提供了在偏差和方差之间的最佳平衡（让·高海威.《交叉验证和准确度评估与模型选择导引研

究》.人工智能国际联合大会（IJCAI），1995，14（12）：1137-43）。

然而，如果训练集的规模相对较小，增加分区可能有益。加大k值，每次迭代会有更多的训练数据，结果用于评估泛化性能的偏差值比较小，偏差值是每个模型评估值的平均。然而，较大的k值也会增加交叉验证算法的计算时间，因为训练子集彼此的相似度更高，所以产生评估值的偏差也比较高。另外，如果数据集比较大，可以选择较小的k值，例如k=5，这样做仍然可以完成对模型平均性能的准确评估，同时降低对不同子集模型反复拟合和评估的计算成本。

k折交叉验证的特例是留存交叉验证（LOOCV）方法。该方法把k值设置为训练子集的样本数（k=n），这样每个迭代只有一个训练子集用于测试，对规模非常小的数据集，我们推荐使用该方法。

分层k折交叉验证略微改善了k折交叉验证方法，所产生的评估偏差和方差比较低，特别是在类属性分布不匀的情况下，正如让·高海威的研究所示（《交叉验证和导引方法在准确度估计和模型选择中的应用研究》，国际人工智能联合大会（IJCAI），1995，14（12）：1137-43）。在分层交叉验证方法中，每个分区都保存类的比例以确保分区。

```
>>> import numpy as np
>>> from sklearn.model_selection import StratifiedKFold

>>> kfold = StratifiedKFold(n_splits=10,
...                         random_state=1).split(X_train,
...                                               y_train)
>>> scores = []
>>> for k, (train, test) in enumerate(kfold):
...     pipe_lr.fit(X_train[train], y_train[train])
...     score = pipe_lr.score(X_train[test], y_train[test])
...     scores.append(score)
...     print('Fold: %2d, Class dist.: %s, Acc: %.3f' % (k+1,
...           np.bincount(y_train[train]), score))
Fold:  1, Class dist.: [256 153], Acc: 0.935
Fold:  2, Class dist.: [256 153], Acc: 0.935
Fold:  3, Class dist.: [256 153], Acc: 0.957
Fold:  4, Class dist.: [256 153], Acc: 0.957
Fold:  5, Class dist.: [256 153], Acc: 0.935
Fold:  6, Class dist.: [257 153], Acc: 0.956
Fold:  7, Class dist.: [257 153], Acc: 0.978
Fold:  8, Class dist.: [257 153], Acc: 0.933
Fold:  9, Class dist.: [257 153], Acc: 0.956
Fold: 10, Class dist.: [257 153], Acc: 0.956

>>> print('\nCV accuracy: %.3f +/- %.3f' %
...       (np.mean(scores), np.std(scores)))
CV accuracy: 0.950 +/- 0.014
```

首先用sklearn.model_selection模块，在以y_train为分类标签的训练集上初始化Stratifiedkfold迭代器，通过参数n_splits指定分区数量。当kfold迭代器遍历k个分区时，用train返回的结果拟合在本章开头建立的逻辑回归训练管道。用pipe_lr管道来确保每次迭代的样本比例尺寸适当（例如标准化）。然后，用test指标来计算模型的准确度得分，并把这些得分存入scores表，以计算评估平均准确度和标准偏差。

前面的代码示例对解释k折交叉验证工作机制很有用，scikit-learn也实现了k折交叉验证得分器，这样可以用分层交叉验证方法更简洁地评估模型：

```
>>> from sklearn.model_selection import cross_val_score

>>> scores = cross_val_score(estimator=pipe_lr,
...                          X=X_train,
...                          y=y_train,
...                          cv=10,
...                          n_jobs=1)
>>> print('CV accuracy scores: %s' % scores)
CV accuracy scores: [ 0.93478261  0.93478261  0.95652174
                      0.95652174  0.93478261  0.95555556
                      0.97777778  0.93333333  0.95555556
                      0.95555556]
>>> print('CV accuracy: %.3f +/- %.3f' % (np.mean(scores),
...       np.std(scores)))
CV accuracy: 0.950 +/- 0.014
```

cross_val_score方法极为有用的功能是可以把不同分区的评估任务分给计算机的多个CPU。假设把n_jobs设为1，只有一个CPU会用于性能评估，就像前面StratifiedKFold示例展示的那样。然而，如果设置n_jobs=2，可以把10轮交叉验证任务分给两个CPU来完成（如果系统有那么多CPU的话），如果设置n_jobs=-1，可以用计算机上所有可用的CPU同时进行计算。

请注意，虽然关于如何用交叉验证评估泛化性能的详细讨论超出了本书的范围，但是我曾经发表过一系列关于模型评估和交叉验证方面的更详细深入的文章。可以从下列网页链接找到这些文章：

·https://sebastianraschka.com/blog/2016/model-evaluation-selection-part1.html

·https://sebastianraschka.com/blog/2016/model-evaluation-selection-part2.html

·https://sebastianraschka.com/blog/2016/model-evaluation-selection-part3.html

另外，也可以从由M.玛咖托和其他作者共同完成的优秀文章中找到更详细的讨论（M.玛咖托，H.田，S.碧斯瓦斯，G.M.里波塞可.《泛化错误交叉验证评估器的方差分析》机器学习研究，2005，第6期：1127-1168）。

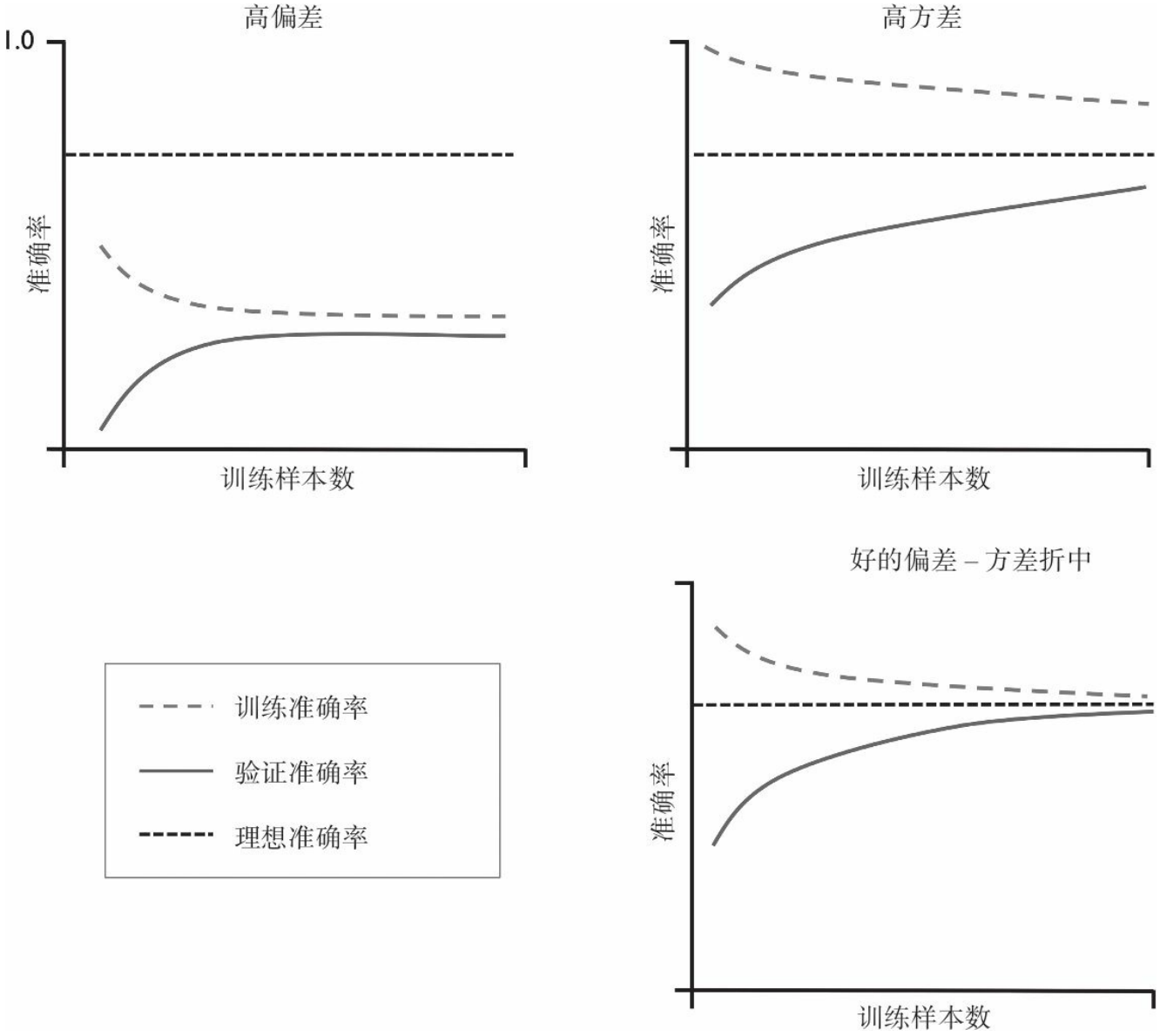
你也可以从书中读到类似.632交叉验证方法的其他交叉验证方法（B.额佛朗和R.替波施蓝尼.《对交叉验证的完善：.632+导引方法》.1997，92（438）：548-560）。

## 6.3 用学习和验证曲线调试算法

本节将讨论学习曲线和验证曲线两种简单且强大的调试工具，助力于改善机器学习算法的性能。下一小节将讨论如何用学习曲线来判别学习算法是否过拟合（大方差）或者欠拟合（高偏差）。我们将会深入讨论有助于解决机器学习算法共性问题的验证曲线。

### 6.3.1 用学习曲线诊断偏差和方差问题

如果模型对于训练集过于复杂，亦即模型中有太多的自由度或者参数，就会有过拟合训练数据的倾向，因而对未见过的数据泛化不良。通常，收集更多的训练样本有助于缓解过拟合。然而，实践中收集更多数据往往代价高昂或者根本就不可能。通过绘制模型训练和验证的准确度与训练集规模之间的关系图，可以很容易地发现模型是否遇到高偏差或者高方差的问题，以及收集更多的数据是否有助于解决问题。在讨论用scikit-learn绘制学习曲线之前，先通过解释下图讨论模型中常见的两个问题：



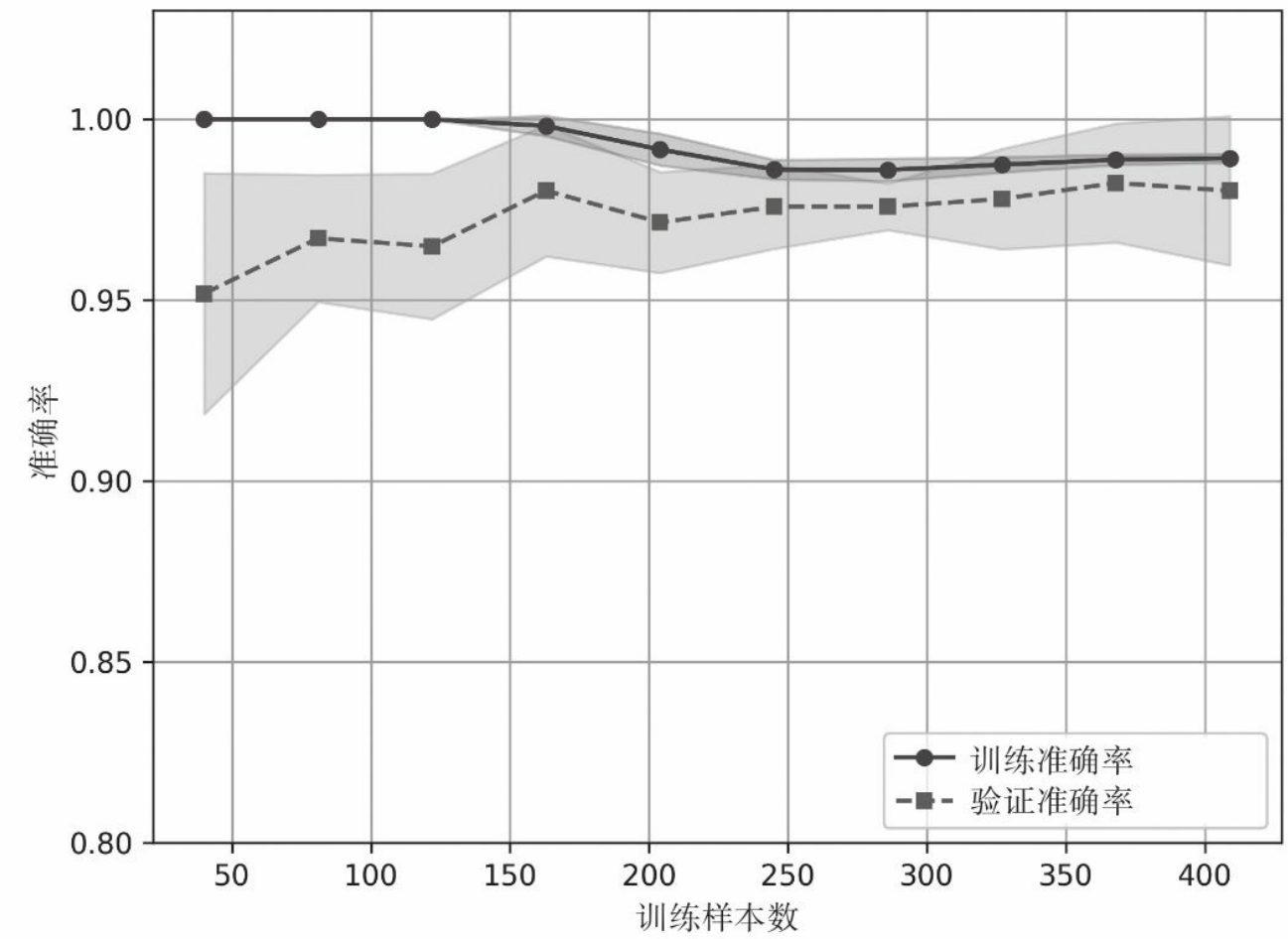

左上图说明模型遇到高偏差问题。该模型的训练和交叉验证准确度均低，这说明模型对训练数据欠拟合。解决该问题的常用办法是增加模型的参数个数，例如，通过收集或构建额外的特征，或者降低正则化的程度，就像在SVM或逻辑回归分类器所做的那样。

右上图说明模型遇到高方差问题，表现是模型在训练和交叉验证的准确度上有比较大的差别。要解决过拟合的问题，可以收集更多的训练数据，减少模型的复杂度，或者增大正则化的参数等。对于非正则化的模型，也有助于通过特征选择或者特征提取（第5章）减少特征的数量（第4章），从而降低过拟合的程度。尽管收集更多的训练数据是为了减少过拟合的机会，但这么做可能并非屡屡奏效，例如，如果训练数据的噪音极大或模型已经非常接近最优的情况下。

下一节将看到如何用验证曲线来解决模型的问题，在此之前先来看看如何利用scikit-learn的学习曲线来评估模型：

```
>>> import matplotlib.pyplot as plt
>>> from sklearn.model_selection import learning_curve

>>> pipe_lr = make_pipeline(StandardScaler(),
...                         LogisticRegression(penalty='l2',
...                                            random_state=1))
>>> train_sizes, train_scores, test_scores =\
...                 learning_curve(estimator=pipe_lr,
...                                X=X_train,
...                                y=y_train,
...                                train_sizes=np.linspace(
...                                  0.1, 1.0, 10),
...                                cv=10,
...                                n_jobs=1)
>>> train_mean = np.mean(train_scores, axis=1)
>>> train_std = np.std(train_scores, axis=1)
>>> test_mean = np.mean(test_scores, axis=1)
>>> test_std = np.std(test_scores, axis=1)

>>> plt.plot(train_sizes, train_mean,
...          color='blue', marker='o',
...          markersize=5, label='training accuracy')

>>> plt.fill_between(train_sizes,
...                  train_mean + train_std,
...                  train_mean - train_std,
...                  alpha=0.15, color='blue')

>>> plt.plot(train_sizes, test_mean,
...          color='green', linestyle='--',
...          marker='s', markersize=5,
...          label='validation accuracy')

>>> plt.fill_between(train_sizes,
...                  test_mean + test_std,
...                  test_mean - test_std,
...                  alpha=0.15, color='green')
>>> plt.grid()
>>> plt.xlabel('Number of training samples')
>>> plt.ylabel('Accuracy')
>>> plt.legend(loc='lower right')
>>> plt.ylim([0.8, 1.0])
>>> plt.show()
```

成功执行前面的代码后可以得到以下的学习曲线图：




可以通过learning_curve函数的参数train_sizes控制用于产生学习曲线的训练样本的绝对或相对数量。这里，通过设置train_sizes=np.linspace（0.1，1.0，10）把训练数据子集之间用10个均匀的间隔分开。默认情况下，learning_curve函数采用分层k折交叉验证来计算分类交叉验证的准确度，通过参数cv设置k=10来实现10折分层交叉验证。然后，根据交叉验证后返回的训练和测试分数直接计算不同规模训练集的平均准确度，最后用Matplotlib的plot函数绘图。此外，用fill_between函数把平均准确度的标准方差加在图上以表示对模型评估的方差。

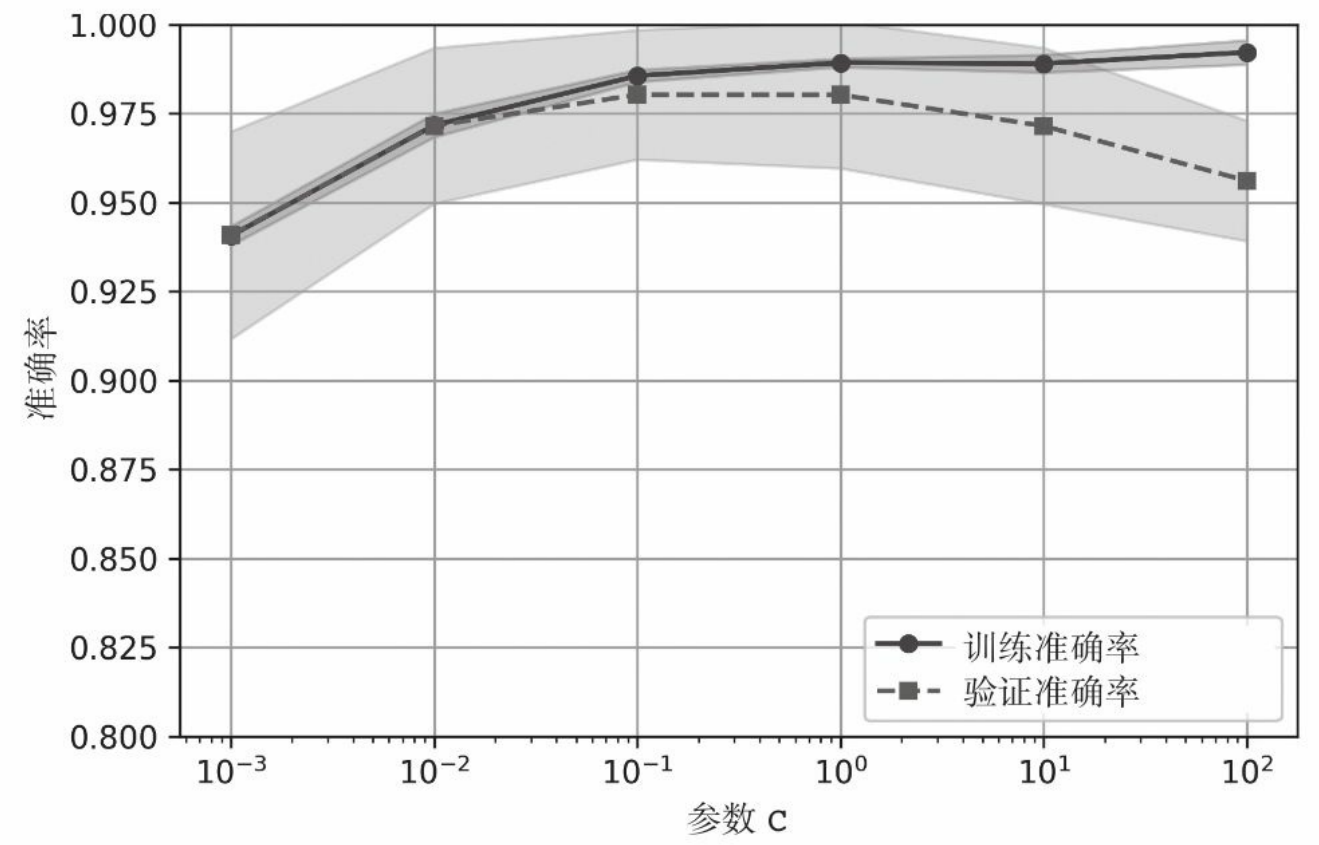
正如在前面的学习曲线图中所看到的，如果模型在训练中见过250多个样本，那么该模型在训练和验证数据集上表现均好。可以看到，对于样本规模少于250的训练集，模型的训练准确度提高，验证和训练准确度之间的差距扩大，是过拟合程度越来越大的标志。

## 6.3.2 用验证曲线解决过拟合和欠拟合问题

验证曲线是通过解决过拟合和欠拟合问题来提高模型性能的有力工具。虽然验证曲线与学习曲线相关，但是我们并不根据训练和测试准确度与样本规模之间的函数关系来绘图，而是通过调整模型参数来调优，例如逻辑回归中的逆正则化参数C。看看如何用scikit-learn来产生验证曲线：

```
>>> from sklearn.model_selection import validation_curve
>>> param_range = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
>>> train_scores, test_scores = validation_curve(
...                 estimator=pipe_lr,
...                 X=X_train,
...                 y=y_train,
...                 param_name='logisticregression__C',
...                 param_range=param_range,
...                 cv=10)
>>> train_mean = np.mean(train_scores, axis=1)
>>> train_std = np.std(train_scores, axis=1)
>>> test_mean = np.mean(test_scores, axis=1)
>>> test_std = np.std(test_scores, axis=1)
>>> plt.plot(param_range, train_mean,
...          color='blue', marker='o',
...          markersize=5, label='training accuracy')
>>> plt.fill_between(param_range, train_mean + train_std,
...                  train_mean - train_std, alpha=0.15,
...                  color='blue')
>>> plt.plot(param_range, test_mean,
...          color='green', linestyle='--',
...          marker='s', markersize=5,
...          label='validation accuracy')
>>> plt.fill_between(param_range,
...                  test_mean + test_std,
...                  test_mean - test_std,
...                  alpha=0.15, color='green')
>>> plt.grid()
>>> plt.xscale('log')
>>> plt.legend(loc='lower right')
>>> plt.xlabel('Parameter C')
>>> plt.ylabel('Accuracy')
>>> plt.ylim([0.8, 1.03])
>>> plt.show()
```

采用前面的代码可以得到参数C的验证曲线图：




与learning_curve相似，validation_curve函数默认采用分层的k折交叉验证来评估分类器的性能。这种情况下在validation_curve函数里定义想要评估的参数。用逻辑回归分类器的逆正则化参数C（即'logisticregression__C'）访问scikit-learn管道中的LogisticRegression对象，通过调用参数param_range定义值域。与前面的学习曲线示例类似，该方法也可以绘出训练和交叉平均准确率以及相应的标准方差。

尽管准确率与不同的C值之间的差别关系很微妙，当提高正则化强度（小的C值）时，可以看到模型拟合数据略显不足。然而，对于大的C值，这意味着降低了正则化强度，所以模型往往会略显过拟合。因此，最佳C值显然是在0.01到0.1之间。

## 6.4 通过网格搜索为机器学习模型调优

机器学习有两类参数：一类是从训练数据中学习到的参数，例如逻辑回归的权重；另一类是单独优化的算法参数。后者为模型的调优参数，也被称为超参数，例如逻辑回归的正则化参数或者决策树的深度参数。

上一节验证曲线通过调整超参数来提高模型的性能。这一节将介绍一种被称为网格搜索的常见超参数调优技术，该技术可以通过寻找最优超参数组合来进一步提高模型的性能。

### 6.4.1 通过网格搜索为超参数调优

网格搜索方法的逻辑非常简单，属于暴力穷尽搜索类型，预先定义好不同的超参数值，然后让计算机针对每种组合分别评估模型的性能，从而获得最佳组合参数值。

```
>>> from sklearn.model_selection import GridSearchCV
>>> from sklearn.svm import SVC

>>> pipe_svc = make_pipeline(StandardScaler(),
...                          SVC(random_state=1))
>>> param_range = [0.0001, 0.001, 0.01, 0.1,
...                1.0, 10.0, 100.0, 1000.0]
>>> param_grid = [{'svc__C': param_range,
...                'svc__kernel': ['linear']},
...               {'svc__C': param_range,
...                'svc__gamma': param_range,
...                'svc__kernel': ['rbf']}]

>>> gs = GridSearchCV(estimator=pipe_svc,
...                   param_grid=param_grid,
...                   scoring='accuracy',
...                   cv=10,
...                   n_jobs=-1)
>>> gs = gs.fit(X_train, y_train)
>>> print(gs.best_score_)
0.984615384615
>>> print(gs.best_params_)
{'svc__C': 100.0, 'svc__gamma': 0.001, 'svc__kernel': 'rbf'}
```

用前面的代码从sklearn.model_selection中初始化GridSearchCV对象来训练和优化支持向量机（SVM）管道。把GridSearchCV的参数param_grid设置为字典的列表来定义想要调优的各个参数。对于线性支持向量机，只评估逆正则化参数C；对于RBF核支持向量机，则对参数svc__c和svc__gamma都调优。请注意参数svc__gamma只适用于核SVM。

用训练数据进行网格搜索后，通过调用best_score_的属性获得表现最佳模型的分数，通过调用best_params_的属性可以访问该模型的参数。在该案例中，RBF核SVM模型以svc_C=100.0得到了最佳的k折交叉验证准确率：98.5%。

最后，用独立的测试集评估所选最佳模型的性能，并通过设置GridSearchCV对象的best_estimator_属性实现：

```
>>> clf = gs.best_estimator_
>>> clf.fit(X_train, y_train)
>>> print('Test accuracy: %.3f' % clf.score(X_test, y_test))
Test accuracy: 0.974
```

尽管网格搜索是寻找模型最佳参数组合的有力手段，从概念上说，评估所有可能的参数组合在计算成本上非常昂贵。另外一种用scikit-learn从不同的参数组合中抽样的方法是随机搜索。用scikit-learn的RandomizedSearchCV类，可以根据预算从样本分布中随机抽取不同的参数组合。从下述网站能找到更多更详细的例子：

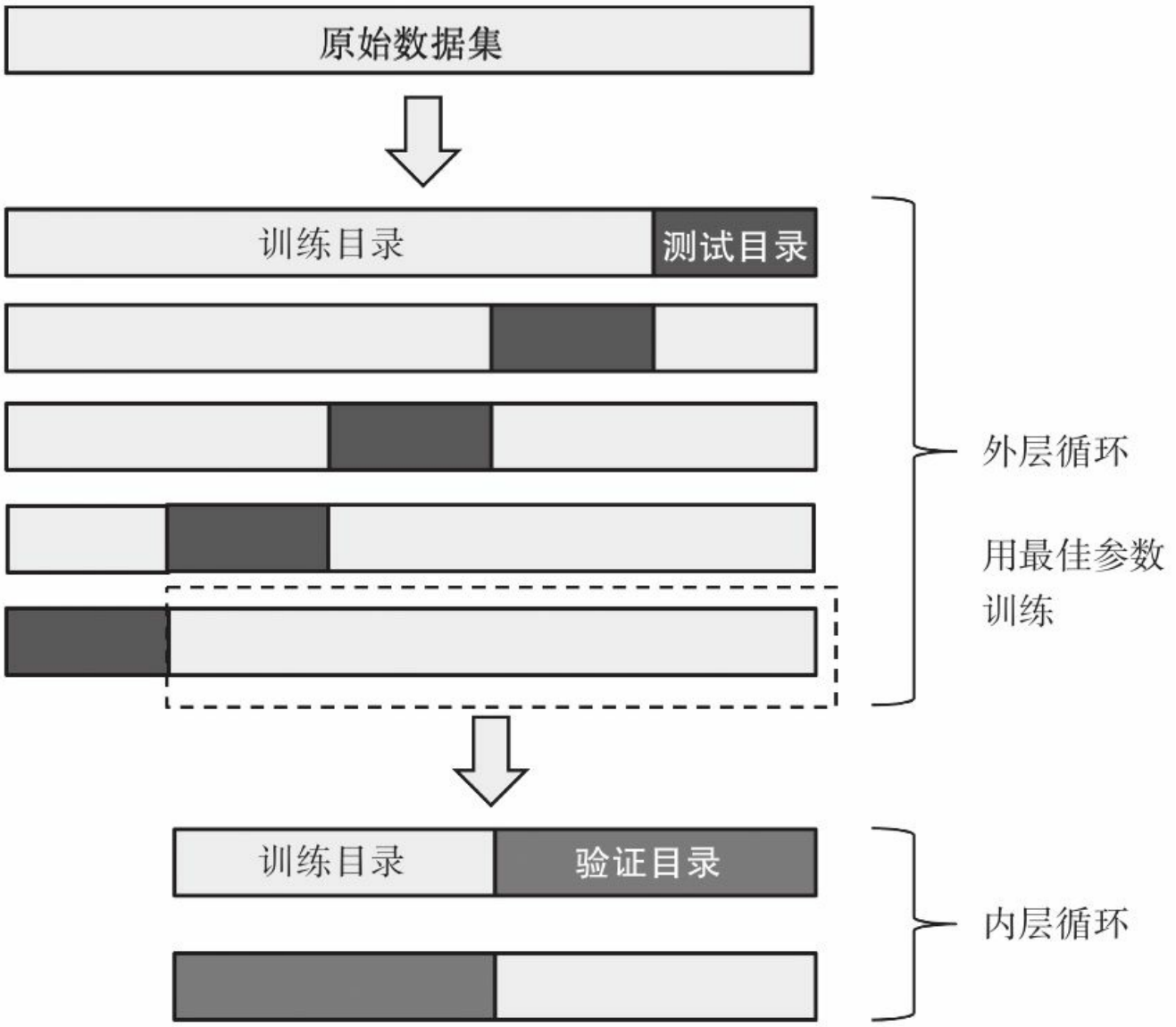
http://scikit-learn.org/stable/modules/grid_search.html#randomized-parameter-optimization。

### 6.4.2 以嵌套式交叉验证来选择算法

正如在上一节中所看到的，使用k折交叉验证结合网格搜索通过改变超参数值进行机器学习，是模型性能调优的有效方法。

如果想要在不同的机器学习算法中进行选择，另一种推荐的方法是嵌套式交叉验证法。通过对误差评估的偏差所做的出色研究，西蒙和瓦玛得出这样的结论：当使用嵌套式交叉验证时，对真实错误的评估与测试集相比几乎没有偏差（S.瓦玛和R.西门.《用交叉验证方法选择模型时的错误估计偏差》.BMC生物信息学，2006，7（1）：91）。

嵌套式交叉验证有一个k折交叉验证外循环，负责把数据分裂为训练和测试子集，而内循环以k折交叉验证法在训练子集上选择模型。模型选择后，用测试子集来评估模型的性能。下图说明了仅有五外两内数据子集的嵌套式交叉验证概念，可用于对计算性能要求高的大型数据集。这种特殊类型的嵌套式交叉验证也被称为5×2交叉验证：




可以用scikit-learn进行嵌套式交叉验证如下：

```
>>> gs = GridSearchCV(estimator=pipe_svc,
...                   param_grid=param_grid,
...                   scoring='accuracy',
...                   cv=2)

>>> scores = cross_val_score(gs, X_train, y_train,
...                          scoring='accuracy', cv=5)
>>> print('CV accuracy: %.3f +/- %.3f' % (np.mean(scores),
...                                       np.std(scores)))
CV accuracy: 0.974 +/- 0.015
```

如果调整模型的超参数并用于预测未见过的数据，所返回的平均交叉验证准确度会对预期有很好的估计。例如，可以用嵌套式交叉验证方法来比较SVM模型和简单的决策树分类器。为简单起见，只调整其深度参数：

```
>>> from sklearn.tree import DecisionTreeClassifier

>>> gs = GridSearchCV(estimator=DecisionTreeClassifier(
                                random_state=0),
...                    param_grid=[{'max_depth': [1, 2, 3,
...                                               4, 5, 6, 7, None]}],
...                    scoring='accuracy',
...                    cv=2)

>>> scores = cross_val_score(gs, X_train, y_train,
...                          scoring='accuracy', cv=5)
>>> print('CV accuracy: %.3f +/- %.3f' % (np.mean(scores),
...                                       np.std(scores)))
CV accuracy: 0.934 +/- 0.016
```

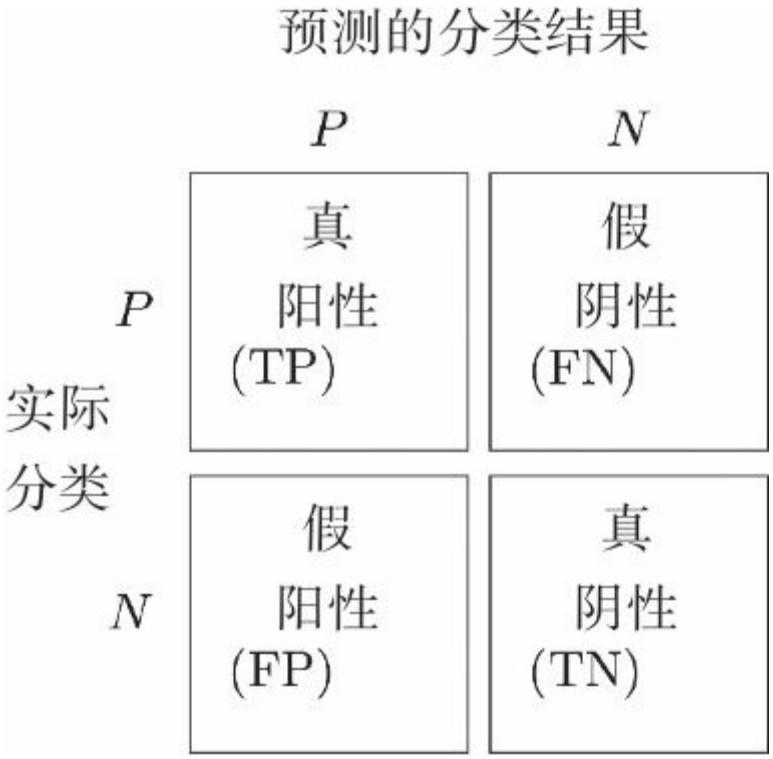
正如所看到的，支持向量机模型（97.4%）的嵌套式交叉验证性能明显优于决策树（93.4%），因此，可以预期使用支持向量机模型对来自某个特定数据集同一样本空间的新数据进行分类可能是更佳选择。

## 6.5 比较不同的性能评估指标

前面的章节用模型准确度来评估模型，这是有效且可量化的模型性能指标。然而，还有几个其他的性能指标也可以度量模型的相关性，如精度、召回率和F1分数。

## 6.5.1 含混矩阵分析

在详细讨论不同的评分指标之前，先看下含混矩阵，这是用来展示算法性能指标的矩阵。含混矩阵根据分类器预测结果报告为真阳性（TP）、真阴性（TN）、假阳性（FP）和假阴性（FN）计数值所构成的方阵，如下图所示：




虽然这些指标可以很容易通过对真实标签和预测标签的比较来手工计算，scikit-learn提供了一个很方便的confusion_matrix函数，示例如下：

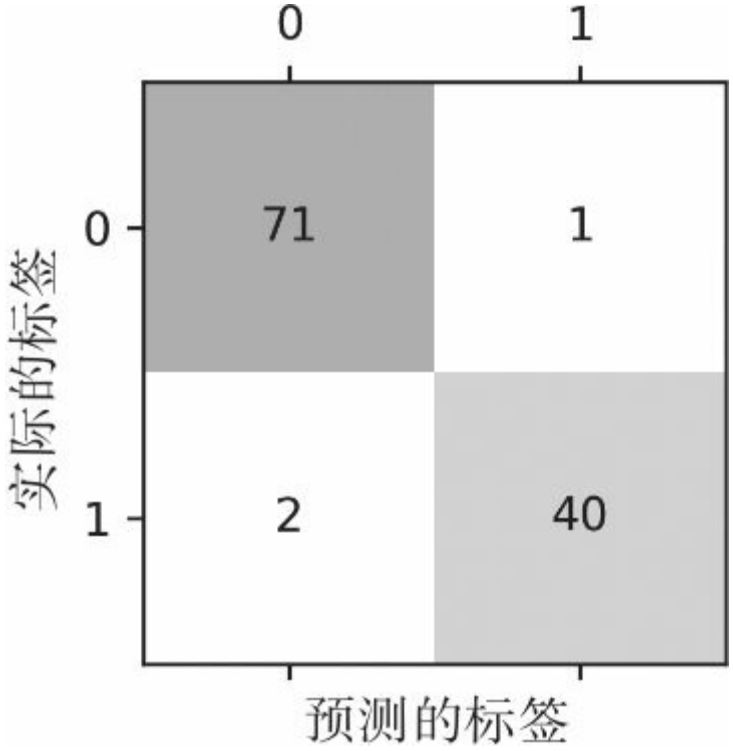
```
>>> from sklearn.metrics import confusion_matrix

>>> pipe_svc.fit(X_train, y_train)
>>> y_pred = pipe_svc.predict(X_test)
>>> confmat = confusion_matrix(y_true=y_test, y_pred=y_pred)
>>> print(confmat)
[[71  1]
 [ 2 40]]
```

执行代码所返回的阵列提供了分类器在测试集上出现的不同类型的错误信息。可以调用matshow把这些信息标示在前面解释过的含混矩阵：

```
>>> fig, ax = plt.subplots(figsize=(2.5, 2.5))
>>> ax.matshow(confmat, cmap=plt.cm.Blues, alpha=0.3)
>>> for i in range(confmat.shape[0]):
...     for j in range(confmat.shape[1]):
...         ax.text(x=j, y=i,
...                 s=confmat[i, j],
...                 va='center', ha='center')
>>> plt.xlabel('predicted label')
>>> plt.ylabel('true label')
>>> plt.show()
```

带有标示的含混矩阵图应该使结果更容易解释：

假设1类（恶性）是该例子中的正向类，模型正确地把71个样本分到属于0类（TN），40个样本分到1类（TP）。然而，模型也错误地把属于1的两个样本分为0类（FN），并把其中一个本为良性的肿瘤样本（FP）预测为恶性。在下一节将学习如何利用这些信息来计算各种错误指标。



## 6.5.2 优化分类模型的准确度和召回率

预测的误差率（ERR）和准确率（ACC）提供了分类错误的总体信息。可以把错误率理解为所有错误预测之和除以预测总数，准确率计算为正确预测之和除以预测总数：

$$ERR = \frac{FP + FN}{FP + FN + TP + TN}$$

预测准确率可以直接根据错误数计算如下：

$$ACC = \frac{TP + TN}{FP + FN + TP + TN} = 1 - ERR$$

真阳率（TPR）和假阳率（FPR）是对非平衡分类问题特别有效的性能指标：

$$FPR = \frac{FP}{N} = \frac{FP}{FP + TN}$$

$$TPR = \frac{TP}{P} = \frac{TP}{FN + TP}$$

例如，在肿瘤诊断更关心恶性肿瘤的检测，以便帮助病人进行适当的治疗。然而，减少恶性肿瘤（FPs）的误诊率对减少病人的不必要担忧也很重要。与FPR相反，TPR提供关于部分正确（或相关）的样本被从阳性池（P）中正确地识别出来的有用信息。

性能指标的精度（PRE）和召回率（REC）与真阳性和真阴性的比率相关，事实上，REC是TPR的同义词：

$$PRE = \frac{TP}{TP + FP}$$

$$REC = TPR = \frac{TP}{P} = \frac{TP}{FN + TP}$$

在实践中，通常是PRE和REC的组合，即所谓的F1-得分：

$$F1 = 2\frac{PRE \times REC}{PRE + REC}$$

这些分数指标都是已经被scikit-learn实现的，可以从sklearn.metrics导入，代码片段如下：

```
>>> from sklearn.metrics import precision_score
>>> from sklearn.metrics import recall_score, f1_score

>>> print('Precision: %.3f' % precision_score(
...                 y_true=y_test, y_pred=y_pred))
Precision: 0.976
>>> print('Recall: %.3f' % recall_score(
...                 y_true=y_test, y_pred=y_pred))
Recall: 0.952
>>> print('F1: %.3f' % f1_score(
...                 y_true=y_test, y_pred=y_pred))
F1: 0.964
```

此外，可以在GridSearchCV通过评分参数使用与准确度不同的各种评分指标。可以在下面的网页链接中找到评分参数可接受的不同值的完整列表。

http://scikit-learn.org/stable/modules/model_evaluation.html

记得在scikit-learn中，阳性类的标签为1类。如果想指定一个不同层级的阳性标签，那就需要通过调用make_scorer函数构建自己的评分器，然后将其作为参数直接提供给GridSearchCV的评分参数（在这个例子中，使用f1_score作为度量）：

```
>>> from sklearn.metrics import make_scorer, f1_score
>>> scorer = make_scorer(f1_score, pos_label=0)
>>> gs = GridSearchCV(estimator=pipe_svc,
...                   param_grid=param_grid,
...                   scoring=scorer,
...                   cv=10)
>>> gs = gs.fit(X_train, y_train)
>>> print(gs.best_score_)
0.986202145696
>>> print(gs.best_params_)
{'svc__C': 10.0, 'svc__gamma': 0.01, 'svc__kernel': 'rbf'}
```

### 6.5.3 绘制受试者操作特性图

ROC（受试者操作特性图）是选择分类模型的有用工具，它以FPR和TPR的性能比较结果为依据，通过移动分类器的阈值完成计算。ROC图的对角线可以解释为随机猜想，落在对角线以下的分类方法被认为比随机猜想要差。TPR为1且FPR为0的最完美分类器会落在图的左上角。基于ROC曲线，可以计算所谓的ROC曲线下面积（ROC AUC）以描述分类模型的特性。

与ROC曲线相似，可以计算分类器在不同概率阈值下的精度与召回率曲线图。scikit-learn也实现了绘制精度与召回率曲线的功能，详细文档可以参考下述网页链接：

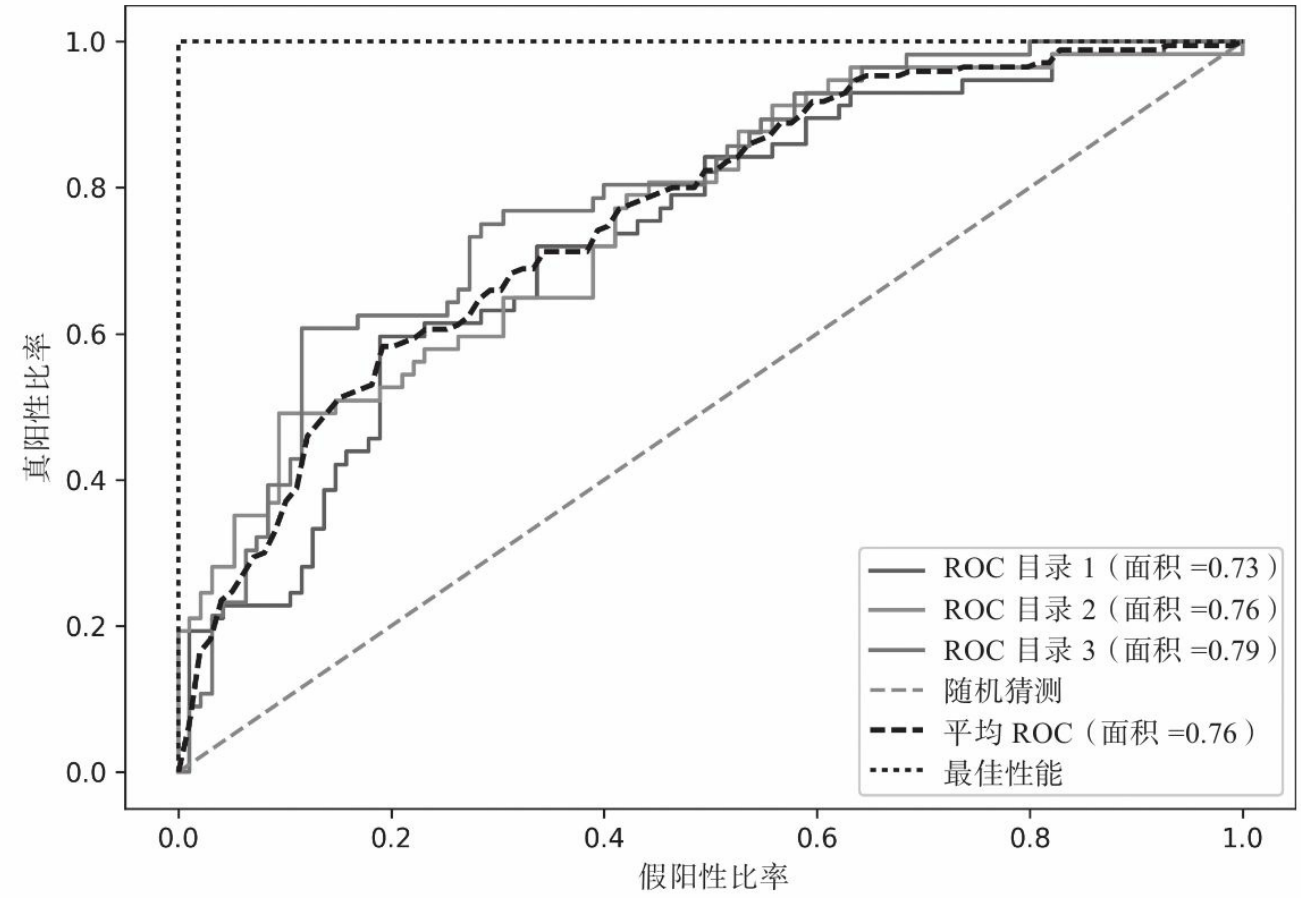
http://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_curve.html

执行下面的代码示例将绘制分类器ROC曲线，该分类器只用威斯康星乳腺癌数据集的两个特征来预测肿瘤为良性或恶性。虽然我们将复用先前定义的逻辑回归管道，但是完成这个分类任务对于分类器更具挑战性，从而使所绘制的ROC曲线在视觉上变得更有趣。出于类似的原因，也把StratifiedKFold验证器分区的数量减少为三个。具体代码如下：

```
>>> from sklearn.metrics import roc_curve, auc
>>> from scipy import interp

>>> pipe_lr = make_pipeline(StandardScaler(),
...                         PCA(n_components=2),
...                         LogisticRegression(penalty='l2',
...                                            random_state=1,
...                                            C=100.0))

>>> X_train2 = X_train[:, [4, 14]]

>>> cv = list(StratifiedKFold(n_splits=3,
...                           random_state=1).split(X_train,
...                                                 y_train))
>>> fig = plt.figure(figsize=(7, 5))

>>> mean_tpr = 0.0
>>> mean_fpr = np.linspace(0, 1, 100)
>>> all_tpr = []

>>> for i, (train, test) in enumerate(cv):
...     probas = pipe_lr.fit(X_train2[train],
...                          y_train[train]).predict_proba(X_train2[test])
...     fpr, tpr, thresholds = roc_curve(y_train[test],
...                                      probas[:, 1],
...                                      pos_label=1)
>>>     mean_tpr += interp(mean_fpr, fpr, tpr)
>>>     mean_tpr[0] = 0.0
>>>     roc_auc = auc(fpr, tpr)
>>>     plt.plot(fpr,
...              tpr,
...              label='ROC fold %d (area = %0.2f)'
...                    % (i+1, roc_auc))
>>> plt.plot([0, 1],
...          [0, 1],
...          linestyle='--',
...          color=(0.6, 0.6, 0.6),
...          label='random guessing')
```

```
>>> mean_tpr /= len(cv)
>>> mean_tpr[-1] = 1.0
>>> mean_auc = auc(mean_fpr, mean_tpr)
>>> plt.plot(mean_fpr, mean_tpr, 'k--',
...          label='mean ROC (area = %0.2f)' % mean_auc, lw=2)
>>> plt.plot([0, 0, 1],
...          [0, 1, 1],
...          linestyle=':',
...          color='black',
...          label='perfect performance')
>>> plt.xlim([-0.05, 1.05])
>>> plt.ylim([-0.05, 1.05])
>>> plt.xlabel('false positive rate')
>>> plt.ylabel('true positive rate')
>>> plt.legend(loc="lower right")
>>> plt.show()
```

前面的代码示例采用已经很熟悉的scikit-learn的StratifiedKFold类，在pipe_lr管道上调用sklearn.metrics模块的roc_curve函数，每次迭代分别计算逻辑回归分类器的ROC性能。此外，调用来自于SciPy的interp函数，把三个子集的平均ROC曲线插入图中，并且调用auc函数计算曲线下方的面积。ROC曲线的结果表明不同的分区之间存在着一定程度上的差异，平均ROC AUC（0.76）介于理想分数（1.0）和随机猜测（0.5）之间：




请注意，如果仅对ROC AUC的分数感兴趣，也可以从sklearn.metrics子模块直接导入roc_auc_score函数。

以ROC AUC来报告分类器的性能，可以对不平衡样本分布下分类器的性能产生更加深刻的认识。然而，尽管准确度分数可以解释为ROC曲线上的一个截点，A.P.布莱德利发现ROC AUC和准确度指标在大多数时间相互吻合（A.P.布莱德利.《用ROC曲线下的面积评估机器学习算法的准确度》.模式识别，1997，30（7）：1145-1159）。

## 6.5.4 多元分类评分指标

本节所讨论的评分指标是针对二元分类系统的。然而，scikit-learn也实现了宏观和微观的平均方法，以把二元分类的评分指标通过一对所有（OvA）扩展到解决多元分类问题。微观平均值是根据各个系统的TP、TN、FP和FN计算出来的。例如，k类系统的精度分数的微观平均值可以计算如下：

$$PRE_{micro} = \frac{TP_1 + \cdots TP_k}{TP_1 + \cdots TP_k + FP_1 + \cdots FP_k}$$

宏观平均值只是计算不同系统的平均分数：

$$PRE_{macro} = \frac{PRE_1 + \cdots PRE_k}{k}$$

如果每个实例或预测的权重相同，那么微观平均值是有用的，而宏观平均值则给予所有的类相同的权重，以评估分类器在最频繁分类标签上的整体性能。

如果用scikit-learn的二元分类模型的性能指标来评价多元分类模型的性能，默认使用归一化或者加权的宏观平均值变量。加权宏观平均值是在计算平均值时，通过计算真实实例的数量，为每个分类标签的评分加权计算而来。如果处理的类不平衡，即每个标签的样本数量不同，加权宏观平均值很有用。

对多元分类问题，scikit-learn默认支持加权宏观平均值，可以调用sklearn.metrics模块的不同评分函数通过参数average来指定平均的方法，例如precision_score或make_scorer函数：

```
>>> pre_scorer = make_scorer(score_func=precision_score,
...                          pos_label=1,
...                          greater_is_better=True,
...                          average='micro')
```

## 6.6 处理类的不平衡问题

本章多次提到了类的不平衡，但还没有讨论过如何适当地处理。在现实世界中，类的不平衡是个常见问题，即当数据集的一个或多个类的样本被过度代表。直观地说，可以想到出现该问题的几个可能的场景，如垃圾邮件过滤、欺诈检测或疾病筛查。

想象一下本章用过的包括90%健康病人的乳腺癌数据集。在这种情况下，可以在无监督机器学习算法的帮助下，通过预测所有样本的多数类（良性肿瘤），在测试集上达到90%的准确度。因此，在这样的数据集上训练一个模型以达到大约90%的测试准确度，将意味着模型还没有从这个数据集所提供的特征中学到任何有用的东西。

本节将简要地介绍一些有助于处理数据集类不平衡的技术。但是在讨论处理这个问题的不同方法之前，先用乳腺癌数据集创建一个不平衡的数据集，该数据集最初包括357个良性肿瘤（0类）和212个恶性肿瘤（1类）样本：

```
>>> X_imb = np.vstack((X[y == 0], X[y == 1][:40]))
>>> y_imb = np.hstack((y[y == 0], y[y == 1][:40]))
```

前一段代码选取了所有357个良性肿瘤的样本，并将它们与前40个恶性肿瘤的样本叠加，形成一个明显的不平衡类。如果要计算能预测大多数类（良性，0类）模型的准确度，将可以达到大约90%的准确度：

```
>>> y_pred = np.zeros(y_imb.shape[0])
>>> np.mean(y_pred == y_imb) * 100
89.924433249370267
```

因此，在这样的数据集上拟合分类器，当比较不同模型的精度、召回率、ROC曲线时，无论在应用中最关心什么，都比将注意力集中在准确度上更有用。例如，如果优先级是找出大多数恶性肿瘤患者，然后推荐他们做额外的筛查，那么召回率应该是要选择的指标。在垃圾邮件过滤中，如果系统并不是很确定的话，并不想把邮件标记为垃圾邮件，那么精度可能是更合适的度量指标。

除了评估机器学习模型外，类的不平衡影响模型拟合过程中的学习算法。由于机器学习算法通常通过优化奖励或成本函数实现，这些函数计算在训练过程中所看到的训练样本的总和，决策规则很可能偏向于多数类。换句话说，该算法基于数据集中最丰富的类来隐式学习优化预测的模型，以便在训练中最小化成本或最大化奖励。

在模型拟合过程中，处理不平衡类比例的一种方法是对少数类的错误预测给予更大的惩罚。在scikit-learn中，只要把参数class_weight设置成class_weight='balanced'，就可以很方便地调整这种惩罚的力度，大多数的分类器都是这么实现的。

处理类不平衡问题的其他常用策略包括对少数类上采样，对多数类下采样以及生成人造训练样本。不幸的是没有万能的最优解决方案，没有对所有问题都最有效的技术。因此，建议在实践中对给定问题尝试不同的策略，通过评估结果选择最合适的技术。

scikit-learn库实现了简单的resample函数，可以通过从数据集中有替换地提取新样本帮助少数类上采样。下面的代码将从不平衡的乳腺癌数据集（这里是1类）中提取少数类，并反复从中提取新样本，直到它包含与分类标签0相同数量的样本为止：

```
>>> from sklearn.utils import resample

>>> print('Number of class 1 samples before:',
...       X_imb[y_imb == 1].shape[0])
Number of class 1 samples before: 40


>>> X_upsampled, y_upsampled = resample(X_imb[y_imb == 1],
...                           y_imb[y_imb == 1],
...                           replace=True,
...                           n_samples=X_imb[y_imb == 0].shape[0],
...                           random_state=123)
>>> print('Number of class 1 samples after:',
...        X_upsampled.shape[0])
Number of class 1 samples after: 357
```

重采样可以把原来的0类样本与上采样的1类样本叠加获得平衡的数据集：

```
>>> X_bal = np.vstack((X[y == 0], X_upsampled))
>>> y_bal = np.hstack((y[y == 0], y_upsampled))
```

因此，多数票预测规则只能达到50%的准确度：

```
>>> y_pred = np.zeros(y_bal.shape[0])
>>> np.mean(y_pred == y_bal) * 100
```

同样，可以删除数据集的训练样本降低采样率。调用resample函数进行下采样，可以直接用前面的示例代码把类0标签与类1标签样本数据进行交换，反之亦然。

另一种处理类不平衡问题的技术是人工生成训练样本，这超出了本书讨论的范围。使用最广泛的人工生成训练样本的算法可能是人工生成少数类的过采样技术（SMOTE），要了解更多关于这项技术的详细信息，可阅读尼太斯·秋拉等撰写的论文（《SMOTE：人工生成少数类的过采样技术》.人工智能研究，2002，16：321-357）。强烈建议了解imbalanced-learn，这是完全聚焦不平衡数据集的Python库，包括SMOTE的实现。可以从下述网站了解更多关于imbalanced-learn的信息：

https://github.com/scikit-learn-contrib/imbalanced-learn

## 6.7 小结

本章的开头讨论了如何在方便的模型管道中连接不同的转换技术和分类器，从而更有效地训练和评估机器学习模型。然后用这些管道进行k折交叉验证，这是一种模型选择和评价的基本方法。用k折交叉验证绘制学习曲线来诊断学习算法中的常见问题，如过拟合和欠拟合。可以用网格搜索进一步对模型的参数调优。我们以介绍含混矩阵和各种性能评价指标结束本章，这些指标对于进一步优化模型和解决特定问题很有用。现在我们已经掌握了必要的技术来为成功分类建立有监督的机器学习模型。

下一章将研究集成的方法：把多个模型和分类算法集成起来进一步提高机器学习系统的预测性能。

# 第7章　综合不同模型的组合学习

前一章重点讨论了为不同分类模型调优和评估的最佳实践。本章将在这些技术的基础上，探索以不同方法构建的集成分类器，它通常比任何单个成员的预测性能都要好。

本章将主要涵盖下述几个方面：

·以多数票机制为基础做出预测

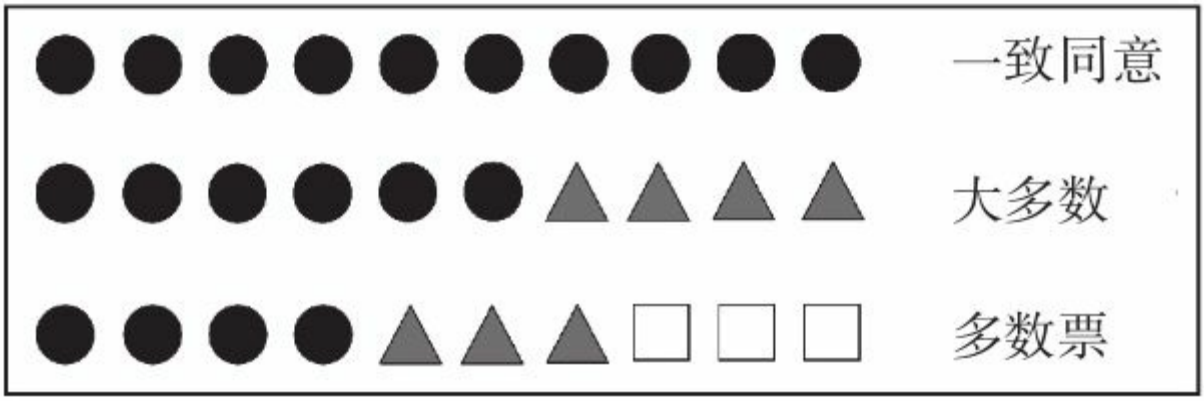
·使用套袋通过反复从训练集可重复地随机抽取样本组合减少过拟合

·利用激励，在从错误中学习的弱学习者的基础上建立强大的模型

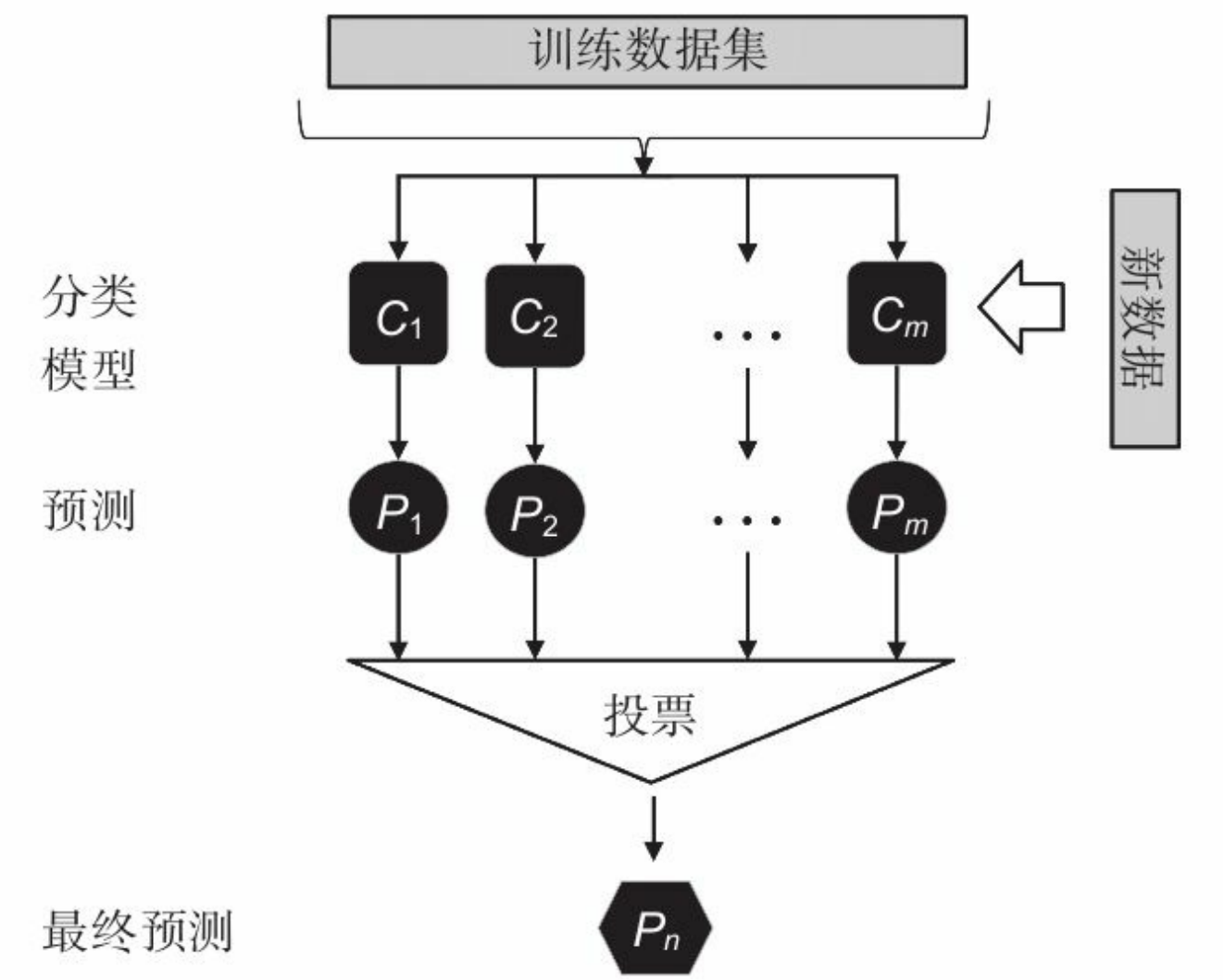
## 7.1 集成学习

集成方法的目标是组合不同的分类器，形成比单个分类器具有更好泛化性能的元分类器。例如，假设收集了来自10位专家的预测，集成方法将策略性地结合10位专家的预测，得出比每位专家的预测更准确、更稳健的结果。正如本章后面将会看到的，创建集成分类器有几种不同的方法。本节将介绍集成的基本概念，包括工作原理以及为什么通常认为它们可以产生良好的泛化性能。

本章将集中讨论最常见的基于多数票机制的集成方法。简单地说多数票机制就是选择大多数分类器所预测的分类标签，也就是说，那些获得超过50%支持的预测结果。严格地说，多数票机制仅指二元分类场景。然而，多数票机制很容易推广到多元分类场景，即所谓的相对多数票。这里选择获得最多票数的分类标签。下图演示了10个分类器集成的多数票和相对多数票概念，每个独特的符号（三角形、正方形和圆形）分别代表一个唯一的分类标签：




用训练集从训练m个不同分类器（$C_1$，…，$C_m$）开始。取决于所用的技术，集成可以由不同的分类算法构建，例如决策树、支持向量机、逻辑回归分类器等等。或者用相同的基本分类算法，分别拟合不同的训练子集。这种方法的突出例子是随机森林算法，它集成了不同的决策树分类器。下图演示了用多数票机制的一般集成方法的概念：




为了通过简单的多数票或相对多数票投票机制预测分类标签，可以把每个单独的分类器$C_j$预测的分类标签组合起来，然后选择分类标签$\hat{y}$，从而得到得票最多的标签：

$$\hat{y}=\text{mode}\{C_1(\boldsymbol{x}),\ C_2(\boldsymbol{x}),\ \cdots,\ C_m(\boldsymbol{x})\}$$

例如，在二元分类任务中，class1=-1而class2=+1，多数票预测可以表示为：

$$C(\boldsymbol{x})=\text{sign}\left[\sum_j^m C_j(\boldsymbol{x})\right]=\begin{cases}1 & \text{if}\sum_i C_j(\boldsymbol{x})\geqslant 0\\ -1 & \text{otherwise}\end{cases}$$

可以用集成理论的简单概念来解释为什么集成方法要比单个分类器的效果更好。在下面的例子中，假设所有n个基本分类器所面对的都是二元分类任务，错误率ε相同。另外，假设分类器都是独立的而且错误率互不相干。

在这些假设条件下，可以直接把基本分类器集成的错误率表达为二项式分布的概率质量函数：

$$P\left(y \geqslant k\right) = \sum_{k}^{n} \langle {}_{k}^{n} \rangle \varepsilon^{k} \left(1 - 0.25\right)^{n-k} = \varepsilon_{\text{ensemble}}$$

这里为二项式系数n选择k。换句话说，我们计算集成出错率。

现在看一个更具体的实例，其中有11个基本分类器（n=11），每个分类器出错率为0.25（ε=0.25）：

$$P\left(y \geqslant k\right) = \sum_{k=6}^{11} \langle {}_{k}^{11} \rangle 0.25^{k} \left(1 - \varepsilon\right)^{11-k} = 0.034$$

二项式系数

二项式系数指从大小为n的集合中选择k个无序元素子集的组合数，因此，它通常被称为“n选k”。因为不考虑顺序，所以二项式系数有时也被称为组合或组合数，表示如下：

$$\frac{n!}{\left(n-k\right)!k!}$$

这里，符号（！）代表阶乘。例如，3！=3×2×1=6。

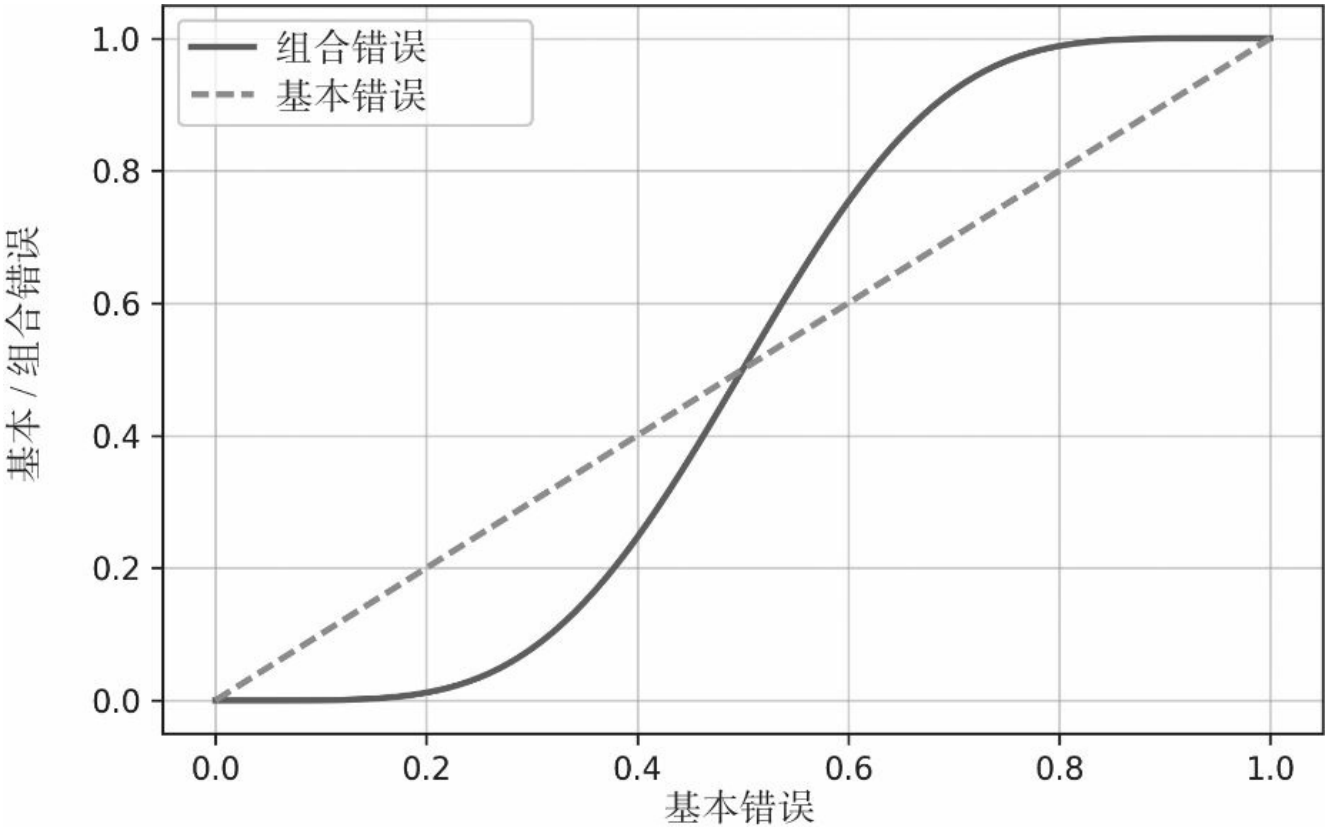
可以看到，如果满足所有的假设条件，那么集成分类器的错误率（0.034）远低于每个分类器的错误率（0.25）。需要注意的是该简化案例的分类器个数为偶数n，而分类结果为50-50的情况被视为错误，因为只有50%正确的机会。为了比较理想集成分类器与基本分类器在不同的错误率范围内的表现，用下述Python代码实现概率质量函数：

```
>>> from scipy.misc import comb
>>> import math
>>> def ensemble_error(n_classifier, error):
...     k_start = int(math.ceil(n_classifier / 2.))
...     probs = [comb(n_classifier, k) *
...              error**k *
...              (1-error)**(n_classifier - k)
...              for k in range(k_start, n_classifier + 1)]
...     return sum(probs)
>>> ensemble_error(n_classifier=11, error=0.25)
0.034327507019042969
```

在实现了ensemble_error函数之后，可以计算从0.0到1.0范围内的不同集成错误率，并将其绘制在图中：

```
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> error_range = np.arange(0.0, 1.01, 0.01)
>>> ens_errors = [ensemble_error(n_classifier=11, error=error)
...               for error in error_range]
>>> plt.plot(error_range, ens_errors,
...          label='Ensemble error',
...          linewidth=2)
>>> plt.plot(error_range, error_range,
...          linestyle='--', label='Base error',
...          linewidth=2)
>>> plt.xlabel('Base error')
>>> plt.ylabel('Base/Ensemble error')
>>> plt.legend(loc='upper left')
>>> plt.grid(alpha=0.5)
>>> plt.show()
```

从下面的结果图中可以看到，只要基本分类器的性能优于随机猜测（ε＜0.5），集成分类器的错误率总是比单一基本分类器的错误率要低。请注意，y轴描述了基本错误率（虚线）以及集成错误率（实线）：


组合错误
基本错误
基本 / 组合错误
基本错误
0.0
0.2
0.4
0.6
0.8
1.0

## 7.2 采用多数票机制的集成分类器

在上一节简要介绍集成学习之后，让我们从热身练习开始，并在Python中实现多数票机制的简单集成分类器。

本节中讨论的多数票机制也可以通过相对多数票机制推广到多元分类的场景中，为了简便，我们依然使用术语多数票机制，因为它也经常在文献中这样使用。

## 7.2.1 实现基于多数票的简单分类器

本节将要实现的算法可以把不同的分类算法及其各自相应的权重组合起来。目标是建立一个更强大的超级分类器，以平衡单个分类器在特定数据集上的弱点。可以用更精确的数学语言把加权多数票的逻辑表示如下：

$$\hat{y} = \arg\max_i \sum_{j=1}^{m} w_j \chi_A \left( C_j(\boldsymbol{x}) = i \right)$$

这里$w_j$是与基本分类器$C_j$相关的权重，$\hat{y}$是分类标签的预测结果，$\chi_A$是特性函数[$C_j$（x）=i∈A]，A为独立分类标签的集合。对于相同权重，可以把该方程简化为：

$$\hat{y} = \mathrm{mode}\left\{ C_1(\boldsymbol{x}), C_2(\boldsymbol{x}), \cdots, C_m(\boldsymbol{x}) \right\}$$

众数是统计学中在集合中最常见的操作。

例如：mode{1，2，1，1，2，4，5，4}=1.

为了更好地理解权重的概念，现在来看一个更具体的例子。假设有三个基本分类器的集成，$C_j$（j∈{0，1}），想要预测某样本实例的分类标签x。其中两个基本分类器的预测分类标签为0，而第三个基本分类器$C_3$预测的分类标签为1。如果对每个基本分类器的预测结果给予相同的权重，基于大多数票原则将会预测那个样本属于分类标签0。

$$C_1(\boldsymbol{x}) \to 0, \quad C_2(\boldsymbol{x}) \to 0, \quad C_3(\boldsymbol{x}) \to 1$$

$$\hat{y} = \mathrm{mode}\{0,0,1\} = 0$$

现在假设$C_3$分类器的权重为0.6，$C_1$和$C_2$的权重为0.2：

$$\hat{y} = \arg\max_i \sum_{j=1}^{m} w_j \chi_A\left(C_j(\boldsymbol{x})\right) = i$$

$$= \arg\max_i \left[0.2 \times i_0 + 0.2 \times i_0 + 0.6 \times i_j\right] = 1$$

显然，因为3×0.2=0.6，所以可以说$C_3$预测结果的权重是$C_1$和$C_2$权重的三倍，可以表示如下：

$$\hat{y}=\text{mode}\{0,0,1,1,1\}=1$$

可以用NumPy的argmax和bincount函数将加权大多数票概念翻译成Python代码：

```
>>> import numpy as np
>>> np.argmax(np.bincount([0, 0, 1],
...                       weights=[0.2, 0.2, 0.6]))
1
```

还记得第3章关于逻辑回归的讨论吗？scikit-learn的某些分类器也可以调用predict_proba方法返回预测分类标签的概率。如果集成分类器预测得够精准，用预测的分类概率代替分类标签进行大多数票表决是有用的。修改后的多数票预测分类标签的概率方法可以表示如下：

$$\hat{y} = \arg\max_i \sum_{j=1}^{m} w_j p_{ij}$$

这里，$p_{ij}$为第j个分类器对分类标签i预测的概率。

继续讨论前面的示例，假设有一个二元分类问题，其分类标签i∈{0，1}，集成了三个分类器$C_j$（j∈{1，2，3}）。假设针对样本x，分类器$C_j$返回下述分类标签的概率：

$$C_1(\boldsymbol{x}) \rightarrow [0.9, 0.1], C_2(\boldsymbol{x}) \rightarrow [0.8, 0.2], C_3(\boldsymbol{x}) \rightarrow [0.4, 0.6]$$

接着计算各分类标签的概率：

$$p(i_0 \mid \boldsymbol{x}) = 0.2 \times 0.9 + 0.2 \times 0.8 + 0.6 \times 0.4 = 0.58$$

$$p(i_0 \mid \boldsymbol{x}) = 0.2 \times 0.1 + 0.2 \times 0.2 + 0.6 \times 0.6 = 0.42$$

$$\hat{y} = \arg\max_i \left[ p(i_0 | \boldsymbol{x},), p(i_1 \mid \boldsymbol{x}) \right] = 0$$

要实现基于分类概率的加权多数票分类方法，可以再次调用NumPy的numpy.average和np.argmax函数：

```
>>> ex = np.array([[0.9, 0.1],
...                [0.8, 0.2],
...                [0.4, 0.6]])
>>> p = np.average(ex, axis=0, weights=[0.2, 0.2, 0.6])
>>> p
array([ 0.58,  0.42])
>>> np.argmax(p)
0
```

综合前面的讨论，现在可以在Python中实现MajorityVoteClassifier：

```
from sklearn.base import BaseEstimator
from sklearn.base import ClassifierMixin
from sklearn.preprocessing import LabelEncoder
from sklearn.externals import six
from sklearn.base import clone
from sklearn.pipeline import _name_estimators
import numpy as np
import operator

class MajorityVoteClassifier(BaseEstimator,
                             ClassifierMixin):
    """ A majority vote ensemble classifier

    Parameters
    ----------
    classifiers : array-like, shape = [n_classifiers]
      Different classifiers for the ensemble

    vote : str, {'classlabel', 'probability'}
```

```
        Default: 'classlabel'
        If 'classlabel' the prediction is based on
        the argmax of class labels. Else if
        'probability', the argmax of the sum of
        probabilities is used to predict the class label
        (recommended for calibrated classifiers).

      weights : array-like, shape = [n_classifiers]
        Optional, default: None
        If a list of `int` or `float` values are
        provided, the classifiers are weighted by
        importance; Uses uniform weights if `weights=None`.

      """
      def __init__(self, classifiers,
                   vote='classlabel', weights=None):

          self.classifiers = classifiers
          self.named_classifiers = {key: value for
                                    key, value in
                                    _name_estimators(classifiers)}
          self.vote = vote
      self.weights = weights

  def fit(self, X, y):
      """ Fit classifiers.

      Parameters
      ----------
      X : {array-like, sparse matrix},
          shape = [n_samples, n_features]
          Matrix of training samples.

      y : array-like, shape = [n_samples]
          Vector of target class labels.

      Returns
      -------
      self : object

      """
      # Use LabelEncoder to ensure class labels start
      # with 0, which is important for np.argmax
      # call in self.predict
      self.lablenc_ = LabelEncoder()
      self.lablenc_.fit(y)
      self.classes_ = self.lablenc_.classes_
      self.classifiers_ = []
      for clf in self.classifiers:
          fitted_clf = clone(clf).fit(X,
                              self.lablenc_.transform(y))
          self.classifiers_.append(fitted_clf)
      return self
```

代码中已经添加了很多注释来解释各个部分。然而，在实现其余的方法之前，先稍微休息一下，讨论那些可能会让人感到困惑的代码。我们用父类BaseEstimator和ClassifierMixin轻而易举地获得了一些基本功能，这包括设置和返回分类器参数的get_params和set_params方法，以及计算预测准确度的score方法。注意，导入six以确保MajorityVoteClassifier兼容Python 2.6。

接下来，如果通过定义vote='classlabel'来初始化MajorityVoteClassifier的新对象，可以增加predict方法来预测分类标签。或者定义vote='probability'来初始化集成分类器，然后基于类中元素的概率来预测分类标签。此外，还将添加一个predict_proba方法，以返回平均概率，这对计算ROC AUC来说很有用：

```
def predict(self, X):
    """ Predict class labels for X.

    Parameters
    ----------
    X : {array-like, sparse matrix},
        Shape = [n_samples, n_features]
        Matrix of training samples.

    Returns
    ----------
    maj_vote : array-like, shape = [n_samples]
        Predicted class labels.

    """
    if self.vote == 'probability':
        maj_vote = np.argmax(self.predict_proba(X),
                             axis=1)
    else:  # 'classlabel' vote

        #  Collect results from clf.predict calls
        predictions = np.asarray([clf.predict(X)
                                  for clf in
                                  self.classifiers_]).T

        maj_vote = np.apply_along_axis(
                       lambda x:
                       np.argmax(np.bincount(x,
                                 weights=self.weights)),
                       axis=1,
                       arr=predictions)
    maj_vote = self.lablenc_.inverse_transform(maj_vote)
    return maj_vote

def predict_proba(self, X):
    """ Predict class probabilities for X.

    Parameters
    ----------
    X : {array-like, sparse matrix},
        shape = [n_samples, n_features]
        Training vectors, where n_samples is
        the number of samples and
```

```
            n_features is the number of features.

        Returns
        ----------
        avg_proba : array-like,
            shape = [n_samples, n_classes]
            Weighted average probability for
            each class per sample.

        """
        probas = np.asarray([clf.predict_proba(X)
                             for clf in self.classifiers_])
        avg_proba = np.average(probas,
                               axis=0, weights=self.weights)
        return avg_proba

    def get_params(self, deep=True):
        """ Get classifier parameter names for GridSearch"""
        if not deep:
            return super(MajorityVoteClassifier,
                         self).get_params(deep=False)
        else:
            out = self.named_classifiers.copy()
            for name, step in\
                    six.iteritems(self.named_classifiers):
                for key, value in six.iteritems(
                        step.get_params(deep=True)):
                    out['%s__%s' % (name, key)] = value
            return out
```

另外请注意，这里定义了我们自己修改后的get_params方法来使用_name_estimators函数访问集成分类器中每个成员分类器的参数。这可能看似复杂，但却很有用，特别是在以后的章节中用网格搜索寻找调优的超参数时。

尽管本书第一版为演示目的而实现的MajorityVoteClassifier非常有用，我们用scikit-learn实现了更复杂的多数票分类器。可以从scikit-learn 0.17或者更新版本中找到sklearn.ensemble.VotingClassifier。

### 7.2.2 用多数票原则进行预测

现在是时候把前面章节中实现的MajorityVoteClassifier付诸于实际应用了。但是首先让我们准备一个可以测试的数据集。既然已经熟悉从CSV文件加载数据集的技术，我们将取捷径从scikit-learn的数据集模块直接加载鸢尾花数据集。此外，为了使演示的分类任务更具有挑战性，我们只选择萼片宽度和花瓣长度两个特征。虽然可以把MajorityVoteClassifier泛化到多元分类问题，但是我们只对Iris-versicolor和Iris-virginica花样本进行分类，并随后计算ROC AUC。具体代码如下：

```
>>> from sklearn import datasets
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.preprocessing import LabelEncoder

>>> iris = datasets.load_iris()
>>> X, y = iris.data[50:, [1, 2]], iris.target[50:]
>>> le = LabelEncoder()
>>> y = le.fit_transform(y)
```

注意，scikit-learn用predict_proba方法（如果适用）来计算ROC AUC的评分。在第3章中，我们看到逻辑回归模型如何计算分类概率。在决策树中，概率根据在训练时为每个节点创建的频率向量计算。

该向量收集从该节点上的分类标签计算出的每个分类标签的频率值。然后把频率归一化，其和为1。同样，k-近邻算法的分类标签从k-近邻算法返回归一化的分类标签频率。尽管决策树和k-近邻分类器返回的归一化概率可能与逻辑回归模型得到的概率相似，但必须意识到这些实际上并非来自于概率质量函数。

接下来，将鸢尾花样本数据分成50%的训练集和50%的测试集：

```
>>> X_train, X_test, y_train, y_test =\
...         train_test_split(X, y,
...                          test_size=0.5,
...                          random_state=1,
...                          stratify=y)
```

现在将使用训练集训练三种不同分类器：

·逻辑回归分类器

·决策树分类器

·k-近邻分类器

在将它们构建成集成分类器之前，先通过对训练集进行10次交叉验证以评估每个分类器模型的性能：

```
>>> from sklearn.model_selection import cross_val_score
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.tree import DecisionTreeClassifier
>>> from sklearn.neighbors import KNeighborsClassifier
>>> from sklearn.pipeline import Pipeline
>>> import numpy as np
>>> clf1 = LogisticRegression(penalty='l2',
...                           C=0.001,
...                           random_state=1)
>>> clf2 = DecisionTreeClassifier(max_depth=1,
...                               criterion='entropy',
...                               random_state=0)
>>> clf3 = KNeighborsClassifier(n_neighbors=1,
...                             p=2,
...                             metric='minkowski')
>>> pipe1 = Pipeline([['sc', StandardScaler()],
...                   ['clf', clf1]])
>>> pipe3 = Pipeline([['sc', StandardScaler()],
...                   ['clf', clf3]])
>>> clf_labels = ['Logistic regression', 'Decision tree', 'KNN']
>>> print('10-fold cross validation:\n')
>>> for clf, label in zip([pipe1, clf2, pipe3], clf_labels):

...     scores = cross_val_score(estimator=clf,
...                              X=X_train,
...                              y=y_train,
...                              cv=10,
...                              scoring='roc_auc')
...     print("ROC AUC: %0.2f (+/- %0.2f) [%s]"
...                % (scores.mean(), scores.std(), label))
```

执行代码后得到下列输出，它显示出单个分类器的预测性能几乎相等。

```
10-fold cross validation:

ROC AUC: 0.87 (+/- 0.17) [Logistic regression]
ROC AUC: 0.89 (+/- 0.16) [Decision tree]
ROC AUC: 0.88 (+/- 0.15) [KNN]
```

读者可能想知道为什么将逻辑回归和k-近邻分类器作为管道的一部分进行训练。其背后的原因正如第3章中所讨论的那样，与决策树相比，逻辑回

归和k近邻算法（使用欧氏距离度量）都是比例尺度可变的。虽然鸢尾花的特征都是在相同比例尺度（cm）上测量的，但是采用标准化特征是一个好习惯。

现在开始更精彩的部分，在MajorityVoteClassifier类中集成独立的分类器进行多数票决策：

```
>>> mv_clf = MajorityVoteClassifier(
...                  classifiers=[pipe1, clf2, pipe3])
>>> clf_labels += ['Majority voting']
>>> all_clf = [pipe1, clf2, pipe3, mv_clf]
>>> for clf, label in zip(all_clf, clf_labels):
...     scores = cross_val_score(estimator=clf,
...                              X=X_train,
...                              y=y_train,
...                              cv=10,
...                              scoring='roc_auc')
...     print("Accuracy: %0.2f (+/- %0.2f) [%s]"
...                % (scores.mean(), scores.std(), label))
ROC AUC: 0.87 (+/- 0.17) [Logistic regression]
ROC AUC: 0.89 (+/- 0.16) [Decision tree]
ROC AUC: 0.88 (+/- 0.15) [KNN]
ROC AUC: 0.94 (+/- 0.13) [Majority voting]
```

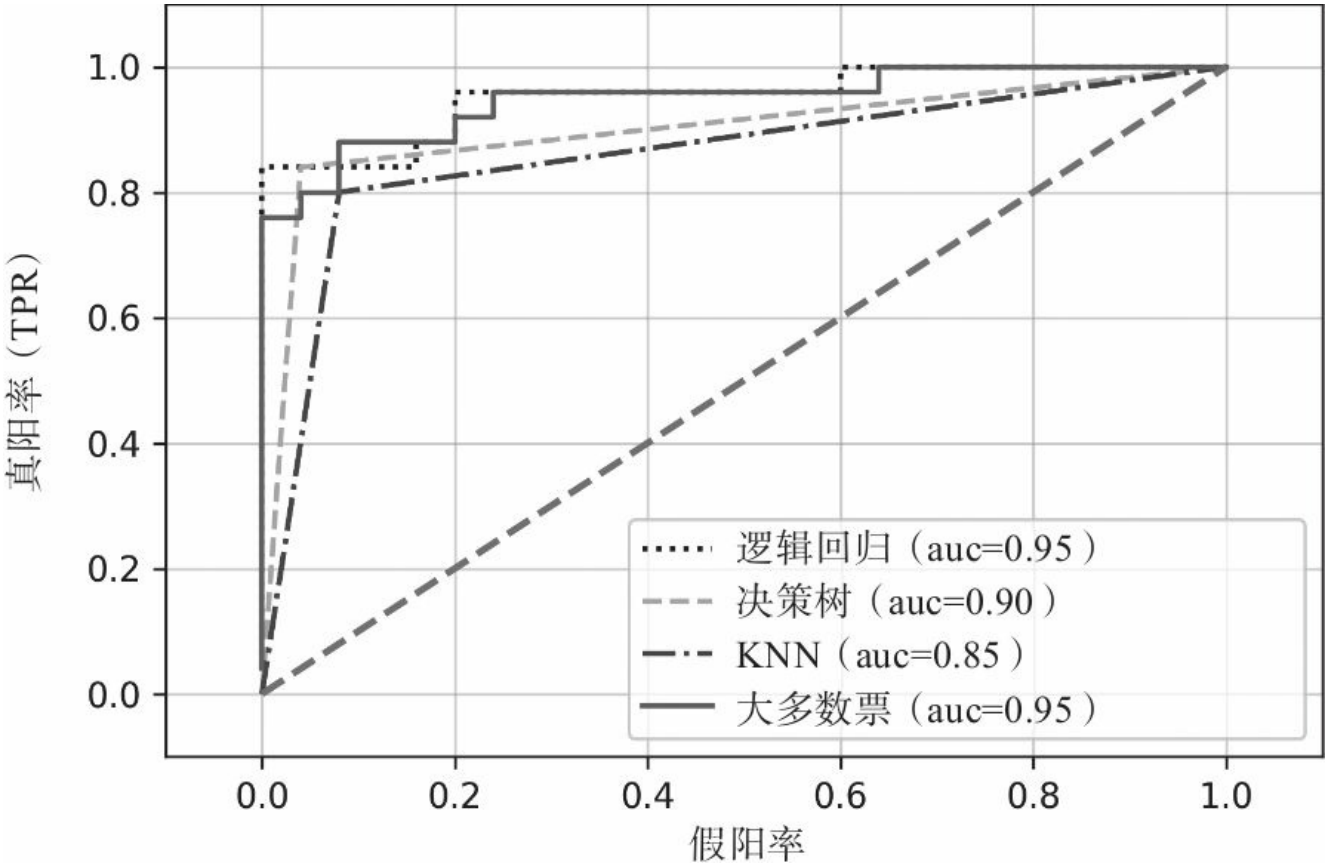
可以看到各个分类器的MajorityVotingClassifier的性能已经在有10个分区的交叉验证评估上得到改善。

### 7.2.3 评估和优化集成分类器

本节将基于测试数据来计算ROC曲线，以检查MajorityVoteClassifier对未见过的数据是否泛化良好。应该记住测试集将不会用于模型选择，其目的仅仅是报告分类器的泛化性能的无偏估计：

```
>>> from sklearn.metrics import roc_curve
>>> from sklearn.metrics import auc
>>> colors = ['black', 'orange', 'blue', 'green']

 >>> linestyles = [':', '--', '-.', '-']
 >>> for clf, label, clr, ls \
 ...         in zip(all_clf, clf_labels, colors, linestyles):
 ...     # assuming the label of the positive class is 1
 ...     y_pred = clf.fit(X_train,
 ...                      y_train).predict_proba(X_test)[:, 1]
 ...     fpr, tpr, thresholds = roc_curve(y_true=y_test,
 ...                                      y_score=y_pred)
 ...     roc_auc = auc(x=fpr, y=tpr)
 ...     plt.plot(fpr, tpr,
 ...              color=clr,
 ...              linestyle=ls,
 ...              label='%s (auc = %0.2f)' % (label, roc_auc))
 >>> plt.legend(loc='lower right')
 >>> plt.plot([0, 1], [0, 1],
 ...          linestyle='--',
 ...          color='gray',
 ...          linewidth=2)
 >>> plt.xlim([-0.1, 1.1])
 >>> plt.ylim([-0.1, 1.1])
 >>> plt.grid(alpha=0.5)
 >>> plt.xlabel('False positive rate (FPR)')
 >>> plt.ylabel('True positive rate (TPR)')
 >>> plt.show()
```

正如在ROC结果图中所看到的，集成分类器在测试集上也表现良好（ROC AUC=0.95）。然而，可以看到逻辑回归分类器在同一数据集上有类似的表现，这可能是由于高方差（在当前情况下，是如何分裂数据集的敏感性），假定数据集的规模很小：

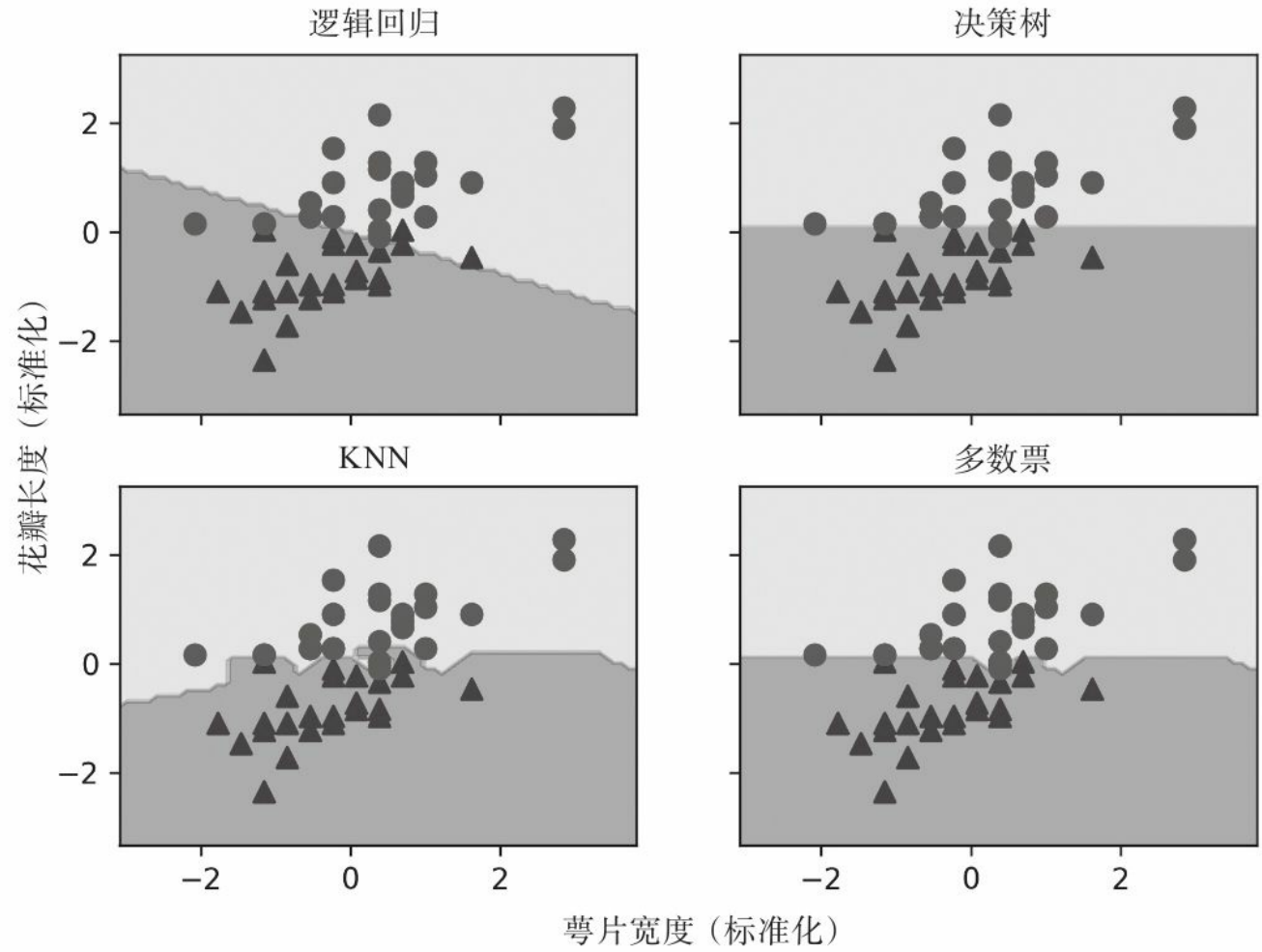



因为我们只为示例分类选择了两个特性，所以看看集成分类器决策区域的实际情况很有意思。虽然在模型拟合之前没有必要对训练特征进行标准化，因为逻辑回归和k-近邻的管道将自动进行处理，我们仍将把训练集标准化，以使决策树的决策区域有相同的比例尺度从而达到可视化的目的。具体代码如下：

```
>>> sc = StandardScaler()
>>> X_train_std = sc.fit_transform(X_train)
>>> from itertools import product
>>> x_min = X_train_std[:, 0].min() - 1
>>> x_max = X_train_std[:, 0].max() + 1
>>> y_min = X_train_std[:, 1].min() - 1
>>> y_max = X_train_std[:, 1].max() + 1
```

```
>>> xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
...                      np.arange(y_min, y_max, 0.1))
>>> f, axarr = plt.subplots(nrows=2, ncols=2,
...                         sharex='col',
...                         sharey='row',
...                         figsize=(7, 5))
>>> for idx, clf, tt in zip(product([0, 1], [0, 1]),
...                         all_clf, clf_labels):
...     clf.fit(X_train_std, y_train)
...     Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
...     Z = Z.reshape(xx.shape)
...     axarr[idx[0], idx[1]].contourf(xx, yy, Z, alpha=0.3)
...     axarr[idx[0], idx[1]].scatter(X_train_std[y_train==0, 0],
...                                   X_train_std[y_train==0, 1],
...                                   c='blue',
...                                   marker='^',
...                                   s=50)
...     axarr[idx[0], idx[1]].scatter(X_train_std[y_train==1, 0],
...                                   X_train_std[y_train==1, 1],
...                                   c='green',
...                                   marker='o',
...                                   s=50)
...     axarr[idx[0], idx[1]].set_title(tt)
>>> plt.text(-3.5, -4.5,
...          s='Sepal width [standardized]',
...          ha='center', va='center', fontsize=12)
>>> plt.text(-10.5, 4.5,
...          s='Petal length [standardized]',
...          ha='center', va='center',
...          fontsize=12, rotation=90)
>>> plt.show()
```

有趣的是，正如预期的那样，集成分类器决策区域似乎是单个分类器决策区域的混合体。乍一看，多数票决策边界看起来更像单层决策树的决策，是当萼片宽度≥1时与y轴的正交。然而，也注意到k-近邻分类器的非线性混合了进来：




在为了集成分类而优化单个分类器参数之前，先调用get_params方法来对我们如何访问GridSearch对象内部的参数有个基本思路：

```
>>> mv_clf.get_params()
{'decisiontreeclassifier': DecisionTreeClassifier(class_weight=None,
criterion='entropy', max_depth=1,
            max_features=None,
            max_leaf_nodes=None,
            min_samples_leaf=1,
            min_samples_split=2, min_weight_fraction_leaf=0.0,
            random_state=0, splitter='best'),
 'decisiontreeclassifier__class_weight': None,
 'decisiontreeclassifier__criterion': 'entropy',
 [...]
 'decisiontreeclassifier__random_state': 0,
 'decisiontreeclassifier__splitter': 'best',
 'pipeline-1': Pipeline(steps=[('sc', StandardScaler(copy=True,
 with_mean=True, with_std=True)), ('clf', LogisticRegression(C=0.001,
 class_weight=None, dual=False, fit_intercept=True,
 intercept_scaling=1, max_iter=100, multi_class='ovr',
 penalty='l2', random_state=0, solver='liblinear', tol=0.0001,
 verbose=0))]),
 'pipeline-1__clf': LogisticRegression(C=0.001, class_weight=None,
dual=False, fit_intercept=True,
          intercept_scaling=1, max_iter=100, multi_class='ovr',
          penalty='l2', random_state=0, solver='liblinear',
tol=0.0001,
          verbose=0),
 'pipeline-1__clf__C': 0.001,
 'pipeline-1__clf__class_weight': None,
 'pipeline-1__clf__dual': False,
 [...]
 'pipeline-1__sc__with_std': True,
 'pipeline-2': Pipeline(steps=[('sc', StandardScaler(copy=True, with_
mean=True, with_std=True)), ('clf', KNeighborsClassifier(algorithm='au
to', leaf_size=30, metric='minkowski',
           metric_params=None, n_neighbors=1, p=2,
weights='uniform'))]),
 'pipeline-2__clf': KNeighborsClassifier(algorithm='auto', leaf_
size=30, metric='minkowski',
           metric_params=None, n_neighbors=1, p=2,
weights='uniform'),
 'pipeline-2__clf__algorithm': 'auto',
 [...]
 'pipeline-2__sc__with_std': True}
```

根据get_params方法的返回值，我们现在知道了应该如何访问单个分类器的属性。为了演示目的，通过网格搜索来优化逻辑回归分类器的逆正则化参数C和决策树深度：

```
>>> from sklearn.model_selection import GridSearchCV
>>> params = {'decisiontreeclassifier__max_depth': [1, 2],
...           'pipeline-1__clf__C': [0.001, 0.1, 100.0]}
>>> grid = GridSearchCV(estimator=mv_clf,
...                     param_grid=params,
...                     cv=10,
```

```
...                           scoring='roc_auc')
>>> grid.fit(X_train, y_train)
```

网格搜索完成后，可以列出不同的超参数值组合，以及通过10个分区的交叉验证计算所获得ROC AUC的平均评分：

```
>>> for params, mean_score, scores in grid.grid_scores_:
...     print("%0.3f+/-%0.2f %r"
...           % (mean_score, scores.std() / 2, params))
0.933 +/- 0.07 {'pipeline-1__clf__C': 0.001, 'decisiontreeclassifier__
max_depth': 1}
0.947 +/- 0.07 {'pipeline-1__clf__C': 0.1, 'decisiontreeclassifier__
max_depth': 1}
0.973 +/- 0.04 {'pipeline-1__clf__C': 100.0, 'decisiontreeclassifier__
max_depth': 1}
0.947 +/- 0.07 {'pipeline-1__clf__C': 0.001, 'decisiontreeclassifier__
max_depth': 2}
0.947 +/- 0.07 {'pipeline-1__clf__C': 0.1, 'decisiontreeclassifier__
max_depth': 2}
0.973 +/- 0.04 {'pipeline-1__clf__C': 100.0, 'decisiontreeclassifier__
max_depth': 2}

>>> print('Best parameters: %s' % grid.best_params_)
Best parameters: {'pipeline-1__clf__C': 100.0,
'decisiontreeclassifier__max_depth': 1}

>>> print('Accuracy: %.2f' % grid.best_score_)
Accuracy: 0.97
```

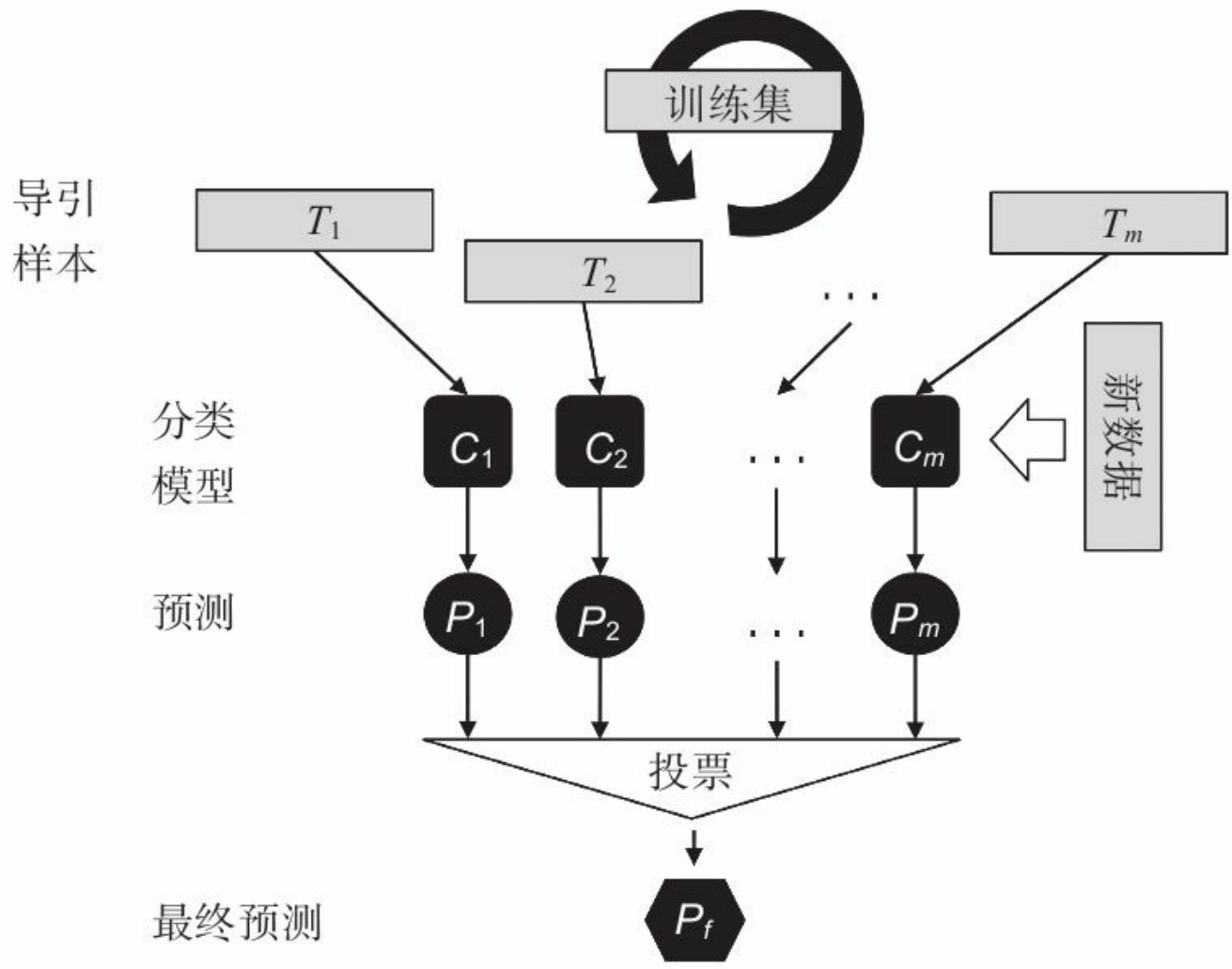
正如所看到的，选择较低的正则化强度（C＝100）得到最佳交叉验证结果，而树的深度似乎根本就不影响性能，这表明单层决策树足以分离数据。为了提醒不止一次地使用测试集来评估模型是不好的做法，我们不打算对本节优化过的超参数的泛化性能进行估计。我们将继续向前推进掌握集成学习的另一种方法：套袋法。

本节实现的多数票投票方法不应与叠加混淆。叠加算法可以理解为一个两层的集成，第一层含有个体分类器，把预测的结果提供给第二层，而另一个分类器（通常是逻辑回归）拟合一级分类器的预测结果，从而作出最终的预测。戴伟·H.沃尔珀特在《神经元网络》（1992，5（2）：241–259）上发表的《叠加泛化》对叠加算法进行了更加具体的描述。不幸的是，在本书撰写时，该算法还没有包括在scikit-learn的实现中。然而，该方法正在实现中。在此期间，你可以从下列网页链接中找到与scikit-learn兼容的叠加算法实现：http://rasbt.github.io/mlxtend/user_guide/classifier/StackingClassifier/

http://rasbt.github.io/mlxtend/user_guide/classifier/StackingCVClassifier/

## 7.3 套袋——基于导引样本构建分类器集成

套袋是一种集成学习技术，它与在上一节中实现的MajorityVoteClassifier紧密相关。然而，我们并没有使用相同的训练集来拟合组合中的各个分类器，而是从初始训练集中抽取自举样本（随机可替换样本），这就是为什么套袋也被称为引导聚合，下图概述了套袋的概念：




下面几节将通过采用scikit-learn实现简单的葡萄酒分类示例来解释套袋技术。

### 7.3.1 套袋简介

为了提供一个更具体的示例，说明套袋分类器引导聚合的工作原理，让我们来考虑下面图中的示例。这里有七个不同的训练实例（用下标1-7表示），每轮套袋的样本随机可更换抽样。然后用每个引导样本来分别拟合不同的分类器$C_j$，这是最典型的修剪决策树：

| 样本索引 | 第一轮抽样 | 第二轮抽样 | … |
|---|---|---|---|
| 1 | 2 | 7 | … |
| 2 | 2 | 3 | … |
| 3 | 1 | 2 | … |
| 4 | 3 | 1 | … |
| 5 | 7 | 1 | … |
| 6 | 2 | 7 | … |
| 7 | 4 | 7 | … |

从上图可以看到，每个分类器随机接收来自于训练集的样本子集。每个子集都包含部分重复样本，有些原始样本因为抽样可重复的原因没有出现在子集中。当每个分类器拟合引导样本时，可以采用多数票的机制进行组合预测。

要注意的是套袋方法与随机森林分类法相关，这部分在第3章中介绍过。事实上，随机森林是套袋方法的一个特例，在随机森林方法中，我们在拟合每个决策树的时候也用到随机抽取的特征子集。

套袋方法是由柳·布莱曼在1994年的技术报告中首先提出，他在报告中证明了套袋方法可以提高不稳定模型的准确度同时降低过拟合的程度。高度推荐读者阅读该研究报告以对套袋方法有更详细的了解。柳·布莱曼的《套袋预测》于1996年发表在《机器学习》期刊（24（2）：123–140）。该报告也可以从网上获得。

## 7.3.2 应用套袋技术对葡萄酒数据集中的样本分类

为了理解套袋技术在实践中的具体应用，我们用葡萄酒数据集创建了一个更复杂的分类问题，该数据集曾在第4章中介绍过。这里仅考虑2、3两类葡萄酒的两个特征：Alcohol和OD280/OD315 of diluted wines。

```
>>> import pandas as pd
>>> df_wine = pd.read_csv('https://archive.ics.uci.edu/ml/'
                          'machine-learning-databases/wine/wine.data',
                          header=None)
>>> df_wine.columns = ['Class label', 'Alcohol',
...                    'Malic acid', 'Ash',
...                    'Alcalinity of ash',
...                    'Magnesium', 'Total phenols',
...                    'Flavanoids', 'Nonflavanoid phenols',
...                    'Proanthocyanins',
...                    'Color intensity', 'Hue',
...                    'OD280/OD315 of diluted wines',
...                    'Proline']
>>> # drop 1 class
>>> df_wine = df_wine[df_wine['Class label'] != 1]
>>> y = df_wine['Class label'].values
>>> X = df_wine[['Alcohol',
...              'OD280/OD315 of diluted wines']].values
```

接着用二进制为分类标签编码，然后按8：2的比例把数据集分裂成训练集和测试集：

```
>>> from sklearn.preprocessing import LabelEncoder
>>> from sklearn.model_selection import train_test_split
>>> le = LabelEncoder()
>>> y = le.fit_transform(y)
>>> X_train, X_test, y_train, y_test =\
...            train_test_split(X, y,
...                             test_size=0.2,
...                             random_state=1,
...                             stratify=y)
```

可以从本书的代码包中找到葡萄酒数据的拷贝（也包括本书讨论过的其他数据集）。也可以在脱机工作或者UCI服务器暂时宕机时，从下述网页链接直接下载：https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data

例如，如果要从本地文件目录加载葡萄酒数据可以用下面的代码：

```
df = pd.read_csv('your/local/path/to/wine.data',
                 header=None)
```

替代下面的部分：

```
df = pd.read_csv('https://archive.ics.uci.edu/ml/'
                 'machine-learning-databases'
                 '/wine/wine.data',
                 header=None)
```

scikit-learn已经实现了BaggingClassifier算法，可以从ensemble子模块导入。这里将用修剪决策树作为基本分类器并创建一个有500棵决策树的组合，这些决策树将拟合训练集中的不同引导样本：

```
>>> from sklearn.ensemble import BaggingClassifier
>>> tree = DecisionTreeClassifier(criterion='entropy',
...                               random_state=1,
...                               max_depth=None)
>>> bag = BaggingClassifier(base_estimator=tree,
...                         n_estimators=500,
...                         max_samples=1.0,
...                         max_features=1.0,
...                         bootstrap=True,
...                         bootstrap_features=False,
...                         n_jobs=1,
...                         random_state=1)
```

下一步将计算模型在训练集和测试集上的预测准确率，以便对套袋分类器的性能和单一修剪决策树的性能进行比较：

```
>>> from sklearn.metrics import accuracy_score
>>> tree = tree.fit(X_train, y_train)
>>> y_train_pred = tree.predict(X_train)
>>> y_test_pred = tree.predict(X_test)
>>> tree_train = accuracy_score(y_train, y_train_pred)
>>> tree_test = accuracy_score(y_test, y_test_pred)
>>> print('Decision tree train/test accuracies %.3f/%.3f'
...       % (tree_train, tree_test))
Decision tree train/test accuracies 1.000/0.833
```

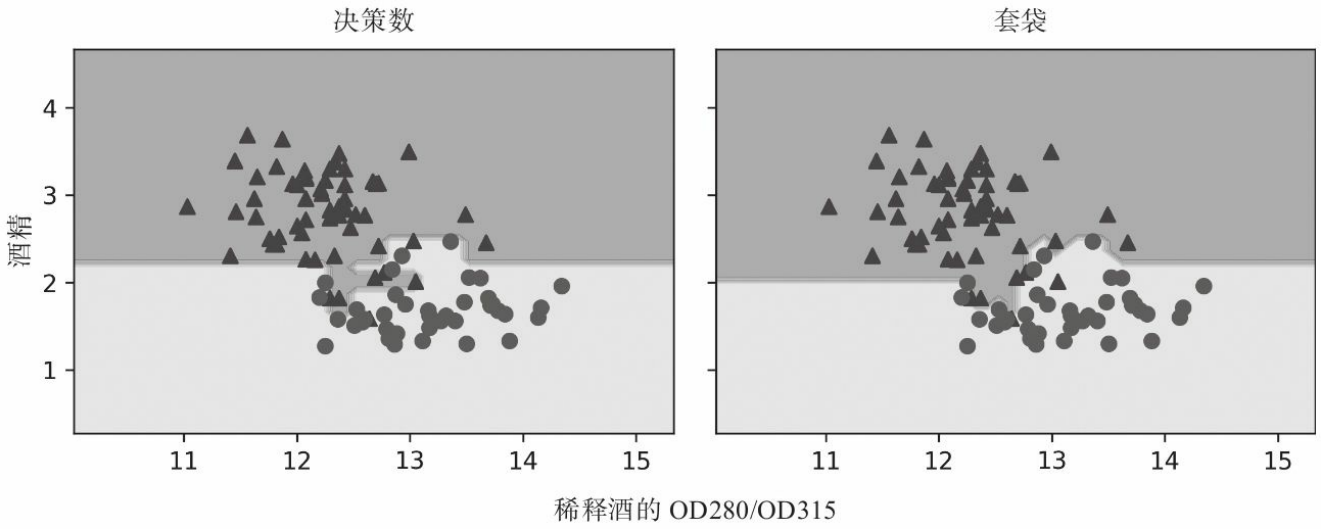
基于下面显示的准确率，修剪决策树正确地预测了所有训练样本的分类标签。然而，测试准确率相当低，这表明模型存在高方差（过拟合）：

```
>>> bag = bag.fit(X_train, y_train)
>>> y_train_pred = bag.predict(X_train)
>>> y_test_pred = bag.predict(X_test)
>>> bag_train = accuracy_score(y_train, y_train_pred)
>>> bag_test = accuracy_score(y_test, y_test_pred)
>>> print('Bagging train/test accuracies %.3f/%.3f'
...       % (bag_train, bag_test))
Bagging train/test accuracies 1.000/0.917
```

虽然在训练集上，决策树和套袋分类器的训练准确度相似（都是100%），但是可以看到，套袋分类器在测试集上估计的泛化性能略好。接下来将比较决策树和套袋分类器之间在决策区域的差别：

```
>>> x_min = X_train[:, 0].min() - 1
>>> x_max = X_train[:, 0].max() + 1
>>> y_min = X_train[:, 1].min() - 1
>>> y_max = X_train[:, 1].max() + 1
>>> xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
...                      np.arange(y_min, y_max, 0.1))
>>> f, axarr = plt.subplots(nrows=1, ncols=2,
...                         sharex='col',
...                         sharey='row',
...                         figsize=(8, 3))
>>> for idx, clf, tt in zip([0, 1],
...                         [tree, bag],
...                         ['Decision tree', 'Bagging']):
...     clf.fit(X_train, y_train)
...
...     Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
...     Z = Z.reshape(xx.shape)
...     axarr[idx].contourf(xx, yy, Z, alpha=0.3)
...     axarr[idx].scatter(X_train[y_train==0, 0],
...                        X_train[y_train==0, 1],
...                        c='blue', marker='^')
...     axarr[idx].scatter(X_train[y_train==1, 0],
...                        X_train[y_train==1, 1],
...                        c='green', marker='o')
...     axarr[idx].set_title(tt)
>>> axarr[0].set_ylabel('Alcohol', fontsize=12)
>>> plt.text(10.2, -1.2,
...          s='OD280/OD315 of diluted wines',
...          ha='center', va='center', fontsize=12)
>>> plt.show()
```

从结果图可以看到，三结点深度决策树的分段线性决策边界在套袋集成中看起来更平滑。




这一节只看了非常简单的套袋示例。实践中更复杂的分类任务和高维度数据集容易导致单个决策树模型过拟合，这正是套袋算法能真正发挥作用的地方。最后注意到套袋算法可以有效地减少模型方差。然而，套袋在减少模型偏差方面却无效，也就是说，模型过于简单以至于无法很好地捕捉数据中的趋势。这就是为什么我们想要在低偏差集成分类器上实现套袋，例如修剪决策树。

## 7.4 通过自适应增强来利用弱学习者

集成方法的最后一节将讨论增强并特别聚焦在最常见的AdaBoost（自适应增强）。

AdaBoost最初的想法是由罗伯特·E.莎普里在1990年提出的（罗伯特·E.莎普里.《弱学习能力的强度》.《机器学习》，1990，5（2）：197-227）。罗伯特·E.莎普里和尤夫·弗莱德在《第十三届国际会议会议录》（ICML 1996）上提出AdaBoost算法后，AdaBoost成为那些年及以后最广泛使用的集成方法（尤夫·弗莱德，罗伯特·E.莎普里等.《采用新增强算法的实验》.ICML，第96册，148-156，1996年）。2003年弗莱德和莎普里因为突破性的工作获得了歌德尔大奖，这是授予计算机科学领域最杰出研究成果的颇具有声望的奖项。

在增强过程中，集成是由很简单的常被称为弱学习者的基本分类器所组成，性能仅比随机猜测略优，弱学习者的典型例子是单层决策树。增强背后的关键概念是专注于难以分类的训练样本，即让弱学习者随后从训练样本的分类错误中学习以提高集成的性能。

下面的小节将介绍增强概念背后的算法过程和一种被称为AdaBoost的常见变体。最后在实际分类案例中用scikit-learn实现。

### 7.4.1 增强是如何实现的

与套袋方法，即增强初始的形式相比，该算法使用从训练集中无替换抽取训练样本的随机子集训练模型，原始的增强过程被总结为以下四个关键步骤：

1.用从训练集D无替换抽取的训练样本随机子集$d_1$来训练弱学习者$C_1$。

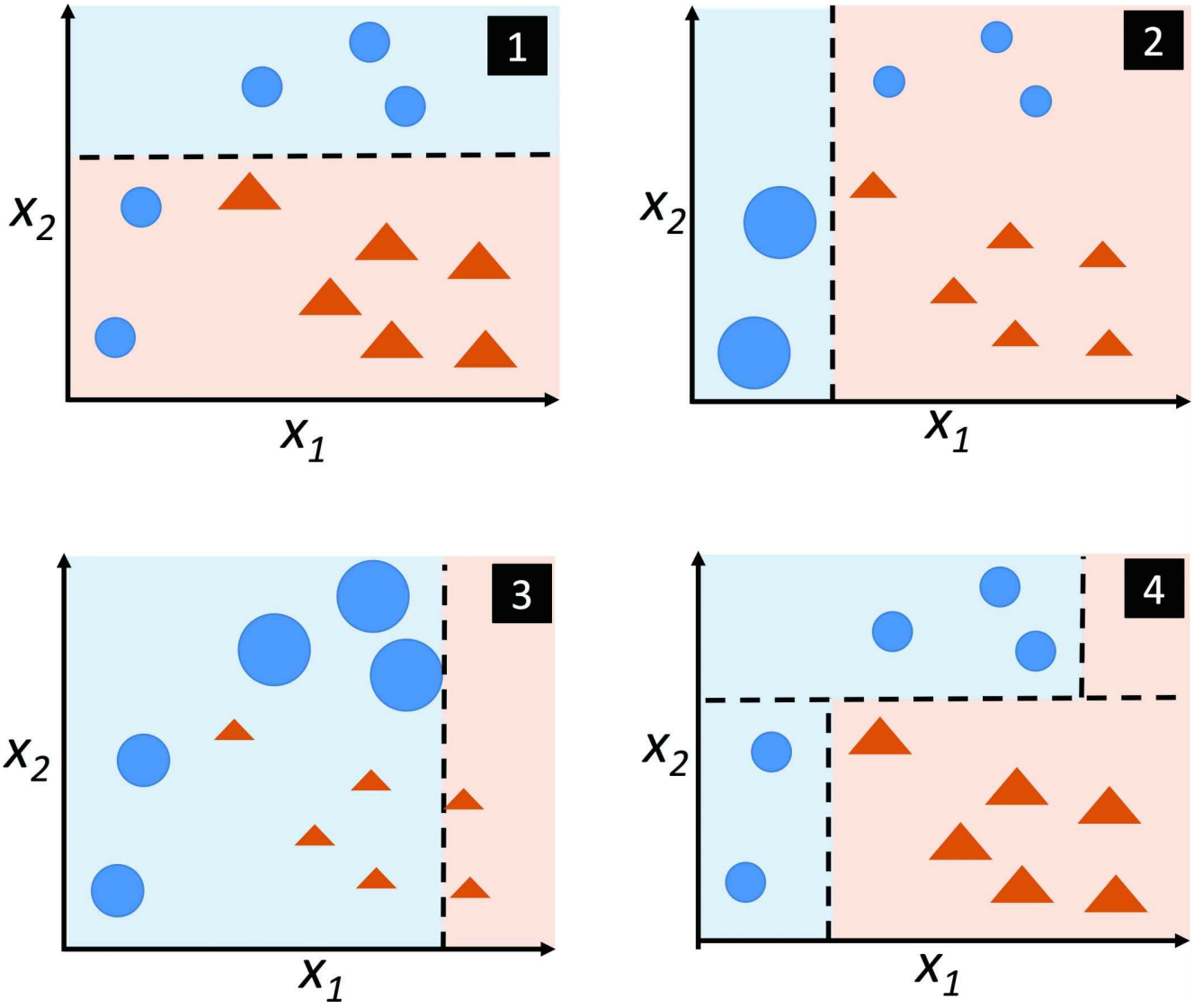
2.从训练集无替换抽取第二个随机训练子集$d_2$并把之前的分类错误样本中的50%加入该子集来训练弱学习者$C_2$。

3.从训练集D找出那些与$C_1$和$C_2$不一致的样本形成训练样本$d_3$来训练第3个弱学习者$C_3$。

4.通过多数票机制集成弱学习者$C_1$，$C_2$和$C_3$。

正如柳·布莱曼所指出的（柳·布莱曼.《偏差、方差和弧线分类器》.1996），与套袋模型相比，增强方法可导致偏差和方差减少。然而增强算法（如Adaboost）在实践中也以其高方差而闻名，即倾向于过拟合（G.瑞泰科，T.奥农达，K.R.穆勒.《改进AdaBoost以避免过拟合》.神经信息处理国际会议录，CiteSeer，1998）。

与这里所描述的原始增强过程相比，AdaBoost用完整的训练集来训练弱学习者，在每次迭代中重新定义训练样本的权重，来构建可以从集成中弱学习者的错误中不断地学习的强大的分类器。在深入研究AdaBoost算法的具体细节之前，分析下面的图，以便更好地理解AdaBoost背后的基本概念：




从子图1开始了解AdaBoost插图的详情，该图是二元分类训练集，为所有训练样本分配相同的权重。基于该训练集训练单层决策树（虚线），试图对三角形和圆形两类样本进行分类，这可能通过最小化代价函数（或在决策树集成的特殊情况下的杂质得分）完成。

下一轮（子图2）为前面误判的两个样本（圆形）分配较大的权重。此外，降低分类正确的样本的权重。下一个单层决策树将更多聚焦在权重最大的训练样本上，这些训练样本很难分类。显示在子图2中的弱学习者把三个圆形类的样本错误地分类，随后这三个样本被赋予较大的权重，如子图3所示。

假设AdaBoost集成只进行三轮增强，将在三个不同的训练子集上通过加权多数票把弱学习者集成起来，如子图4所示。

现在对AdaBoost的基本概念有了更好的理解，让我们用伪代码更详细地观察该算法。为清楚起见，用×表示元素之间的相乘，用·来表示两个矢量之间的点积：

1.把权重向量w定义为统一的权重，其中$\sum_i w_i=1$

2.For j in m轮增强，做以下的事：

a.训练有权重的弱学习者：$C_j$=train（X，y，w）

b.预测分类标签：$\hat{y}$=predict($C_j$,$\boldsymbol{X}$)

c.计算权重错误率：$\varepsilon=\boldsymbol{w}\cdot(\hat{\boldsymbol{y}}\neq\boldsymbol{y})$

d.计算系数：$\alpha_j=0.5\log\frac{1-\varepsilon}{\varepsilon}$

e.更新权重：$\boldsymbol{w}:=\boldsymbol{w}\times\exp(-\alpha_j\times\hat{\boldsymbol{y}}\times\boldsymbol{y})$

f.归一化权重使其和为1：$\boldsymbol{w}:=\boldsymbol{w}/\sum_i w_i$

3.计算最终的预测结果：

$$\hat{\boldsymbol{y}}=\left(\sum_{j=1}^{m}\left(\boldsymbol{\alpha}_j\times\text{predict}\left(C_j,\boldsymbol{X}\right)\right)>0\right)$$

注意2c步的表达式$\hat{\boldsymbol{y}}\neq\boldsymbol{y}$代表由多个0和1组成的二元向量，如果预测结果正确则数值为1，否则为0。

尽管AdaBoost算法看似简单，但我们还是用由10个训练样本组成的训练集通过更具体的例子来进一步解释，详见下表：

| 样本索引 | x | y | 权重 | ŷ(x<=3.0)？ | 正确吗？ | 更新后的权重 |
|---|---|---|---|---|---|---|
| 1 | 1.0 | 1 | 0.1 | 1 | Yes | 0.072 |
| 2 | 2.0 | 1 | 0.1 | 1 | Yes | 0.072 |
| 3 | 3.0 | 1 | 0.1 | 1 | Yes | 0.072 |
| 4 | 4.0 | -1 | 0.1 | -1 | Yes | 0.072 |
| 5 | 5.0 | -1 | 0.1 | -1 | Yes | 0.072 |
| 6 | 6.0 | -1 | 0.1 | -1 | Yes | 0.072 |
| 7 | 7.0 | 1 | 0.1 | -1 | No | 0.167 |
| 8 | 8.0 | 1 | 0.1 | -1 | No | 0.167 |
| 9 | 9.0 | 1 | 0.1 | -1 | No | 0.167 |
| 10 | 10.0 | -1 | 0.1 | -1 | Yes | 0.072 |

表的第一列为训练样本1到10的索引。第二列为单个样本的特征值，这里假设这是一维数据集。第三列显示了真正的类标签$y_i$，对每个样本$x_i$，$y_i \in \{1, -1\}$。初始的权重值显示在第四列，我们初始化每个样本时赋予它们同样的权重并对其进行归一化处理。因为是10个样本的训练集，所以为权重向量w的每个元素$w_i$赋值0.1。预测得到的分类标签$\hat{y}$显示在第五列，此处假设数据分裂标准为x≤3.0。最后一列显示了根据伪代码规则更新后的权重值。

因为权重更新的计算起初看起来有点儿复杂，现在把计算过程一步一步地分解来看。从计算的2c步描述权重错误率ε开始：

$$\varepsilon = 0.1\times 0 + 0.1\times 0 + 0.1\times 0 + 0.1\times 0 + 0.1\times 0 + 0.1\times 0 + 0.1\times 1 + 0.1\times 1 + 0.1\times 1 + 0.1\times 0 = \frac{3}{10} = 0.3$$

接着计算系数$\alpha_j$（这在2d步讨论过），将在后面的2d步更新权重时使用，第4步的多数票权重计算也会用到：

$$a_j = 0.5\log\left(\frac{1-\varepsilon}{\varepsilon}\right) \approx 0.424$$

计算了系数$\alpha_j$之后，可以通过下面的公式更新权重向量：

$$\boldsymbol{w} := \boldsymbol{w} \times \exp\left(-a_j \times \hat{\boldsymbol{y}} \times \boldsymbol{y}\right)$$

这里，$\hat{\boldsymbol{y}} \times \boldsymbol{y}$是预测结果向量与真实分类标签向量之间的元素乘法。因此如果预测结果$\hat{y}_i$正确，那么$\hat{y}_i \times y_i$将会是个正值，这样权重i降低，因为$\alpha_j$也是正数：

$$0.1 \times \exp(-0.424 \times 1 \times 1) \approx 0.065$$

与此类似，如果预测的分类结果$\hat{y}_i$不正确，我们会增加第i个权重的值：

$$0.1 \times \exp(-0.424 \times 1 \times (-1)) \approx 0.153$$

也可以是这样：

$$0.1 \times \exp(-0.424 \times (-1) \times (1)) \approx 0.153$$

在更新了权重向量中每个元素的值之后，我们把权重做归一处理（第2f步）：

$$\boldsymbol{w} := \frac{\boldsymbol{w}}{\sum_i w_i}$$

这里$\sum_i w_i = 7 \times 0.065 + 3 \times 0.153 = 0.914$.

因此，与正确分类样本相对应的每个权重将会从初始值0.1降低到0.065/0.914≈0.071，这将是下一轮增强过程的权重。与之类似，分类不正确样本的权重将会从0.1上升到0.153/0.914≈0.167。

## 7.4.2 用scikit-learn实现AdaBoost

前面的小节简单地介绍了AdaBoost。现在进入更为实用的部分，用scikit-learn训练AdaBoost集成分类器。将用前一节中提到的相同的葡萄酒数据子集来训练装套袋元分类器。基于base_estimator的属性在500棵单层决策树基础上训练AdaBoostClassifier：

```
>>> from sklearn.ensemble import AdaBoostClassifier
>>> tree = DecisionTreeClassifier(criterion='entropy',
                                  random_state=1,
...                               max_depth=1)
>>> ada = AdaBoostClassifier(base_estimator=tree,
...                          n_estimators=500,
...                          learning_rate=0.1,
...                          random_state=1)
>>> tree = tree.fit(X_train, y_train)
>>> y_train_pred = tree.predict(X_train)
>>> y_test_pred = tree.predict(X_test)
>>> tree_train = accuracy_score(y_train, y_train_pred)
>>> tree_test = accuracy_score(y_test, y_test_pred)
>>> print('Decision tree train/test accuracies %.3f/%.3f'

...       % (tree_train, tree_test))
Decision tree train/test accuracies 0.916/0.875
```

结果表明单层决策树似乎对训练数据欠拟合，这与前节中见到的修剪决策树形成对比：

```
>>> ada = ada.fit(X_train, y_train)
>>> y_train_pred = ada.predict(X_train)
>>> y_test_pred = ada.predict(X_test)
>>> ada_train = accuracy_score(y_train, y_train_pred)
>>> ada_test = accuracy_score(y_test, y_test_pred)
>>> print('AdaBoost train/test accuracies %.3f/%.3f'
...       % (ada_train, ada_test))
AdaBoost train/test accuracies 1.000/0.917
```

可以看到，AdaBoost模型正确地预测了训练集的所有分类标签，而且与单层决策树相比，其测试性能也略有改善。然而，也看到因为试图减少模型偏差，在训练集和测试集之间的性能上存在着较大的差距，因此也引入了额外的方差。

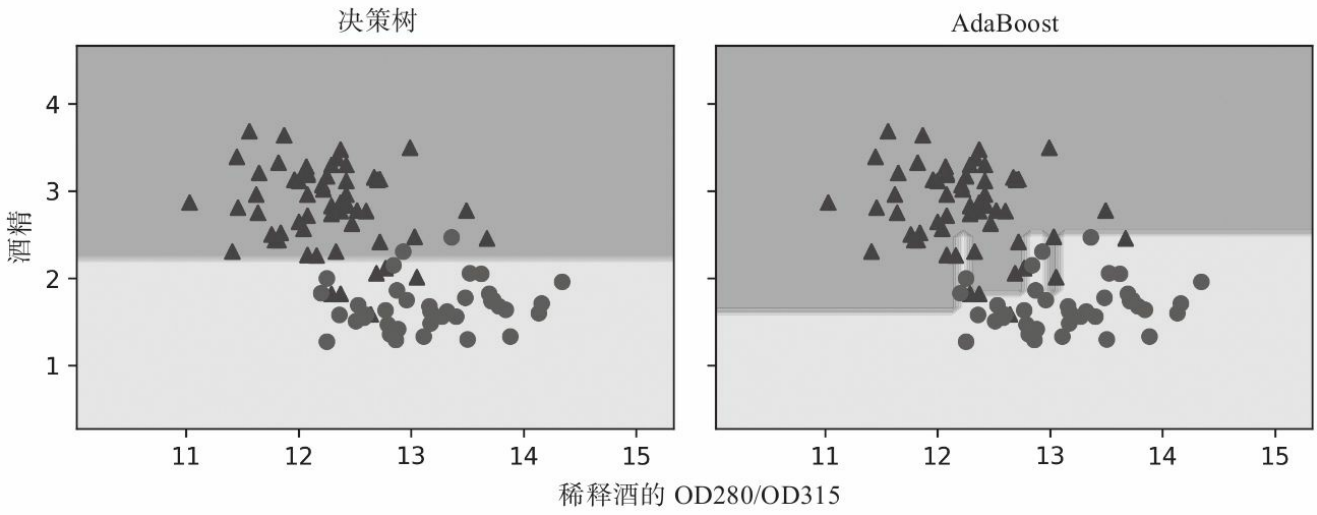
尽管为演示目的用了另外一个简单的示例，但是可以看到AdaBoost分类器的性能与单层决策树相比略有改进，并且达到了与前一节所描述的套袋分类器非常相似的准确率。但是应该注意，重复使用测试集来选择模型是不好

的做法，可能会使我们对泛化性能的评估过于乐观，第6章曾对此做过详细的讨论。

最后看看决策区域到底是什么情况：

```
>>> x_min = X_train[:, 0].min() - 1
>>> x_max = X_train[:, 0].max() + 1
>>> y_min = X_train[:, 1].min() - 1
>>> y_max = X_train[:, 1].max() + 1
>>> xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
...                      np.arange(y_min, y_max, 0.1))
>>> f, axarr = plt.subplots(1, 2,
...                         sharex='col',
...                         sharey='row',
...                         figsize=(8, 3))
>>> for idx, clf, tt in zip([0, 1],
...                         [tree, ada],
...                         ['Decision Tree', 'AdaBoost']):
...     clf.fit(X_train, y_train)
...     Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
...     Z = Z.reshape(xx.shape)
...     axarr[idx].contourf(xx, yy, Z, alpha=0.3)
...     axarr[idx].scatter(X_train[y_train==0, 0],
...                        X_train[y_train==0, 1],
...                        c='blue',
...                        marker='^')
...     axarr[idx].scatter(X_train[y_train==1, 0],
...                        X_train[y_train==1, 1],
...                        c='red',
...                        marker='o')
...     axarr[idx].set_title(tt)
...     axarr[0].set_ylabel('Alcohol', fontsize=12)
>>> plt.text(10.2, -0.5,
...          s='OD280/OD315 of diluted wines',
...          ha='center',
...          va='center',
...          fontsize=12)
>>> plt.show()
```

通过对决策区域的观察，可以看出AdaBoost模型的决策边界比单层决策树的决策边界要复杂得多。此外，还注意到AdaBoost模型划分特征空间的结果与前一节训练的套袋分类器非常相似：




作为集成技术的总结，与单个分类器相比，值得注意的是集成学习增加了计算的复杂度。需要在实践中仔细考虑是否愿意为预测性能相对温和的改善而增加计算成本。

著名的Netflix100万美元大奖是个经常被引用的案例，它是靠集成技术被赢得的。关于该算法的细节发表在2009年，由A.透彻、M.佳合尔以及R.M.贝尔撰写的《Netflix大奖大杂烩解决方案》上，见《Netflix大奖文集》，可以从下述网站链接查到该文。

http://www.stat.osu.edu/~dmsl/GrandPrize2009_BPC_BigChaos.pdf

获胜团队得到了100万美元的大奖，不过由于模型自身的复杂性，Netflix从来就没有实现该模型，因为不太可能将其应用到现实世界：

“我们离线评估了一些新方法，但是所测量到的额外准确度改善似乎并不能平衡将其引入生产环境所需耗费的工程努力。”（http://techblog.netflix.com/2012/04/netflix-recommendations-beyond-5-stars.html）。

## 7.5 小结

本章研究了一些最常见和广泛使用的集成学习技术。集成方法组合不同的分类模型来消除各自的弱点，这往往能带来稳定和性能良好的模型，对于工业应用和机器学习竞赛非常有吸引力。

本章的开始部分用Python实现了MajorityVoteClassifier，它允许组合不同的分类算法。然后研究了套袋技术，这是一种可以减少模型方差的有用技术，该方法从训练集随机提取引导样本，通过多数票机制把个别训练的分类器组合起来。最后学习了AdaBoost算法，它是一种基于弱学习者从分类错误中学习的算法。

前几章讨论了许多种不同的学习算法、调优和评估技术。下一章将讨论机器学习的特殊应用，即情感分析，它已经成为互联网和社交媒体时代的一个有趣话题。

# 第8章　应用机器学习于情感分析

在这个互联网和社会媒体时代，人们的意见、评论和建议已经成为政治科学和商业的宝贵资源。得益于现代技术人们现在才能够卓有成效地收集和分析数据。本章将深入研究自然语言处理（NLP）的一个分支，即情感分析，学习如何使用机器学习算法并根据作者的态度对文档进行分类。特别是要基于互联网电影数据库（IMDb）的50 000个电影评论数据集构建可以区分正反面评论的预测器。

本章将主要涵盖下述几个方面：

·清洗和准备文本数据

·根据文本数据建立特征向量

·训练机器学习模型来区分正面或者负面评论

·用基于外存的学习方法来处理大型文本数据集

·根据文档推断主题进行分类

## 8.1 为文本处理预备好IMDb电影评论数据

情感分析有时也被称为意见挖掘，是NLP广泛领域中的一个分支，着重于分析文档的倾向。情感分析的一个热门任务是根据作者对特定主题所表达的观点或情感为文档分类。

本章将处理IMDb的大量数据，这些数据是由马斯等人收集的电影评论（A.L.马斯，R.E.达雷，P.T.潘，D.黄，A.Y.梁，以及C.波茨.《情感分析机器学习词语向量》.49届计算语言学协会年会：人类语言技术，2011年6月：142–150，波特兰，俄勒冈，美国，计算语言学协会）。

电影评论数据集由50 000个带有倾向性的电影评论所组成，每个评论都被标记为正面或负面。这里正面评论意味着对该部电影的评价超过IMDb的六星级，而负面评论意味着对该部电影的评论低于IMDb的五星级。下面的章节将下载数据集，然后将其预处理成机器学习工具可用的格式，并从这些电影评论数据的子集中提取有意义的信息，以建立机器学习模型，然后预测某个评论者是否喜欢某部电影。

### 8.1.1 获取电影评论数据集

可以从下列网站链接下载Gzip压缩后的tarball格式的电影评论数据集（84.1 MB）：

http://ai.stanford.edu/~amaas/data/sentiment/

·如果你用Linux或者macOS系统，可以打开一个新的终端窗口，用cd命令进入下载文件目录，然后执行tar-zxf aclImdb_v1.tar.gz对数据集解压。

·如果你用Windows系统，可以从7Zip（http://www.7-zip.org）免费下载一个压缩应用来从下载的压缩文件中抽取解压后的文件。

·也可以用下述命令直接从Python中解压Gzip压缩的tarball：

```
>>> import tarfile
>>> with tarfile.open('aclImdb_v1.tar.gz', 'r:gz') as tar:
...     tar.extractall()
```

### 8.1.2 把电影评论数据预处理成更方便格式的数据

成功地提取数据后，现在将从下载的压缩文档解压出单个CSV文件。下面的代码片段将把电影评论读入pandas DataFrame对象，在标准台式电脑上这可能需要运行10分钟。为了观察进度和估计完成时间，将用几年前为此目的而研发的Python进度指示器包PyPrind（https://pypi.python.org/pypi/PyPrind/），可以通过执行pip install pyprind命令完成该包的安装。

```
>>> import pyprind
>>> import pandas as pd
>>> import os

>>> # change the `basepath` to the directory of the
>>> # unzipped movie dataset

>>> basepath = 'aclImdb'
>>>
>>> labels = {'pos': 1, 'neg': 0}
>>> pbar = pyprind.ProgBar(50000)
>>> df = pd.DataFrame()
>>> for s in ('test', 'train'):
...     for l in ('pos', 'neg'):
...         path = os.path.join(basepath, s, l)
...         for file in os.listdir(path):
...             with open(os.path.join(path, file),
...                       'r', encoding='utf-8') as infile:
...                 txt = infile.read()
...             df = df.append([[txt, labels[l]]],
...                            ignore_index=True)
...             pbar.update()
>>> df.columns = ['review', 'sentiment']
0%                            100%
[##############################] | ETA: 00:00:00
Total time elapsed: 00:03:37
```

上面的代码首先初始化新的进度条对象pbar，并定义迭代次数为50 000，这是要读入的文件数量。使用嵌套的for循环，遍历aclImdb主目录下的train和test子目录，并从子目录pos和neg下读入单个文本文件，这两个目录连同整数类标签（1=正面和0=负面）最终将会被映射到pandas的DataFrame对象df上。

因为数据集中的分类标签已经排过序，所以现在可以调用np.random子模块的permutation函数对DataFrame洗牌，这对后期将数据集分裂成训练集和测试集很有用，特别是以后的章节要从本地磁盘目录获得数据流。为了方便

起见，将组装好和洗过牌的电影评论数据集以CSV文件格式保存：

```
>>> import numpy as np

>>> np.random.seed(0)
>>> df = df.reindex(np.random.permutation(df.index))
>>> df.to_csv('movie_data.csv', index=False, encoding='utf-8')
```

本章稍后将使用此数据集，通过读取CSV文件并显示前三个示例的摘录，可以快速确认我们已经成功地将数据以合适的格式保存。

```
>>> df = pd.read_csv('movie_data.csv', encoding='utf-8')
>>> df.head(3)
```

如果用Jupyter笔记本运行代码示例，现在应该能看到该数据集的前三行，如右表所示：

| | 评论 | 情绪 |
|---|---|---|
| 0 | In 1974, the teenager Martha Moxley (Maggie Gr... | 1 |
| 1 | OK... so... I really like Kris Kristofferson a... | 0 |
| 2 | ***SPOILER*** Do not read this, if you think a... | 0 |

## 8.2 词袋模型介绍

你可能还记得在第4章中，必须将分类数据（如文本或文字）转换成数字形式，然后才能将其传递给机器学习算法。本节将介绍词袋，它将文本转换为数字类型的特征向量。词袋模型背后的逻辑非常简单，可以概括如下：

1.从整个文档集中创建一个基于独立令牌（例如单词）的词汇表。

2.为每个文档构建一个特征向量，其中包含每个词在特定文档中出现的频率。

由于每个文档的独立单词只代表了词袋词汇表中所有单词的一小部分，所以特征向量主要由零组成，因此称之为稀疏向量。如果这听起来太抽象，请不要担心，下面的小节将介绍创建简单词袋模型的具体步骤。

### 8.2.1 把词转换成特征向量

可以用scikit-learn实现的CountVectorizer类根据单词在各文件中出现的频率构建词袋模型。正如下面代码段所见到的那样，CountVectorizer以文本数据阵列（可以是文档或句子）构建词袋模型：

```
>>> import numpy as np
>>> from sklearn.feature_extraction.text import CountVectorizer
>>> count = CountVectorizer()
>>> docs = np.array([
...         'The sun is shining',
...         'The weather is sweet',
...         'The sun is shining and the weather is sweet'])
>>> bag = count.fit_transform(docs)
```

调用CountVectorizer的fit_transform方法处理构建词袋模型的词汇表，并且把以下三个句子转化为稀疏特征向量：

- 'The sun is shining'
- 'The weather is sweet'
- 'The sun is shining, the weather is sweet, and one and one is two'

现在打印出词汇表的内容，以便更好地理解其中所包含的概念：

```
>>> print(count.vocabulary_)
{'and': 0,
 'two': 7,
 'shining': 3,
 'one': 2,
 'sun': 4,
 'weather': 8,
 'the': 6,
 'sweet': 5,
 'is': 1}
```

执行前面的命令后可以看到词汇表存储在Python字典，该字典将每个独立的单词映射为整数索引。接着显示刚刚创建的特征向量：

```
>>> print(bag.toarray())
[[0 1 0 1 1 0 1 0 0]
 [0 1 0 0 0 1 1 0 1]
 [2 3 2 1 1 1 2 1 1]]
```

特征向量的每个索引位置对应于词汇存储在CountVectorizer字典项中的整数值。例如，索引位置0上的第一个特征等同于单词'and'的词频，它只出现在最后一个文档中，单词'is'在索引位置1（文档向量中的第二个特征），它出现在所有三个句子中。这些值的特征向量也叫原词频率：tf（t，d），即t项在文档中出现的次数d。

刚创建的词袋模型也被称为1克或者单克模型，词汇中的每项或每个令牌代表一个单词。更普遍的是NLP中的连续序列项，即单词、字母或者符号，也被称为n克。在n克模型中所选择的数量n取决于特定应用。例如，坎纳瑞斯和其他人的研究发现，在反垃圾邮件过滤过程中，n克模型的规模设置为3和4时性能最佳（伊安尼丝·卡纳里斯，伊安尼丝·胡瓦达斯，埃夫斯塔·西奥斯.《单词与字符的n克反垃圾邮件过滤模型》.国际人工智能工具，世界科学出版公司，2007，16（06）：1047-1067）。总结n克模型的概念，表示第一个文件“the sun is shining”的1克和2克模型的构成如下：

·1克："the"，"sun"，"is"，"shining"

·2克："the sun"，"sun is"，"is shining"

scikit-learn的CountVectorizer类通过调整参数ngram_range来使用不同的n克模型。默认情况为1克模型，可以通过始化一个新的CountVectorizer实例，同时通过定义ngram_range=（2，2）将其转换为2克模型。

## 8.2.2 通过词频逆反文档频率评估单词相关性

在分析文本数据时，经常会发现好的和坏的词在多个文档中出现。经常出现的单词基本上不包含有用或者判断性的信息。本节将介绍一种被称为词频逆反文档频率（tf-idf）的实用技术，用于减少特征向量中频繁出现的词。tf-idf可以定义为词频与逆反文档频率的乘积：

$$\text{tf-idf}(t,d)=\text{tf}(t,d)\times\text{idf}(t,d)$$

tf（t，d）为前一节引入的词频，idf（t，d）为逆反文档频率，其计算过程如下：

$$\text{idf}\left(t,d\right)=\log\frac{n_d}{1+\text{df}\left(d,t\right)}$$

nd为文档总数，df（d，t）为含有单词t的文档数量。请注意，为分母添加常数1为可选，目的在于为所有训练样本中出现的单词赋予非零值，用对数来确保低文档频率的权重不会过大。

scikit-learn实现了另外一个转换器TfidfTransformer类，它以来自于CountVectorizer类，以原始词频为输入，然后转换为tf-idfs格式：

```
>>> from sklearn.feature_extraction.text import TfidfTransformer
>>> tfidf = TfidfTransformer(use_idf=True,
...                          norm='l2',
...                          smooth_idf=True)
>>> np.set_printoptions(precision=2)
>>> print(tfidf.fit_transform(count.fit_transform(docs))
...       .toarray())
[[ 0.    0.43  0.    0.56  0.56  0.    0.43  0.    0.  ]
 [ 0.    0.43  0.    0.    0.    0.56  0.43  0.    0.56]
 [ 0.5   0.45  0.5   0.19  0.19  0.19  0.3   0.25  0.19]]
```

正如前一节所看到的，单词'is'在第三个文档中的词频最高，是最常出现的单词。然而，在把相同的特征向量转换成tf-idf之后，发现单词'is'在第三个文档中与相对较小的tf-idf（0.45）相关联，因为该页也在第一和第二个文档中出现，因此不太可能包含任何具有判断性的信息。

然而，如果手工计算特征向量中的每个单词的tf-idfs，就会注意到TfidfTransformer对tf-idf的计算与之前书中定义的标准公式稍有不同。scikit-learn实现的逆文档频率计算公式如下：

$$\text{idf}(t,d)=\log\frac{1+n_d}{1+\text{df}(d,t)}$$

类似，在scikit-learn中计算的tf-idf与前面定义的默认公式稍有不同：

$$\text{tf-idf}(t,d)=\text{tf}(t,d)\times(\text{idf}(t,d)+1)$$

在调用TfidfTransformer类直接归一化并计算tf-idf之前，归一化原始单词的词频更具代表性。定义默认参数norm='l2'，用scikit-learn的TfidfTransformer进行L2归一化，返回长度为1的向量，

$$v_{\text{norm}}=\frac{v}{\|v\|_2}=\frac{v}{\sqrt{v_1^2+v_2^2+\cdots+v_n^2}}=\frac{v}{\left(\sum_{i=1}^{n}v_i^2\right)^{\frac{1}{2}}}$$

为了确保理解TfidfTransformer的工作机制，让我们来看下在第三个文件中如何计算单词'is'的tf-idf。

单词'is'在第三个文档中的词频为3（tf=3），其文档频率也为3因为单词'is'在所有的三个文档中都出现过（df=3）。因此可以计算逆文档频率如下：

$$\text{idf}(\text{"is"},\text{d3})=\log\frac{1+3}{1+3}=0$$

现在，为了计算tf-idf，只需要在逆文档频率上加1，然后乘以词频

$$\text{tf-idf}(\text{"is"},\text{d3})=3\times(0+1)=3$$

如果重复对第三个文档中所有术语的计算，将获得tf-idf向量：[3.39，3.0，3.39，1.29，1.29，1.29，2.0，1.69，1.29]。然而，请注意，这个特征向量中的值与以前用TfidfTransformer所获得的不同。在tf-idf计算中缺少的最后一步是L2归一化，具体计算如下：

$$\begin{aligned}\text{tf-idf}(\text{d3})_{\text{norm}}&=\frac{[3.39,3.0,3.39,1.29,1.29,1.29,2.0,1.69,1.29]}{\sqrt{3.39^2+3.0^2+3.39^2+1.29^2+1.29^2+1.29^2+2.0^2+1.69^2+1.29^2}}\\&=[0.5,0.45,0.5,0.19,0.19,0.19,0.3,0.25,0.19]\end{aligned}$$

$$\text{tf-idf}(\text{"is"},\text{d3})=0.45$$

可以看到，现在的结果与调用scikit-learn的TfidfTransformer计算的结果吻合，理解了tf-idf是如何计算的，就可以进入下一节把这些概念应用到电影评论数据集上。

## 8.2.3 清洗文本数据

上一小节了解了词袋模型、单词词袋模型、词频和tf-idfs。然而，在构建词袋模型之前，第一个重要的步骤是通过去掉所有不需要的字符来清洗文本数据。为了说明为什么这个环节很重要，让我们显示电影评论数据集清洗后第一个文件的最后50个字符：

```
>>> df.loc[0, 'review'][-50:]
'is seven.<br /><br />Title (Brazil): Not Available'
```

正如在这里看到的，文本中包含HTML标记和标点符号以及其他的非字母字符。虽然HTML标记所包含的有用语义不多，但在某些NLP场景，标点符号可以包含有用的附加信息。然而，为了简单起见，将删除所有的标点符号除了表情特征如：），因为这些表情符号当然是有用的情感分析数据。为了完成这一任务，我们将使用Python的正则表达式库re，如下所示：

```
>>> import re
>>> def preprocessor(text):
...     text = re.sub('<[^>]*>', '', text)
...     emoticons = re.findall('(?::|;|=)(?:-)?(?:\)|\(|D|P)',
...                            text)
...     text = (re.sub('[\W]+', ' ', text.lower()) +
...             ' '.join(emoticons).replace('-', ''))
...     return text
```

第一轮调用正则表达式<[^>]*>试图删除电影评论数据集中的所有HTML标记。虽然很多程序员通常建议不要用正则表达式来解析HTML，但是对清洗该特定数据集应该足够有效率。在去除HTML标记之后，用稍微复杂的正则表达式来找到表情符号，然后把这些表情符号暂时存储为emoticons。接下来，从文本中通过正则表达式[\w]+去除所有的非单词字符并把文本转换为小写字符。

在这个分析场景下，我们假设大写的单词，无论该单词是否出现在句子的开头，都不含有与语义相关的信息。然而，请注意，也有例外的情况，例如我们去掉了专有名词的符号。但是，在当前场景下，这是一个简化的假设，即字母的大小写并不包含与情绪分析相关的信息。

最终在整理过的文档字符串的末尾增加临时存储的emoticons表情符号。此外，也从表情符号去除特殊字符鼻子（-）以确保一致性。

尽管正则表达式提供了一种高效而且方便的方法来搜索字符串中

的字符，但它们也伴随着陡峭的学习曲线。不幸的是对正则表达式的深入讨论超出了本书的范围。然而，你可以从谷歌开发者平台找到优秀的教程（https://developers.google.com/edu/python/regular-expressions）或者查询相关的Python官方文档（https://docs.python.org/3.6/library/re.html）。

尽管把表情字符加在清洗干净的文档字符串的结尾看起来可能不是最优雅的方法，但是我们应当注意到如果词汇中只包含单词令牌，词袋模型中词语的顺序就无关紧要了。但在更深入地讨论如何将文档分裂成单个术语、单词或记号之前，我们先确认预处理程序的工作状态：

```
>>> preprocessor(df.loc[0, 'review'][-50:])
'is seven title brazil not available'
>>> preprocessor("</a>This :) is :( a test :-)!")
'this is a test :) :( :)'
```

最后，因为在下一节中将要反复使用干净的文本数据，所以我们用preprocessor函数来处理DataFrame上的所有电影评论。

```
>>> df['review'] = df['review'].apply(preprocessor)
```

## 8.2.4 把文档处理为令牌

在成功地准备好电影评论数据集之后，现在需要考虑如何将文本语料库拆分成独立的元素。标记文件的一种方法是通过把清洗后的文件以空白字符拆分为单词：

```
>>> def tokenizer(text):
...     return text.split()
>>> tokenizer('runners like running and thus they run')
['runners', 'like', 'running', 'and', 'thus', 'they', 'run']
```

关于文档的标记化，还有另外一种有用的技术是词干，就是将单词转换为词根的过程。可以把相关的词映射到同一个词干上。原始的算法是由马丁·波特在1979年开发的，因此被称为波特分词算法（马丁·波特.《一种后缀移除算法》.程序：电子图书馆和信息系统，1980，14（3）：130-137）。Python的自然语言处理工具集（NLTK，http://www.nltk.org）实现了波特分词算法，会用在下述的代码片段中。可以直接执行conda install nltk或pip install nltk完成NLTK的安装。

尽管NLTK并不是本章的重点，但如果你对自然语言处理的更高级应用感兴趣。我强烈建议你访问NLTK网站并阅读NLTK的官方书籍，可以从http://www.nltk.org/book/免费获得。

下面的代码演示了如何使用波特词干生成算法：

```
>>> from nltk.stem.porter import PorterStemmer
>>> porter = PorterStemmer()
>>> def tokenizer_porter(text):
...     return [porter.stem(word) for word in text.split()]
>>> tokenizer_porter('runners like running and thus they run')
['runner', 'like', 'run', 'and', 'thu', 'they', 'run']
```

使用NLTK软件包的PorterStemmer函数，修改tokenizer函数把相关的词都归纳为相应的词根，这就是前面的代码段所展现的，抽取'running'其他部分后获得词根'run'。

波特词干分析算法可能是最古老且最简单的词干生成算法。其他常见的词干分析算法包括较新的雪球词干算法（也被称为第二波特算法或英语词干算法）和兰开斯特词干分析器（帕斯/稻壳词干分析器），速度更快且比波特词干分析算法更激进。这些替代性的词干分析算法可以从NLTK的软件包中获得（http://www.nltk.org/api/nltk.stem.html）。

词干分析法会创建现实中不存在的单词，比如前面的例子中展示的'thu'（源于'thus'）。词元法是一种旨在获得每个单词规范格式（语法正确）的技术。然而，词元法的计算比普通的词干提取法更难而且更贵，实践证明词元法和词干提取法在文本分类性能上差别不大（迈克·图门，罗马·特萨，凯瑞·杰之克.《单词规范化对文本分类的影响》.InScit处理2006：354–358）。

在进入下一节之前，将使用词袋模型来训练机器学习模型，让我们先简要地讨论另一个有用的主题，叫停用词删除。停用词就是那些在各种各样的文本中常见的单词，这些单词可能并不包含（或只有很少）可用来区分不同类别文档的有用信息。is，and，has，和like都是停用词的例子。去除停用词可能对处理原始或者正则词频而非tf-idfs有益，因为它已经降低了那些频繁出现单词的权重。

为了去除电影评论中的停用词，先从NLTK库下载含127个英语停用词的包，这可以通过调用nltk.download函数完成：

```
>>> import nltk

>>> nltk.download('stopwords')
```

下载完停用词集之后，可以像下面描述的这样加载和使用英语停用词：

```
>>> from nltk.corpus import stopwords

>>> stop = stopwords.words('english')
>>> [w for w in tokenizer_porter('a runner likes running and runs a
lot')[-10:] if w not in stop]

['runner', 'like', 'run', 'run', 'lot']
```

## 8.3 训练文档分类的逻辑回归模型

本节将训练一个逻辑回归模型来把电影评论分类为正面和负面。首先将清理过的文本文档DataFrame分成25000个训练文档和25000个测试文档。

```
>>> X_train = df.loc[:25000, 'review'].values
>>> y_train = df.loc[:25000, 'sentiment'].values
>>> X_test = df.loc[25000:, 'review'].values
>>> y_test = df.loc[25000:, 'sentiment'].values
```

接着调用GridSearchCV对象，采用5倍分层交叉验证方法，为逻辑回归模型寻找最佳的参数集：

```
>>> from sklearn.model_selection import GridSearchCV
>>> from sklearn.pipeline import Pipeline
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.feature_extraction.text import TfidfVectorizer

>>> tfidf = TfidfVectorizer(strip_accents=None,
...                         lowercase=False,
...                         preprocessor=None)
>>> param_grid = [{'vect__ngram_range': [(1,1)],
...                'vect__stop_words': [stop, None],
...                'vect__tokenizer': [tokenizer,
...                                    tokenizer_porter],
...                'clf__penalty': ['l1', 'l2'],
...                'clf__C': [1.0, 10.0, 100.0]},
...               {'vect__ngram_range': [(1,1)],
...                'vect__stop_words': [stop, None],
...                'vect__tokenizer': [tokenizer,
...                                    tokenizer_porter],
...                'vect__use_idf':[False],
...                'vect__norm':[None],
...                'clf__penalty': ['l1', 'l2'],
...                'clf__C': [1.0, 10.0, 100.0]}
...               ]
>>> lr_tfidf = Pipeline([('vect', tfidf),
...                      ('clf',
...                       LogisticRegression(random_state=0))])
>>> gs_lr_tfidf = GridSearchCV(lr_tfidf, param_grid,
...                            scoring='accuracy',
...                            cv=5, verbose=1,
...                            n_jobs=1)
>>> gs_lr_tfidf.fit(X_train, y_train)
```

请注意，我们高度推荐设置参数n_jobs=-1（而不是n_jobs=1），在之前的示例代码中讨论过，这样的设置可以充分利用计算机的全部处理器核以加快网格搜索速度。另外，有些Windows用户在执行前面的示例代码过程中，采用参数n_jobs=-1的设置时却发现了问题，这些问题与Windows的多重处理tokenizer和tokenizer_porter函数相关联。

避免该问题的其他方法是用[str.split]替换[tokenizer，tokenizer_porter]两个函数。然而，值得注意的是，替换后，简单的str.split函数将不支持词干抽取。

复用前面的代码，初始化GridSearchCV对象，定义网格搜索的参数，限制参数组合数量，由于特征向量和词汇很多，网格搜索的计算成本相当昂贵。如果使用标准的台式计算机，这样的网格搜索可能需要40分钟才能完成。

前面的代码示例用上一小节的TfidfVectorizer替换CountVectorizer和

TfidfTransformer，TfidfTransformer包含了转换器对象。param_grid包括了两个参数文件目录。第一个目录用TfidfVectorizer的默认设置（use_idf=true，smooth_idf=true和norm='l2'）来计算tf-idfs；为了训练原始单词的词频模型，第二目录设置use_idf=false，smooth_IDF=false，norm=None。此外，对于逻辑回归分类器本身，可以通过惩罚参数用L2和L1完成正则化，并通过定义逆正则化参数C的取值范围来比较正则化的强度。

网格搜索完成后，可以显示得到的最佳参数：

```
>>> print('Best parameter set: %s ' % gs_lr_tfidf.best_params_)
Best parameter set: {'clf__C': 10.0, 'vect__stop_words': None,
'clf__penalty': 'l2', 'vect__tokenizer': <function tokenizer at
0x7f6c704948c8>, 'vect__ngram_range': (1, 1)}
```

从上面的输出可以看到，在采用常规tokenizer，没有波特词干分析且没有停用词库的条件下，综合使用tf-idfs与正则化强度参数C为10.0的L2正则化逻辑回归分类器，得到最佳网格搜索结果。

使用此网格搜索的最佳模型，打印在训练集上的平均5折交叉验证准确率和在测试集上的分类准确率：

```
>>> print('CV Accuracy: %.3f'
...       % gs_lr_tfidf.best_score_)
CV Accuracy: 0.892
>>> clf = gs_lr_tfidf.best_estimator_
>>> print('Test Accuracy: %.3f'
...     % clf.score(X_test, y_test))
Test Accuracy: 0.899
```

结果表明机器学习模型能以90%的准确率来预测电影评论是正面还是负面。

Naïve贝叶斯分类器常用于文本分类，且在垃圾电子邮件过滤方面得到普遍的应用。该分类器实现简单且计算效率高，与其他算法相比往往在相对较小的数据集上表现良好。虽然本书不讨论Naïve贝叶斯分类器，但感兴趣的读者可以从网上找到我写的关于Naïve文本分类的文章。这些文章可以免费从arXiv获得（S.拉施卡.《Naïve贝叶斯与文本分类介绍和理论》.计算研究文库（CoRR），abs/1410.5329，2014.http://arxiv.org/PDF/1410.5329v3.PDF）。

## 8.4 处理更大的数据集——在线算法和核心学习

从上一节执行过的代码示例中你可能已经注意到，在做网格搜索时，为50 000个电影评论数据集构造特征向量的代价可能非常昂贵。在许多实际应用中，超出计算机内存的规模处理更大数据集的情况并不少见。但并不是每个人都能访问超级计算机，所以现在将采用一种被称为“核外学习”的技术，这种技术可以通过对数据集的小批增量来模拟分类器完成大型数据集的处理工作。

第2章引入了随机梯度下降的概念，这是一种每次用一个样本更新模型权重的优化算法。本节将用scikit-learn的SGDClassifier的partial_fit函数从本地驱动器直接获取流式文件，并用文件的小批次文档训练逻辑回归模型。

首先定义tokenizer函数来清理来自于movie_data.csv文件的文本数据（本章开头时已经构建了该文件），然后分解成单词，在标记的同时去除了停用词：

```
>>> import numpy as np
>>> import re
>>> from nltk.corpus import stopwords
>>> stop = stopwords.words('english')
>>> def tokenizer(text):
...     text = re.sub('<[^>]*>', '', text)
...     emoticons = re.findall('(?::|;|=)(?:-)?(?:\)|\(|D|P)',
...                            text.lower())
...     text = re.sub('[\W]+', ' ', text.lower()) \
...            + ' '.join(emoticons).replace('-', '')
...     tokenized = [w for w in text.split() if w not in stop]
...     return tokenized
```

接着定义生成器函数stream_docs，每次读入并返回一个文档：

```
>>> def stream_docs(path):
...     with open(path, 'r', encoding='utf-8') as csv:
...         next(csv) # skip header
...         for line in csv:
...             text, label = line[:-3], int(line[-2])
...             yield text, label
```

为了验证stream_docs函数是否能正常工作，可以从movie_data.csv读取第一个文件，它应该返回由评论文本以及相应分类标签所组成的元组：

```
>>> next(stream_docs(path='movie_data.csv'))
('"In 1974, the teenager Martha Moxley ... ',1)
```

现在将定义get_minibatch函数，该函数调用stream_docs读入文件流并返回大小由参数size定义的文件：

```
>>> def get_minibatch(doc_stream, size):
...     docs, y = [], []
...     try:
...         for _ in range(size):
...                 text, label = next(doc_stream)
...                 docs.append(text)
...                 y.append(label)
...     except StopIteration:
...         return None, None
...     return docs, y
```

不幸的是，因为需要把全部的单词保存在内存，所以无法调用CountVectorizer函数做核心学习。另外，TfidfVectorizer需要把训练集的所有特征向量保存在内存，以计算逆文档频率。然而，scikit-learn实现的另一个有用的向量化工具是HashingVectorizer，该工具用于文本处理并且独立于数据，调用由奥斯汀·爱珀白提出的基于哈希技术的32位MurmurHash3函数（https://sites.google.com/site/murmurhash/）：

```
>>> from sklearn.feature_extraction.text import HashingVectorizer
>>> from sklearn.linear_model import SGDClassifier
>>> vect = HashingVectorizer(decode_error='ignore',
...                          n_features=2**21,
...                          preprocessor=None,
...                          tokenizer=tokenizer)
>>> clf = SGDClassifier(loss='log', random_state=1, n_iter=1)
>>> doc_stream = stream_docs(path='movie_data.csv')
```

可以用scikit-learn 0.18以上的Perceptron（...，max_iter=1，...）取代Perceptron（...，n_iter=1，...）。这里有意采用参数n_iter，因为它仍然在scikit-learn 0.18广泛使用。

用前面的代码，通过标记化函数并设置特征数量为2**21来初始化Hashing-Vectorizer。另外，设置loss参数SGDClassifier的值为'log'，重新初始化逻辑回归分类器，请注意如果在HashingVectorizer中选择较大的特征数，可以减少哈希碰撞的机会，但是这么做也增加了逻辑回归模型系数的个数。在所有的函数都调优之后，我们可以开始用下面的代码进行核外学习了：

```
>>> import pyprind
>>> pbar = pyprind.ProgBar(45)
>>> classes = np.array([0, 1])
>>> for _ in range(45):
...     X_train, y_train = get_minibatch(doc_stream, size=1000)
...     if not X_train:
...         break
...     X_train = vect.transform(X_train)
...     clf.partial_fit(X_train, y_train, classes=classes)
...     pbar.update()
0%                          100%
[##############################] | ETA: 00:00:00
Total time elapsed: 00:00:39
```

另外，为了用PyPrind软件包来评估机器学习算法的进度。我们初始化进度条对象，并且定义迭代次数为45，在下面的for循环中，迭代45个迷你批次的文档，每个迷你批次都包括1000个文档。在完成增量学习后，将用至少5000个文档来评估模型的性能。

```
>>> X_test, y_test = get_minibatch(doc_stream, size=5000)
>>> X_test = vect.transform(X_test)
>>> print('Accuracy: %.3f' % clf.score(X_test, y_test))
Accuracy: 0.878
```

正如所看到的，模型的准确率约为88%，略低于前一节用网格搜索确定超参数时所取得的准确率。然而，核外学习的内存效率很高，只需少于1分钟的时间就能完成。最后，可以用前面的5000个文档来更新模型：

```
>>> clf = clf.partial_fit(X_test, y_test)
```

如果打算继续阅读第9章，我建议保持当前Python的会话状态。下一章将用刚训练过的模型来学习如何将状态保存到磁盘以供日后使用以及如何将模型嵌入到网络应用。

另外一种比词袋更现代化的模型是word2vec，该算法由谷歌在2013年发布（T.米克罗夫，K.陈，G.考拉多，J.丁.《矢量空间中词表达的有效估计》.arXiv预印本arXiv：1301.3781，2013）。word2vec算法是基于神经元网络的无监督学习算法，试图自动学习单词之间的关系。word2vec的逻辑是将含义相似的单词放入相似的群，并且通过巧妙的矢量空间，该模型可以用简单的矢量计算再现某些单词，例如，king–man+woman=queen.关于基于C语言的代码实现、相关论文链接以及其他语言的实现案例可在https://code.google.com/p/word2vec/找到。

## 8.5 具有潜在狄氏分配的主题建模

主题建模描述了为无标签文本文档分配主题这个范围很广的任务。例如，典型应用是在大文本语料库对不确定哪个特定页面或类别中出现的报纸文章这样的文档进行分类。主题建模应用的目标是为这些文章指定分类标签，例如体育、金融、世界新闻、政治、当地新闻等。因此，就第1章所讨论的广泛的机器学习范畴而言，可以将主题建模看作是一个聚类任务，即无监督学习的子类别。

本节将介绍一种常用的被称为潜在狄氏分配（LDA）的主题建模技术。然而，请注意，虽然潜在狄氏分配通常缩写为LDA，但不要与线性判别分析混淆，那是一种有监督的降维技术，第5章曾经介绍过。

LDA不同于本章所用的将电影评论分为正面和负面的有监督学习方法。因此，如果有意用以电影评论员为示例将scikit learn模型嵌入到网络应用例如Flask框架，可以随时跳到下一章，以后再回来阅读有关主题建模的讨论，本章是完全独立的章节。

### 8.5.1 使用LDA分解文本文档

由于LDA涉及许多数学知识，需要了解贝叶斯推理，我们将从实操的角度来探讨这个问题，用通俗易懂的术语解释LDA。然而，有兴趣的读者可以阅读下面的研究论文以了解更多关于LDA的知识：大卫·布雷，安德鲁·梁，麦克·乔丹.《潜在狄氏分配》.机器学习研究3，2003年1月：993-1022。

LDA是一种生成概率模型，试图找出经常出现在不同文档中的单词。假设每个文档都是由不同单词组成的混合体，那么经常出现的单词就代表着主题。LDA的输入是在本章前面讨论过的词袋模型。LDA将把词袋矩阵作为输入然后分解成两个新矩阵：

·文档主题矩阵

·单词主题矩阵

LDA以这样的方式来分解词袋矩阵，即如果把分解后的两个矩阵相乘会还原成原来输入的词袋矩阵，而且出现错误的机会最小。在实践中，我们对LDA在词袋矩阵中所发现的主题感兴趣。LDA唯一的缺点可能是必须预先定义好主题数量，这是必须手动定义的一个超参数。

## 8.5.2 LDA与scikit-learn

本节将用scikit-learn的LatentDirichletAllocation类来分解电影评论数据集，然后将它们归入不同的主题。下面的例子把主题数目限制在10个以内，但我们鼓励读者测试算法的超参数并继续探索这个数据集中的其他主题。

首先将本章开始时产生的本地电影评论文件数据集movie_data.csv加载到pandas的DataFrame：

```
>>> import pandas as pd
>>> df = pd.read_csv('movie_data.csv', encoding='utf-8')
```

下一步将用已经熟悉的CountVectorizer创建词袋矩阵作为LDA的输入。为方便起见，将通过设置参数stop_words='english'用scikit-learn内置的英文停用词库：

```
>>> from sklearn.feature_extraction.text import CountVectorizer
>>> count = CountVectorizer(stop_words='english',
...                         max_df=.1,
...                         max_features=5000)
>>> X = count.fit_transform(df['review'].values)
```

请注意把要考虑单词的最大文档频率设置为10%（max_df=.1），以排除在文档间频繁出现的那些单词。删除频繁出现的单词，其背后的逻辑是这些单词可能是在所有文档中都出现的常见单词，因此不太可能与给定文档特定主题的类别相关联。此外，把要考虑单词的数量限制为最常出现的5 000个单词（max_features=5000），以限制此数据集的维度，加快LDA的推理速度。但是max_df=.1和max_features=5000是我所选择的超参数值，鼓励读者们在比较结果时对其进一步调优。

下面的代码示例演示了如何拟合应用于词袋矩阵的LatentDirichletAllocation评估器，并从文档中推断出10个不同的主题（请注意拟合模型在笔记本电脑或标准桌面计算机上可能需要运行五分钟或更长的时间）：

```
>>> from sklearn.decomposition import LatentDirichletAllocation
>>> lda = LatentDirichletAllocation(n_topics=10,
...                                 random_state=123,
...                                 learning_method='batch')
>>> X_topics = lda.fit_transform(X)
```

通过设置参数learning_method='batch'，让lda评估器在一次迭代中根据所有可用的训练数据（词袋矩阵）进行估计，这比在线学习方法慢，但可能会

带来更准确的预测结果（设置参数learning_method='online'，则与第2章以及本章讨论的在线或小批量学习类似）。

scikit-learn的LDA实现采用期望最大化（EM）算法来迭代更新参数估计。本章尚未讨论EM算法，但如果想了解更多信息，请参阅维基百科上的精彩概述（https://en.wikipedia.org/wiki/Expectation-maximization_algorithm）以及科罗拉多里德讲解的如何使用LDA的详细教程《LDA：迈向更深入的了解》，该教程可以免费从下述网站获得：http://obphio.us/pdfs/lda_tutorial.pdf。

拟合LDA之后，现在可以访问lda实例的components_属性，该属性存储包含每10个主题的按升序排列的单词重要性（此处为5000）的矩阵。

```
>>> lda.components_.shape
(10, 5000)
```

为了分析，我们列出10个主题中每个主题的5个最重要的单词。请注意，单词的重要性按升序排列。因此，要显示这前五个单词，需要按相反的顺序对主题阵列排序：

```
>>> n_top_words = 5
>>> feature_names = count.get_feature_names()
>>> for topic_idx, topic in enumerate(lda.components_):
...     print("Topic %d:" % (topic_idx + 1))
...     print(" ".join([feature_names[i]
...                     for i in topic.argsort()\
...                         [:-n_top_words - 1:-1]]))
```

```
Topic 1:
worst minutes awful script stupid
Topic 2:
family mother father children girl
Topic 3:
american war dvd music tv
Topic 4:
human audience cinema art sense
Topic 5:
police guy car dead murder
Topic 6:
horror house sex girl woman
Topic 7:
role performance comedy actor performances
Topic 8:
series episode war episodes tv
Topic 9:
book version original read novel
Topic 10:
action fight guy guys cool
```

通过阅读每个主题最重要的5个单词可以猜测LDA发现了以下的主题：

1.差的电影（非真正主题类别）

2.家庭电影

3.战争电影

4.艺术电影

5.犯罪电影

6.恐怖电影

7.喜剧电影

8.电视电影

9.书籍电影

10.动作电影

为了确定基于评论的分类有道理，选出三部恐怖电影的主要情节描述（恐怖片在索引位置5属于类别6）：

```
>>> horror = X_topics[:, 5].argsort()[::-1]
>>> for iter_idx, movie_idx in enumerate(horror[:3]):
...     print('\nHorror movie #%d:' % (iter_idx + 1))
...     print(df['review'][movie_idx][:300], '...')
Horror movie #1:
House of Dracula works from the same basic premise as House of
Frankenstein from the year before; namely that Universal's three most
famous monsters; Dracula, Frankenstein's Monster and The Wolf Man are
appearing in the movie together. Naturally, the film is rather messy
therefore, but the fact that ...

Horror movie #2:
Okay, what the hell kind of TRASH have I been watching now? "The
Witches' Mountain" has got to be one of the most incoherent and insane
Spanish exploitation flicks ever and yet, at the same time, it's also
strangely compelling. There's absolutely nothing that makes sense here
and I even doubt there  ...

Horror movie #3:
<br /><br />Horror movie time, Japanese style. Uzumaki/Spiral was a
total freakfest from start to finish. A fun freakfest at that, but at
times it was a tad too reliant on kitsch rather than the horror. The
story is difficult to summarize succinctly: a carefree, normal teenage
girl starts coming fac ...
```

用前面的代码示例打印出三部恐怖片的前300个描述字符，可以看到，尽管我们并不知道它们确切地属于哪类，但这些评论听起来像是针对恐怖片的（不过，有人可能会争辩说，恐怖电影#2也可能属于主题类别1：差的电影）。

## 8.6 小结

本章学习了如何利用机器学习算法根据文本文档的倾向性对其分类，这是NLP领域情感分析的基本任务。本章不仅学习了如何使用词袋模型将文档编码为特征向量，而且还学习了如何使用tf-idf通过相关性对词频进行加权。

由于在该过程中创建了大量特征向量，处理这样的文本数据在计算上可能非常昂贵。上一节学习了如何利用外部存储或增量学习来训练机器学习算法，而不需要将整个数据集加载到计算机内存。

最后引入了LDA主题建模的概念，以无监督的方式将电影评论分为不同的类别。

下一章中我们将使用自己实现的文档分类器并学习如何将其嵌入到网络应用。

# 第9章　将机器学习模型嵌入网络应用

前几章了解了许多不同的机器学习概念和算法，这些算法可以帮助我们更好和更高效地进行决策。然而，机器学习技术并不局限于离线应用和分析，它们也可以成为网络服务的预测引擎。例如，网络应用中常用和有价值的机器学习模型应用包括：提交表单中的垃圾检测、搜索引擎、媒体或购物门户推荐系统等。

本章学习如何将机器学习模型嵌入网络应用，让应用不仅可以进行实时分类，而且还可以从数据中学习。本章将主要涵盖下述几个方面：

·保存训练过的机器学习模型的当前状态

·用SQLite数据库做存储

·用常用的Flask网络框架研发网络应用

·把机器学习应用部署到面向公众的网络服务器上

## 9.1 序列化拟合scikit-learn评估器

训练机器学习模型的计算成本相当昂贵，正如在第8章中所看到的那样。我们当然不想在每次退出Python解释器之后，当希望进行新的预测或重新加载网络应用时，又要重新训练模型。其中一种模型持久化解决方案是使用Python内置的pickle模块（https://docs.python.org/3.6/library/pickle.html），使用该模块可以通过序列化和反序列化，将Python对象结构压缩为字节码，以存储分类器的当前状态，而且当需要再次加载分类器来分类新样本时，没有必要再把需要的模型在所有训练数据上重新训练一遍。在执行下面的代码之前，请确保已经在第8章最后一节训练了核心逻辑回归模型，并在当前的Python会话中已做好准备。

```
>>> import pickle
>>> import os
>>> dest = os.path.join('movieclassifier', 'pkl_objects')
>>> if not os.path.exists(dest):
...     os.makedirs(dest)

>>> pickle.dump(stop,
...              open(os.path.join(dest, 'stopwords.pkl'),'wb'),
...              protocol=4)
>>> pickle.dump(clf,

...              open(os.path.join(dest, 'classifier.pkl'), 'wb'),
...              protocol=4)
```

前面的代码所创建的movieclassifier目录将在以后用来存储网络应用文件和数据。在movieclassifier目录下创建pkl_objects子目录，以把序列化的Python对象保存到本地磁盘。调用pickle模块的dump方法，序列化训练过的逻辑回归模型和自然语言工具库（NLTK）的停用词数据集，这样就不必在服务器上安装NLTK词汇表了。

dump方法把想要处理的对象作为第一个参数，把所提供的打开文件对象作为第二个参数，把Python对象写入打开文件的对象。通过open函数中的参数wb以二进制模式打开要处理的文件，设置protocol=4来选择Python 3.4中添加的最新和效率最高的处理协议，该协议与Python 3.4或更新版本兼容。如果在使用protocol=4时遇到问题，请检查是否使用了最新的Python 3版本。或者可以考虑选择较低的版本。

逻辑回归模型包含几个NumPy阵列，如权重向量。NumPy阵列更有效的序列化方式是使用joblib库。为了确保与稍后将使用的服务器环境兼容，我们将使用标准的pickle处理方法。如果有兴趣可以从下述网站的链接中找到更多关于joblib的信息：

https://pypi.org/project/joblib

因为它不需要拟合，所以也不需要处理HashingVectorizer。相反，可以创建一个新的Python脚本文件把向量导入到现有的Python会话。现在，复制下面的代码并将其保存在movieclassifier目录下的vectorizer.py文件中：

```
from sklearn.feature_extraction.text import HashingVectorizer
import re
import os
import pickle

cur_dir = os.path.dirname(__file__)
stop = pickle.load(open(
                os.path.join(cur_dir,
                'pkl_objects',
                'stopwords.pkl'), 'rb'))

def tokenizer(text):
    text = re.sub('<[^>]*>', '', text)
    emoticons = re.findall('(?::|;|=)(?:-)?(?:\)|\(|D|P)',
                           text.lower())
    text = re.sub('[\W]+', ' ', text.lower()) \
                + ' '.join(emoticons).replace('-', '')
    tokenized = [w for w in text.split() if w not in stop]
    return tokenized

vect = HashingVectorizer(decode_error='ignore',
                         n_features=2**21,
                         preprocessor=None,
                         tokenizer=tokenizer)
```

在处理完Python对象并且创建了vectorizer.py文件之后，重新启动Python解释器或IPythonNotebook内核，测试是否可以正确地恢复序列化对象。

请注意打开来自于不信任来源的数据可能会带来潜在的安全风险，因为pickle模块无法抵御恶意代码。由于pickle被用来处理任意对象的序列化，unpickle的进程将执行存储在pickle文件上的代码。因此，如果从不受信任的来源接收pickle文件（例如从网上下载），请务必多加小心，最好应该把项目unpickle在虚拟系统或者不敏感、不存储重要数据的系统里，而且除了你能访问外，其他人无法访问。

从终端进入movieclassifier目录，打开一个新的Python会话，同时执行以下的代码来验证是否可以导入vectorizer并unpickle分类器：

```
>>> import pickle
>>> import re
>>> import os
>>> from vectorizer import vect
>>> clf = pickle.load(open(
...        os.path.join('pkl_objects',
...                     'classifier.pkl'), 'rb'))
```

在成功加载vectorizer和unpickle分类器之后，现在可以用这些对象来预处理文件样本并对其所表达的情绪做出预测：

```
>>> import numpy as np
>>> label = {0:'negative', 1:'positive'}

>>> example = ['I love this movie']
>>> X = vect.transform(example)
>>> print('Prediction: %s\nProbability: %.2f%%' %\
...       (label[clf.predict(X)[0]],
...        np.max(clf.predict_proba(X))*100))
Prediction: positive
Probability: 91.56%
```

由于分类器将分类标签的返回值定义为整数，所以定义了简单的Python字典来将这些整数映射为相应的情感。然后用HashingVectorizer把简单的示例文档变换为单词向量X。最后用逻辑回归分类器的predict方法来预测分类标签，同时调用predict_proba函数返回与前面预测相对应的概率。请注意，调用predict_proba方法返回一个阵列，其中每个元素与每个独立分类标签的概率相对应。由于具有最大概率的分类标签与调用predict函数返回的分类标签相对应，所以调用np.max函数返回预测的分类概率。

## 9.2 搭建SQLite数据库存储数据

本节将搭建一个简单的SQLite数据库，收集网络应用中用户对预测的可选反馈。可以用这个反馈来更新分类模型。SQLite是开源的SQL数据库引擎，不需要在单独的服务器上运行，这使其非常适合小项目和简单的网络应用。从本质上讲，可以把SQLite数据库理解为一个单一的、独立的数据库文件，它允许直接访问存储文件。

此外，SQLite不需要任何特定的系统配置，而且支持所有常见的操作系统。并且因为它被像谷歌、Mozilla、Adobe、苹果、微软等主流公司广泛地使用，所以其可靠性已经获得认可。如果想了解更多关于SQLite的信息，建议访其官方网站：http://www.sqlite.org。

幸运的是，遵循Python提供便捷配套服务的哲学，已经有存在于Python标准库的API sqlite3，允许我们直接使用SQLite数据库（更多关于sqlite3的信息请访问下述网站https://docs.python.org/3.6/library/sqlite3.html）。

执行下述代码将在movieclassifier目录下建立SQLite数据库并存储两个电影评论的样例：

```
>>> import sqlite3
>>> import os

>>> if os.path.exists('reviews.sqlite'):
...     os.remove('reviews.sqlite')
>>> conn = sqlite3.connect('reviews.sqlite')
>>> c = conn.cursor()
>>> c.execute('CREATE TABLE review_db'\
...           ' (review TEXT, sentiment INTEGER, date TEXT)')

>>> example1 = 'I love this movie'
>>> c.execute("INSERT INTO review_db"\
...           " (review, sentiment, date) VALUES"\
...           " (?, ?, DATETIME('now'))", (example1, 1))

>>> example2 = 'I disliked this movie'
>>> c.execute("INSERT INTO review_db"\
...           " (review, sentiment, date) VALUES"\
...           " (?, ?, DATETIME('now'))", (example2, 0))
>>> conn.commit()
>>> conn.close()
```

前面的代码示例通过调用sqlite3库的connect方法创建了一个到SQLite数据库文件的连接（conn），这也同时在movieclassifier目录下创建了新的数据

库文件reviews.sqlite，如果该文件不存在的话。请注意，如果想要再执行一次代码，需要先从文件浏览器上手动删除数据库文件，因为SQLite没有实现对现有数据表的替换功能。

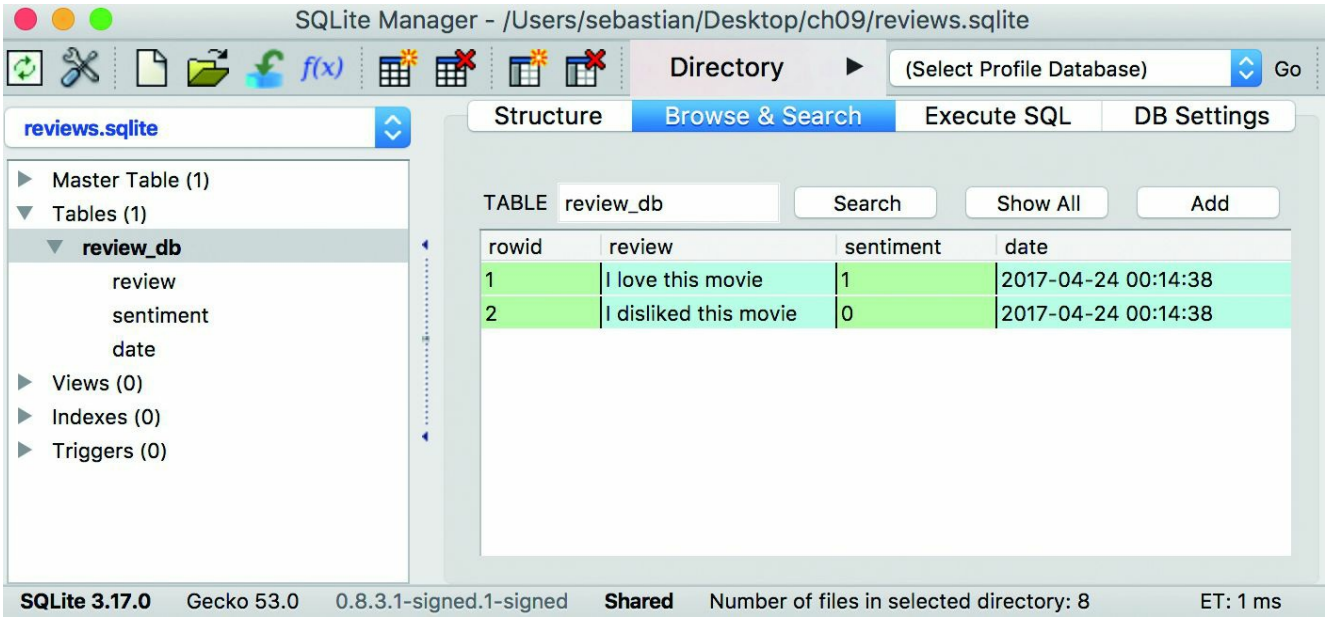
接着通过调用cursor方法创建游标，它允许我们用通用的SQL语法遍历数据库记录。通过第一次调用execute命令创建新的数据库表review_db。用该表来存储和访问数据库记录。与review_db一起创建的还有数据库表的三个列，即review、sentiment和date。该库用来存储两个电影评论样例以及相关的分类标签（情绪）。

执行SQL命令DATETIME（'now'）为记录增加日期和时间戳。除了时间戳以外，也用问号（？）来把电影评论的文本（example1和example2）及其相应的分类标签（1和0）作为位置参数传递给execute方法作为元组成员。最后调用commit方法保存对数据库所做的变更，并通过调用close方法关闭连接。

为了检查记录是否已经正确地存储在数据库表中，现在重新打开数据库的连接，并用SQL的SELECT命令获取从2017年年初到今天所提交的所有数据库表的行：

```
>>> conn = sqlite3.connect('reviews.sqlite')
>>> c = conn.cursor()
>>> c.execute("SELECT * FROM review_db WHERE date"\
... " BETWEEN '2017-01-01 00:00:00' AND DATETIME('now')")
>>> results = c.fetchall()

>>> conn.close()
>>> print(results)
[('I love this movie', 1, '2017-04-24 00:14:38'),
 ('I disliked this movie', 0, '2017-04-24 00:14:38')
```

另外，还可以用免费的Firefox浏览器插件SQLite管理器（可以从https://addons.mozilla.org/en-US/firefox/addon/sqlite-manager/下载），它提供了与SQLite数据库很好的用户图形交互界面（GUI），如下图所示：


SQLite Manager - /Users/sebastian/Desktop/ch09/reviews.sqlite
Directory
(Select Profile Database)
Go
reviews.sqlite
Structure
Browse & Search
Execute SQL
DB Settings
Master Table (1)
Tables (1)
review_db
review
sentiment
date
Views (0)
Indexes (0)
Triggers (0)
TABLE review_db
Search
Show All
Add
rowid
review
sentiment
date
1
I love this movie
1
2017-04-24 00:14:38
2
I disliked this movie
0
2017-04-24 00:14:38
SQLite 3.17.0
Gecko 53.0
0.8.3.1-signed.1-signed
Shared
Number of files in selected directory: 8
ET: 1 ms

## 9.3 用Flask开发网络应用

上一节为电影评论分类准备好了代码，现在讨论开发网络应用的基本框架。阿明·容那车于2010年首次发布了Flask，该框架多年来已经获得了巨大的人气，包括LinkedIn和Pinterest都是使用Flask框架的常用应用范例。由于Flask是用Python编写的，所以它为Python程序员提供了方便的接口来嵌入现有的Python代码，例如电影分类器。

Flask也被称为微框架，这意味着它保持了精简的核心，但可以很容易用其他库来扩展。虽然轻量化的Flask API的学习曲线不像其他流行Python网络框架如Django那么陡峭，但是我鼓励你去Flask的官方网站阅读更多相关的文档，以学习更多的功能。（http://flask.pocoo.org/docs/0.12/。）

如果当前的Python环境尚未安装Flask库，那么可以在终端上执行conda或者pip直接安装（在本书写作时，最新的稳定版本为0.12.1）：

```
conda install flask
# or: pip install flask
```

### 9.3.1 第一个Flask网络应用

这一小节将开发一个非常简单的网络应用，以便在实现电影分类器之前更加熟悉Flask API。要构建的第一个应用由简单的网络页面所组成，含有允许输入姓名的表单字段。把名字提交给网络应用后，该名字将出现在新页面上。虽然这是网络应用中非常简单的示例，但它有助于训练直觉，理解在Flask框架下代码的不同部分之间如何存储和传递变量与值。

首先创建一个目录树：

```
1st_flask_app_1/
    app.py
    templates/
        first_app.html
```

Python解释器通过执行包含主要代码的app.py文件来运行Flask框架下的网络应用。Flask将在templates目录下寻找静态HTML文件，以在网络浏览器中呈现。现在来看看app.py的内容：

```
from flask import Flask, render_template

app = Flask(__name__)
@app.route('/')
def index():
    return render_template('first_app.html')

if __name__ == '__main__':
    app.run()
```

看过前面的代码示例后，可以逐步讨论代码的各个组成部分：

1.我们把应用作为单一模块运行；使用__name__来初始化新的Flask实例，其目的是让Flask知道可以在同一目录找到HTML的模板文件夹（templates）。

2.接着用路由装饰器（@app.route（'/'））来指定应该触发执行index函数的URL。

3.这里的index函数直接渲染模板文件夹的HTML文件first_app.html。

4.最后，仅在服务器上由Python解释器直接执行脚本运行应用时才调用run函数，这通过使用if语句对__name__=='__main__'进行判断来实现。

现在查看一下文件first_app.html的内容：

```
<!doctype html>
<html>
  <head>
    <title>First app</title>
  </head>
  <body>
    <div>Hi, this is my first Flask web app!</div>
  </body>
</html>
```

如果对HTML的语法还不熟悉，我推荐你去下述网站学习有关HTML基本知识的有用教程：

https://developer.mozilla.org/en-US/docs/Web/HTML

这里简单地用了一个空的HTML模板文件，用 （一个块级元素）包含了这句话：Hi，this is my first Flask web app！。

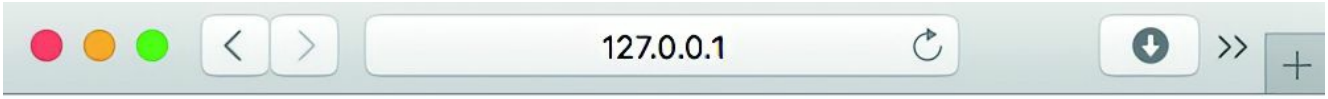
Flask允许我们很方便地在本地运行应用，这对于在公共网络服务器上部署网络应用之前进行开发和测试非常有用。现在，从终端在1st_flask_app_1目录执行下述命令以启动网络应用：

```
python3 app.py
```

终端屏幕上应当显示下面这样的一行信息：

```
* Running on http://127.0.0.1:5000/
```

该行包含本地服务器的地址。你可以在网络浏览器中输入该地址，以观察网络应用的运行情况。如果一切都执行得正确，那么应该看到如下图所示的简单的网站内容：“Hi，this is my first Flask web app！”：



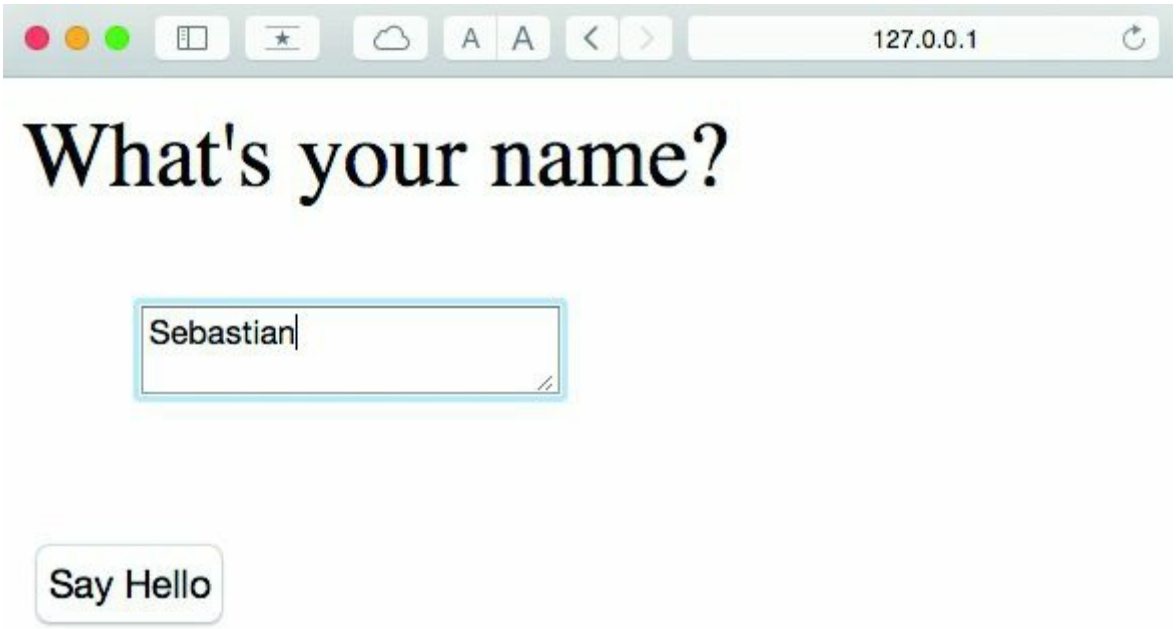

Hi, this is my first Flask web app!

## 9.3.2 表单验证与渲染

本小节将把简单的Flask网络应用扩展为带有表格元素的HTML，学习如何使用WTForms库从用户那里收集数据（https://wtforms.readthedocs.org/en/latest/）。该软件可以通过conda或pip安装：

```
conda install wtforms
# or pip install wtforms
```

该网络应用将提示用户输入他的名字到文本字段中，如下面的截图所示：




点击提交按钮（Say hello），表单得到验证后将启用新的HTML页面来显示用户名字：

### 9.3.2.1 创建目录结构

需要为这个应用创建如下所示的新目录结构：

```
1st_flask_app_2/
        app.py
        static/
            style.css
        templates/
            _formhelpers.html
            first_app.html
            hello.html
```

下面是修改后的app.py文件的内容：

```
from flask import Flask, render_template, request
from wtforms import Form, TextAreaField, validators

app = Flask(__name__)

class HelloForm(Form):
    sayhello = TextAreaField('',[validators.DataRequired()])

@app.route('/')
def index():
    form = HelloForm(request.form)
    return render_template('first_app.html', form=form)

@app.route('/hello', methods=['POST'])
def hello():
    form = HelloForm(request.form)
    if request.method == 'POST' and form.validate():
        name = request.form['sayhello']
        return render_template('hello.html', name=name)
    return render_template('first_app.html', form=form)
if __name__ == '__main__':
    app.run(debug=True)
```

下面分步讨论前面的代码：

1.我们使用wtforms扩展index函数，增加了嵌在开始页面上的文本域，该域属于TextAreaField类，可以自动检查用户是否输入了要验证的文本。

2.定义一个新函数hello，在验证HTML表单后渲染hello.html的HTML页面。

3.用POST方法把消息体的表单数据传递给服务器。最后设置参数debug=True并调用app.run方法进一步激活Flask的调试器。该功能对网络应用

研发很有用。

#### 9.3.2.2 通过Jinja2模板引擎实现一个宏

现在，在_formhelpers.html文件中通过Jinja2模板引擎实现一个通用的宏，然后将它导入到first_app.html文件以渲染文本字段：

```
{% macro render_field(field) %}
  <dt>{{ field.label }}
  <dd>{{ field(**kwargs)|safe }}
  {% if field.errors %}
    <ul class=errors>
  {% for error in field.errors %}
    <li>{{ error }}</li>
  {% endfor %}
  </ul>
{% endif %}
</dd>
</dt>
{% endmacro %}
```

关于Jinja2模板语言的更详细讨论超出了本书的范围。然而，可以在下述网站找到更全面的关于Jinja2语法的描述文档：http://jinja.pocoo.org。

#### 9.3.2.3 通过CSS添加样式

为了演示如何改变HTML文档的外观，下一步将创建一个简单的层叠样式表（CSS）文件style.css。如果想把HTML正文字体的大小增加一倍，就必须把下面的CSS文件存入static子目录，Flask在这个默认目录中寻找像CSS这样的静态文件。该文件的内容如下：

```
body {
      font-size: 2em;
}
```

下面的代码是修改后的first_app.html文件，用来渲染供用户输入名字的文本表单：

```
<!doctype html>
<html>
  <head>
    <title>First app</title>
      <link rel="stylesheet" href="{{ url_for('static',
       filename='style.css') }}">
  </head>
  <body>
    {% from "_formhelpers.html" import render_field %}
    <div>What's your name?</div>
    <form method=post action="/hello">
      <dl>
        {{ render_field(form.sayhello) }}
      </dl>
      <input type=submit value='Say Hello' name='submit_btn'>
    </form>
  </body>
</html>
```

在first_app.html的头部加载CSS文件。现在HTML正文中所有文本元素的大小都应该发生了改变。在HTML文档的正文部分，从_formhelpers.html中导入表单宏，然后渲染app.py文件所定义的sayhello表单。此外，为同一表单元素添加一个按钮，这样用户就可以提交文本字段数据了。

#### 9.3.2.4 创建结果页面

最后创建hello.html文件，该文件将在app.py脚本hello函数的代码行render_template（'hello.html'，name=name）中被渲染，显示用户通过文本字段所提交的文本。文件的内容如下：

```
<!doctype html>
<html>
  <head>
    <title>First app</title>
      <link rel="stylesheet" href="{{ url_for('static',
       filename='style.css') }}">
  </head>
  <body>
    <div>Hello {{ name }}</div>
  </body>
</html>
```

在建立了修改后的Flask网络应用之后，可以在应用程序主目录下执行

以下命令在本地运行该应用，结果可以通过网页浏览器在http://127.0.0.1:5000/查看：

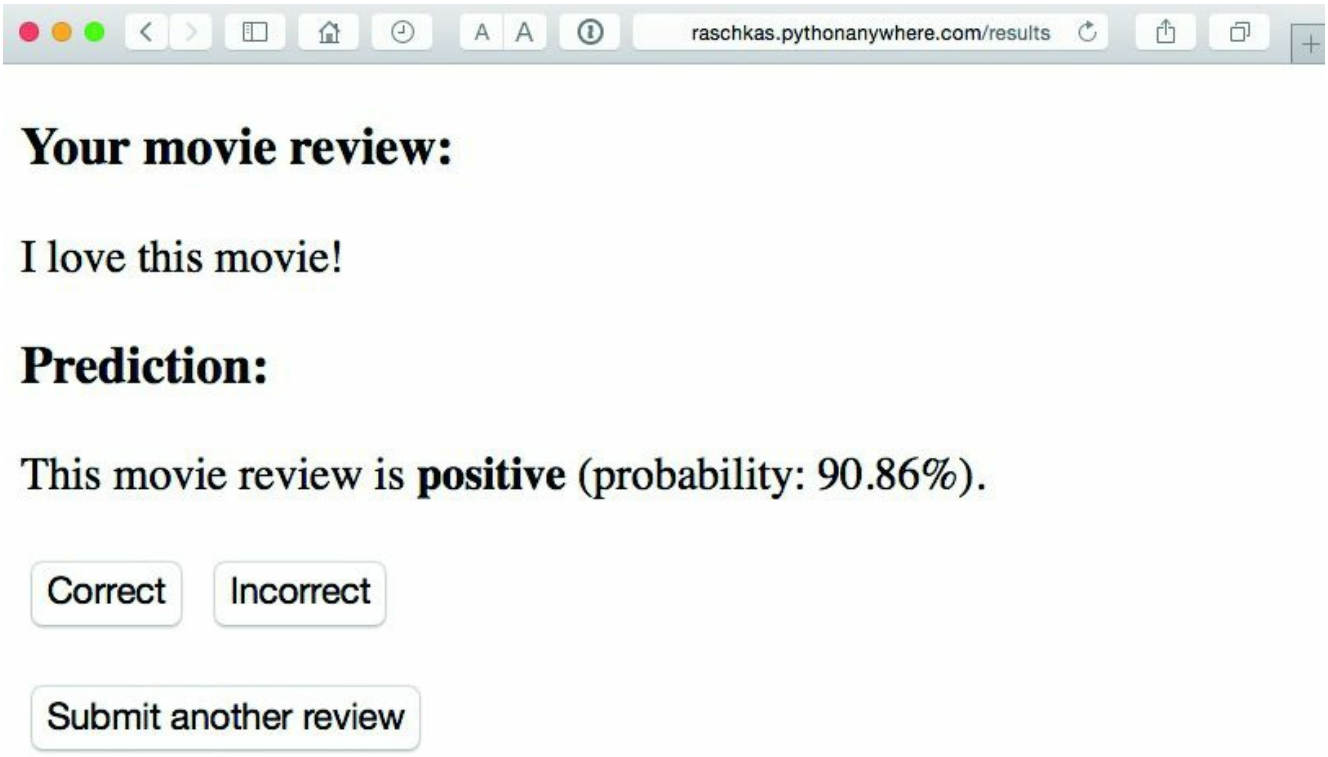
```
python3 app.py
```

如果你是网络开发的新手，有些概念乍看起来可能非常复杂。在这种情况下，我鼓励你动手尝试，只需在硬盘的目录建立前面的文件。然后仔细分析研究就会发现Flask框架相当简单，比最初想象的要简单得多！同时，记得参考优秀的Flask文档和示例以获得更多的帮助：http://flask.pocoo.org/docs/0.12/。

## 9.4 将电影评论分类器转换为网络应用

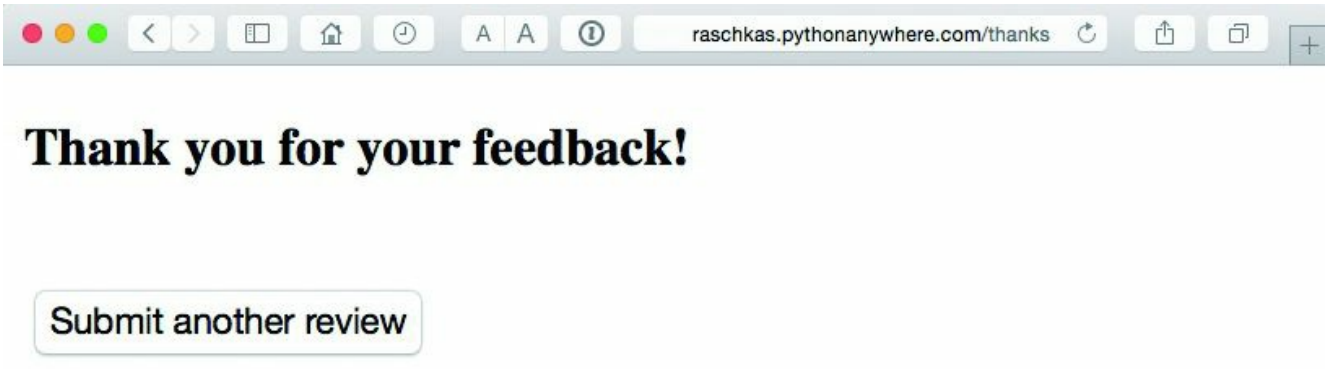
既然已经熟悉了Flask网络开发的基本知识，那么让我们继续前进将电影分类器嵌入网络应用。本节将开发一个网络应用，首先提示用户输入电影评论，如下面的截图所示：

在提交了评论之后，用户将看到一个新页面，显示预测的分类标签及其概率。此外，用户可以通过单击正确或错误按钮来提供对该预测结果的反馈，见下面的截图：




分类模型将根据用户单击的正确或错误按钮所提供的反馈意见进行更新。此外，还将把由用户提供的电影评论文字以及建议（从按钮上推断）的分类标签存储在SQLite数据库中，以留待未来复用。（用户也可以跳过更新的步骤，单击提交另一个评论。）

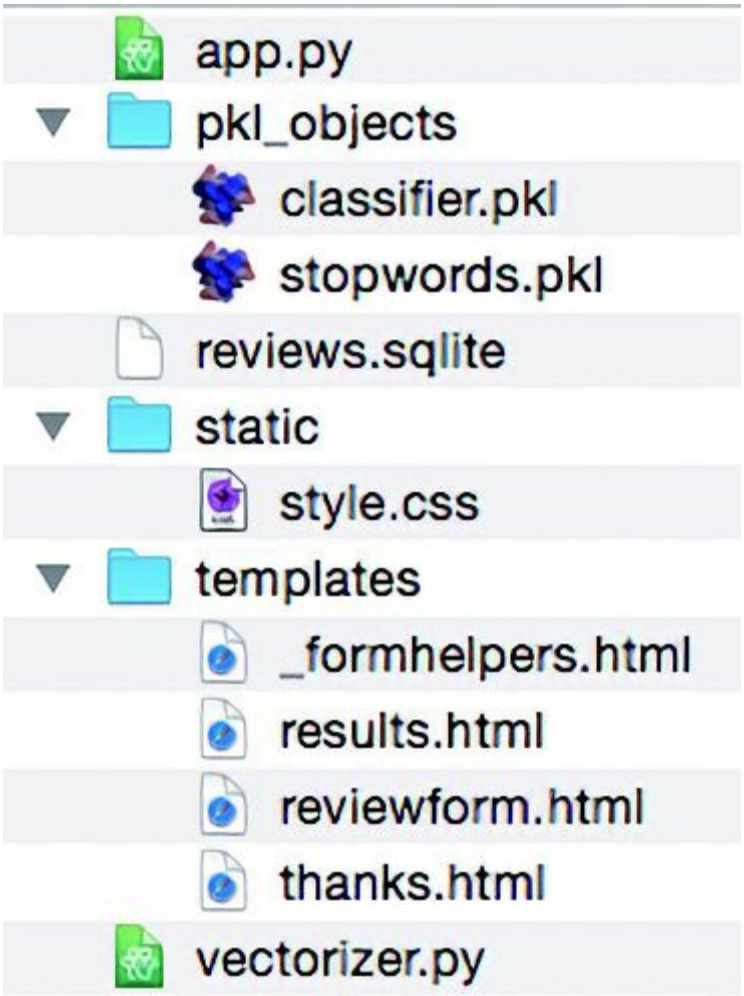
在点击反馈按钮后，用户将看到第三个页面，这是一个简单的“thank you”下面带有提交另一个评论的按钮的界面，按钮可以将用户重定向到起始页。见下面的截图：



在仔细研究这个网络应用的代码实现之前，我鼓励你看一下我上传到下述网站的应用实例演示，感受本节试图要传达的思想。（http://raschkas.pythonanywhere.com。）

### 9.4.1 文件与文件夹——研究目录树

从宏观入手，先看下为这个电影分类应用所创建的目录树，截屏如下：




本章前面的小节创建了vectorizer.py文件、SQLit数据库reviews.sqlite以及处理Python对象的pkl_objects目录。

主目录下的app.py是包含Flask代码的Python脚本文件，用数据库文件review.sqlite（本章前面创建的）来存储提交到网络应用的电影评论。templates子目录包含了HTML模板，将由Flask渲染并在浏览器中显示。static子目录包含简单的CSS文件以调整所渲染的HTML代码的外观。

本书所提供的代码示例中有一个单独的目录，包含了电影评论分类应用以及在本节讨论过的代码。可以直接从GitHub或Packt下载：https://github.com/rasbt/python-machine-learning-book-2nd-edition/。本节的代码可以从.../code/ch09/movieclassifier子目录中找到。

### 9.4.2 实现主应用app.py

由于app.py文件相当长，需要分两步来掌握。app.py的第一部分导入要用到的Python模块和对象，处理数据并建立分类模型的代码：

```
from flask import Flask, render_template, request
from wtforms import Form, TextAreaField, validators
import pickle
import sqlite3
import os
import numpy as np

# import HashingVectorizer from local dir
from vectorizer import vect

app = Flask(__name__)

######## Preparing the Classifier
cur_dir = os.path.dirname(__file__)
clf = pickle.load(open(os.path.join(cur_dir,
                  'pkl_objects',
                  'classifier.pkl'), 'rb'))
db = os.path.join(cur_dir, 'reviews.sqlite')

def classify(document):
    label = {0: 'negative', 1: 'positive'}
    X = vect.transform([document])
    y = clf.predict(X)[0]
    proba = np.max(clf.predict_proba(X))
    return label[y], proba

def train(document, y):
    X = vect.transform([document])
    clf.partial_fit(X, [y])

def sqlite_entry(path, document, y):
    conn = sqlite3.connect(path)
    c = conn.cursor()
    c.execute("INSERT INTO review_db (review, sentiment, date)"\
    " VALUES (?, ?, DATETIME('now'))", (document, y))
    conn.commit()
    conn.close()
```

app.py脚本的第一部分现在应该很熟悉了。直接导入HashingVectorizer并且unpickle逻辑回归分类器。接下来，定义classify函数以返回对给定类文档分类标签的预测及相应概率。如果提供文档和分类标签的话，可以调用train函数来更新分类器。

可以用sqlite_entry函数把提交的影评存储在SQLite数据库备用，数据包括分类标签和时间戳。注意，如果重新启动网络应用，clf对象将被重置为原始的、被pickle的状态。本章最后将学习如何利用收集并存储在SQLite数据库的数据来永久更新分类器。

app.py脚本第二部分的概念应该也不陌生：

```
######## Flask
class ReviewForm(Form):
    moviereview = TextAreaField('',
                                [validators.DataRequired(),
                                validators.length(min=15)])

@app.route('/')
def index():
    form = ReviewForm(request.form)
    return render_template('reviewform.html', form=form)

@app.route('/results', methods=['POST'])
def results():
    form = ReviewForm(request.form)
    if request.method == 'POST' and form.validate():
        review = request.form['moviereview']
        y, proba = classify(review)
        return render_template('results.html',
                                content=review,
                                prediction=y,
                                probability=round(proba*100, 2))
    return render_template('reviewform.html', form=form)

@app.route('/thanks', methods=['POST'])
def feedback():
    feedback = request.form['feedback_button']
    review = request.form['review']
    prediction = request.form['prediction']

    inv_label = {'negative': 0, 'positive': 1}
    y = inv_label[prediction]
    if feedback == 'Incorrect':
        y = int(not(y))
    train(review, y)
    sqlite_entry(db, review, y)
    return render_template('thanks.html')

if __name__ == '__main__':
    app.run(debug=True)
```

我们定义了实例化TextAreaField的ReviewForm类，表单将在reviewform.html模板文件中渲染网络应用的登录页面时被渲染。模板文件反过来由index函数渲染。通过设定参数validators.length（min=15）要求用户至少输入15个字符的评论。结果函数将从提交的网络表单中提取内容，并把它传给分类器来预测情绪，然后在results.html模板中渲染显示。

上一小节app.py中实现的feedback函数乍看起来有点复杂。基本上results.html模板根据用户点击正确或错误的反馈按钮来获取预测的分类标签，并将预测的情绪返回为整数型的分类标签，然后通过train函数来更新分类器，这在app.py脚本的第一部分已经实现了。同时，如果用户提供了反馈

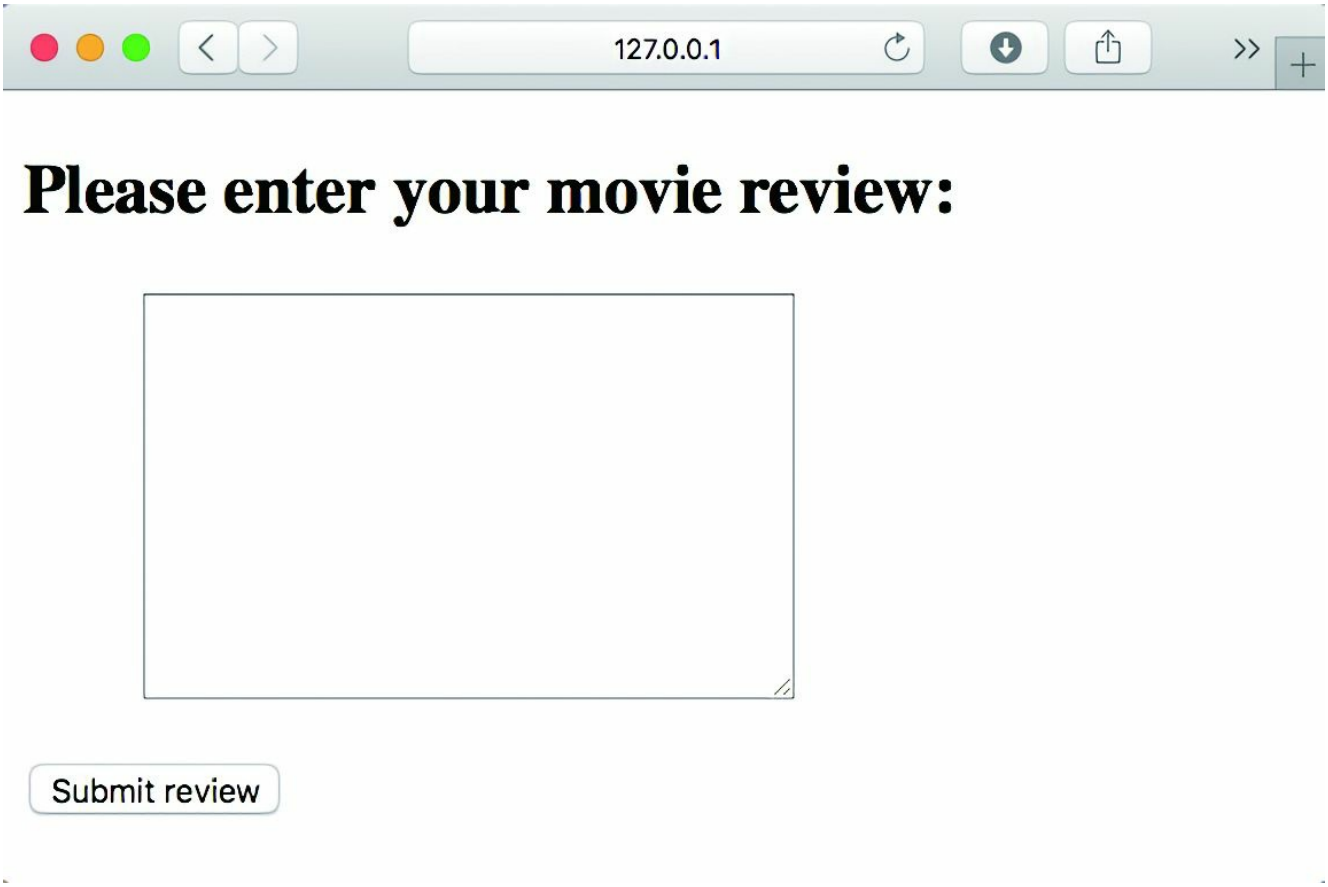
信息，将通过sqlite_entry为SQLite数据库增加新记录，并最终通过thanks.html模板渲染感谢用户的反馈。

### 9.4.3 建立评论表单

下一步讨论reviewform.html模板，其中包含了应用的开始页面。

```
<!doctype html>
<html>
  <head>
    <title>Movie Classification</title>
      <link rel="stylesheet"
       href="{{ url_for('static', filename='style.css') }}">
  </head>
  <body>

    <h2>Please enter your movie review:</h2>

    {% from "_formhelpers.html" import render_field %}

    <form method=post action="/results">
      <dl>
        {{ render_field(form.moviereview, cols='30', rows='10') }}
      </dl>
      <div>
        <input type=submit value='Submit review'
        name='submit_btn'>
      </div>
    </form>

  </body>
</html>
```

这里直接导入与本章前面所定义的_formhelpers.html相同的模板。用这个宏的render_field函数来渲染TextAreaField，用户利用该字段提供电影评论并通过显示在页面底部的按钮提交。这个TextAreaField字段30列宽10行高，截图如下：


127.0.0.1
Please enter your movie review:
Submit review

### 9.4.4 创建一个结果页面的模板

下一个模板results.html看起来有点儿意思：

```
<!doctype html>
<html>
  <head>
    <title>Movie Classification</title>
      <link rel="stylesheet"
      href="{{ url_for('static', filename='style.css') }}">
  </head>
  <body>

    <h3>Your movie review:</h3>
    <div>{{ content }}</div>

    <h3>Prediction:</h3>
    <div>This movie review is <strong>{{ prediction }}</strong>
    (probability: {{ probability }}%).</div>

    <div id='button'>
      <form action="/thanks" method="post">
        <input type=submit value='Correct'
        name='feedback_button'>
        <input type=submit value='Incorrect'
        name='feedback_button'>
        <input type=hidden value='{{ prediction }}'
        name='prediction'>
        <input type=hidden value='{{ content }}' name='review'>
      </form>
    </div>

    <div id='button'>
      <form action="/">
       <input type=submit value='Submit another review'>
      </form>
    </div>

  </body>
</html>
```

首先将提交的评论以及预测结果插入相应的域{{content}}，{{prediction}}和{{probability}}。你可能注意到，在包含正确和错误按钮的表单中，第二次使用了{{content}}和{{prediction}}占位符变量。这是对POST的一种变通，万一用户点击这两个按钮，系统会把这些值推送给服务器，以更新分类器并存储评论。

此外，在results.html的开头导入CSS文件（style.css）。该文件的设置非常简单，它将网络应用内容的宽度限制为600个像素，并将标有div ID button的错误和正确按钮向下移动了20个像素：

```
body{
  width:600px;
}

.button{
   padding-top: 20px;
}
```

这个CSS文件只是个占位符，可随意调整以改变网络应用的观感直到你满意。

为网络应用实现的最后一个HTML文件是thanks.html模板。顾名思义，它只是通过正确或错误按钮提供反馈意见，然后向用户提供感谢信息。此外，把“提交另一个评论”的按钮放在页面底部，该按钮将用户重定向到起始页。thanks.html文件的内容如下：

```
<!doctype html>
<html>
  <head>
    <title>Movie Classification</title>
      <link rel="stylesheet"
      href="{{ url_for('static', filename='style.css') }}">
  </head>
  <body>

    <h3>Thank you for your feedback!</h3>

    <div id='button'>
      <form action="/">
        <input type=submit value='Submit another review'>
      </form>
    </div>

  </body>
</html>
```

在进入下一个小节将其部署到面向公众的网络服务器之前，在终端键入下面的命令并从本地启动网络应用：

```
python3 app.py
```

完成应用测试后，不要忘记删除app.py脚本中app.run（）命令的参数debug=True。

## 9.5 在面向公众的服务器上部署网络应用

在本地完成网络应用测试之后，现在可以将网络应用部署到面向公众的网络服务器上了。本教程将用PythonAnywher的托管服务，该服务专门托管基于Python的网络应用，使应用托管服务变得非常简单和轻松。此外，PythonAnywhere为初学者提供账户以免费运行单个网络应用。

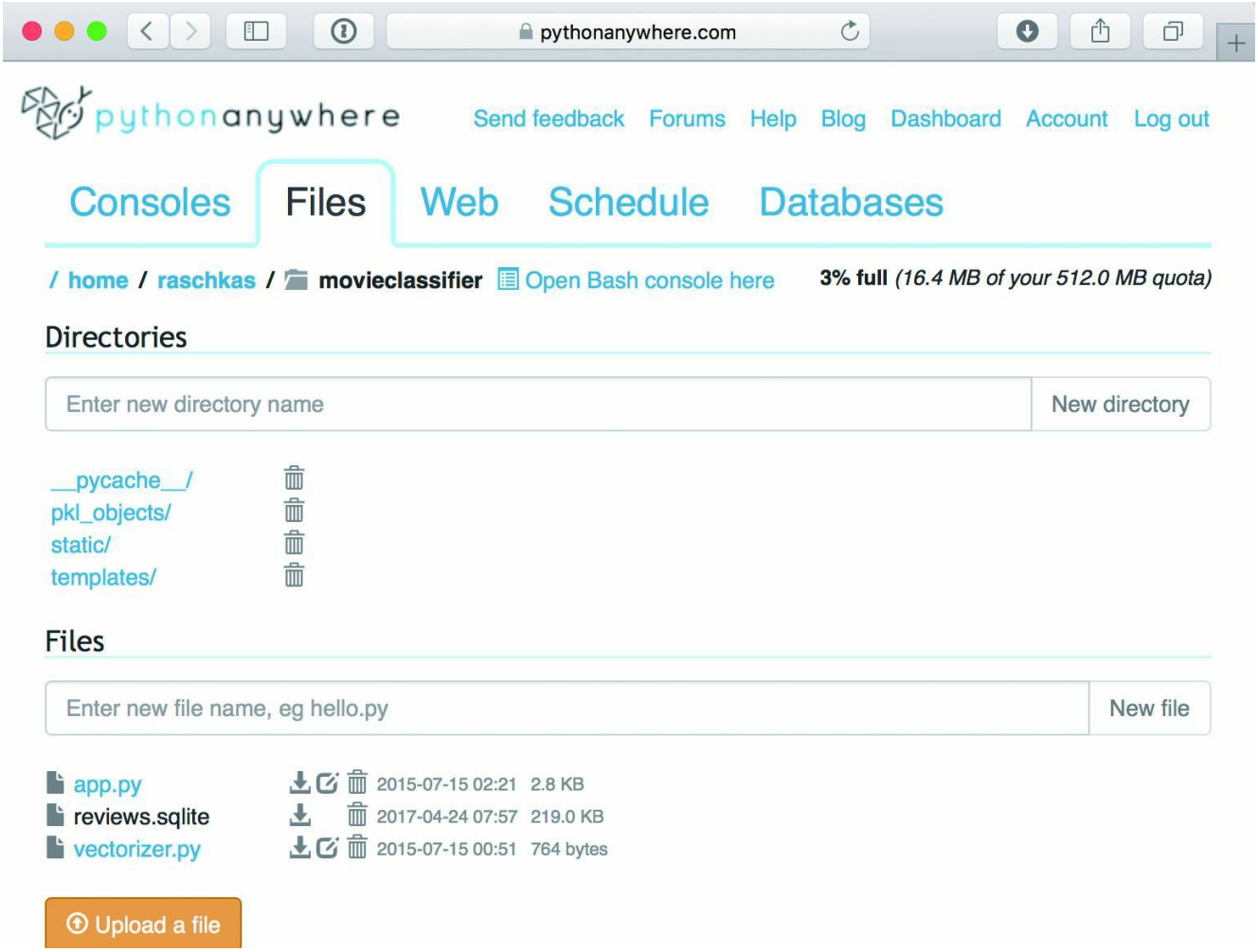
### 9.5.1 创建PythonAnywhere账户

要创建新的PythonAnywhere账户，首先访问 https://www.pythonanywhere.com/，点击位于右上角的“Pricing<amp;signup”链接。接下来，点击“Create a Beginner account”，这里需要提供用户名、密码和有效的电子邮件地址。在阅读并同意条款和条件后，你应该就拥有新账户了。

不幸的是，免费的初级账户不允许从终端以SSH协议访问远程服务器。为此，需要用PythonAnywhere的网络界面管理网络应用。但在把本地应用文件上传到服务器之前，首先要为新账户创建PythonAnywhere网络应用。点击右上角的“Dashboard”按钮后，可以访问页面顶部的控制面板。接着，点击页面顶部可见的“Web”选项卡。继续点击左边的“+Add a new web app”按钮，创建基于Python 3.5的名为movieclassifier的新Flask网络应用。

### 9.5.2 上传电影分类应用

创建PythonAnywhere账户的新应用后，打开文件选项卡，用PythonAnywhere的网络界面从本地目录movieclassifier上传文件。上传在电脑本地上创建的网络应用文件后，现在应该在PythonAnywhere账户上有一个movieclassifier目录。其中包含与本地的movieclassifier相同的目录和文件，见下面的截图：




最后，再次切换至“Web”标签页并点击Reload<username>.pythonanywhere.com按钮，来应用更改并更新网络应用。现在终于应该可以启动和运行网络应用了，公众可以通过<username>.pythonanywhere.com看到应用页面。

## 故障检修

不幸的是网络服务器即使对网络应用中最小的问题可能也相当敏感。如果在PythonAnywhere上运行网络应用后在浏览器上收到错误信息，可以检查服务器及错误日志来诊断问题，日志可以从PythonAnywhere账户的“Web”选项卡访问。

### 9.5.3 更新电影分类器

每当用户提供有关分类的反馈时，预测模型就在运行过程中实时更新，如果网络服务器死机或重启，对clf对象的更新将被重置。如果重新加载网络应用，clf对象将从pickle文件classifier.pkl重新初始化。永久应用更新的一个选择是每次更新都pickle clf对象。然而，随着用户数量的增加，计算效率将变低，如果用户同时提供反馈，则有损坏pickle文件的可能。

另一种方法是根据SQLite数据库所收集的反馈数据更新预测模型。一种方法是从PythonAnywhere服务器下载SQLite数据库，更新本地计算机的clf对象，然后上传新的pickle文件到PythonAnywhere。要更新在本地电脑上的分类器，可以用在movie-classifier目录下创建的update.py脚本文件，细节如下：

```
import pickle
import sqlite3
import numpy as np
import os

# import HashingVectorizer from local dir
from vectorizer import vect

def update_model(db_path, model, batch_size=10000):

    conn = sqlite3.connect(db_path)
    c = conn.cursor()
    c.execute('SELECT * from review_db')

    results = c.fetchmany(batch_size)
    while results:
        data = np.array(results)
        X = data[:, 0]
        y = data[:, 1].astype(int)

        classes = np.array([0, 1])
        X_train = vect.transform(X)
        model.partial_fit(X_train, y, classes=classes)
        results = c.fetchmany(batch_size)

    conn.close()
    return model

cur_dir = os.path.dirname(__file__)

clf = pickle.load(open(os.path.join(cur_dir,
                  'pkl_objects',
                  'classifier.pkl'), 'rb'))
db = os.path.join(cur_dir, 'reviews.sqlite')

clf = update_model(db_path=db, model=clf, batch_size=10000)

# Uncomment the following lines if you are sure that
# you want to update your classifier.pkl file
# permanently.

# pickle.dump(clf, open(os.path.join(cur_dir,

#              'pkl_objects', 'classifier.pkl'), 'wb')
#              , protocol=4)
```

包含具有更新功能的电影评论分类器应用的单独目录来自于本书的示例代码，你可以直接从Packt获取或者从GitHub下载（https://github.com/rasbt/python-machine-learning-book-2nd-edition/）

本节代码可以从.../code/ch09/movieclassifier_with_update子目录中找到。

update_model会从SQLite数据库以批量方式每次抽取10 000条记录，除非数据库的记录数量太少。另外也可以用fetchone每次取一条记录而不是fetchmany，这么做的计算效率很低。但是请记住如果面对规模超过计算机或服务器内存容量的大型数据集，用可选的fetchall方法将会出现问题。

现在已经创建了update.py脚本，可以将其上传到PythonAnywhere的movie-classifier目录，并在主应用程序脚本app.py中导入update_model函数，当重新启动网络应用时，用来自于SQLite数据库的数据更新分类器。为此，只需要在app.py顶部添加一行代码，从update.py脚本导入update_model函数。

```
# import update function from local dir
from update import update_model
```

然后需要从主应用体调用update_model函数：

```
...
if __name__ == '__main__':
    clf = update_model(db_path=db,
                       model=clf,
                       batch_size=10000)
...
```

如前所述，前面代码片段的修改将更新PythonAnywhere上的pickle文件。然而，实际上并不需要经常重启网络应用，而且在更新之前验证SQLite数据库的用户评论确保评论信息对分类器有价值是有意义的。

## 9.6 小结

本章学习了许多实用的主题，扩展了对机器学习理论的认识。我们学会了如何序列化训练模型，以及如何加载使用。此外，也建立了SQLite数据库用于高效存储，并且创建了网络应用让我们有机会把电影分类器提供给外面的世界使用。

本书已经讨论了很多关于机器学习的概念、最佳实践和有监督学习的分类模型。下一章将讨论有监督学习的另一子类——回归分析，确保在一个连续尺度上预测结果变量，这与一直在用的标签分类模型不同。

# 第10章　用回归分析预测连续目标变量

前几章学到了很多关于有监督学习的主要概念，并且训练了许多用于不同分类任务的模型，这些模型可以预测群组成员的身份或者分类变量。这一章将开始讨论另一种有监督学习：回归分析。

回归分析主要应用在预测连续目标变量，有助于解决科学以及工业应用中的许多问题，有助于理解变量之间的关系，评估或预测趋势。例如预测公司未来几个月的销售额。

本章将介绍回归分析模型的主要概念，主要涵盖下述几个方面：

·探索和可视化数据集

·研究实现线性回归模型的不同方法

·训练有效解决离群值问题的回归模型

·评估回归模型并诊断常见问题

·拟合非线性数据的回归模型

## 10.1 线性回归简介

线性回归的目的是针对一个或多个特征与连续目标变量之间的关系建模。第1章曾讨论过，回归分析是机器学习的一个分支。与有监督学习分类相反，回归分析的主要目标是在连续尺度上预测输出，而非分类标签。

下面的小节将介绍线性回归的最基本类型，即简单线性回归，并将它与更一般的多元情形（多特征的线性回归）联系起来。

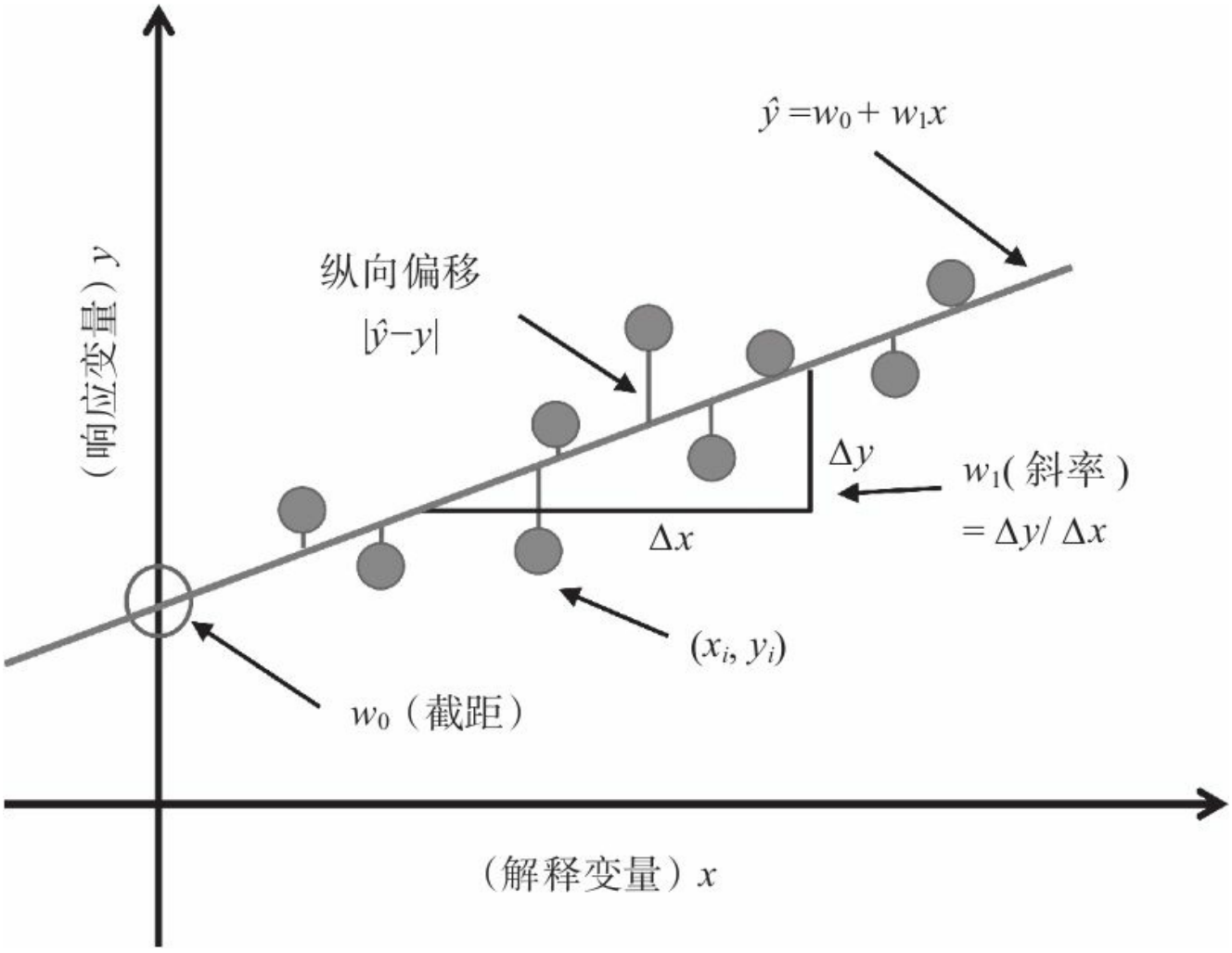
## 10.1.1 简单线性回归

简单（单变量）线性回归的目的是针对单个特征（解释变量x）和连续响应值（目标变量y）之间的关系建模。拥有一个解释变量的线性回归模型的方程定义如下：

$$y = w_0 + w_1 x$$

这里权重$w_0$代表y轴截距，$w_1$为解释变量的加权系数。目标是学习线性方程的权重，以描述解释变量和目标变量之间的关系，然后预测训练集里未见过的新响应变量。

根据前面的定义，线性回归可以理解为通过采样点找到最佳拟合直线，见下图：




这条最佳拟合线也被称为回归线，从回归线到样本点的垂直线就是所谓的误差或残差。

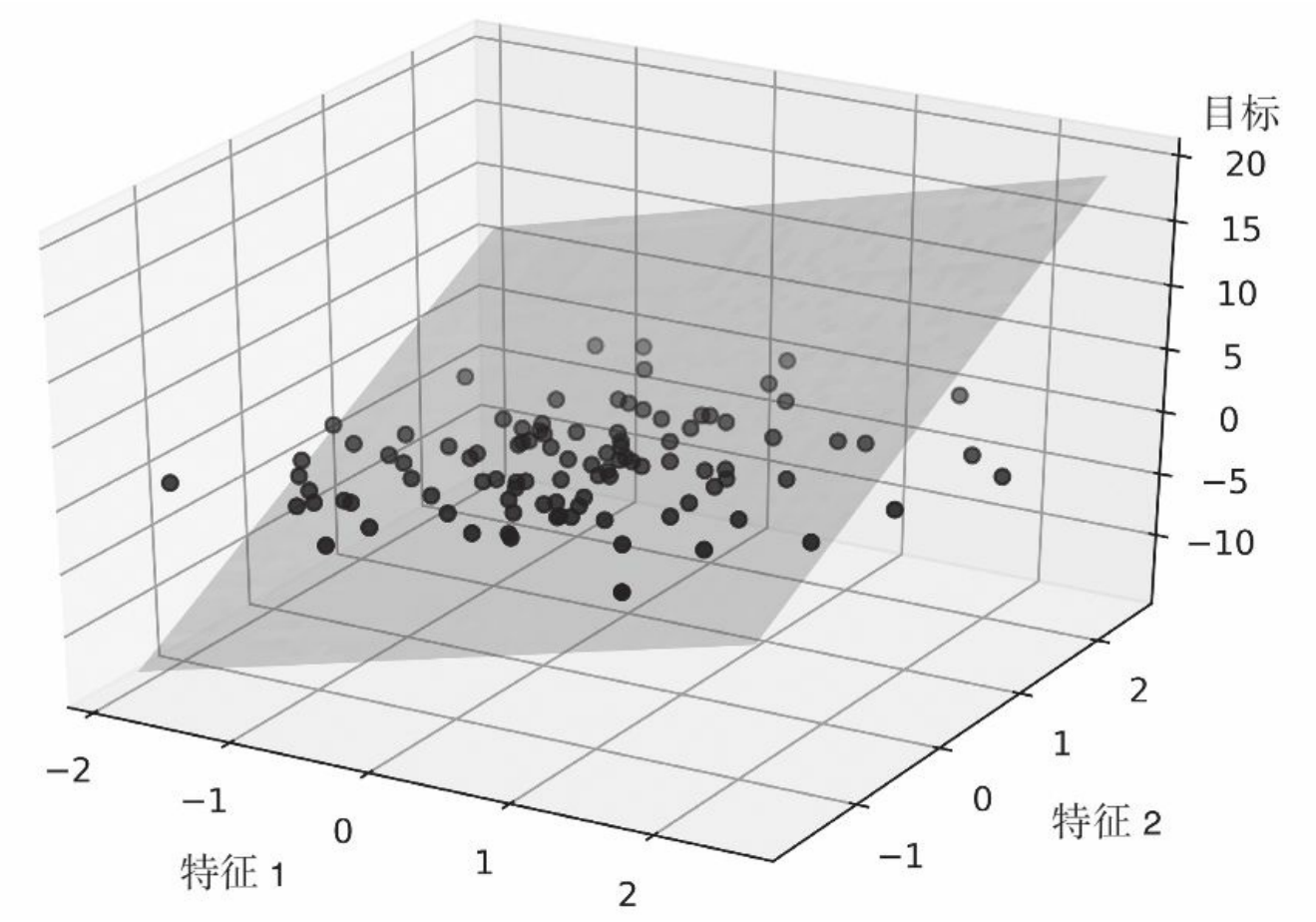
## 10.1.2 多元线性回归

前一小节引入了单解释变量的线性回归分析，也称之为简单的线性回归。当然，也可以将线性回归模型推广到多个解释变量，这个过程叫作多元线性回归：

$$y = w_0 x_0 + w_1 x_1 + \cdots + w_m x_m = \sum_{i=0}^{m} w_i x_i = w^{\mathrm{T}} x$$

这里，$w_0$是当$x_0$=1时的y轴截距。

下图显示了具有两个特征的多元线性回归模型的二维拟合超平面：




正如所看到的，适合于三维散点图的多元线性回归的可视化，在静态图像时已经很难描述了。因为没有更好的方法来可视化二维平面的散点图（拟合三个或多个特征的数据集的多元线性回归模型），本章的案例和可视化将主要集中在采用简单线性回归的单变量情况。然而，简单和多元线性回归基于相同的概念和评估技术，本章所讨论的代码实现也与这两种回归模型兼容。

## 10.2 探索住房数据集

在实现第一个线性回归模型之前，先介绍一个新的数据集，即住房数据集，其中包含了由D·哈里斯和慕·鲁宾菲尔德两位在1978年收集的关于波士顿郊区住房的信息。住房数据集现已免费提供，并包含在本书的代码包。该数据集最近已被UCI从机器学习库中删除，但仍然可以在网站 https://raw.githubusercontent.com/rasbt/python-machine-learning-book-2nd-edition/master/code/ch10/housing.data.txt中查阅。像每个新数据集一样，通过简单的可视化来探索总会很有所帮助，可以使我们更好地理解所面对的工作和数据。

### 10.2.1 加载住房数据

本节将调用pandas的read_csv函数加载住房数据，该工具快速灵活，是我们推荐的处理纯文本格式的表格数据处理工具。

这里总结了住房数据集中所包括的506个样本的特征，取自于先前共享的原始数据源（https://archive.ics.uci.edu/ml/datasets/Housing）：

·CRIM：城镇人均犯罪率

·ZN：占地面积超过25 000平方英尺的住宅用地比例

·INDUS：城镇非零售营业面积占比

·CHAS：查尔斯河哑变量（如果临河有大片土地为1，否则为0）

·NOX：一氧化氮浓度（千万分之一）

·RM：平均每户的房间数

·AGE：1940以前建造的自用房单位比例

·DIS：波士顿五个就业中心的加权距离

·RAD：辐射可达的公路的索引

·TAX：$10 000—每10 000美元全额财产的税率

·PTRATIO：城镇师生比例

·B：1000人中非裔美国人的比例

·LSTAT：地位较低人口的百分比

·MEDV：自住房的中位价（千美元）

本章的其余部分将把房价（MEDV）作为目标变量，即基于13个解释变量中的1个或多个来预测房价。在进一步探讨该数据之前，先把它从UCI复制到pandas的数据帧：

```
>>> import pandas as pd
>>> df = pd.read_csv('https://raw.githubusercontent.com/rasbt/'
...                  'python-machine-learning-book-2nd-edition'
...                  '/master/code/ch10/housing.data.txt',
...                  header=None,
...                  sep='\s+')
>>> df.columns = ['CRIM', 'ZN', 'INDUS', 'CHAS',
...               'NOX', 'RM', 'AGE', 'DIS', 'RAD',
...               'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV']
>>> df.head()
```

显示数据集的前五行以确认数据集已成功加载，如下表所示：

| | CRIM | ZN | INDUS | CHAS | NOX | RM | AGE | DIS | RAD | TAX | PTRATIO | B | LSTAT | MEDV |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0.00632 | 18.0 | 2.31 | 0 | 0.538 | 6.575 | 65.2 | 4.0900 | 1 | 296.0 | 15.3 | 396.90 | 4.98 | 24.0 |
| 1 | 0.02731 | 0.0 | 7.07 | 0 | 0.469 | 6.421 | 78.9 | 4.9671 | 2 | 242.0 | 17.8 | 396.90 | 9.14 | 21.6 |
| 2 | 0.02729 | 0.0 | 7.07 | 0 | 0.469 | 7.185 | 61.1 | 4.9671 | 2 | 242.0 | 17.8 | 392.83 | 4.03 | 34.7 |
| 3 | 0.03237 | 0.0 | 2.18 | 0 | 0.458 | 6.998 | 45.8 | 6.0622 | 3 | 222.0 | 18.7 | 394.63 | 2.94 | 33.4 |
| 4 | 0.06905 | 0.0 | 2.18 | 0 | 0.458 | 7.147 | 54.2 | 6.0622 | 3 | 222.0 | 18.7 | 396.90 | 5.33 | 36.2 |

你可以在本书的代码包中找到该住房数据集（以及本书中用到的所有其他数据集）的副本，在脱机工作或网络链接中断的情况下，可以从下面的链接获得：https://raw.githubusercontent.com/rasbt/python-machine-learning-book-2nd-edition/master/code/ch10/housing.data.txt。要从本地目录加载住房数据集，可以如示例替换掉这些行：

```
df = pd.read_csv(
                 'https://raw.githubusercontent.com/rasbt/'
                 'python-machine-learning-book-2nd-edition'
                 '/master/code/ch10/housing.data.txt',
                 sep='\s+')
```

以下面这行取而代之：

```
df = pd.read_csv('./housing.data.txt'), sep='\s+')
```

## 10.2.2 可视化数据集的重要特点

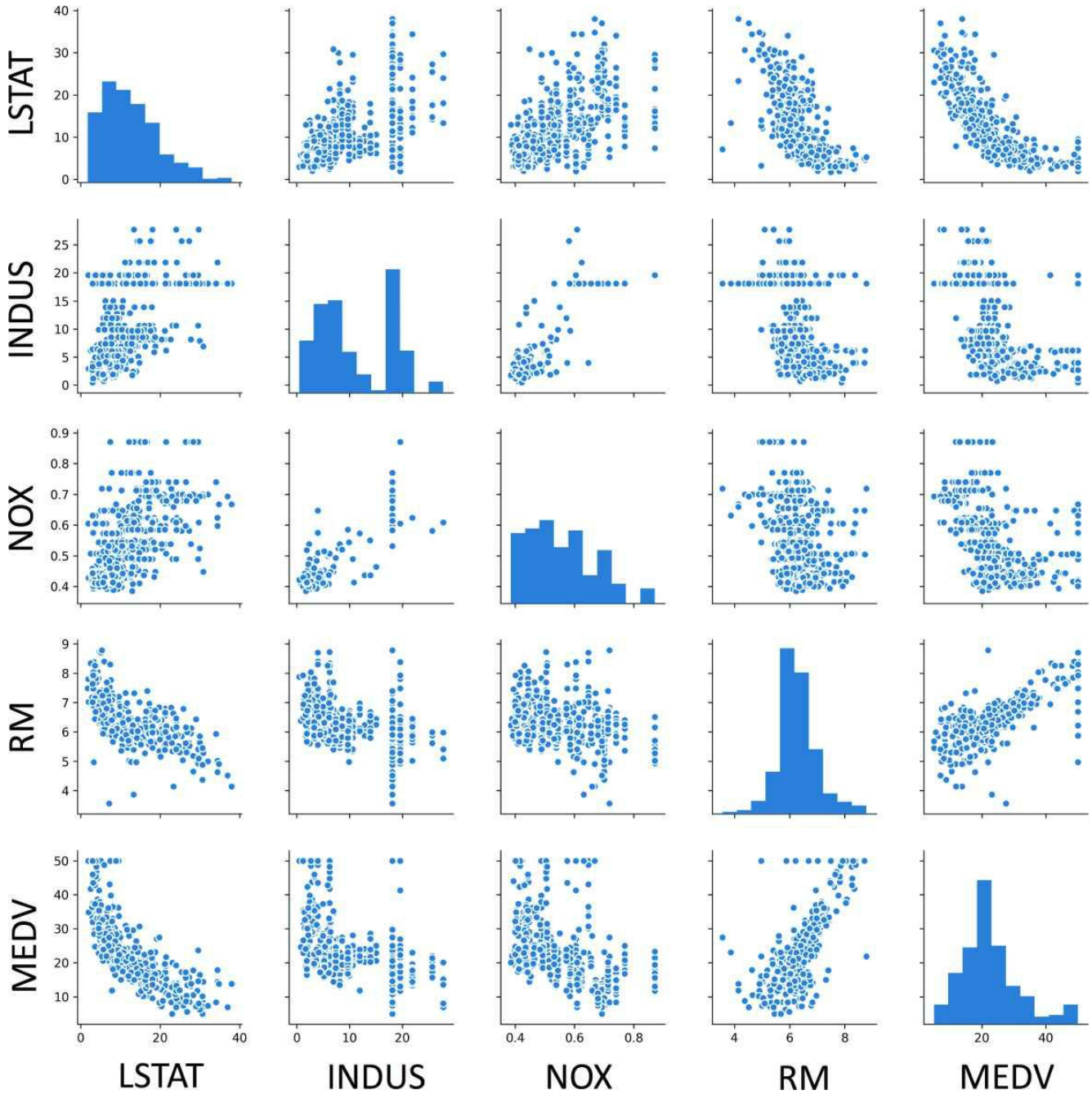
探索性数据分析（EDA）是在进行机器学习模型训练之前值得推荐的重要一步。本节的其余部分将使用图形化EDA工具箱中的一些简单而有用的技术，这些工具有助于直观地发现异常值的存在、数据的分布以及特征之间的关系。

先创建一个散点图矩阵，把数据集中不同特征之间的成对相关性在一张图上直观地表示出来。调用Seaborn库（http://stanford.edu/~mwaskom/software/seaborn/）的pairplot函数绘制散点图矩阵，这是一个基于Matplotlib统计图的Python库。

可以通过conda install seaborn或者pip install seaborn安装seaborn软件包。安装完成后，导入软件包并创建散点图矩阵，具体代码如下：

```
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
>>> cols = ['LSTAT', 'INDUS', 'NOX', 'RM', 'MEDV']
>>> sns.pairplot(df[cols], size=2.5)
>>> plt.tight_layout()
>>> plt.show()
```

从下面的图可以看到，散点图矩阵展示了数据集内部特征之间的关系，是一种有价值的图形汇总：




由于空间的限制同时考虑到读者的兴趣，本章只绘制了数据集中的五列：LSTAT、INDUS、NOX、RM和MEDV。然而，你可以创建一个基于DataFrame的全散点图矩阵，调用以前的sns.pairplot函数来选择不同列的名称，或通过省略列选择器（sns.pairplot（df））选择包括在散点图矩阵中的所有变量，以进一步探索数据集。

可以利用散点图矩阵快速分辨出数据的分布情况以及是否含有异常值。例如，可以看到房间数RM和房价MEDV（第四行第五列）之间存在着线性关系。此外，可以从散点图矩阵右下部分的直方图看到，MEDV变量似乎呈正态分布，但包含了几个离群样本。

注意，与一般的想法相反，训练线性回归模型并不要求解释变量

或目标变量呈正态分布。正态假设只是针对某些统计和假设检验的要求，关于这部分的讨论超出了本书范围。（道格拉斯·C.蒙特高美丽，伊丽莎白·A.派克，G.焦佛瑞·威宁.《线性回归分析导论》.威利出版社，2012：318-319。）

## 10.2.3 用关联矩阵查看关系

上一节把住房数据集变量的数据分布以直方图和散点图的形式可视化。下一步将创建关联矩阵来量化和概括变量之间的线性关系。关联矩阵与协方差矩阵密切相关，这部分在第5章讨论主成分分析（PCA）时了解过。直观地说可以把关联矩阵理解为协方差矩阵的修正。事实上，关联矩阵与协方差矩阵在标准化特征计算方面一致。

关联矩阵是包含皮尔森积矩相关系数（通常简称为皮尔森的r）的正方形矩阵，用来度量特征对之间的线性依赖关系。相关系数在-1到1之间。如果r=1，则两个特征之间呈完美的正相关；如果r=0，则两者之间没有关系；如果r=-1，则两者之间呈完全相反的关系。如前所述，皮尔森相关系数可以简单地计算为特征x和y之间的协方差（分子）除以标准偏差的乘积（分母）：

$$r = \frac{\sum_{i=1}^{n}\left[\left(x^{(i)}-\mu_x\right)\left(y^{(i)}-\mu_y\right)\right]}{\sqrt{\sum_{i=1}^{n}\left(x^{(i)}-\mu_x\right)^2}\sqrt{\sum_{i=1}^{n}\left(y^{(i)}-\mu_y\right)^2}} = \frac{\sigma_{xy}}{\sigma_x\sigma_y}$$

这里，μ为样本的均值，$\sigma_{xy}$为样本x和y之间的协方差，$\sigma_x$和$\sigma_y$为样本的标准偏差。

可以证明，一对标准化的特征之间的协方差实际上等于它们的线性相关系数。为了证明这一点，先标准化特征x和y以获得它们的z评分，分别用x′和y′来表示：

$$x' = \frac{x-u_x}{\sigma_x}, y' = \frac{y-u_y}{\sigma_y}$$

记住我们计算两个特征之间的（总数）协方差如下：

$$\sigma_{xy} = \frac{1}{n}\sum_{i}^{n}\left(x^{(i)}-\mu_x\right)\left(y^{(i)}-\mu_y\right)$$

因为标准化让一个特征变量以平均值中心，所以现在可以计算比例调整后特征之间的协方差如下：

$$\sigma'_{xy} = \frac{1}{n}\sum_{i}^{n}(x'-0)(y'-0)$$

通过回代得到下面的结果：

$$\frac{1}{n}\sum_{i}^{n}\left(\frac{x-\mu_x}{\sigma_x}\right)\left(\frac{y-\mu_y}{\sigma_y}\right)$$

$$\frac{1}{n\cdot\sigma_x\sigma_y}\sum_{1}^{n}\left(x^{(i)}-\mu_x\right)\left(y^{(i)}-\mu_y\right)$$

最后简化该等式如下：

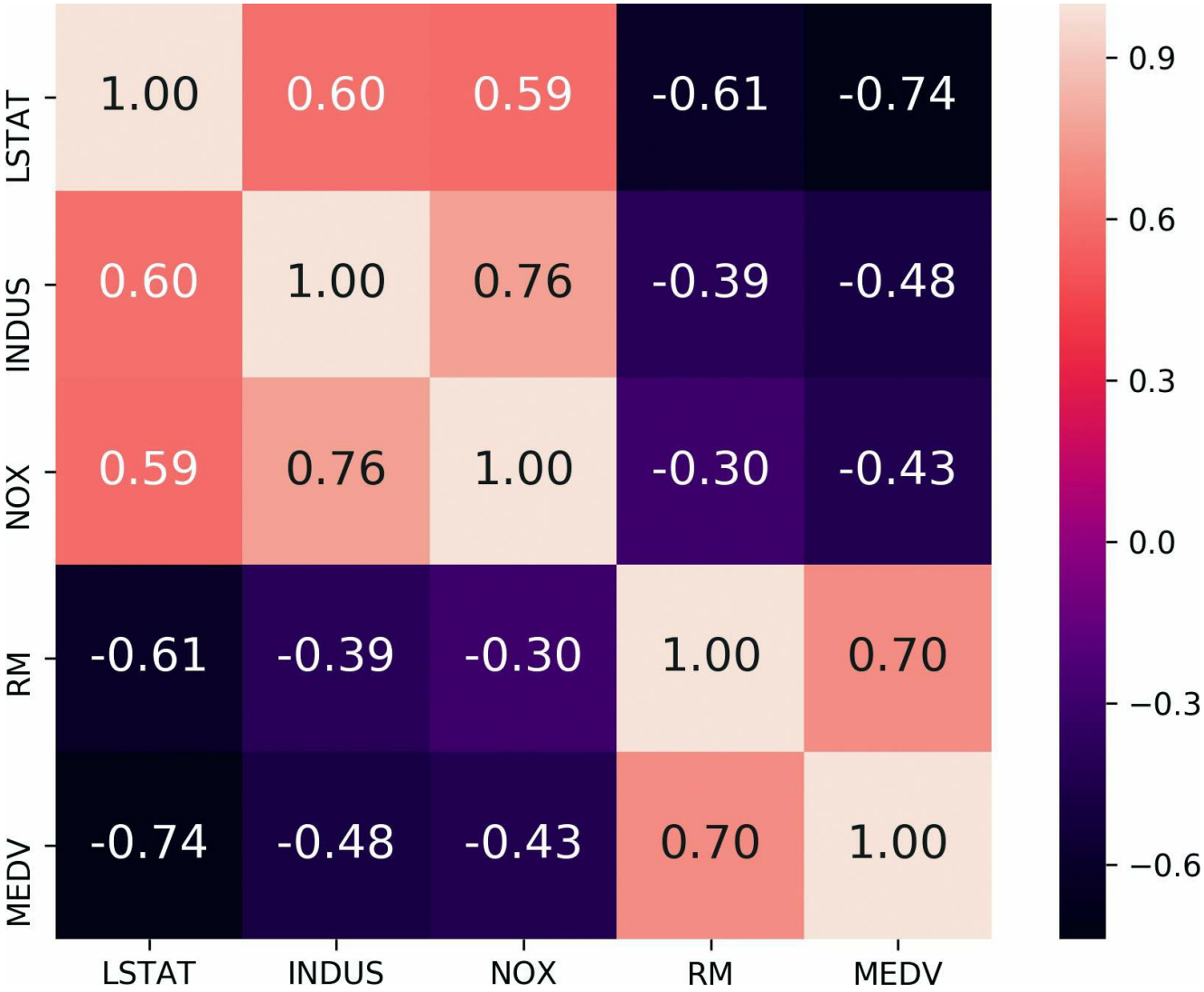
$$\sigma'_{xy} = \frac{\sigma_{xy}}{\sigma_x\sigma_y}$$

下面的代码示例将调用NumPy的corrcoef函数处理前面可视化散点图矩阵的5个特征列，调用Seaborn的heatmap函数基于关联矩阵阵列绘制热力图：

```
>>> import numpy as np
>>> cm = np.corrcoef(df[cols].values.T)
>>> sns.set(font_scale=1.5)
>>> hm = sns.heatmap(cm,
...                  cbar=True,
...                  annot=True,
...                  square=True,
...                  fmt='.2f',
...                  annot_kws={'size': 15},
...                  yticklabels=cols,
...                  xticklabels=cols)
>>> plt.show()
```

正如结果图中所见到的，关联矩阵提供了另一个有用的概要图，有助于

根据各自的线性相关性选择特征：




要拟合线性回归模型，我们只对那些与目标变量MEDV高度相关的特征感兴趣。分析前面的关联矩阵发现目标变量MEDV与LSTAT变量（-0.74）的相关性最大。然而，你可能还记得在检查散点图矩阵时发现LSTAT和MEDV之间存在着明显的非线性关系。另一方面，RM和MEDV之间的相关性相对来说也比较高（0.70）。因为从散点图观察到两个变量之间存在着线性关系，RM似乎是下面要介绍的简单线性回归模型中探索变量的恰当选择。

## 10.3 普通最小二乘线性回归模型的实现

本章的开头部分提到过可以把线性回归理解为，通过训练数据的样本点获得最佳拟合直线。然而，我们既没有定义最佳拟合，也没有讨论过拟合模型的不同技术。下面的小节将用普通最小二乘法（OLS）（有时也称为线性最小二乘法）来填补缺失的部分，估计线性回归线的参数，从而使样本点的垂直距离（残差或误差）之和最小化。

### 10.3.1 用梯度下降方法求解回归参数

考虑第2章实现的自适应线性神经元（ADALINE），我们记得人工神经元采用线性激活函数。同时，定义了成本函数J（·），通过优化算法最小化成本函数学习权重，这些算法包括梯度下降（GD）和随机梯度下降（SGD）。Adaline的成本函数是平方和误差（SSE），与OLS中用到的成本函数相同。

$$J(w)=\frac{1}{2}\sum_{i=1}^{n}\left(y^{(i)}-\hat{y}^{(i)}\right)^2$$

这里，$\hat{y}$为预测值$\hat{y}=w^{\mathrm{T}}x$（注意$\frac{1}{2}$只是为了方便推导GD的更新规则）。OLS回归基本上可以理解为没有单位阶跃函数的Adaline，这样就可以得到连续的目标值，而不是分类标签-1和1。为了证明这一点，以第2章中的Adaline GD实施为基础，去除单位阶跃函数来实现第一个线性回归模型：

```
class LinearRegressionGD(object):

    def __init__(self, eta=0.001, n_iter=20):
        self.eta = eta
        self.n_iter = n_iter

    def fit(self, X, y):
        self.w_ = np.zeros(1 + X.shape[1])
        self.cost_ = []

        for i in range(self.n_iter):
            output = self.net_input(X)
            errors = (y - output)
            self.w_[1:] += self.eta * X.T.dot(errors)
            self.w_[0] += self.eta * errors.sum()
            cost = (errors**2).sum() / 2.0
            self.cost_.append(cost)
        return self

    def net_input(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]

    def predict(self, X):
        return self.net_input(X)
```

如果需要了解如何更新权重，可以沿梯级的方向后退一步，请回顾第2章《训练简单的机器学习分类算法》的《自适应神经元和学习收敛》部分。

为了要观察LinearRegressionGD回归器的具体行为，我们用住房数据的RM（房间数）变量作为解释变量，训练可以预测MEDV（房价）的模型。此外标准化变量以确保GD算法具有更好的收敛性。代码如下：

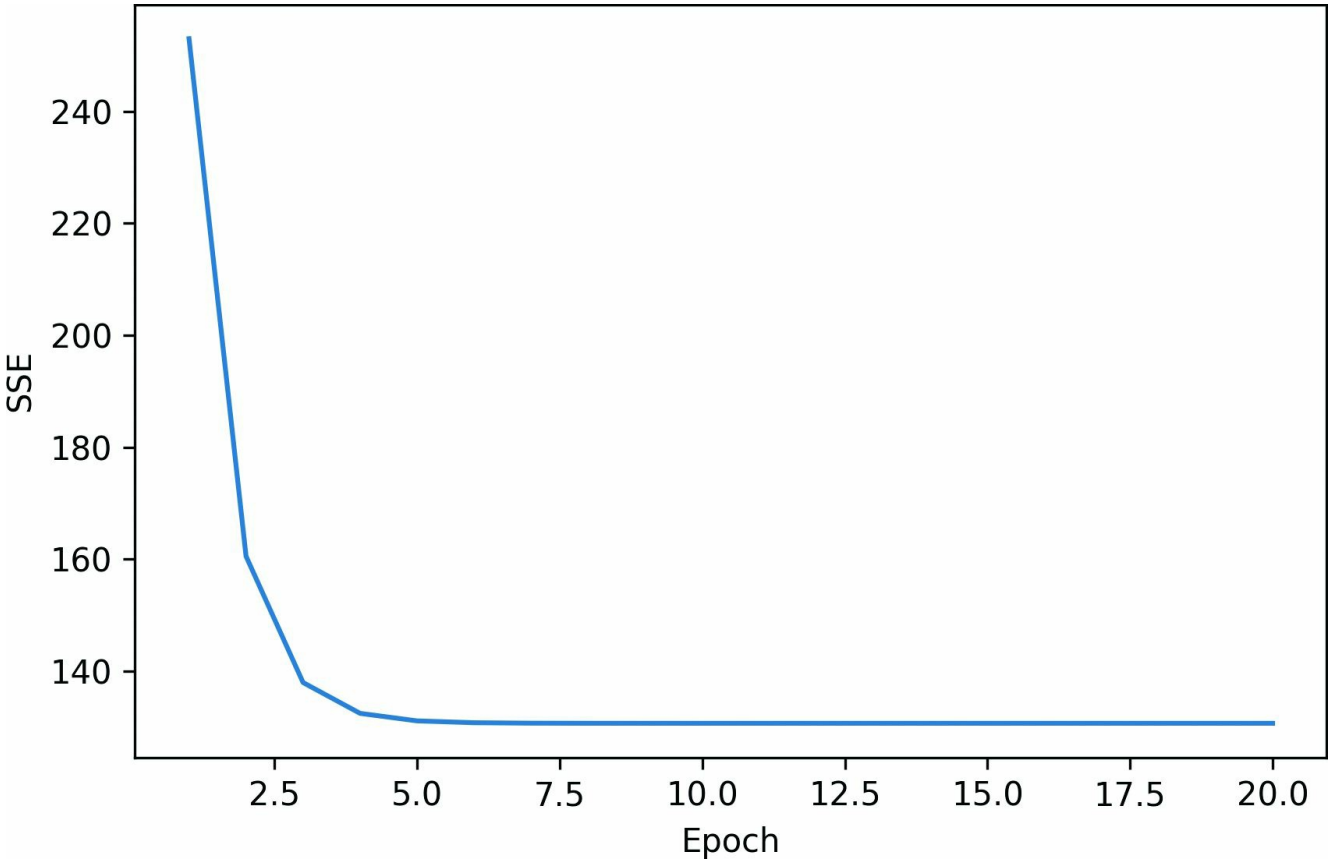
```
>>> X = df[['RM']].values
>>> y = df['MEDV'].values
>>> from sklearn.preprocessing import StandardScaler
>>> sc_x = StandardScaler()
>>> sc_y = StandardScaler()
>>> X_std = sc_x.fit_transform(X)
>>> y_std = sc_y.fit_transform(y[:, np.newaxis]).flatten()
>>> lr = LinearRegressionGD()
>>> lr.fit(X_std, y_std)
```

你可能已经注意到使用np.newaxisx和flatten变通y_std的方法。scikit-learn的大多数转换器期望数据存储在二维阵列。前面的代码示例用y[：，np.newaxis]为阵列添加了一个新维度。然后，StandardScaler返回比例尺度调整后的变量，为了方便用flatten（）方法将其转换回原来的一维阵列。

第2章曾经讨论过当使用像梯度下降的优化算法时，以训练集迭代次数作为成本函数绘制成本图来检查算法是否收敛到了最低成本（这里指全局性最小成本值）确实是个不错的主意：

```
>>> sns.reset_orig() # resets matplotlib style
>>> plt.plot(range(1, lr.n_iter+1), lr.cost_)
>>> plt.ylabel('SSE')
>>> plt.xlabel('Epoch')
>>> plt.show()
```

正如下图所见，GD算法在第五次迭代后开始收敛：



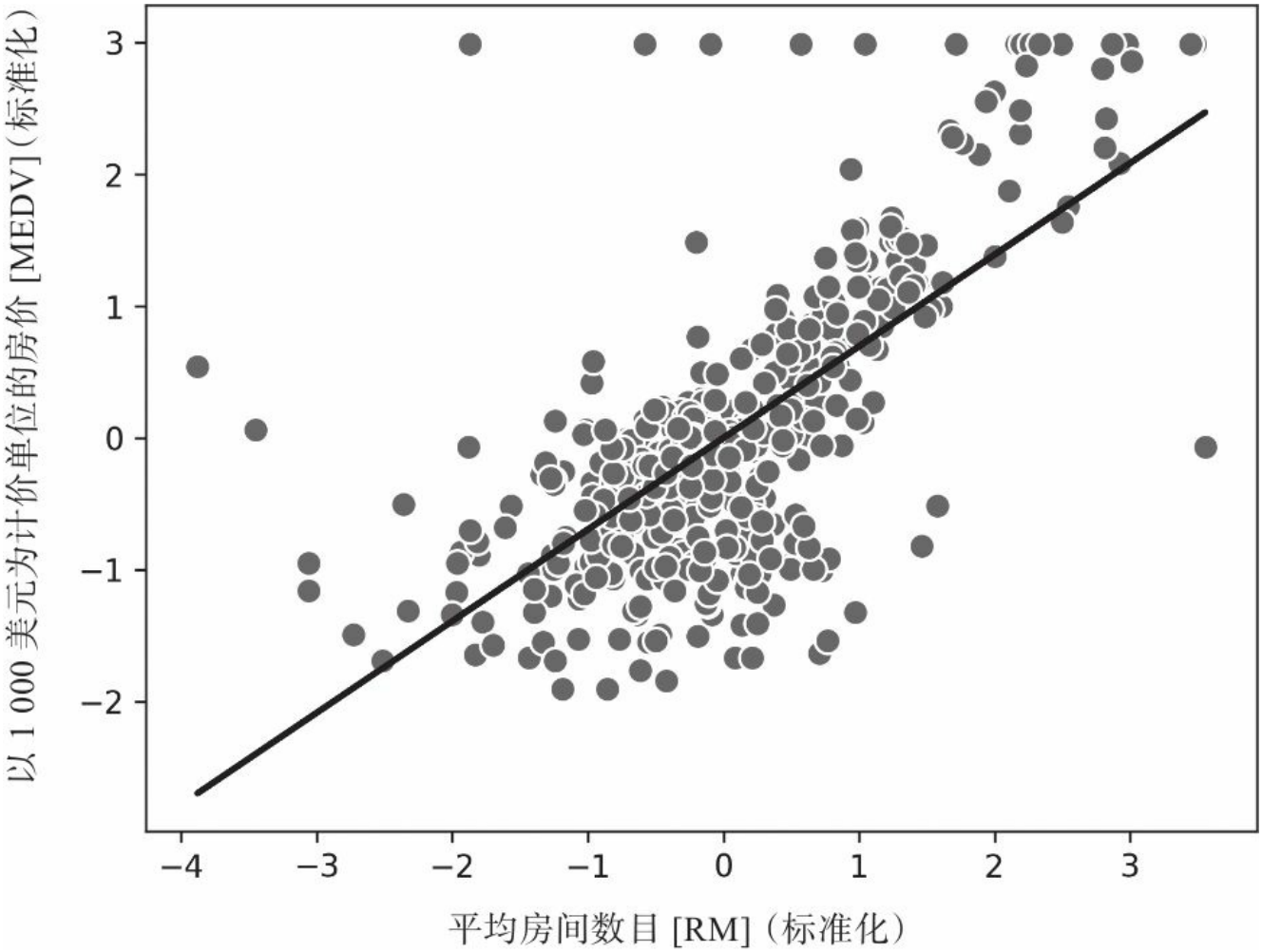

接着通过可视化手段，观察线性回归与训练数据的吻合程度。为此，定义简单的辅助函数lin_regplot绘制训练样本的散点图并添加回归线：

```
>>> def lin_regplot(X, y, model):
...     plt.scatter(X, y, c='steelblue', edgecolor='white', s=70)
...     plt.plot(X, model.predict(X), color='black', lw=2)
...     return None
```

现在用lin_regplot函数绘制房间数与房价的关系图：

```
>>> lin_regplot(X_std, y_std, lr)
>>> plt.xlabel('Average number of rooms [RM] (standardized)')
>>> plt.ylabel('Price in $1000s [MEDV] (standardized)')
>>> plt.show()
```

如下图所示，线性回归反映了房价随房间数增加的基本趋势：




虽然该观察结果很自然，但数据也告诉我们，在许多情况下，房间数并不能很好地解释房价。本章后面将讨论如何量化评估回归模型的性能。有趣的是观察到在y=3的时候有几个数据点形成了一个横排，这表明价格对房间数可能已经不再敏感。在某些应用中，将预测结果变量以原来的比例尺度进行报告也很重要。可以直接调用StandardScaler的inverse_transform方法，把价格的预测结果恢复到以1000美元为单位的坐标轴：

```
>>> num_rooms_std = sc_x.transform([5.0])
>>> price_std = lr.predict(num_rooms_std)
>>> print("Price in $1000s: %.3f" % \
...       sc_y.inverse_transform(price_std))
Price in $1000s: 10.840
```

这个代码示例用以前训练过的线性回归模型来预测有五个房间房屋的价格。根据模型计算这样的房屋价值10 840美元。

另一方面，值得一提的是如果处理标准化变量，从技术角度来说，不需要更新截距的权重，因为在这些情况下，y轴的截距总是0。可以通过打印权重来快速确认这一点：

```
>>> print('Slope: %.3f' % lr.w_[1])
Slope: 0.695
>>> print('Intercept: %.3f' % lr.w_[0])
Intercept: -0.000
```

## 10.3.2 通过scikit-learn估计回归模型的系数

上一节实现了一个可用的回归分析模型。然而，在实际应用中，可能对所实现模型的高效性更感兴趣。例如，许多scikit-learn回归估计器使用LIBLINEAR库，它有先进的优化算法，以及对非标准变量更有效的代码优化手段，这对某些应用来说很有用：

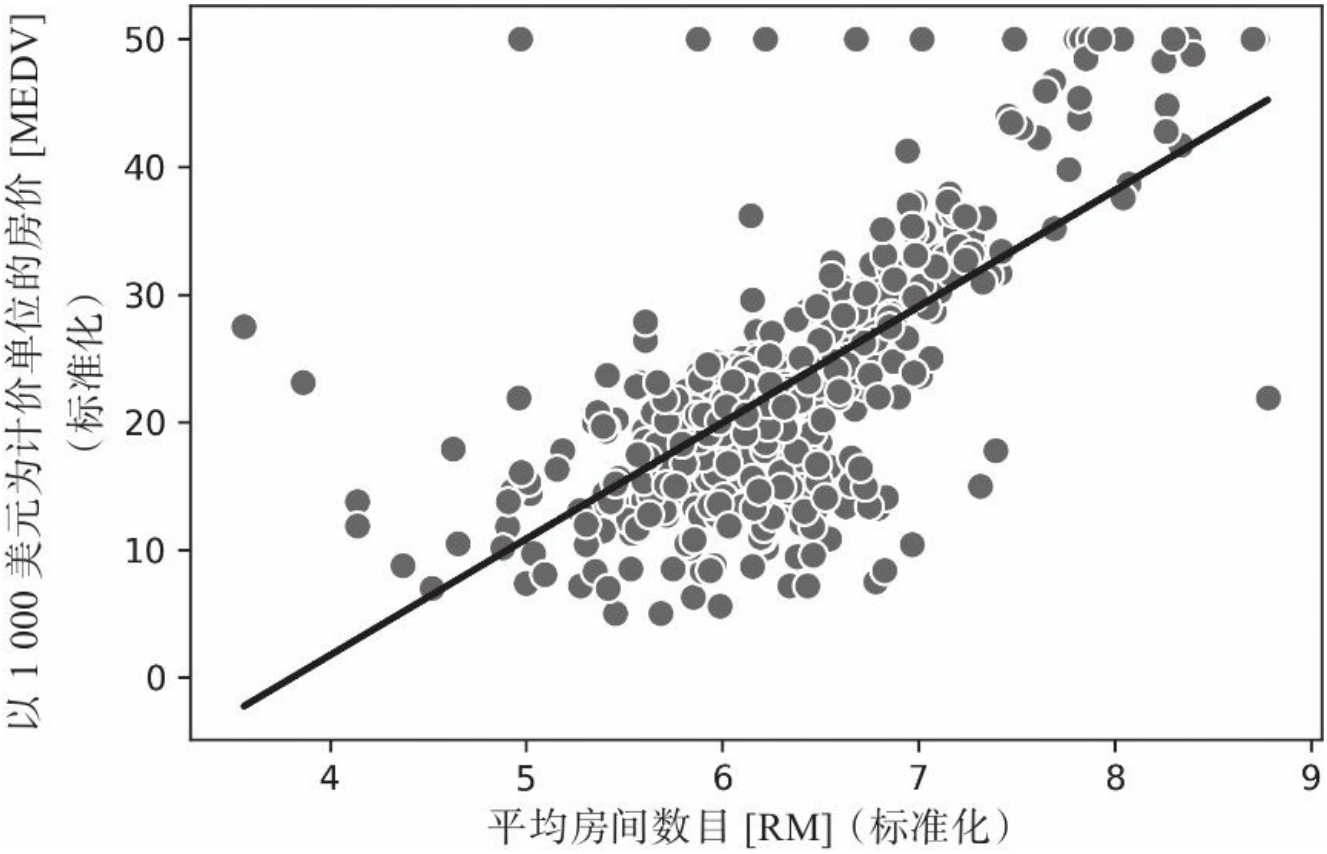
```
>>> from sklearn.linear_model import LinearRegression
>>> slr = LinearRegression()
>>> slr.fit(X, y)
```

```
>>> print('Slope: %.3f' % slr.coef_[0])
Slope: 9.102
>>> print('Intercept: %.3f' % slr.intercept_)
Intercept: -34.671
```

执行此代码片段可以看到scikit-learn的LinearRegression模型拟合非标准的RM和MEDV变量，产生不同的模型系数。用MEDV与RM的关系图来与GD实现做个比较：

```
>>> lin_regplot(X, y, slr)
>>> plt.xlabel('Average number of rooms [RM] (standardized)')
>>> plt.ylabel('Price in $1000s [MEDV] (standardized)')
>>> plt.show()
```

执行这些代码绘制训练数据和拟合模型，从下图可以看到预测结果在总体上与前面所实现的GD一致：




除机器学习库外，另一个选择是用封闭形态的OLS解决方案，这涉及到线性方程组，可以从大多数入门级统计教科书中找到相应的介绍：

$$\boldsymbol{w} = (X^{\mathrm{T}} X)^{-1} X^{\mathrm{T}} y$$

用Python实现如下：

```
# adding a column vector of "ones"
>>> Xb = np.hstack((np.ones((X.shape[0], 1)), X))
>>> w = np.zeros(X.shape[1])
>>> z = np.linalg.inv(np.dot(Xb.T, Xb))
>>> w = np.dot(z, np.dot(Xb.T, y))
>>> print('Slope: %.3f' % w[1])
Slope: 9.102
>>> print('Intercept: %.3f' % w[0])
Intercept: -34.671
```

这种方法的优点是保证能通过分析找到最优解。然而，如果面对非常大的数据集，那么计算可能过于昂贵，以至于无法完成（有时也称为正规方程）逆矩阵，或者样本矩阵可能是奇异的（非可逆的），这就是为什么在某些情况下我们可能更喜欢迭代法。

如果想了解更多关于正规方程的信息，建议看一下莱斯特大学的斯蒂芬

·佩洛克博士的《经典线性回归模型》课程，这些课程可以从下述网站免费获得：

http://www.le.ac.uk/users/dsgp1/COURSES/MESOMET/ECMETXT/06mesm

## 10.4 利用RANSAC拟合稳健的回归模型

线性回归模型可能会受到离群值的严重影响。在某些情况下，一小部分数据可能会对估计的模型系数有很大的影响。有许多统计测试可以用来检测异常值，这超出了本书的范围。然而，去除离群值需要数据科学家以及领域知识的判断。

除了淘汰离群值之外，还有一种更为稳健的回归方法，即采用随机抽样一致性（RANSAC）的算法，根据数据子集（所谓的内点）拟合回归模型。

总结迭代RANSAC算法如下：

1.随机选择一定数量的样本作为内点来拟合模型。

2.用模型测试所有其他的点，把落在用户给定容限范围内的点放入内点集。

3.调整模型中使用的所有的内点。

4.用内点重新拟合模型。

5.评估模型预测结果与内点集相比较的误差。

6.如果性能达到用户定义的阈值或指定的迭代数，则终止；否则返回到步骤1。

现在用scikit-learn的RANSACRegressor类结束基于RANSAC算法的线性模型讨论。

```
>>> from sklearn.linear_model import RANSACRegressor
>>> ransac = RANSACRegressor(LinearRegression(),
...                          max_trials=100,
...                          min_samples=50,
...                          loss='absolute_loss',
...                          residual_threshold=5.0,
...                          random_state=0)
>>> ransac.fit(X, y)
```

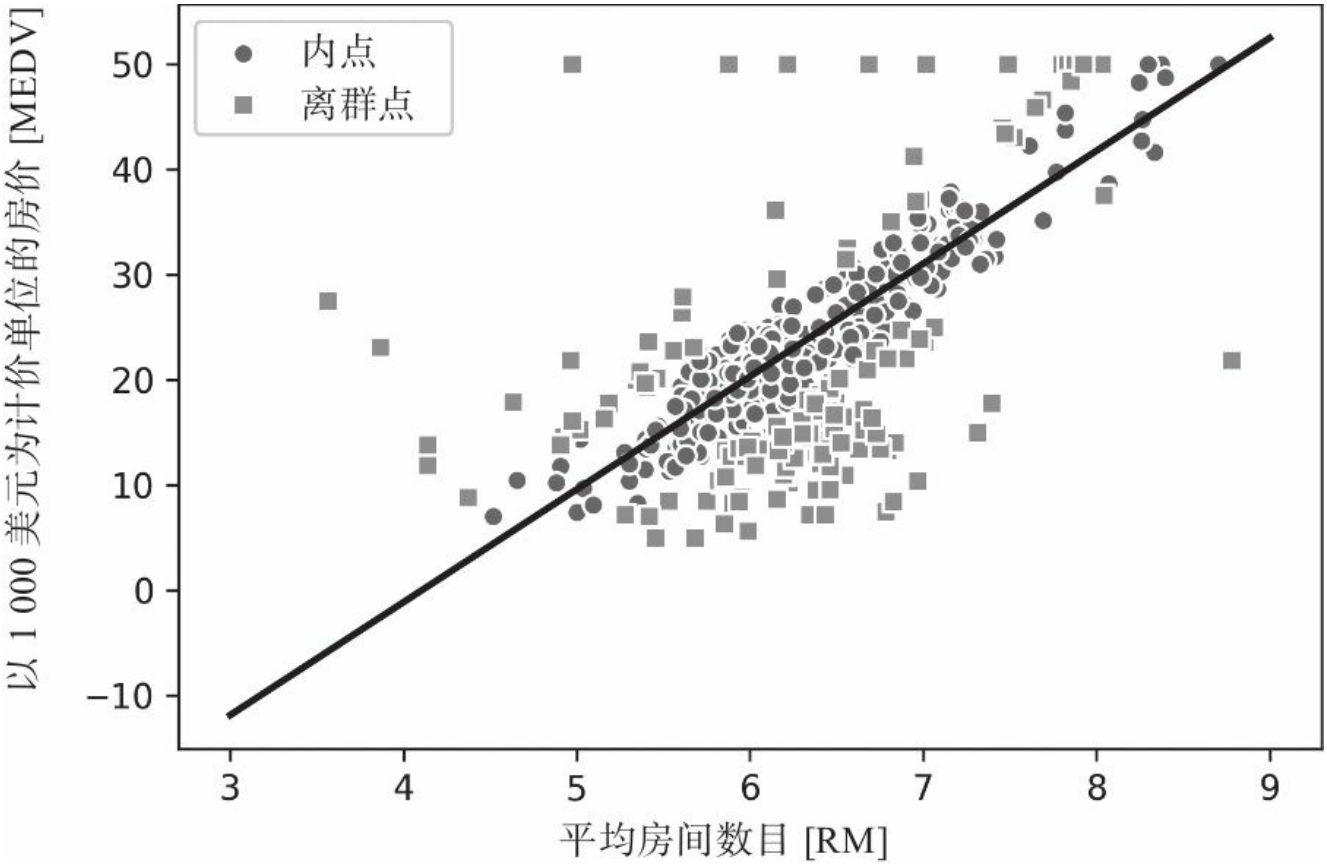
设置RANSACRegressor的最大迭代次数为100，设置随机选择样本的最低数量至少为50，min_samples=50。用'absolute_loss'作为形式参数residual_metric的实际参数，该算法计算拟合线和采样点之间的绝对垂直距离。通过设置residual_threshold参数为5.0，我们只允许与拟合线垂直距离在5个单位以内的采样点被包括在内点集，这对该特定数据集效果很好。

在默认的情况下，scikit-learn用MAD评估内点选择的阈值，MAD是目标值y的平均绝对偏差（Median Absolute Deviation）的缩写。然而，选择适当的内点阈值因问题而宜，这是RANSAC的不利点。最近几年已经研究了许多种不同的方法来自动选择最佳内点阈值。如果想进行更详细的讨论，可以参考：R.托多、A.费塞罗《在稳健的多重结构拟合中自动估计内点阈值》Springer出版社，2009年ICIAP，图像分析与处理，2009：123-131。

在拟合RANSAC模型之后，可以根据用RANSAC算法拟合的线性回归模型获得内点和离群点，并且把这些点和线性拟合的情况绘制成图：

```
>>> inlier_mask = ransac.inlier_mask_
>>> outlier_mask = np.logical_not(inlier_mask)
>>> line_X = np.arange(3, 10, 1)
>>> line_y_ransac = ransac.predict(line_X[:, np.newaxis])
>>> plt.scatter(X[inlier_mask], y[inlier_mask],
...             c='steelblue', edgecolor='white',
...             marker='o', label='Inliers')
>>> plt.scatter(X[outlier_mask], y[outlier_mask],
...             c='limegreen', edgecolor='white',
...             marker='s', label='Outliers')
>>> plt.plot(line_X, line_y_ransac, color='black', lw=2)
>>> plt.xlabel('Average number of rooms [RM]')
>>> plt.ylabel('Price in $1000s [MEDV]')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

下面的散点图可以看到线性回归模型拟合检测出的以圆形表示的内点数据集：




执行下面的代码利用模型计算出斜率和截距后可以看到线性回归拟合的线与前一节未用RANSAC拟合的结果有些不同：

```
>>> print('Slope: %.3f' % ransac.estimator_.coef_[0])
Slope: 10.735
>>> print('Intercept: %.3f' % ransac.estimator_.intercept_)
Intercept: -44.089
```

RANSAC降低了数据集中离群值的潜在影响，但并不知道这种方法对未见过数据的预测性能是否有良性影响。因此，下一节将研究评估回归模型的不同方法，这是建立预测模型系统的关键。

## 10.5 评估线性回归模型的性能

前一节学习了如何在训练数据上拟合回归模型。然而，在前几章中了解到用训练期间未见过的数据对模型进行测试可以获得对性能更客观的估计，这非常重要。

在第6章中我们把数据集分成单独的训练集和测试集，用前者拟合模型，用后者评估当模型推广到未见过数据时的性能。我们不再使用简单的回归模型，而是用数据集中的所有变量来训练多元回归模型：

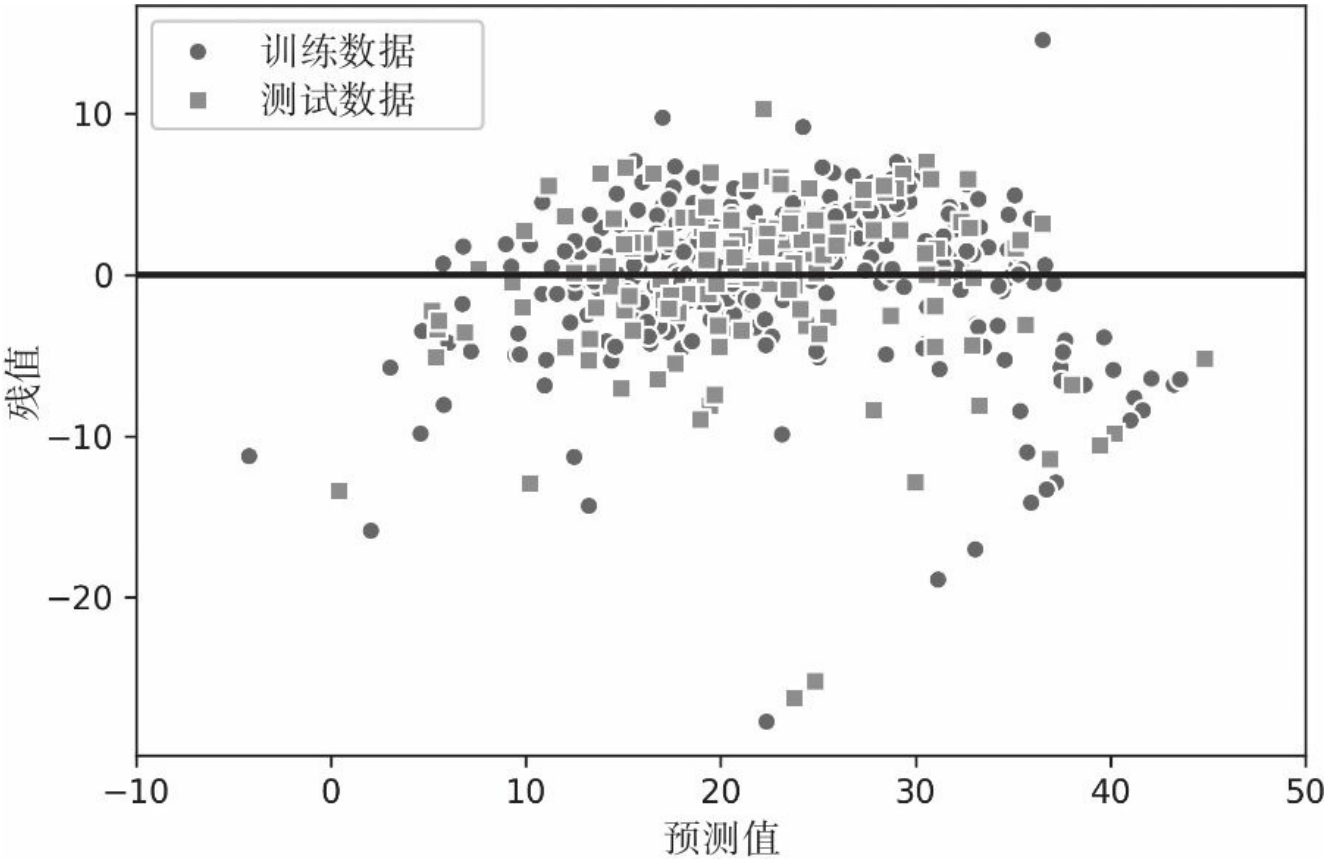
```
>>> from sklearn.model_selection import train_test_split
>>> X = df.iloc[:, :-1].values
>>> y = df['MEDV'].values
>>> X_train, X_test, y_train, y_test = train_test_split(
...         X, y, test_size=0.3, random_state=0)
>>> slr = LinearRegression()
>>> slr.fit(X_train, y_train)
>>> y_train_pred = slr.predict(X_train)
>>> y_test_pred = slr.predict(X_test)
```

由于模型使用了多个解释变量，所以无法在二维图中可视化线性回归线（更准确地说是超平面），但可以通过绘制残差（实际值和预测值之间的差异或垂直距离）与预测值来判断回归模型。残差图是判断回归模型常用的图形工具。这有助于检测非线性和异常值，并检查这些错误是否呈随机分布。

执行下面的代码可以绘制残差图，这里直接用预测值减去真实的目标变量：

```
>>> plt.scatter(y_train_pred,  y_train_pred - y_train,
...              c='steelblue', marker='o', edgecolor='white',
...              label='Training data')
>>> plt.scatter(y_test_pred,  y_test_pred - y_test,
...              c='limegreen', marker='s', edgecolor='white',
...              label='Test data')
>>> plt.xlabel('Predicted values')
>>> plt.ylabel('Residuals')
>>> plt.legend(loc='upper left')
>>> plt.hlines(y=0, xmin=-10, xmax=50, color='black', lw=2)
>>> plt.xlim([-10, 50])
>>> plt.show()
```

执行代码将会看到残差图中有一行通过x轴原点，如下所示：




在完美预测的情况下，残差刚好为零，这是在现实和实际应用中可能永远都不会遇到的。然而，我们期望好的回归模型的误差呈随机分布，残差应该随机分布在中心线附近。如果从残差图中看到模式存在，这意味着模型无法捕捉到一些解释性信息，这些信息已经泄露到了残差中，像在前面的残差图中所看到那样。此外，还可以用残差图来检测离群点，这些离群点由图中与中心线存在很大偏差的那些点来表示。

另一个有用的模型性能定量度量是所谓的均方误差（MSE），它仅仅是为了拟合线性回归模型，而将SSE成本均值最小化的结果。MSE对比较不同的回归模型或通过网格搜索和交叉验证调整其参数很有用，因为它通过调整样本比例尺寸使SSE归一化。

$$MSE = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2$$

下面的代码将计算训练和测试预测的MSE：

```
>>> from sklearn.metrics import mean_squared_error
>>> print('MSE train: %.3f, test: %.3f' % (
...         mean_squared_error(y_train, y_train_pred),
...         mean_squared_error(y_test, y_test_pred)))
MSE train: 19.958, test: 27.196
```

可以看到训练集的MSE为19.96，而测试集的MSE为27.20，比较大，这是模型过拟合训练数据的一个标志。

有时候报告决定系数（$R^2$）可能更有用，可以把这理解为MSE的标准版，目的是为更好地解释模型的性能。换句话说，$R^2$是模型捕获到的响应方差函数的一部分。定义$R^2$值如下：

$$R^2 = 1 - \frac{SSE}{SST}$$

这里，SSE是平方误差之和，而SST是平方和之总和：

$$SST = \sum_{i=1}^{n} \left( y^{(i)} - \mu_y \right)^2$$

换句话说，SST只是反应的方差。

下面将很快证明R2的确只是修正版的MSE：

$$R^2 = 1 - \frac{SSE}{SST}$$

$$1 - \frac{\frac{1}{n}\sum_{i=1}^{n} \left( y^{(i)} - \hat{y}^{(i)} \right)^2}{\frac{1}{n}\sum_{i=1}^{n} \left( y^{(i)} - \mu_{(y)} \right)^2}$$

$$1 - \frac{MSE}{Var(y)}$$

对于训练集，$R^2$的取值范围在0到1之间，但它也可以是负值。如果$R^2$=1，当相应的MSE=0时，模型完美地拟合数据。

在训练数据上进行评估，模型的$R^2$是0.765，这听起来并不太坏。但是，在测试集上的$R^2$只有0.673，这可以通过执行下面的代码来计算：

```
>>> from sklearn.metrics import r2_score
>>> print('R^2 train: %.3f, test: %.3f' %
...       (r2_score(y_train, y_train_pred),
...        r2_score(y_test, y_test_pred)))
R^2 train: 0.765, test: 0.673
```

## 10.6 用正则化方法进行回归

正如第3章所讨论的，正则化是通过添加额外信息解决过拟合问题的一种方法，而缩小模型参数值却引来了复杂性的惩罚。正则线性回归最常用的方法包括所谓的岭回归、最小绝对收缩与选择算子（LASSO）以及弹性网络。

岭回归是一个L2惩罚模型，只需把加权平方添加到最小二乘代价函数：

$$J(w)_{Ridge} = \sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2 + \lambda\|w\|_2^2$$

这里：

$$L2: \lambda\|w\|_2^2 = \lambda\sum_{j=1}^{m} w_j^2$$

通过加大超参数λ的值，我们增加了正则化的强度，同时收缩了模型的权重。注意，不要正则化截距项$w_0$。

另一种可能导致稀疏模型的方法是LASSO。取决于正则化的强度，某些权重可能成为零，这也使LASSO成为有监督特征选择的有用技术：

$$J(w)_{LASSO} = \sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2 + \lambda\|w\|_1$$

这里：

$$L1: \lambda\|w\|_1 = \lambda\sum_{j=1}^{m}|w_j|$$

然而，如果m>n，LASSO的限制是最多可以选择n个变量。处于岭回归和LASSO之间的折中方案是弹性网络，它以一个L1惩罚来产生稀疏性，以一个L2惩罚来克服LASSO的一些限制，例如选择变量的数量：

$$J(w)_{\text{ElasticNet}} = \sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2 + \lambda_1 \sum_{j=1}^{m} w_j^2 + \lambda_2 \sum_{j=1}^{m} |w_j|$$

那些正则化回归模型都可以通过scikit-learn获得，其用法与普通回归模型类似，除了必须通过参数优化λ指定正则化强度，例如，通过k折交叉验证优化。

可以通过下述代码初始化岭回归模型：

```
>>> from sklearn.linear_model import Ridge
>>> ridge = Ridge(alpha=1.0)
```

需要注意的是，正则化强度依靠参数alpha调节，这类似于参数λ。同样可以从linear_model模块初始化LASSO回归：

```
>>> from sklearn.linear_model import Lasso
>>> lasso = Lasso(alpha=1.0)
```

最后，ElasticNet能够调整L1与L2的比例：

```
>>> from sklearn.linear_model import ElasticNet
>>> elanet = ElasticNet(alpha=1.0, l1_ratio=0.5)
```

例如，如果把l1_ratio设置为0，ElasticNet回归等同于LASSO回归。有关线性回归不同实现的详细信息，请参阅下述文档：

http://scikit-learn.org/stable/modules/linear_model.html

## 10.7 将线性回归模型转换为曲线——多项式回归

前面的部分假设解释变量和响应变量之间存在着线性关系。一种有效解决违背线性假设的方法是通过增加项数采用多项式回归模型：

$$y = w_0 + w_1 x + w_2 x^2 + \cdots + w_d x^d$$

这里d为多项式的次数。虽然可以用多项式回归来模拟非线性关系，但是因为存在线性回归系数w，它仍然被认为是多元线性回归模型，下面的小节将看到如何更方便地在现有数据集上，通过增加多项式的项来拟合多项式回归模型。

### 10.7.1 用scikit-learn增加多项式的项

现在开始学习如何在scikit-learn用PolynomialFeatures转换器类，为解释变量增加二次项（d=2）以解决简单的回归问题。然后按照下面步骤来比较多项式拟合与线性拟合：

1.增加二次多项式：

```
from sklearn.preprocessing import PolynomialFeatures
>>> X = np.array([ 258.0, 270.0, 294.0, 320.0, 342.0,
...               368.0, 396.0, 446.0, 480.0, 586.0])\
...               [:, np.newaxis]
>>> y = np.array([ 236.4, 234.4, 252.8, 298.6, 314.2,
...                342.2, 360.8, 368.0, 391.2, 390.8])
>>> lr = LinearRegression()
>>> pr = LinearRegression()
>>> quadratic = PolynomialFeatures(degree=2)
>>> X_quad = quadratic.fit_transform(X)
```

2.为了比较，拟合一个简单的线性回归模型：

```
>>> lr.fit(X, y)
>>> X_fit = np.arange(250,600,10)[:, np.newaxis]
>>> y_lin_fit = lr.predict(X_fit)
```

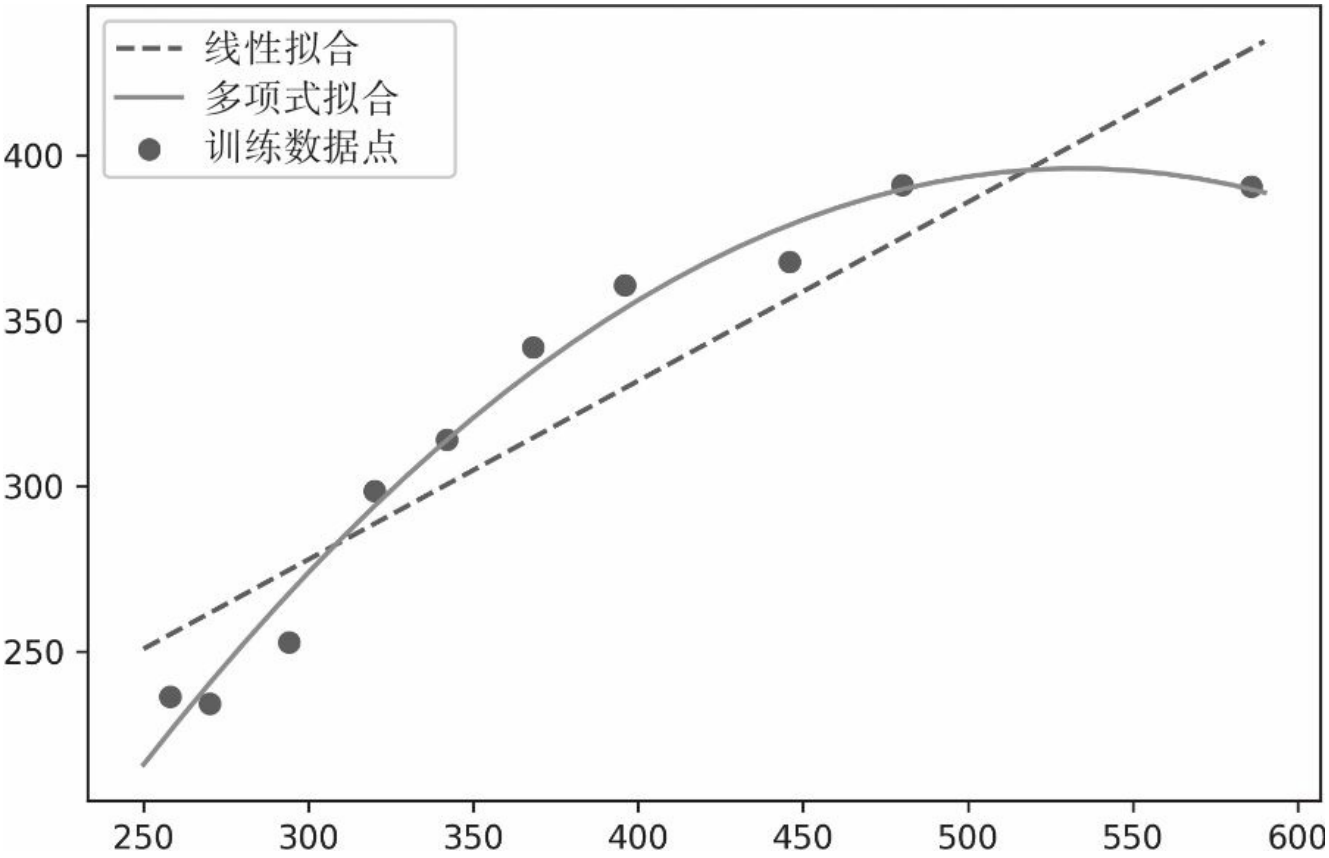
3.用变换后的特征以多项式回归拟合多元回归模型：

```
>>> pr.fit(X_quad, y)
>>> y_quad_fit = pr.predict(quadratic.fit_transform(X_fit))
```

4.绘制出结果：

```
>>> plt.scatter(X, y, label='training points')
>>> plt.plot(X_fit, y_lin_fit,
...          label='linear fit', linestyle='--')
>>> plt.plot(X_fit, y_quad_fit,
...          label='quadratic fit')
>>> plt.legend(loc='upper left')
>>> plt.show()
```

从结果图可以看到多项式拟合比线性拟合更好地反映了响应和解释变量之间的关系：




```
>>> y_lin_pred = lr.predict(X)
>>> y_quad_pred = pr.predict(X_quad)
>>> print('Training MSE linear: %.3f, quadratic: %.3f' % (
...         mean_squared_error(y, y_lin_pred),
...         mean_squared_error(y, y_quad_pred)))
Training MSE linear: 569.780, quadratic: 61.330
>>> print('Training  R^2 linear: %.3f, quadratic: %.3f' % (
...         r2_score(y, y_lin_pred),
...         r2_score(y, y_quad_pred)))
Training  R^2 linear: 0.832, quadratic: 0.982
```

执行代码后可以看到MSE从570（线性拟合）下降到61（二次项拟合）。同时，对这个特定问题，决定系数反映出二次项模型拟合（$R^2$=0.982）比线性拟合（$R^2$=0.832）更加合适。

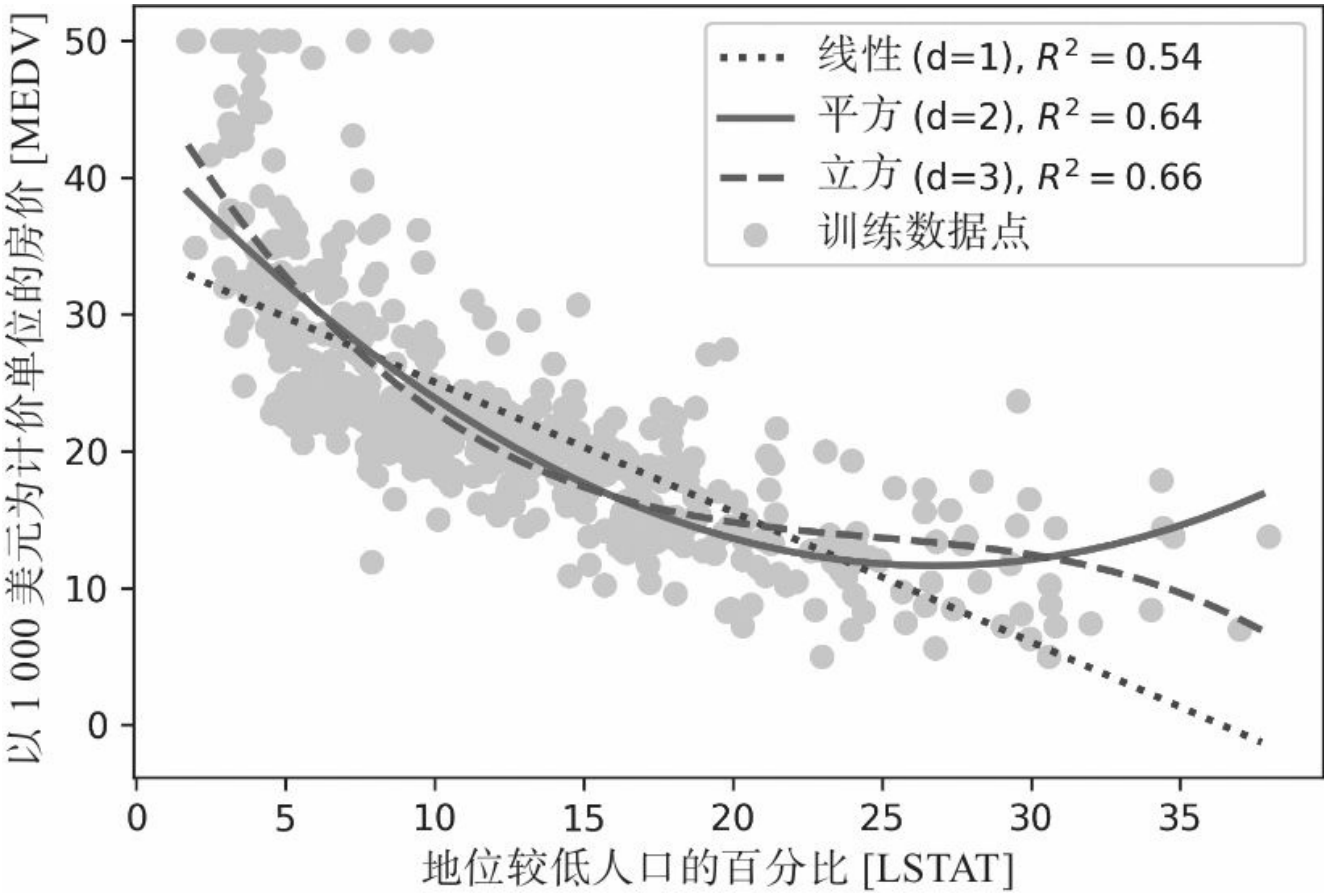
## 10.7.2 为住房数据集中的非线性关系建模

学习了如何构建多项式特征以拟合非线性关系后，现在通过一个更具体的例子将这些概念应用于住房数据集。通过执行下面的代码，将用第二度（二次）和第三度（三方）多项式构建房价和LSTAT（人口的地位低于某个百分点）之间关系的模型，并将它与线性拟合进行比较：

```
>>> X = df[['LSTAT']].values
>>> y = df['MEDV'].values

>>> regr = LinearRegression()

# create quadratic features
>>> quadratic = PolynomialFeatures(degree=2)
>>> cubic = PolynomialFeatures(degree=3)
>>> X_quad = quadratic.fit_transform(X)
>>> X_cubic = cubic.fit_transform(X)

# fit features
>>> X_fit = np.arange(X.min(), X.max(), 1)[:, np.newaxis]

>>> regr = regr.fit(X, y)
>>> y_lin_fit = regr.predict(X_fit)
>>> linear_r2 = r2_score(y, regr.predict(X))
```

```
>>> regr = regr.fit(X_quad, y)
>>> y_quad_fit = regr.predict(quadratic.fit_transform(X_fit))
>>> quadratic_r2 = r2_score(y, regr.predict(X_quad))

>>> regr = regr.fit(X_cubic, y)
>>> y_cubic_fit = regr.predict(cubic.fit_transform(X_fit))
>>> cubic_r2 = r2_score(y, regr.predict(X_cubic))


# plot results
>>> plt.scatter(X, y, label='training points', color='lightgray')

>>> plt.plot(X_fit, y_lin_fit,
...          label='linear (d=1), $R^2=%.2f$' % linear_r2,
...          color='blue',
...          lw=2,
...          linestyle=':')

>>> plt.plot(X_fit, y_quad_fit,
...          label='quadratic (d=2), $R^2=%.2f$' % quadratic_r2,
...          color='red',
...          lw=2,
...          linestyle='-')

>>> plt.plot(X_fit, y_cubic_fit,
...          label='cubic (d=3), $R^2=%.2f$' % cubic_r2,
...          color='green',
...          lw=2,
...          linestyle='--')

>>> plt.xlabel('% lower status of the population [LSTAT]')
>>> plt.ylabel('Price in $1000s [MEDV]')
>>> plt.legend(loc='upper right')
>>> plt.show()
```

结果图如下：




可以看到，三次项拟合捕捉房价与LSTAT之间的关系优于线性和二次项的拟合。然而应该意识到，增加越来越多的多项式特征也提高了模型的复杂性，因此增加了过拟合的机会。因此，在实践中，我们总是建议在单独的测试集上评估模型的泛化性能。

此外，多项式特征并非总是对非线性关系建模的最佳选择。例如，如果有一些经验或直觉，只是看看MEDV-LSTAT的散点图就可能会得出这样的假设，即LSTAT特征变量的对数变换和MEDV的平方根，可能把数据映射到适合线性回归拟合的线性特征空间。例如，我的看法是这两个变量之间的关系看起来与指数函数非常相似：

$$f(x) = 2^{-x}$$

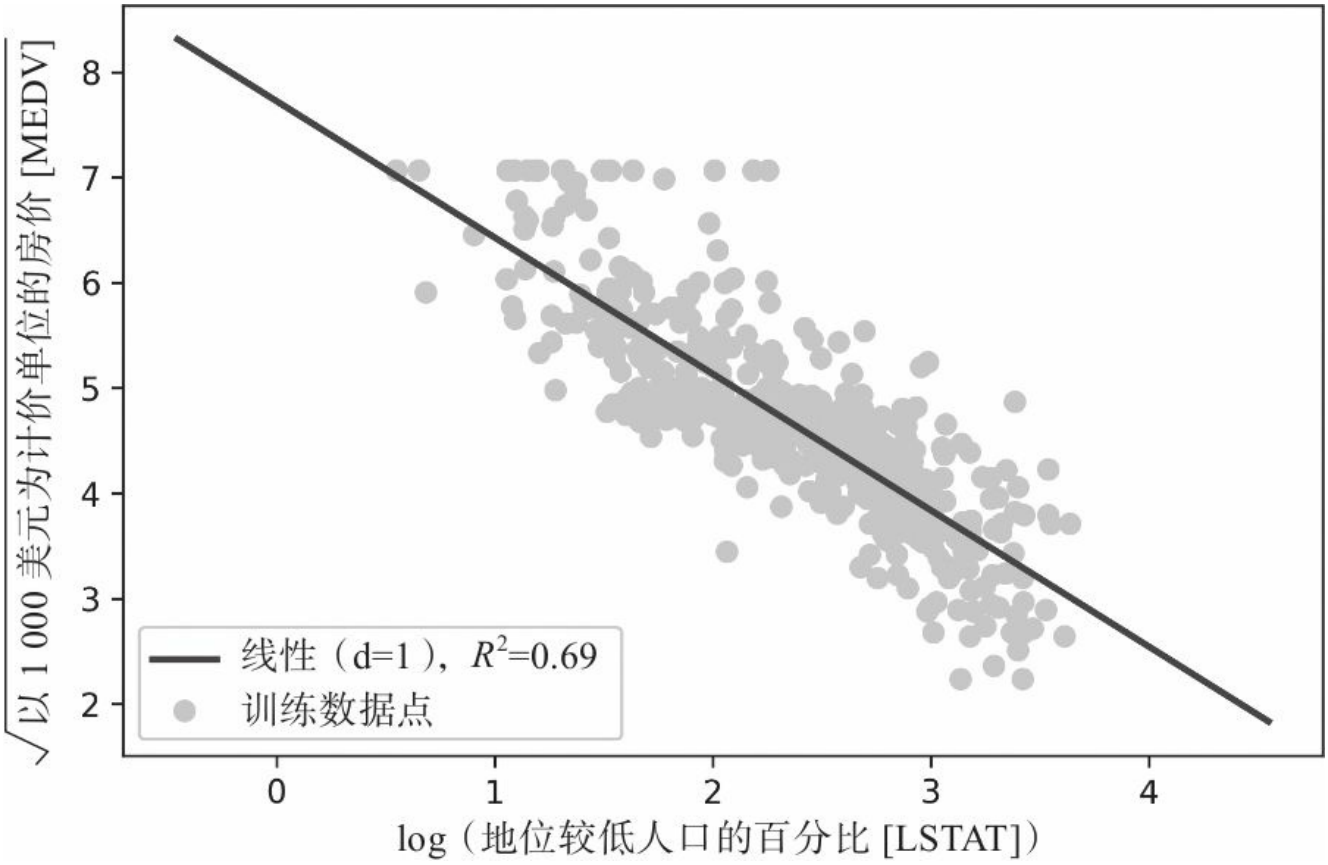
由于指数函数的自然对数是一条直线，所以我认为这种对数变换在这里应用会很有效：

$$\log\left(f(x)\right) = -x$$

让我们通过执行下面的代码来测试这个假设：

```
# transform features
>>> X_log = np.log(X)
>>> y_sqrt = np.sqrt(y)

# fit features
>>> X_fit = np.arange(X_log.min()-1,
...                   X_log.max()+1, 1)[:, np.newaxis]
>>> regr = regr.fit(X_log, y_sqrt)
>>> y_lin_fit = regr.predict(X_fit)
>>> linear_r2 = r2_score(y_sqrt, regr.predict(X_log))

# plot results
>>> plt.scatter(X_log, y_sqrt,
...             label='training points',
...             color='lightgray')
>>> plt.plot(X_fit, y_lin_fit,
...          label='linear (d=1), $R^2=%.2f$' % linear_r2,
...          color='blue',
...          lw=2)
>>> plt.xlabel('log(% lower status of the population [LSTAT])')
>>> plt.ylabel('$\sqrt{Price \; in \; \$1000s \; [MEDV]}$')
>>> plt.legend(loc='lower left')
>>> plt.show()
```

在将解释变量转换到对数空间并对目标变量取平方根之后，就能够用线性回归线捕捉两个变量之间的关系。这似乎比以前任何多项式特征变换都能更好地拟合数据变量（$R^2$=0.69）：


√以 1 000 美元为计价单位的房价 [MEDV]
8
7
6
5
4
3
2
0
1
2
3
4
log（地位较低人口的百分比 [LSTAT]）
线性（d=1），$R^2$=0.69
训练数据点

## 10.8 用随机森林处理非线性关系

本节将讨论随机森林回归，它与本章以前的回归模型在概念上有所不同。随机森林是多个决策树的集合，可以理解为分段线性函数之和，与以前讨论过的全局性线性和多项式回归模型相反。换句话说，通过决策树算法，可以进一步将输入空间细分为更小的区域，这些区域也因此变得更易于管理。

### 10.8.1 决策树回归

决策树算法的一个优点是，如果处理非线性数据，它不需要对特征进行任何转换。在第3章中我们曾经通过迭代分裂节点，直到当叶子是纯的，或者满足停止分裂的标准为止来培养一棵决策树。当用决策树进行分类时，定义熵作为杂质的指标以确定哪个特征分割可以最大化信息增益（IG），二元分裂可以定义为：

$$IG\left(D_p, x_i\right) = I\left(D_p\right) - \frac{N_{\text{left}}}{N_p} I\left(D_{left}\right) - \frac{N_{\text{right}}}{N_p} I\left(D_{right}\right)$$

这里x是要分裂的样本特征，$N_p$为父节点的样本数，I为杂质函数，$D_p$是父节点训练样本子集。$D_{left}$和$D_{right}$为分裂后左右子节点的训练样本集。记住，目标是要找到可以最大化信息增益的特征分裂；换句话说，希望找到可以减少子节点中杂质的特征分裂。第3章讨论过基尼杂质和度量杂质的熵，这两者都是有用的分类标准。然而，为了把回归决策树用于回归，需要一个适合连续变量的杂质指标，所以，作为替换，我们把节点t的杂质指标定义为MSE：

$$I\left(t\right) = MSE\left(t\right) = \frac{1}{N_t} \sum_{i \in D_i} \left(y^{(i)} - \hat{y}_t\right)^2$$

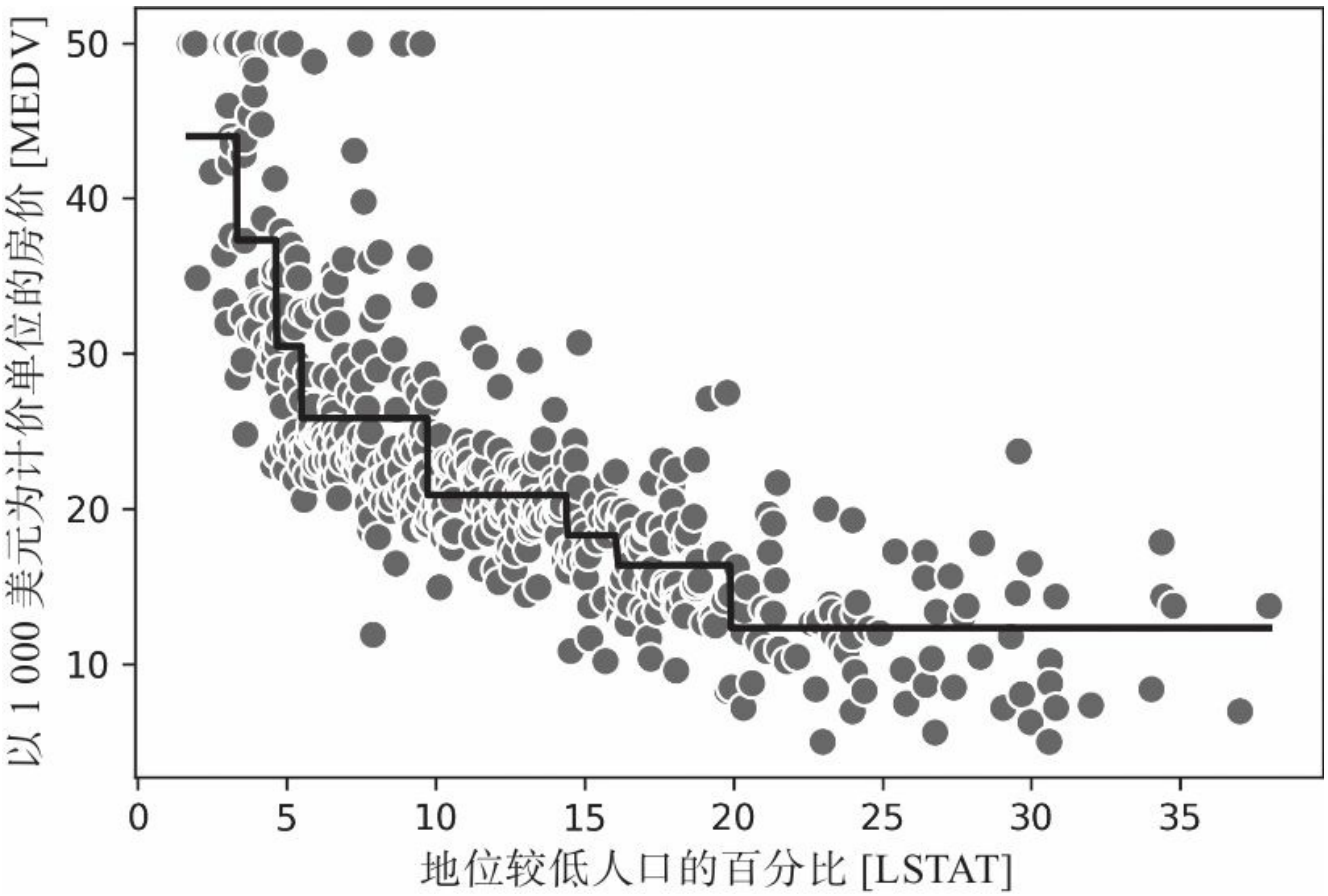
这里$N_t$为节点t的训练样本数，$D_t$为节点t的训练子集。$y^{(i)}$为真实的目标值，$\hat{y}_t$为预测的目标值（样本均值）：

$$\hat{y}_t = \frac{1}{N_t} \sum_{i \in D_i} y^{(i)}$$

在决策树回归的背景下，MSE通常也被称为节点方差，这就是为什么分裂标准也被称为方差缩减。用scikit-learn实现的DecisionTreeRegressor为变量MEDV和LSTAT之间的非线性关系建立模型，让我们看看决策树拟合的情况：

```
>>> from sklearn.tree import DecisionTreeRegressor
>>> X = df[['LSTAT']].values
>>> y = df['MEDV'].values
>>> tree = DecisionTreeRegressor(max_depth=3)
>>> tree.fit(X, y)
>>> sort_idx = X.flatten().argsort()
>>> lin_regplot(X[sort_idx], y[sort_idx], tree)
>>> plt.xlabel('% lower status of the population [LSTAT]')
>>> plt.ylabel('Price in $1000s [MEDV]')
>>> plt.show()
```

正如在结果图中所看到的，决策树捕获了数据中的基本趋势。然而，这种模式的一个局限是它并不捕获所需预测的连续性和可微性。此外，在选择树的深度值时要小心，既要确保不过拟合也要避免欠拟合。这里深度为3似乎是个不错的选择：




下一节将研究一种更稳健的回归树拟合方法：随机森林。

### 10.8.2 随机森林回归

正如第3章中学到的，随机森林算法是一种组合了多个决策树的技术。由于随机性，随机森林通常比单个决策树具有更好的泛化性能，这有助于减少模型的方差。随机森林的其他优点还包括它对数据集中的离群值不敏感，而且也不需要太多的参数优化。随机森林中通常唯一需要试验的参数是组合中决策树的个数。基本的回归随机森林算法与第3章中讨论的随机森林分类算法几乎相同，唯一不同的是用MSE准则来培育每棵决策树，并用决策树平均预测值来计算预测的目标变量。

现在，让我们使用住房数据集中的所有特征来拟合随机森林回归模型，用其中60%的样本做拟合，其余40%的样本做性能评估。代码如下：

```
>>> X = df.iloc[:, :-1].values
>>> y = df['MEDV'].values
>>> X_train, X_test, y_train, y_test =\
...       train_test_split(X, y,
...                        test_size=0.4,
...                        random_state=1)

>>> from sklearn.ensemble import RandomForestRegressor
>>> forest = RandomForestRegressor(n_estimators=1000,
...                                criterion='mse',
...                                random_state=1,
...                                n_jobs=-1)
>>> forest.fit(X_train, y_train)
>>> y_train_pred = forest.predict(X_train)
>>> y_test_pred = forest.predict(X_test)
>>> print('MSE train: %.3f, test: %.3f' % (
...        mean_squared_error(y_train, y_train_pred),
...        mean_squared_error(y_test, y_test_pred)))
MSE train: 1.642, test: 11.052
>>> print('R^2 train: %.3f, test: %.3f' % (
...        r2_score(y_train, y_train_pred),
...        r2_score(y_test, y_test_pred)))
R^2 train: 0.979, test: 0.878
```

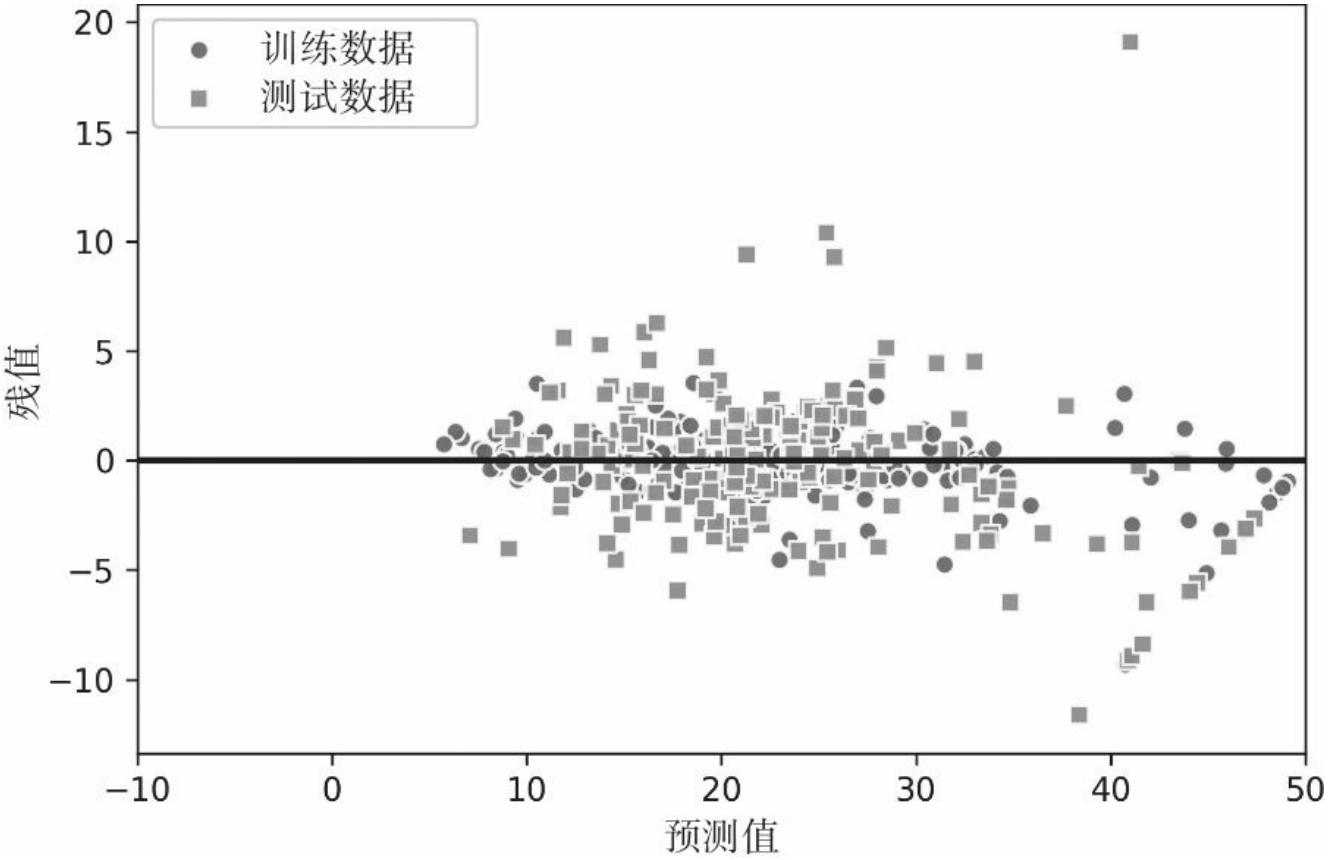
不幸的是，我们看到随机森林趋向于过拟合训练数据。然而，它仍然能够很好地解释目标变量和解释变量之间的关系（测试集$R^2$=0.871）。

最后，来看看预测残差：

```
>>> plt.scatter(y_train_pred,
...             y_train_pred - y_train,
...             c='steelblue',
...             edgecolor='white',
...             marker='o',
...             s=35,
...             alpha=0.9,
...             label='Training data')
>>> plt.scatter(y_test_pred,
...             y_test_pred - y_test,
...             c='limegreen',
...             edgecolor='white',
...             marker='s',
...             s=35,
...             alpha=0.9,
...             label='Test data')
>>> plt.xlabel('Predicted values')
>>> plt.ylabel('Residuals')
>>> plt.legend(loc='upper left')
>>> plt.hlines(y=0, xmin=-10, xmax=50, lw=2, color='black')
>>> plt.xlim([-10, 50])
>>> plt.show()
```

正如$R^2$系数所总结的，如y轴方向上的离群值所示，可以看到该模型拟合训练数据比拟合测试数据更好。此外，残差分布似乎并不是围绕零中心点完全随机，这表明该模型不能捕捉所有的探索性信息。

然而，下述残差图比本章前面所述的线性模型残差图有了很大的改进。




在理想情况下的模型错误应该是随机或不可预测的。换言之，预测的错误不应该与解释变量中所包含的任何信息有关，而应反映现实世界的分布或模式的随机性。如果通过检查残差图观察到了预测误差的模式，那就意味着残差图中包含着预测信息。一个常见的原因可能是解释性信息泄露到了这些残差中。

幸好现在有处理残差图中非随机性的通用方法，不过它仍需要实验。取决于能得到的数据，或许能通过变量变换、优化学习算法的超参数、选择更简单或更复杂的模型、除去离群值或者增加额外的变量来改善模型。

第3章介绍了核技巧，它可以与支持向量机（SVM）组合完成分类任务，这对处理非线性问题很有用。虽然这方面的讨论超出了本书的范围，但支持向量机也可以用于非线性回归任务。感兴趣的读者可以在下面这篇优秀的研究报告中找到有关SVM用于回归的更多信息：S.R.干等.《用于分类和回归的支持向量机》.ISIS技术报告，1998年第14期。scikit-learn也实现了SVM回归，可以从下述网页链接找到更多关于其使用的信息：

http://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html#sklearn.svm.SVR

## 10.9 小结

本章开篇学习了如何构建简单的线性回归模型，以分析单个解释变量和连续响应变量之间的关系。然后讨论了一种有用的解释数据分析技术，以查看数据中的模式和异常，这是在预测建模任务中迈出的重要的第一步。

我们采用基于梯度的优化方法实现线性回归建立了第一个模型。然后看到了如何把scikit-learn线性模型用于回归，实现用于处理离群情况的稳健回归方法（RANSAC）。为了评估回归模型的预测性能，我们计算了平均误差平方和以及相关的$R^2$度量。此外，还讨论了判断回归模型问题的图解方法：残差图。

之后我们讨论了如何把正则化方法应用于回归模型以降低模型复杂度，同时避免过拟合。然后介绍了为非线性关系建模的几种方法，包括多项式特征变换和随机森林回归。

前几章详细地讨论了有监督学习、分类和回归分析。下一章将进入另一个有趣的机器学习子领域，即无监督学习，还将学习如何在没有目标变量的情况下使用聚类分析来发现数据中的隐藏结构。

# 第11章　用聚类分析处理无标签数据

前面的章节用有监督学习技术建立了机器学习模型，用已知答案的数据训练模型对标签进行分类和预测。本章将开始探索聚类分析，这是一种无监督学习技术，在无法预先知道正确答案的情况下可以发现数据中的隐藏结构。聚类的目标是在数据中找到自然分组，确保相同集群中的元素比不同集群中的元素更相似。

鉴于其探索性，集群是一个令人兴奋的主题，主要涵盖下述几个方面的概念，这将有助于将数据组织成有意义的结构：

·使用流行的k-均值算法寻找相似中心

·采用自下而上的方法构建层次形聚类树

·使用基于密度的聚类方法识别任意形状的物体

## 11.1 用k-均值进行相似性分组

本节将学习最常用的聚类算法——k-均值算法，它在学术界和工业界都有广泛的应用。聚类（或聚类分析）是一种能够找到相似对象的技术，这些对象彼此之间的关系比其他群组对象之间的关系更为密切。面向业务的聚类应用示例包括对不同主题的文档、音乐和电影的分组，或者作为推荐引擎寻找与之有共同购买行为和兴趣的客户。

### 11.1.1 scikit-learn的k-均值聚类

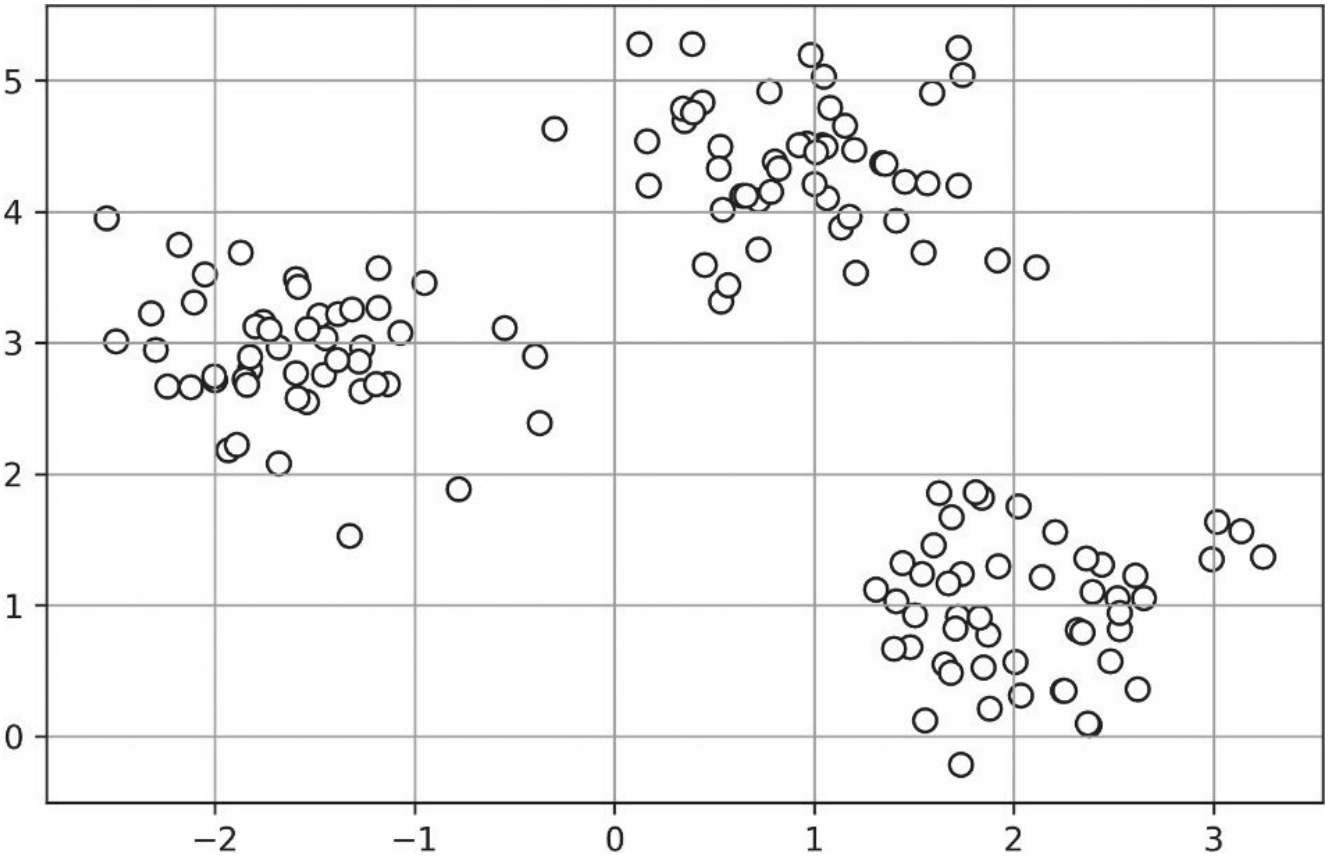
稍后可以看到，与其他聚类算法相比，k-均值算法极易实现而且计算效率也很高，这也许可以解释为什么它这么流行。k-均值算法属于基于原型聚类的范畴。本章后面将会讨论层次化和基于密度的另外两类聚类方法。

基于原型的聚类方法意味着每个集群代表一个原型，可以是有类似连续性特征点的重心，或者是在分类特征最具代表性或最频繁出现的中心。k-均值方法非常擅长识别球形集群，其缺点是必须指定集群数k，所以它是个先验方法。如果k值选择不当会导致聚类性能不良。本章的后面将讨论肘法和轮廓图，这是评估聚类质量以确定最佳聚类集群数k的有效技术。

尽管k-均值集群可以应用于更高维度的数据，为了方便可视化，我们将展示简单的二维数据集：

```
>>> from sklearn.datasets import make_blobs
>>> X, y = make_blobs(n_samples=150,
...                   n_features=2,
...                   centers=3,
...                   cluster_std=0.5,
...                   shuffle=True,
...                   random_state=0)

>>> import matplotlib.pyplot as plt
>>> plt.scatter(X[:,0],
...             X[:,1],
...             c='white',
...             marker='o',
...             edgecolor='black',
...             s=50)
>>> plt.grid()
>>> plt.show()
```

前面刚创建的数据集包括150个随机生成的点，大致有三个高密度区域，可以通过二维散点图实现可视化：




实际的聚类应用并没有任何关于标定样本的真实分类信息，否则它将属于有监督学习的范畴。因此目标是根据特征的相似性对样本进行分组，这可以通过以下四个步骤所总结出的k-均值算法来实现：

1.随机从样本中挑选k个重心作为初始聚类中心。

2.将每个样本分配到最近的重心$\mu^{(j)}$，$j \in \{1, \ldots, k\}$。

3.把重心移到已分配样本的中心。

4.重复步骤2和3，直到集群赋值不再改变，或者达到用户定义的容限或最大迭代数。

下一个问题是如何度量对象之间的相似性？可以把相似性定义为距离的反义，具有连续特征的聚类样本，其常用距离为m维空间中x和y之间的欧氏距离的平方：

$$d(\boldsymbol{x}, \boldsymbol{y})^2 = \sum_{j=1}^{m} (x_j - y_j)^2 = \|\boldsymbol{x} - \boldsymbol{y}\|_2^2$$

注意，在前面的方程中，指数j是指样本点x和y的第j维（特征列）。本节的其余部分将用下标i和j分别代表样本索引和聚类索引。

基于欧几里得距离度量，可以把k-均值算法描述为一种简单的优化问题，是一种最小化群内误差平方和（SSE）的迭代方法，有时也被称为群惯性：

$$SSE = \sum_{i=1}^{n}\sum_{j=1}^{k} w^{(i,j)} \left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(i)} \right\|_2^2$$

这里$\mu^{(j)}$为集群j的代表点（重心），如果样本$x^{(i)}$在集群j中，$w^{(i,j)}=0$，否则$w^{(i,j)}=1$。

现在已经理解了简单的k-均值算法的工作原理，我们将采用KMeans类scikit-learn cluster模块将其应用到样本数据：

```
>>> from sklearn.cluster import KMeans
>>> km = KMeans(n_clusters=3,
...             init='random',
...             n_init=10,
...             max_iter=300,
...             tol=1e-04,
...             random_state=0)
>>> y_km = km.fit_predict(X)
```

前面的代码将期望集群的数目设置为3，在没有推理的基础上指定集群数目是k-均值的局限之一。我们设置n_init=10执行10次独立的k-均值聚类算法，每次计算随机选用不同的中心，选择SSE最低的为最终模型。通过参数max_iter指定每次运行的最大迭代数（这里是300）。值得注意的是，如果在收敛之前达到最大迭代数，scikit-learn实现的k-均值就会提早停止计算。然而在特定运行过程中，如果选择较大的max_iter值，k-均值有可能无法达致收敛，这会产生巨大的计算成本。解决收敛性问题的一种方法是选择较大的tol值，该参数控制集群内误差平方和的变化以定义收敛标准。前面的代码选择的容限为1e-04（=0.0001）。

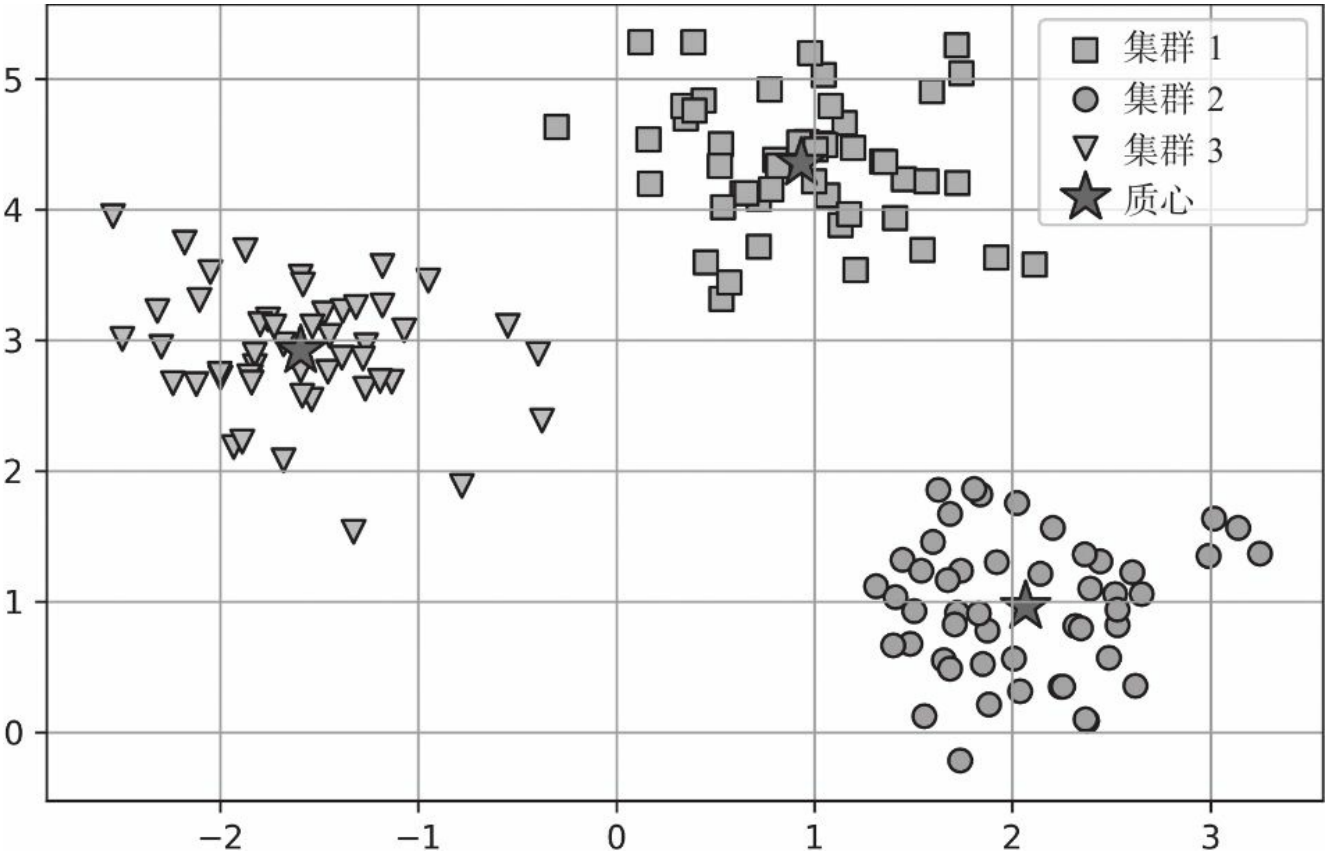
k-均值的问题是一个或多个集群可能为空。注意，该问题对k-中心点或模糊C-均值算法不存在，本节稍后将会讨论模糊C-均值算法。然而，这个问题会对scikit-learn目前实现的k-均值算法有影响。如果一个集群为空，该算法将搜索与集群中心相距最远的样本。然后将中心重新分配为最远点。

当基于欧氏距离度量把k-均值算法应用到真实数据时，要确保特征度量的比例尺度一致，必要时可以用z-score标准化或最小最大化方法处理。

在预测了聚类标签y_km和讨论了一些k-均值算法的挑战之后，把k-均值在数据集中发现的集群以及聚类中心用图表示出来。这些数据存储在拟合对象KMeans的属性cluster_centers_中：

```
>>> plt.scatter(X[y_km == 0, 0],
...             X[y_km == 0, 1],
...             s=50, c='lightgreen',
...             marker='s', edgecolor='black',
...             label='cluster 1')
>>> plt.scatter(X[y_km == 1, 0],
...             X[y_km == 1, 1],
...             s=50, c='orange',
...             marker='o', edgecolor='black',
...             label='cluster 2')
>>> plt.scatter(X[y_km == 2, 0],
...             X[y_km == 2, 1],
...             s=50, c='lightblue',
...             marker='v', edgecolor='black',
...             label='cluster 3')
>>> plt.scatter(km.cluster_centers_[:, 0],
...             km.cluster_centers_[:, 1],
...             s=250, marker='*',
...             c='red', edgecolor='black',
...             label='centroids')
>>> plt.legend(scatterpoints=1)
>>> plt.grid()
>>> plt.show()
```

从下面的散点图可以看到k-均值把三个重心放在球形中，对所给的数据来说看起来分组合理：




虽然k-均值在这个实验数据集上工作得很好，但必须指出k-均值的另一个缺点：必须在无推理的前提下指定集群的数目k。在实际应用中，选择的集群数目可能并不总是那么明显，特别是在处理无法可视化的高维数据集时。k-均值的其他特性是假设每个集群中至少有一个成员，集群不重叠而且不分层。本章后面将遇到不同类型的聚类算法，基于层次和密度的聚类。这两种算法都不要求预先指定集群数目或者在数据集中假设球形结构。

下一小节将介绍被称为k-均值++的经典k-均值算法变体。虽然它没有解决上一段中讨论过的k-均值假设和缺点，但可以通过更精确地设置初始聚类中心极大地改善聚类的结果。

## 11.1.2 k-均值++——更聪明地设置初始聚类中心的方法

到目前为止，已经讨论了用随机种子设置初始值中心的经典k-均值算法，有时如果初始聚类中心选择不当会导致聚类不良或收敛缓慢。解决这个问题的一种方法是在数据集上多次运行k-均值算法，并根据SSE选择性能最佳的模型。另一个策略是通过k-均值++算法，将初始中心彼此的距离设置得足够远，带来比经典k-均值更好和更一致的结果（D.阿瑟和S.瓦斯威斯基.《k-均值++：精心播种的好处》.第十八届ACM-SIAM离散算法研讨会论文集，2007：1027-1035，工业和应用数学学会）。

概括k-均值++的初始化过程如下：

1.初始化空集合M来存储选择的k个重心。

2.从输入样本中随机选择第一个重心$\mu^{(j)}$然后加入M。

3.找出不在M中的每个样本$x^{(i)}$与M中每个点的最小距离的平方$d(x^{(i)}, M)^2$。

4.随机选择下一个重心$\mu^{(p)}$，计算加权概率分布$\frac{d\left(\boldsymbol{\mu}^{(p)}, M\right)^2}{\sum_i d\left(\boldsymbol{x}^{(i)}, M\right)}$。

5.重复步骤2和3直至选中k个重心。

6.继续进行经典的k-均值算法。

要用scikit-learn的Kmeans对象的k-均值++，只需要把k-均值++的参数init设置成'k-means++'。事实上，'k-means++'是形式参数init的默认实际参数，在实践中强烈推荐该选择。在前面的例子中没有使用它的唯一原因是不想一次引入过多的概念。本节的其余部分将使用k-均值++，但是鼓励读者尝试更多不同的方法（通过init='random'设置的经典的k-均值以及通过init='k-means++'设置的k-均值++）设置聚类的初始中心。

### 11.1.3 硬聚类与软聚类

硬聚类描述了一类算法，把数据集中的每个样本分配到一个集群，例如在前面小节中讨论过的k-均值算法。与此相反，软聚类算法（有时也称为模糊聚类）将一个样本分配给一个或多个集群。常见的软聚类例子是模糊C-均值算法（FCM）（也被称为软k-均值或模糊k-均值）。最初的想法可以追溯到20世纪70年代，约瑟夫·C.杜安首次提出了模糊聚类的改进版本（J.C.杜安.《ISODATA方法的模糊近亲及其在检测分离良好的紧凑集群中的应用》.1973）。大约十年后，詹姆斯·C.拜佐克发表了他对模糊聚类算法的改进工作，这就是现在已知的FCM算法（J.C.拜佐克.《模式识别与模糊目标函数的算法》.施普林格科学与商业媒体，2013）。

FCM与k-均值程序非常相似。用集群中的每个点的概率替换硬集群分配。在k-均值中，可以用二进制的稀疏向量表示样本x的集群成员：

$$\begin{bmatrix} \mu^{(1)} \rightarrow 0 \\ \mu^{(2)} \rightarrow 0 \\ \mu^{(3)} \rightarrow 0 \end{bmatrix}$$

这里，值为1的索引位置指示集群重心$\mu^{(j)}$赋予样本的值（假设k=3，$j \in \{1, 2, 3\}$）。与之相反，可以把FCM的成员向量表示如下：

$$\begin{bmatrix} \mu^{(1)} \rightarrow 0.10 \\ \mu^{(2)} \rightarrow 0.85 \\ \mu^{(3)} \rightarrow 0.05 \end{bmatrix}$$

每个成员的取值范围为[0，1]，代表各个成员相对于集群重心的概率。给定样本成员概率之和等于1。与k-均值算法类似，可以把FCM算法总结为四个关键步骤：

1.指定k重心的数量然后随机为每个重心点分配集群成员。

2.计算集群的中心$\mu^{(j)}$，$j \in \{1, \ldots, k\}$。

3.更新每个点的集群成员。

4.重复步骤2和3直到成员系数不再变化，或达到用户定义的容限或最大迭代数。

我们把FCM的目标函数缩写为$J_m$，该函数看起来非常像在k-均值中见过的群内误差平方和：

$$J_m = \sum_{i=1}^{n}\sum_{j=1}^{k} w^{m(i,j)} \left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2^2$$

然而，值得注意的是会员指数$w^{(i,\ j)}$不像在k-均值（$w^{(i,\ j)} \in \{0, 1\}$）中那样是二进制值形式，而是代表集群成员概率（$w^{(i,\ j)} \in \{0, 1\}$）的实际值。你或许注意到$w^{(i,\ j)}$增加了一个指数，指数m为任何大于或者等与1的数（通常m=2），被称为模糊系数（或简称模糊化），用于控制模糊的程度。m值越大，集群成员数就越小，集群就越模糊。集群成员概率本身计算如下：

$$w^{(i,j)} = \left[ \sum_{p=1}^{k} \left( \frac{\left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2}{\left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(p)} \right\|_2} \right)^{\frac{2}{m-1}} \right]^{-1}$$

例如，假设像以前k-均值的例子那样选择三个聚类中心，可以计算出属于集群$\mu^{(j)}$的样本$x^{(i)}$的会员如下：

$$w^{(i,j)} = \left[ \left( \frac{\left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2}{\left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(1)} \right\|_2} \right)^{\frac{2}{m-1}} + \left( \frac{\left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2}{\left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(2)} \right\|_2} \right)^{\frac{2}{m-1}} + \left( \frac{\left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2}{\left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(3)} \right\|_2} \right)^{\frac{2}{m-1}} \right]^{-1}$$

计算所有样本的加权平均为集群的中心$\mu^{(j)}$，权重（$w^{m\,(i,\ j)}$）为每个样本属于集群的程度：

$$\boldsymbol{\mu}^{(j)} = \frac{\sum_{i=1}^{n} w^{m(i,j)} \boldsymbol{x}^{(i)}}{\sum_{i=1}^{n} w^{m(i,j)}}$$

只要看一下计算集群成员的等式，就可以直观地说FCM的每次迭代都比k-均值中的迭代更为昂贵。然而FCM通常需要较少的迭代就能收敛。不幸的是，scikit-learn目前还没有实现FCM算法。然而，实际应用中已经发现，k-均值和FCM产生非常相似的聚类结果，如在下述研究中所描述的：S.高石，S.K.杜贝.《k-均值算法和模糊C-均值算法的比较分析》.IJACSA，2013，4：35-38。

### 11.1.4 用肘法求解最佳聚类数

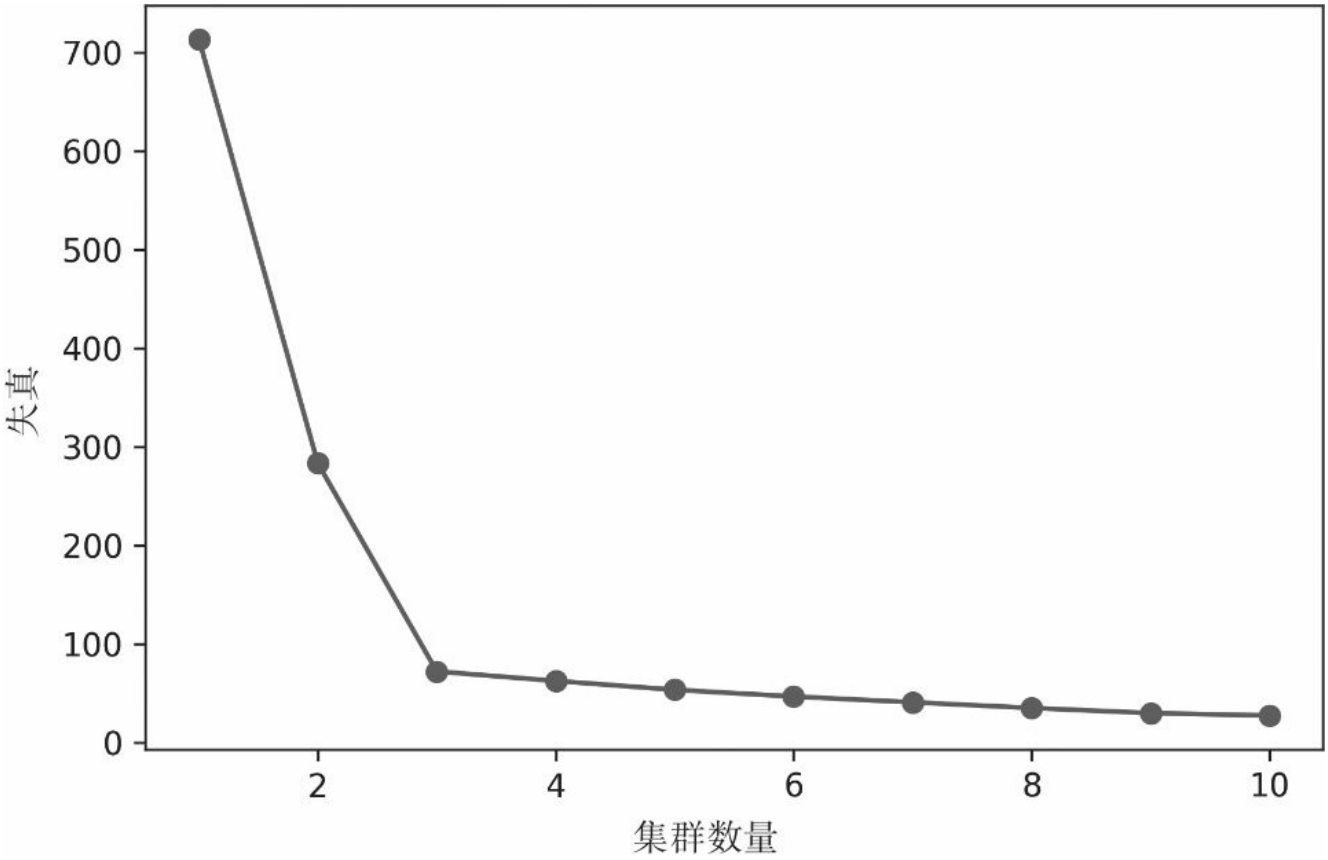
无监督学习的主要挑战之一是我们不知道明确的答案。数据集中没有标定过的真实分类标签，使我们能够应用第6章用过的技术来评估有监督学习模型的性能。因此，要量化聚类的质量需要用有内在联系的指标，如本章前面讨论过的群内SSE（失真）来比较不同k-均值聚类性能。在KMeans模型拟合之后，可以非常方便地用scikit-learn，不需要具体计算群内的SSE，因为它已经可以通过inertia_属性访问：

```
>>> print('Distortion: %.2f' % km.inertia_)
Distortion: 72.48
```

可以使用图形工具基于群内SSE用所谓的肘法来评估给定任务的最佳集群数k。直观地说，如果k增大，失真会减小。这是因为样本将接近它们被分配的中心。肘法的逻辑是识别当失真增速最快时的k值，如果为不同k值绘制失真图，情况就会变得更清楚：

```
>>> distortions = []
>>> for i in range(1, 11):
...     km = KMeans(n_clusters=i,
...                 init='k-means++',
...                 n_init=10,
...                 max_iter=300,
...                 random_state=0)
>>>     km.fit(X)
>>>     distortions.append(km.inertia_)
>>> plt.plot(range(1,11), distortions, marker='o')
>>> plt.xlabel('Number of clusters')
>>> plt.ylabel('Distortion')
>>> plt.show()
```

正如下图可以看到的那样，肘位于k=3证明它确实是该数据集的一个很好的选择：


700
600
500
400
300
200
100
0
失真
2
4
6
8
10
集群数量

### 11.1.5 通过轮廓图量化聚类质量

评估聚类质量的另一个有内在联系的度量是轮廓分析，它也可以应用于k-均值以外的聚类算法，本章后面将讨论这些算法。轮廓分析可以作为图形工具来绘制度量集群中样本分组的紧密程度。可以用以下三个步骤来计算数据集中单个样本的轮廓系数：

1.计算集群的内聚度$a^{(i)}$，即样本$x^{(i)}$与集群内所有其他点之间的平均距离。

2.计算集群与最近集群的分离度$b^{(i)}$，即样本$x^{(i)}$与最近集群内所有样本之间的平均距离。

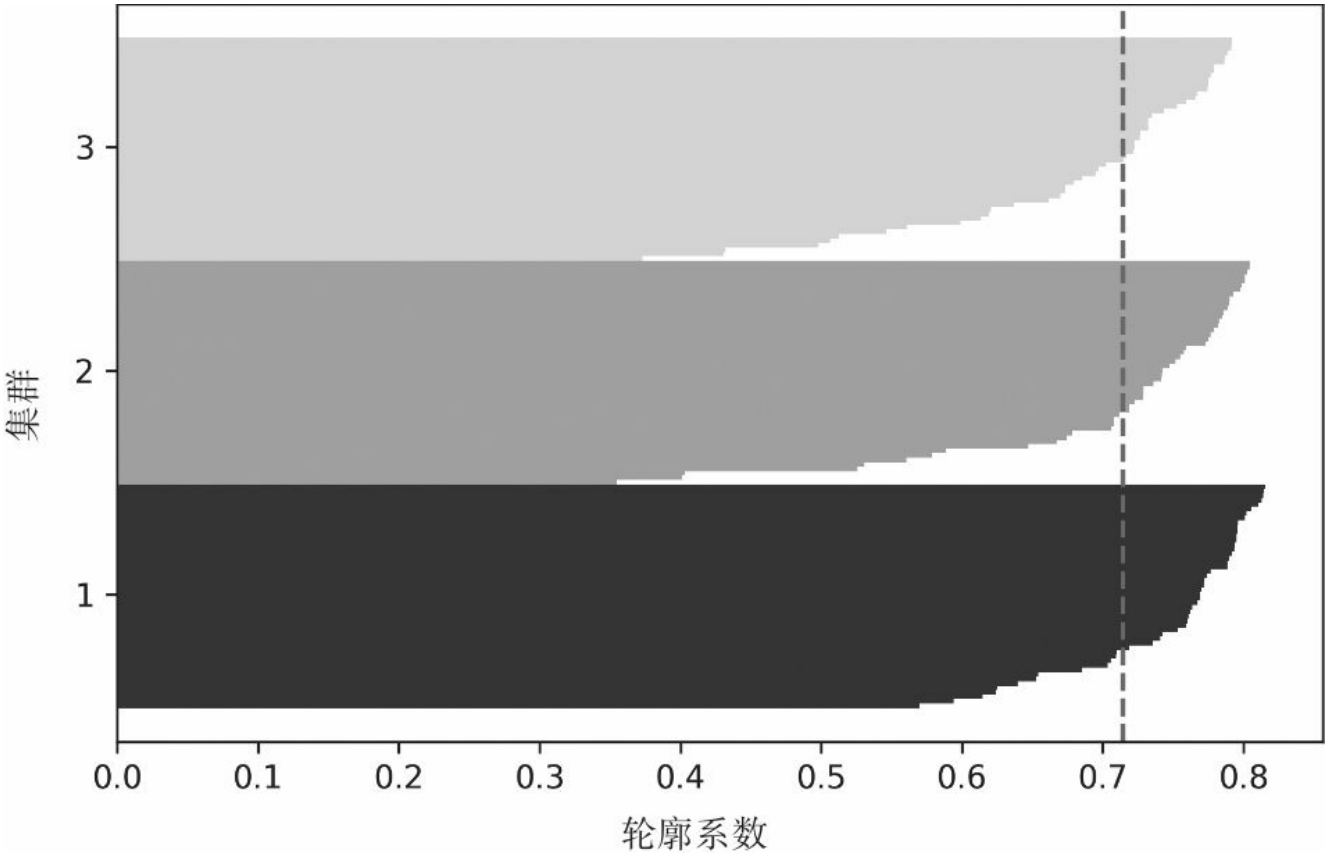
3.计算轮廓系数$s^{(i)}$，即集群内聚度与集群分离度之差，除以两者中较大的那一个，公式如下：

$$s^{(i)} = \frac{b^{(i)} - a^{(i)}}{\max\left\{b^{(i)}, a^{(i)}\right\}}$$

轮廓系数的范围在-1到1之间。如果集群分离度和集群内聚度相等，即$b^{(i)}=a^{(i)}$，那么从前面的方程可以看到轮廓系数为0。此外，如果$b^{(i)} >> a^{(i)}$，则接近理想的轮廓系数1，因为$b^{(i)}$量化该样本与其他集群样本的差异程度，而a（i）则告诉样本在自己的集群内与其他样本的相似程度。可以从scikit-learn的metric模块中找到silhouette_samples来计算轮廓系数，也可以很方便地导入silhouette_scores。silhouette_scores函数计算所有样本的平均轮廓系数，相当于numpy.mean（silhouette_samples（...））。执行下面的代码绘制一个基于k-均值方法（k=3）的聚类轮廓系数图：

```
>>> km = KMeans(n_clusters=3,
...             init='k-means++',
...             n_init=10,
...             max_iter=300,
...             tol=1e-04,
...             random_state=0)
>>> y_km = km.fit_predict(X)

>>> import numpy as np
>>> from matplotlib import cm
>>> from sklearn.metrics import silhouette_samples
>>> cluster_labels = np.unique(y_km)
>>> n_clusters = cluster_labels.shape[0]
>>> silhouette_vals = silhouette_samples(X,
...                                      y_km,
...                                      metric='euclidean')
>>> y_ax_lower, y_ax_upper = 0, 0
>>> yticks = []
>>> for i, c in enumerate(cluster_labels):
...     c_silhouette_vals = silhouette_vals[y_km == c]
...     c_silhouette_vals.sort()
...     y_ax_upper += len(c_silhouette_vals)
...     color = cm.jet(float(i) / n_clusters)
...     plt.barh(range(y_ax_lower, y_ax_upper),
...              c_silhouette_vals,
...              height=1.0,
...              edgecolor='none',
...              color=color)
...     yticks.append((y_ax_lower + y_ax_upper) / 2.)
...     y_ax_lower += len(c_silhouette_vals)
>>> silhouette_avg = np.mean(silhouette_vals)
>>> plt.axvline(silhouette_avg,
...             color="red",
...             linestyle="--")
>>> plt.yticks(yticks, cluster_labels + 1)
>>> plt.ylabel('Cluster')
>>> plt.xlabel('Silhouette coefficient')
>>> plt.show()
```

检查轮廓图可以快速确定不同集群的大小，并识别出包含离群值的集群：

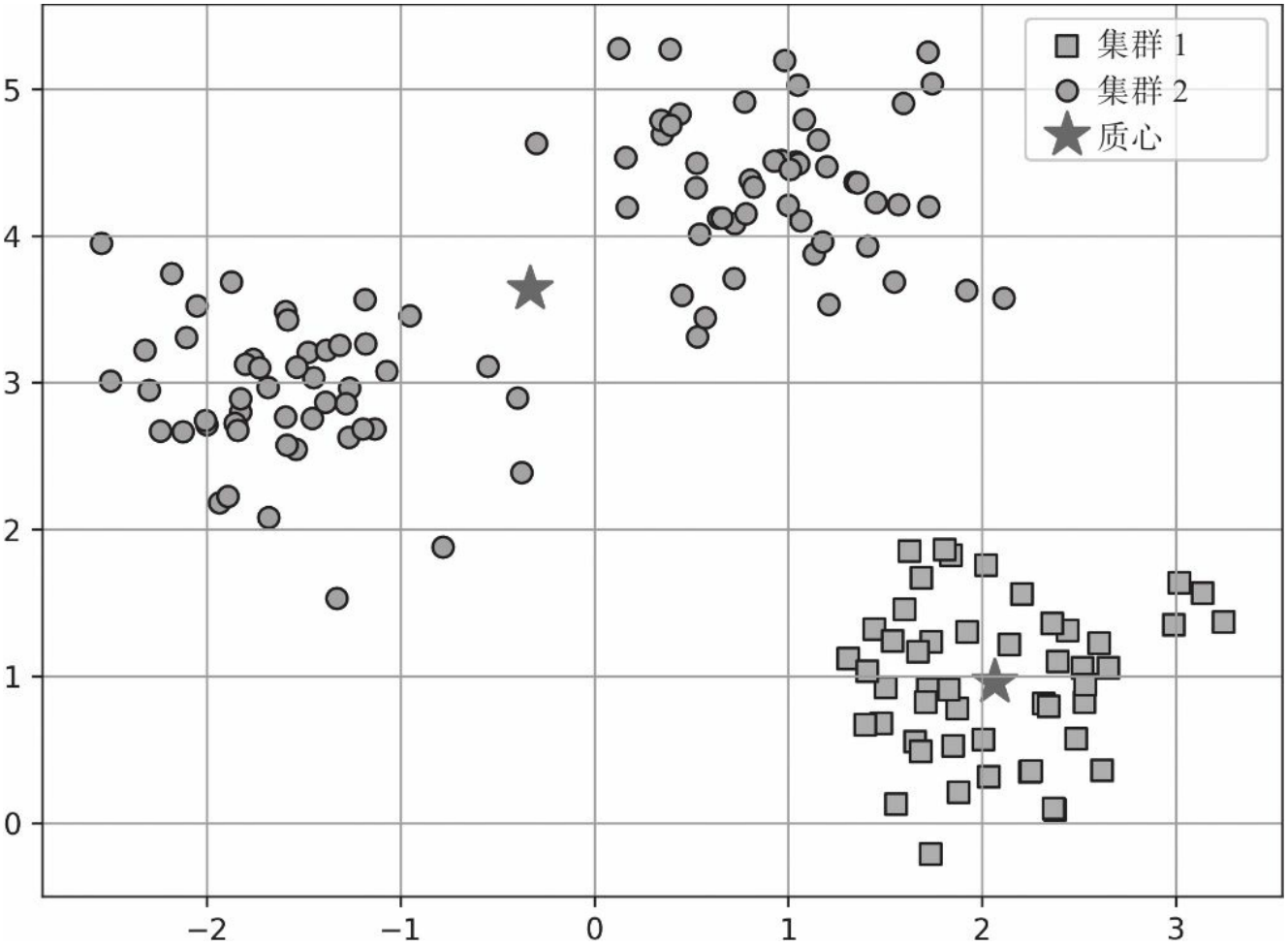



然而，正如前面的轮廓图中所看到的，轮廓系数甚至不接近于0，在这种情况表明聚类良好。此外，为了总结聚类的优点，我们在图中加上了平均轮廓系数（虚线）。

要想在轮廓图上看到较差聚类的具体情况，可以用只有两个重心的k-均值算法：

```
>>> km = KMeans(n_clusters=2,
...             init='k-means++',
...             n_init=10,
...             max_iter=300,
...             tol=1e-04,
...             random_state=0)
>>> y_km = km.fit_predict(X)

>>> plt.scatter(X[y_km==0,0],
...             X[y_km==0,1],
...             s=50, c='lightgreen',
...             edgecolor='black',
...             marker='s',
...             label='cluster 1')
>>> plt.scatter(X[y_km==1,0],
...             X[y_km==1,1],
...             s=50,
...             c='orange',
...             edgecolor='black',
...             marker='o',
...             label='cluster 2')
>>> plt.scatter(km.cluster_centers_[:,0],
...             km.cluster_centers_[:,1],
...             s=250,
...             marker='*',
...             c='red',
...             label='centroids')
>>> plt.legend()
>>> plt.grid()
>>> plt.show()
```

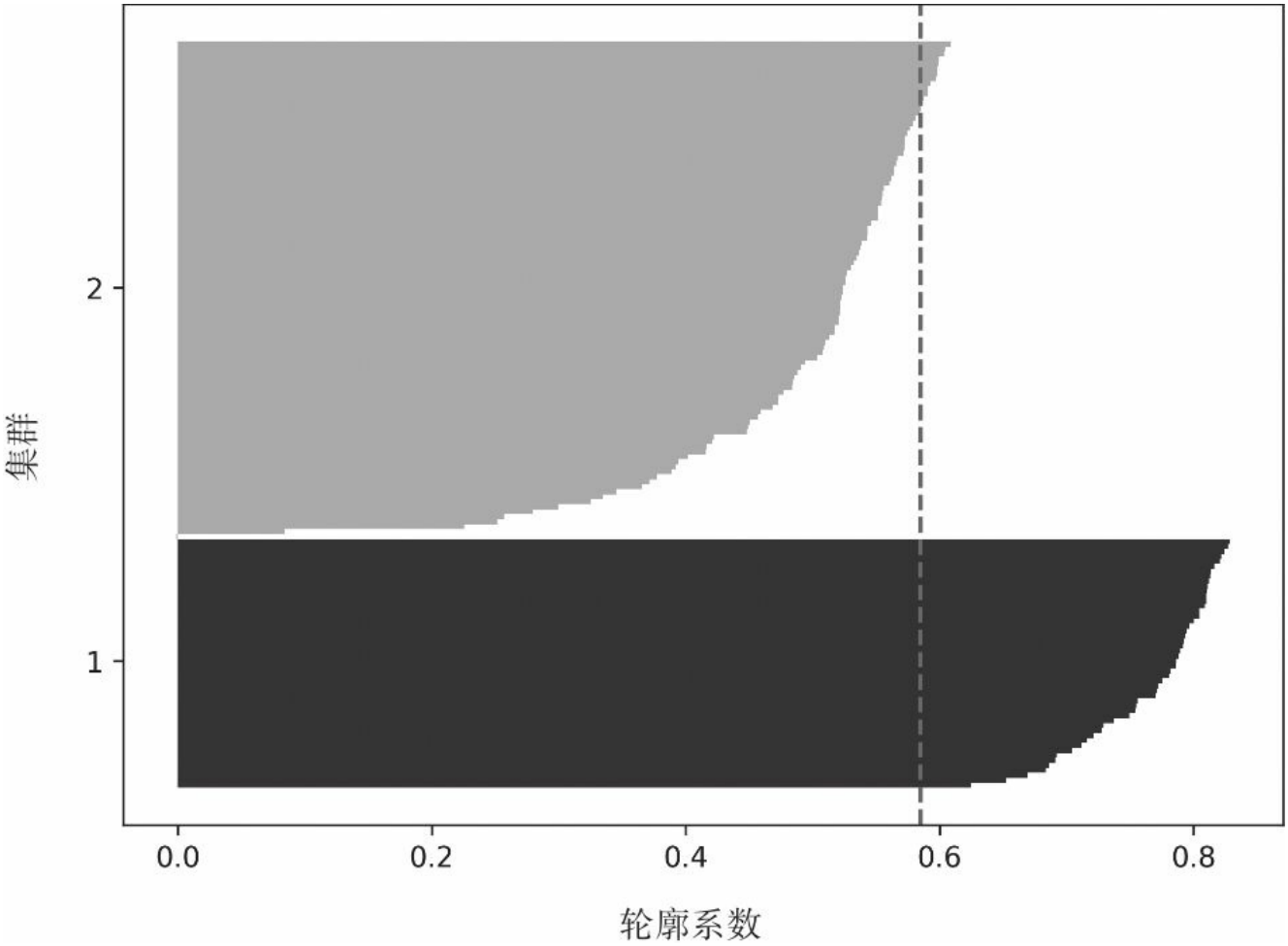
从结果图中可以看到，有一个重心落在三个样本空间中的两个球形区域之间。虽然聚类结果看起来并不是那么糟糕，但它不是最优的：




请记住，现实世界的问题通常没有条件把数据可视化在二维散点图上。因为通常模型在更高维的数据集上工作。接下来将创建轮廓图来评价结果：

```
>>> cluster_labels = np.unique(y_km)
>>> n_clusters = cluster_labels.shape[0]
>>> silhouette_vals = silhouette_samples(X,
...                                      y_km,
...                                      metric='euclidean')
>>> y_ax_lower, y_ax_upper = 0, 0
>>> yticks = []
>>> for i, c in enumerate(cluster_labels):
...     c_silhouette_vals = silhouette_vals[y_km == c]
...     c_silhouette_vals.sort()
...     y_ax_upper += len(c_silhouette_vals)
...     color = cm.jet(i / n_clusters)
...     plt.barh(range(y_ax_lower, y_ax_upper),
...              c_silhouette_vals,
...              height=1.0,
...              edgecolor='none',
...              color=color)
...     yticks.append((y_ax_lower + y_ax_upper) / 2)
...     y_ax_lower += len(c_silhouette_vals)
>>> silhouette_avg = np.mean(silhouette_vals)
>>> plt.axvline(silhouette_avg, color="red", linestyle="--")
>>> plt.yticks(yticks, cluster_labels + 1)
>>> plt.ylabel('Cluster')
>>> plt.xlabel('Silhouette coefficient')
>>> plt.show()
```

正如从结果图中所看到的，轮廓现在的长度和宽度有明显的不同，这是相对较差或至少不是最优聚类的证据：


集群
2
1
0.0
0.2
0.4
0.6
0.8
轮廓系数

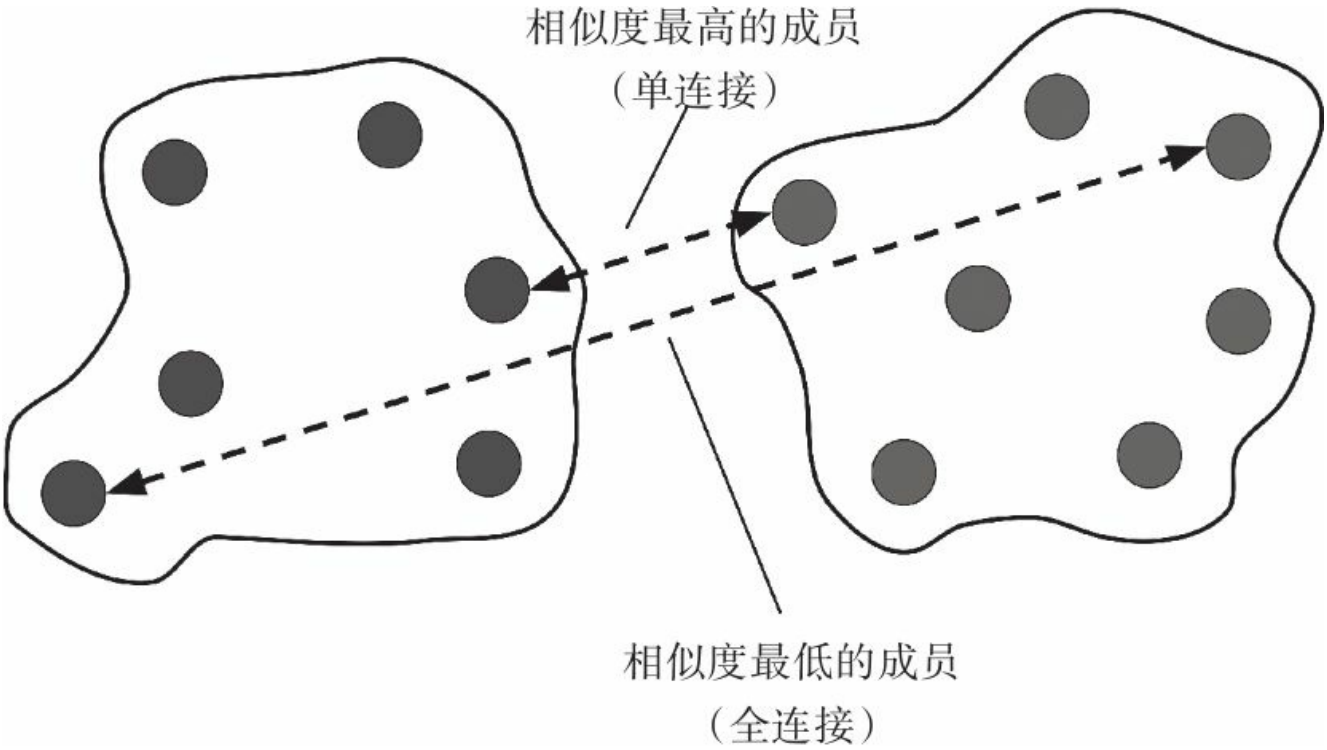
## 11.2 把集群组织成有层次的树

本节将研究另一种基于原型的聚类方法：层次聚类。层次聚类算法的优点是它允许绘制树状图（二进制层次聚类的可视化），这有助于解释创建有意义分类的结果。这种分层方法的另一个优点是不需要预先指定集群数目。

层次聚类的两种主要方法是凝聚和分裂。在分裂层次聚类中，首先从包含所有样本的集群开始，然后逐步迭代，将集群分裂成更小的集群，直到每个集群只包含一个样本为止。本节将重点讨论以相反方式进行的凝聚聚类。首先从每个集群包括单个样本开始，合并最接近的集群，直到只剩下一个集群为止。

### 11.2.1 以自下而上的方式聚类

凝聚层次聚类的两个标准算法分别是单连接和全连接。单连接算法计算两个集群中最相似成员之间的距离，然后合并两个集群，其中两个最相似成员之间的距离最小。全连接方法类似于单连接，但是，不是比较每对集群中最相似的成员，而是比较最不相似的成员然后合并。如下图所示：



凝聚层次聚类算法中其他常用的方法包括平均连接法和沃德连接法。平均连接法基于两个集群中所有组成员之间的最小平均距离来合并集群。沃德连接法合并引起群内总SSE增长最小的两个集群。

本节将聚焦用全连接的凝聚层次聚类算法。层次型全连接聚类是一个迭代过程，可以把具体步骤总结如下：

1.计算所有样本的距离矩阵。

2.将每个数据点表示为单例集群。

3.根据最不相同的（遥远的）成员之间的距离合并两个最近的集群。

4.更新相似度矩阵。

5.重复步骤2-4直到一个集群保持不变。

接下来将讨论如何计算距离矩阵（步骤1）。但先产生一些随机样本数据：行代表样本的不同观察（ID 0-4），列代表样本的不同特征（X，Y，Z）：

```
>>> import pandas as pd
>>> import numpy as np
>>> np.random.seed(123)
>>> variables = ['X', 'Y', 'Z']
>>> labels = ['ID_0','ID_1','ID_2','ID_3','ID_4']
>>> X = np.random.random_sample([5,3])*10
>>> df = pd.DataFrame(X, columns=variables, index=labels)
>>> df
```

执行前面的代码之后，我们现在将看到包含随机生成样本的以下数据框：

| | X | Y | Z |
|---|---|---|---|
| ID_0 | 6.964692 | 2.861393 | 2.268515 |
| ID_1 | 5.513148 | 7.194690 | 4.231065 |
| ID_2 | 9.807642 | 6.848297 | 4.809319 |
| ID_3 | 3.921175 | 3.431780 | 7.290497 |
| ID_4 | 4.385722 | 0.596779 | 3.980443 |

## 11.2.2 在距离矩阵上进行层次聚类

用SciPy的spatial.distance子模块的pdist函数计算距离矩阵作为层次聚类算法的输入：

```
>>> from scipy.spatial.distance import pdist, squareform
>>> row_dist = pd.DataFrame(squareform(
...             pdist(df, metric='euclidean')),
...             columns=labels, index=labels)
>>> row_dist
```

用前面的代码，根据特征X，Y和Z计算数据集中每对样本点之间的欧氏距离，提供pdist返回的简明距离矩阵作为squareform函数的输入，创建一个成对距离的对称矩阵，如下所示：

| | X | Y | Z |
|---|---|---|---|
| ID_0 | 6.964692 | 2.861393 | 2.268515 |
| ID_1 | 5.513148 | 7.194690 | 4.231065 |
| ID_2 | 9.807642 | 6.848297 | 4.809319 |
| ID_3 | 3.921175 | 3.431780 | 7.290497 |
| ID_4 | 4.385722 | 0.596779 | 3.980443 |

接下来将通过调用SciPy中的cluster.hiera-rchy子模块的linkage函数来应用全连接凝聚方法处理集群，处理结果将返回在所谓的连接矩阵中。

但是，在调用linkage函数之前，先仔细看看该函数的文档：

```
>>> from scipy.cluster.hierarchy import linkage
>>> help(linkage)
```

```
    [...]
    Parameters:
      y : ndarray
        A condensed or redundant distance matrix. A condensed
        distance matrix is a flat array containing the upper
        triangular of the distance matrix. This is the form
        that pdist returns. Alternatively, a collection of m
        observation vectors in n dimensions may be passed as
        an m by n array.

      method : str, optional
        The linkage algorithm to use. See the Linkage Methods
        section below for full descriptions.

      metric : str, optional
        The distance metric to use. See the distance.pdist
        function for a list of valid distance metrics.
    Returns:
      Z : ndarray
        The hierarchical clustering encoded as a linkage matrix.
    [...]
```

根据对该函数的描述得出结论，即可以用从pdist函数返回的简明距离矩阵（上三角）作为输入的属性。或者提供初始数据阵列，并用“eulidean”度量作为linkage函数的参数。然而，不应该用先前定义的squareform距离矩阵，因为这会产生与预期值不同的距离。概括起来，这里列出了三种可能的场景：

·不正确的方法：用下面代码片段所显示的squareform距离矩阵会带来不正确的结果：

```
>>> from scipy.cluster.hierarchy import linkage
>>> row_clusters = linkage(row_dist,
...                        method='complete',
...                        metric='euclidean')
```

·正确的方法：用下面代码示例所示的简明距离矩阵会产生正确的成对距离矩阵：

```
>>> row_clusters = linkage(pdist(df, metric='euclidean'),
...                        method='complete')
```

·正确的方法：用下面代码片段所示的完整输入样本矩阵会产生与前面方法相似的正确距离矩阵：

```
>>> row_clusters = linkage(df.values,
...                         method='complete',
...                         metric='euclidean')
```

为了能仔细观察聚类结果，可以把聚类结果转换成pandas的DataFrame（最好在Jupyter笔记本上看），具体代码如下：

```
>>> pd.DataFrame(row_clusters,
...       columns=['row label 1',
...                'row label 2',
...                'distance',
...                'no. of items in clust.'],
...       index=['cluster %d' %(i+1) for i in
...              range(row_clusters.shape[0])])
```

如下面的截图所示，连接矩阵由若干行组成，其中每行代表一个合并。第一列和第二列代表每个集群中相似度最低的成员，第三列报告这些成员之间的距离。最后一列返回每个集群成员的个数。

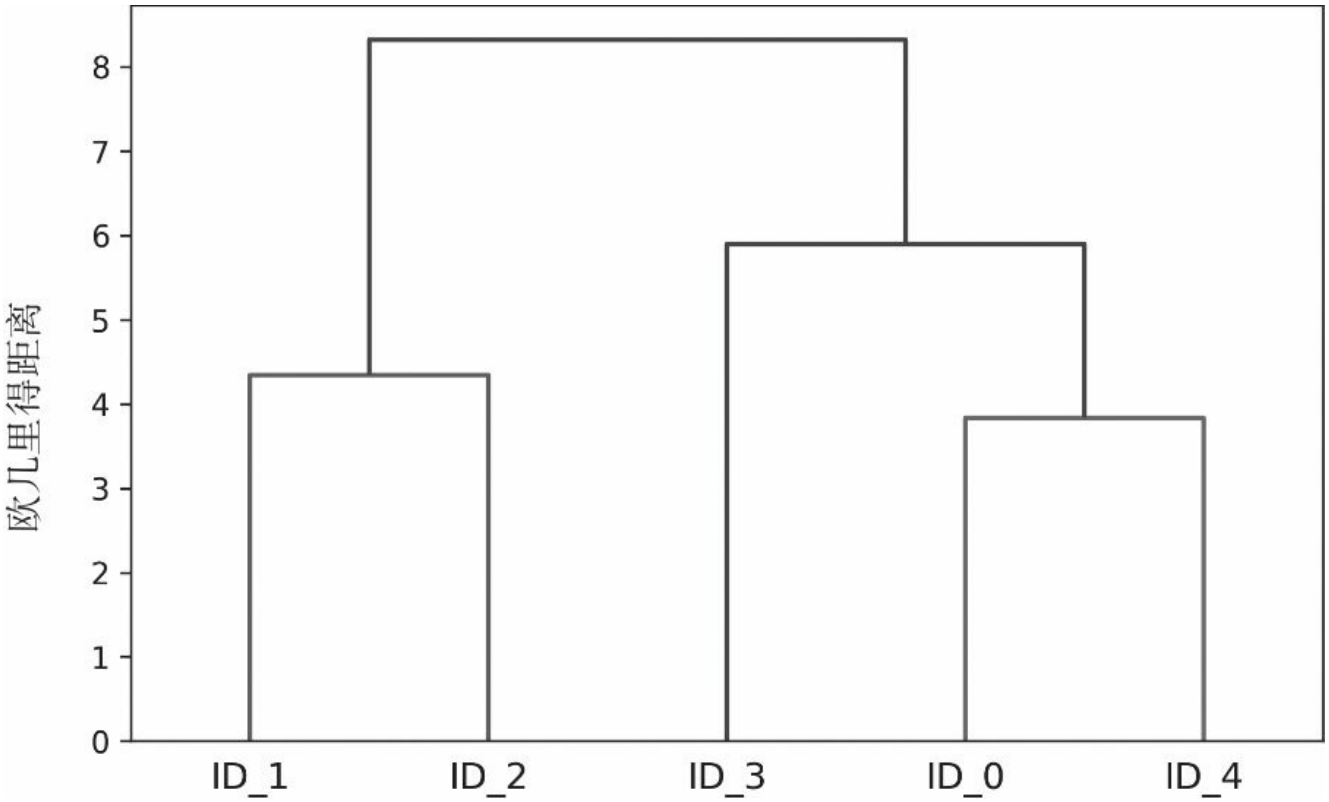
| | row label 1 | row label 2 | distance | no. of items in clust. |
|---|---|---|---|---|
| cluster 1 | 0.0 | 4.0 | 3.835396 | 2.0 |
| cluster 2 | 1.0 | 2.0 | 4.347073 | 2.0 |
| cluster 3 | 3.0 | 5.0 | 5.899885 | 3.0 |
| cluster 4 | 6.0 | 7.0 | 8.316594 | 5.0 |

现在已经计算好了连接矩阵，接着可以用树状图显示结果：

```
>>> from scipy.cluster.hierarchy import dendrogram
# make dendrogram black (part 1/2)
# from scipy.cluster.hierarchy import set_link_color_palette
# set_link_color_palette(['black'])
>>> row_dendr = dendrogram(row_clusters,
...                        labels=labels,
...                        # make dendrogram black (part 2/2)
...                        # color_threshold=np.inf
...                        )
>>> plt.tight_layout()
>>> plt.ylabel('Euclidean distance')
>>> plt.show()
```

如果你在执行上述代码或者在读本书的电子版，会发现在结果的树状图

中不同的分支以不同的颜色显示。颜色安排来自于Matplotlib的颜色列表，根据树的距离阈值分配不同颜色。例如，要显示黑色树状图，可以把前面代码相应部分的注释去掉：




这样的树状图概括了在凝聚层次聚类中形成的不同集群。例如，可以看到样本ID_0和ID_4，接着是ID_1和ID_2，它们在欧几里得距离度量上最为相似。

### 11.2.3 热度图附加树状图

在实际应用中，层次聚类树状图通常与热度图结合使用，使我们能够用颜色代码代表样本矩阵个体的数值。本节将讨论如何把树状图附加到热度图，并对热度图中对应的行排序。

然而，把树状图附加到热度图上可能有点儿复杂，下面分步完成这个过程：

1.创建一个新的图形对象，并通过add_axes函数的属性，定义x轴和y轴的位置以及树状图的宽度和高度。此外，逆时针旋转树状图90度。代码如下：

```
>>> fig = plt.figure(figsize=(8,8), facecolor='white')
>>> axd = fig.add_axes([0.09,0.1,0.2,0.6])
>>> row_dendr = dendrogram(row_clusters, orientation='left')
>>> # note: for matplotlib < v1.5.1, please use
orientation='right'
```

2.接着对在最初DataFrame中的数据按照树状图对象可以访问的聚类标签排序，本质上这是一个以leaves为关键词的Python字典。代码如下：

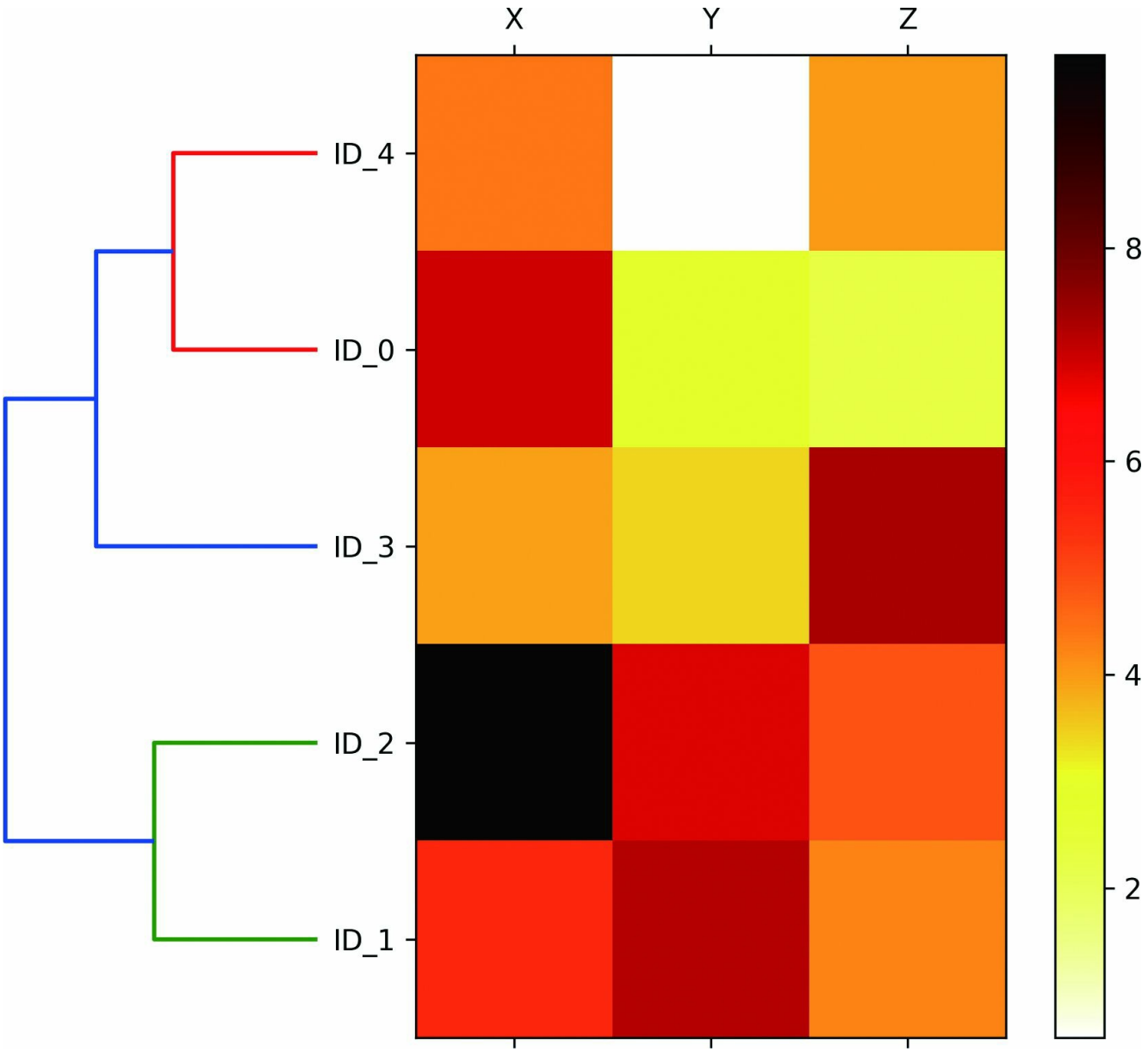
```
>>> df_rowclust = df.iloc[row_dendr['leaves'][::-1]]
```

3.现在根据排序后的DataFrame构建热度图，并将其放在树状图的旁边：

```
>>> axm = fig.add_axes([0.23,0.1,0.6,0.6])
>>> cax = axm.matshow(df_rowclust,
...                   interpolation='nearest', cmap='hot_r')
```

4.最后通过去除坐标轴的间隔标记和隐藏坐标轴的轴线来美化修树状图。另外，添加颜色条，并将特征和样本名称分别标注在x轴和y轴的刻度标签上：

```
>>> axd.set_xticks([])
>>> axd.set_yticks([])
>>> for i in axd.spines.values():
...     i.set_visible(False)
>>> fig.colorbar(cax)
>>> axm.set_xticklabels(['']  + list(df_rowclust.columns))
>>> axm.set_yticklabels(['']  + list(df_rowclust.index))
>>> plt.show()
```




完成前述步骤之后，热度图以及旁边附带的树状图就应该展示出来了：

可以看到热度图中行的顺序反映了树状图中样本的聚类情况。除了简单的树状图，在热度图中每个样本的颜色编码和特征为数据集做了很好的概括。

## 11.2.4 scikit-learn凝聚聚类方法

上一小节看到如何用SciPy进行凝聚层次聚类。然而scikit-learn实现的Agglo-merative-Clustering允许我们选择要返回的集群数量。这对修剪层次结构的集群树很有用。通过设置参数n_cluster为3，现在可以采用相同的全连接方法基于欧几里得距离度量将样本聚集成三个集群，代码如下：

```
>>> from sklearn.cluster import AgglomerativeClustering
>>> ac = AgglomerativeClustering(n_clusters=3,
...                              affinity='euclidean',
...                              linkage='complete')
>>> labels = ac.fit_predict(X)
>>> print('Cluster labels: %s' % labels)
Cluster labels: [1 0 0 2 1]
```

从预测的聚类标签可以看到，第一个和第五个样本（ID_0和ID_4）被分配到一个集群（标签1），样本ID_1和ID_2被分配到第二集群（标签0）。样本ID_3被放入自己的集群（标签2）。总体而言，该结果与我们从树状图中所观察到的一致。需要注意的是ID_3与ID_4以及ID_0的相似度比它与ID_1和ID_2的相似度更高，如前面的树状图所示。这在scikit-learn的聚类结果中并不明显。现在在n_cluster=2的条件下重新运行AgglomerativeClustering，见下面的代码片段：

```
>>> ac = AgglomerativeClustering(n_clusters=2,
...                              affinity='euclidean',
...                              linkage='complete')
>>> labels = ac.fit_predict(X)
>>> print('Cluster labels: %s' % labels)
Cluster labels: [0 1 1 0 0]
```

正如所看到的，在这个修剪过的层次聚类过程中，正如所预期的那样，标签ID_3没有被分到ID_0和ID_4相同的集群中。

## 11.3 通过DBSCAN定位高密度区域

虽然本章不能覆盖大量不同的聚类算法，让我们至少再引入一种新的聚类方法，即基于密度的有噪声的空间聚类应用（DBSCAN），不像k-均值方法那样假设集群呈球形，或是把数据集分成不同的需要人工设定分界点的层级。顾名思义，基于密度的聚类把标签分配给样本点密集的区域。DBSCAN的密度定义为在指定半径ε范围内的点数。采用DBSCAN算法根据下列标准把特殊的标签分配给每个样本点：

·如果有指定数量（MinPts）的相邻点落在以该点为圆心的指定半径ε范围内，则该点为核心点。

·在核心点半径ε范围内，相邻点比半径ε范围内的MinPts少，则该点叫边界点。

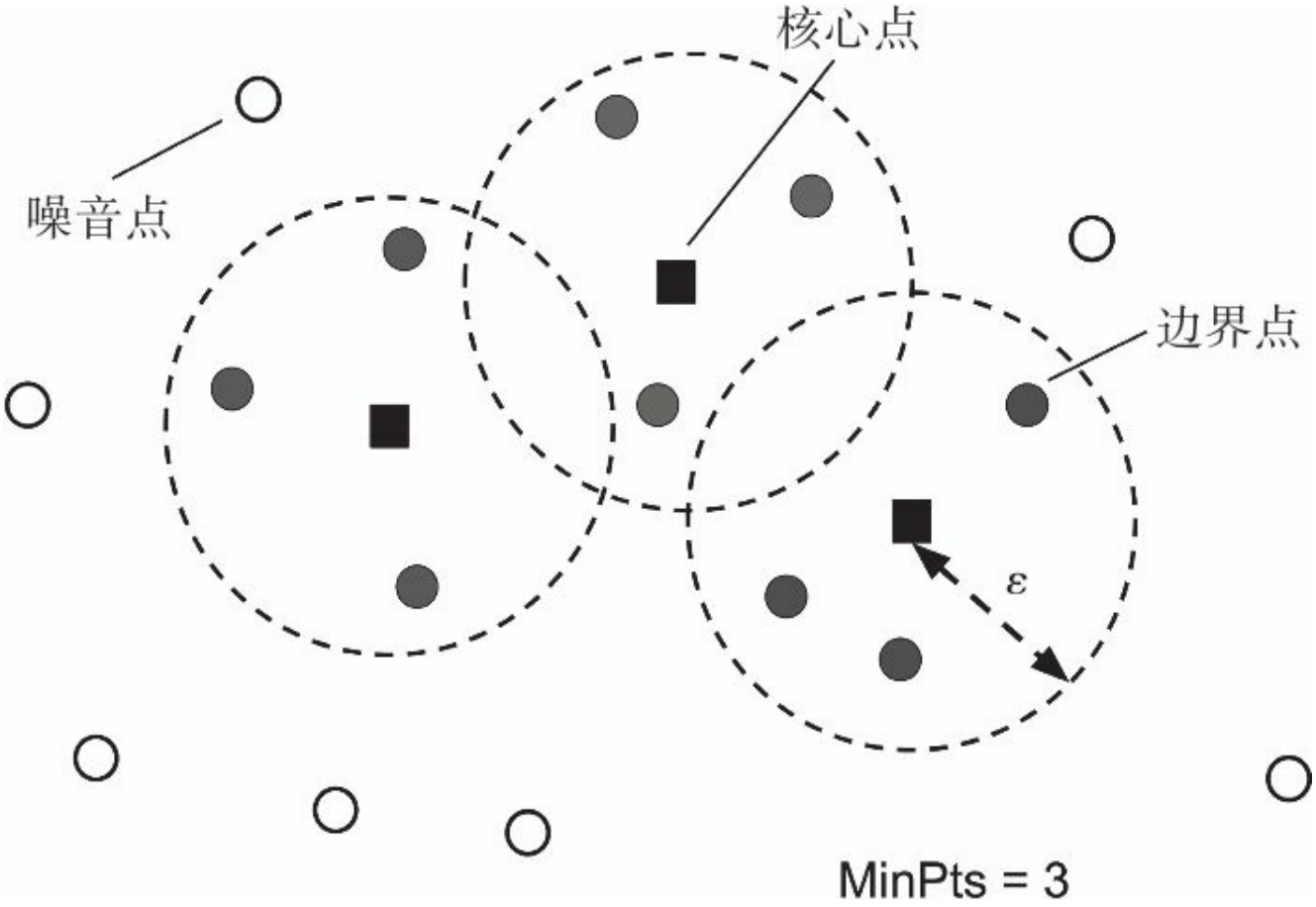
·所有既不是核心点也不是边界点的其他点被认为是噪声点。

在把所有的点分别标示为核心点、边界点和噪声点之后。可以用两步来概括DBSCAN算法：

1.用每个核心点或连接的核心点组成单独的集群（如果不超过ε的范围，核心点被视为连接的）。

2.把每个边界点分配到与其核心点相对应的集群。

为了更好地了解DBSCAN算法的聚类结果，在实现该算法之前，先从下面的图总结刚才所学习到的核心点、边界点和噪声点：

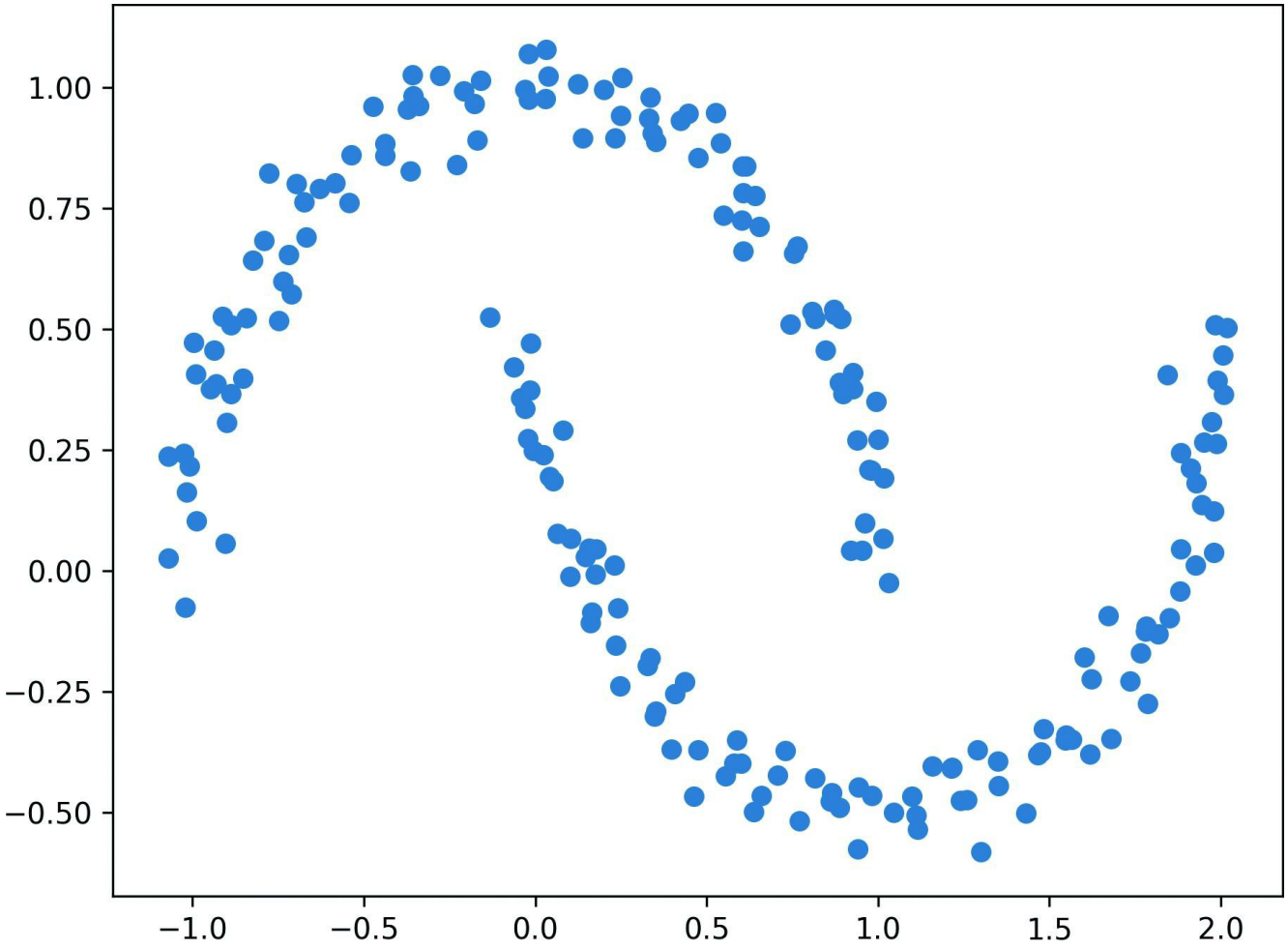



使用DBSCAN算法的主要优点是，它不像k-均值那样假设集群呈球形分布。此外，DBSCAN不同于k-均值和层次聚类的是它不一定把每个点都分配到集群中去，但是能够去除噪声点。

举一个更具有说明性的例子，创建新的半月形结构数据集来比较k-均值聚类、层次聚类和DBSCAN：

```
>>> from sklearn.datasets import make_moons
>>> X, y = make_moons(n_samples=200,
...                   noise=0.05,
...                   random_state=0)
>>> plt.scatter(X[:,0], X[:,1])
>>> plt.show()
```

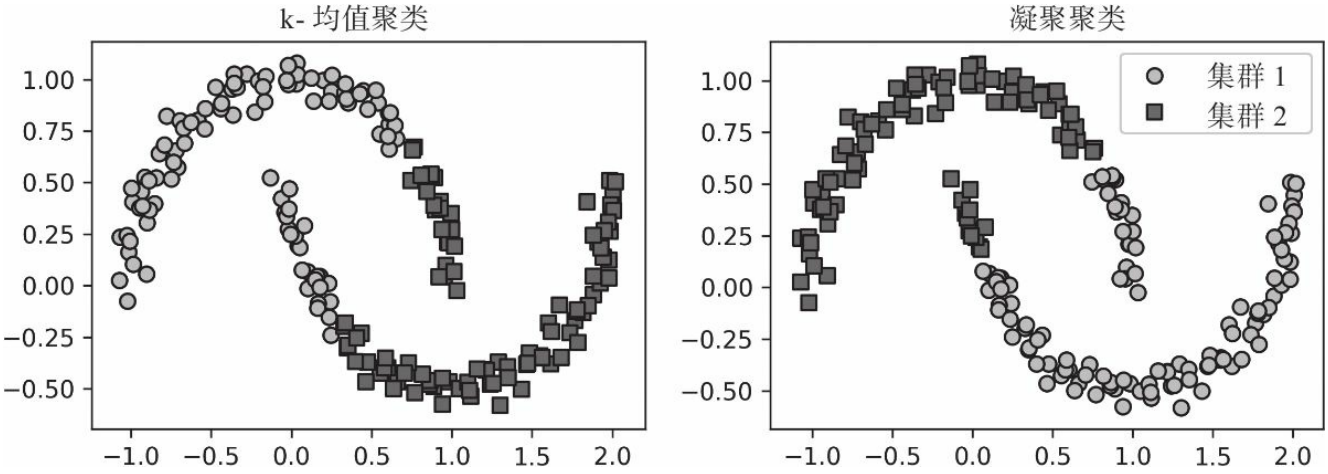
正如从结果图中可以看到的，有两个半月形的集群，分别包含100个样本点：




从使用k-均值算法和全连接聚类开始，来看一下前面讨论过的那些聚类算法是否能成功地把半月形集群彼此分开。代码如下：

```
>>> f, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 3))
>>> km = KMeans(n_clusters=2,
...             random_state=0)
>>> y_km = km.fit_predict(X)
>>> ax1.scatter(X[y_km==0,0],
...             X[y_km==0,1],
...             c='lightblue',
...             edgecolor='black',
...             marker='o',
...             s=40,
...             label='cluster 1')
>>> ax1.scatter(X[y_km==1,0],
...             X[y_km==1,1],
...             c='red',
...             edgecolor='black',
...             marker='s',
...             s=40,
...             label='cluster 2')
>>> ax1.set_title('K-means clustering')
>>> ac = AgglomerativeClustering(n_clusters=2,
...                              affinity='euclidean',
...                              linkage='complete')
>>> y_ac = ac.fit_predict(X)
>>> ax2.scatter(X[y_ac==0,0],
...             X[y_ac==0,1],
...             c='lightblue',
...             edgecolor='black',
...             marker='o',
...             s=40,
...             label='cluster 1')
>>> ax2.scatter(X[y_ac==1,0],
...             X[y_ac==1,1],
...             c='red',
...             edgecolor='black',
...             marker='s',
...             s=40,
...             label='cluster 2')
>>> ax2.set_title('Agglomerative clustering')
>>> plt.legend()
>>> plt.show()
```

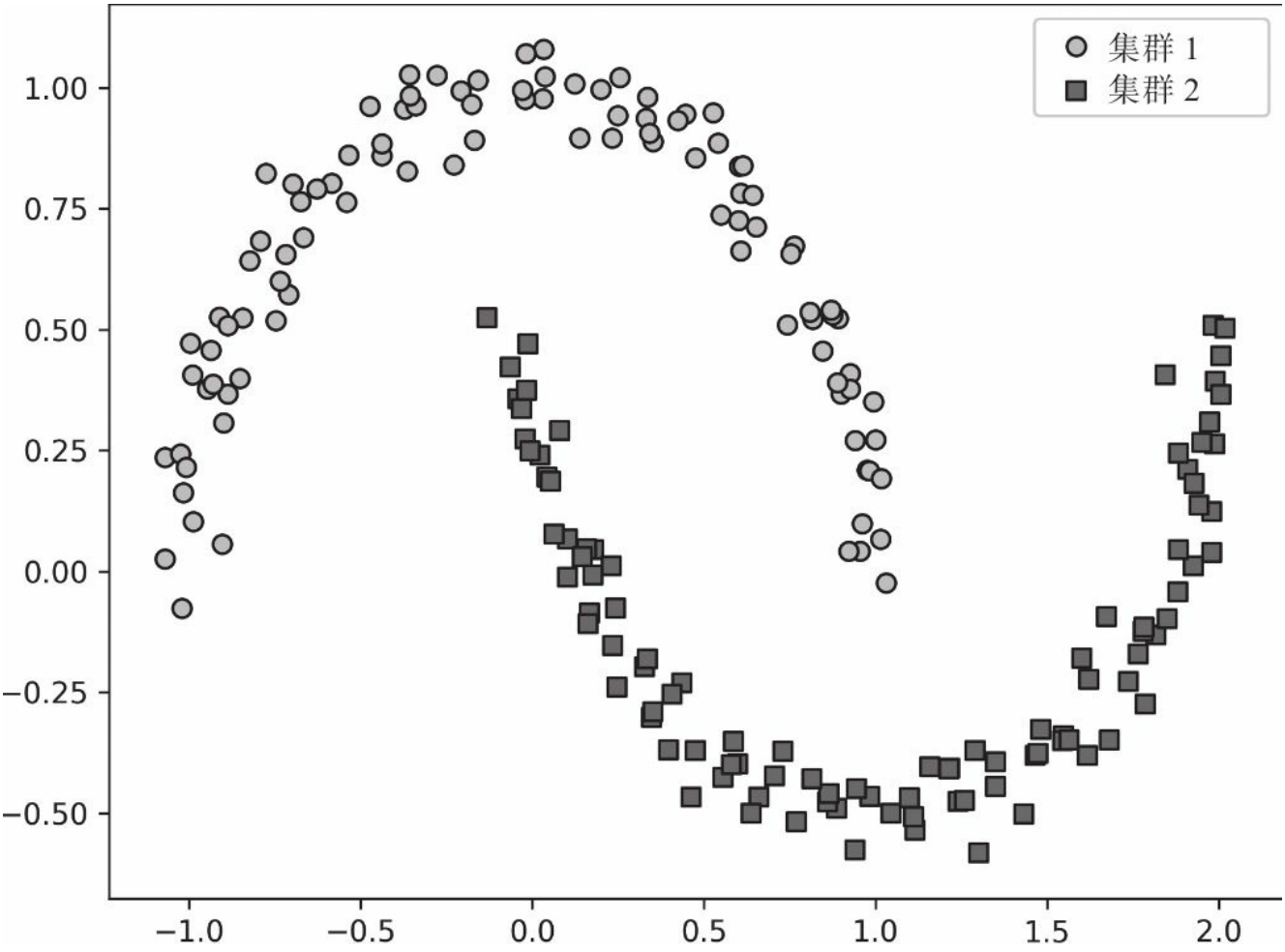
从可视化聚类结果可以看到k-均值算法不能在两个集群之间分出彼此，层次聚类算法也遇到了复杂形状的挑战：




最后，再尝试采用DBSCAN算法，看它是否能用基于密度的方法分辨出两个集群：

```
>>> from sklearn.cluster import DBSCAN
>>> db = DBSCAN(eps=0.2,
...             min_samples=5,
...             metric='euclidean')
>>> y_db = db.fit_predict(X)
>>> plt.scatter(X[y_db==0,0],
...             X[y_db==0,1],
...             c='lightblue',
...             edgecolor='black',
...             marker='o',
...             s=40,
...             label='cluster 1')
>>> plt.scatter(X[y_db==1,0],
...             X[y_db==1,1],
...             c='red',
...             edgecolor='black',
...             marker='s',
...             s=40,
...             label='cluster 2')
>>> plt.legend()
>>> plt.show()
```

DBSCAN算法可以成功地检测出半月形，这凸显出DBSCAN算法的优势——可以完成任意形状的数据聚类：




然而，应当留意DBSCAN的一些缺点。假定训练样本的数量保持不变，如果数据集的特征数目增加，维数诅咒的副作用就会增大。特别是在使用欧几里得距离度量时会出现问题。然而，维数诅咒问题并非只针对DBSCAN，它也影响使用欧几里得距离度量的其他聚类算法，例如k-均值和层次聚类算法。另外，需要优化DBSCAN的两个超参数（MinPts和ε）以获得良好的聚类结果。如果数据集的密度差别相对较大，找到MinPts与ε的优化组合可能会比较困难。

到目前为止已经看到了三种最基本的聚类算法：基于原型的k-均值聚类，凝聚层次聚类以及基于密度的DBSCAN聚类。然而，我想提出本章未讨论过的更加高级的第四种聚类算法，即基于图形的聚类。基于图形的聚类算法家族中最为著名的成员很可能是种群聚类。尽管这种算法有许多不同的实现，但是这些实现的共同点是使用相似度特征向量，或者距离矩阵来推导集群之间的关系。因为种群聚类超出了本书的范围，你可以通过学习乌利克·冯·拉申博格准备的优秀教程，了解更多关于该主题的内容（乌利克·冯·拉申博格.《种群聚类教程》.统计学与计算，2007，17（4）：395–416）。可以免费从arXiv网站获得：http://arxiv.org/pdf/0711.0189v1.pdf。

请注意，在实践中，哪种算法在给定的数据集上表现最佳并不总是很明显，特别是当数据中有很多维度使其很难甚至无法完成可视化时。另外，成

功的聚类不仅仅取决于算法及其超参数。相反，合适距离度量的选择和有助于引导实验性的设置的领域知识的使用甚至更为重要。

因此，在维数诅咒的背景下，在进行聚类之前应用降维技术很常见，对无监督学习数据集而言，这类降维技术包括在第5章中讨论过的主成分分析和RBF核主成分分析。另外，把数据集压缩到只有两个维度的情况特别常见，这使我们可以用二维散点图来进行聚类和分配标签，对评估聚类结果特别有帮助。

## 11.4 小结

本章学习了三种不同的聚类算法，这有助于发现隐藏在数据中的结构或者信息。从介绍基于原型的k-均值算法开始，该算法基于定义的集群中心点，把样本聚集成球形。因为聚类属于无监督学习方法，所以无法依靠标定清楚的分类标签来评估模型性能。因此，采用有内在联系的性能指标，如肘法或者轮廓分析法作为试图量化聚类质量的工具。

接着讨论不同的聚类方法，即凝聚层次聚类。层次聚类不要求预先定义集群的数量，而且可以通过树状图完成结果的可视化，这有助于解释聚类的结果。本章最后看到的聚类算法是DBSCAN，该算法基于本地样本点的密度来聚集，能够处理离群点并发现非球状分布的集群。

在进入无监督学习领域之后，现在是时候为有监督学习引入一些最令人兴奋的机器学习算法：多层人工神经网络。神经网络在最近复兴之后，再次成为机器学习研究中最热门的话题。由于最近开发的深层学习算法，神经网络被认为是可以完成许多像图像分类和语音识别等复杂任务的最先进的工具。第12章将从头开始构建本书自己的多层神经网络。第13章将介绍可以最有效地帮助我们训练复杂网络体系结构的强大软件库。

# 第12章　从零开始实现多层人工神经网络

正如你所知道的那样，深度学习正受到媒体的广泛关注，毫无疑问是机器学习领域中最热门的话题。可以把深度学习理解为一组专门研发来最有效地训练多层人工神经网络的算法。本章将学习人工神经网络的基本概念，为接下来的章节做好准备，下面的章节将介绍先进的基于Python的深度学习库和深层神经网络（DNN）体系，它们特别适合图像和文本分析。

本章将主要涵盖下述几个方面：

·获得对多层神经网络的概念性理解

·从零开始实现神经网络训练的基本反向传播算法

·训练基本的多层神经网络并用于图像分类

## 12.1 用人工神经网络为复杂函数建模

本书从第2章开始了人工神经元机器学习算法的旅程。人工神经元是本章将要讨论的多层人工神经网络的组成部分。人工神经网络背后的基本概念建立在人脑如何完成解决复杂问题的假设和模型之上。虽然近几年人工神经网络已经获得了广泛的普及，早期的神经网络研究要追溯到20世纪40年代，沃伦·麦卡洛克和沃尔特·皮特在那个时候第一次描述了神经元的工作原理。

然而，在麦卡洛克——皮特神经元模型作为20世纪50年代的罗森布拉特的感知器首次实施后的几十年里，因为对多层神经网络的训练没有更好的解决方案，许多研究人员和机器学习实践者慢慢开始对神经网络失去了兴趣。对神经网络的兴趣最终在1986年被重新点燃，大卫·E.鲁姆哈特、杰弗里·E.欣顿和罗纳德·J.威廉姆斯参与了重新发现和推广更有效地训练神经网络的反向传播算法，本章的后面将对此做更详细的讨论（大卫·E.鲁姆哈特，杰弗里·E.欣顿，罗纳德·J.威廉姆斯.《通过反向传播误差学习表示》.自然，1986，323（6088）：533–536）。我们也鼓励对人工智能（AI）、机器学习和神经网络的历史感兴趣的读者阅读维基百科关于AI冬天的文章，在这段时间里，大部分的研究社区对神经网络失去了兴趣（https://en.wikipedia.org/wiki/AI_winter）。

然而，过去十年中取得的许多重大突破使神经网络达到了史无前例的受欢迎程度，这也带来了现在称之为深度学习的算法和体系，即由许多层组成的神经网络。神经网络不仅是学术研究的热门话题，也是脸谱、微软和谷歌等大型科技公司的热门话题，他们在人工神经网络和深度学习的研究方面投入了大量资源。时至今日，以深度学习算法为动力的复杂神经网络，在诸如图像和语音识别等复杂问题的求解中被认为是最先进的技术。在日常生活中，常见的以深度学习驱动的产品案例是谷歌图片搜索和翻译，这些智能手机应用可以自动识别图像中的文本并实时翻译成20多种语言。

很多令人兴奋的深度神经网络（DNN）应用已经在主要的科技公司和医药行业研发，以下列出的是不完全的案例清单：

·脸谱研发的DeepFace专门为图像打标签（Y.泰格曼，M.杨，M.兰扎托，L.沃尔夫.《DeepFace：缩小人脸验证与人工性能水平的差距》.IEEE国际会议《计算机视觉与模式识别》（CVPR），2014：1701–1708页）。

·百度的DeepSpeech能够处理普通话的语音查询（A.韩南，C.凯斯，J.卡斯珀，B.卡坦扎罗，G.戴莫斯，E.埃尔森，R.普伦格，S.沙得士，S.森古普塔，A.科茨，安德鲁.Y.梁.《DeepSpeech：端到端的语音识别扩展》.预印本arXiv：1412.5567，2014）。

·谷歌的新语言翻译服务（《谷歌的神经机器翻译系统：缩小人与机器

翻译的差距》.预印本arXiv：1412.5567，2016）。

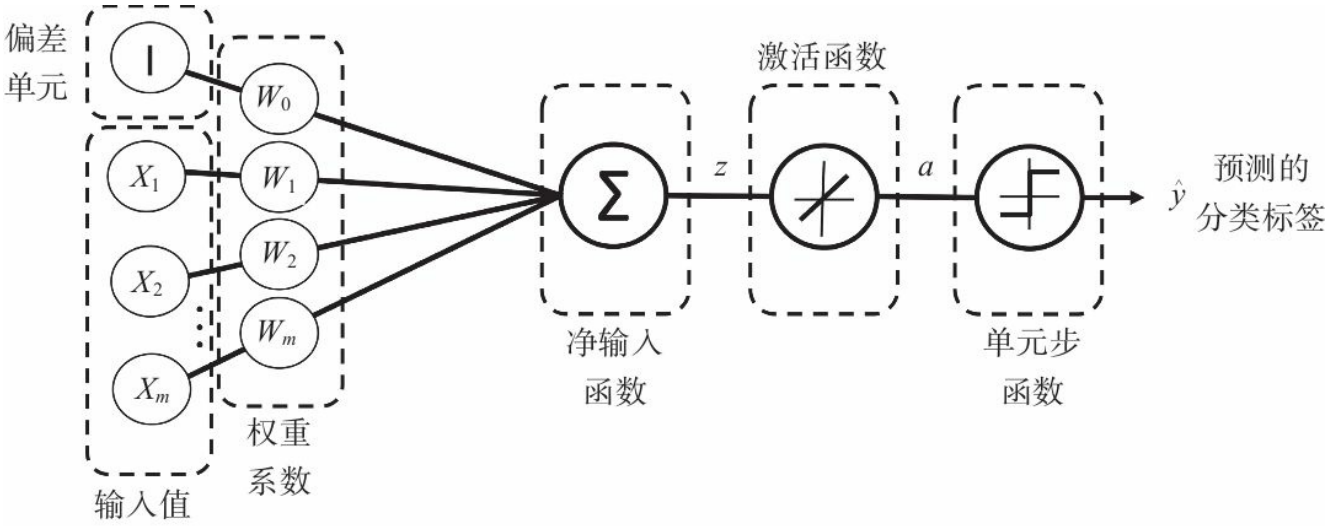
·Novel的新药发现和毒性预测技术（T.昂特信内，A.迈尔，G.克拉姆宝和S.霍克赖特.《深度学习毒性预测》.arXiv预印本arXiv：1503.01445，2015）。

·检测皮肤癌的移动应用，其准确度相当于受过专业训练的皮肤科医生（A.爱斯特瓦，B.库普里，R.A.诺沃亚，J.考，S.M.斯威特，H.M.布劳和S.特龙.《用深度神经网络对皮肤癌进行皮肤科医生水平的分级》.自然542，第7639，2017：115-118页）。

## 12.1.1 单层神经网络扼要重述

本章讨论多层神经网络的工作原理，以及如何训练它们来解决复杂的问题。然而，在深入挖掘特定的多层神经网络体系之前，先简单地回顾第2章简单介绍过的一些单层神经网络概念，即自适应线性神经元算法（Adaline），如下图所示：

第2章实现了用于分类的自适应算法Adaline，并采用梯度下降算法来学习模型的权重系数。用以下的更新规则在训练集的每个迭代更新权重向量w：




$$\boldsymbol{w} := \boldsymbol{w} + \Delta\boldsymbol{w}, \quad 其中\ \Delta\boldsymbol{w} = -\eta\nabla J(\boldsymbol{w})$$

换句话说计算基于整个训练集的梯度，并通过向梯度相反的方向退一步来更新模型的权重。优化定义为误差平方和（SSE）的目标函数J（w）以找到该模型的最优权重。此外把梯度乘以学习率η，必须慎重选择该参数以平衡学习速度与全局最小成本函数过度所带来的风险。

在梯度下降的优化过程中，每次迭代都同时更新所有的权重。为权重向量w的每个权重元素$w_j$定义的偏导数如下：

$$\frac{\partial}{\partial w_j} J(\boldsymbol{w}) = -\sum_i \left( y^{(i)} - a^{(i)} \right) x_j^{(i)}$$

这里$y^{(i)}$为特定样本$x^{(i)}$的目标分类标签，$a^{(i)}$为神经元的激发值是Adaline特例的线性函数。另外定义激发函数φ（·）如下：

$$\phi(z) = z = a$$

这里净输入z是连接输入层到输出层权重的线性组合：

$$z = \sum_j w_j x_j = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x}$$

用活化值φ（z）计算梯度更新，通过实现的阈值函数把连续值预测输出转换为二进制分类标签：

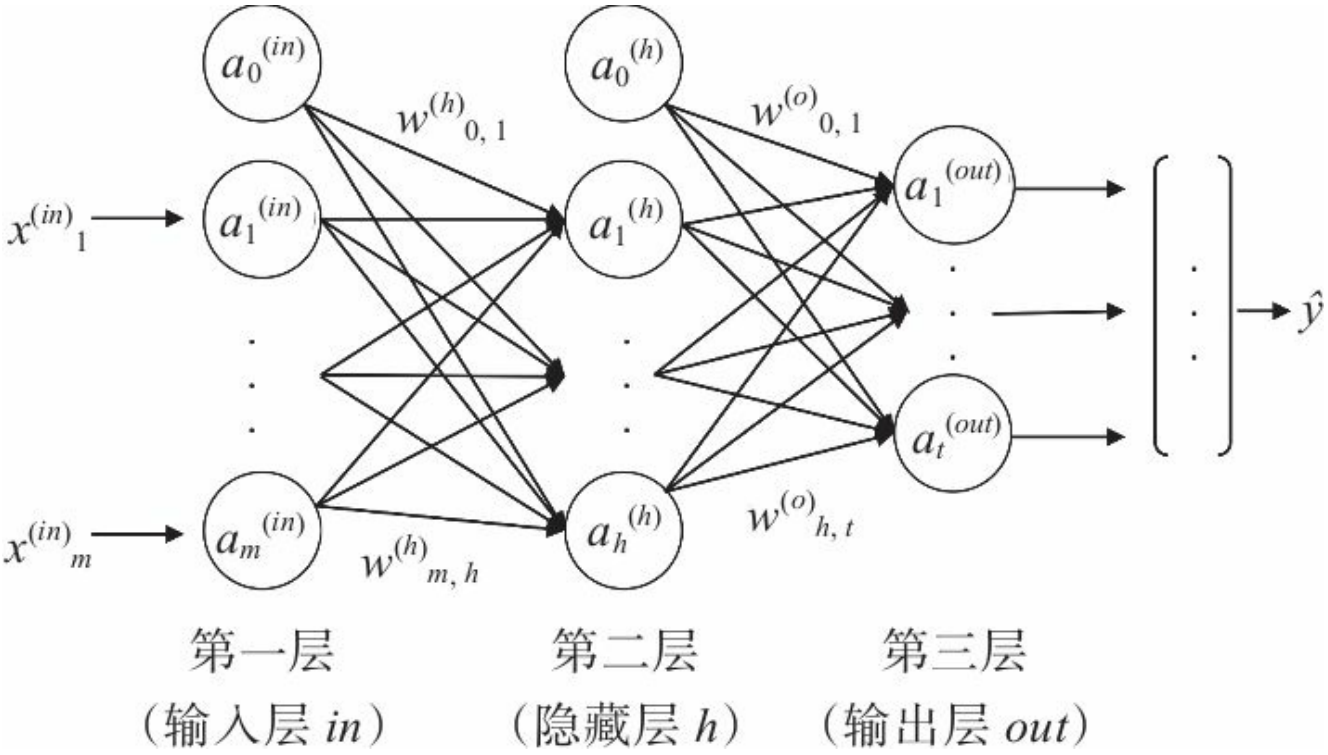
$$\hat{y} = \begin{cases} 1, & \text{if } g(z) \geqslant 0 \\ -1, & \text{otherwise} \end{cases}$$

值得注意的是，虽然Adaline由输入层和输出层两层组成，但它被称为单层网络，因为在输入层和输出层之间存在着单链接。

此外，我们还学习了一些加速模型学习的技巧，即所谓的随机梯度下降优化算法。随机梯度下降估计单个训练样本（在线学习）或小规模训练样本子集（小批量学习）的成本。本章稍后将用这个概念来实现和训练多层感知器。除了更快的学习速度（与梯度下降相比更频繁的权重更新），其噪声性质也被视为对训练多层神经网络的非线性激活函数有益，不存在凸成本函数。增加的噪声有助于避免局部成本极小化，本章后面将会更详细地讨论该主题。

## 12.1.2 介绍多层神经网络体系

本节将学习如何将多个单神经元连接到多层前馈神经网络。这种特殊类型的全连接网络也被称为多层感知器（MLP）。下图展示了包含三个层次的MLP概念：




上图描述的MLP有一个输入层、一个隐藏层和一个输出层。隐藏层中的单元完全连接到输入层，同时输出层完全连接到隐藏层。如果这样的网络有一个以上的隐藏层，我们也称之为深度人工神经网络。

可以为MLP添加任意数量的隐藏层，以创建更深入的网络体系。实际上可以把神经网络的层数和单元数作为要优化的额外超参数，对该问题将采用在第6章讨论过的交叉验证技术。然而因为网络中加入了更多层，所以通过反向传播计算的误差梯度将越来越小。这个梯度消失问题使模型学习更具挑战性。因此，已经研发了专门的算法来帮助训练这种深度神经网络结构，这就是所谓的深度学习。

上图用$a_i^{(l)}$来表示第l层的第i个活化单元。为了使数学和代码实现更直观，我们将不用数值来表示各层，而用in上标表示输入层，h上标表示隐藏层，out上标表示输出层。例如，$a_i^{(in)}$指输入层的第i个数值，$a_i^{(h)}$指隐藏层的第i个数值，$a_i^{(out)}$指输出层的第i个数值。这里激发单元$a_0^{(in)}$和$a_0^{(h)}$为偏差单

元，定义为1。输入层的激发单元只是输入单元加上偏差单元：

$$a^{in} = \begin{bmatrix} a_0^{(in)} \\ a_1^{(in)} \\ \vdots \\ a_m^{(in)} \end{bmatrix} = \begin{bmatrix} 1 \\ x_1^{(in)} \\ \vdots \\ x_m^{(in)} \end{bmatrix}$$

本章后面将用偏置单元的独立向量实现多层感知器，这可以使代码的实现更高效而且更易于阅读。这个概念也被用于TensorFlow（深度学习库），第13章将对此讨论。然而如果必须为偏置增加额外的变量，那么随之而来的数学方程式会显得错综复杂。

但是，请注意，通过对输入向量加上1的计算（如前面所示）和使用权重变量作为偏置与单独的偏置向量操作完全相同。它仅仅是个不同的约定。

l层中的每个单元通过权重系数与l+1层中的所有单元连接。例如l层中的第k个单元与l+1层中的第j个单元之间的连接可以表示为：$w_{k,j}^{(l)}$。回顾前面的图，我们把连接输入层到隐藏层的权重矩阵表示为$W^{(h)}$，把连接隐藏层到输出层的矩阵表示为W（out）。

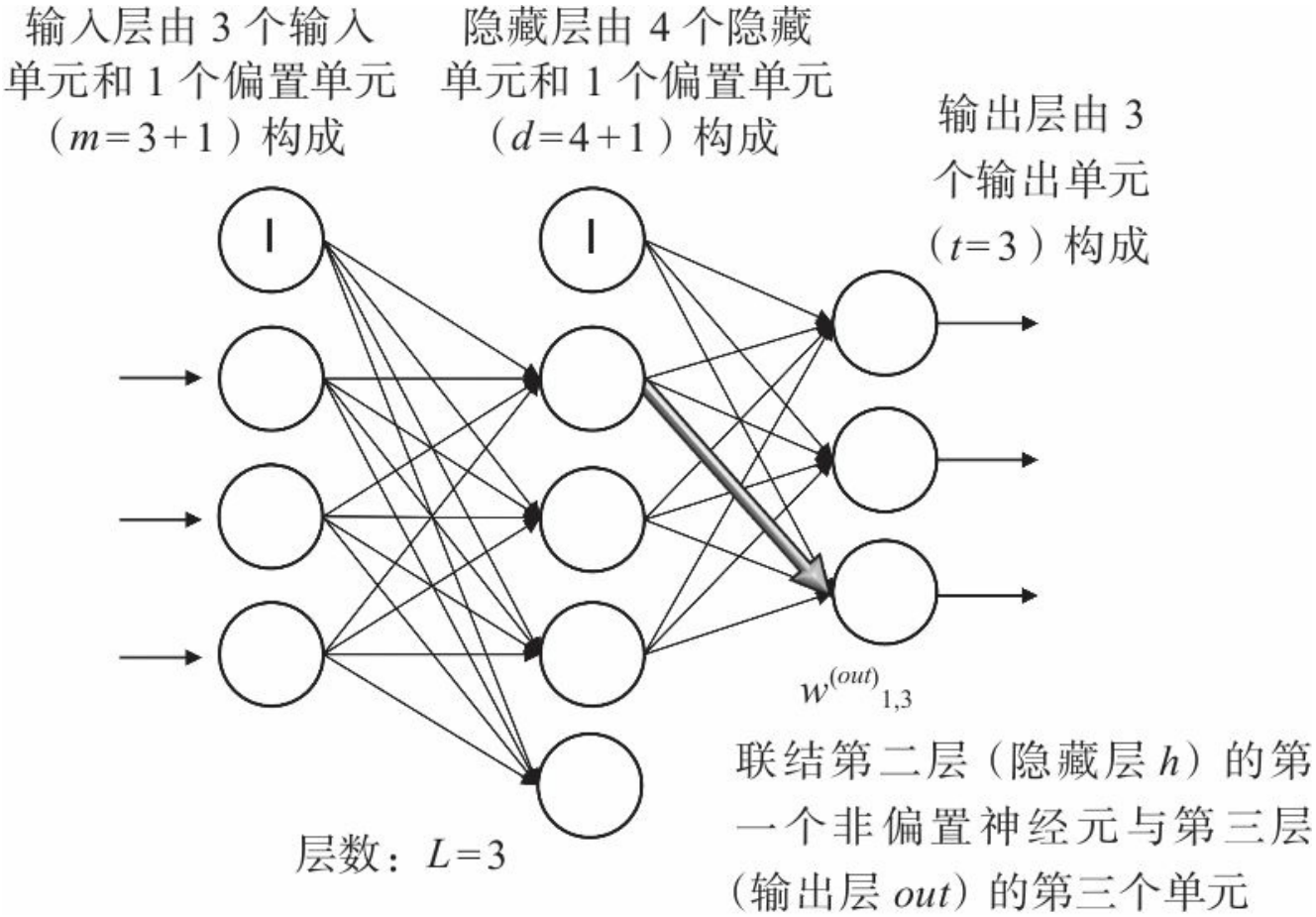
虽然输出层中的一个单元就足以满足二进制的分类任务，但是从前面的图中看到了一个更一般的神经网络形式，它允许使用一对多（OvA）的泛化技术来完成多元分类任务。为了更好地理解其工作原理，请记住第4章中介绍的独热分类变量表示。例如可以采用熟悉的鸢尾花数据集的三个分类标签（0=Setosa，1=Versicolor，2=Virginica）：

$$0 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, 1 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, 2 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

独热向量表示可以处理在训练集中有任意数量独立标签的分类任务。

如果刚接触神经网络的表示方式，标引符号（下标和上标）初看起来可能有点儿混乱。它们起初也许看起来过于复杂，但在以后的章节中向量化神

经网络表示时会更有意义。如前面介绍的那样，用矩阵$W^{(h)} \in R^{m \times d}$来概括地表示连接输入层和隐藏层的权重，d为隐藏层的单元数量，m为输入层单元（包括偏置单元）的数量。因为内化这个符号以继续学习本章后面的概念很重要，下图把刚学到的知识总结在描述简化的3-4-3多层感知器中：



## 12.1.3 利用正向传播激活神经网络

本节将描述正向传播计算MLP模型输出的过程。为了理解它为什么能适合学习MLP模型，把MLP的学习过程总结为下述三个简单步骤：

1.从输入层开始，把训练数据的模型通过网络传播输出。

2.基于网络输出，用稍后将描述的代价函数计算想要最小化的错误。

3.反向传播误差，匹配网络中相应的权重结果并更新模型。

最后，在上述三个步骤经过多次迭代并学习了MLP权重之后，用正向传播计算网络输出，并应用阈值函数以前面描述的独热表示方式获得预测的分类标签。

让我们详细解释正向传播输出训练数据模型的各个步骤。由于隐藏层中的每个单元都与输入层中的所有单元相连接，所以首先对隐藏层的激活单元$a_1^{(h)}$进行计算：

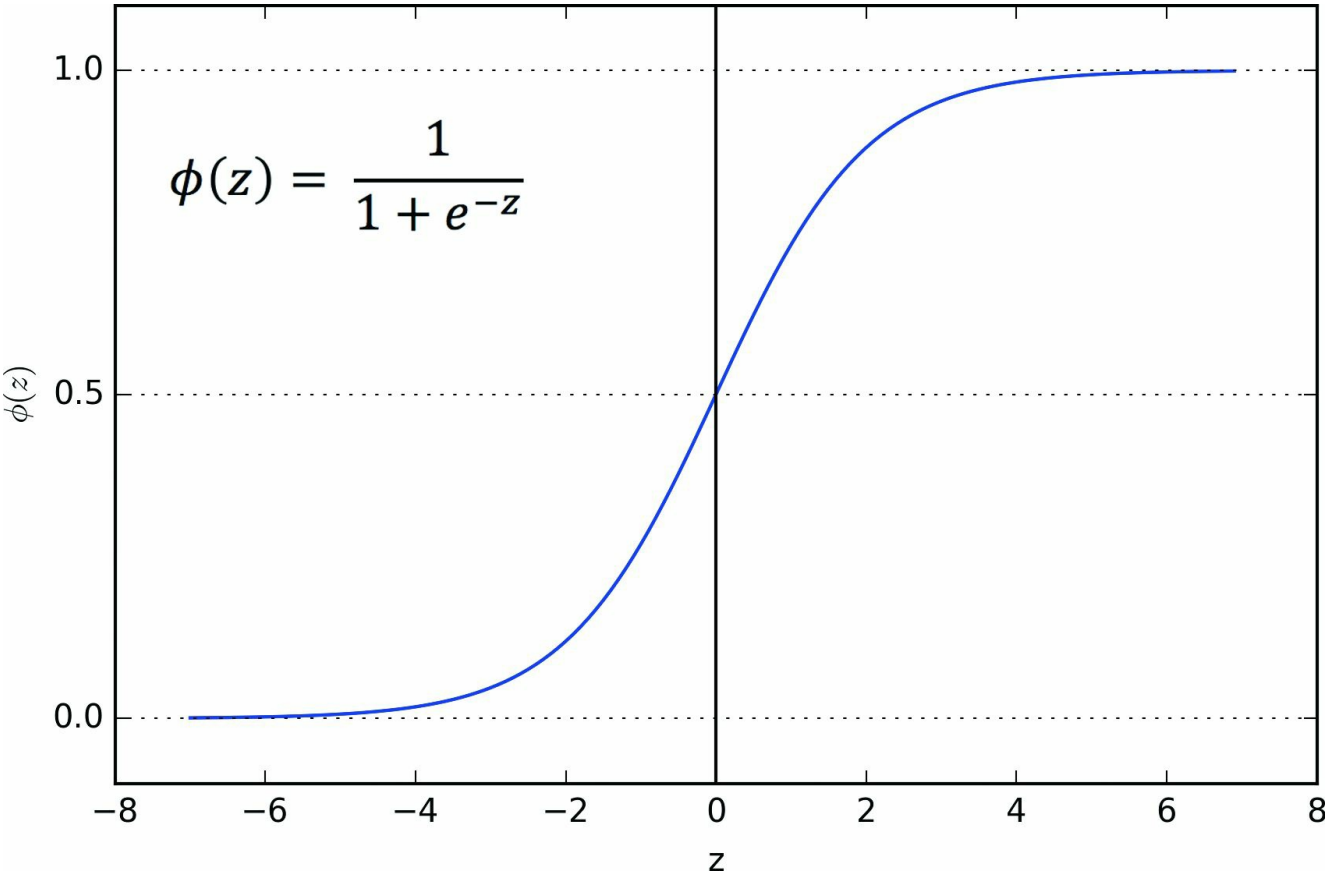
$$z_1^{(h)} = a_0^{(in)} w_{0,1}^{(h)} + a_1^{(in)} w_{1,1}^{(h)} + \cdots + a_m^{(in)} w_{m,1}^{(h)}$$

$$a_1^{(h)} = \phi\left(z_1^{(h)}\right)$$

这里$z_1^{(h)}$为净输入，φ（·）为激活函数，要以基于梯度的方法学习与神经元连接的权重，则该函数必须可微。为了能解决诸如图像分类这种复杂的问题，需要MLP模型中存在非线性激活函数，例如S状激活函数，回顾一下第3章中曾经讨论过的逻辑回归部分：

$$\phi(z) = \frac{1}{1+e^{-z}}$$

如前所述，S形函数是一条S状曲线，它将净输入z映射到逻辑分布在0到1的区间，在z=0处切割y轴，如下图所示：




MLP是前馈人工神经网络的典型例子。前馈一词指的是每一层都是不循环的下一层的输入，与本章后面将要讨论的递归神经网络相反，第16章将对此进行更详细的讨论。多层感知器一词听起来可能有点儿乱，因为这种网络体系的人工神经元通常是S单元，并不是感知器。直观地说，可以把MLP中的神经元看作是逻辑回归单位，其返回值在0和1之间的连续范围内。

考虑到代码效率和可读性，我们将用线性代数的基本概念以更紧凑的形式表示激活，这可以用NumPy向量化代码实现，比计算成本昂贵的Python for多嵌套循环要好得多：

$$\boldsymbol{z}^{(h)} = \boldsymbol{a}^{(in)}\boldsymbol{W}^{(h)}$$

$$\boldsymbol{a}^{(h)} = \phi(\boldsymbol{z}^{(h)})$$

这里$a^{(in)}$为样本$x^{(in)}$的1×m维特征向量再加上偏置单元。$W^{(h)}$为m×d维权重矩阵，d为隐藏层单元的个数。在完成矩阵–向量计算之后，得到1×d维净输入向量$z^{(h)}$用以计算激活值$a^{(h)}$（$a^{(h)} \in R^{1\times d}$）。可以进一步把这种计算推广到训练集的所有n个样本。

$$\boldsymbol{Z}^{(h)} = \boldsymbol{A}^{(in)}\boldsymbol{W}^{(h)}$$

这里，$A^{(in)}$为n×m矩阵，矩阵相乘的结果为n×d维的净输入矩阵

$Z^{(h)}$。最后用激活函数φ（·）计算净输入矩阵中的每个值，从而得到下一层n×d维的激活矩阵$A^{(h)}$（这里是输出层）：

$$A^{(h)} = \phi(z^{(h)})$$

类似，可以把多个样本的输出层激发值计算表示为向量形式：

$$Z^{(out)} = A^{(h)} W^{(out)}$$

这里，用d×t矩阵（t为数据单元的个数）$W^{(out)}$乘以n×d维矩阵$A^{(h)}$，以获得n×t维的矩阵$Z^{(out)}$（该矩阵的列代表每个样本的输出）。

最后，调用S激发函数来获得网络的连续性值输出。

$$A^{(out)} = \phi(z^{(out)}),\ A^{(out)} \in \mathbb{R}^{n \times t}$$

## 12.2 识别手写数字

上一节介绍了很多关于神经网络的理论，如果初次接触这个主题可能会有点儿不知所措。在继续讨论学习MLP模型的权重和反向传播算法之前，让我们把理论先暂时放在一边，看看实践中的神经网络。

神经网络理论可能相当复杂，因此我想推荐另外两个资源，它们对本章所讨论的一些概念做了更详细的阐述：

《深度前馈网络，深度学习》的第6章，I.古德菲洛，Y.本希奥，A.考维尔，麻省理工学院2016年出版。（可以从下述网站免费获得手稿：http://www.deeplearningbook.org。）

《模式识别和机器学习》，C.M.毕晓普等，斯普林格出版社，第一卷，2006年纽约。

本节将实现并训练第一个多层神经网络，来对美国国家标准与技术研究院（MNIST）常用的手写数字数据集进行分类，该数据集是由燕乐存等人构建，作为机器学习算法的常用基准数据集（Y.乐存，L.包头，Y.本希奥，P.哈夫纳.《把基于梯度的学习应用于文档识别》.IEEE，1998年11月，86（11）：2278-2324）。

### 12.2.1 获取MNIST数据集

MNIST数据集可以从下述公开网站获得：http://yann.lecun.com/exdb/mnist/

它包括下述四个部分：

·训练集图像：train-images-idx3-ubyte.gz（9.9 MB，解压后47 MB，60 000样本）

·训练集标签：train-labels-idx1-ubyte.gz（29 KB，解压后60 KB，60 000标签）

·测试集图像：t10k-images-idx3-ubyte.gz（1.6 MB，解压后7.8 MB，10 000样本）

·测试集标签：t10k-labels-idx1-ubyte.gz（5 KB，解压后10 KB，10 000标签）

MNIST数据集来自于美国国家标准与技术研究所（NIST）的两个数据集。训练集包含了250个人的手写数字，其中50%为高中学生，另外50%为人口普查局职员。请注意，测试集包含同样比例构成的不同人群的手写数字。为了快捷，下载该文件之后，建议从Unix或Linux终端上用gzip工具解压，在本地的MNIST目录下，用以下命令解压：

```
gzip *ubyte.gz -d
```

如果用微软Windows的计算机，也可以使用自己喜欢的解压工具。图像以字节的格式存储，会被读进NumPy阵列里，然后用MLP算法来训练和测试。为了做到这一点，需定义以下的辅助函数：

```
import os
import struct
import numpy as np

def load_mnist(path, kind='train'):
    """Load MNIST data from `path`"""
    labels_path = os.path.join(path,
                               '%s-labels-idx1-ubyte' % kind)
    images_path = os.path.join(path,
                               '%s-images-idx3-ubyte' % kind)

    with open(labels_path, 'rb') as lbpath:
        magic, n = struct.unpack('>II',
                                 lbpath.read(8))
        labels = np.fromfile(lbpath,
                             dtype=np.uint8)

    with open(images_path, 'rb') as imgpath:
        magic, num, rows, cols = struct.unpack(">IIII",
                                               imgpath.read(16))
        images = np.fromfile(imgpath,
                             dtype=np.uint8).reshape(
                             len(labels), 784)
        images = ((images / 255.) - .5) * 2

    return images, labels
```

调用load_mnist函数将返回两个阵列，第一个是n×m维的NumPy阵列（images），其中n为样本数，m为特征数（在这里是像素）。训练集由60 000个训练数字组成，测试集包含10 000个样本。MNIST数据集的图像由28×28像素组成，每个像素由灰度值表示。我们把28×28的像素展开变成一维行向量，代表image阵列的行（784个行或图像）。load_mnist函数返回第二个阵列（labels），包含相应的目标变量、手写数字分类标签（整数0-9）。

初看起来读入图像的方式似乎有点奇怪：

```
>>> magic, n = struct.unpack('>II', lbpath.read(8))
>>> labels = np.fromfile(lbpath, dtype=np.int8)
```

要理解上面这两行代码，必须先了解MNIST网站对该数据的描述：

```
[offset] [type]          [value]          [description]
0000     32 bit integer  0x00000801(2049) magic number (MSB first)
0004     32 bit integer  60000            number of items
0008     unsigned byte   ??               label
0009     unsigned byte   ??               label
........
xxxx     unsigned byte   ??               label
```

前面两行代码首先读入奇妙的数字，该数字描述文件缓存中的协议和样本个数（n），用fromfile方法把下面的字节读入到NumPy阵列，把参数fmt的值'>II'传递给struct.unpack，该参数值包含以下两部分：

·>：这是大端字节序，它定义一串字节存储的顺序。如果你对术语大端和小端不熟悉，可以参考维基百科关于此术语的一篇优秀文章：https://en.wikipedia.org/wiki/Endianness。

·I：代表无符号整数。

最后，通过执行以下的代码，把MNIST中的像素值归一化为-1和1之间的数值（原来为0至255）：

```
images = ((images / 255.) - .5) * 2
```

背后的逻辑是基于梯度的优化远比第2章所讨论的这些条件更稳定。请注意，逐个像素调整图像比例与前几章中采用的特征比例调整方法有所不同。在此之前，我们从训练集导出比例参数，然后用该参数对训练集和测试集中的每列进行比例调整。但在处理图像像素时，把数据的中心置于零，同时在[-1，1]区间调整比例，这在实践中很常见也行之有效。

近期开发的另一种通过输入比例调整来提高基于梯度优化的收敛技术是批量归一化，这是本书不涉及的一个高级主题。但是，如果你对深度学习的应用和研究感兴趣，我强烈建议阅读更多关于批量归一化的内容，如谢尔盖·约飞和克里斯汀·盖塞迪写的《批量归一化：通过减少内部协变量变化提高深度网络训练的速度》（2015，https://arxiv.org/abs/1502.03167）。

执行下面的代码从本地目录解压MNIST数据集，加载60 000个训练实例以及10 000个测试样本（下面的代码片段假定下载的MNIST文件被解压到与代码相同的目录下）：

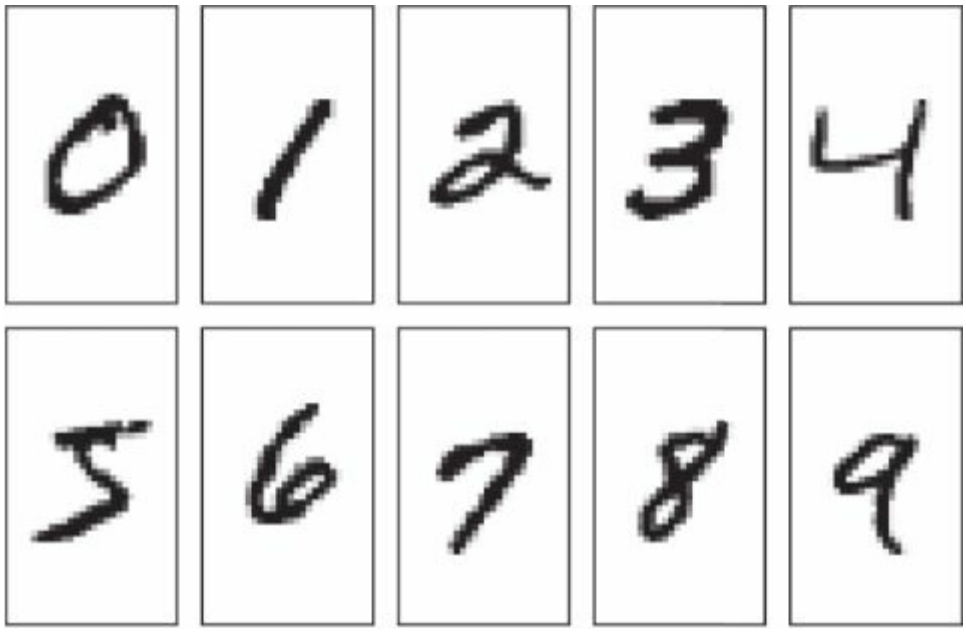
```
>>> X_train, y_train = load_mnist('', kind='train')
>>> print('Rows: %d, columns: %d'
...       % (X_train.shape[0], X_train.shape[1]))
Rows: 60000, columns: 784
```

```
>>> X_test, y_test = load_mnist('', kind='t10k')
>>> print('Rows: %d, columns: %d'
...       % (X_test.shape[0], X_test.shape[1]))
Rows: 10000, columns: 784
```

调用Matplotlib的imshow函数把拥有784个像素的向量特征矩阵变换回原来的28×28图像，可以图示出数字0-9，来看看MNIST这些图像的模样：

```
>>> import matplotlib.pyplot as plt

>>> fig, ax = plt.subplots(nrows=2, ncols=5,
...                        sharex=True, sharey=True)
>>> ax = ax.flatten()
>>> for i in range(10):
...     img = X_train[y_train == i][0].reshape(28, 28)
...     ax[i].imshow(img, cmap='Greys')

>>> ax[0].set_xticks([])
>>> ax[0].set_yticks([])
>>> plt.tight_layout()
>>> plt.show()
```

现在可以看到一个2×5的图集，它显示出每个独立数字的代表性图像：

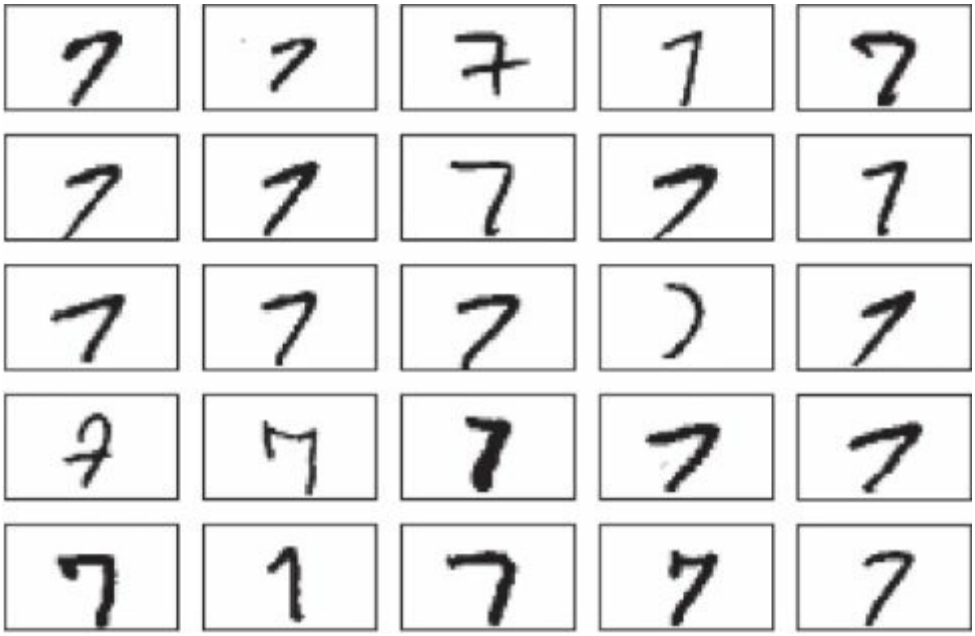


另外，也可以画出同一数字的多个例子，看看这些笔迹到底有多么不同：

```
>>> fig, ax = plt.subplots(nrows=5,
...                        ncols=5,
...                        sharex=True,
...                        sharey=True)
>>> ax = ax.flatten()
>>> for i in range(25):
...     img = X_train[y_train == 7][i].reshape(28, 28)
...     ax[i].imshow(img, cmap='Greys')
>>> ax[0].set_xticks([])
>>> ax[0].set_yticks([])
>>> plt.tight_layout()
>>> plt.show()
```

执行代码，现在应该看到数字7的前25个变体：

在完成前面所有的步骤之后，建议把比例调整后的图像以某种格式保存，这样就可以避免再次读取和处理数据，以便更快地把数据加载到新的Python会话。要在本地磁盘保存NumPy多维阵列，一个方便而有效的方法是调用savez函数（可以从下述网站找到官方文档：https://docs.scipy.org/doc/numpy/reference/generated/numpy.savez.html）。



总之，savez函数与第9章中用到的Python pickle模块类似，但为了存储NumPy阵列，对其进行了优化。savez函数输出压缩的归档数据，所产生的.npz文件也包含.npy格式的文件。如果想了解更多关于该格式的信息，可以从下述NumPy文档网站找到更清楚的解释，其中包括了有关其利弊的讨论：

https://docs.scipy.org/doc/NumPy/neps/npy-format.html

另外savez_compressed与savez的用法相同，但输出的文件被压缩得更小（大约是22MB与440MB的差别）。下述代码片段将把训练集和测试集存储

到归档文件'mnist_scaled.npz'：

```
>>> import numpy as np

>>> np.savez_compressed('mnist_scaled.npz',
...                     X_train=X_train,
...                     y_train=y_train,
...                     X_test=X_test,
...                     y_test=y_test)
```

产生.npz文件后，可以调用NumPy的load函数加载预处理后的MNIST图像阵列：

```
>>> mnist = np.load('mnist_scaled.npz')
```

现在mnist变量指向可以访问的对象，该对象包含在调用savez_compressed函数时所提供的四个关键参数相对应的四个阵列中，存储在对象mnist的文件属性列表：

```
>>> mnist.files
['X_train', 'y_train', 'X_test', 'y_test']
```

例如，可以通过下述方式访问'X_train'阵列（与Python字典类似）把训练数据加载到现在的Python会话：

```
>>> X_train = mnist['X_train']
```

通过下述方式穷举列表访问所有的四个阵列：

```
>>> X_train, y_train, X_test, y_test = [mnist[f] for
...                                     f in mnist.files]
```

请注意，处理np.savez_compressed和np.load的示例与执行本章的其他代码并非绝对相关，目的仅为了演示如何方便且有效地存储和加载NumPy阵列。

### 12.2.2 实现一个多层感知器

本小节将实现一个多层感知器（MLP），用于识别MNIST数据集中的图像，其中涉及一个输入层、一个隐藏层和一个输出层。我会尽可能保持代码简单易懂。但是起初看起来可能会有些复杂，我鼓励读者从Packt出版社或者GitHub的网站（https://github.com/rasbt/python-machine-learning-book-2nd-edition）下载本章的示例代码，这样可以通过代码注释和语法解释更好地理解MLP的实现逻辑：

如果不用Jupyter笔记本所附带的文件执行代码或者无法上网，我建议把本章NeuralNetMLP的代码拷贝到当前工作目录下的Python脚本上，例如neuralnet.py，然后用下述命令导入到当前的Python会话中：

```
from neuralnet import NeuralNetMLP
```

代码将包含部分尚未讨论过的内容，如反向传播算法，但大部分应该在第2章中基于Adaline的实现和前面讨论过的前向传播过程中见过。

如果有些代码不明白别太担心，本章后面将跟进解释。然而，在这个阶段浏览这些代码可以更容易理解本章后面的理论。

下述代码是对多层感知器的具体实现：

```
import numpy as np
import sys

class NeuralNetMLP(object):
    """ Feedforward neural network / Multi-layer perceptron
        classifier.
Parameters

------------
n_hidden : int (default: 30)
    Number of hidden units.
l2 : float (default: 0.)
    Lambda value for L2-regularization.
    No regularization if l2=0. (default)
epochs : int (default: 100)
    Number of passes over the training set.
eta : float (default: 0.001)
    Learning rate.
shuffle : bool (default: True)
    Shuffles training data every epoch
    if True to prevent circles.
minibatch_size : int (default: 1)
    Number of training samples per minibatch.
seed : int (default: None)
    Random seed for initializing weights and shuffling.

Attributes
-----------
eval_ : dict
  Dictionary collecting the cost, training accuracy,
  and validation accuracy for each epoch during training.

"""
def __init__(self, n_hidden=30,
             l2=0., epochs=100, eta=0.001,
             shuffle=True, minibatch_size=1, seed=None):

    self.random = np.random.RandomState(seed)
    self.n_hidden = n_hidden
    self.l2 = l2
    self.epochs = epochs
    self.eta = eta
    self.shuffle = shuffle
    self.minibatch_size = minibatch_size

def _onehot(self, y, n_classes):
    """Encode labels into one-hot representation
```

```
    Parameters
    ------------
    y : array, shape = [n_samples]
        Target values.

    Returns
    -----------
    onehot : array, shape = (n_samples, n_labels)

    """
    onehot = np.zeros((n_classes, y.shape[0]))
    for idx, val in enumerate(y.astype(int)):
        onehot[val, idx] = 1.
    return onehot.T

def _sigmoid(self, z):
    """Compute logistic function (sigmoid)"""
    return 1. / (1. + np.exp(-np.clip(z, -250, 250)))

def _forward(self, X):
    """Compute forward propagation step"""

    # step 1: net input of hidden layer
    # [n_samples, n_features] dot [n_features, n_hidden]
    # -> [n_samples, n_hidden]
    z_h = np.dot(X, self.w_h) + self.b_h

    # step 2: activation of hidden layer
    a_h = self._sigmoid(z_h)

    # step 3: net input of output layer
    # [n_samples, n_hidden] dot [n_hidden, n_classlabels]
    # -> [n_samples, n_classlabels]

    z_out = np.dot(a_h, self.w_out) + self.b_out

    # step 4: activation output layer
    a_out = self._sigmoid(z_out)

    return z_h, a_h, z_out, a_out

def _compute_cost(self, y_enc, output):
    """Compute cost function.

    Parameters
    ----------
    y_enc : array, shape = (n_samples, n_labels)
        one-hot encoded class labels.
    output : array, shape = [n_samples, n_output_units]
        Activation of the output layer (forward propagation)

    Returns
    ---------
    cost : float
        Regularized cost
```

```
    """
    L2_term = (self.l2 *
               (np.sum(self.w_h ** 2.) +
                np.sum(self.w_out ** 2.)))

    term1 = -y_enc * (np.log(output))
    term2 = (1. - y_enc) * np.log(1. - output)
    cost = np.sum(term1 - term2) + L2_term
    return cost

def predict(self, X):
    """Predict class labels

    Parameters
    -----------
    X : array, shape = [n_samples, n_features]
        Input layer with original features.

    Returns:
    ----------
    y_pred : array, shape = [n_samples]
        Predicted class labels.

    """
    z_h, a_h, z_out, a_out = self._forward(X)
    y_pred = np.argmax(z_out, axis=1)
    return y_pred

def fit(self, X_train, y_train, X_valid, y_valid):
    """ Learn weights from training data.

    Parameters
    -----------
    X_train : array, shape = [n_samples, n_features]
    Input layer with original features.
y_train : array, shape = [n_samples]
    Target class labels.
X_valid : array, shape = [n_samples, n_features]
    Sample features for validation during training
y_valid : array, shape = [n_samples]
    Sample labels for validation during training

Returns:
----------
self

"""
n_output = np.unique(y_train).shape[0] # no. of class
                                       #labels

n_features = X_train.shape[1]

########################
# Weight initialization
########################
```

```
        # weights for input -> hidden
        self.b_h = np.zeros(self.n_hidden)
        self.w_h = self.random.normal(loc=0.0, scale=0.1,
                                      size=(n_features,
                                            self.n_hidden))

        # weights for hidden -> output
        self.b_out = np.zeros(n_output)
        self.w_out = self.random.normal(loc=0.0, scale=0.1,
                                        size=(self.n_hidden,
                                              n_output))

        epoch_strlen = len(str(self.epochs))  # for progr. format.
        self.eval_ = {'cost': [], 'train_acc': [], 'valid_acc': \
                      []}

        y_train_enc = self._onehot(y_train, n_output)

        # iterate over training epochs
        for i in range(self.epochs):

            # iterate over minibatches
        indices = np.arange(X_train.shape[0])

        if self.shuffle:
            self.random.shuffle(indices)

        for start_idx in range(0, indices.shape[0] -\
                               self.minibatch_size +\
                               1, self.minibatch_size):
            batch_idx = indices[start_idx:start_idx +\
                                self.minibatch_size]

            # forward propagation
            z_h, a_h, z_out, a_out = \
                self._forward(X_train[batch_idx])

            ##################
            # Backpropagation
            ##################

            # [n_samples, n_classlabels]
            sigma_out = a_out - y_train_enc[batch_idx]

            # [n_samples, n_hidden]
            sigmoid_derivative_h = a_h * (1. - a_h)

            # [n_samples, n_classlabels] dot [n_classlabels,
            #                                 n_hidden]
            # -> [n_samples, n_hidden]
            sigma_h = (np.dot(sigma_out, self.w_out.T) *
                       sigmoid_derivative_h)

            # [n_features, n_samples] dot [n_samples,
            #                              n_hidden]
```

```
    # -> [n_features, n_hidden]
    grad_w_h = np.dot(X_train[batch_idx].T, sigma_h)
    grad_b_h = np.sum(sigma_h, axis=0)

    # [n_hidden, n_samples] dot [n_samples,
    #                            n_classlabels]
    # -> [n_hidden, n_classlabels]
    grad_w_out = np.dot(a_h.T, sigma_out)
    grad_b_out = np.sum(sigma_out, axis=0)

    # Regularization and weight updates

        delta_w_h = (grad_w_h + self.l2*self.w_h)
        delta_b_h = grad_b_h # bias is not regularized
        self.w_h -= self.eta * delta_w_h
        self.b_h -= self.eta * delta_b_h

        delta_w_out = (grad_w_out + self.l2*self.w_out)
        delta_b_out = grad_b_out # bias is not regularized
        self.w_out -= self.eta * delta_w_out
        self.b_out -= self.eta * delta_b_out

    ##############
    # Evaluation
    ##############

    # Evaluation after each epoch during training
    z_h, a_h, z_out, a_out = self._forward(X_train)

    cost = self._compute_cost(y_enc=y_train_enc,
                              output=a_out)

    y_train_pred = self.predict(X_train)
    y_valid_pred = self.predict(X_valid)

    train_acc = ((np.sum(y_train ==
                  y_train_pred)).astype(np.float) /
                 X_train.shape[0])
    valid_acc = ((np.sum(y_valid ==
                  y_valid_pred)).astype(np.float) /
                 X_valid.shape[0])

    sys.stderr.write('\r%0*d/%d | Cost: %.2f '
                     '| Train/Valid Acc.: %.2f%%/%.2f%% '
                     %
                     (epoch_strlen, i+1, self.epochs,
                      cost,
                      train_acc*100, valid_acc*100))
    sys.stderr.flush()

    self.eval_['cost'].append(cost)
    self.eval_['train_acc'].append(train_acc)
    self.eval_['valid_acc'].append(valid_acc)

return self
```

代码执行后可以初始化一个有784个输入层单元（n_features），100个隐藏层单元（n_hidden）以及10个输出层单元（n_output）的神经网络：

```
>>>nn = NeuralNetMLP(n_hidden=100,
...                   l2=0.01,
...                   epochs=200,
...                   eta=0.0005,
...                   minibatch_size=100,
...                   shuffle=True,
...                   seed=1)
```

如果已经读过NeuralNetMLP代码，可能已经猜到这些参数的作用。下面是对这些参数的简短摘要。

·l2：L2的参数λ，用来正则化以减少过拟合的机会。

·epochs：训练集的迭代次数。

·eta：学习率η。

·shuffle：为了避免算法陷入循环而定义的迭代前洗牌。

·seed：用于洗牌和初始化权重的随机种子。

在随机梯度下降的过程中，训练集在每次迭代被分裂成小批量，而每个小批量所包含的训练样本个数被定义为minibatch_size。为了更快地学习，我们不计算全部训练数据的梯度，而只单独计算每个小批量的梯度。

接着用55 000个洗过牌的MNIST样本训练MLP，把剩余的5000个样本用在训练验证过程中。注意，在标准的台式计算机上训练神经网络可能需要5分钟。

在前面的代码实现中可能已经注意到，fit方法接受四个输入参数：训练图像、训练标签、验证图像和验证标签。神经网络训练中，在训练阶段完成对训练和验证准确度的比较很有用，这有助于判断网络模型在已选定体系结构和超参数情况下的表现是否良好。

一般来说，训练深度神经网络与讨论过的其他模型相比，相对来说比较昂贵。因此，在某些情况下，我们要提早终止训练，并且采用不同的超参数设置重新开始。另外，如果发现有过拟合训练数据的倾向（可以观察到训练和验证数据集之间的性能差距越来越大），那可能就要提早停止训练。

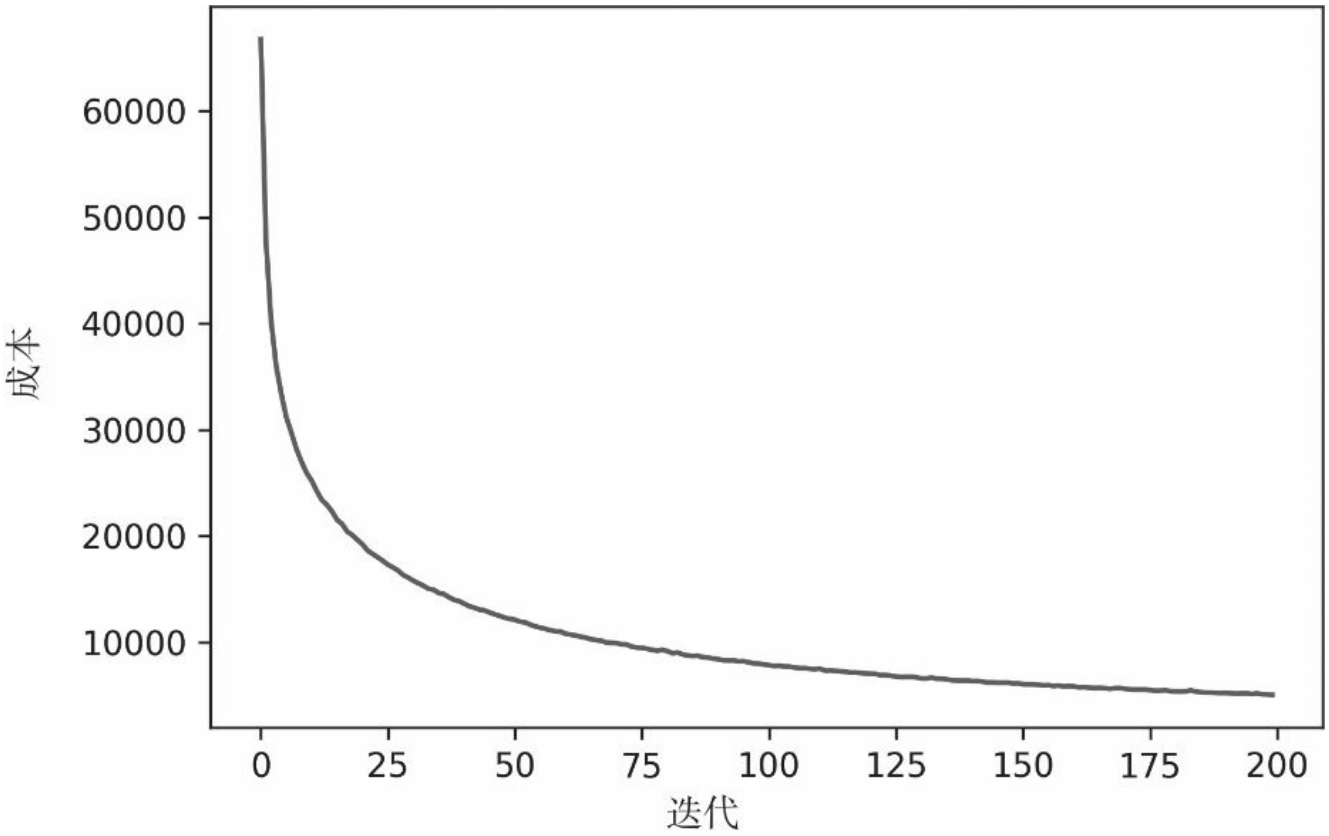
执行以下代码开始训练：

```
>>> nn.fit(X_train=X_train[:55000],
...        y_train=y_train[:55000],
...        X_valid=X_train[55000:],
...        y_valid=y_train[55000:])
200/200 | Cost: 5065.78 | Train/Valid Acc.: 99.28%/97.98%
```

在实现NeuralNetMLP的过程中，我们还定义了eval_属性，用来收集每个迭代的成本、训练和验证准确度，因此可以用Matplotlib来完成结果的可视化：

```
>>> import matplotlib.pyplot as plt
>>> plt.plot(range(nn.epochs), nn.eval_['cost'])
>>> plt.ylabel('Cost')
>>> plt.xlabel('Epochs')
>>> plt.show()
```

上面的代码在下图中绘出了200个迭代的成本情况：

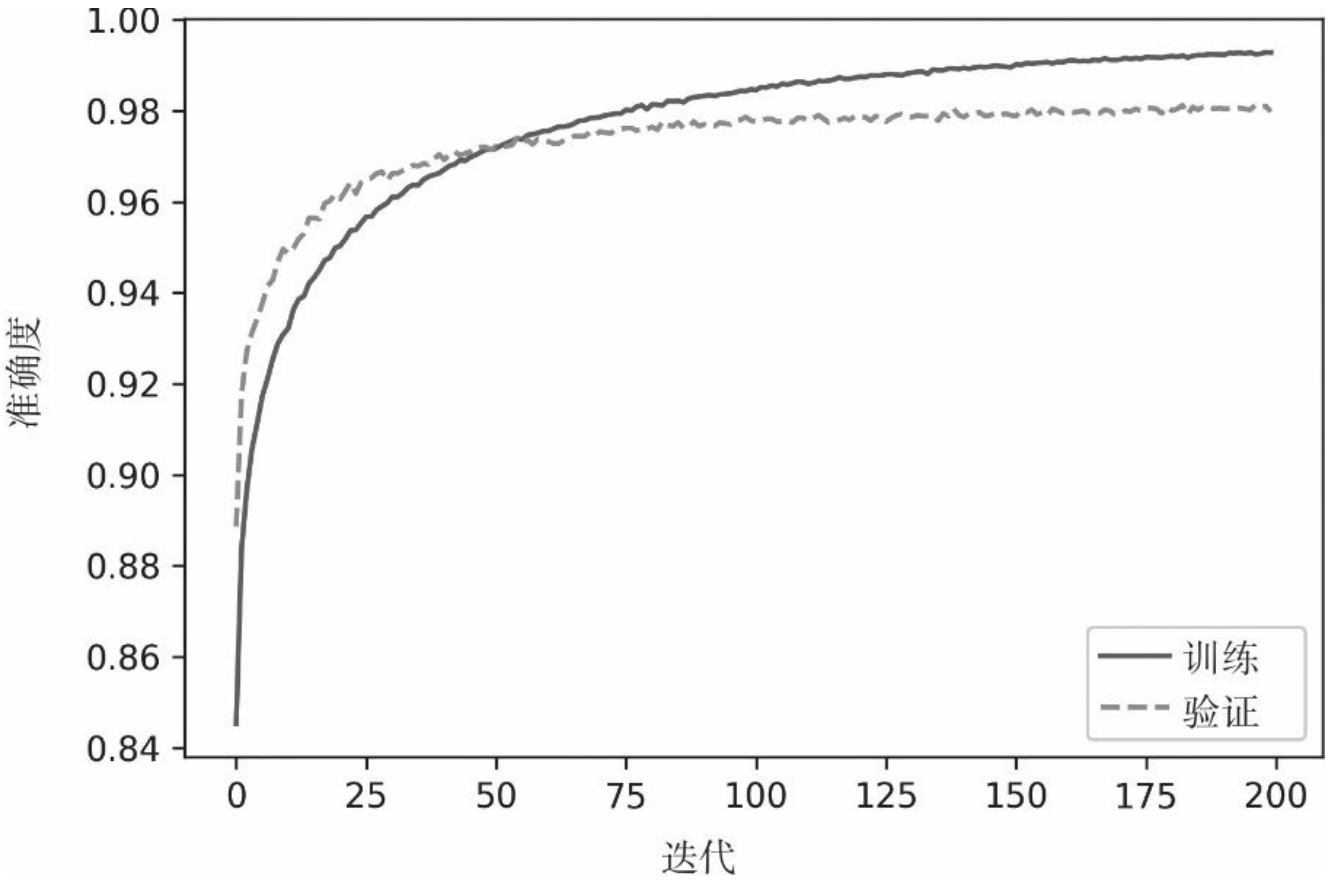



正如可以看到的，前100次迭代的成本大幅下降，而后100次迭代似乎收敛缓慢。然而，第175次迭代和第200次迭代之间的小坡度表明，随着训练的额外迭代次数增加，成本将进一步降低。

接下来，让我们看看训练和验证的准确度：

```
>>> plt.plot(range(nn.epochs), nn.eval_['train_acc'],
...           label='training')
>>> plt.plot(range(nn.epochs), nn.eval_['valid_acc'],
...           label='validation', linestyle='--')
>>> plt.ylabel('Accuracy')
>>> plt.xlabel('Epochs')
>>> plt.legend()
>>> plt.show()
```

前面的代码示例绘制了200次训练迭代的准确度变化情况，如下图所示：




上图表明训练网络的迭代次数越多，训练和验证准确度之间的差距就越大。在大约第50次迭代时，训练和验证的准确度值相等，之后网络就开始过拟合训练数据。

请注意，我们特意选择这个例子是来说明过拟合的影响，并且说明为什么在训练过程中比较验证和训练的准确度很有用。减少过拟合影响的一种方法是增加正则化的强度，例如设置l2=0.1。解决神经网络过拟合的另一种有效方法是停止学习，第15章将对此进行讨论。

最后，通过计算模型在测试集上的预测准确度来评估其泛化性能：

```
>>> y_test_pred = nn.predict(X_test)
>>> acc = (np.sum(y_test == y_test_pred)
...        .astype(np.float) / X_test.shape[0])
>>> print('Training accuracy: %.2f%%' % (acc * 100))
Test accuracy: 97.54%
```

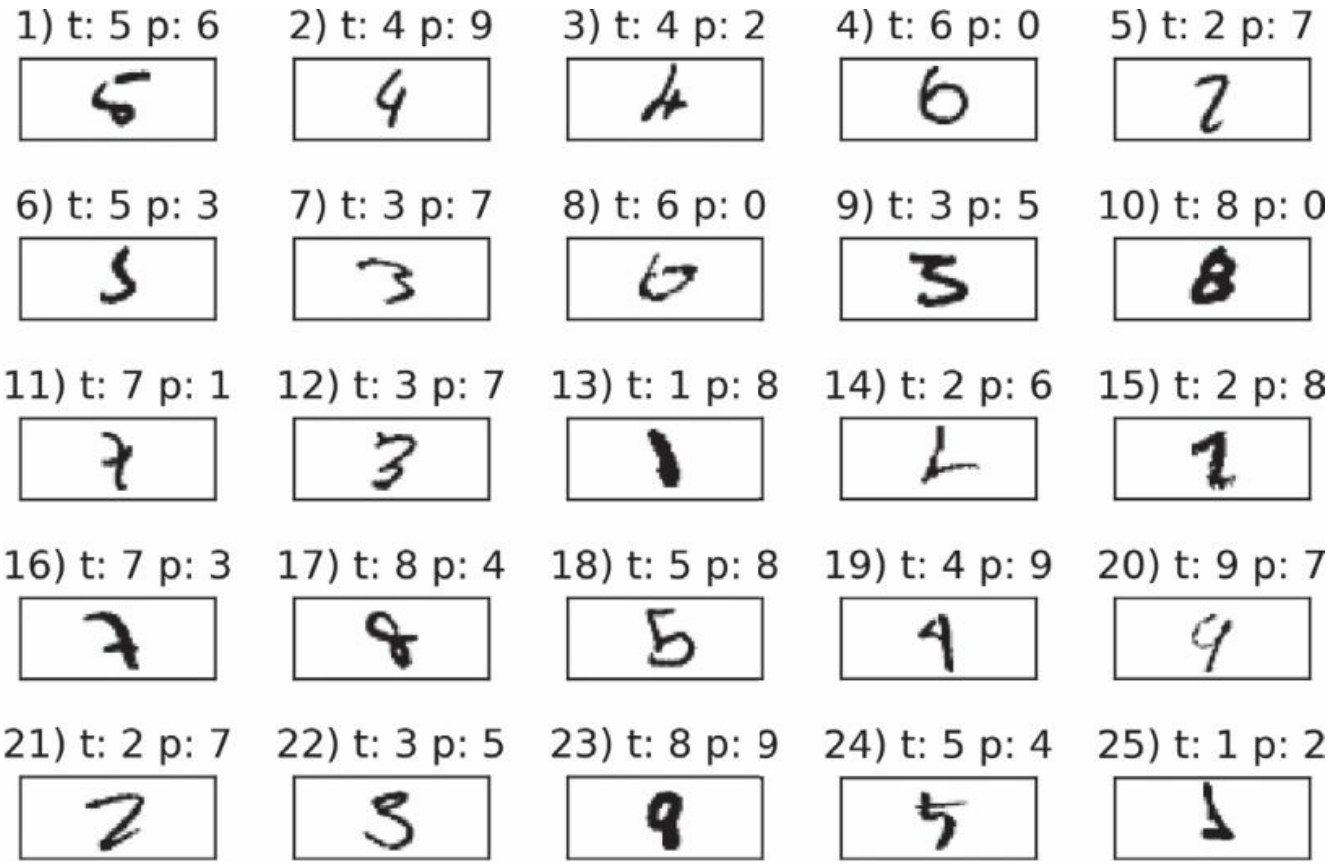
尽管在训练集上出现了轻微的过拟合现象，我们的比较简单的单隐藏层神经网络在测试集上取得了相对较好的性能，这和在验证数据集上的准确度（97.98%）类似。

可以调整隐藏层的单元数、正则化的参数值以及学习率或者采用其他各种手段来进一步优化模型。在14章中，你将会了解一种不同的神经网络体系结构，以其对图像数据集表现优异而著称。另外，本章将介绍更多增强性能的技巧，如自适应学习率、动量学习和停止学习。

最后，让我们看看MLP难以处理的一些图像：

```
>>> miscl_img = X_test[y_test != y_test_pred][:25]
>>> correct_lab = y_test[y_test != y_test_pred][:25]
>>> miscl_lab= y_test_pred[y_test != y_test_pred][:25]

>>> fig, ax = plt.subplots(nrows=5,
...                        ncols=5,
...                        sharex=True,
...                        sharey=True,)
>>> ax = ax.flatten()
>>> for i in range(25):
...     img = miscl_img[i].reshape(28, 28)
...     ax[i].imshow(img,
...                  cmap='Greys',
...                  interpolation='nearest')
...     ax[i].set_title('%d) t: %d p: %d'
...                     % (i+1, correct_lab[i], miscl_lab[i]))

>>> ax[0].set_xticks([])
>>> ax[0].set_yticks([])
>>> plt.tight_layout()
>>> plt.show()
```

现在应该看到一个由子图构成的5×5矩阵，每个小标题的第一个数字代表图的顺序号，第二个数字代表真正的分类标签（t），第三个数字代表预测的分类标签（p）：




正如前面图中所看到的，其中一些图像甚至对人类来说都难以进行正确的分类。例如，子图8中的数字6看起来真像一个随手写的0，子图23中的数字8，由于狭窄的下部与粗线粘连在一起，看上去更像9。

## 12.3 训练人工神经网络

现在已经看到了一个实践中的神经网络，并通过阅读代码基本了解了它的工作原理，让我们更深入了解一些概念，比如用来实现权重学习的逻辑成本函数和反向传播算法。

### 12.3.1 逻辑成本函数的计算

实现_compute_cost的方法实际上非常简单，因为它与第3章中的逻辑回归部分所描述的成本函数相同：

$$J(\boldsymbol{w}) = -\sum_{i=1}^{n} y^{[i]} \log\left(a^{[i]}\right) + \left(1 - y^{[i]}\right) \log\left(1 - a^{[i]}\right)$$

这里a[i]为数据集中第i个样本用前向传播算法计算出的S形激活值：

$$a^{[i]} = \phi(z^{[i]})$$

同样，请注意，这个表达式上标[i]指训练样本的索引而不是层。

现在添加一个正则化项以减少过拟合的机会。正如前面章节中讨论过的那样，L2正则化项的定义如下（记住，不正则偏差单元）：

$$L2 = \lambda \|\boldsymbol{w}\|_2^2 = \lambda \sum_{j=1}^{m} w_j^2$$

通过将L2正则项加入到逻辑成本函数得到以下的等式：

$$J(\boldsymbol{w}) = -\left[\sum_{i=1}^{n} y^{[i]} \log\left(a^{[i]}\right) + \left(1 - y^{[i]}\right) \log\left(1 - a^{[i]}\right)\right] + \frac{\lambda}{2} \|\boldsymbol{w}\|_2^2$$

因为实现了用于多元分类的MLP，它返回一个有t个元素的输出向量，需要与t×1维独热编码表示的目标向量进行比较，例如，某个样本的第三层和目标类（这里是2类）的激活可能以如下的方式来表示：

$$a^{(out)} = \begin{bmatrix} 0.1 \\ 0.9 \\ \vdots \\ 0.3 \end{bmatrix}, y = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

因此，需要将逻辑成本函数推广到网络中所有的t激活单元。因此成本函数（不包含正则化项）变成下面这样：

$$J(\boldsymbol{W}) = -\sum_{i=1}^{n}\sum_{j=1}^{t} y_j^{[i]} \log\left(a^{[i]}{}_j\right) + \left(1 - y_j^{[i]}\right)\log\left(1 - a^{[i]}{}_j\right)$$

在这里，上标（i）再次成为训练集中特定样本的索引。

下面的广义正则化项初看起来有点复杂，但这里只计算添加到第1列的一个l层（无偏差项）所有权重的总和：

$$J(\boldsymbol{W}) = -\left[\sum_{i=1}^{n}\sum_{j=1}^{t} y_j^{[i]} \log\left(a^{[i]}{}_j\right) + \left(1 - y_j^{[i]}\right)\log\left(1 - a^{[i]}{}_j\right)\right] + \frac{\lambda}{2}\sum_{l=1}^{L-1}\sum_{i=1}^{u_l}\sum_{j=1}^{u_{l+1}}\left(w_{j,i}^{(l)}\right)^2$$

这里$u_l$是一个给定的l层单位数，下面的表达式代表惩罚项：

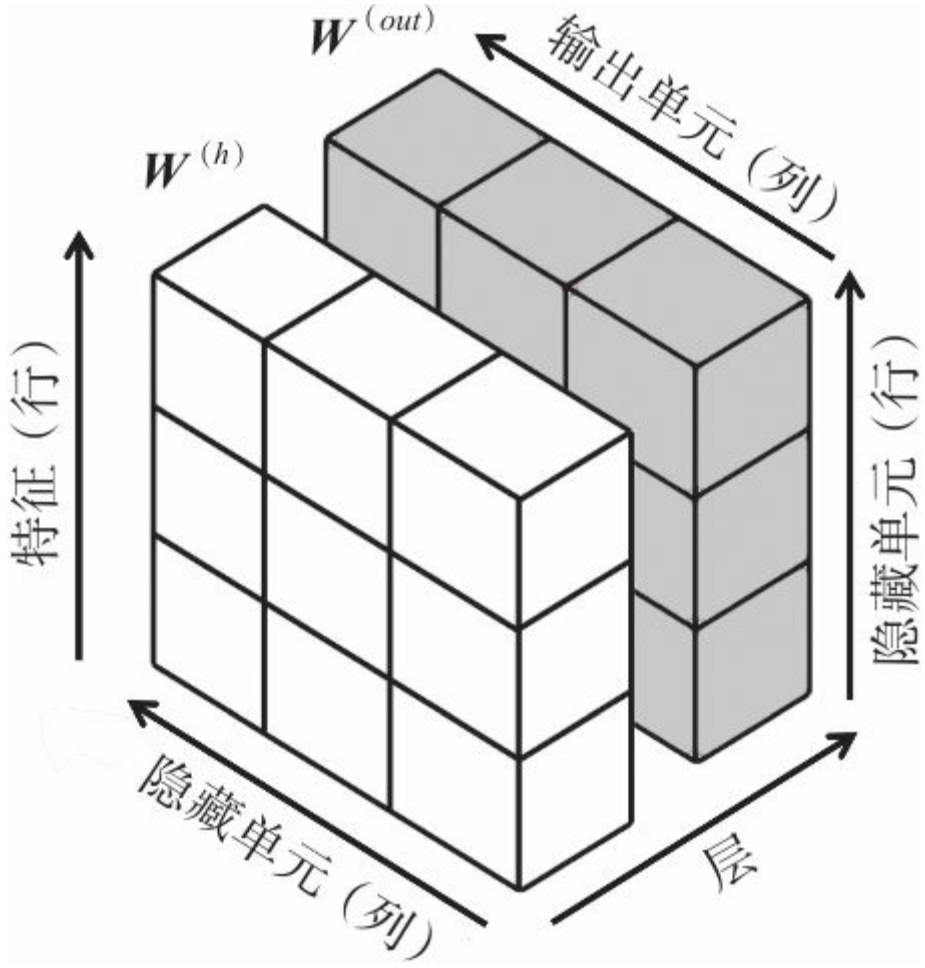
$$\frac{\lambda}{2}\sum_{l=1}^{L-1}\sum_{i=1}^{u_l}\sum_{j=1}^{u_{l+1}}\left(w_{j,i}^{(l)}\right)^2$$

记住，目标是最小化成本函数J（W）；因此，需要计算参数W的偏导数，W相应于网络中每层的每个权重

$$\frac{\partial}{\partial w_{j,i}^{(l)}} J(\boldsymbol{W})$$

下一节将讨论反向传播算法，通过计算偏导数来最小化代价函数。

请注意，W是由多个矩阵组成的。在有隐藏单位的多层感知器中，权重矩阵$W^{(h)}$连接输入层与隐藏层，矩阵$W^{(out)}$连接隐藏层与输出层。下图提供了直观的三维张量W的可视化效果：




上面的简图看上去好像$W^{(h)}$和$W^{(out)}$有同样的行数和列数，通常情况并不是这样，只有在初始化MLP时才用同样的隐藏单元数、输出单元数和输入样本数。

如果对此感到困惑，可以暂时把它先放到一边，等到下一节介绍反向传播时，再更深入地讨论$W^{(h)}$和$W^{(out)}$的维度问题。另外，我鼓励你再读一遍NeuralNetMLP代码，其中加了不少注释，用来帮助你理解有关不同类型矩阵和向量转换的维度问题。可以从Packt网站或者本书在GitHub的库下载这些加了注释的代码：

https://github.com/rasbt/python-machine-learning-book-2nd-edition

## 12.3.2 开发反向传播的直觉

虽然重新发现和推广反向传播已经是30多年前的事（D.E.鲁迈尔哈特，G.E.新顿，R.J.威廉姆斯.《通过反向传播错误学习表现》.自然，1986，323：6088，533-536），但它仍然是有效训练人工神经网络应用最广泛的算法之一。如果你对反向传播的历史参考文献感兴趣，可以在下述网站找到约尔根·米德胡贝写的一篇优秀的调研报告《谁发明了反向传播？》：http://people.idsia.ch/~juergen/who-invented-backpropagation.html

在更深入讨论数学细节之前，本节打算提供一个简单而且直观的总结，以及对这个迷人算法工作机理的宏观描述。本质上可以认为反向传播是一种计算多层神经网络的复杂成本函数的偏导数的非常有效的方法。目标是利用这些导数来学习权重系数，以实现多层人工神经网络的参数化。神经网络参数化通常所面临的挑战是处理高维特征空间的大量权重系数。与在前面章节中见过的Adaline和逻辑回归这样的单层神经网络成本函数相反，神经网络成本函数与参数相关的误差表面无凸起或光滑。高维成本表面必须要克服很多的凸块（局部极小值）才能找到目标函数的全局最小值。

你可能还记得微积分入门课程的链式规则概念。链式规则是一种计算复杂嵌套函数导数的方法，如f（g（x））， 如下所示：

$$\frac{d}{dx}\left[f\left(g(x)\right)\right]=\frac{df}{dg}\cdot\frac{dg}{dx}$$

类似地，可以用链规则来处理任意长的函数组合。例如，假设有五个不同的函数f（x），g（x），h（x），u（x）和v（x），F为函数组合：F（x）=f（g（h（u（v（x））））。应用链式规则，可以计算该函数的导数如下：

$$\frac{dF}{dx}=\frac{d}{dx}F(x)=\frac{d}{dx}f\left(g\left(h\left(u\left(v(x)\right)\right)\right)\right)=\frac{df}{dg}\cdot\frac{dg}{dh}\cdot\frac{dh}{du}\cdot\frac{du}{dv}\cdot\frac{dv}{dx}$$

计算机代数已经开发出了一套非常有效地解决这类问题的技术，也就是所谓的自动微分。如果你有兴趣了解更多关于机器学习应用中的自动微分知识，我推荐阅读A.G.贝丁和B.A.珀缪特的文章《机器学习的自动微分算法》（arXiv预印本arXiv：1404.7456，2014年）可以从下述arXiv网站免费获得：

http://arxiv.org/pdf/1404.7456.pdf

自动微分有正向和反向两种模式，反向传播仅仅是反向模式自动微分的特例。关键是正向模式应用链式法则可能相当昂贵，因为要与每层的大矩阵（雅可比矩阵）相乘，最终乘以一个向量以获得输出。反向模式的技巧是从右向左：用一个矩阵乘以一个向量，从而产生另一个向量，然后再乘以下一个矩阵，以此类推。矩阵向量乘法比矩阵矩阵乘法在计算成本上要便宜得多，这就是为什么反向传播算法是神经网络训练中最常用的算法之一。

为了充分理解反向传播，需要借用微分学中的某些概念，这超出了本书的范围。然而，我已经把对最基本概念的回顾写成了一个章节，你可能会从中发现有用的概念。该章讨论了函数导数、偏导数、梯度和雅可比矩阵。可以从下述网站免费获得该章节：

https://sebastianraschka.com/pdf/books/dlb/appendix_d_calculus.pdf

如果你对此积分不熟悉或者需要简单的回顾，在学习下一章之前，可以考虑阅读该章节。

### 12.3.3 通过反向传播训练神经网络

本节将学习反向传播所涉及的数学，以了解如何有效地学习神经网络中的权重。取决于对数学表达式掌握的程度，下面的方程一开始看起来可能会比较复杂。

前一节已经看过如何计算成本，这里定义成本为最后一层的激活与目标分类标签之间的差别。现在将从数学角度来看反向传播算法如何更新MLP模型的权重，该算法在#Backpropagation部分的fit方法中实现。正如本章开头所提到的，首先要应用前向传播以获得输出层的激活，公式如下：

$$\boldsymbol{Z}^{(h)} = \boldsymbol{A}^{(in)}\boldsymbol{W}^{(h)} \text{（隐藏层的净输入）}$$

$$\boldsymbol{A}^{(h)} = \phi(\boldsymbol{Z}^{(h)}) \text{（隐藏层的激活）}$$

$$\boldsymbol{Z}^{(out)} = \boldsymbol{A}^{(h)}\boldsymbol{W}^{(out)} \text{（输出层的净输入）}$$

$$\boldsymbol{A}^{(out)} = \phi(\boldsymbol{Z}^{(out)}) \text{（输出层的激活）}$$

简单地说，只是通过网络连接前向传播输入的特征，如下图所示：

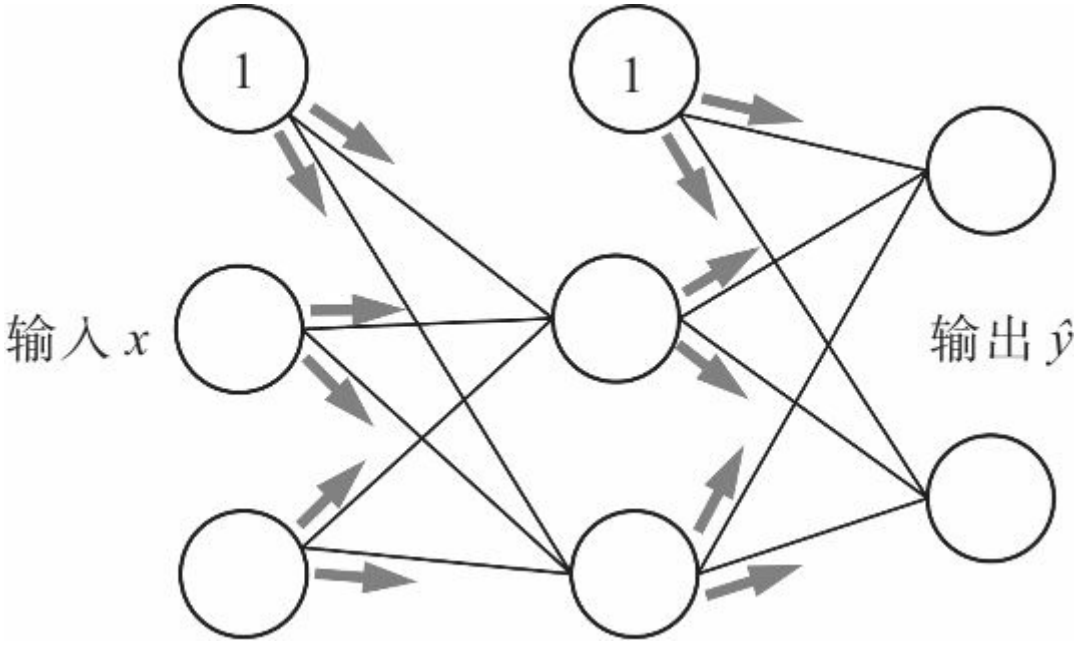
反向传播中从右向左传播错误。首先从计算输出层的误差向量开始：

$$\boldsymbol{\delta}^{(out)} = \boldsymbol{a}^{(out)} - \boldsymbol{y}$$

这里y为真实分类标签的向量（在Neural-NetMLP代码中的相应变量名为sigma_out）。

接着计算隐藏层的错误项：

$$\boldsymbol{\delta}^{(h)} = \boldsymbol{\delta}^{(out)}\left(\boldsymbol{W}^{(out)^{\mathrm{T}}}\right) \odot \frac{\partial \phi\left(z^{(h)}\right)}{\partial z^{(h)}}$$




这里$\frac{\partial\phi\left(z^{(h)}\right)}{\partial z^{(h)}}$为S形激活函数的导数，在Neural-NetMLP代码的fit方法中这样计算sigmoid_derivative_h=a_h*（1.-a_h）：

$$\frac{\partial\phi(z)}{\partial z}=\left(a^{(h)}\odot\left(1-a^{(h)}\right)\right)$$

注意，符号⊙在这里表示元素层面上的乘法。

尽管继续讨论下面这些方程并不太重要，但你可能会对如何得到激活函数的导数感到好奇，为此下面将分步概述推导过程：

$$\phi'(z)=\frac{\partial}{\partial z}\left(\frac{1}{1+e^{-z}}\right)$$

$$=\frac{e^{-z}}{\left(1+e^{-z}\right)}$$

$$=\frac{1+e^{-z}}{\left(1+e^{-z}\right)^2}-\left(\frac{1}{\left(1+e^{-z}\right)}\right)^2$$

$$=\frac{1}{\left(1+e^{-z}\right)}-\left(\frac{1}{\left(1+e^{-z}\right)}\right)^2$$

$$=\phi(z)-\left(\phi(z)\right)^2$$

$$=\phi(z)\left(1-\phi(z)\right)$$

$$=a(1-a)$$

计算δ（h）层误差的矩阵（sigma_h）如下：

$$\delta^{(h)}=\delta^{(out)}\left(\boldsymbol{W}^{(out)}\right)^{\mathrm{T}}\odot\left(a^{(h)}\odot\left(1-a^{(h)}\right)\right)$$

要更好地理解如何计算δ（h）项，需要先详细了解模型。上面的公式用到了h×t维矩阵W（out）的转置矩阵（W（out））T。这里t为输出的分类标签数量，h为隐藏层的单元个数。n×t维的δ（out）矩阵与t×h维的（W（out））T矩阵相乘，结果是一个n×t维的矩阵，该矩阵与同样维度的S形导数在元素层面相乘从而得到δ（h）。

最终，在获得δ项之后，描述成本函数的具体推导如下：

$$\frac{\partial}{\partial w_{i,j}^{(out)}} J(\boldsymbol{W}) = a_j^{(h)} \delta_i^{(out)}$$

$$\frac{\partial}{\partial w_{i,j}^{(h)}} J(\boldsymbol{W}) = a_j^{(in)} \delta_i^{(h)}$$

下一步将要计算每层各节点偏导数之和以及下一层各节点误差之和。

然而，记得还要计算训练集中每个样本的$\triangle^{(l)}_{i,j}$，因此用像在代码NeuralNetMLP中那样的向量化来实现更加容易：

$$\Delta^{(h)} = \Delta^{(h)} + \left(\boldsymbol{A}^{(in)}\right)^{\mathrm{T}} \delta^{(h)}$$

$$\Delta^{(out)} = \Delta^{(out)} + \left(\boldsymbol{A}^{(h)}\right)^{\mathrm{T}} \delta^{(out)}$$

在求取了偏导数之和以后，可以添加正则化项如下：

$$\Delta^{(l)} := \Delta^{(l)} + \lambda^{(l)} \quad (\text{除偏置项以外})$$

前面两个公式分别对应NeuralNetMLP代码中的变量delta_w_h，delta_b_h，delta_w_out和delta_b_out。

最后，在梯度计算完成之后，可以通过向每层l梯度相反的方向走一步来更新权重。

$$\boldsymbol{W}^{(l)} := \boldsymbol{W}^{(l)} - \eta \Delta^{(l)}$$

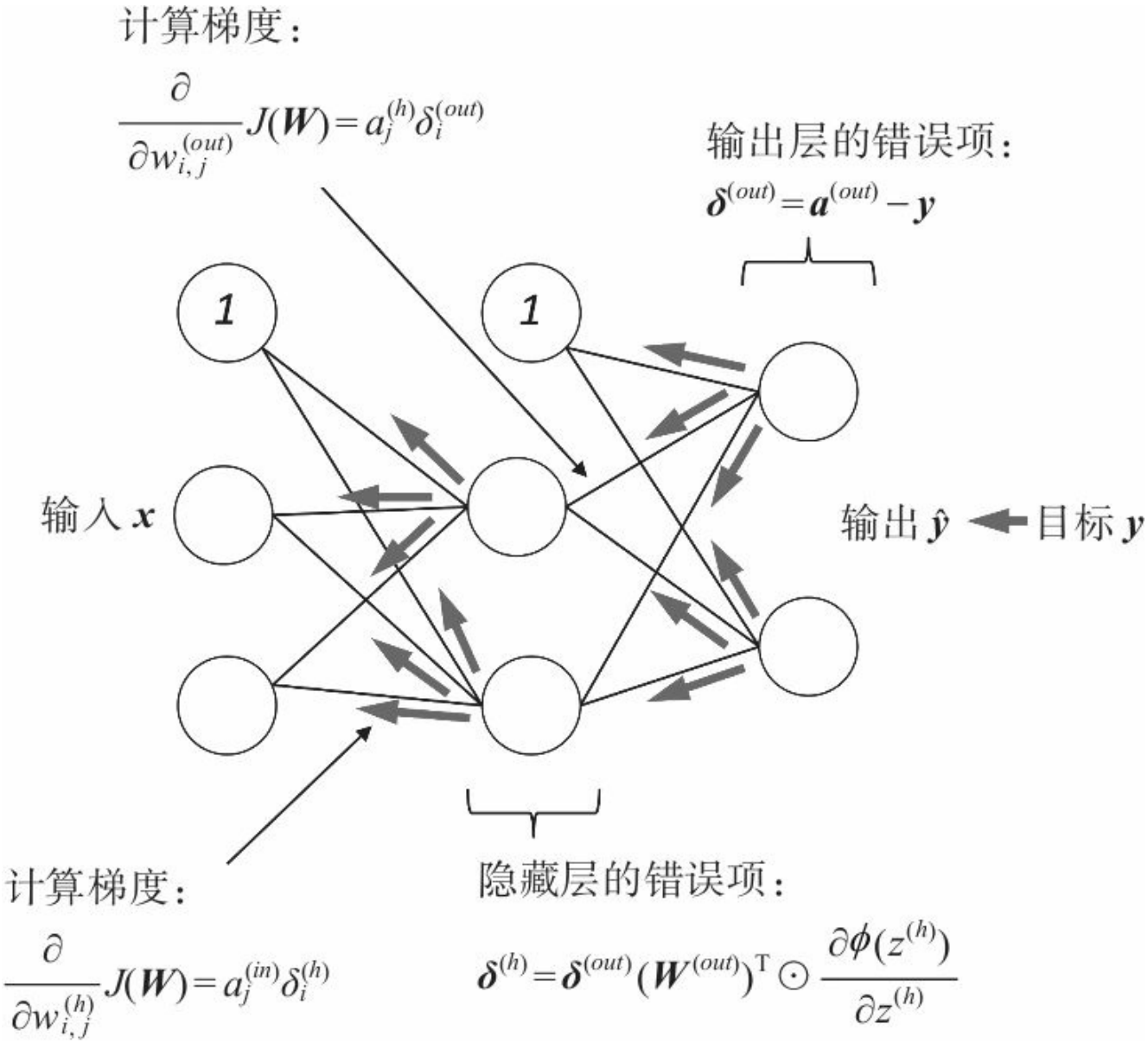
具体实现如下：

```
self.w_h -= self.eta * delta_w_h
self.b_h -= self.eta * delta_b_h
self.w_out -= self.eta * delta_w_out
self.b_out -= self.eta * delta_b_out
```

下图为反向传播的总结，它把本章讨论过的相关内容串在了一起：

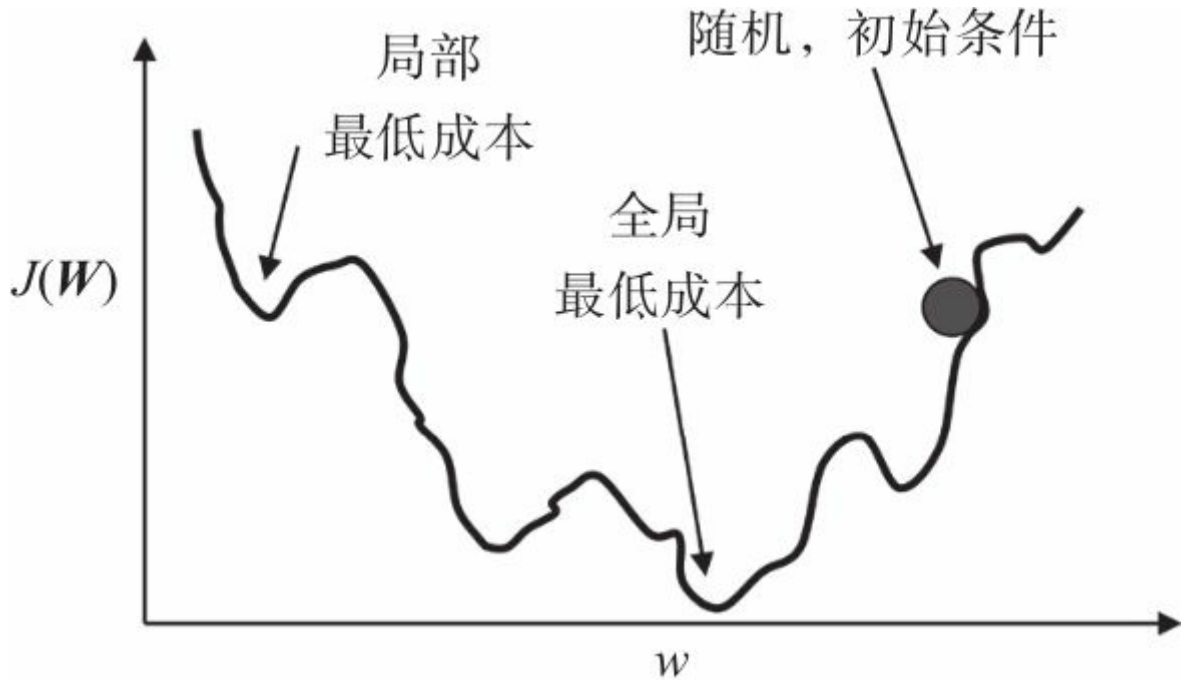
## 12.4 关于神经网络的收敛性

你可能想知道为什么不用正则梯度下降，而用小批量学习来训练神经网络识别手写数字。你可能还记得关于用随机梯度下降来实现在线学习的讨论。在线学习每次基于单个训练样本（k=1）来计算梯度以更新权重。虽然这是一个随机方法，但它往往会带来非常精确的解决方案，其收敛速度远比常规的梯度下降更快。小批量学习是随机梯度下降的一种特殊形式，在1＜k＜n的情况下，基于n个训练样本的子集k来计算梯度。小批量学习比在线学习更有优势，可以利用向量化提高计算效率。比常规梯度下降更快地更新权重。直观地说，可以把小批量学习看作是根据某组投票情况来预测总统选举的结果，仅询问有代表部分选民的投票情况，而不是对全部人口做调查（这相当于实际选举）。




多层神经网络比像Adaline、逻辑回归或者支持向量机这样简单的算法

更难训练。在多层神经网络通常需要优化数百、数千、甚至数十亿的权重。不幸的是输出函数的表面粗糙，优化算法很容易陷入局部极小的情况，如下图所示。




注意，因为神经网络有很多维度，所以表示极其简单，不可能把实际成本的表面可视化。这里只显示x轴上的一个权重成本表面。然而，关键是不希望算法陷入局部极小。通过增加学习率可以更容易逃离这种局部极小值。另一方面，如果学习速率太大，也会增加忽略全局最大值的机会。由于随机初始化权重，我们首先从解决一个通常无可救药错误的优化问题开始。

## 12.5 关于神经网络实现的最后几句话

你可能会问，既然要分辨手写数字，为什么不用开源的Python机器学习库，偏要学习这些理论来实现简单的多层人工神经网络？事实上，下一章将引入更复杂的开源软件TensorFlow来训练神经网络模型（https://www.tensorflow.org）。虽然从头实现初看起来似乎有点乏味，但是了解反向传播和神经网络训练的基本概念大有好处，对算法有基本的了解对于适当和成功地运用机器学习技术至关重要。

现在已经学会了前馈神经网络的工作原理，下一步准备探索更复杂的深度神经网络，如TensorFlow和Keras（https://keras.io），这些技术可以更加有效地构建神经网络，第13章将会详细讨论。从2015年11月发布迄今的两年时间里，TensorFlow已经在机器学习研究人员当中受到广泛的欢迎，因为它有能力优化数学表达式进而利用图形处理单元（GPU）计算多维阵列，所以研究人员用它来构建深度神经网络。可以把TensorFlow当成是底层的深度学习库，现在已经为TensorFlow开发出像Keras这样的简易API，这使构建常见的深度学习模型更为方便，我们将在第13章中学习。

## 12.6 小结

本章学习了多层人工神经网络的基本概念，这是当前机器学习研究中最热门的课题。从第2章的单层神经网络开始，现在已经可以把多个神经元连接到强大的神经网络体系结构，以解决诸如手写数字识别这样的复杂问题。我们揭开了常见的反向传播算法的神秘外衣，该算法是构建深度学习的众多神经网络模型的基础。在学习了反向传播算法之后，本章已经为探索更复杂的深度神经网络体系结构做好了准备。本书剩下的几个章节将介绍面向深度学习的开源系统TensorFlow，以便更有效地实现和训练多层神经网络。

# 第13章　用TensorFlow并行训练神经网络

本章将在机器学习和深度学习的数学基础上进一步介绍TensorFlow。它是目前可用的最受欢迎的深度学习软件库，比之前的NumPy更高效地实现了神经网络。本章将开始使用TensorFlow，你可以看到它为提高模型训练性能所带来的显著好处。

本章将进入训练机器学习和深度学习的下一段旅程，主要涵盖下述几个方面：

·如何用TensorFlow提高模型训练的性能

·如何用TensorFlow编写优化的机器学习代码

·如何用TensorFlow的高级API构建多层神经网络

·为人工神经网络选择激活函数

·介绍封装了TensorFlow的高级应用接口Keras，学习如何用它非常方便地实现深度学习

## 13.1 TensorFlow与模型训练的性能

TensorFlow可以显著提高机器学习的速度。要理解其原理，可以从讨论在硬件上运行昂贵的计算时通常所遇到的性能挑战开始。

近年来计算机处理器性能的不断提高使训练更强大且更复杂的机器学习系统成为可能，因此也改善了机器学习模型的预测性能。即使是目前最便宜的台式计算机的处理器也有多个核。

另外，从前面章节中可以看到scikit-learn有许多功能可以把这些计算分散到多个处理器。但在默认情况下，Python受限于全局解释器锁（GIL），仅能在单核上执行代码。诚然可以调用多处理器软件库把计算分发到多个核，但仍要考虑即使最先进桌面电脑的配置也很少超过8或16个核。

第12章实现了一个非常简单的多层感知器，它包含由100个单元组成的单个隐藏层，为此必须优化大约80000个权重参数（[784*100+100]+[100]*10+10=79510）才能训练出一个非常简单的图像识别模型。MNIST的图像相当小（28 x 28像素），如果要添加额外的隐藏层或处理更高像素密度的图像，就能想象到参数量的大爆炸。

这样的任务会很快会因为单处理器而变得不可行。那么焦点就变成：怎样才能更有效地解决这些难题呢？

显然解决这个问题的正确方案是使用GPU（图形处理器）。可以把图形处理器看作是计算机内部的一个小型计算机集群。与最先进的处理器相比，现代GPU的另一个优势是相对比较便宜，这可以从下面的对比中看出来：

| 配置 | Intel® Core™ i7-6900K Processor Extreme Ed. | NVIDIA GeForce® GTX™ 1080 Ti |
|---|---|---|
| 基础时钟频率 | 3.2 GHz | <1.5 GHz |
| 核 | 8 | 3584 |
| 内存带宽 | 64 GB/s | 484 GB/s |
| 浮点计算 | 409 GFLOPS | 11300 GFLOPS |
| 成本 | ~$1000.00 | ~$700.00 |

表中信息来源于下述网站：

·https://www.intel.com/content/www/us/en/products/processors/core/x-series/i7-6900k.html

·https://www.nvidia.com/en-us/geforce/products/10series/geforce-gtx-1080-ti/

（日期：2017年8月）

一个GPU的成本相当于一个现代CPU价格的70%，但却比CPU多450倍以上的核，并且每秒能够多处理大约15倍的浮点运算。那么还有什么因素阻碍我们利用GPU来解决图机器学习的任务呢？

我们所面临的挑战是编写针对GPU的代码，这并不像在解释器上执行Python代码那么简单。需要像CUDA和OpenCL这样的特殊软件包才能与GPU交互。然而，用CUDA或OpenCL编写代码，可能并不是实现和运行机器学习算法最方便的环境。好消息是这正是研发TensorFlow的目的之所在！

### 13.1.1 什么是TensorFlow

TensorFlow是实现和运行机器学习算法的编程接口，具有可扩展和跨平台的优点，包括了为深度学习特别准备的方便封装。

TensorFlow由谷歌大脑团队的研究人员和工程师开发。尽管研发主要是由谷歌的研究人员和软件工程师带领，但其发展也涉及许多来自于开源社区的贡献。TensorFlow最初构建时仅在谷歌内部使用，后来在2015年11月对外公布了源代码，该软件受宽松的开放源代码许可证的保护。

为了提高训练机器学习模型的性能，TensorFlow同时支持CPU和GPU。然而，最佳性能是在使用GPU的时候。TensorFlow正式支持有CUDA处理能力的GPU。对有OpenCL处理能力设备的支持仍然在实验中。应该会在不久的将来正式支持OpenCL。

TensorFlow的前端接口目前支持多种编程语言。作为Python用户，幸运的是TensorFlow的Python API是目前最为完整的，因此吸引了众多机器学习和深度学习的实践者。此外，TensorFlow有C++的官方API。

像Java、Haskell、Node.js和Go等其他语言的API尚不稳定，开源社区和TensorFlow研发人员正在不断地改善。TensorFlow的计算依靠构建计算图来表示数据流。尽管构建计算图听起来可能有点复杂，但是TensorFlow本身自带的高级API使其变得很容易。

### 13.1.2 如何学习TensorFlow

首先学习关于TensorFlow底层API的所有知识。虽然刚开始在这个层次上实现模型可能会有点儿麻烦，但底层API的好处在于它提供了更多的灵活性，使程序员能够组合基本操作从而研发出复杂的机器学习模型。从TensorFlow版本1.1.0开始，在底层API的基础上添加了高级API（例如，所谓的层和评估API），这样就可以快速构建模型的原型。

在了解了底层API之后，我们将进一步探索两个高级的API，它们分别是TensorFlow Layers和Keras。但是，先从TensorFlow的底层API开始迈出第一步，以便理解其工作原理。

### 13.1.3 学习TensorFlow的第一步

本节将从TensorFlow的底层API开始迈出第一步。取决于系统的设置，通常只需要用Python的pip安装器就可以在终端上执行以下命令从PyPI上安装TensorFlow：

```
pip install tensorflow
```

如果打算使用GPU，那么就需要安装CUDA工具包以及NVIDIA的cuDNN库，然后你就可以执行下述命令安装TensorFlow的GPU的支持软件：

```
pip install tensorflow-gpu
```

TensorFlow目前尚处在积极发展阶段。因此，每隔几个月便会有包含重大变化的新版本发布。在写作本章的时候，TensorFlow的最新版本是1.3.0。你可以用如下的命令从终端验证TensorFlow的版本：

```
python -c 'import tensorflow as tf; print(tf.__version__)'
```

如果在安装过程中遇到问题，可以去下述网站阅读更多有关特定系统和平台的相关信息：

https://www.tensorflow.org/install/

注意本章的所有代码都可以在CPU上运行，使用GPU完全是个可选项，我们推荐使用GPU以充分享受TensorFlow的好处。如果你有图形卡，那么请参阅安装页面以完成适当的设置。此外，你会发现下述网站的TensorFlow-GPU设置指南很有帮助，因为它对在Ubuntu上如何安装NVIDIA图形卡的驱动程序、CUDA和cuDNN做了非常好的解释（非必需但却是在GPU上运行TensorFlow的推荐）：

https://sebastianraschka.com/pdf/books/dlb/appendix_h_cloud-computing.pdf

TensorFlow围绕着由一组节点组成的计算图构建。每个节点代表一个操作，该操作可能有零个或多个输入或输出。流经计算图边缘的值被称为张量。

可以把张量理解为一个抽象的标量、向量、矩阵等。更具体地说，一个标量可以定义为rank-0张量，一个矢量可以定义为rank-1张量，一个矩阵可以定义为rank-2张量，一个三维堆叠矩阵定义为rank-3张量。

建立了计算图之后，就可以在TensorFlow的会话中启动该图，并对图中的不同节点进行处理。第14章将对如何构建计算图以及在会话中启动计算图进行更详细的讨论。

作为热身运动，我们将使用TensorFlow的简单标量来计算一维数据集中权重为w偏置为b的样本点x的净输入z：

$$z = w \times x + b$$

下面的代码显示了如何用TensorFlow底层的API来实现该等式。

```
import tensorflow as tf

## create a graph
g = tf.Graph()
with g.as_default():
    x = tf.placeholder(dtype=tf.float32,
                       shape=(None), name='x')
    w = tf.Variable(2.0, name='weight')
    b = tf.Variable(0.7, name='bias')

    z = w*x + b

    init = tf.global_variables_initializer()
## create a session and pass in graph g
with tf.Session(graph=g) as sess:
    ## initialize w and b:
    sess.run(init)
    ## evaluate z:
    for t in [1.0, 0.6, -1.8]:
        print('x=%4.1f --> z=%4.1f'%(
              t, sess.run(z, feed_dict={x:t})))
```

执行前面的代码将看到以下的输出：

```
x= 1.0 --> z= 2.7
x= 0.6 --> z= 1.9
x=-1.8 --> z=-2.9
```

觉得直截了当，对吧？一般来说，当用TensorFlow的底层API研发模型时，首先要定义输入数据的占位符（x，y，以及其他的可调参数）。然后，定义权重矩阵并构建从输入到输出的模型。如果是优化问题，那就应该先定义损失或成本函数，然后决定应该采用哪种优化算法。TensorFlow将创建一个包含所有以定义的符号为节点的图。

这里首先创建了一个占位符x，其中shape=（None）。然后逐个元素输入数值并作为一批同时输入的数据，如下所示：

```
>>> with tf.Session(graph=g) as sess:
...     sess.run(init)
...     print(sess.run(z, feed_dict={x:[1., 2., 3.]}))

[ 2.70000005  4.69999981  6.69999981]
```

请注意，为提高较长代码示例的可读性，本章的几个地方省略了Python命令行的提示，以避免不必要的文字折行，原因是TensorFlow的函数和方法名可以很冗长。

同时也可以参考下述网站（https://www.tensorflow.org/community/style_guide），尽管TensorFlow的官方风格指南推荐使用两个字符间距作为代码的缩进。然而，我们却选择用四个字符的缩进，这更符合Python的官方风格，也有助于在很多文本编辑器以及伴随本书的Jupyter代码笔记本上正确地突出显示代码的语法：

https://github.com/rasbt/python-machine-learning-book-2nd-edition

### 13.1.4 使用阵列结构

让我们来讨论一下如何在TensorFlow上使用阵列结构。执行下面的代码将创建一个简单的大小为batchsize×2×3的三阶张量，先对其进行重塑，然后用TensorFlow优化的表达式来计算列的和。由于事先并不知道批的大小，所以在占位符x的参数shape中定义批的大小为None：

```
import tensorflow as tf
import numpy as np

g = tf.Graph()
with g.as_default():
    x = tf.placeholder(dtype=tf.float32,
                       shape=(None, 2, 3),
                       name='input_x')

    x2 = tf.reshape(x, shape=(-1, 6),
                    name='x2')

    ## calculate the sum of each column
    xsum = tf.reduce_sum(x2, axis=0, name='col_sum')

    ## calculate the mean of each column
    xmean = tf.reduce_mean(x2, axis=0, name='col_mean')

with tf.Session(graph=g) as sess:
    x_array = np.arange(18).reshape(3, 2, 3)

    print('input shape: ', x_array.shape)
    print('Reshaped:\n',
          sess.run(x2, feed_dict={x:x_array}))
    print('Column Sums:\n',
          sess.run(xsum, feed_dict={x:x_array}))
    print('Column Means:\n',
          sess.run(xmean, feed_dict={x:x_array}))
```

执行上述代码后输出如下的结果：

```
input shape:  (3, 2, 3)
Reshaped:
 [[  0.   1.   2.   3.   4.   5.]
  [  6.   7.   8.   9.  10.  11.]
  [ 12.  13.  14.  15.  16.  17.]]

Column Sums:
  [ 18.  21.  24.  27.  30.  33.]

Column Means:
  [  6.   7.   8.   9.  10.  11.]
```

这个例子用到了三个函数：tf.reduce_sum、tf.reshape和tf.reduce_mean。请注意，重塑后第一个维度的值为-1，这是因为我们不知道批的大小。当对张量进行重塑时，如果某个特定维度的值为-1，那么该维度的大小将根据张量的总规模及其他维度的大小进行计算。因此可以通过调用tf.reshape（tensor，shape=（-1，））来扁平化张量。可以随时从下述网站阅读官方文档来了解TensorFlow的功能：

https://www.tensorflow.org/api_docs/python/tf

### 13.1.5 用TensorFlow的底层API开发简单的模型

现在已经熟悉了TensorFlow，让我们通过一个非常实际的例子来实现普通最小二乘法（OLS）回归。如果想要对回归分析做个快速回顾，请参阅第10章。

首先创建一个小的一维试验数据集，其中包括10个训练样本：

```
>>> import tensorflow as tf
>>> import numpy as np
>>>
>>> X_train = np.arange(10).reshape((10, 1))
>>> y_train = np.array([1.0, 1.3, 3.1,
...                     2.0, 5.0, 6.3,
...                     6.6, 7.4, 8.0,
...                     9.0])
```

基于该数据集训练线性回归模型，根据输入的x值预测输出值y。在名为TfLinreg的类中实现该模型。为此需要两个占位符，一个是输入x，另一个是把数据提供给模型的y。接着需要定义可训练的变量，即权重w和偏差b。

然后，定义线性回归模型为z=w×x+b，把成本函数定义为平方差均值（MSE）。用梯度下降优化器来学习模型的权重参数，代码如下：

```
class TfLinreg(object):
    def __init__(self, x_dim, learning_rate=0.01,
                 random_seed=None):
        self.x_dim = x_dim
        self.learning_rate = learning_rate
        self.g = tf.Graph()
        ## build the model
        with self.g.as_default():
            ## set graph-level random-seed
            tf.set_random_seed(random_seed)

            self.build()
            ## create initializer
            self.init_op = tf.global_variables_initializer()

    def build(self):
        ## define placeholders for inputs
        self.X = tf.placeholder(dtype=tf.float32,
                                shape=(None, self.x_dim),
                                name='x_input')
        self.y = tf.placeholder(dtype=tf.float32,
                                shape=(None),
                                name='y_input')
  print(self.X)
  print(self.y)
  ## define weight matrix and bias vector
  w = tf.Variable(tf.zeros(shape=(1)),
                  name='weight')
  b = tf.Variable(tf.zeros(shape=(1)),
                  name="bias")
  print(w)
  print(b)

  self.z_net = tf.squeeze(w*self.X + b,
                          name='z_net')
  print(self.z_net)

  sqr_errors = tf.square(self.y - self.z_net,
                         name='sqr_errors')
  print(sqr_errors)
  self.mean_cost = tf.reduce_mean(sqr_errors,
                                  name='mean_cost')

  optimizer = tf.train.GradientDescentOptimizer(
              learning_rate=self.learning_rate,
              name='GradientDescent')
  self.optimizer = optimizer.minimize(self.mean_cost)
```

到目前为止，定义了构建模型的类。接下来将创建该类的一个实例lrmodel：

```
>>> lrmodel = TfLinreg(x_dim=X_train.shape[1], learning_rate=0.01)
```

build方法中的print语句将显示计算图中的x，y，w，b，z_net和sqr_errors 6个节点的名称及其形状。

可以根据实际需要选择print语句。然而，检查变量形状对调试复杂模型很有帮助。在构建模型时显示以下的信息：

```
Tensor("x_input:0", shape=(?, 1), dtype=float32)
Tensor("y_input:0", dtype=float32)
<tf.Variable 'weight:0' shape=(1,) dtype=float32_ref>
<tf.Variable 'bias:0' shape=(1,) dtype=float32_ref>
Tensor("z_net:0", dtype=float32)
Tensor("sqr_errors:0", dtype=float32)
```

下一步将实现训练函数来学习线性回归模型的权重。注意b为偏置单元（x=0时y轴的截距）。

我们为训练专门实现了一个独立的函数，它需要TensorFlow会话，以模型实例、训练数据和迭代次数作为输入参数。该函数首先调用模型中定义的init_op初始化TensorFlow会话变量。然后，随着训练数据的不断提供，迭代并调用模型的optimizer操作。该函数将把训练成本作为副产品返回：

```
def train_linreg(sess, model, X_train, y_train, num_epochs=10):
    ## initialiaze all variables: W and b
    sess.run(model.init_op)

    training_costs = []

 for i in range(num_epochs):
     _, cost = sess.run([model.optimizer, model.mean_cost],
                        feed_dict={model.X:X_train,
                                   model.y:y_train})
     training_costs.append(cost)

 return training_costs
```

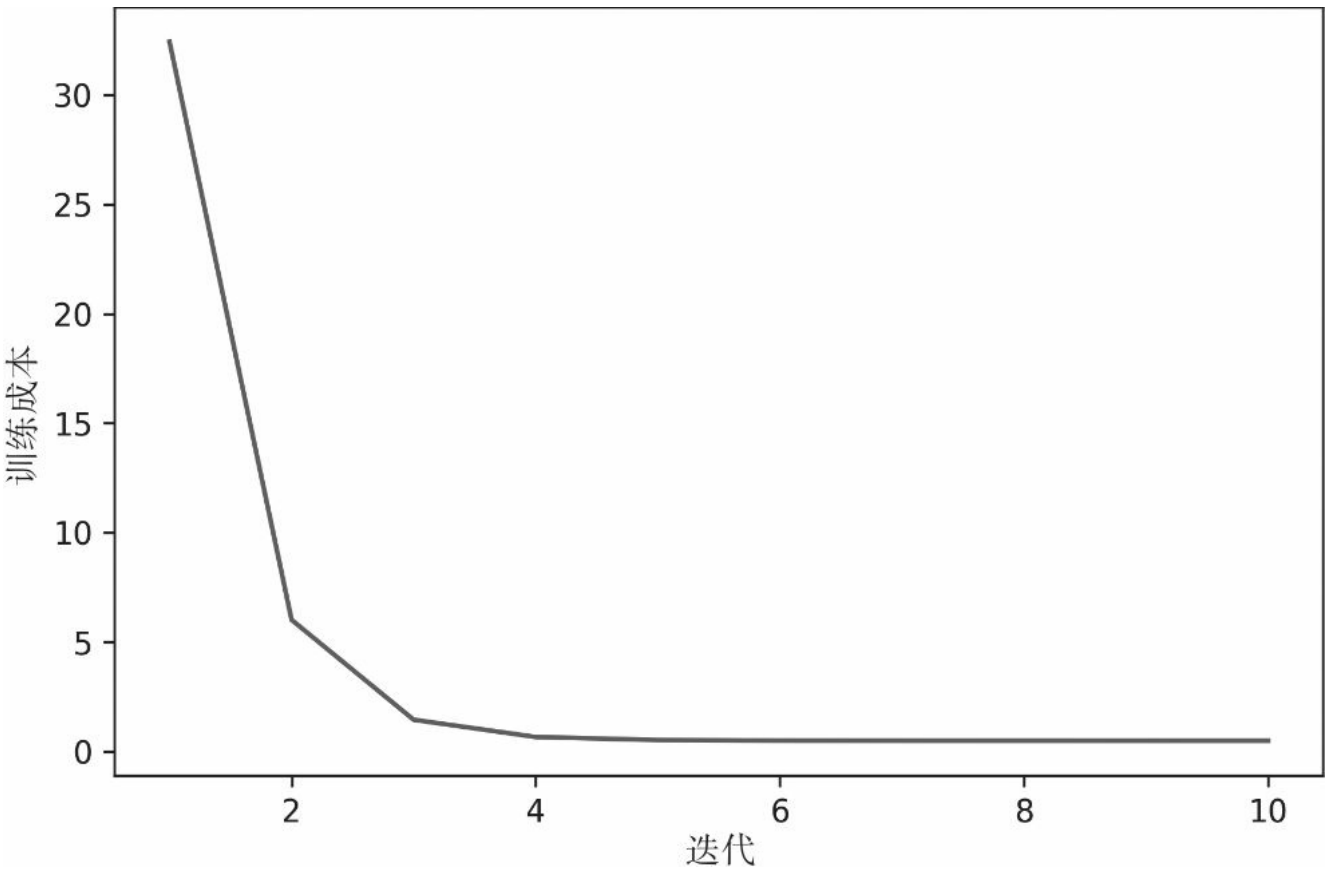
现在可以创建一个新的TensorFlow会话来启动lrmodel.g计算图，并把所有必需的参数传给train_linreg函数来进行训练：

```
>>> sess = tf.Session(graph=lrmodel.g)
>>> training_costs = train_linreg(sess, lrmodel, X_train, y_train)
```

可视化10个迭代后的训练成本以检查该模型是否已经收敛：

```
>>> import matplotlib.pyplot as plt
>>> plt.plot(range(1,len(training_costs) + 1), training_costs)
>>> plt.tight_layout()
>>> plt.xlabel('Epoch')
>>> plt.ylabel('Training Cost')
>>> plt.show()
```

从下图可以看到这个简单的模型在几次迭代后已经快速收敛：



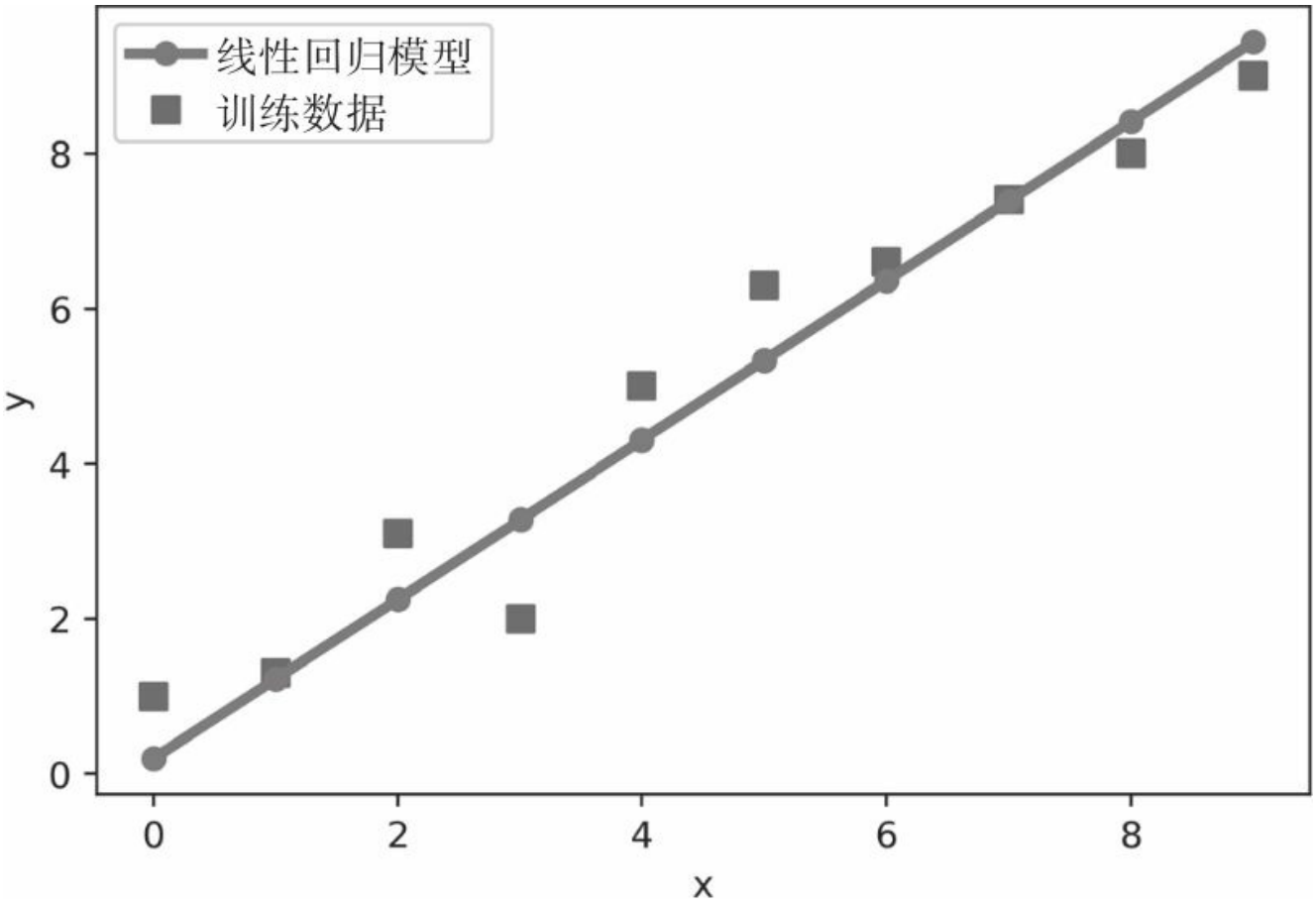

到现在为止，一切顺利。从成本函数看，似乎已经在这个特定数据集的基础上成功构建了回归模型。现在要建立一个新函数，根据输入样本进行预测。该函数需要TensorFlow会话、模型以及测试集：

```
def predict_linreg(sess, model, X_test):
    y_pred = sess.run(model.z_net,
                      feed_dict={model.X:X_test})
    return y_pred
```

实现该预测函数相当简单。只需要运行图中定义的z_net计算预测的输出值。接着把拟合训练数据的线性回归绘制出来：

```
>>> plt.scatter(X_train, y_train,
...             marker='s', s=50,
...             label='Training Data')
>>> plt.plot(range(X_train.shape[0]),
...          predict_linreg(sess, lrmodel, X_train),
...          color='gray', marker='o',
...          markersize=6, linewidth=3,
...          label='LinReg Model')
>>> plt.xlabel('x')
>>> plt.ylabel('y')
>>> plt.legend()
>>> plt.tight_layout()
>>> plt.show()
```

从结果图可以看到模型很好地拟合了训练数据点：



## 13.2 用TensorFlow的高级API高效率地训练神经网络

本节将讨论TensorFlow的两个高级API，即Layers API（tensorflow.layers或tf.layers）和Keras API（tensorflow.contrib.keras）。

可以把Keras作为独立软件包进行安装。它支持Theano或TensorFlow后台（更多信息可以参见Keras的官方网站https://keras.io/）

然而，在TensorFlow版本1.1.0发布以后，Keras已经被作为TensorFlow中contrib的子模块。Keras非常有可能在不久后被移出试验性的contrib子模块成为TensorFlow的子模块。

### 13.2.1 用TensorFlow的Layers API构建多层神经网络

要想了解通过tensorflow.layers（tf.layers）的高级API进行神经网络训练，实现一个多层感知器来分辨MNIST数据集中的手写数字，前面的章节曾对此做过介绍。你可以从下述网站下载MNIST数据集：http://yann.lecun.com/exdb/mnist/，该数据集由下述四个部分组成：

·训练集图像：train-images-idx3-ubyte.gz（9.5 MB）

·训练集标签：train-labels-idx1-ubyte.gz（32 KB）

·测试集图像：t10k-images-idx3-ubyte.gz（1.6 MB）

·测试集标签：t10k-labels-idx1-ubyte.gz（8.0 KB）

请注意TensorFlow也以下述方式提供相同的数据集：

```
import tensorflow as tf
from tensorflow.examples.tutorials.mnist import input_data
```

也可以只用MNIST数据集作为外部数据来学习数据预处理的所有步骤。通过这种方式可以了解到针对自己的数据集需要做些什么。

在下载并对压缩文档解压之后，把这些文件放在当前工作目录下的mnist子目录，这样就可以调用load_mnist（path，kind）函数来加载训练集和测试集数据，该函数此前在第12章中实现过。

然后，会像下面代码所示加载数据集：

```
>>> ## loading the data
>>> X_train, y_train = load_mnist('./mnist/', kind='train')
>>> print('Rows: %d,  Columns: %d' %(X_train.shape[0],
...                                  X_train.shape[1]))
Rows: 60000,  Columns: 784
>>> X_test, y_test = load_mnist('./mnist/', kind='t10k')
>>> print('Rows: %d,  Columns: %d' %(X_test.shape[0],
...                                  X_test.shape[1]))
Rows: 10000,  Columns: 784
>>> ## mean centering and normalization:
>>> mean_vals = np.mean(X_train, axis=0)
>>> std_val = np.std(X_train)
>>>
>>> X_train_centered = (X_train - mean_vals)/std_val
>>> X_test_centered = (X_test - mean_vals)/std_val
>>>
>>> del X_train, X_test
>>>
>>> print(X_train_centered.shape, y_train.shape)
(60000, 784) (60000,)
>>> print(X_test_centered.shape, y_test.shape)
(10000, 784) (10000,)
```

现在可以开始建立模型了。首先从创建两个占位符开始，这两个占位符分别是tf_x和tf_y，然后构建一个多层感知器，这部分曾在第12章中讨论过。所不同的是它有三个完全连接层。

其他不同的地方包括用双曲正切激活函数取代隐藏层的逻辑单元（tanh），用softmax取代输出层的逻辑函数，并且添加了一个额外的隐藏层。

tanh和softmax函数是新的激活函数。下一节《多层网络激活函数的选择》将学习更多关于这些激活函数的知识。

```
import tensorflow as tf

n_features = X_train_centered.shape[1]
n_classes = 10
random_seed = 123
np.random.seed(random_seed)

g = tf.Graph()
with g.as_default():
    tf.set_random_seed(random_seed)
    tf_x = tf.placeholder(dtype=tf.float32,
                          shape=(None, n_features),
                          name='tf_x')

    tf_y = tf.placeholder(dtype=tf.int32,
                          shape=None, name='tf_y')
    y_onehot = tf.one_hot(indices=tf_y, depth=n_classes)

    h1 = tf.layers.dense(inputs=tf_x, units=50,
                         activation=tf.tanh,
                         name='layer1')

    h2 = tf.layers.dense(inputs=h1, units=50,
                         activation=tf.tanh,
                         name='layer2')

    logits = tf.layers.dense(inputs=h2,
                             units=10,
                             activation=None,
                             name='layer3')

    predictions = {
        'classes' : tf.argmax(logits, axis=1,
                              name='predicted_classes'),
        'probabilities' : tf.nn.softmax(logits,
                              name='softmax_tensor')
    }
```

接下来将定义成本函数，并为初始化模型变量增加一个操作符，另外为优化模型增加一个操作符：

```
## define cost function and optimizer:
with g.as_default():
    cost = tf.losses.softmax_cross_entropy(
            onehot_labels=y_onehot, logits=logits)

    optimizer = tf.train.GradientDescentOptimizer(
            learning_rate=0.001)

    train_op = optimizer.minimize(
            loss=cost)

    init_op = tf.global_variables_initializer()
```

在开始训练网络模型之前，需要一种生成大量数据的方法。为此实现了以下的函数，该函数将返回一个生成器：

```
def create_batch_generator(X, y, batch_size=128, shuffle=False):
    X_copy = np.array(X)
    y_copy = np.array(y)

    if shuffle:
        data = np.column_stack((X_copy, y_copy))
        np.random.shuffle(data)
        X_copy = data[:, :-1]
        y_copy = data[:, -1].astype(int)

    for i in range(0, X.shape[0], batch_size):
        yield (X_copy[i:i+batch_size, :], y_copy[i:i+batch_size])
```

下一步将创建一个新的TensorFlow会话，初始化网络中的所有变量并对其进行训练。同时也显示每次迭代后的平均训练损失，为后期监测学习过程做好准备：

```
>>> ## create a session to launch the graph
>>> sess =  tf.Session(graph=g)
>>> ## run the variable initialization operator
>>> sess.run(init_op)
>>>
>>> ## 50 epochs of training:
>>> for epoch in range(50):
...     training_costs = []
...     batch_generator = create_batch_generator(
...             X_train_centered, y_train,
...             batch_size=64)
...     for batch_X, batch_y in batch_generator:
...         ## prepare a dict to feed data to our network:
...         feed = {tf_x:batch_X, tf_y:batch_y}
...         _, batch_cost = sess.run([train_op, cost], feed_dict=feed)
...         training_costs.append(batch_cost)
...     print(' -- Epoch %2d  '
...           'Avg. Training Loss: %.4f' % (
...               epoch+1, np.mean(training_costs)
...     ))


 -- Epoch  1  Avg. Training Loss: 1.5573
 -- Epoch  2  Avg. Training Loss: 1.2532
 -- Epoch  3  Avg. Training Loss: 1.0854
 -- Epoch  4  Avg. Training Loss: 0.9738

 […]
 -- Epoch 49  Avg. Training Loss: 0.3527
 -- Epoch 50  Avg. Training Loss: 0.3498
```

训练的过程可能需要几分钟。最后可以用经过训练的模型对测试集进行预测：

```
>>> ## do prediction on the test set:
>>> feed = {tf_x : X_test_centered}
>>> y_pred = sess.run(predictions['classes'],
...                   feed_dict=feed)

>>>
>>> print('Test Accuracy: %.2f%%' % (
...       100*np.sum(y_pred == y_test)/y_test.shape[0]))

Test Accuracy: 93.89%
```

可以看到利用高级API可以快速地构建并测试模型。因此，高级API对建立原型验证想法以及快速检查结果都非常有用。

下一步将用Keras开发一个类似MNIST的分类模型，Keras是TensorFlow的另一个高级API。

## 13.2.2 用Keras研发多层神经网络

Keras的发展始于2015年初。时至今日，它已演变成一个最受欢迎和广泛使用的软件库，Keras建立在Theano和TensorFlow上。

与TensorFlow类似，Keras能够利用GPU来加速神经网络训练。其突出特点是它有非常直观和用户友好的API，可以只用几行代码实现神经网络。

Keras最初作为一个以Theano作为后台的独立的API发布，它对TensorFlow的支持是后来才加进去的。Keras从1.1.0版本开始融入TensorFlow。因此，如果你在用TensorFlow的1.1.0版本，就不需要再安装Keras。关于Keras的更多信息，请访问其官方网站：http://keras.io。

Keras目前是contrib模块的一部分（包含TensorFlow的贡献者提供的软件包，被认为是试验性的代码）。在TensorFlow的未来版本中，Keras有可能被移出，成为TensorFlow主要API中的独立模块。请访问TensorFlow官网以获得更多相关信息：https://www.tensorflow.org/api_docs/python/tf/contrib/keras。

请注意，在未来的TensorFlow版本中，下面的代码示例有可能要做改变，把import tensorflow.contrib.keras as keras变成import tensorflow.keras as keras。

接下来将通过示例代码分步地解释如何使用Keras。为了调用前面一节所描述的相同函数，需要将数据以下述方式加载：

```
>>> X_train, y_train = load_mnist('mnist/', kind='train')
>>> print('Rows: %d,  Columns: %d' %(X_train.shape[0],
...                                   X_train.shape[1]))
>>> X_test, y_test = load_mnist('mnist/', kind='t10k')
>>> print('Rows: %d,  Columns: %d' %(X_test.shape[0],
...                                   X_test.shape[1]))
Rows: 10000,  Columns: 784
>>>
>>> ## mean centering and normalization:
>>> mean_vals = np.mean(X_train, axis=0)
>>> std_val = np.std(X_train)
>>>
>>> X_train_centered = (X_train - mean_vals)/std_val
>>> X_test_centered = (X_test - mean_vals)/std_val
>>>
```

```
>>> del X_train, X_test
>>>
>>> print(X_train_centered.shape, y_train.shape)
(60000, 784) (60000,)
>>> print(X_test_centered.shape, y_test.shape)
(10000, 784) (10000,)
```

首先为NumPy和TensorFlow设置随机种子以确保可以得到一致的结果：

```
>>> import tensorflow as tf
>>> import tensorflow.contrib.keras as keras

>>> np.random.seed(123)
>>> tf.set_random_seed(123)
```

为了继续准备训练数据，需要把分类标签（整数0-9）转换为独热格式。幸运的是，Keras为此提供了一个非常方便的工具：

```
>>> y_train_onehot = keras.utils.to_categorical(y_train)
>>>
>>> print('First 3 labels: ', y_train[:3])
First 3 labels:  [5 0 4]
>>> print('\nFirst 3 labels (one-hot):\n', y_train_onehot[:3])
First 3 labels (one-hot):
 [[ 0.  0.  0.  0.  0.  1.  0.  0.  0.  0.]
  [ 1.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
  [ 0.  0.  0.  0.  1.  0.  0.  0.  0.  0.]]
```

现在我们可以看到有趣的部分并实现神经网络。简而言之，我们将有三个层，其中前两层各有50个隐藏层单元，它们以双曲正切为激活函数，最后一层有10层对应10种不同的分类标签，并采用softmax计算每个类的概率。Keras使这些任务变得异常简单，可以从下面的代码实现看到这一点：

```
model = keras.models.Sequential()

model.add(
    keras.layers.Dense(
        units=50,
        input_dim=X_train_centered.shape[1],
        kernel_initializer='glorot_uniform',
        bias_initializer='zeros',
        activation='tanh'))

model.add(
    keras.layers.Dense(
        units=50,
        input_dim=50,
        kernel_initializer='glorot_uniform',
        bias_initializer='zeros',
        activation='tanh'))

model.add(
    keras.layers.Dense(
        units=y_train_onehot.shape[1],
        input_dim=50,
        kernel_initializer='glorot_uniform',
        bias_initializer='zeros',
        activation='softmax'))


sgd_optimizer = keras.optimizers.SGD(
        lr=0.001, decay=1e-7, momentum=.9)

model.compile(optimizer=sgd_optimizer,
              loss='categorical_crossentropy')
```

首先用Sequential类来初始化一个新模型以实现前馈神经网络。然后可以随意添加很多层。然而，由于添加的第一层是输入层，所以必须确保input_dim属性与训练集中的特征数（列）相匹配（在神经网络实现中有784个特征或者像素）。

同时，必须确保相邻两层的输出单元个数（units）与输入单元个数（input_dim）相匹配。前面的例子添加了两个隐藏层，每个包含50个隐藏单元和1个偏置单元。输出层中的单元数应该等于独立分类标签的数量，即一个热编码分类标签阵列中的列数。

注意，通过设置kernel_initializer='glorot_uniform'，为权重矩阵增

加了新的初始化算法。Glorot初始化（又称为沙维尔初始化）是更可靠的深度神经网络初始化方法（沙维尔·格罗饶特和约书亚·本希奥.了解训练深度前馈神经网络的困难.人工智能和统计第9卷，2010年：249-256）

把偏置初始化为零比较常见。事实上，这是Keras的默认设置。第14章对权重值的初始化方案将进行详细的讨论。

在编译模型之前，还必须定义好优化器。前面的例子选择了随机梯度下降作为优化的算法，这在前面章节中已经熟悉了。此外，我们可以设置权重衰减常数和动量学习来调整每个迭代的学习速度，如第12章所讨论的那样。最后把成本（或损失）函数设置为categorical_crossentropy。

二进制交叉熵只是逻辑回归成本函数的一个技术术语，而分类交叉熵是通过softmax进行的多元预测泛化，本章后面《在多元分类中调用softmax函数评估类别概率》一节将涵盖该部分。

在编译模型之后，现在可以通过调用fit方法来训练。这里采用每批64个训练样本的小批量随机梯度算法。MLP的训练超过50次迭代，可以在训练过程中通过设置verbose=1来跟踪成本函数的优化。

参数validation_split尤其有用，因为它储备了10%的训练数据（该例有6000个样本）用于在每次迭代之后验证，这样就能够监测到模型在训练过程中是否出现过拟合问题：

```
>>> history = model.fit(X_train_centered, y_train_onehot,
...                     batch_size=64, epochs=50,
...                     verbose=1,
...                     validation_split=0.1)

Train on 54000 samples, validate on 6000 samples
Epoch 1/50
54000/54000 [==============================] - 3s - loss: 0.7247 -
val_loss: 0.3616
Epoch 2/50

54000/54000 [==============================] - 3s - loss: 0.3718 -
val_loss: 0.2815
Epoch 3/50
54000/54000 [==============================] - 3s - loss: 0.3087 -
val_loss: 0.2447

[…]
Epoch 50/50
54000/54000 [==============================] - 3s - loss: 0.0485 -
val_loss: 0.1174
```

在训练过程中打印成本函数的值非常有用。因为能够在训练过程中很快

地发现成本是否降低从而提早中止算法，否则就要调整超参数的值。

要预测分类标签，可以用predict_classes方法直接返回整数型的分类标签：

```
>>> y_train_pred = model.predict_classes(X_train_centered, verbose=0
>>> print('First 3 predictions: ', y_train_pred[:3])
First 3 predictions:  [5 0 4]
```

最后把模型在训练集和测试集上的准确度打印出来：

```
>>> y_train_pred = model.predict_classes(X_train_centered,
...                                      verbose=0)
>>> correct_preds = np.sum(y_train == y_train_pred, axis=0)
>>> train_acc = correct_preds / y_train.shape[0]
>>>
>>> print('First 3 predictions: ', y_train_pred[:3])
First 3 predictions:  [5 0 4]
>>>
>>> print('Training accuracy: %.2f%%' % (train_acc * 100))
Training accuracy: 98.88%
>>>
>>> y_test_pred = model.predict_classes(X_test_centered,
...                                     verbose=0)
>>> correct_preds = np.sum(y_test == y_test_pred, axis=0)
>>> test_acc = correct_preds / y_test.shape[0]
>>> print('Test accuracy: %.2f%%' % (test_acc * 100))
Test accuracy: 96.04%
```

注意，这只是一个没有优化调优参数的非常简单的神经网络。如果你有兴趣更多地尝试Keras，可以进一步调整学习速度、动量、质量衰减和隐藏单元数。

## 13.3 多层网络激活函数的选择

为了简单起见，我们仅在多层前馈神经网络中讨论了S形激活函数。在第12章的多层感知器实现中，曾在隐藏层和输出层中使用了S形激活函数。

虽然这种激活函数被称为S形函数（在文献中通常也这么称呼），但是更精确的定义是逻辑函数或负对数似然函数。在下面的小节中，你会了解更多关于各种S型函数用于多层神经网络实现的信息。

在技术上，可以用任何函数作为多层神经网络的激活函数，只要它是可微的。甚至可以用像Adaline这样的线性激活函数（见第2章）。然而，隐藏层和输出层在实践中使用线性激活函数的意义不太大，因为我们希望在典型的人工神经网络中引入非线性来处理复杂的问题。线性函数之和所产生的毕竟还只是线性函数。

第12章中用到的逻辑激活函数可能是最接近大脑的神经元概念，我们可以把它看作是神经元被激活的概率。

然而，如果有非常高的负输入，逻辑激活函数就可能会出现问题，因为在这种情况下，S形函数的输出将接近零。如果S形函数返回接近零的结果，那么神经网络的学习将变得非常慢，而且在训练中更容易陷入局部极小值的情况。这就是为什么人们通常希望在隐藏层中用双曲正切作为激活函数。

在讨论什么是双曲正切之前，先简要概括逻辑函数的基础概念，并了解一下泛化，这对多标签分类问题更为有用。

### 13.3.1 逻辑函数回顾

正如在本节介绍中提到的，通常把逻辑函数称为S形函数，实际上它是S形函数的特例。记得在第3章中的逻辑回归部分曾经讨论过，可以在二元分类任务中用逻辑函数建模预测样本x属于正类（1类）的概率。给定的净输入z显示在下面的等式中：

$$z = w_0 x_0 + w_1 x_1 + \cdots + w_m x_m = \sum_{i=0}^{m} w_i x_i = w^{\mathrm{T}} x$$

逻辑函数将进行下述计算：

$$\phi_{\text{logistic}}(z) = \frac{1}{1 + \mathrm{e}^{-z}}$$

注意，$w_0$为偏置单元（当$x_0$=1时的y轴截距）。给一个更具体的例子，假设有一个二维数据点x的模型和一个把下述权重系数分配给w向量的模型：

```
>>> import numpy as np

>>> X = np.array([1, 1.4, 2.5]) ## first value must be 1
>>> w = np.array([0.4, 0.3, 0.5])

>>> def net_input(X, w):
...     return np.dot(X, w)
...
>>> def logistic(z):
...     return 1.0 / (1.0 + np.exp(-z))
...
>>> def logistic_activation(X, w):
...     z = net_input(X, w)
...     return logistic(z)
...
>>> print('P(y=1|x) = %.3f' % logistic_activation(X, w))
P(y=1|x) = 0.888
```

如果计算净输入，并用它来激活具有那些特征值和权重系数的逻辑神经元，就会得到0.888的值，这可以解读为该特定样本x属于正类的概率为88.8%。

第12章用独热编码技术计算由多个逻辑激活单元所组成的输出层中的值。但是，正如下面的代码示例演示的那样，由多个逻辑激活单元组成的输出层不会产生有意义的、可解释的概率值：

```
>>> # W : array with shape = (n_output_units, n_hidden_units+1)
... #     note that the first column are the bias units
...
>>> W = np.array([[1.1, 1.2, 0.8, 0.4],
...               [0.2, 0.4, 1.0, 0.2],
...               [0.6, 1.5, 1.2, 0.7]])
>>>
>>> # A : data array with shape = (n_hidden_units + 1, n_samples)
... #     note that the first column of this array must be 1
...
>>> A = np.array([[1, 0.1, 0.4, 0.6]])
>>>
>>> Z = np.dot(W, A[0])
>>> y_probas = logistic(Z)
>>> print('Net Input: \n', Z)
Net Input:
 [ 1.78  0.76  1.65]
>>> print('Output Units:\n', y_probas)
Output Units:
 [ 0.85569687  0.68135373  0.83889105]
```

正如在输出中所看到的，不能把结果值解释为三元分类问题的概率。原因是这三个数值之和不等于1。然而，如果只用模型来预测分类标签，而并不预测类成员属于某个类的概率，那么实际上这就并不是个大问题。根据前面得到的输出单元预测分类标签的一种方法是使用最大值：

```
>>> y_class = np.argmax(Z, axis=0)
>>> print('Predicted class label: %d' % y_class)
Predicted class label: 0
```

在某些情况下，计算有意义的分类概率对多元分类预测很有用。下一节将讨论逻辑函数和softmax函数泛化问题，这对完成任务会有帮助。

## 13.3.2 在多元分类中调用softmax函数评估类别概率

上一节看到了如何使用argmax函数获得分类标签。softmax函数实际上是argmax函数的软形式。它提供每个类的概率，而不是给出一个类指标。因此可以在多元分类环境下计算有意义的分类概率（多项式逻辑回归）。

在softmax中，一个有净输入z的特定样本属于第i类的概率，可以用以归一化项为分母的公式来计算，即所有M个线性函数之和：

$$p(y=i|z)=\phi(z)=\frac{e^{z_i}}{\sum_{i=1}^{M} e^{z_i}}$$

可以用Python编写一段代码来看看softmax的实际应用：

```
>>> def softmax(z):
...     return np.exp(z) / np.sum(np.exp(z))
...
>>> y_probas = softmax(Z)
>>> print('Probabilities:\n', y_probas)

 Probabilities:
  [ 0.44668973   0.16107406   0.39223621]


 >>> np.sum(y_probas)
 1.0
```

正如所看到的，预测的分类概率之和现在如我们所愿达到1。值得注意的是预测的分类标签与把argmax函数应用到逻辑输出的结果相同。直观地说，把softmax函数作为归一化输出，对多元情况下获得有意义的类成员预测有用。

## 13.3.3 利用双曲正切拓宽输出范围

另外一个在人工神经网络的隐藏层常用到S形函数是双曲正切（也被称作tanh），可以理解为一种比例调整后的逻辑函数：

$$\phi_{\text{logistic}}(z) = \frac{1}{1+e^{-z}}$$

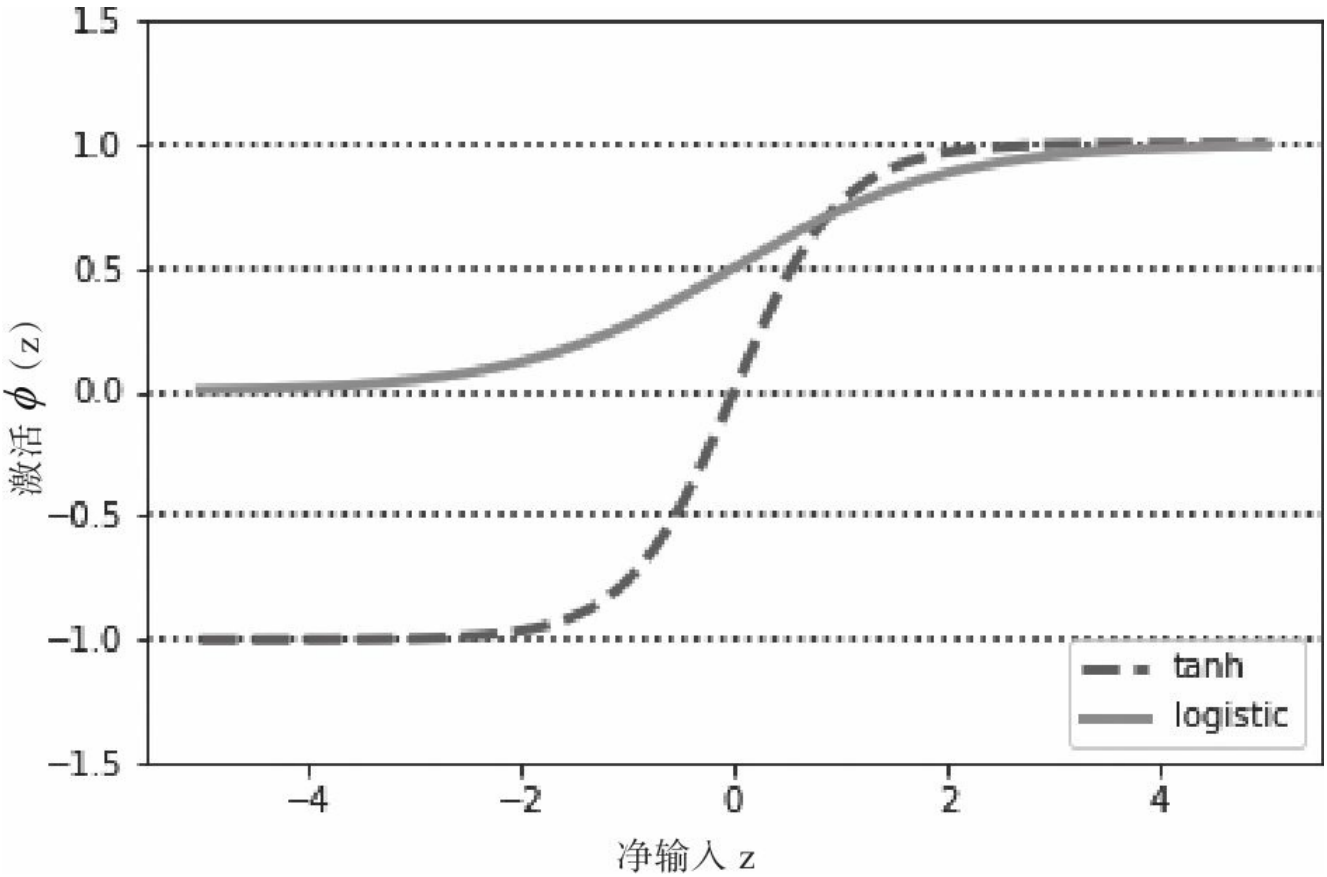
$$\phi_{\text{tanh}} = 2 \times \phi_{\text{logistic}}(2z) - 1 = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

与逻辑函数相比，双曲正切函数的优点是它有一个更广泛的输出范围，而且取值范围在开放的区间（-1，1），这有助于提高反向传播算法的收敛性（C.M.毕晓普.《模式识别的神经网络》.牛津大学出版社，1995：500-501）。

相反，逻辑函数返回一个在开放区间（0，1）的输出信号。为了直观地比较逻辑函数和双曲正切函数，我们将通过下述代码绘制两个S形函数：

```
>>> import matplotlib.pyplot as plt

>>> def tanh(z):
...     e_p = np.exp(z)
...     e_m = np.exp(-z)
...     return (e_p - e_m) / (e_p + e_m)

>>> z = np.arange(-5, 5, 0.005)
>>> log_act = logistic(z)
>>> tanh_act = tanh(z)
>>> plt.ylim([-1.5, 1.5])
>>> plt.xlabel('net input $z$')
>>> plt.ylabel('activation $\phi(z)$')
>>> plt.axhline(1, color='black', linestyle=':')
>>> plt.axhline(0.5, color='black', linestyle=':')
>>> plt.axhline(0, color='black', linestyle=':')
>>> plt.axhline(-0.5, color='black', linestyle=':')
>>> plt.axhline(-1, color='black', linestyle=':')
>>> plt.plot(z, tanh_act,
...          linewidth=3, linestyle='--',
...          label='tanh')
>>> plt.plot(z, log_act,
...          linewidth=3,
...          label='logistic')
>>> plt.legend(loc='lower right')
>>> plt.tight_layout()
>>> plt.show()
```

可以看到，这两个S形曲线的形状非常相似，但双曲正切函数的输出空间是逻辑函数的两倍：




请注意之所以要详尽地实现逻辑函数和双曲正切函数是为了能清楚地说明和解释。可以在实践中用NumPy的tanh函数来获得相同的结果。

```
>>> tanh_act = np.tanh(z)
```

此外可以从SciPy的专用模块直接倒入逻辑函数：

```
>>> from scipy.special import expit
>>> log_act = expit(z)
```

### 13.3.4 修正线性单元激活函数

修正线性单元（ReLU）是另一个在深度神经网络中常用到的激活函数。在了解ReLU之前，应该先退一步，理解双曲正切激活函数和逻辑激活函数中出现的梯度消失问题。

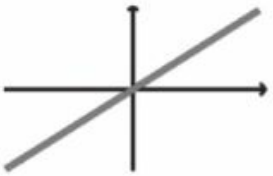
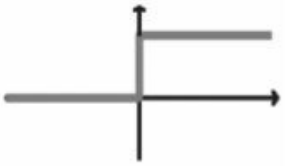
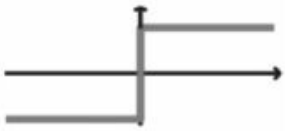
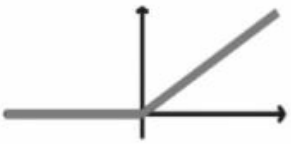
为了解释这个问题，假设最初的净输入$z_1$=20，之后变成$z_2$=25。计算双曲正切激活值得到（$z_1$）≈φ（$z_2$）≈1.0，这表明结果没变。

这意味着随着z值变大，与净输入相对应的激活值导数减小。因此，训练阶段的学习权重变得非常缓慢，因为梯度项可能非常接近于零。ReLU激活函数解决了这个问题。定义ReLU的数学形式如下：

$$\phi(z)=\max(0, z)$$

ReLU仍然是非线性函数，但它对学习神经网络的复杂函数有益。除此之外，对于与输入相对应ReLU导数，正面输入值的结果总是1。因此解决了梯度消失的问题，适用于深层神经网络。下一章将用ReLU作为多层卷积神经网络的激活函数。

了解了人工神经网络中常用的激活函数之后，现在概述本书中所遇到的不同激活函数来结束本节：

| 激活函数 | 方程 | 示例 | 一维图 |
| --- | --- | --- | --- |
| 线性 | $\phi(z) = z$ | Adaline，线性回归 |  |
| 单位阶跃（Heaviside 函数） | $\phi(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$ | 感知器变体 |  |
| 符号（signum） | $\phi(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$ | 感知器变体 |  |
| 分段线性 | $\phi(z) = \begin{cases} 0 & z \leq -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \leq z \leq \frac{1}{2} \\ 1 & z \geq \frac{1}{2} \end{cases}$ | 支持向量机 | |
| 逻辑（sigmoid） | $\phi(z) = \frac{1}{1 + e^{-z}}$ | 逻辑回归<br>多层神经网络 | |
| 双曲正切（tanh） | $\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$ | 多层神经网络，RNN | |
| ReLU | $\phi(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$ | 多层神经卷积神经网络 |  |

## 13.4 小结

这一章学会了如何使用开源的TensorFlow在深度学习中进行数值计算。虽然因为TensorFlow支持GPU所带来的额外复杂性使其用起来不如NumPy方便，但是它可以非常有效地定义和训练大型多层神经网络。

同样你也学习了如何调用TensorFlow的API来构建复杂的机器学习和神经网络模型并有效地运行。我们首先探讨了TensorFlow的低级API编程。当为矩阵向量乘法编程并定义每个操作的细节时，调用低级API建模可能会很麻烦。然而好处是它允许研发人员组合这些基本操作以构建更复杂的模型。此外也讨论了TensorFlow如何利用GPU来加快大型神经网络训练和测试的计算速度。如果不用GPU，训练网络通常可能需要几个月的计算时间！

然后探索了两个高级API，用它们构建神经网络模型要比低级API容易得多。具体来说，使用TensorFlow的Layers和Keras建立多层神经网络，并学习如何使用这些API来建立模型。

最后，理解了不同的激活函数及其行为和应用。具体地说，本章讨论了tanh、softmax、和ReLU。第12章从实现简单的多层感知器（MLP）识别MNIST手写图像数据集开始。而从零开始的底层实现有助于说明多层神经网络的核心概念，如正向和反向传播的神经网络，用NumPy训练神经网络非常低效而且不切合实际。

下一章将继续我们的旅程，深入学习TensorFlow，包括处理图像和会话对象。这将学到很多新概念，如占位符、变量以及如何保存和恢复TensorFlow的模型。

# 第14章　深入探讨TensorFlow的工作原理

第13章用TensorFlow Python API的各种功能训练多层感知器来识别MNIST的手写数字。这是直接动手深入学习以TensorFlow训练神经网络以及机器学习的非常好的方式。

本章将把重点转移到TensorFlow本身，详细地探讨TensorFlow所能提供的令人印象深刻的过程和功能，主要涵盖下述几个方面：

·TensorFlow的主要功能和优点

·TensorFlow的排序与张量

·理解与使用TensorFlow的计算图

·使用TensorFlow的变量

·TensorFlow在不同区间的操作

·常用的张量变换：排序、形状和类型

·把张量转换为多维阵列

·在TensorFlow中存储和恢复模型

·用TensorBoard实现神经网络图的可视化

本章仍注重实战，通过实现图探讨TensorFlow的主要功能和概念，也会在该过程中重新讨论回归模型，用TensorBoard探讨神经网络的可视化，并推荐一些图可视化方法来进行更多的探索。

## 14.1 TensorFlow的主要功能

TensorFlow提供了可扩展和多平台的编程接口来实现并运行机器学习算法。从2017年1.0版发布以来，TensorFlow API已经相对稳定和成熟。虽然目前也有其他的深度学习软件库可用，但与TensorFlow API相比尚不成熟。

在第13章中，我们已经注意到TensorFlow有一个关键的功能，它具有使用单个或多个GPU工作的能力。这允许用户在大型系统上非常有效地训练机器学习模型。

TensorFlow由谷歌资助和支持，具有较强的增长动力。谷歌有一个庞大的软件工程师团队在不断地对其进行改进。TensorFlow也获得了来自于开源研发者的大力支持，他们热心地为用户提供反馈。这使TensorFlow对学术研究人员和企业开发人员更有用，也为TensorFlow带来大量的文档和教程以帮助新用户。

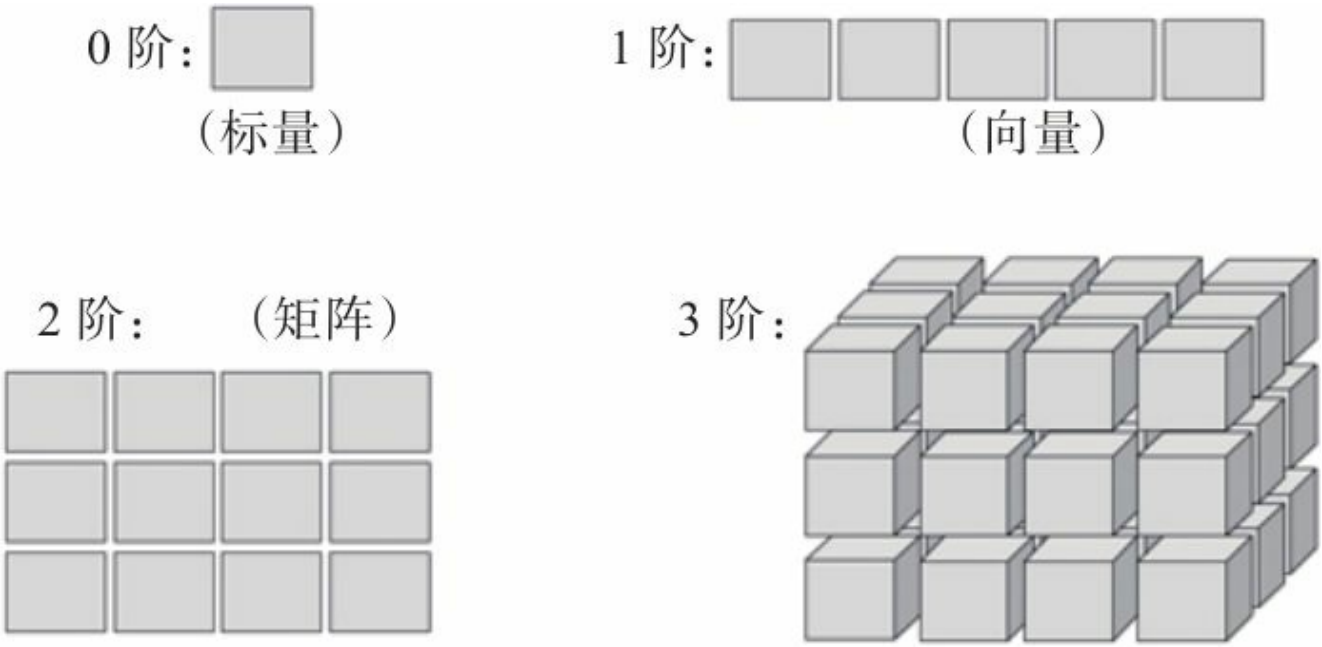
在这些主要功能中，最后但并非不重要的是TensorFlow支持移动部署，使其成为非常适合生产的工具。

## 14.2 TensorFlow的排序与张量

TensorFlow允许用户把张量操作和功能定义为计算图。张量是通用的数学符号，代表保存数据值的多维阵列，张量的维数通常被称为阶。

到目前为止，我们一直使用0到2阶张量。例如标量，像整数或浮点这样的单个数是0阶张量。向量是1阶张量，矩阵是2阶张量。但并没有到此为止。张量符号可以推广到更高维度，就像下一章将看到的，用3阶输入张量以及4阶权重张量来支持具有多个颜色通道的图像。

为了使张量概念更直观，考虑下面的图，第一行是0阶和1阶张量，第二行是2阶和3阶张量：




### 如何获得张量的阶和形状

调用tf.rank函数可以得到张量的阶。值得注意的是tf.rank将返回一个张量作为输出，因此需要评估张量以得到实际值。

除了张量的阶以外，还有张量形状（与NumPy阵列形状相似）。例如，如果X是一个张量，那么可以通过调用X.get_shape（）得到其形状，结果返回在一个被称为TensorShape的特殊类中。

在创建其他张量时，可以输出形状并把它直接作为形状参数。但不能直接对该对象进行索引或切片。如果想对该对象的不同元素进行索引或切片，那么可以用张量类的as_list方法将其转换为Python列表。

关于如何使用张量的tf.rank函数和get_shape方法，请参考下面的示例代

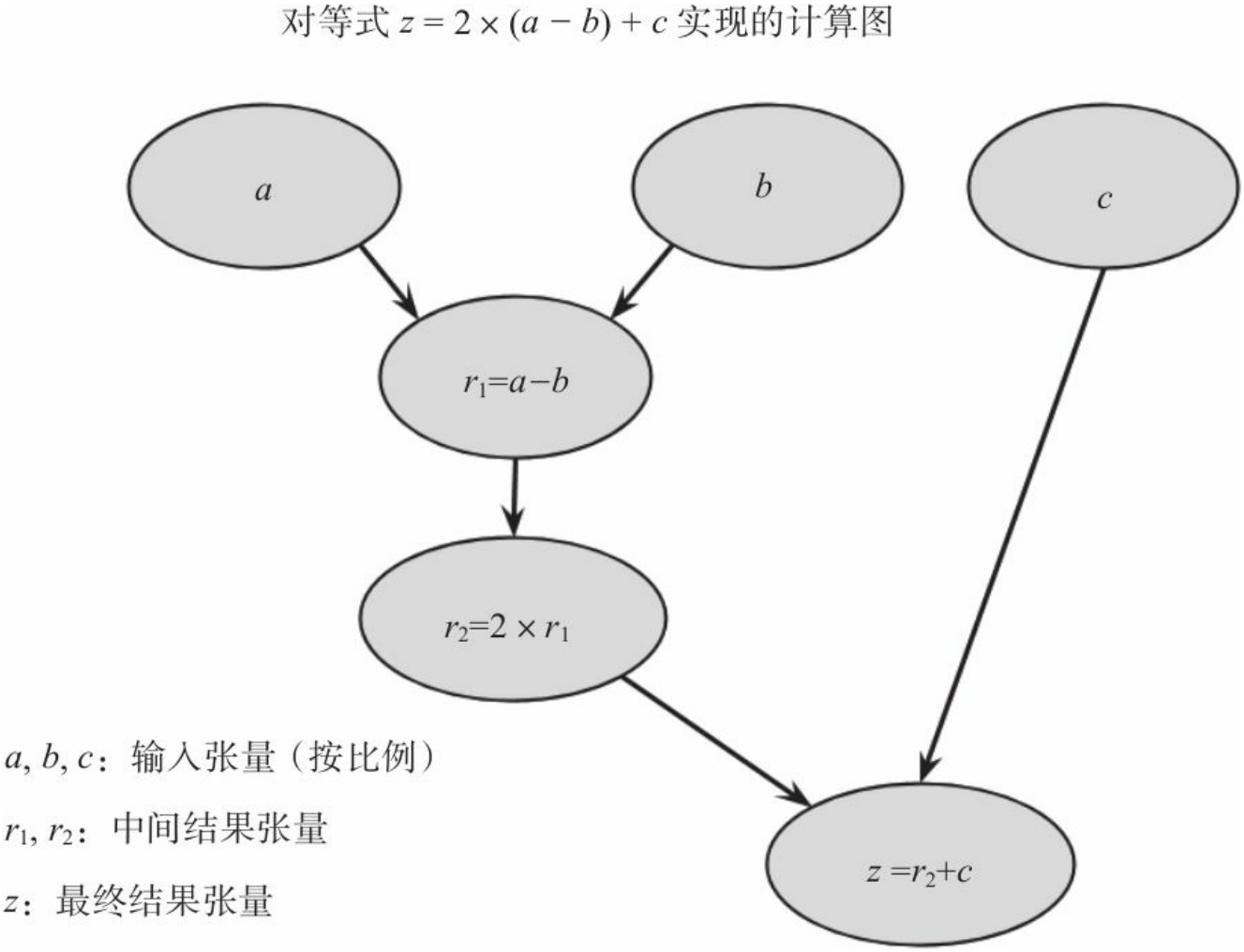
码。该段代码说明了如何在一个TensorFlow会话中检索张量的阶和形状。

```
>>> import tensorflow as tf
>>> import numpy as np
>>>
>>> g = tf.Graph()
>>>
>>> ## define the computation graph
>>> with g.as_default():

...     ## define tensors t1, t2, t3
...     t1 = tf.constant(np.pi)

...     t2 = tf.constant([1, 2, 3, 4])
...     t3 = tf.constant([[1, 2], [3, 4]])
...
...     ## get their ranks
...     r1 = tf.rank(t1)
...     r2 = tf.rank(t2)
...     r3 = tf.rank(t3)
...
...     ## get their shapes
...     s1 = t1.get_shape()
...     s2 = t2.get_shape()
...     s3 = t3.get_shape()
...     print('Shapes:', s1, s2, s3)
Shapes: [] (4,) (2, 2)
>>> with tf.Session(graph=g) as sess:
...     print('Ranks:',
...           r1.eval(),
...           r2.eval(),
...           r3.eval())

Ranks: 0 1 2
```

正如所看到的，张量t1的阶为0，因为它只是一个标量（对应于形状[]）。张量t2的阶为1，因为它有4个元素，其形状是一个元素元组（4，）。最后，2×2矩阵t3的阶为2，其相应的形状存储在（2，2）元组中。

## 14.3 了解TensorFlow的计算图

TensorFlow的核心在于构建计算图，并用计算图推导出从输入到输出的所有张量之间的关系。假设有0阶张量a、b和c，要评估z=2×（a-b）+c。可以把该评估表示为下图所示的计算图：




正如所看到的，计算图就是一个节点网络。每个节点就像一个操作，将函数应用到输入张量，然后返回0个或更多个张量作为输出。

用Tensorflow来构建计算图并计算相应的梯度。在TensorFlow中构建和编制这样一个计算图的具体步骤如下：

1.初始化一个新的空白计算图。

2.为该计算图增加节点（张量和操作）。

3.执行计算图：

a.开始一个新的会话。

b.初始化图中的变量。

c.运行会话的计算图。

让我们产生一张图来评估z=2×（a-b）+c，从前面的图中可以看到a、b和c为标量（单个数字）。这里将其定义为TensorFlow的常数。通过调用tf.Graph（）产生图，然后把节点以下述方式添加到图上：

```
>>> g = tf.Graph()
>>>
>>> with g.as_default():
...     a = tf.constant(1, name='a')
...     b = tf.constant(2, name='b')
...     c = tf.constant(3, name='c')
...
...     z = 2*(a-b) + c
```

这段代码用with g.as_default（）把节点添加到图g上。如果不具体产生图，就会有一张默认的图，因此，所有的节点都会被添加到图上。为清晰起见，本书尽量避免使用默认图。这种做法对在Jupyter笔记本中研发代码特别有用，因为这样可以避免不小心在默认图上堆叠一些不想要的节点。

TensorFlow的会话是执行图的操作和张量的环境。通过调用tf.Session产生会话对象，该调用可以接收一个图作为参数（在这里是g），例如，tf.Session（graph=g）。否则将会启动默认的空图。

在TensorFlow会话启动计算图以后，可以执行程序处理这些节点，即评估张量或者完成操作。评估每个节点涉及在当前会话中调用eval方法。在评估图中的特定张量时，TensorFlow要在图中执行所有的前述代码，直到遇到特定的节点。如果存在占位符就需要予以提供，下一节将会看到。

非常相似地，可以用会话的run方法执行操作。在前面的例子里，train_op仅是一个操作符号，不返回任何张量。可以通过调用train_op.run（）来执行该操作符。有一种通用的方法可以既运行张量又运行操作符：tf.Session（）.run（），它将返回大小均匀的列表。

我们将在TensorFlow的会话里启动前面的图来评估张量z：

```
>>> with tf.Session(graph=g) as sess:
...     print('2*(a-b)+c => ', sess.run(z))
2*(a-b)+c =>  1
```

记得曾在TensorFlow内定义张量和对计算图的操作。接着用TensorFlow的会话来对图执行操作，并取得评估结果。

本节看到了如何定义计算图，如何为图添加节点，如何在TensorFlow的会话中评估张量。现在将深入讨论在计算图中可能出现的不同类型节点，包括占位符和变量。同时，会看到其他一些在输出中不返回张量的操作符。

## 14.4 TensorFlow中的占位符

TensorFlow有提供数据的特别机制。其中一种机制是使用占位符，它们是一些预先定义好了具体类型和形状的张量。

通过调用tf.placeholder函数把这些张量加到计算图上，而且它们不包含任何数据。然而，一旦执行了图中的特定节点就需要提供数据阵列。

下面的章节将看到如何在图中定义占位符，如何在节点执行后提供数据值。

### 14.4.1 定义占位符

如你所知道的那样，通过调用tf.placeholder函数来定义占位符。占位符的形状和类型取决于节点执行时需要提供数据的形状和类型。

先看一个简单的例子。下面的代码将沿用在前一节评估z=2×（a-b）+c时用过的图，但这次用占位符替代标量a、b和c，同时存储与r1和r2相关联张量的中间结果：

```
>>> import tensorflow as tf
>>>
>>> g = tf.Graph()


>>> with g.as_default():
...     tf_a = tf.placeholder(tf.int32, shape=[],
...                           name='tf_a')
...     tf_b = tf.placeholder(tf.int32, shape=[],
...                           name='tf_b')
...     tf_c = tf.placeholder(tf.int32, shape=[],
...                           name='tf_c')
...
...     r1 = tf_a-tf_b
...     r2 = 2*r1
...     z  = r2 + tf_c
```

代码里定义了三个占位符，分别是类型为tf.int32（32位整数型）的tf_a、tf_b、tf_c，而且通过shape=[]设置其形状，因为它们是标量（0阶张量）。本书总是在占位对象的前面加上tf_，以便清楚地区别于其他张量。

请注意，前面的示例代码处理标量，所以形状定义为shape=[]。然而，定义高维度的占位符是顺理成章的事。例如，一个类型为float、阶为3、形状为3×4×5的占位符可以定义为tf.placeholder（dtype=tf.float32，shape=[2，3，4]）。

## 14.4.2 为占位符提供数据

当在图中处理节点的时候，需要产生Python字典来为占位符提供数据阵列。这需要根据占位符的类型和形状来决定。该词典将作为输入的参数feed_dict传递给会话的run方法。

在前面的图中，添加了三个类型为tf.int32的占位符来提供标量以计算z。现在，为了评估结果张量z，可以随意为占位符提供整数型的数值（这里是1、2和3）如下：

```
>>> with tf.Session(graph=g) as sess:
...     feed = {tf_a: 1,
...             tf_b: 2,
...             tf_c: 3}
...     print('z:',
...           sess.run(z, feed_dict=feed))
z: 1
```

这意味着占位符有多余的阵列并不会导致错误，只不过有些多余。然而，如果在处理特定节点时需要占位符，却没有通过参数feed_dict提供的话，就会造成运行错误。

### 14.4.3 用batchsizes为数据阵列定义占位符

在研发神经网络模型的过程中，有时会碰到规模不一致的小批量数据。例如可能用某个规模的小批量来训练神经网络，但我们想用网络来对一个或者多个数据输入进行预测。

占位符的一个有用功能是可以把大小无法确定的维度定义为None。例如可以产生一个2阶占位符，其第一个维度为未知（或者可变），详情如下所示：

```
>>> import tensorflow as tf
>>>
>>> g = tf.Graph()
>>>
>>> with g.as_default():
...     tf_x = tf.placeholder(tf.float32,
...                           shape=[None, 2],
...                           name='tf_x')
...
...     x_mean = tf.reduce_mean(tf_x,
...                             axis=0,
...                             name='mean')
```

接着可以用两个不同的输入x1和x2来评估x_mean，它们是NumPy形状阵列，大小分别是（5，2）和（10，2），具体代码实现如下：

```
>>> import numpy as np
>>> np.random.seed(123)
>>> np.set_printoptions(precision=2)
>>> with tf.Session(graph=g) as sess:
...     x1 = np.random.uniform(low=0, high=1,
...                            size=(5, 2))
...     print('Feeding data with shape ', x1.shape)
...     print('Result:', sess.run(x_mean,
...                               feed_dict={tf_x: x1}))
...     x2 = np.random.uniform(low=0, high=1,
...                            size=(10,2))
...     print('Feeding data with shape', x2.shape)
...     print('Result:', sess.run(x_mean,
...                               feed_dict={tf_x: x2}))
```

显示输出如下：

```
Feeding data with shape (5, 2)
Result: [ 0.62  0.47]
Feeding data with shape (10, 2)
Result: [ 0.46  0.49]
```

最后尝试显示对象tf_x，将得到Tensor（"tf_x：0"，shape=（？，2），dtype=float32），其中显示张量的形状为（？，2）。

## 14.5 TensorFlow中的变量

就TensorFlow而言，变量是一种特殊类型的张量对象，它允许我们在训练模型期间，在TensorFlow会话中存储和更新模型的参数。下面这部分解释了如何在计算图中定义变量，初始化会话中的变量，通过所谓的变量范围组织变量，以及复用现有变量。

### 14.5.1 定义变量

TensorFlow的变量存储在训练过程中可以更新的模型参数中，例如，神经网络的输入层、隐藏层和输出层的权重。当定义参数时，需要用张量值对其进行初始化。可以在下述网站阅读到更多关于TensorFlow变量的介绍：

https://www.tensorflow.org/programmers_guide/variables

TensorFlow提供定义变量的两种方式：

·tf.Variable（，name="variable-name"）

·tf.get_variable（name，...）

第一种方式，tf.Variable是为新变量创建对象并将其添加到计算图的类。注意，tf.Variable变量没有明确的方式来确定shape和dtype，形状和类型将和那些最初值相同。

第二种方式，假设某个变量名在计算图中，tf.get_variable可以复用给定变量名的现有值，如果不存在，可以创建一个新变量名。因此名称变得非常重要。这可能就是为什么它必须作为函数的第一个参数。此外，tf.get_variable提供了设置shape和dtype的显式方法，只有在创建新变量时才需要这些参数，而不是复用现有的参数时。

tf.get_variable与tf.Variable相比有两个优点：tf.get_variable允许复用现有变量，这些变量已经采用常用的Xavier/Glorot默认的初始化方案。

除了初始化以外，get_variable函数还提供其他的参数来控制张量，例如为变量正则化。如果有兴趣想了解更多关于tf.get_variable参数的信息，可随时访问下述网站：

https://www.tensorflow.org/api_docs/python/tf/get_variable

Xavier/Glorot初始化

在深度学习的早期，已经发现随机均匀初始化或随机正则权重初始化常常会导致模型在训练过程中表现不佳。

2010年，沙维尔·格罗饶特和约书华·本希奥研究了初始化的影响，提出了一种新的、更强大的初始化方案来促进深层网络的训练。Xavier初始化的逻辑是大致平衡不同层间的梯度方差。否则，某层可能在训练过程中受到过

多关注，而另一层却乏人问津。

格罗饶特和本希奥的研究发现，如果要均匀地初始化权重分布，应该选择均匀分布区间：

$$W \sim Uniform\left(-\frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}\right)$$

这里nin为输入神经元数量与权重的乘积，而$n_{out}$为输出到下一层神经元的数量。为使初始化权重呈现正态分布，作者推荐选用下述标准方差：

$$\sigma = \frac{\sqrt{2}}{\sqrt{n_{in}}+\sqrt{n_{out}}}$$

TensorFlow支持Xavier权重均匀或呈正态分布的初始化。下述文档提供了关于TensorFlow使用Xavier初始化方面的详细信息：

https://www.tensorflow.org/api_docs/python/tf/contrib/layers/xavier_initialize

如果想要了解更多关于格罗饶特和本希奥初始化方案的信息，包括数学推导及证明，我建议阅读最初发表的论文（沙维尔·格罗饶特，约书华·本希奥，“理解深度前馈神经网络的难点”，2010年），该论文可以从下述网站免费获取：

http://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf

无论采用哪种初始化技术，需要格外注意的是，直到通过调用tf.Session启动计算图并且在会话中具体运行初始化操作时才设置初始化值。事实上，只有初始化TensorFlow的变量之后，才会为计算图分配内存。

下面是一段创建变量对象的示例代码，其初始值由NumPy阵列产生。该张量的数据类型dtype为tf.int64，自动根据NumPy阵列推算：

```
>>> import tensorflow as tf
>>> import numpy as np
>>>
>>> g1 = tf.Graph()
>>>
>>> with g1.as_default():
...     w = tf.Variable(np.array([[1, 2, 3, 4],
...                               [5, 6, 7, 8]]), name='w')
...     print(w)
<tf.Variable 'w:0' shape=(2, 4) dtype=int64_ref>
```

## 14.5.2 初始化变量

理解定义为变量的张量不会被分配内存空间，而且在初始化之前这些变量不包含任何值是很重要的。因此，在计算图中处理任何节点之前，必须初始化存在于要执行节点路径内的变量。

这个初始化过程包括为相关张量分配内存空间并为其赋予初始值。TensorFlow提供了一个名为tf.global_variables_initializer的函数，该函数返回初始化所有计算图中现存变量的操作符。执行该操作符初始化变量如下：

```
>>> with tf.Session(graph=g1) as sess:
...     sess.run(tf.global_variables_initializer())
...     print(sess.run(w))

[[1 2 3 4]
 [5 6 7 8]]
```

也可以把该操作符存储在像init_op=tf.global_variables_initializer（）这样的对象中，然后用sess.run（init_op）或者init_op.run（）执行该操作符。然而，需要确保该操作符在定义了所有的变量后才创建。

例如，下面的代码先定义变量w1，然后定义操作符init_op，接着是变量w2：

```
>>> import tensorflow as tf
>>>
>>> g2 = tf.Graph()
>>>
>>> with g2.as_default():
...     w1 = tf.Variable(1, name='w1')
...     init_op = tf.global_variables_initializer()
...     w2 = tf.Variable(2, name='w2')
```

现在用下面的方式评估w1：

```
>>> with tf.Session(graph=g2) as sess:
...     sess.run(init_op)
...     print('w1:', sess.run(w1))
w1: 1
```

这段代码运行良好。现在尝试评估w2：

```
>>> with tf.Session(graph=g2) as sess:
...     sess.run(init_op)
...     print('w2:', sess.run(w2))
FailedPreconditionError
Attempting to use uninitialized value w2
    [[Node: _retval_w2_0_0 = _Retval[T=DT_INT32, index=0, _device="/
job:localhost/replica:0/task:0/cpu:0"](w2)]]
```

正如实例代码所示，处理计算图发生了错误，w2在处理计算图时报告了一个错误，因为w2并不是通过调用sess.run（init_op）完成的初始化，因此无法评估。init_op是在把w2添加到图上之前定义的，因此，执行init_op无法初始化w2。

### 14.5.3 变量范围

本节将要讨论变量的域，这是TensorFlow的一个重要概念，对建设大型神经网络计算图特别有用。

可以把变量的域划分成独立的子部分。在创建变量域时，该域内创建的操作和张量的名称都以域名为前缀，而且这些域可以嵌套。例如，如果有两个子网络，每个子网络有好几层，那么可以分别把这两个域定义为net_A和net_B。然后，再在这些域中定义每个层。

让我们看看下述代码示例中的变量是如何命名的：

```
>>> import tensorflow as tf
>>>
>>> g = tf.Graph()
>>>
>>> with g.as_default():
...     with tf.variable_scope('net_A'):
...         with tf.variable_scope('layer-1'):
...             w1 = tf.Variable(tf.random_normal(
...                 shape=(10,4)), name='weights')
...         with tf.variable_scope('layer-2'):
...             w2 = tf.Variable(tf.random_normal(
...                 shape=(20,10)), name='weights')
...     with tf.variable_scope('net_B'):
...         with tf.variable_scope('layer-1'):
...             w3 = tf.Variable(tf.random_normal(
...                 shape=(10,4)), name='weights')
...
...     print(w1)
...     print(w2)
...     print(w3)


<tf.Variable 'net_A/layer-1/weights:0' shape=(10, 4) dtype=float32_
ref>
<tf.Variable 'net_A/layer-2/weights:0' shape=(20, 10) dtype=float32_
ref>
<tf.Variable 'net_B/layer-1/weights:0' shape=(10, 4) dtype=float32_
ref>
```

注意变量名称现在有嵌套域的前缀，用正斜杠（/）符号分隔。

要了解更多关于变量域的信息，可在下述网站找到更多文档：

https://www.tensorflow.org/programmers_guide/variable_scope

https://www.tensorflow.org/api_docs/python/tf/variable_scope

### 14.5.4 变量复用

想象我们正在开发一个有点儿复杂的神经网络模型，其分类器的输入数据有多个来源。假设数据（$X_A$，$y_A$）来自于自源A，数据（$X_B$，$y_B$）来自于自源B。

这个例子将以这样的方式设计计算图：它将来自于一个源的数据作为输入张量来构建网络，然后将另一个源的数据送到同一个分类器。

下面的示例代码假设来自于源A的数据通过占位符馈送，并且源B是数据生成器网络的输出。通过调用build_generator函数在generator域建立生成器网络，然后调用build_classifier在classifier域添加分类器：

```
>>> import tensorflow as tf
>>>
>>> ############################
... ##   Helper functions    ##
... ############################
>>>
>>> def build_classifier(data, labels, n_classes=2):
...     data_shape = data.get_shape().as_list()
...     weights = tf.get_variable(name='weights',
...                               shape=(data_shape[1],
...                                      n_classes),
...                               dtype=tf.float32)
...     bias = tf.get_variable(name='bias',
...                            initializer=tf.zeros(
...                                        shape=n_classes))
...     logits = tf.add(tf.matmul(data, weights),
...                     bias,
...                     name='logits')
...     return logits, tf.nn.softmax(logits)
>>>
>>>
>>> def build_generator(data, n_hidden):
...     data_shape = data.get_shape().as_list()
...     w1 = tf.Variable(
...         tf.random_normal(shape=(data_shape[1],
...                                 n_hidden)),
...         name='w1')
...     b1 = tf.Variable(tf.zeros(shape=n_hidden),
...                      name='b1')
...     hidden = tf.add(tf.matmul(data, w1), b1,
...                     name='hidden_pre-activation')
...     hidden = tf.nn.relu(hidden, 'hidden_activation')
...
...     w2 = tf.Variable(
...         tf.random_normal(shape=(n_hidden,
...                                 data_shape[1])),
...         name='w2')
...     b2 = tf.Variable(tf.zeros(shape=data_shape[1]),
...                      name='b2')
...     output = tf.add(tf.matmul(hidden, w2), b2,
...                     name = 'output')
...     return output, tf.nn.sigmoid(output)
>>>
>>> ############################
... ##  Build the graph      ##
... ############################
>>>
>>> batch_size=64
>>> g = tf.Graph()
>>>
>>> with g.as_default():
...     tf_X = tf.placeholder(shape=(batch_size, 100),
...                           dtype=tf.float32,
...                           name='tf_X')
...
```

```
...     ## build the generator
...     with tf.variable_scope('generator'):
...         gen_out1 = build_generator(data=tf_X,
...                                    n_hidden=50)
...
...     ## build the classifier
...     with tf.variable_scope('classifier') as scope:
...         ## classifier for the original data:
...         cls_out1 = build_classifier(data=tf_X,
...                                     labels=tf.ones(
...                                         shape=batch_size))
...
...         ## reuse the classifier for generated data
...         scope.reuse_variables()
...         cls_out2 = build_classifier(data=gen_out1[1],
...                                     labels=tf.zeros(
...                                         shape=batch_size))
```

注意，调用build_classifier函数两次。先建立网络，然后在scope.reuse_variables（）中再次调用build_classifier。从结果看，第二次调用并不创建新变量而是复用相同的变量，或者可以通过定义参数reuse=True来复用变量，如下所示：

```
>>> g = tf.Graph()
>>>
>>> with g.as_default():
...     tf_X = tf.placeholder(shape=(batch_size, 100),
...                           dtype=tf.float32,
...                           name='tf_X')
...     ## build the generator
...     with tf.variable_scope('generator'):
...         gen_out1 = build_generator(data=tf_X,
...                                    n_hidden=50)
...
...     ## build the classifier
...     with tf.variable_scope('classifier'):
...         ## classifier for the original data:
...         cls_out1 = build_classifier(data=tf_X,
...                                     labels=tf.ones(
...                                         shape=batch_size))
...
...     with tf.variable_scope('classifier', reuse=True):


...         ## reuse the classifier for generated data
...         cls_out2 = build_classifier(data=gen_out1[1],
...                                     labels=tf.zeros(
...                                         shape=batch_size))
```

虽然已经讨论了如何在TensorFlow上定义计算图和变量，但对计算图的梯度计算超出了本书的讨论范围，这里用TensorFlow方便的优化器类来自动进行反向传播。如果对计算图梯度计算以及TensorFlow的不同计算方式感兴趣，请参阅由塞巴斯蒂安·拉施卡写的《PyData talk》（https://github.com/rasbt/pydata-annarbor2017-dl-tutorial）。

## 14.6 建立回归模型

既然已经探索了占位符和变量，那么我们构建一个回归分析的实例模型，类似于第13章中创建的那个，目标是实现线性回归模型：$\hat{y} = wx + b$。

在该模型中，w和b是这个简单回归模型的两个参数，需要被定义为变量。请注意，x是模型的输入，可以将其定义为占位符。此外，为了训练这个模型，需要建立一个成本函数。这里用在第10章中定义的均方误差（MSE）代价函数。

$$MSE = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2$$

这里y为真值，是为训练该模型而提供的输入。因此也需要定义y为占位符。最后，将用TensorFlow的tf.matmul和tf.add操作来计算预测的输出值$\hat{y}$。前面讨论过TensorFlow的操作会返回0或多个张量。这里tf.matmul和tf.add返回0个张量。

我们也可以用过载操作符+来增加两个张量。然而，tf.add的好处就是可以通过参数name为结果张量提供一个额外的名字。

所以，用数学符号和代码名称把所有的张量总结如下：

·输入x：定义为占位符tf_x

·输入y：定义为占位符tf_y

·模型参数w：定义为变量weight

·模型参数b：定义为变量bias

·模型输出$\hat{y}$：由TensorFlow的操作计算返回的回归模型预测结果y_hat

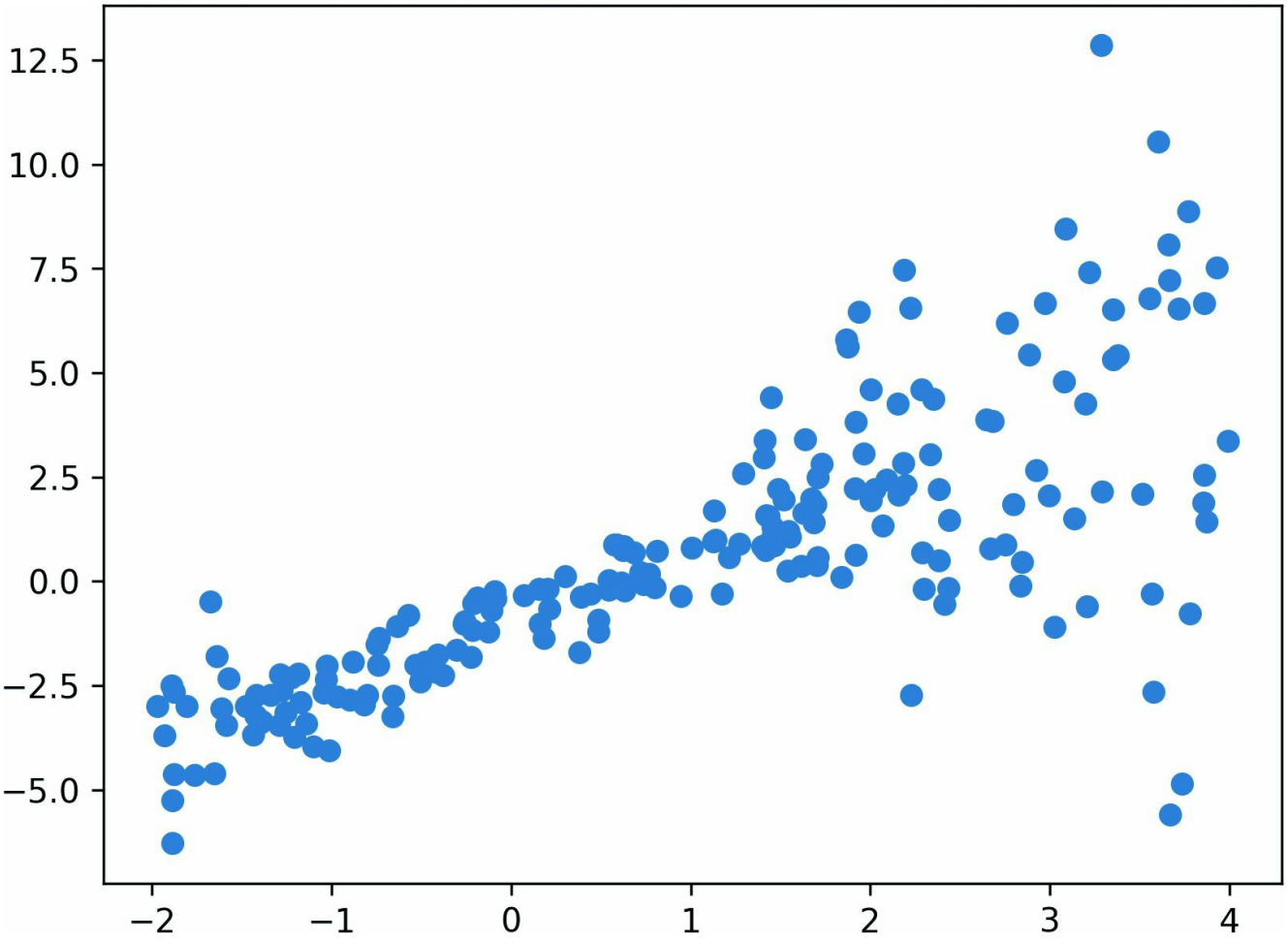
实现这个简单回归模型的代码如下：

```
>>> import tensorflow as tf
>>> import numpy as np
>>>
>>> g = tf.Graph()
>>>
>>> with g.as_default():
...     tf.set_random_seed(123)
...     ## placeholders
...     tf_x = tf.placeholder(shape=(None),
...                           dtype=tf.float32,
...                           name='tf_x')
...     tf_y = tf.placeholder(shape=(None),
...                           dtype=tf.float32,
...                           name='tf_y')
...
...     ## define the variable (model parameters)
...     weight = tf.Variable(
...         tf.random_normal(
...             shape=(1, 1),
...             stddev=0.25),
...         name='weight')
...     bias = tf.Variable(0.0, name='bias')
...
...     ## build the model
...     y_hat = tf.add(weight * tf_x, bias,

...                    name='y_hat')
...
...      ## compute the cost
...      cost = tf.reduce_mean(tf.square(tf_y - y_hat),
...                            name='cost')
...
...      ## train the model
...      optim = tf.train.GradientDescentOptimizer(
...          learning_rate=0.001)
...      train_op = optim.minimize(cost, name='train_op')
```

现在已经建立了计算图，下一步要创建会话来启动计算图并训练模型。但在进一步讨论之前，先看看如何评估张量并执行操作。我们将生成随机回归数据，并调用make_random_data函数来实现其可视化：

```
>>> ## create a random toy dataset for regression
>>>
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> np.random.seed(0)
>>>
>>> def make_random_data():
...     x = np.random.uniform(low=-2, high=4, size=200)
...     y = []
...     for t in x:
...         r = np.random.normal(loc=0.0,
...                              scale=(0.5 + t*t/3),
...                              size=None)
...         y.append(r)
...     return  x, 1.726*x -0.84 + np.array(y)
>>>
>>>
>>> x, y = make_random_data()
>>>
>>> plt.plot(x, y, 'o')
>>> plt.show()
```

下图显示了所生成的随机回归数据：

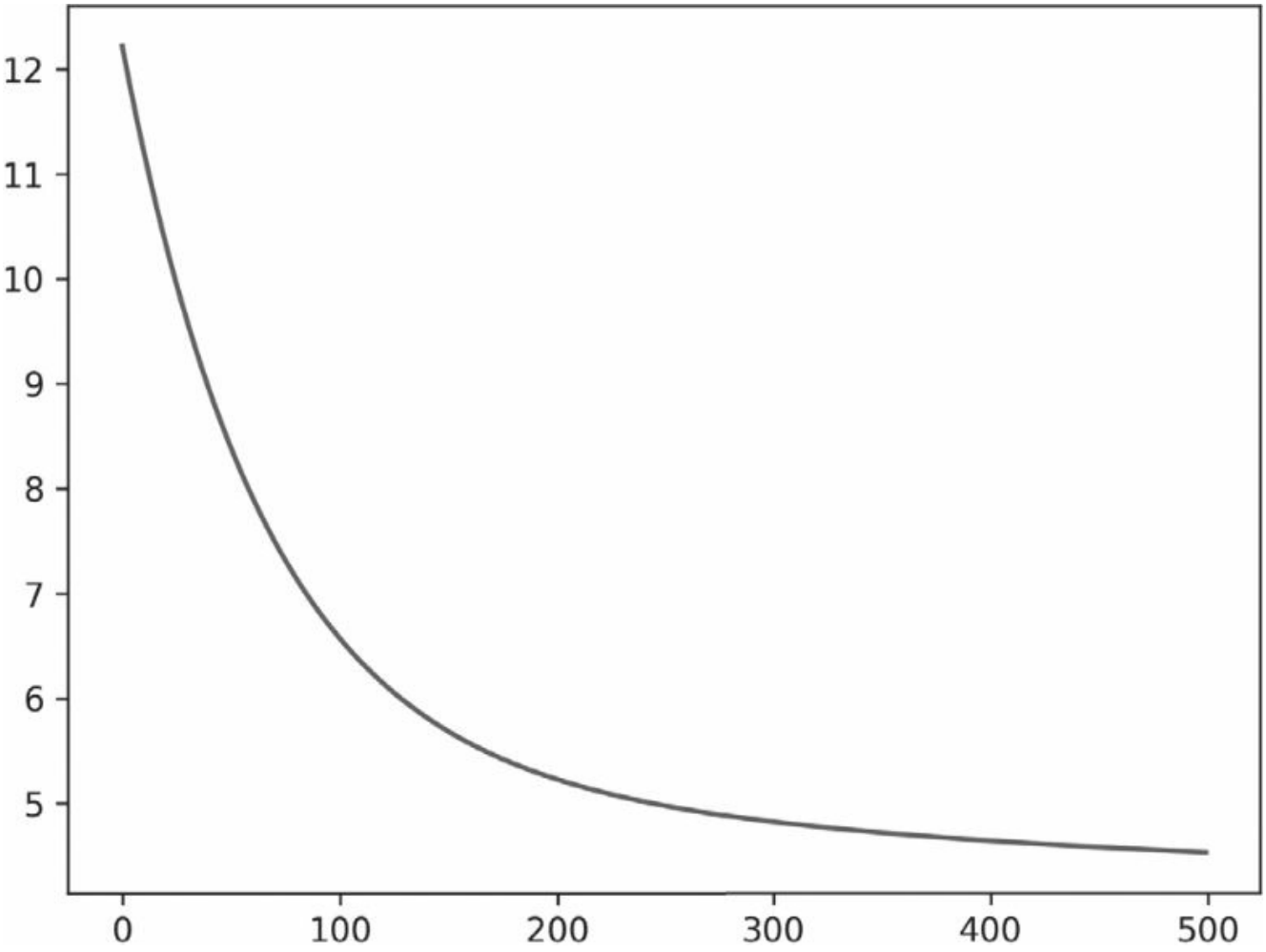



现在万事俱备，可以训练以前的模型了。让我们从创建被称为sess的TensorFlow会话对象开始。然后初始化变量，这可以通过调用sess.run（tf.global_variables_initializer（））完成。此后也可以创建for循环来执行训练操作符同时计算训练成本。

我们有两个任务：第一个执行操作符，第二个评估张量。现在把这两个任务整合成一个sess.run方法，具体代码如下：

```
>>> ## train/test splits
>>> x_train, y_train = x[:100], y[:100]
>>> x_test, y_test = x[100:], y[100:]
>>>
>>>
>>> n_epochs = 500
>>> training_costs = []
>>> with tf.Session(graph=g) as sess:
...     sess.run(tf.global_variables_initializer())
...
...     ## train the model for n_epochs
...     for e in range(n_epochs):
...         c, _ = sess.run([cost, train_op],
...                         feed_dict={tf_x: x_train,
...                                    tf_y: y_train})
...         training_costs.append(c)
...         if not e % 50:
...             print('Epoch %4d: %.4f' % (e, c))
Epoch    0: 12.2230
Epoch   50: 8.3876
Epoch  100: 6.5721
Epoch  150: 5.6844
Epoch  200: 5.2269
Epoch  250: 4.9725
Epoch  300: 4.8169
Epoch  350: 4.7119
Epoch  400: 4.6347
Epoch  450: 4.5742

>>> plt.plot(training_costs)
>>> plt.show()
```

执行代码生成以下图表，显示每次迭代后的训练成本，如右图所示。


12
11
10
9
8
7
6
5
0
100
200
300
400
500

## 14.7 在TensorFlow计算图中用张量名执行对象

在许多情况下，通过名称来执行变量和操作符非常有用。例如，我们可能在某个独立的模块开发了一个模型，根据Python的作用域规则，从其他不同的Python域无法访问该模型。然而，如果有计算图，就可以用计算图上的张量名来执行图中的节点。

这可以通过修改前面代码示例中的sess.run方法轻松实现，用图中的变量名cost而不是Python变量cost，把sess.run（[cost，train_op]，...）修改为sess.run（['cost：0'，'train_op']，...）。

```
>>> n_epochs = 500
>>> training_costs = []
>>> with tf.Session(graph=g) as sess:
...     ## first, run the variables initializer
...     sess.run(tf.global_variables_initializer())
...
...     ## train the model for n_eopchs
...     for e in range(n_epochs):
...         c, _ = sess.run(['cost:0', 'train_op'],
...                         feed_dict={'tf_x:0':x_train,
...                                    'tf_y:0':y_train})
...         training_costs.append(c)
...         if e%50 == 0:
...             print('Epoch {:4d} : {:.4f}'
...                   .format(e, c))
```

请注意，我们根据张量名'cost：0'评估成本，根据张量名'train_op'执行训练操作符。同样，在feed_dict方法中，使用'tf_x：0'：x_train而不是tf_x：x_train。

如果把注意力放在张量名上，将会注意到TensorFlow在张量名之后增加了后缀'：0'。然而，操作符名却没有类似的后缀。例如，在以name='my_tensor'创建张量时，TensorFlow在其后加上'：0'，这样张量名就变成'my_tensor：0'。

如果在同一计算图中创建同名张量，TensorFlow将为名称加上后缀'_1：0'。因此，未来的张量会有诸如'my_tensor_1：0'、'my_tensor_2：0'等名称。这种命名方法可以确保不会复用已创建的张量。

## 14.8 在TensorFlow中存储和恢复模型

前一节建立了图并完成了训练。如何在保留的测试集上进行实际的预测？问题是我们并没有保存模型的参数。因此，一旦执行完前面的语句就会退出tf.session环境，所有变量以及分配的内存都将被释放。

一种解决方案是一旦完成模型训练，就为它提供测试集。然而，这并不是一个好方法，因为训练深度神经网络模型通常需要几小时、几天甚至几周的时间。

最好的方法是保存经过训练的模型以备将来使用。为此，需要给计算图添加一个新节点，它也是tf.train.Saver类的一个实例，我们称之为saver。

```
>>> with g.as_default():
...     saver = tf.train.Saver()
```

下面的语句为特定的计算图添加更多节点。当前情况下，我们在图g中添加了saver。

可以接着通过调用saver.save（），以下面的方式保存模型：

```
>>> n_epochs = 500
>>> training_costs = []
>>> with tf.Session(graph=g) as sess:
...     sess.run(tf.global_variables_initializer())
...
...     ## train the model for n_epochs
...     for e in range(n_epochs):
...         c, _ = sess.run([cost, train_op],
...                         feed_dict={tf_x: x_train,
...                                    tf_y: y_train})
...         training_costs.append(c)
...         if not e % 50:
...             print('Epoch %4d: %.4f' % (e, c))
...
...     saver.save(sess, './trained-model')
```

执行完这个新语句后，将产生三个分别以.data、.index和.meta为扩展名的文件。TensorFlow用缓存协议（https://developers.google.com/protocol-buffers/，一种非语言的方式）把结构化的数据序列化存储。

恢复保存的模型需要以下两步：

1.根据存储的模型重构有相同节点和变量名的计算图。

2.把存储的变量恢复到新的tf.Session环境。

执行前面用过的语句可以完成第一步构建图g的任务。但是有更加简单的办法来完成这个任务。注意所有与图相关的信息都以元数据的形式保存在以.meta为扩展名的文件中。执行下述代码，可以通过导入元数据来重构计算图：

```
>>> with tf.Session() as sess:
...     new_saver = tf.train.import_meta_graph(
...                               './trained-model.meta')
```

tf.train.import_meta_graph函数重新创建存储在'./trained-model.meta'文件中的计算图。重新创建计算图之后，可以用new_saver对象恢复会话中的模型参数，然后执行。下面给出了在测试集上运行该模型的全部代码：

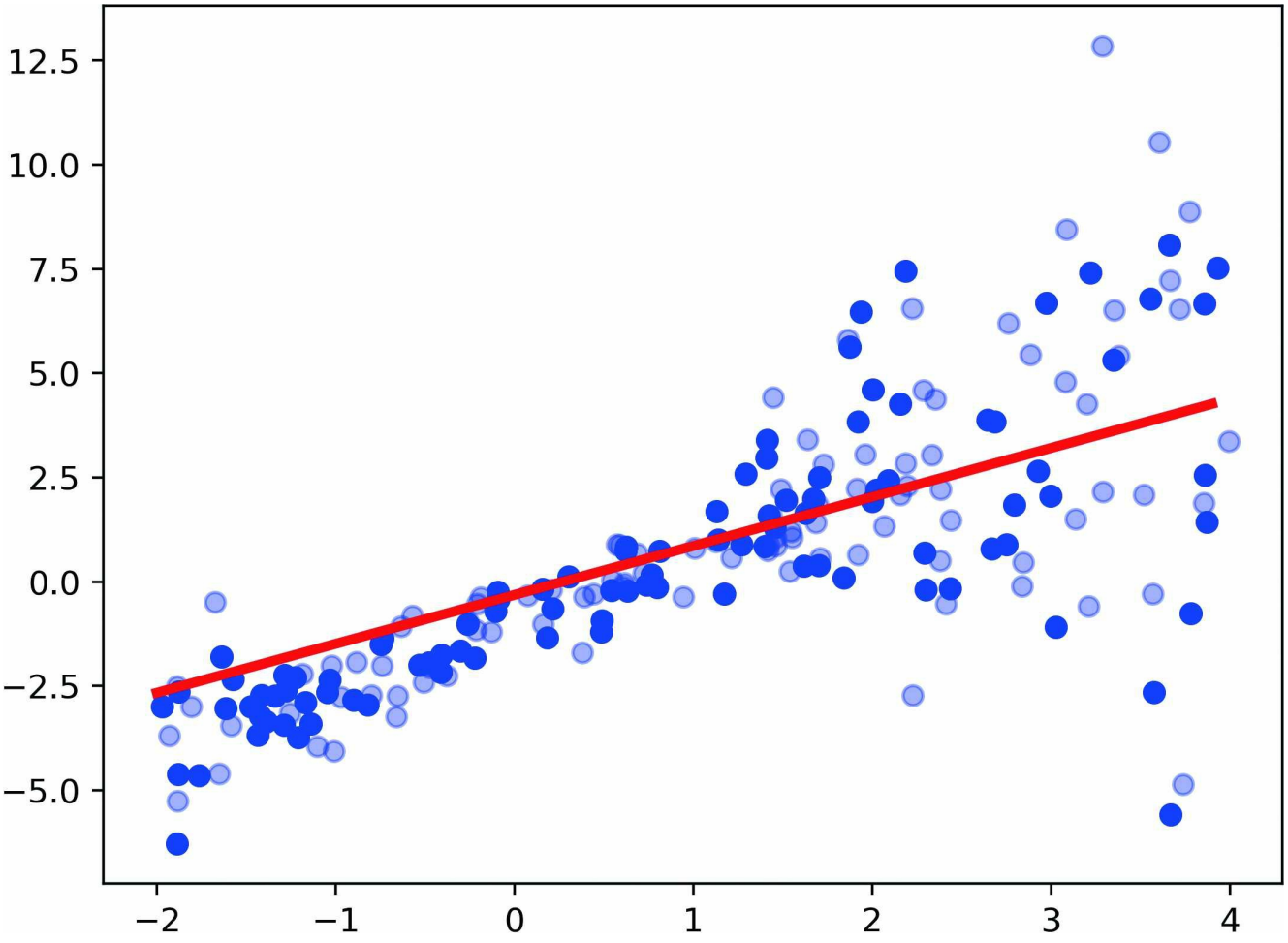
```
>>> import tensorflow as tf
>>> import numpy as np
>>>
>>> g2 = tf.Graph()
>>> with tf.Session(graph=g2) as sess:
...     new_saver = tf.train.import_meta_graph(
...         './trained-model.meta')
...     new_saver.restore(sess, './trained-model')
...
...     y_pred = sess.run('y_hat:0',
...                       feed_dict={'tf_x:0': x_test})
```

请注意，这里根据之前的张量名'y_hat：0'评估张量$\hat{y}$。同时，需要为张量名为'tf_x：0'的占位符tf_x提供数值。在这种情况下，没有必要为真值y提供数值。因为在重构的计算图中，执行节点y_hat并不依赖tf_y。

现在，执行下面的代码完成对预测结果的可视化：

```
>>> import matplotlib.pyplot as plt
>>>
>>> x_arr = np.arange(-2, 4, 0.1)
>>>
>>> g2 = tf.Graph()
>>> with tf.Session(graph=g2) as sess:
...     new_saver = tf.train.import_meta_graph(
...         './trained-model.meta')
...     new_saver.restore(sess, './trained-model')
...
...     y_arr = sess.run('y_hat:0',
...                      feed_dict={'tf_x:0' : x_arr})
>>>
>>> plt.figure()
>>> plt.plot(x_train, y_train, 'bo')
>>> plt.plot(x_test, y_test, 'bo', alpha=0.3)
>>> plt.plot(x_arr, y_arr.T[:, 0], '-r', lw=3)
>>> plt.show()
```

显示在下图中的结果既包括了训练数据，也包括了测试数据：




在大型模型的训练阶段经常会存储和恢复模型。因为训练大型模型可能会耗费几小时甚至几天的时间，可以把训练进一步细分成更小的任务。例如，计划中的迭代次数为100，可以将其分成25个任务，每个任务有4次迭代，一个任务接着一个任务运行。为此，可以把训练后的模型存储起来，在执行下一个任务时再恢复。

## 14.9 把张量转换成多维数据阵列

本节将探讨如何选择可用于转换张量的操作符。可以注意到有些操作符与NumPy阵列转换相似。然而，当处理二阶以上张量时，需要小心使用像转置这样的转换器。

首先，在NumPy中可以用arr.shape属性来获得NumPy阵列的形状。而在Tensor-Flow中，则用tf.get_shape函数完成：

```
>>> import tensorflow as tf
>>> import numpy as np
>>>
>>> g = tf.Graph()
>>> with g.as_default():
...     arr = np.array([[1., 2., 3., 3.5],
...                     [4., 5., 6., 6.5],
...                     [7., 8., 9., 9.5]])
...     T1 = tf.constant(arr, name='T1')
...     print(T1)
...     s = T1.get_shape()
...     print('Shape of T1 is', s)
...     T2 = tf.Variable(tf.random_normal(
...         shape=s))
...     print(T2)
...     T3 = tf.Variable(tf.random_normal(
...         shape=(s.as_list()[0],)))
...     print(T3)
```

执行前面的代码得到下述输出：

```
Tensor("T1:0", shape=(3, 4), dtype=float64)
Shape of T1 is (3, 4)
<tf.Variable 'Variable:0' shape=(3, 4) dtype=float32_ref>
<tf.Variable 'Variable_1:0' shape=(3,) dtype=float32_ref>
```

注意到我们用s创建T2，但无法用切片或者索引创建T3。因此通过调用s.as_list（）把s转换成常规的Python列表，然后用一般的索引方法访问。

现在来看如何改变张量的形状。在NumPy中曾用np.reshape或arr.reshape来完成此任务。TensorFlow调用tf.reshape函数来改变张量的形状。至于NumPy，1个维度可以设置为-1，这样就可以根据阵列的总规模和剩余的其他维度来推算新维度。

下面的代码把张量T1转换成3阶的T4和T5：

```
>>> with g.as_default():
...     T4 = tf.reshape(T1, shape=[1, 1, -1],
...                     name='T4')
...     print(T4)
...     T5 = tf.reshape(T1, shape=[1, 3, -1],
...                     name='T5')
...     print(T5)
```

结果输出如下：

```
Tensor("T4:0", shape=(1, 1, 12), dtype=float64)
Tensor("T5:0", shape=(1, 3, 4), dtype=float64)
```

接着输出T4和T5的内容：

```
>>> with tf.Session(graph = g) as sess:
...     print(sess.run(T4))
...     print()
...     print(sess.run(T5))
```

```
[[[ 1.   2.   3.   3.5  4.   5.   6.   6.5  7.   8.   9.   9.5]]]

[[[ 1.   2.   3.   3.5]
  [ 4.   5.   6.   6.5]
  [ 7.   8.   9.   9.5]]]
```

我们已经知道NumPy有三种阵列转置的方法：arr.T，arr.transpose（）和np.transpose（arr）。在TensorFlow中，可以使用替代的tf.transpose函数，并且在一般的转置操作中，我们可以按照自己的意愿通过定义perm=[...]来改变维度的顺序。示例如下：

```
>>> with g.as_default():
...     t5_splt = tf.split(T5,
...                        num_or_size_splits=2,
...                        axis=2, name='T8')
...     print(t5_splt)

[<tf.Tensor 'T8:0' shape=(1, 3, 2) dtype=float64>,
 <tf.Tensor 'T8:1' shape=(1, 3, 2) dtype=float64>]
```

接下来，还可以用tf.split函数将张量分裂为张量列表，代码如下：

```
>>> with g.as_default():
...     t5_splt = tf.split(T5,
...                        num_or_size_splits=2,
...                        axis=2, name='T8')
...     print(t5_splt)

[<tf.Tensor 'T8:0' shape=(1, 3, 2) dtype=float64>,
 <tf.Tensor 'T8:1' shape=(1, 3, 2) dtype=float64>]
```

这里，重要的是要注意输出不再是张量对象而是张量列表。这些子张量名分别是'T8：0'和'T8：1'。

最后，另一个有用的变换是多个张量的拼接。如果有相同形状和dtype的张量列表，可以调用tf.concat函数将其组合成一个大张量。下面的代码给出了一个例子：

```
>>> g = tf.Graph()
>>> with g.as_default():
...     t1 = tf.ones(shape=(5, 1),
...                  dtype=tf.float32, name='t1')
...     t2 = tf.zeros(shape=(5, 1),
...                  dtype=tf.float32, name='t2')
...     print(t1)
...     print(t2)
>>> with g.as_default():
...     t3 = tf.concat([t1, t2], axis=0, name='t3')
...     print(t3)
...     t4 = tf.concat([t1, t2], axis=1, name='t4')
...     print(t4)

Tensor("t1:0", shape=(5, 1), dtype=float32)
Tensor("t2:0", shape=(5, 1), dtype=float32)

Tensor("t3:0", shape=(10, 1), dtype=float32)
Tensor("t4:0", shape=(5, 2), dtype=float32)
```

让我们把这些拼接张量的值打印出来：

```
>>> with tf.Session(graph=g) as sess:
...     print(t3.eval())
...     print()
...     print(t4.eval())

[[ 1.]
 [ 1.]
 [ 1.]
 [ 1.]
 [ 1.]
 [ 0.]
 [ 0.]
 [ 0.]
 [ 0.]
 [ 0.]]

[[ 1.  0.]
 [ 1.  0.]
 [ 1.  0.]
 [ 1.  0.]
 [ 1.  0.]]
```

## 14.10 利用控制流构图

现在来学习TensorFlow的一个有趣的机制。TensorFlow提供了构图决策机制。但在构建计算图时，与TensorFlow相比，Python的流控制语句有些细微的差别。为了用简单的代码示例来说明这些差异，考虑在TensorFlow中实现以下方程：

$$res = \begin{cases} x + y & \text{if } x < y \\ x - y & \text{otherwise} \end{cases}$$

下面的代码可以简单地用Python的if语句来构建与前面的方程相对应的计算图：

```
>>> import tensorflow as tf
>>>
>>> x, y = 1.0, 2.0
>>>
>>> g = tf.Graph()
>>> with g.as_default():
...     tf_x = tf.placeholder(dtype=tf.float32,
...                           shape=None, name='tf_x')
...     tf_y = tf.placeholder(dtype=tf.float32,
...                           shape=None, name='tf_y')
...     if x < y:
...         res = tf.add(tf_x, tf_y, name='result_add')
...     else:
...         res = tf.subtract(tf_x, tf_y, name='result_sub')
...
...     print('Object:', res)
>>>
>>> with tf.Session(graph=g) as sess:

...     print('x < y: %s -> Result:' % (x < y),
...           res.eval(feed_dict={'tf_x:0': x,
...                               'tf_y:0': y}))
...     x, y = 2.0, 1.0
...     print('x < y: %s -> Result:' % (x < y),
...           res.eval(feed_dict={'tf_x:0': x,
...                               'tf_y:0': y}))
```

执行代码后得到的以下结果：

```
Object: Tensor("result_add:0", dtype=float32)
x < y: True -> Result: 3.0
x < y: False -> Result: 3.0
```

正如所看到的，res对象是名为'result_add：0'的张量。重要的是要理解在先前的机制中，计算图只有一个分支与加法运算符相关联，并且没有调用减法运算符。

TensorFlow的计算图是静态的，这意味着一旦建立了计算图，它在执行过程中保持不变。因此，即使改变x和y的值并将新值输入到图中，这些新张量还将会在图中经过相同的路径。因此，在x＝2，y＝1和x＝1，y＝2两种情况下，得到了相同的输出值3.0。

现在来应用TensorFlow的流控制机制。下面的代码用tf.cond函数代替Python的if语句来实现前面的方程：

```
>>> import tensorflow as tf
>>>
>>> x, y = 1.0, 2.0
>>>
>>> g = tf.Graph()
>>> with g.as_default():
...     tf_x = tf.placeholder(dtype=tf.float32,
...                           shape=None, name='tf_x')
...     tf_y = tf.placeholder(dtype=tf.float32,
...                           shape=None, name='tf_y')
...     res = tf.cond(tf_x < tf_y,
...                   lambda: tf.add(tf_x, tf_y,
...                                  name='result_add'),
...                   lambda: tf.subtract(tf_x, tf_y,
...                                       name='result_sub'))
...     print('Object:', res)
...
>>> with tf.Session(graph=g) as sess:
...     print('x < y: %s -> Result:' % (x < y),
...           res.eval(feed_dict={'tf_x:0': x,
...                               'tf_y:0': y}))
...     x, y = 2.0, 1.0
...     print('x < y: %s -> Result:' % (x < y),
...           res.eval(feed_dict={'tf_x:0': x,
...                               'tf_y:0': y}))
```

结果如下：

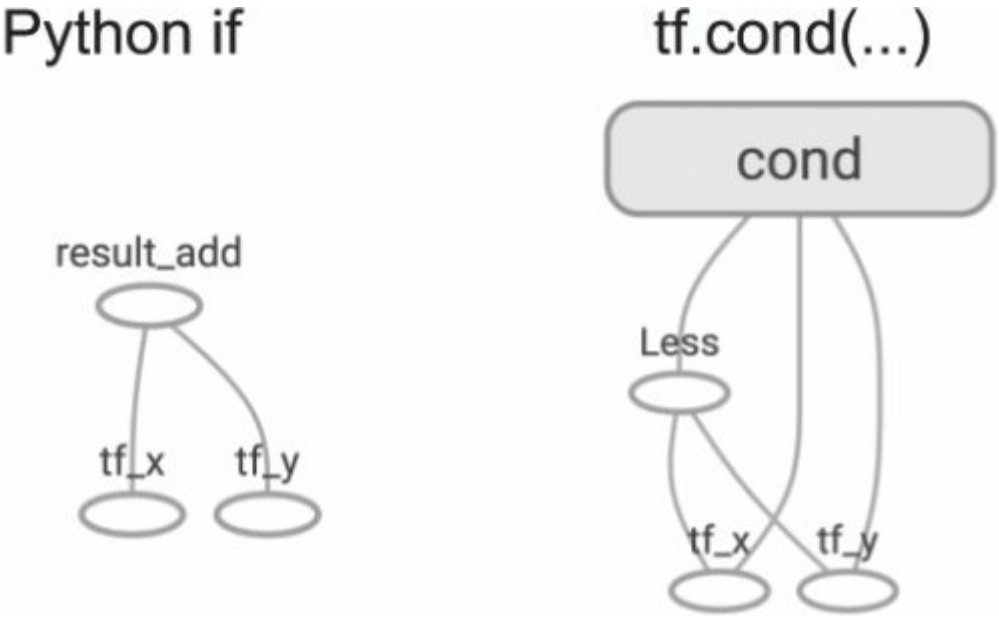
```
Object: Tensor("cond/Merge:0", dtype=float32)
x < y: True -> Result: 3.0
x < y: False -> Result: 1.0
```

这里可以看到res对象被命名为"cond/Merge：0"。在当前情况下，计算图有两个分支以及一个在执行时决定经过哪个分支的机构。因此，当x=2，y=1时，它经过加法分支，输出值为3.0，而当x=1，y=2时，经过减法分支，其结果为1.0。

下图对比了Python的if语句与TensorFlow的tf.cond函数在实现计算图时的差异。

除tf.cond之外，TensorFlow还提供了其他一些流控制运算符，例如tf.case和tf.while_loop。tf.case是TensorFlow的流控制命令，等同于Python的if…else语句。考虑下面的Python表达式：

```
if (x < y):
    result = 1
else:
    result = 0
```




tf.case相当于前面TensorFlow计算图中的条件执行语句，可以用如下代码实现：

```
f1 = lambda: tf.constant(1)
f2 = lambda: tf.constant(0)
result = tf.case([(tf.less(x, y), f1)], default=f2)
```

类似地，可以把while循环添加到TensorFlow的计算图中，每次循环变量

i将递增1直到达到阈值（threshold），代码如下：

```
i = tf.constant(0)
threshold = 100
c = lambda i: tf.less(i, 100)
b = lambda i: tf.add(i, 1)
r = tf.while_loop(cond=c, body=b, loop_vars=[i])
```

当然，你可以查阅官方文档https://www.tensorflow.org/api_guides/python/control_flow_ops以了解更多关于各种流控制运算符的信息。

你可能已经注意到这些计算图是由TensorBoard构建的，下一节将讨论TensorBoard。

## 14.11 用TensorBoard可视化图

TensorBoard是TensorFlow一个非常好的功能，它负责可视化和模型学习。可视化允许我们看到节点之间的连接，探索它们之间的依赖关系，并在需要时进行模型调试。

让我们把前面构建的由生成器和分类器组成的网络可视化。在定义辅助函数时，将复用一些代码。因此，请回顾14.5.4节，以了解函数build_generator和build_classifier的定义。这两个辅助函数将用于构建下面的图：

```
>>> batch_size=64
>>> g = tf.Graph()
>>>
>>> with g.as_default():
...     tf_X = tf.placeholder(shape=(batch_size, 100),
...                           dtype=tf.float32,
...                           name='tf_X')

...     ## build the generator
...     with tf.variable_scope('generator'):
...         gen_out1 = build_generator(data=tf_X,
...                                    n_hidden=50)
...
...     ## build the classifier
...     with tf.variable_scope('classifier') as scope:
...         ## classifier for the original data:
...         cls_out1 = build_classifier(data=tf_X,
...                                     labels=tf.ones(
...                                        shape=batch_size))
...
...         ## reuse the classifier for generated data
...         scope.reuse_variables()
...         cls_out2 = build_classifier(data=gen_out1[1],
...                                     labels=tf.zeros(
...                                         shape=batch_size))
```

注意，到目前为止构建图的过程并没有什么变化。构建图之后的可视化也很直接。下面几行代码可以实现可视化并导出图：

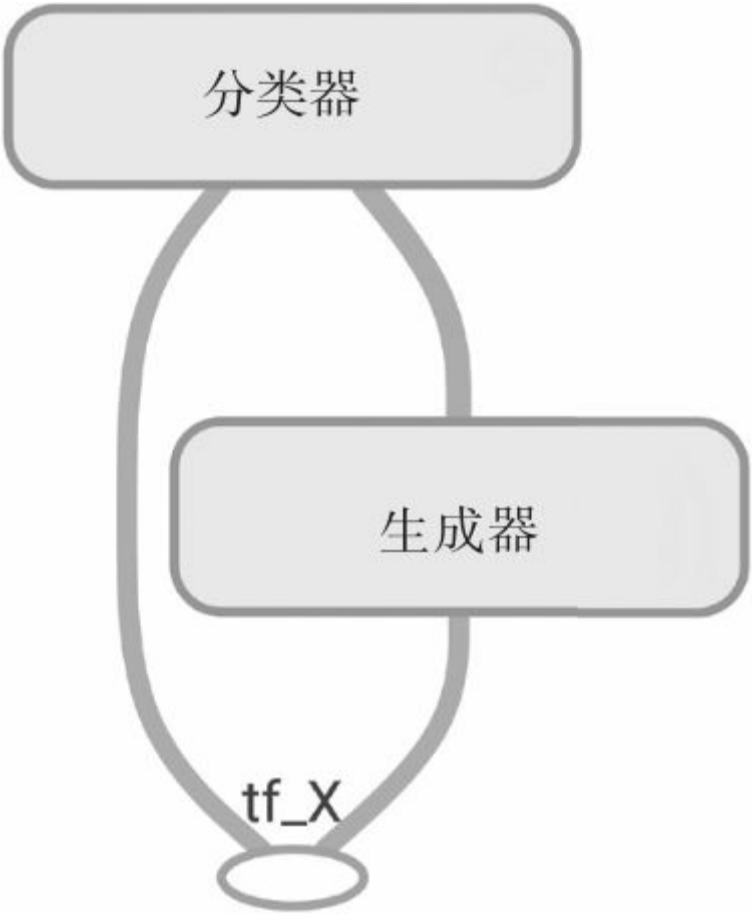
```
>>> with tf.Session(graph=g) as sess:
...     sess.run(tf.global_variables_initializer())
...
...     file_writer = tf.summary.FileWriter(
...                                 logdir='./logs/', graph=g)
```

这将会创建新目录：logs/，现在只需要在Linux或者macOS上运行下述命

令：

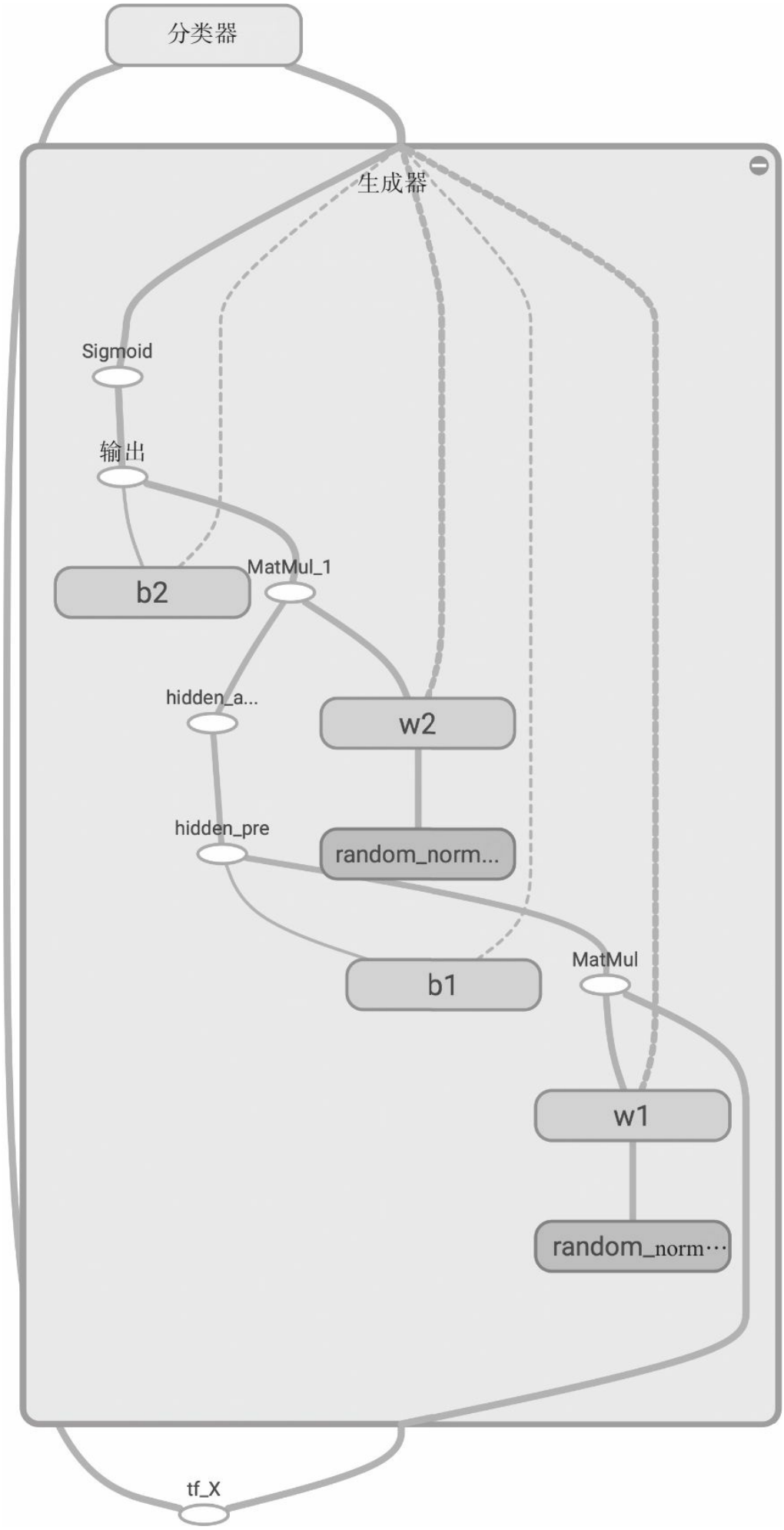
```
tensorboard --logdir logs/
```

该命令将显示一个URL地址。可以把诸如http://localhost:6006/#graphs的地址拷贝到浏览器的地址栏来启动TensorBoard。将会看到如下所示的与该模型相对应的图：

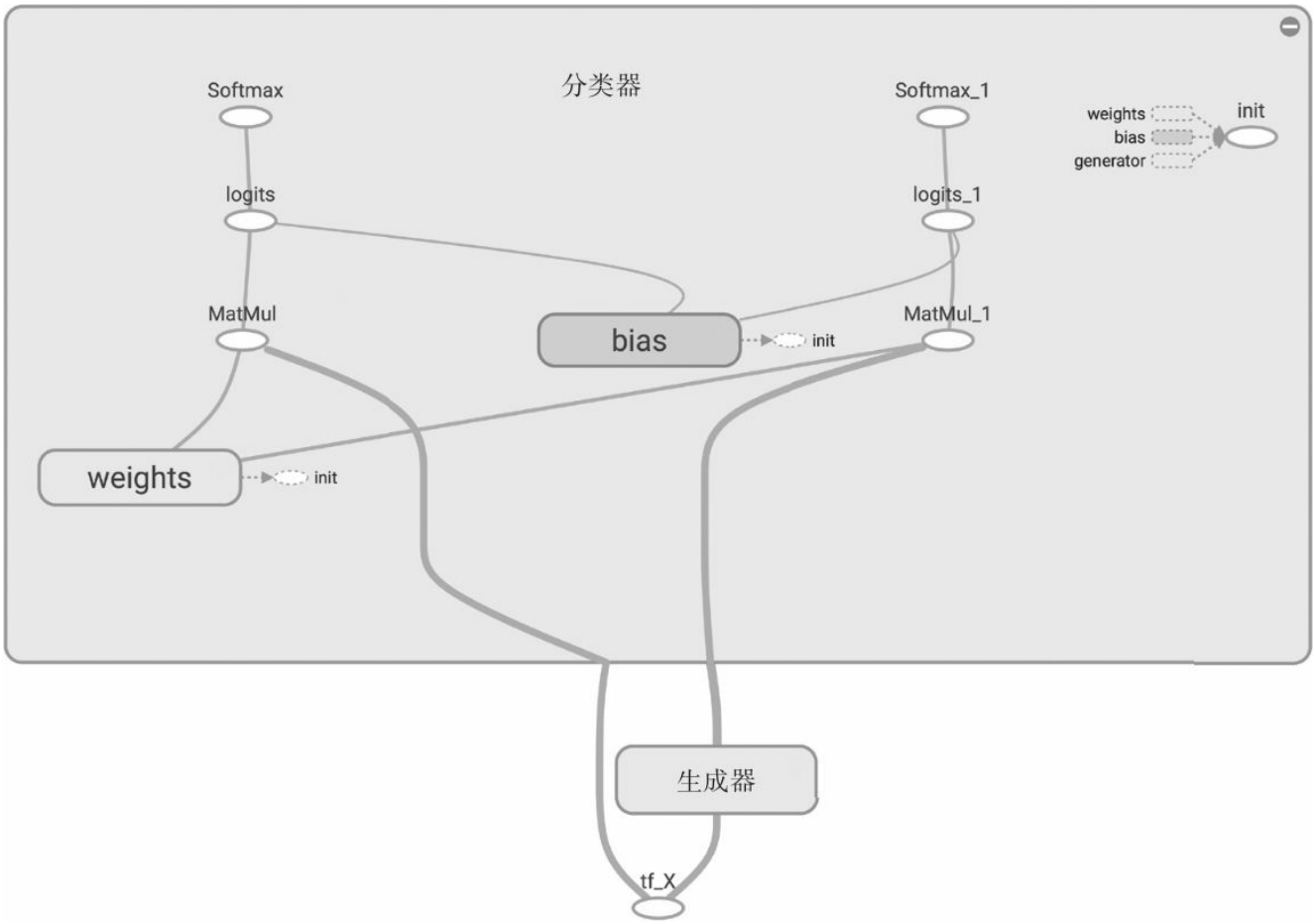



大的长方形框表示所构建的生成器和分类器两个子网络。调用tf.variable_scope函数构建该图，两个子网络的所有组成部分被收纳在上图的长方形框图中。

我们可以展开这两个框图以探索其细节：点击框图右上角的“+”展开，进而可以看到生成器子网络的细节，见下图：


分类器
生成器
Sigmoid
输出
b2
MatMul_1
hidden_a...
w2
hidden_pre
random_norm...
b1
MatMul
w1
random_norm…
tf_X

探索该图可以很容易地看到生成器有w1和w2两个权重张量。接着可以展开分类器子网络，见下图：




如上图所示，分类器有两个输入源，一个来自于tf_X占位符，另一个实际上是生成器子网络的输出。

### 拓展TensorBoard经验

建议用TensorBoard来可视化本章实现的不同图，以此作为一个有趣的练习。例如，可以用类似的步骤来构建计算图，然后为其可视化添加额外的线。还可以为流控制部分绘制图表，从而看到Python if语句和tf.cond函数之间在计算图上的差异。

有关可视化的更多信息和示例，请访问下述TensorFlow的官方教程网页：

https://www.tensorflow.org/get_started/graph_viz

## 14.12 小结

本章详细介绍了TensorFlow的主要功能和概念。首先讨论了TensorFlow的主要功能和优势，以及阶和张量等TensorFlow概念。然后研究了TensorFlow计算图，讨论了如何在会话环境中启动图，并学习了占位符和变量。接着介绍了评估张量和执行运算符的不同方法，即使用Python变量或引用图中的张量名。

我们进一步探讨了一些基本的TensorFlow运算符和张量变换函数，例如tf.transpose、tf.reshape、tf.split和tf.concat。最后介绍了如何使用TensorBoard可视化Tensor-Flow计算图。用该模块可视化计算图非常有用，特别是在调试复杂模型时。

下一章将实现先进的图像分类器：卷积神经网络（CNN）。CNN是一种功能强大的模型，在图像分类和计算机视觉方面性能卓著。我们将覆盖CNN的基本操作，用TensorFlow实现深度卷积网络并完成图像分类任务。

# 第15章　深度卷积神经网络图像识别

前一章深入研究了TensorFlow API的不同方面，熟悉了张量、变量命名和运算符，并且学会了如何使用变量域。本章将学习卷积神经网络（CNN）以及如何在TensorFlow中实现CNN。本章将开始一段有趣的旅程，将这种类型的深度神经网络架构应用于图像识别。

因此，我们将采用自下而上的方法讨论构建CNN的基本模块。然后深入研究CNN的体系结构，以及如何在TensorFlow中实现深度CNN。本章将按照这个思路讨论以下几个主题：

·理解一维和二维卷积运算

·了解构建CNN体系结构的模块

·在TensorFlow中实现深度卷积神经网络

## 15.1 构建卷积神经网络的模块

卷积神经网络（CNN）模型的灵感来自于人类大脑视觉皮层识别物体的工作原理。

CNN的发展可以追溯到上世纪90年代，当时燕乐存及其同事提出了一种新的神经网络体系结构用于识别手写数字（燕乐存等于1989年在《神经信息处理系统（NIPS）》大会上发表了《带有反向传播网络的手写数字识别》。

由于CNN在图像识别任务中表现突出，因此获得了广泛的关注，进而为机器学习和CNN计算机视觉应用带来了巨大的进步。

接下来的几节将看到如何将CNN用作特征提取引擎。接着我们将深入研究卷积理论的定义以及如何在一维和二维上计算卷积。

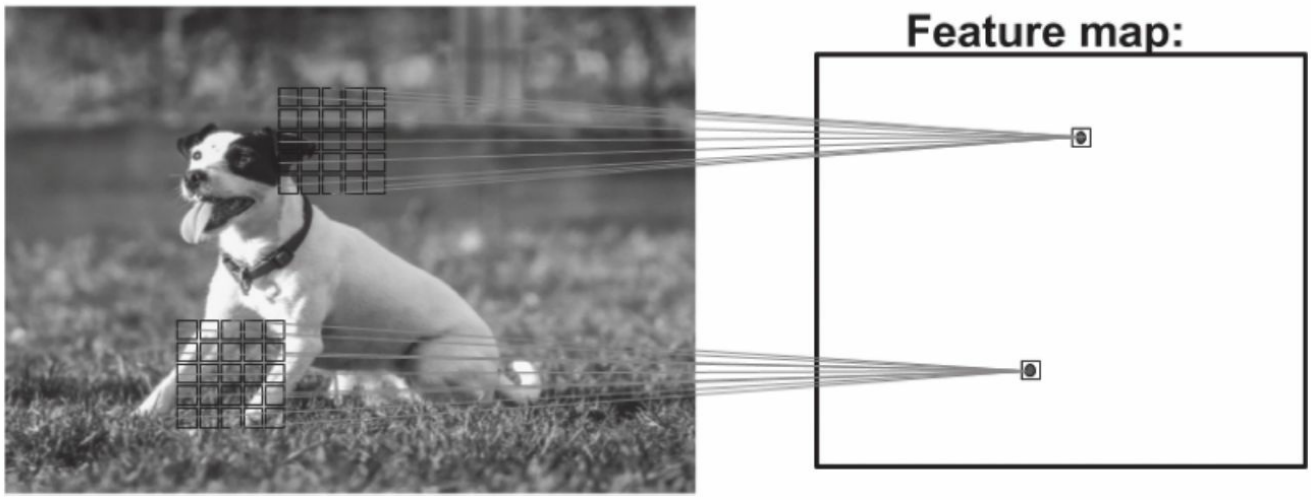
### 15.1.1 理解CNN与学习特征的层次

当然，成功地提取显著相关的特征是保障任何机器学习算法成功的关键，传统的机器学习模型依赖领域专家的输入特征，或者基于计算特征提取技术。神经网络能够自动地从原始数据中学习对特定任务最有用的特征。因此通常把神经网络作为特征提取引擎：即从紧靠着输入层后面的早期层提取低级特征。

多层神经网络，特别是深度卷积神经网络，通过逐层组合低级特征来构造所谓的特征层次，从而形成高级特征。例如可以在处理图像时从早期层把如边缘和斑点之类的低级特征提取出来，然后把它们组合在一起形成如建筑物、汽车或狗的形状的高级特征。

正如在下面的图像中可以看到的，CNN根据输入图像计算特征映射，其中每个元素都来自于输入图像中的局部像素区：

（图片来自 Unsplash，作者 Alexander Dummer）




该局部像素区被称为局部接收场。CNN通常会很好地完成与图像相关的任务，这主要是因为下面两个重要思想：

·稀疏连通：特征映射中的单个元素只连接到一小块像素区。（这与感知器连接整个输入图像非常不同。在第12章中回顾和比较我们如何实现与整个图像完全连接的网络将会很有帮助。

·参数共享：对于输入图像的不同区域使用相同的权重。

作为这两种思想的直接结果，网络的权重数急剧下降，并且可以看到模型捕获显著特征的能力得以提高。直观地说，彼此邻近的像素可能要比彼此远离的像素更加相关。

CNN通常由若干卷积层（conv）和子采样层（也称其为池层（P））所组成，尾随着一个或多个全连接（FC）层。全连接层在本质上是多层感知器，其中每个输入单元i以权重$w_{ij}$连接到每个输出单元j（第12章曾学习过）。

请注意，通常称为池层的子采样层没有任何可学习的参数。例如，池层没有权重或偏置单元。然而，卷积和完全连接层却都有这样的权重和偏置。

后续的章节将会更详细地研究卷积层和池层，并了解其工作机理。我们先从一维卷积讨论开始，理解卷积运算的具体原理，然后在典型的二维情况下将其应用到图像处理。

## 15.1.2 执行离散卷积

离散卷积（或简单称作卷积）是CNN的基本操作。因此了解该操作的工作原理很重要。本节将学习数学定义并讨论一些朴素的算法，以计算两个一维向量或两个二维矩阵的卷积。

请注意，此描述仅用于理解卷积的工作原理。将在本章后面看到，像TensorFlow这样的软件包事实上已经实现了更有效的卷积操作。

### 数据定义

本章将用下标表示多维阵列的大小。例如$A_{n1\times n2}$代表大小为$n_1\times n_2$的二维阵列。用[.]表示多维阵列的索引。例如A[i，j]表示矩阵A中索引为i，j的元素。另外请注意用特殊的符号*来表示两个向量或矩阵之间的卷积操作，不要与Python中的乘号混淆。

#### 15.1.2.1 计算一维离散卷积

首先从要用到的基本定义和符号开始。两个一维向量x和w的离散卷积表示为y=x*w，x为输入（有时也叫信号），而w为过滤器或者核。以数学语言表示离散卷积如下：

$$\boldsymbol{y}=\boldsymbol{x}*\boldsymbol{w}\rightarrow \boldsymbol{y}[i]=\sum_{k=-\infty}^{+\infty}\boldsymbol{x}[i-k]\boldsymbol{w}[k]$$

在这里，方括弧[]用来表示向量元素的索引。索引i遍历输出向量的每个元素。前面公式中有两个特别的问题需要澄清：-∞到+∞的指标和x的负索引。

### 交叉关联

输入向量x与过滤器w之间的交叉关联（或简单地说相关性）表示为y=x*w，与卷积非常类似，但稍有差别。其差别在于交叉关联的乘法在同一方向上进行。因此，不需要在每个维度中旋转滤波器矩阵w。交叉关联的数学定义如下：

$$y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{+\infty} x[i+k]w[k]$$

填充和步幅的相同规则也可以应用于交叉关联。

第一个问题是求和的上下指标为-∞到+∞，这似乎有些奇怪，因为机器学习应用所面对的特征向量总是有限。例如，如果x有10个特征，那么其索引为0，1，2，…，8，9，而-∞：-1和10：+∞不在x的范围内。因此，为了正确地计算前面公式中的和，假设以0填充x和w。这将产生输出向量y，同样无穷大而且有许多个0。由于在实际情况下这样做没有意义，所以只为x填充有限个0。

该过程被称为0填充或简称填充。这里每个边上填充0的个数由p表示。一维向量x的填充示例如下图所示：

假设原始输入x和过滤器w分别有n和m个单元，其中m≤n，因此，填充后的向量$x^p$的大小为n+2p。计算离散卷积的公式就变成下面这样：

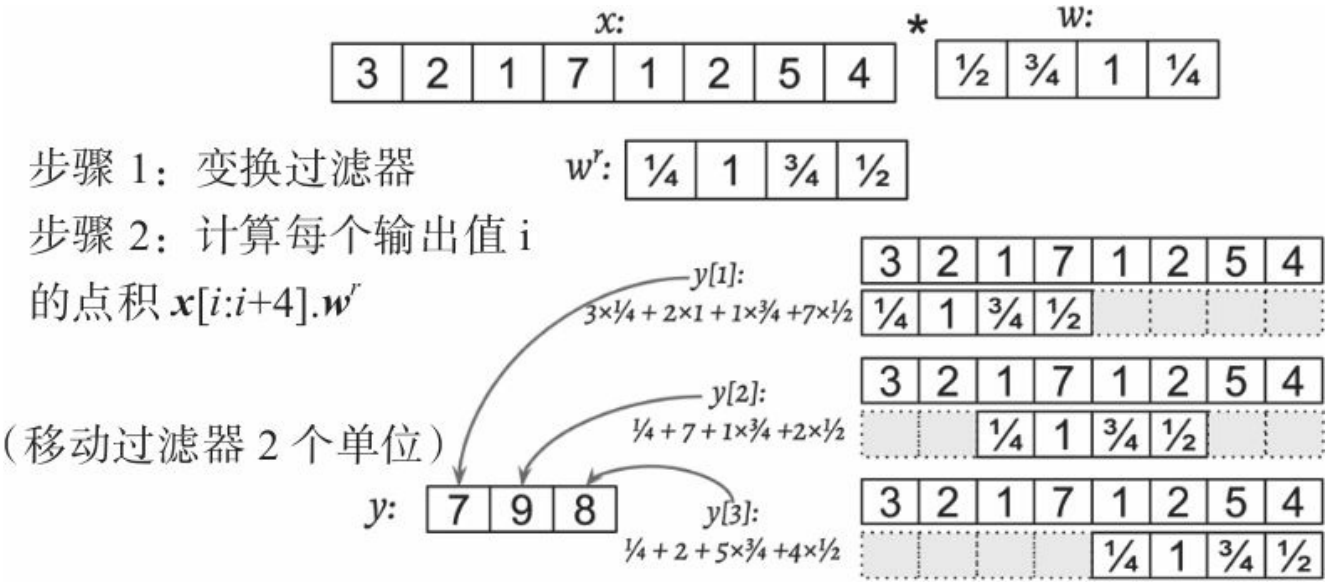
$$y = x * w \rightarrow y[i] = \sum_{k=0}^{k=m-1} x^p[i+m-k]w[k]$$

我们已经解决了无限索引问题，第二个问题是x的索引为i+m-k。重点是要注意x和w在求和过程中索引的方向不同。因此可以在填充之后，反转矩阵x和w。这样就可以计算它们的向量点积。

假设把过滤器向量w反转为$w^r$。那么计算点积x[i：i+m].$w^r$就得到y[i]，其中x[i：i+m]是向量x中大小为m的部分。

该操作就像滑动窗口那样，反复不断地计算从而得出所有的输出元素。下图以x=（3，2，1，7，1，2，5，4）和 $w = \left[\frac{1}{2}, \frac{3}{4}, 1, \frac{1}{4}\right]$ 为例计算前三个

输出元素：




前面的示例可以看到填充量为零（p=0）。注意到过滤器$w^r$通过两个元素旋转移位完成。这个移位是卷积的另一个超参数步幅s。该例的步幅为2，即s=2。注意，步幅必须是小于输入向量大小的正数。下一节将更详细地讨论填充和跨步。

#### 15.1.2.2 卷积中的零填充效应

到目前为止，卷积使用零填充来计算规模有限的输出向量。从技术上讲，当p≥0时可以采用填充。根据所选择的p值，向量x边界元素的处理方法可能不同于中间元素。

假设n=5，m=3。p=0，x[0]仅用于计算输出元素（例如y[0]），而x[1]用于计算两个输出元素（例如y[0]和y[1]）。因此可以看到对向量x元素的这种不同处理可以人为地把更多重点放在中间元素x[2]，因为它早就出现在大多数的计算中。如果选择p=2，则可以避免这个问题，在这种情况下，x的每个元素将参与计算y的三个元素。

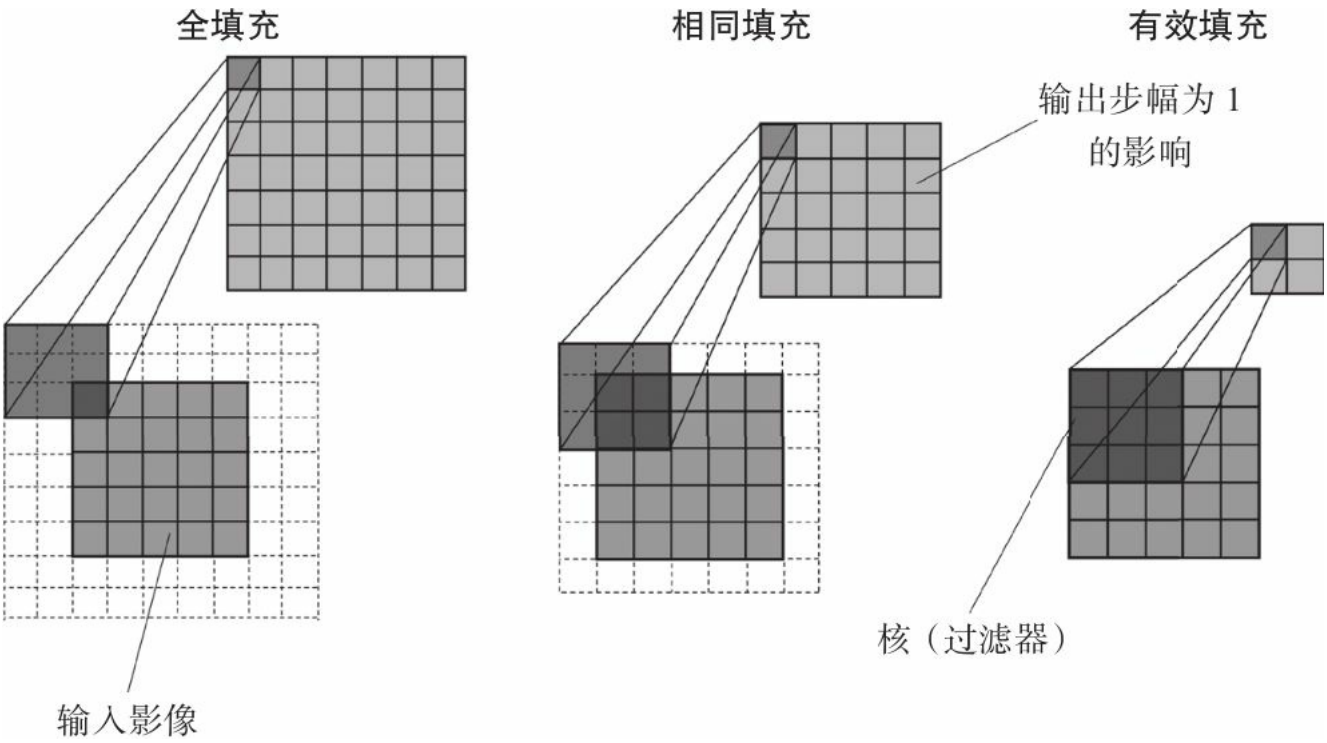
此外，输出向量y的大小也取决于所选择的填充策略。在实践中常用的填充模式有三种：完全、相同和有效：

·完全模式：填充参数p设置为p=m–1，因为增加了输出向量的维数，所以很少用于卷积神经网络的体系结构。

·相同模式：如果希望输出与输入向量大小相同，通常用该模式填充。除了要求输入和输出向量大小相同以外，还根据过滤器大小计算填充参数p。

·有效模式：最后在p=0，即无填充情况下计算卷积。

下图举例说明了三种不同的填充模式，该例有一个简单的5×5像素作为输入向量，其中核为3×3的向量，其步幅为1：




卷积神经网络中最常用的填充模式是相同填充。与其他填充模式相比，其优点是相同的填充保持了输入图像或张量的高度和宽度，使设计网络体系结构更加方便。

有效填充与完全填充和相同填充相比，存在着一个大缺点，例如张量的体积会在多层神经网络中显著减少，可能会对网络性能带来不利的影响。

建议在实践中使用相同填充的卷积层来保持空间的大小，而不是通过汇集层来减小空间。至于完整填充，其大小导致输出大于输入。全填充通常应用于着重于最小化边界效应的信号处理。然而在深度学习的场景中，边界效应通常不是问题，所以很少使用完全填充。

#### 15.1.2.3 确定卷积输出的大小

卷积输出的大小由沿着输入向量移动过滤器w的总次数来确定。假设输入向量的大小为n，过滤器的大小为m。在填充为p和步幅为的s条件下根据x*w计算输出向量的大小：

$$o = \left\lfloor \frac{n + 2p - m}{s} \right\rfloor + 1$$

这里⌊ ⌋代表向下取整操作：

向下取整返回相等或较小的最大整数，例如：

$$floor(1.77)=\lfloor 1.77 \rfloor=1$$

考虑下述两个例子：

·大小为10的输入向量，其卷积核的大小为5，填充为2和步幅1，据此计算输出向量的大小如下：

$$n-10,\ m=5,\ p=2,\ s=1 \rightarrow o=\left\lfloor \frac{10+2\times 2-5}{1} \right\rfloor+1=10$$

（注意在该例中输出向量的大小与输入向量的大小相同，因此得出结论它们是相同模式的填充，即mode='same'）

·如果输入向量相同，但卷积核的大小为3，步幅为2，输出向量的大小会发生什么样的变化？

$$n=10,\ m=3,\ p=2,\ s=2 \rightarrow o=\left\lfloor \frac{10+2\times 2-3}{2} \right\rfloor+1=6$$

如果有兴趣了解更多有关卷积输出向量大小的信息，推荐阅读文森特·杜穆林和弗朗西斯科·维辛在2016年撰写的《深度学习卷积算法指南》，可以从下述网站免费获取该论文：

https://arxiv.org/abs/1603.07285

最后，为了学习如何计算一维卷积，下面的代码片段中展示了一个不太成熟的实现，并将其结果与numpy.convolve函数做了比较，代码如下：

```
>>> import numpy as np
>>> def conv1d(x, w, p=0, s=1):
...     w_rot = np.array(w[::-1])
...     x_padded = np.array(x)
...     if p > 0:
...         zero_pad = np.zeros(shape=p)
...         x_padded = np.concatenate([zero_pad,
...                                    x_padded,
...                                    zero_pad])
```

```
...     res = []
...     for i in range(0, int(len(x)/s),s):
...         res.append(np.sum(x_padded[i:i+w_rot.shape[0]] *
...                           w_rot))
...     return np.array(res)

>>> ## Testing:
>>> x = [1, 3, 2, 4, 5, 6, 1, 3]
>>> w = [1, 0, 3, 1, 2]

>>> print('Conv1d Implementation:',
... conv1d(x, w, p=2, s=1))
Conv1d Implementation: [  5. 14. 16. 26. 24. 34. 19. 22.]

>>> print('Numpy Results:',
... np.convolve(x, w, mode='same'))
Numpy Results: [ 5 14 16 26 24 34 19 22]
```

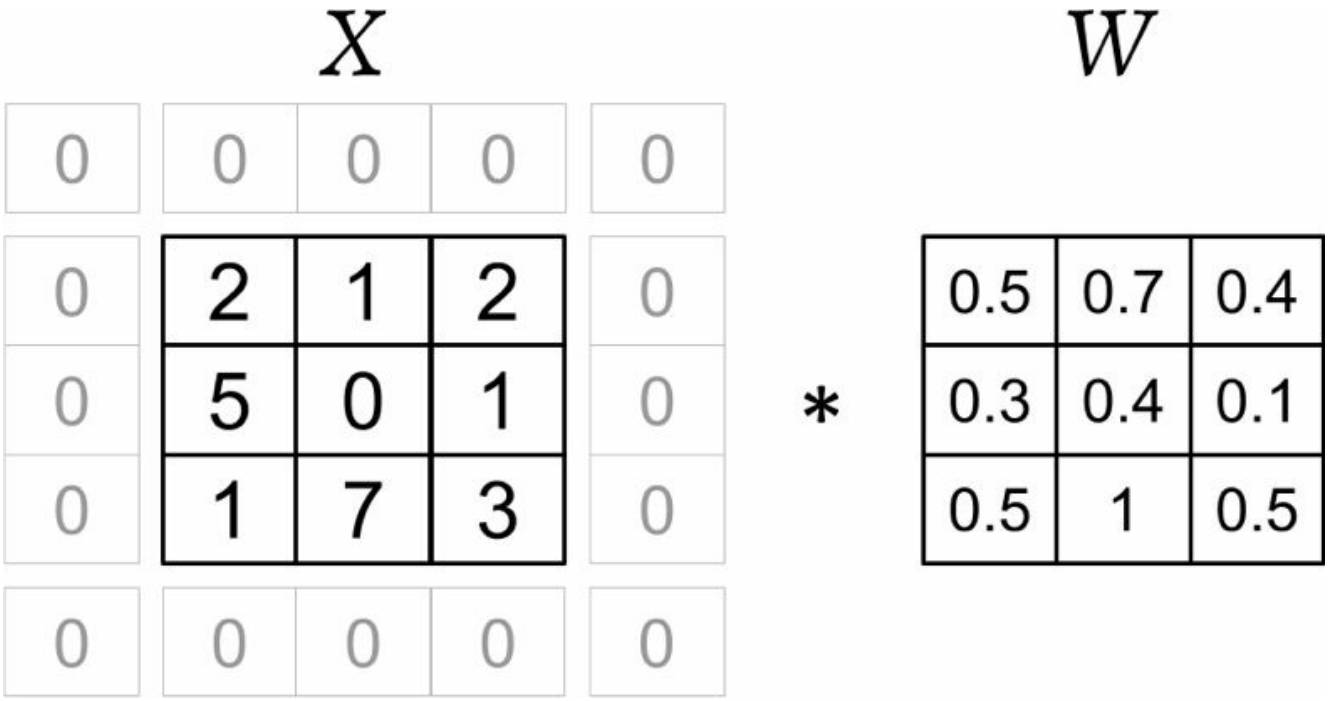
目前已经研究了一维卷积。从一维开始的原因是概念更容易理解。下一节我们将扩展到二维。

#### 15.1.2.4 二维离散卷积计算

前一节学到的一维概念很容易延伸到二维。在处理二维输入向量时，假设输入向量为$\boldsymbol{X}_{n_1\times n_2}$，过滤器为$\boldsymbol{W}_{m_1\times m_2}$，其中$m_1\le n_1$而且$m_2\le n_2$，那么矩阵Y=X*W就是X和W的二维卷积计算结果。以数学的语言表示如下：

$$\boldsymbol{Y}=\boldsymbol{X}*\boldsymbol{W}\rightarrow \boldsymbol{Y}[i,j]=\sum_{k_1=-\infty}^{+\infty}\sum_{k_2=-\infty}^{+\infty}\boldsymbol{X}[i-k_1,j-k_2]\boldsymbol{W}[k_1,k_2]$$

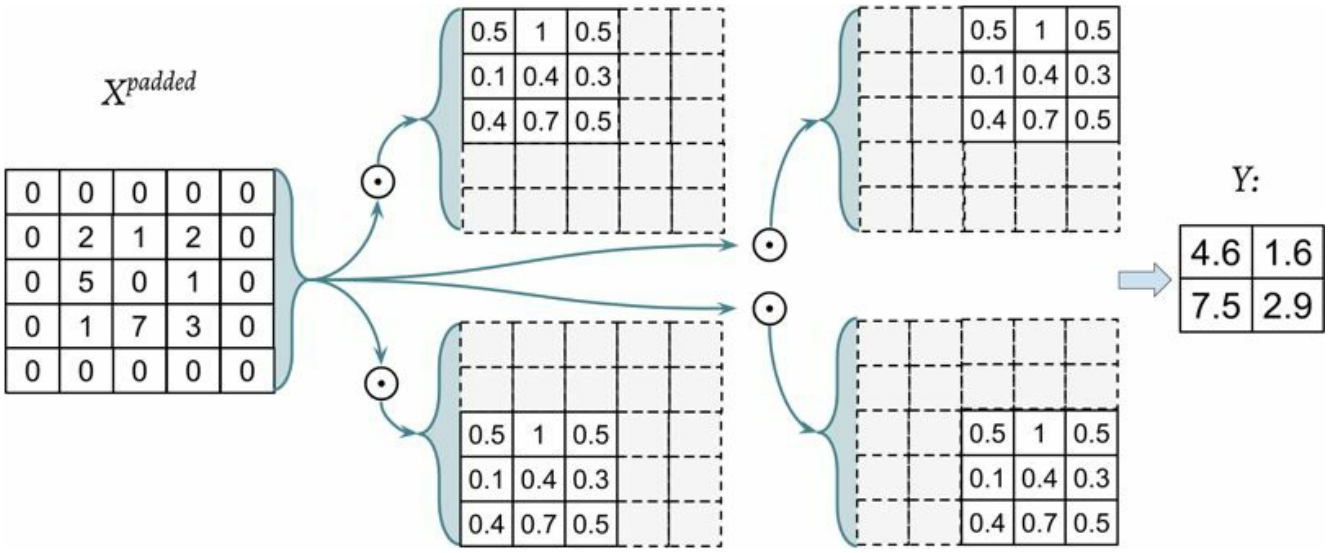
可以发现，如果忽略一个维度，剩下的公式与前面用到的计算一维卷积的公式一样。事实上，假设可以独立扩展到各自的维度，所有前面提到的技术，例如0填充、过滤器矩阵旋转，以及步幅都可以应用到二维卷积。下面的例子说明了如何计算输入矩阵$X_{3\times3}$与填充p=（1，1）且步幅s=（2，2）的核矩阵$W_{3\times3}$之间的二维卷积。根据所定义的填充，输入矩阵的每个边都填充0形成$X^{padded}_{5\times5}$矩阵。




前面的过滤器在旋转之后得到如下的结果：

$$\boldsymbol{W}^{r} = \begin{bmatrix} 0.5 & 1 & 0.5 \\ 0.1 & 0.4 & 0.3 \\ 0.4 & 0.7 & 0.5 \end{bmatrix}$$

注意该转换与转置矩阵不同。在NumPy中要变换过滤器向量，可以表示为W_rot=W[：：-1，：：-1]。接着，可以把转换后的过滤器矩阵沿着输入矩阵$X^{padded}$，像不断移动滑动窗口那样来计算元素乘积的和，可以图示该过程如下：



结果是2×2矩阵Y。

根据前面描述的初级算法来实现二维卷积。scipy.signal调用函数scipy.signal.convolve2d来计算二维卷积：

```
>>> import numpy as np
>>> import scipy.signal

>>> def conv2d(X, W, p=(0, 0), s=(1, 1)):
...     W_rot = np.array(W)[::-1,::-1]
...     X_orig = np.array(X)
...     n1 = X_orig.shape[0] + 2*p[0]
...     n2 = X_orig.shape[1] + 2*p[1]
...     X_padded = np.zeros(shape=(n1, n2))
...     X_padded[p[0]:p[0]+X_orig.shape[0],
...              p[1]:p[1]+X_orig.shape[1]] = X_orig
...
...     res = []
...     for i in range(0, int((X_padded.shape[0] - \
...                            W_rot.shape[0])/s[0])+1, s[0]):
...         res.append([])
...         for j in range(0, int((X_padded.shape[1] - \

...                                 W_rot.shape[1])/s[1])+1, s[1]):
...             X_sub = X_padded[i:i+W_rot.shape[0],
...                              j:j+W_rot.shape[1]]
...             res[-1].append(np.sum(X_sub * W_rot))
...     return(np.array(res))

>>> X = [[1, 3, 2, 4], [5, 6, 1, 3], [1, 2, 0, 2], [3, 4, 3, 2]]
>>> W = [[1, 0, 3], [1, 2, 1], [0, 1, 1]]

>>> print('Conv2d Implementation:\n',
...       conv2d(X, W, p=(1, 1), s=(1, 1)))
Conv2d Implementation:
 [[ 11.  25.  32.  13.]
 [ 19.  25.  24.  13.]
 [ 13.  28.  25.  17.]
 [ 11.  17.  14.   9.]]

>>> print('SciPy Results:\n',
...       scipy.signal.convolve2d(X, W, mode='same'))
SciPy Results:
 [[11 25 32 13]
 [19 25 24 13]
 [13 28 25 17]
 [11 17 14  9]]
```

上面实现了一个初级的二维卷积计算，目的是为了理解概念。然而，就存储需求和计算复杂度来说，这样的实现非常低效。因此不应该把它应用于现实世界的神经网络。

近年来，使用傅里叶变换计算卷积的算法更为有效。同样重要的是要注

意神经网络卷积核的大小通常比输入图像要小得多。例如，目前CNN常用的核大小为1×1，3×3或5×5，这对于可以完成卷积操作的高效算法而言非常有效，例如威诺格拉德的最小过滤器算法。

对这些算法的讨论超出了本书的范围，但是如果你有兴趣了解更多，可以阅读由安得烈·拉文和史考特·葛瑞在2015年撰写的《卷积神经网络的快速算法》，该论文可以从下面的网址免费获得：

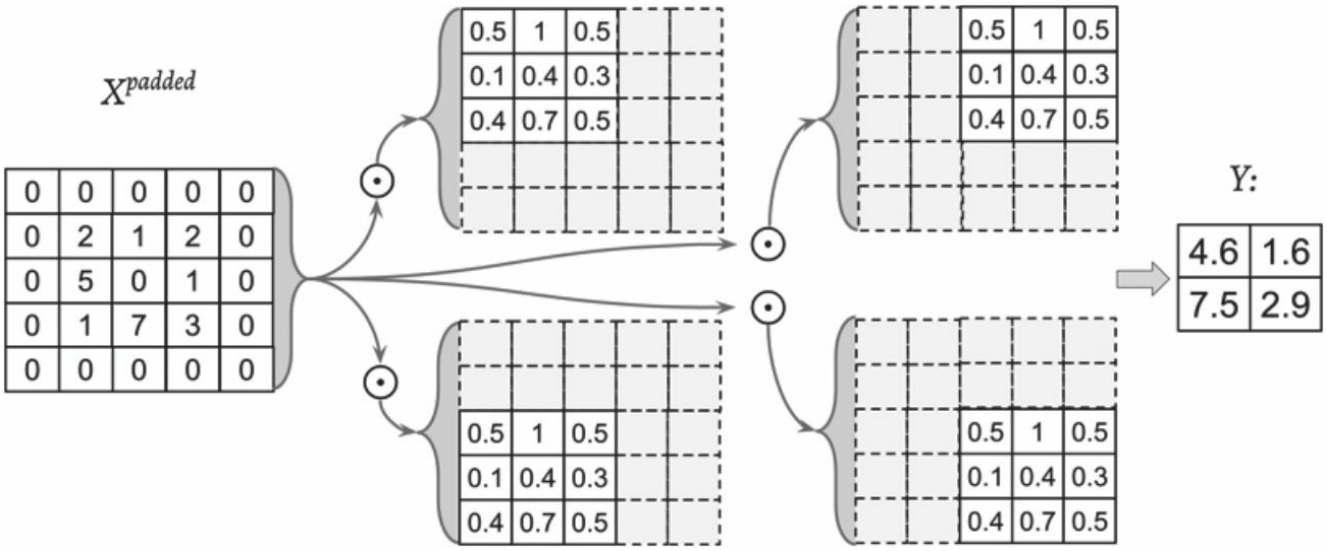
https://arxiv.org/abs/1509.09308

下一节将讨论子采样，这是CNN经常使用的另外一个重要操作。

## 15.1.3 子采样

子采样通常用于卷积神经网络两种形式的池操作：最大池（max-pooling）和平均池（mean-pooling）（也被称为average-pooling）。池层通常表示为$\boldsymbol{P}_{n_1 \times n_2}$，这里下标确定了邻区的大小（每个维度上相邻区域的像素数量），在这里进行最大或者平均池操作。该相邻区域被称为池大小（pooling size）。

下图将描述该操作。这里最大池从像素的邻域取得最大值，平均池计算它们的平均值：




池化的好处有两点：

·最大池（max-pooling）引入了局部的不变性。这意味着在局部邻域的小变化不会改变最大池的结果。因此有助于生成抗输入数据噪声的特性。下面的示例显示出对两个不同的输入矩阵$X_1$和$X_2$进行最大池操作得到相同的输出结果。

$$\left.\begin{matrix} \boldsymbol{X}_1 = \begin{bmatrix} 10 & 255 & 125 & 0 & 170 & 100 \\ 70 & 255 & 105 & 25 & 25 & 70 \\ 255 & 0 & 150 & 0 & 10 & 10 \\ 0 & 255 & 10 & 10 & 150 & 20 \\ 70 & 15 & 200 & 100 & 95 & 0 \\ 35 & 25 & 100 & 20 & 0 & 60 \end{bmatrix} \\ \boldsymbol{X}_2 = \begin{bmatrix} 100 & 100 & 100 & 50 & 100 & 50 \\ 95 & 255 & 100 & 125 & 125 & 170 \\ 80 & 40 & 10 & 10 & 125 & 150 \\ 255 & 30 & 150 & 20 & 120 & 125 \\ 30 & 30 & 150 & 100 & 70 & 70 \\ 70 & 30 & 100 & 200 & 70 & 95 \end{bmatrix} \end{matrix}\right\} \underrightarrow{\max - \text{pooling} P_{2\times 2}} \begin{bmatrix} 255 & 125 & 170 \\ 255 & 150 & 150 \\ 70 & 200 & 95 \end{bmatrix}$$

·池化减少特征数量，因而提高计算效率。此外，减少特征数量也可以减轻过拟合程度。

传统上假设池不重叠。通常如果要在非重叠邻域上进行池化，可以通过设置步长参数等于池的大小来实现。例如，非重叠池层$\boldsymbol{P}_{n_1\times n_2}$需要步长参数s=（$n_1$，$n_2$）。

另一方面，如果步幅小于池的大小，则会发生池重叠。由A.克里什夫茨基，I.苏特斯克和G.欣顿，于2012年撰写的《基于深度卷积神经网络的ImageNet分类》，描述了在卷积网络中使用重叠池的例子，该论文可以从下述网站免费获得：

https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks

## 15.2 拼装构建CNN

到目前为止，我们已经了解了卷积神经网络的基本模块。本章所阐明的概念并不比传统的多层神经网络更难理解。直观地说，传统的神经网络中最重要的运算是矩阵向量乘法。

例如，我们用矩阵向量乘法来进行预激活（或净输入），如a=Wx+b。这里x代表像素的列向量，W为连接像素与每个隐藏单元的权重矩阵。在卷积神经网络中，该操作由像A=W*X+b这样的卷积运算代替，其中X代表高度×宽度的像素矩阵。在这两种情况下，预激活被传递给激活函数以获得隐藏单元H=φ（A）的激活，其中是激活函数。此外，回想一下子采样是卷积神经网络的另一个构建模块，就像前一节所描述的那样以池的形式出现。

### 15.2.1 处理多个输入或者彩色频道

卷积层的输入样本可能包含一个或者多个二维阵列或$N_1 \times N_2$矩阵（例如图像的高度像素和宽度像素）。这些$N_1 \times N_2$矩阵被称为频道。用多频道作为卷积层的输入需要使用3阶张量或者三维阵列：$X_{N_1 \times N_2 \times C_{in}}$，其中$C_{in}$为输入频道的个数。

例如，考虑以图像作为CNN的第一层输入。如果是彩色图像而且采用RGB色彩表示，那么$C_{in}=3$（RGB的红、绿、蓝三原色）。然而，如果是灰度图像，那么$C_{in}=1$，因为只有一个频道有像素的灰度值。

在处理图像时，可以用'uint8'（无符号8位整数）数据类型把图像读入NumPy阵列，与16、32或64位整数型相比，这样做可以减少内存的使用量。无符号8位整数的取值范围在[0，255]，足以存储RGB图像的像素信息，因为这也是RGB的取值范围。接下来，通过一个例子看一下如何用SciPy把图像读入Python会话。然而，请记住，用SciPy读入图像需要系统先安装PIL（Python图像库）。可以用比PIL更友好的Pillow（https://python-pillow.org）命来令安装：pip install pillow

一旦Pillow安装完毕，就可以从scipy.misc模块调用imread函数来读入RGB图像（示例图像可以从本章所提供的代码汇总文件夹中找到，具体的网络链接如下：

https://github.com/rasbt/python-machine-learning-book-2nd-edition/tree/master/code/ch15

```
>>> import scipy.misc
>>> img = scipy.misc.imread('./example-image.png',
...                         mode='RGB')
>>> print('Image shape:', img.shape)
Image shape: (252, 221, 3)
>>> print('Number of channels:', img.shape[2])
Number of channels: 3
>>> print('Image data type:', img.dtype)
Image data type: uint8
>>> print(img[100:102, 100:102, :])
 [[[179 134 110]
  [182 136 112]]

 [[180 135 111]
  [182 137 113]]]
```

到目前为止，我们已经熟悉了输入数据的结构，下面的问题就是如何整合在前节中讨论过的卷积操作的多个输入频道。

答案很简单：为每个频道独立进行卷积操作，然后用矩阵求和把结果累加起来。与每个频道（c）相关联的卷积有自己的核矩阵W[：，：，c]。预激活的结果通过下列方程计算：

$$
\begin{aligned}
&\text{样本}\boldsymbol{X}_{n_1\times n_2\times c_{in}'}\\
&\text{核矩阵}\ \boldsymbol{W}_{m_1\times m_2\times c_{in}'}\\
&\text{偏置值为}\ b
\end{aligned}
\Rightarrow
\begin{cases}
\boldsymbol{Y}^{Conv}=\sum_{c=1}^{C_{in}}\boldsymbol{W}[:,:,c]*\boldsymbol{X}[:,:,c]\\
\text{预激活：}\ \boldsymbol{A}=\boldsymbol{Y}^{Conv}+b\\
\text{特征分布图：}\boldsymbol{H}=\phi(\boldsymbol{A})
\end{cases}
$$

我们把最后的结果h称为特征分布图。CNN卷积层通常可以有多个特征分布图。如果用多个特征分布图，核张量就变成宽度×高度×$C_{in}$×$C_{out}$四个维度。在这里，宽度×高度为核大小，$C_{in}$是输入的频道数量，$C_{out}$为输出的特征分布图的数量。因此，可以在前面的公式中包含输出特征分布图的数量，如下所示：

样本 $\boldsymbol{X}_{n_1\times n_2\times C_{in}}$

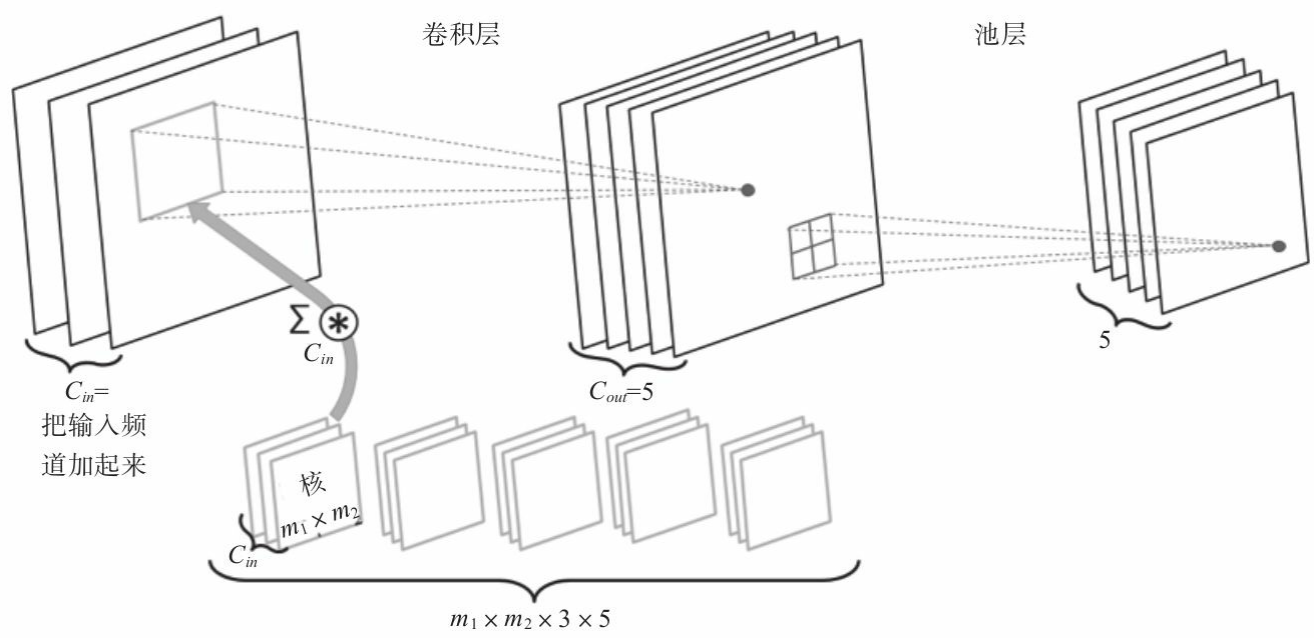
核矩阵 $\boldsymbol{W}_{m_1\times m_2\times C_{in}\times C_{out}}$

偏置值为 $\boldsymbol{b}_{C_{out}}$

$$\Rightarrow\begin{cases}\boldsymbol{Y}^{Conv}[:,:k]=\sum_{c=1}^{C_{in}}\boldsymbol{W}[:,:,c,k]*\boldsymbol{X}[:,:,c]\\ \boldsymbol{A}[:,:,k]=\boldsymbol{Y}^{Conv}[:,:k]+\boldsymbol{b}[k]\\ \boldsymbol{H}[:,:,k]=\phi(\boldsymbol{A}[:,:k])\end{cases}$$

为了对神经网络卷积计算的讨论做出总结，让我们观察连接着一个池层的卷积层的例子。

该例有三个输入频道。核张量有四个维度。每个核矩阵用$m_1\times m_2$表示，总共有3个核矩阵，分别对应一个输入频道。另外还有5个这样的核负责5个特征分布图。最后，有一个池化层对应特征分布图的子取样，详情见下图：




在上面的例子中，总共有多少个可以训练的参数存在呢？

下面通过实例解释卷积在参数共享和稀疏链接方面的优势。上图网络中的卷积层是一个四维张量。所以有$m_1\times m_2\times 3\times 5$个与核相关的参数。另外，还有与卷积层的每个输出特征图相对应的偏置向量。因此偏置向量的大小为5。池层并没有可以训练的参数存在，所以可以表示为：

$$m_1\times m_2\times 3\times 5+5$$

如果输入大小为$n_1\times n_2\times 3$的张量，假设以mode='same'执行卷积，那么数据特征图的大小为$n_1\times n_2\times 5$。请注意如果想要全连结层而不是卷积层，这个数字就会小很多。在全连结层情况下，要达到同样输出单元的权重矩阵参数

量将会是：

$$(n_1 \times n_2 \times 3) \times (n_1 \times n_2 \times 5) = (n_1 \times n_2) 2 \times 3 \times 5$$

假设$m_1 < n_1$且$m_2 < n_2$，我们可以看到在可训练参数量方面存在着巨大的差异。

下一节将讨论如何正则化神经网络。

### 15.2.2 通过淘汰正则化神经网络

无论是处理传统的全连接神经网络还是CNN，如何选择网络的大小一直是个具有挑战性的问题。例如需要调优权重矩阵的大小和层数以获得最佳性能。

网络容量指要学习特征的复杂水平。小网络的参数相对较少，容量较低，因为在复杂数据集的结构下无法学习，很可能欠拟合导致性能较差。

然而，大网络可能更容易过拟合，网络会记住训练数据，因此在训练集上表现得特别好，而在预留的测试集上却表现不佳。在处理现实世界机器学习问题时，并没有办法预先知道网络的规模。

解决该问题的办法是构建容量相对较大的网络（实际上选择比需求略大的网络）以确保模型在训练集上表现良好。然后为了避免过拟合，应用一个或者多个正则化规则以在新数据（例如预留数据）上获得好的泛化性能。较常用的正则化规则是本书前面讨论过的L2规则。

近年来，出现了另外一种被称为淘汰（dropout）的正则化技术，对正则化（深度）神经网络效果奇佳（尼提斯·斯里瓦斯塔瓦等.《淘汰；一种避免神经网络过拟合的简单方法》.机器学习研究15.1，2014：1929-1958，http://www.jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf）。

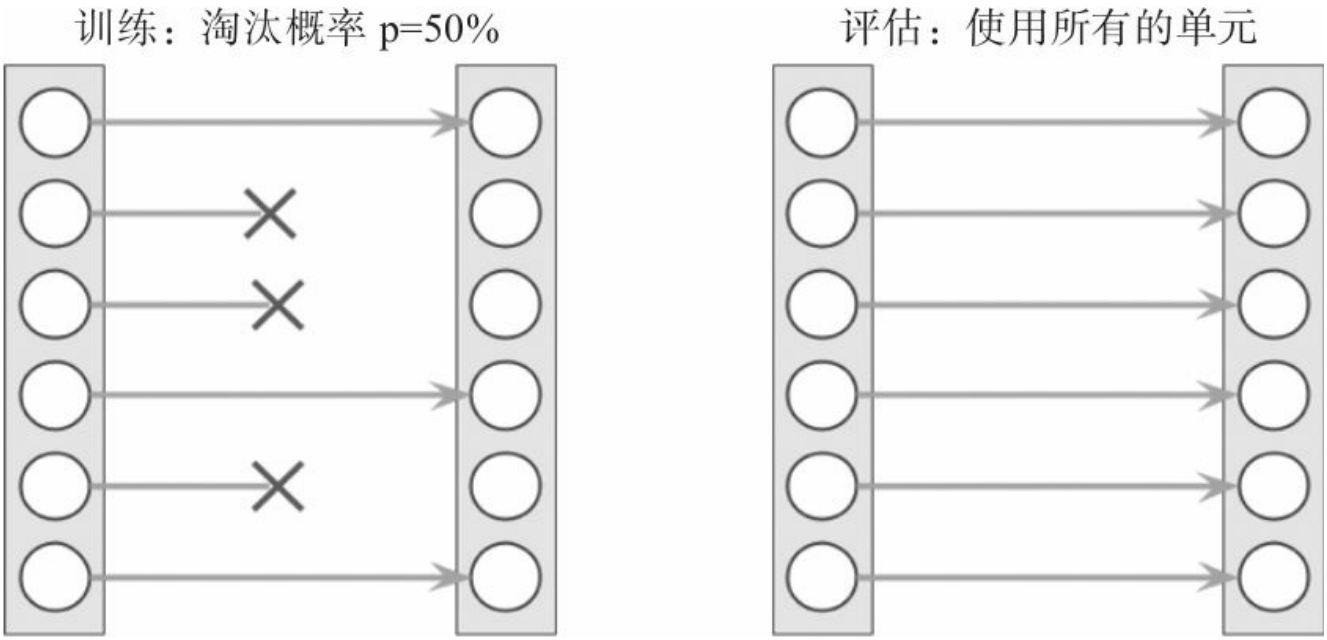
形象地说，淘汰可以被认为是一组模型的共识或平均方法。组合学习时独立训练模型。预测时则采用所有训练模型预测结果的共识。然而，训练几个模型并收集和平均多个模型的输出结果所带来的计算成本昂贵。因此，淘汰在提供了一种有效方法来同时训练许多模型，并计算它们在测试和预测时的平均结果。

淘汰技术通常应用于较高层的隐藏单元。在神经网络模型的训练阶段，每次迭代都会有一小部分的隐藏单元根据淘汰率$p_{drop}$（换句话说$p_{keep}=1-p_{drop}$）被随机淘汰。

淘汰率是由用户设置的，通常为p=0.5，正如前面提到的尼提斯·斯里瓦斯塔瓦等人在2014年撰写的论文中所讨论的那样。当淘汰某些输入神经元时，与剩余神经元相关联的权重会重新调整以补偿淘汰的神经元。

这一随机淘汰的效果是逼迫网络向冗余的数据代表学习。因此，网络无法依赖任何单一组别隐藏单元的激活，因为在训练过程中它们随时有可能被淘汰，结果被逼从数据中学习更多通用且强有力的模式。

该淘汰技术可以有效地避免过拟合。下图展示了在训练集上淘汰率为p=0.5的应用案例，一半的神经元都随机进入不活跃状态。然后，在预测的过程中，所有的神经元会为计算预激活做出贡献。




如上所示，关键是单元可能会在训练过程中被随机淘汰，而在评估阶段，所有的隐藏单元又必须被激活（例如$p_{drop}=0$或$p_{keep}=1$）。要确保在训练和预测阶段所有的激活函数保持相同的比例，活跃神经元的激发比例必须进行适当的调整（例如淘汰概率设置为p=0.5意味着激活一半）。然而，因为在进行预测的实际过程中不方便持续调整比例，所以TensorFlow及其他工具在训练阶段调整激活比例（例如，如果淘汰概率设置为p=0.5，激活数量将变为两倍）。

那么淘汰和组合学习之间是什么关系呢？因为每次迭代都要淘汰不同的隐藏神经元，所以实际上在训练不同的模型。当所有这些模型最后训练完成时，把保持率设置为1从而使用所有的隐藏单元。这意味着对所有隐藏单元取平均激活数。

## 15.3 用TensorFlow实现深度卷积神经网络

在第13章中，你可能还记得我们为解决手写数字识别问题而采用不同层级的TensorFlow API实现了多层神经网络。或许还记得我们达到了97%的准确率。

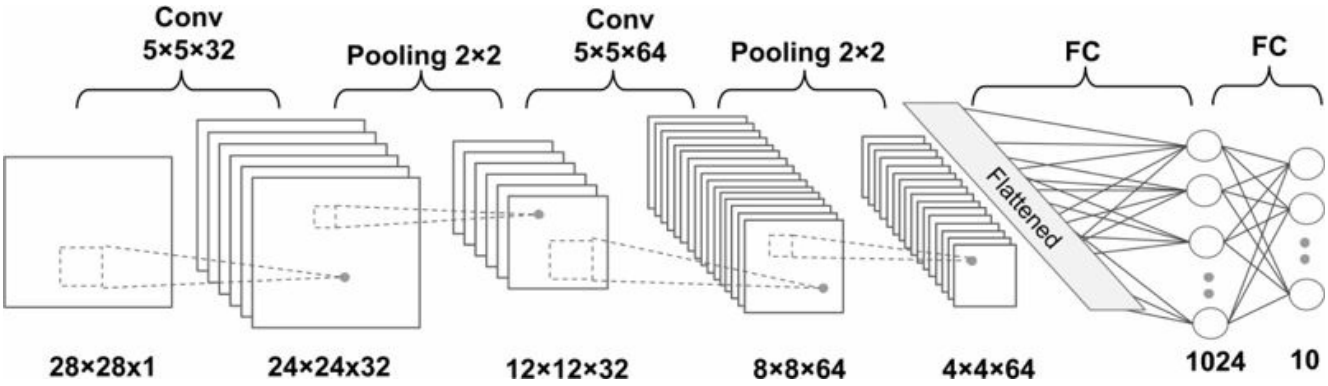
现在想要实现CNN来解决同样的问题，观察CNN识别手写数字的能力。请注意，第13章所看到的全连接层能够很好地解决这个问题。然而，在有些应用中，诸如读取银行账户的手写数字，即使微小的错误也会导致非常重大的损失。因此，尽可能减少错误极为关键。

## 15.3.1 多层CNN体系结构

下图展示了将要实现网络的体系结构。其输入是28×28像素的灰度图像。考虑到频道数量（灰度图像频道为1）和批量输入图像，输入张量维度为批量大小×28×28×1。

输入数据遍历两个核大小为5×5的卷积层。第一个卷积层有32个输出特征分布图，第二个有64个输出特征分布图。每个卷积层都与以最大池（max-pooling）操作的子采样层相连接。

接着全连接层把输出传递给第二个全连接层，该层被当成是最终的softmax输出层。要实现网络的体系结构如下图所示：




每层张量的维度说明如下：

5.Input：[batchsize×28×28×1]

6.Conv_1：[batchsize×24×24×32]

7.Pooling_1：[batchsize×12×12×32]

8.Conv_2：[batchsize×8×8×64]

9.Pooling_2：[batchsize×4×4×64]

10.FC_1：[batchsize×1024]

11.FC_2与softmax层：[batchsize×10]

下面将要用TensorFlow的低级API和TensorFlow的Layers API来实现该网络。但是要先定义几个辅助函数。

## 15.3.2 加载和预处理数据

还记得第13章调用load_mnist函数读入MNIST的手写数字数据集吗？现在需要重复同样的过程：

```
>>> #### Loading the data
>>> X_data, y_data = load_mnist('./mnist/', kind='train')
>>> print('Rows: {},  Columns: {}'.format(
...             X_data.shape[0], X_data.shape[1]))
>>> X_test, y_test = load_mnist('./mnist/', kind='t10k')
>>> print('Rows: {},  Columns: {}'.format(
...             X_test.shape[0], X_test.shape[1]))

>>> X_train, y_train = X_data[:50000,:], y_data[:50000]
>>> X_valid, y_valid = X_data[50000:,:], y_data[50000:]

>>> print('Training:   ', X_train.shape, y_train.shape)
>>> print('Validation: ', X_valid.shape, y_valid.shape)
>>> print('Test Set:   ', X_test.shape, y_test.shape)
```

把数据分裂成训练集、验证集和测试集。下面的结果描述了各个数据子集：

```
Rows: 60000,  Columns: 784
Rows: 10000,  Columns: 784
Training:    (50000, 784) (50000,)
Validation:  (10000, 784) (10000,)
Test Set:    (10000, 784) (10000,)
```

加载数据之后，只要一个函数就可以迭代小批量的数据，如下所示：

```
>>> def batch_generator(X, y, batch_size=64,
...                     shuffle=False, random_seed=None):
...
...     idx = np.arange(y.shape[0])
...
...     if shuffle:
...         rng = np.random.RandomState(random_seed)
...         rng.shuffle(idx)
...         X = X[idx]
...         y = y[idx]

...     for i in range(0, X.shape[0], batch_size):
...         yield (X[i:i+batch_size, :], y[i:i+batch_size])
```

这个函数返回匹配样本的元组的生成器，例如数据X和标签y。接着，为了能更好地训练性能和收敛性，需要归一化数据（均值居中且除以标准方差）。

用训练数据（X_train）计算每个特征的均值和所有特征的标准方差。不计算每个特征标准方差的原因是像MNIST这样的图像数据集中的有些特征（像素位置），所有灰度图像所对应的白色像素的常数值为255。

因为所有样本的常数无变化，所以那些特征的标准方差为零，结果会出现分母为零的错误，这就是为什么在计算X_train阵列标准方差时用np.std而不定义参数axis：

```
>>> mean_vals = np.mean(X_train, axis=0)
>>> std_val = np.std(X_train)

>>> X_train_centered = (X_train - mean_vals)/std_val
>>> X_valid_centered = (X_valid - mean_vals)/std_val
>>> X_test_centered = (X_test - mean_vals)/std_val
```

现在已经准备好实现刚才描述过的CNN了。下面将在TensorFlow中实现模型。

### 15.3.3 用TensorFlow的低级API实现CNN模型

为了在TensorFlow中实现CNN模型，首先要定义两个封装函数来简化网络构建的过程：一个封装函数是为卷积层准备的，另外一个是为构建全连接层准备的。

为卷积层准备的第一个函数如下：

```
import tensorflow as tf
import numpy as np

def conv_layer(input_tensor, name,
               kernel_size, n_output_channels,
               padding_mode='SAME', strides=(1, 1, 1, 1)):
    with tf.variable_scope(name):
        ## get n_input_channels:
        ##   input tensor shape:
        ##   [batch x width x height x channels_in]

input_shape = input_tensor.get_shape().as_list()
n_input_channels = input_shape[-1]

weights_shape = list(kernel_size) + \
                [n_input_channels, n_output_channels]

weights = tf.get_variable(name='_weights',
                          shape=weights_shape)
print(weights)
biases = tf.get_variable(name='_biases',
                         initializer=tf.zeros(
                             shape=[n_output_channels]))
print(biases)
conv = tf.nn.conv2d(input=input_tensor,
                    filter=weights,
                    strides=strides,
                    padding=padding_mode)
print(conv)
conv = tf.nn.bias_add(conv, biases,
                      name='net_pre-activation')
print(conv)
conv = tf.nn.relu(conv, name='activation')
print(conv)

return conv
```

这个封装的函数将完成构建卷积层的必要工作，包括定义权重和偏差并初始化，以及包括tf.nn.conv2d函数在内的卷积操作。共需要四个参数：

·input_tensor：作为输入提供给卷积层的张量

·name：层名，也用作域名

·kernel_size：以列表形式提供的核张量维度

·n_output_channels：输出特征分布图的数量

注意，在用tf.get_variable时（第14章曾讨论过Xavier/Glorot的初始化方案），在默认情况下用Xavier（或Glorot）的初始化模型完成权重初始化，用tf.zeros函数把偏差初始化为0。净预激活被传递给ReLU激活函数。我们可

以通过打印TensorFlow的操作和节点观察张量的形状和类型。通过定义占位符作为简单的输入来测试该函数，详情如下：

```
>>> g = tf.Graph()
>>> with g.as_default():
...     x = tf.placeholder(tf.float32, shape=[None, 28, 28, 1])
...     conv_layer(x, name='convtest',
...                kernel_size=(3, 3),
...                n_output_channels=32)
>>>
>>> del g, x

<tf.Variable 'convtest/_weights:0' shape=(3, 3, 1, 32) dtype=float32_
ref>
<tf.Variable 'convtest/_biases:0' shape=(32,) dtype=float32_ref>
Tensor("convtest/Conv2D:0", shape=(?, 28, 28, 32), dtype=float32)
Tensor("convtest/net_pre-activaiton:0", shape=(?, 28, 28, 32),
dtype=float32)
Tensor("convtest/activation:0", shape=(?, 28, 28, 32), dtype=float32)
```

下一个封装函数是为了定义全连接层：

```
def fc_layer(input_tensor, name,
             n_output_units, activation_fn=None):
    with tf.variable_scope(name):
        input_shape = input_tensor.get_shape().as_list()[1:]
        n_input_units = np.prod(input_shape)
        if len(input_shape) > 1:
            input_tensor = tf.reshape(input_tensor,
                                      shape=(-1, n_input_units))

        weights_shape = [n_input_units, n_output_units]
        weights = tf.get_variable(name='_weights',
                                  shape=weights_shape)
        print(weights)
        biases = tf.get_variable(name='_biases',
                                 initializer=tf.zeros(
                                     shape=[n_output_units]))
print(biases)
layer = tf.matmul(input_tensor, weights)
print(layer)
layer = tf.nn.bias_add(layer, biases,
                       name='net_pre-activaiton')
print(layer)
```

```
if activation_fn is None:
    return layer

layer = activation_fn(layer, name='activation')
print(layer)
return layer
```

这个封装函数fc_layer也构建权重和偏差，用类似conv_layer函数的方法初始化，然后用tf.matmul函数进行矩阵乘法。fc_layer函数需要三个参数：

·input_tensor：输入张量

·name：层名，也用来作为域名

·n_output_units：输出单元的个数

可以用简单的输入张量来测试该函数：

```
>>> g = tf.Graph()
>>> with g.as_default():
...     x = tf.placeholder(tf.float32,
...                        shape=[None, 28, 28, 1])
...     fc_layer(x, name='fctest', n_output_units=32,
...              activation_fn=tf.nn.relu)
>>>
>>> del g, x
<tf.Variable 'fctest/_weights:0' shape=(784, 32) dtype=float32_ref>
<tf.Variable 'fctest/_biases:0' shape=(32,) dtype=float32_ref>
Tensor("fctest/MatMul:0", shape=(?, 32), dtype=float32)
Tensor("fctest/net_pre-activaiton:0", shape=(?, 32), dtype=float32)
Tensor("fctest/activation:0", shape=(?, 32), dtype=float32)
```

这个函数对模型中的两个全连接层的表现有点儿不同。第一个全连接层紧接着卷积层获得输入，因此输入仍然是一个4维的张量。第二个全连接层需要调用tf.reshape函数来扁平化输入的张量。另外，来自于FC层的净预激活被传递给了ReLU激活函数，但是第二个与logits相对应，所以必须使用线性激活。

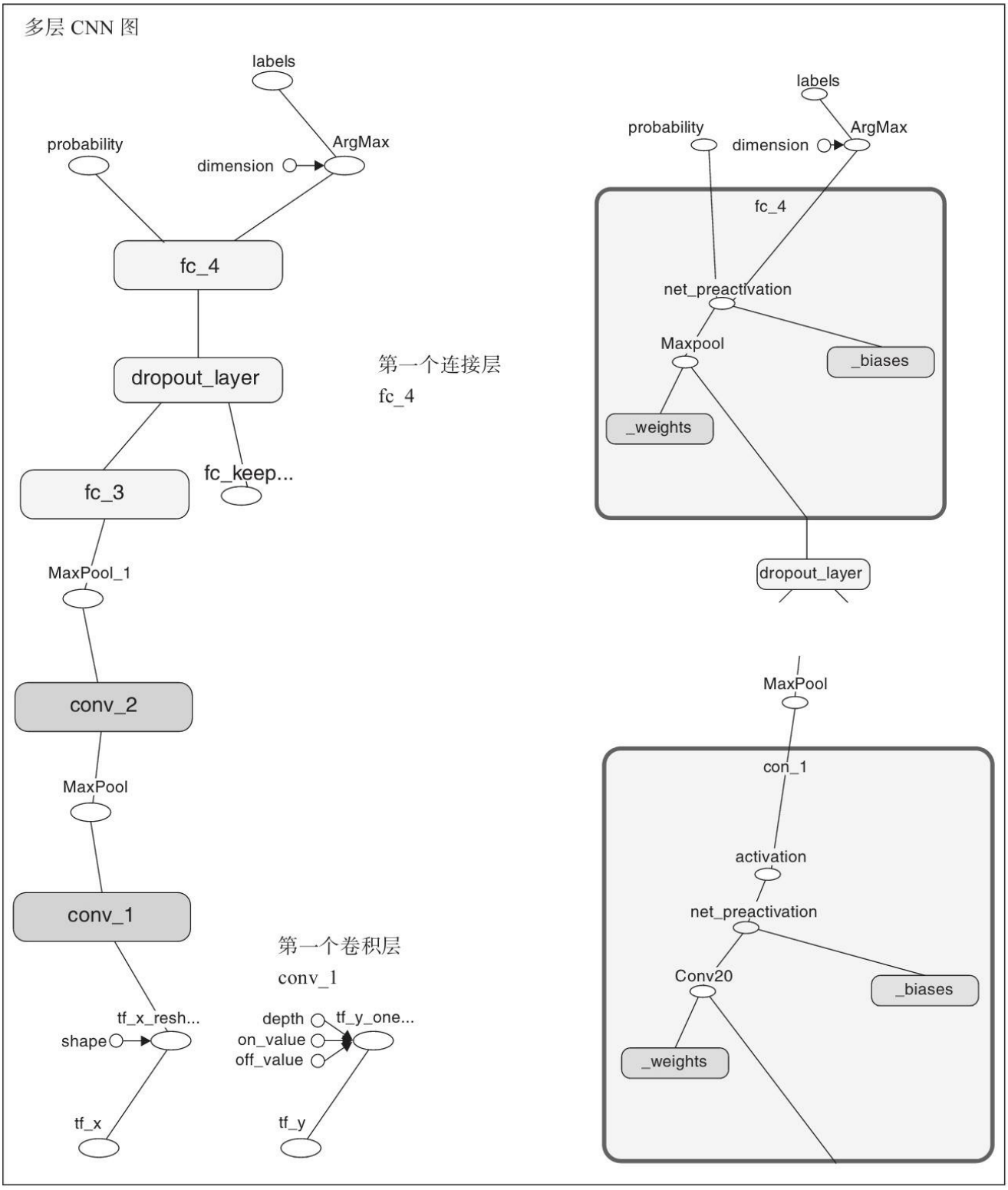
现在可以用这些封装函数来构建完整的卷积网络。定义一个名为build_cnn的函数来构建CNN模型，代码如下所示：

```
def build_cnn():
    ## Placeholders for X and y:
    tf_x = tf.placeholder(tf.float32, shape=[None, 784],
                          name='tf_x')
    tf_y = tf.placeholder(tf.int32, shape=[None],
                          name='tf_y')

    # reshape x to a 4D tensor:
    # [batchsize, width, height, 1]
    tf_x_image = tf.reshape(tf_x, shape=[-1, 28, 28, 1],
                            name='tf_x_reshaped')
    ## One-hot encoding:
    tf_y_onehot = tf.one_hot(indices=tf_y, depth=10,
                             dtype=tf.float32,
                             name='tf_y_onehot')

    ## 1st layer: Conv_1
```

```
    print('\nBuilding 1st layer:')
    h1 = conv_layer(tf_x_image, name='conv_1',
                    kernel_size=(5, 5),
                    padding_mode='VALID',
                    n_output_channels=32)
    ## MaxPooling
    h1_pool = tf.nn.max_pool(h1,
                             ksize=[1, 2, 2, 1],
                             strides=[1, 2, 2, 1],
                             padding='SAME')
    ## 2n layer: Conv_2
    print('\nBuilding 2nd layer:')
    h2 = conv_layer(h1_pool, name='conv_2',
                    kernel_size=(5, 5),
                    padding_mode='VALID',
                    n_output_channels=64)
    ## MaxPooling
    h2_pool = tf.nn.max_pool(h2,
                             ksize=[1, 2, 2, 1],
                             strides=[1, 2, 2, 1],
                             padding='SAME')

    ## 3rd layer: Fully Connected
print('\nBuilding 3rd layer:')
h3 = fc_layer(h2_pool, name='fc_3',
              n_output_units=1024,
              activation_fn=tf.nn.relu)

## Dropout
keep_prob = tf.placeholder(tf.float32, name='fc_keep_prob')
h3_drop = tf.nn.dropout(h3, keep_prob=keep_prob,
                        name='dropout_layer')

## 4th layer: Fully Connected (linear activation)
print('\nBuilding 4th layer:')
h4 = fc_layer(h3_drop, name='fc_4',
              n_output_units=10,
              activation_fn=None)

## Prediction
predictions = {
    'probabilities': tf.nn.softmax(h4, name='probabilities'),
    'labels': tf.cast(tf.argmax(h4, axis=1), tf.int32,
                      name='labels')
}

## Visualize the graph with TensorBoard:

## Loss Function and Optimization
cross_entropy_loss = tf.reduce_mean(
    tf.nn.softmax_cross_entropy_with_logits(
        logits=h4, labels=tf_y_onehot),
    name='cross_entropy_loss')

## Optimizer:
optimizer = tf.train.AdamOptimizer(learning_rate)
```

```
optimizer = optimizer.minimize(cross_entropy_loss,
                               name='train_op')
## Computing the prediction accuracy
correct_predictions = tf.equal(
    predictions['labels'],
    tf_y, name='correct_preds')

accuracy = tf.reduce_mean(
    tf.cast(correct_predictions, tf.float32),
    name='accuracy')
```

为了得到稳定的结果，需要用到NumPy和TensorFlow的随机种子。稍后会看到，可以在图级别中图的范围内调用tf.set_random_seed函数设置TensorFlow的随机种子。下图利用TensorBoard完成了与多层CNN相关联的TensorFlow可视化：




请注意，这个实现了用tf.train.AdamOptimizer函数来训练CNN模型。Adam优化器是一个是强大的基于梯度的优化方法，适合非顶点优化和机器学习。有RMSProp和AdaGrad两个受Adam影响的优化算法。

Adam方法的主要好处在于可以根据梯度变化时的平均值推导更新步幅。可以阅读更多关于Adam优化器的手稿：《Adam：一种随机优化方法》，狄得里克·P.金玛，吉米·雷·巴·2014。该手稿可以从该网站免费获

得：https://arxiv.org/abs/1412.6980。

下面将更进一步定义其他四个函数：save和load用来存储和加载训练模型的检查点，train用training_set来训练模型，predict用来获取测试数据的预测概率或者预测标签。实现这些函数的代码如下：

```
def save(saver, sess, epoch, path='./model/'):
    if not os.path.isdir(path):
        os.makedirs(path)
    print('Saving model in %s' % path)
    saver.save(sess, os.path.join(path,'cnn-model.ckpt'),
               global_step=epoch)

def load(saver, sess, path, epoch):
    print('Loading model from %s' % path)
    saver.restore(sess, os.path.join(
            path, 'cnn-model.ckpt-%d' % epoch))

def train(sess, training_set, validation_set=None,
          initialize=True, epochs=20, shuffle=True,
          dropout=0.5, random_seed=None):

    X_data = np.array(training_set[0])
    y_data = np.array(training_set[1])
    training_loss = []

    ## initialize variables
    if initialize:
        sess.run(tf.global_variables_initializer())

    np.random.seed(random_seed) # for shuflling in batch_generator
    for epoch in range(1, epochs+1):
        batch_gen = batch_generator(
                        X_data, y_data,
                        shuffle=shuffle)
        avg_loss = 0.0
        for i,(batch_x,batch_y) in enumerate(batch_gen):
            feed = {'tf_x:0': batch_x,
                    'tf_y:0': batch_y,
                    'fc_keep_prob:0': dropout}
            loss, _ = sess.run(
                    ['cross_entropy_loss:0', 'train_op'],
                    feed_dict=feed)
            avg_loss += loss

        training_loss.append(avg_loss / (i+1))
```

```
        print('Epoch %02d Training Avg. Loss: %7.3f' % (
            epoch, avg_loss), end=' ')
        if validation_set is not None:
            feed = {'tf_x:0': validation_set[0],
                    'tf_y:0': validation_set[1],
                    'fc_keep_prob:0': 1.0}
            valid_acc = sess.run('accuracy:0', feed_dict=feed)
            print(' Validation Acc: %7.3f' % valid_acc)
        else:
            print()

def predict(sess, X_test, return_proba=False):
    feed = {'tf_x:0': X_test,
            'fc_keep_prob:0': 1.0}
    if return_proba:
        return sess.run('probabilities:0', feed_dict=feed)
    else:
        return sess.run('labels:0', feed_dict=feed)
```

现在，可以创建TensorFlow的计算图对象了，设置图级别的随机种子，然后在那张计算图上构建CNN模型，步骤如下：

```
>>> ## Define hyperparameters
>>> learning_rate = 1e-4
>>> random_seed = 123
>>>
>>>
>>> ## create a graph
>>> g = tf.Graph()
>>> with g.as_default():
...     tf.set_random_seed(random_seed)

...     ## build the graph
...     build_cnn()
...
...     ## saver:
...     saver = tf.train.Saver()
```

请留意，前面的代码在调用build_cnn函数构建模型之后，又基于tf.train.Saver类创建了saver对象，以存储和恢复训练过的模型，我们曾在第14章中见过。

下一步开始训练CNN模型。为此需要创建TensorFlow会话来启动计算图，然后调用train函数。第一次训练模型时，要初始化网络中的所有变量。

为此定义了一个名为initialize的参数来负责初始化过程。如果设置

initialize=True，将在整个session.run过程中执行tf.global_variables_initializer。你应该避免该初始化步骤，除非想做额外的迭代训练。例如你可以恢复已经训练过的模型，再对其进行10次迭代的训练。用来做第一次模型训练的代码如下所示：

```
>>> ## create a TF session
>>> ## and train the CNN model
>>>
>>> with tf.Session(graph=g) as sess:
...     train(sess,
...           training_set=(X_train_centered, y_train),
...           validation_set=(X_valid_centered, y_valid),
...           initialize=True,
...           random_seed=123)
...     save(saver, sess, epoch=20)

Epoch 01 Training Avg. Loss: 272.772  Validation Acc:   0.973
Epoch 02 Training Avg. Loss:  76.053  Validation Acc:   0.981
Epoch 03 Training Avg. Loss:  51.309  Validation Acc:   0.984
Epoch 04 Training Avg. Loss:  39.740  Validation Acc:   0.986
Epoch 05 Training Avg. Loss:  31.508  Validation Acc:   0.987
...
Epoch 19 Training Avg. Loss:   5.386  Validation Acc:   0.991
Epoch 20 Training Avg. Loss:   3.965  Validation Acc:   0.992
Saving model in ./model/
```

在完成20次迭代之后，把训练过的模型存储起来以备未来之用，这样可以避免每次都保存模型，因此也可以节省计算时间。下面的代码展示了如何恢复保存的模型。这里把图定义为g，然后再创建一张新图g2，重新加载训练过的模型以在测试集上进行预测：

```
>>> ### Calculate prediction accuracy
>>> ### on test set
>>> ### restoring the saved model
>>>
>>> del g
>>>
>>> ## create a new graph
>>> ## and build the model
>>> g2 = tf.Graph()
>>> with g2.as_default():
...     tf.set_random_seed(random_seed)
...     ## build the graph
...     build_cnn()
...
...     ## saver:
...     saver = tf.train.Saver()
>>>
>>> ## create a new session
>>> ## and restore the model
>>> with tf.Session(graph=g2) as sess:
...     load(saver, sess,
...          epoch=20, path='./model/')
...
...     preds = predict(sess, X_test_centered,
...                     return_proba=False)
...
...     print('Test Accuracy: %.3f%%' % (100*
               np.sum(preds == y_test)/len(y_test)))

Building 1st layer:
..
Building 2nd layer:
..
Building 3rd layer:
..

Building 4th layer:
..
Test Accuracy: 99.310%
```

build_cnn函数的print语句输出了多余的几行，但为了简捷起见并没有把它们显示在这里。正如你所看到的，对测试数据预测的准确度已经比第13章使用的多层感知器要好。

请确保你用的是X_test_centered，这是测试数据的预处理版本，如果使用了X_test数据集会得到较低的准确率。

现在看一下预测出的标签及它们在前10个测试样本中的概率。代码已经把预测结果存储在preds。然而，为了能有更多使用会话以及启动计算图的实际经验，特别重复这些步骤如下：

```
>>> ## run the prediction on
>>> ##  some test samples
>>> np.set_printoptions(precision=2, suppress=True)
>>>
>>> with tf.Session(graph=g2) as sess:
...     load(saver, sess,
...          epoch=20, path='./model/')
...
...     print(predict(sess, X_test_centered[:10],
...                return_proba=False))
...
...     print(predict(sess, X_test_centered[:10],
...                   return_proba=True))

Loading model from ./model/
INFO:tensorflow:Restoring parameters from ./model/cnn-model.ckpt-20
[7 2 1 0 4 1 4 9 5 9]
[[ 0.    0.    0.    0.    0.    0.    0.    1.    0.    0.  ]
 [ 0.    0.    1.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    1.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 1.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    1.    0.    0.    0.    0.    0.  ]
 [ 0.    1.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    1.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    1.  ]
 [ 0.    0.    0.    0.    0.    0.99  0.01  0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    1.  ]]
```

最后讨论一下如何才能进一步训练模型，以达到40个迭代的水平。自从初始化权重和偏差之后，模型已经训练了20次。我们可以恢复训练过的模型，然后再继续进行另外20次的训练以节省时间。跟着我们的步骤来做可以很容易达到这个目的。这里需要再次调用train函数，但这一次设置initialize=False以免再次初始化。代码如下：

```
## continue training for 20 more epochs
## without re-initializing :: initialize=False
## create a new session
## and restore the model
with tf.Session(graph=g2) as sess:
    load(saver, sess,
         epoch=20, path='./model/')
```

```
train(sess,
      training_set=(X_train_centered, y_train),
      validation_set=(X_valid_centered, y_valid),
      initialize=False,
      epochs=20,
      random_seed=123)

save(saver, sess, epoch=40, path='./model/')

preds = predict(sess, X_test_centered,
                return_proba=False)

print('Test Accuracy: %.3f%%' % (100*
            np.sum(preds == y_test)/len(y_test)))
```

结果显示：20次额外的训练使性能略有提高，测试集的预测结果达到了99.37%。

本节将会看到如何用TensorFlow的低级API来实现多层卷积神经网络。下一节将实现同样的网络，但是使用TensorFlow的Layers API。

### 15.3.4 用TensorFlow的Layers API实现CNN

为了用TensorFlow的Layers API实现CNN，需要调用X_train_centered，X_valid_centered和X_test_centered，重复同样的数据加载和预处理过程。

然后，用一个新类来实现模型：

```
import tensorflow as tf
import numpy as np


class ConvNN(object):
    def __init__(self, batchsize=64,
                 epochs=20, learning_rate=1e-4,
                 dropout_rate=0.5,
                 shuffle=True, random_seed=None):
        np.random.seed(random_seed)
        self.batchsize = batchsize
        self.epochs = epochs
        self.learning_rate = learning_rate
        self.dropout_rate = dropout_rate
        self.shuffle = shuffle

        g = tf.Graph()
        with g.as_default():
            ## set random-seed:
            tf.set_random_seed(random_seed)

            ## build the network:
            self.build()

            ## initializer
            self.init_op = \
                tf.global_variables_initializer()
```

```
            ## saver
            self.saver = tf.train.Saver()

        ## create a session
    self.sess = tf.Session(graph=g)

def build(self):

    ## Placeholders for X and y:
    tf_x = tf.placeholder(tf.float32,
                          shape=[None, 784],
                          name='tf_x')
    tf_y = tf.placeholder(tf.int32,
                          shape=[None],
                          name='tf_y')
    is_train = tf.placeholder(tf.bool,
                          shape=(),
                          name='is_train')

    ## reshape x to a 4D tensor:
    ##  [batchsize, width, height, 1]
    tf_x_image = tf.reshape(tf_x, shape=[-1, 28, 28, 1],
                          name='input_x_2dimages')
    ## One-hot encoding:
    tf_y_onehot = tf.one_hot(indices=tf_y, depth=10,
                             dtype=tf.float32,
                             name='input_y_onehot')

    ## 1st layer: Conv_1
    h1 = tf.layers.conv2d(tf_x_image,
                          kernel_size=(5, 5),
                          filters=32,
                          activation=tf.nn.relu)
    ## MaxPooling
    h1_pool = tf.layers.max_pooling2d(h1,
                          pool_size=(2, 2),
                          strides=(2, 2))
    ## 2n layer: Conv_2
    h2 = tf.layers.conv2d(h1_pool, kernel_size=(5, 5),
                          filters=64,
                          activation=tf.nn.relu)
    ## MaxPooling
    h2_pool = tf.layers.max_pooling2d(h2,
                          pool_size=(2, 2),
                          strides=(2, 2))

    ## 3rd layer: Fully Connected
input_shape = h2_pool.get_shape().as_list()
n_input_units = np.prod(input_shape[1:])
h2_pool_flat = tf.reshape(h2_pool,
                      shape=[-1, n_input_units])
h3 = tf.layers.dense(h2_pool_flat, 1024,
                     activation=tf.nn.relu)

## Dropout
```

```
h3_drop = tf.layers.dropout(h3,
                      rate=self.dropout_rate,
                      training=is_train)

## 4th layer: Fully Connected (linear activation)
h4 = tf.layers.dense(h3_drop, 10,
                     activation=None)

## Prediction
predictions = {
    'probabilities': tf.nn.softmax(h4,
                      name='probabilities'),
    'labels': tf.cast(tf.argmax(h4, axis=1),
                      tf.int32, name='labels')
}

## Loss Function and Optimization
cross_entropy_loss = tf.reduce_mean(
    tf.nn.softmax_cross_entropy_with_logits(
        logits=h4, labels=tf_y_onehot),
    name='cross_entropy_loss')

## Optimizer:
optimizer = tf.train.AdamOptimizer(self.learning_rate)
optimizer = optimizer.minimize(cross_entropy_loss,
                      name='train_op')

## Finding accuracy
correct_predictions = tf.equal(
    predictions['labels'],
    tf_y, name='correct_preds')

accuracy = tf.reduce_mean(
        tf.cast(correct_predictions, tf.float32),
        name='accuracy')

def save(self, epoch, path='./tflayers-model/'):
    if not os.path.isdir(path):
        os.makedirs(path)
    print('Saving model in %s' % path)
    self.saver.save(self.sess,
                    os.path.join(path, 'model.ckpt'),
                    global_step=epoch)

def load(self, epoch, path):
    print('Loading model from %s' % path)
    self.saver.restore(self.sess,
         os.path.join(path, 'model.ckpt-%d' % epoch))

def train(self, training_set,
          validation_set=None,
          initialize=True):
    ## initialize variables
    if initialize:
        self.sess.run(self.init_op)
```

```
        self.train_cost_ = []
        X_data = np.array(training_set[0])
        y_data = np.array(training_set[1])

        for epoch in range(1, self.epochs+1):
            batch_gen = \
                batch_generator(X_data, y_data,
                                shuffle=self.shuffle)
            avg_loss = 0.0
            for i, (batch_x,batch_y) in \
                enumerate(batch_gen):
                feed = {'tf_x:0': batch_x,
                        'tf_y:0': batch_y,
                        'is_train:0': True} ## for dropout
                loss, _ = self.sess.run(
                        ['cross_entropy_loss:0', 'train_op'],
                        feed_dict=feed)
                avg_loss += loss

            print('Epoch %02d: Training Avg. Loss: '
                  '%7.3f' % (epoch, avg_loss), end=' ')

            if validation_set is not None:
                feed = {'tf_x:0': batch_x,
                        'tf_y:0': batch_y,
                        'is_train:0' : False} ## for dropout
                valid_acc = self.sess.run('accuracy:0',
                        feed_dict=feed)
                print('Validation Acc: %7.3f' % valid_acc)
            else:
                print()

    def predict(self, X_test, return_proba=False):
        feed = {'tf_x:0' : X_test,
                'is_train:0' : False} ## for dropout
        if return_proba:
            return self.sess.run('probabilities:0',
                                 feed_dict=feed)
        else:
            return self.sess.run('labels:0',
                                 feed_dict=feed)
```

该类的结构与前一节TensorFlow的低级API非常相似。这个类有一个创建器用来设置训练参数，创建计算图g，然后构建模型。除了创建器外，还有五个主要的方法：

·.build：构建模型

·.save：存储训练过的模型

·.load：恢复存储的模型

·.train：训练模型

·.predict：在测试集上进行预测

与上一节的实现相似，在第一个全连接层之后采用淘汰层。之前使用了TensorFlow的低级API的实现中用到了tf.nn.dropout函数，但是在这里却用了tf.layers.dropout，它是tf.nn.dropout函数的封装。这两个函数之间存在着较大的差别，应当小心：

·tf.nn.dropout：有个名为keep_prob的参数，用来表示保存单元的概率，而tf.layers.dropout有一个rate参数表示淘汰该单元的概率，因此rate=1-keep_prob。

·tf.nn.dropout函数用占位符为参数keep_prob提供数据，我们将设置训练过程的参数keep_prob=0.5，设置推理或预测过程的参数keep_prob=1。然而，tf.layers.dropout的rate的值取决于计算图中创建的淘汰层，而且在训练和推理时无法改变。相反，只需要提供一个布尔型的参数来确定是否要淘汰。这可以用一个tf.bool类型的占位符来实现，在训练状态下，提供的值为True，而在推理或者预测状态下提供的值为False。

可以创建一个ConvNN类，用20次迭代来训练并存储模型。代码如下：

```
>>> cnn = ConvNN(random_seed=123)
>>>
>>> ## train the model
>>> cnn.train(training_set=(X_train_centered, y_train),
...           validation_set=(X_valid_centered, y_valid),
...           initialize=True)
>>> cnn.save(epoch=20)
```

完成训练以后，可以用模型来对测试集进行预测，具体如下：

```
>>> del cnn
>>>
>>> cnn2 = ConvNN(random_seed=123)
>>> cnn2.load(epoch=20, path='./tflayers-model/')
>>>
>>> print(cnn2.predict(X_test_centered[:10, :]))

Loading model from ./tflayers-model/
INFO:tensorflow:Restoring parameters from ./tflayers-model/model.ckpt-
20
[7 2 1 0 4 1 4 9 5 9]
```

最后，可以像下面这样来度量测试集的准确度：

```
>>> preds = cnn2.predict(X_test_centered)
>>>
>>> print('Test Accuracy: %.2f%%' % (100*
...       np.sum(y_test == preds)/len(y_test)))

Test Accuracy: 99.32%
```

所获得的预测准确度为99.32%，这意味着错误分类的测试样本只有68个！

这样就结束了使用TensorFlow的低级API和Layers API实现卷积神经网络的讨论。我们用低级API为第一个实现定义了一些封装函数。因为可以用tf.layers.conv2d和tf.layers.dense函数建立卷积和全连接层，所以第二个实现更加简单。

## 15.4 小结

本章学习了CNN或卷积神经网络，并探索了构建不同CNN体系结构所需要的模块。从定义卷积运算开始，然后通过讨论一维和二维的实现了解了它的基本原理。

通过讨论最大池和平均池两种形式的池操作，我们也介绍了子采样。然后，将所有这些模块放在一起构建了深度卷积神经网络，用TensorFlow的核心API和Layers API进行了实现，并将其应用于图像识别。

下一章将转向递归神经网络（RNN）。RNN用于学习序列数据的结构，并且有一些包括语言翻译和图像字幕在内的有趣应用！

# 第16章　用递归神经网络为序列数据建模

前一章专注于用卷积神经网络（CNN）进行图像识别。本章将探讨递归神经网络（RNN），并看看它们在序列数据和序列数据的特定子集建模方面的情况，即RNN在时间序列数据中的应用。本章将主要涵盖下述几个方面：

·介绍序列数据

·用RNN构建序列模型

·长短时记忆（LSTM）

·时间截断反向传播（T-BPTT）

·用TensorFlow实现多层RNN序列建模

·项目1：IMDb电影评论数据集的RNN情感分析

·项目2：基于莎士比亚的作品《哈姆雷特》，进行带有LSTM单元的RNN字符级语言建模

·使用梯度裁剪来避免梯度爆炸

本章是Python机器学习之旅的最后一站，我们将以对RNN的学习的总结，和对引领我们贯穿全书的机器学习和深度学习的所有主题的回顾来结束本章。然后将通过为你分享一些我们最喜欢的人和在这个奇妙领域的事，引导你沿着机器学习和深度学习的方向继续前进。

## 16.1 序列数据

让我们从观察顺序数据的性质开始讨论RNN，顺序数据更常见的叫法是序列数据。本章将研究那些能使序列数据与其他类型的数据区别开来的特性。然后研究如何表示序列数据，并基于输入和输出探索针对序列数据的各类模型。这将有助于本章稍后探讨RNN和序列数据之间的关系。

### 16.1.1 序列数据建模——顺序很重要

从其他数据类型的角度看，使序列独树一帜的特性是其元素以一定的顺序出现，并且彼此不独立。

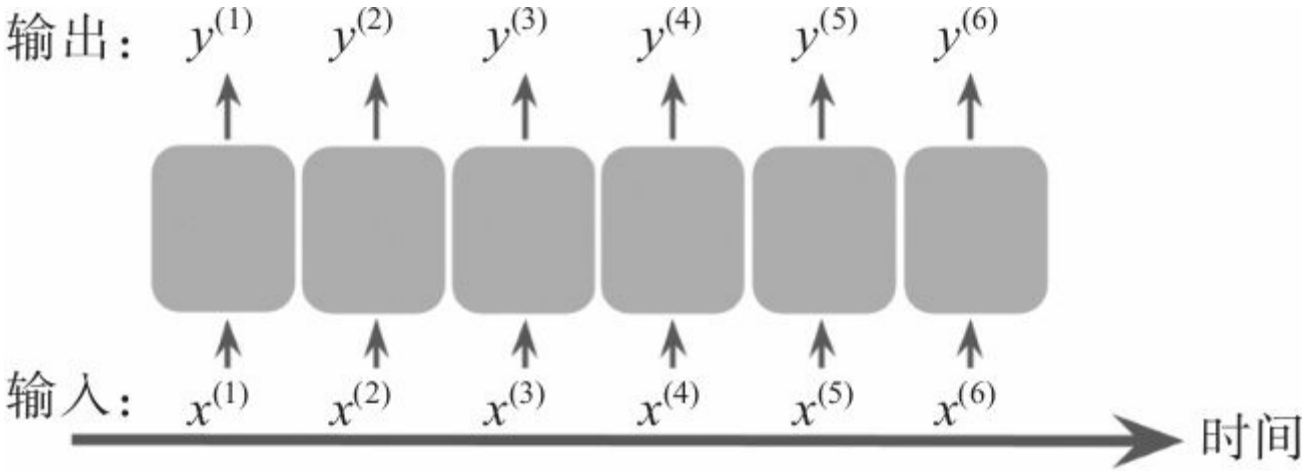
还记得第6章曾讨论过，典型的有监督学习的机器学习算法中，我们假设输入数据是独立和均匀分布（IID）的。例如，如果有n个数据样本 $x^{(1)}$，$x^{(2)}$，...，$x^{(n)}$，那么用来训练机器学习算法的数据顺序并不重要。

然而当根据定义处理顺序很重要的序列问题时，这种假设就不再有效。

## 16.1.2 表示序列

我们已经建立了序列，其中输入数据是非独立的顺序。接下来要找到方法利用机器学习模型在其中提取有价值信息。

贯穿本章，我们将一直用（$x^{(1)}$，$x^{(2)}$，...，$x^{(T)}$）来表示序列实例，序列长度为T。举一个更有意义的案例，考虑时间序列数据，每个样本$x^{(t)}$属于某个特定时间点t。下图展示了时间序列数据，x和y都很自然地按照时间轴的顺序展开；因此x和y都是序列的。



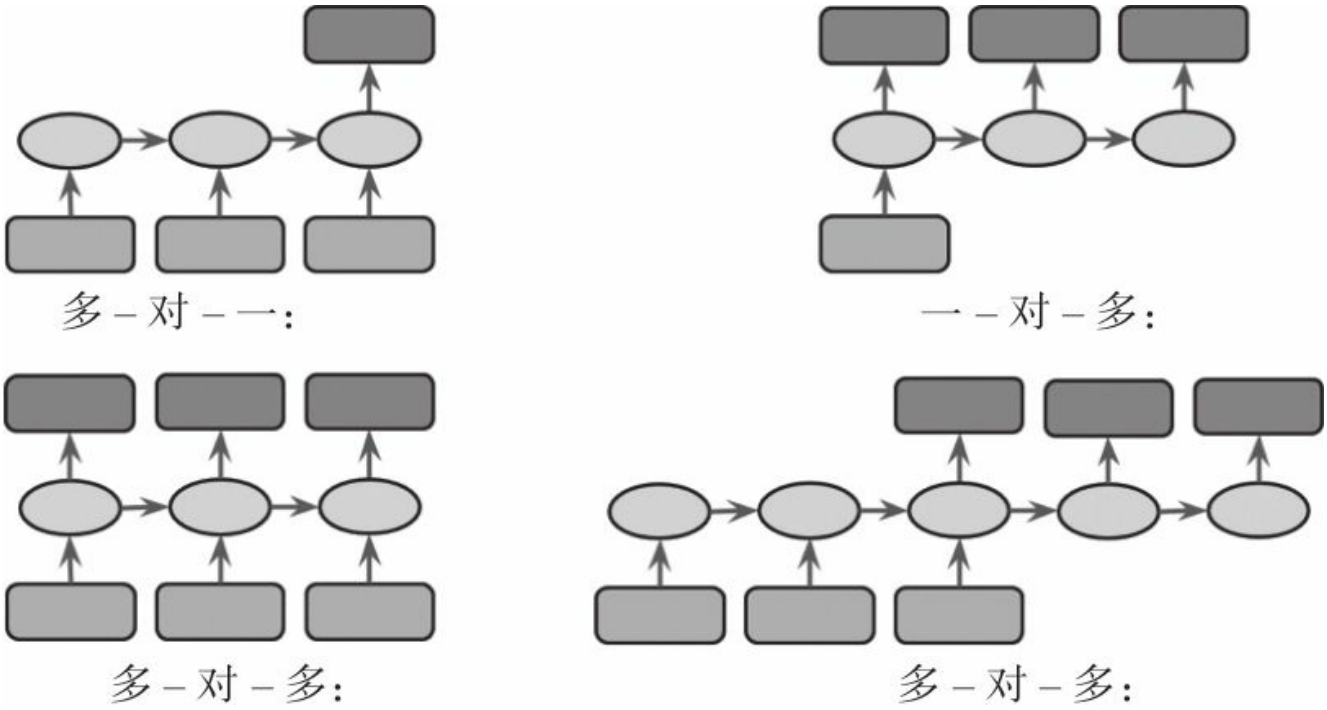

到目前为止，已经讨论过的MLP和CNN这样的标准神经网络模型无法处理输入样本的顺序。形象地说，这类模型对曾见过的样本没有记忆。例如，样本通过前馈和后馈步骤传播，而且更新权重与样本处理顺序无关。

相比之下，设计RNN的目的是为序列数据建模。RNN能够记住过去看到的信息，而且有能力根据情况处理新事件。

## 16.1.3 不同类别的序列建模

序列建模有许多像语言翻译、图像字幕和文本生成等的有趣应用。

然而，需要了解不同类型的序列建模任务以研发合适的模型。下图是对一篇由安德烈·卡帕斯发表的优秀文章《递归神经网络的不合理有效性》的解释（http://karpathy.github.io/2015/05/21/rnn-effectiveness/），其中显示了输入和输出数据的几种不同的关系类别：




因此，我们来考虑输入和输出数据。如果输入或输出数据都不代表序列，那么所处理的是标准数据，就可以采用先前讨论过的任何方法建模处理。但如果输入或输出的数据都不是序列的，那数据将会属于以下三种不同的类别：

·多对一：输入数据是一个序列，但输出数据不是序列而是固定的向量。例如，情感分析的输入基于文本，而输出是分类标签。

·一对多：输入数据是标准格式，不是序列，而输出数据是序列。一个例子是图像字幕，输入是图像，输出是英语短语。

·多对多：输入和输出阵列都是序列。可以根据输入和输出是否同步来进一步划分该类别。同步多对多建模任务的例子是视频分类，标记视频的每帧。延迟多对多的例子是把一种语言翻译成另一种语言。例如，一个完整的英语句子必须在机器翻译成德语之前先被机器阅读和处理。

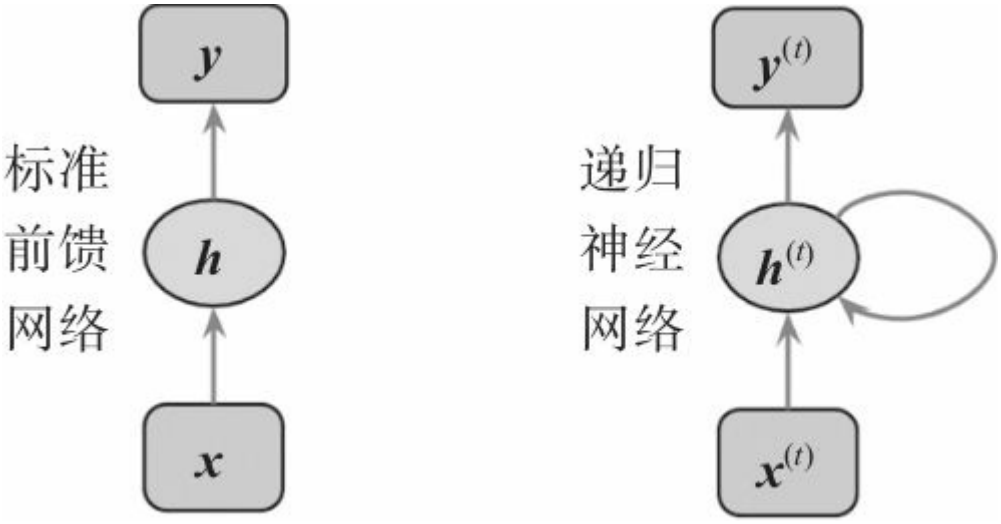
既然现在已经掌握了序列建模的类别，那么可以继续深入讨论RNN的结构。

## 16.2 用于序列建模的RNN

现在已经理解了序列，可以在这一节讨论RNN的基础。我们将从介绍RNN的典型结构开始，看数据是如何流过一个或多个隐藏层。然后，将研究在典型的RNN中如何计算神经元的激活。这将为应对训练RNNs模型所遇到的共同挑战做好准备，探索针对这些挑战的现代解决方案——LSTM。

## 16.2.1 理解RNN的结构和数据流

从介绍RNN的体系结构开始。下图显示了标准的前馈神经网络和RNN，并对两者逐一进行了比较：



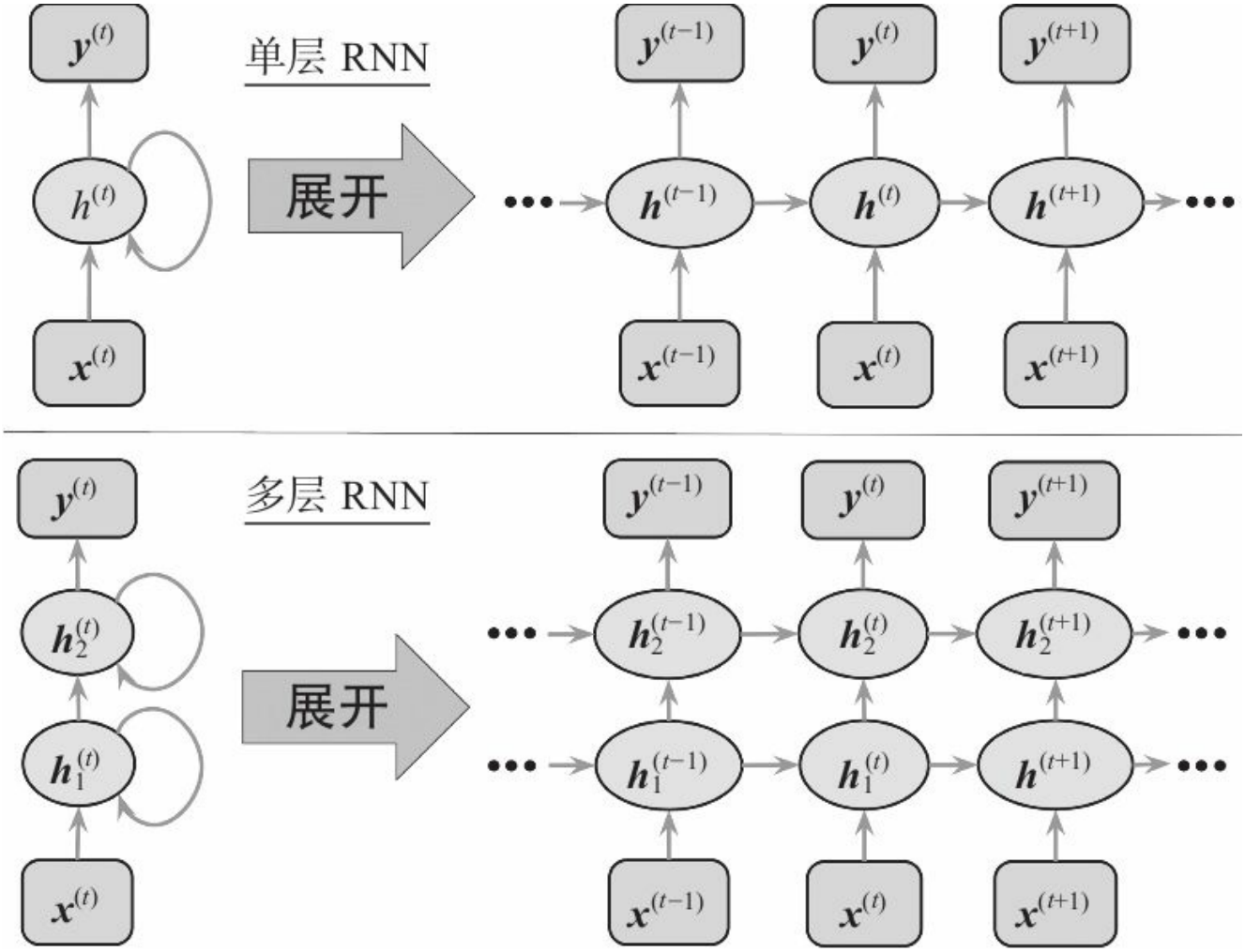

这两种网络都只有一个隐藏层。该图没有显示出单元，但是假设输入层（x）、隐藏层（h）和输出层（y）是包含多个单元的向量。

通用的RNN体系结构可以对应于两个序列建模类别，其中输入为序列。因此，如果考虑$y^{(t)}$作为最终的输出，它可以是多对多，如果只用$y^{(t)}$的最后一个元素作为最终的输出，那么它可以是多对一。后面会看到输出序列$y^{(t)}$是如何转换成标准以及非序列输出的。

在标准的前馈网络中，信息从输入流到隐藏层，然后从隐藏层流向输出层。另外，递归网络的隐藏层从输入层和先前迭代的隐藏层获得其输入。

隐藏层中相邻迭代的信息流允许网络对事件保持记忆。这种信息流通常被显示为循环，在图记法中也被称为递归边缘，这就是这个通用体系结构的命名逻辑。

下图通过对比单隐藏层网络和多隐藏层网络说明了两种不同的体系结构：




由上图可见为检查RNN的体系结构和信息流，可以展开具有递归边缘的紧凑表示。

正如所知道的那样，标准神经网络的每个隐藏单元只接收一个输入，即与输入层相关联的网络预激活。相反，RNN的每个隐藏单元接收来自输入层的预激活和时间步t=1之前的相同激活隐藏层的两组不同的输入。

第一个时间步t=0，隐藏的单元被初始化为零或小的随机值。然后在t>0时，隐藏单元从当前时间x（t）的数据点获得它们的输入，且隐藏单元之前在t-1处的值为h（t-1）。

与此类似，可以将多层RNN情况下的信息流概述如下：

·layer=1：这里，隐藏层表示为$\boldsymbol{h}_1^{(t)}$，从数据点x（t）获得输入的数据，以及本层所隐藏的上一个时间步的数值$\boldsymbol{h}_1^{(t-1)}$。

·layer=2：第二隐藏层从当前层下面的隐藏单元$\boldsymbol{h}_2^{(t)}$得到输入，也从自己在上一个时间步得到的$\boldsymbol{h}_2^{(t-1)}$。

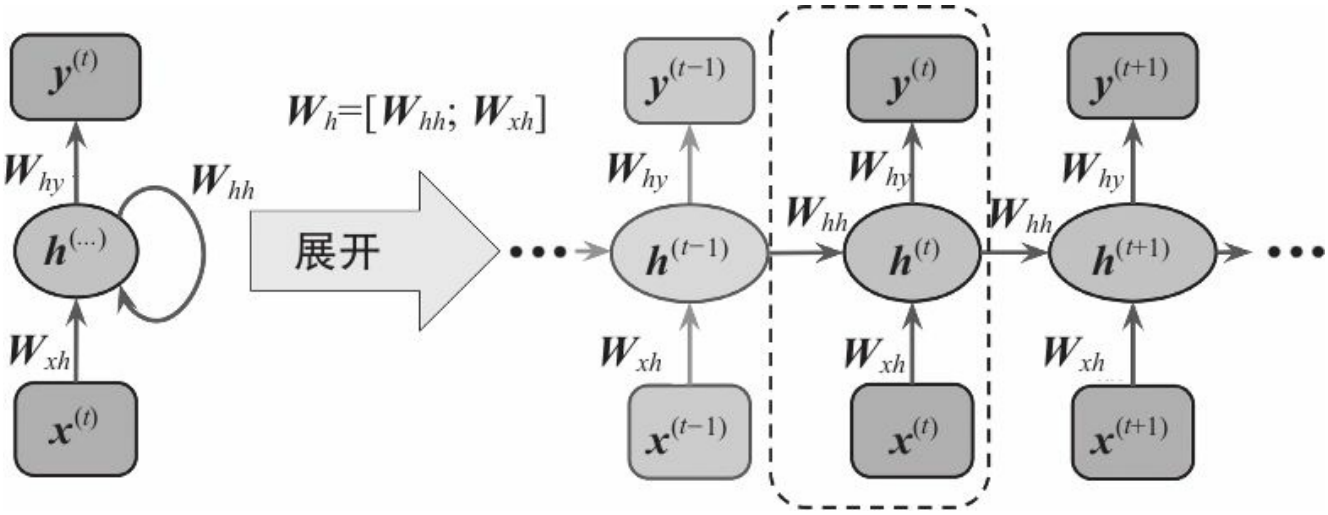
## 16.2.2 在RNN中计算激活值

了解了RNN的结构和一般信息流的情况，让我们更具体地计算隐藏层和输出层的实际激活值。为了简单起见，只考虑单个隐藏层。然而，同样的概念适用于多层RNN。

在刚看到的RNN表示中，每个有向边（两个方框之间的连接）与权重矩阵相关联。那些权重不依赖于时间t，因此可以在时间轴上共享。单层RNN中的不同权重矩阵如下：

- $W_{xh}$：输入层$x^{(t)}$和隐藏层h之间的权重矩阵
- $W_{hh}$：与递归边相关联的权重矩阵
- $W_{hy}$：隐藏层与输出层之间的权重矩阵

你可以在下面的图中看到这些权重矩阵：




在某些实现中，你可能会观察到权重矩阵$W_{xh}$和$W_{hh}$相互连接成为$W_h=[W_{xh}; W_{hh}]$。

以后我们也会采用这种表示方法。计算激活与标准多层感知器和其他类型的前馈神经网络非常相似。对于隐藏层，通过线性组合计算净输入zh（预激活）。也就是说计算权重矩阵与相应向量乘法之和，然后加上偏差单元，即：

$$z_h^{(t)} = W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h$$

然后，计算隐藏单元在时间步t的激活如下：

$$\boldsymbol{h}^{(t)} = \phi_h(z_h^{(t)}) = \phi_h(\boldsymbol{W}_{xh}\boldsymbol{x}^{(t)} + \boldsymbol{W}_{hh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_h)$$

这里，$b_h$为隐藏单元的偏差变量而$\varphi_h$（·）是隐藏层的激活函数。

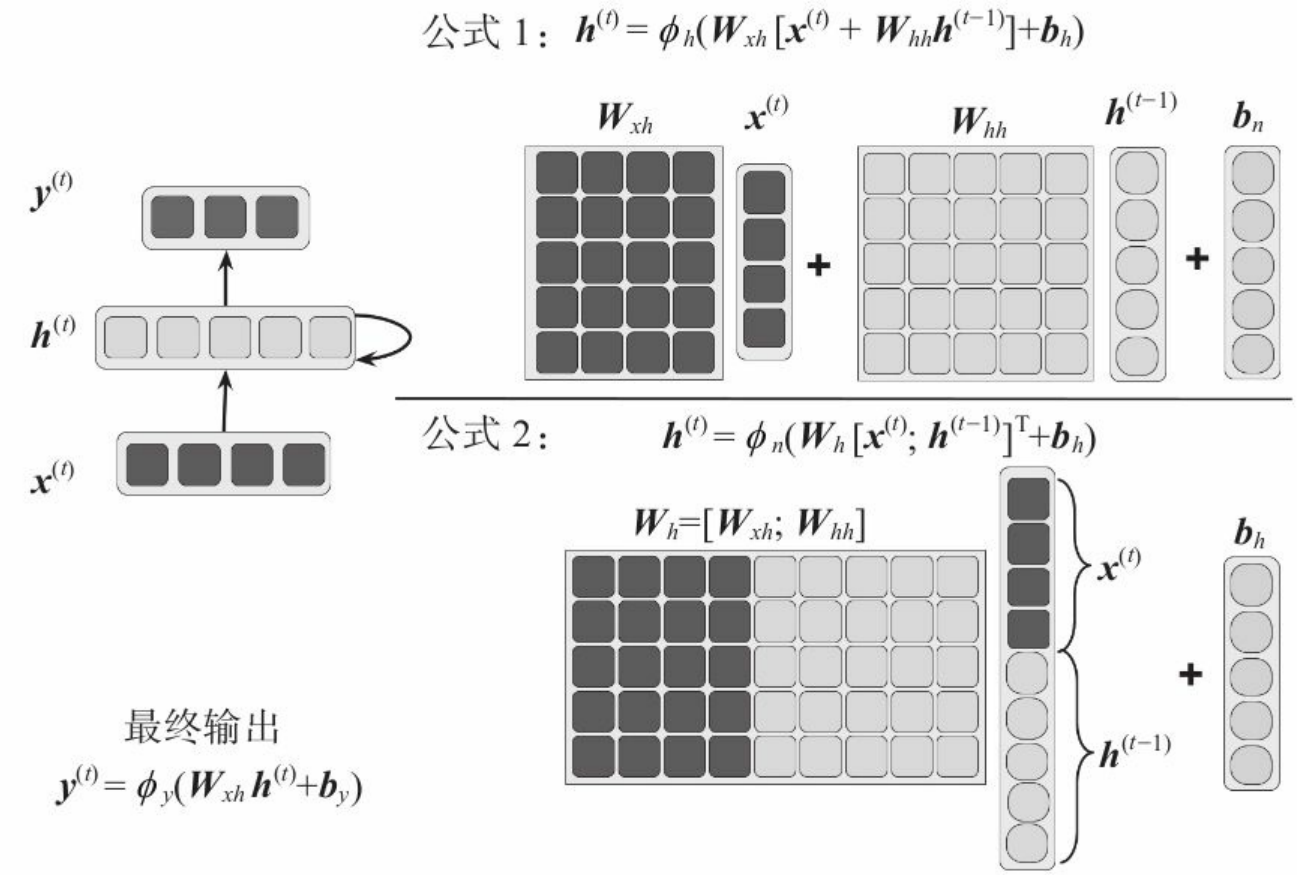
如果想要用联结后的权重矩阵$W_h=[W_{xh}; W_{hh}]$，那么计算隐藏单元的公式就变成：

$$\boldsymbol{h}^{(t)} = \phi_h\left([\boldsymbol{W}_{xh}; \boldsymbol{W}_{hh}]\begin{bmatrix}\boldsymbol{x}^{(t)}\\ \boldsymbol{h}^{(t-1)}\end{bmatrix} + \boldsymbol{b}_h\right)$$

一旦计算出隐藏单元当前时间步的激活，则计算输出单元的激活如下：

$$\boldsymbol{y}^{(t)} = \phi_y(\boldsymbol{W}_{hy}\boldsymbol{h}^{(t)} + \boldsymbol{b}_y)$$

为了进一步阐明这一点，下图显示了用两种公式计算激活的过程：



用BPTT训练RNN

20世纪90年代RNN引入了时间反向传播的机器学习算法（保罗·韦伯，1990年IEEE会议录：《它是做什么的以及如何做？》78（10）：1550-1560）。

梯度推导可能有点复杂，但基本思想是总损失L等于从t=1到t=T之间所有损失函数的总和。

$$L = \sum_{t=1}^{T} L^{(t)}$$

因为时间1：t之间的损失取决于隐藏单元所有之前的时间步长1：t，梯度计算如下：

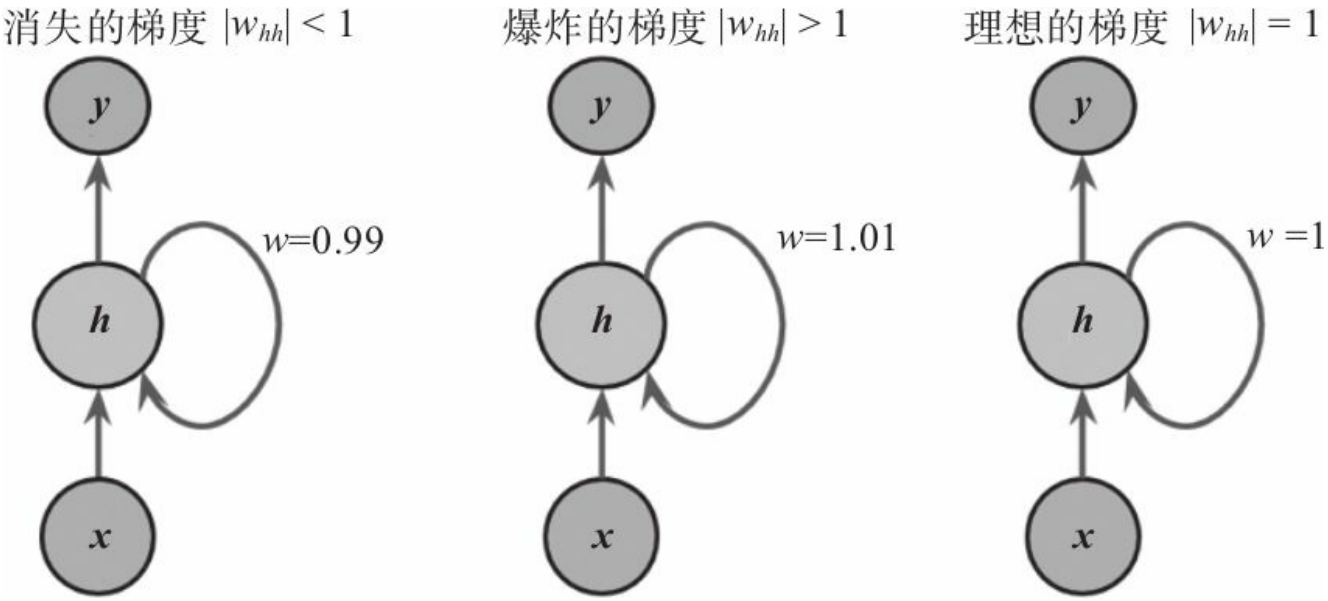
$$\frac{\partial L^{(t)}}{\partial \boldsymbol{W}_{hh}} = \frac{\partial L^{(t)}}{\partial \boldsymbol{y}^{(t)}} \times \frac{\partial \boldsymbol{y}^{(t)}}{\partial \boldsymbol{h}^{(t)}} \times \left( \sum_{k=1}^{t} \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}} \times \frac{\partial \boldsymbol{h}^{(k)}}{\partial \boldsymbol{W}_{hh}} \right)$$

这里，$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}}$为相邻时间步之积：

$$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}} = \prod_{i=k+1}^{t} \frac{\partial \boldsymbol{h}^{(i)}}{\partial \boldsymbol{h}^{(i-1)}}$$

## 16.2.3 长期交互学习的挑战

在前面的信息栏简单地提到时间反向传播或者BPTT，它们带来了一些新的挑战。在计算损失函数的梯度时，由于乘法因子$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}}$的存在，会出现所谓的梯度消失或爆炸的问题。这可以通过下图的例子来解释，为简单起见，该图只显示了拥有单个隐藏单元的RNN：




$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}}$基本上要做t-k次乘法运算。因此，t-k次乘以权重w得到因子$w^{t-k}$。结果，如果|w|＞1，该因子在t-k很大的情况下就变得非常小。另一方面，如果递归边缘的权重|w|＞1，那么在t-k很大的情况下$w^{t-k}$就变得非常大。注意t-k很大的意思是长期依赖。

可以凭直觉通过简单地确保|w|=1避免梯度消失或爆炸。如果你对此感兴趣并想更详细地研究，我鼓励你阅读R.帕斯卡努、T.米科洛夫和Y.本杰明在2012年撰写的论文《论递归神经网络进行的训练难度》（http://ARXIV.org/pdf/1211.5063.pdf）。

实际上，对这个问题有两种解决办法：

1.截断的时间反向传播法（TBPTT）

2.长期记忆法（LSTM）

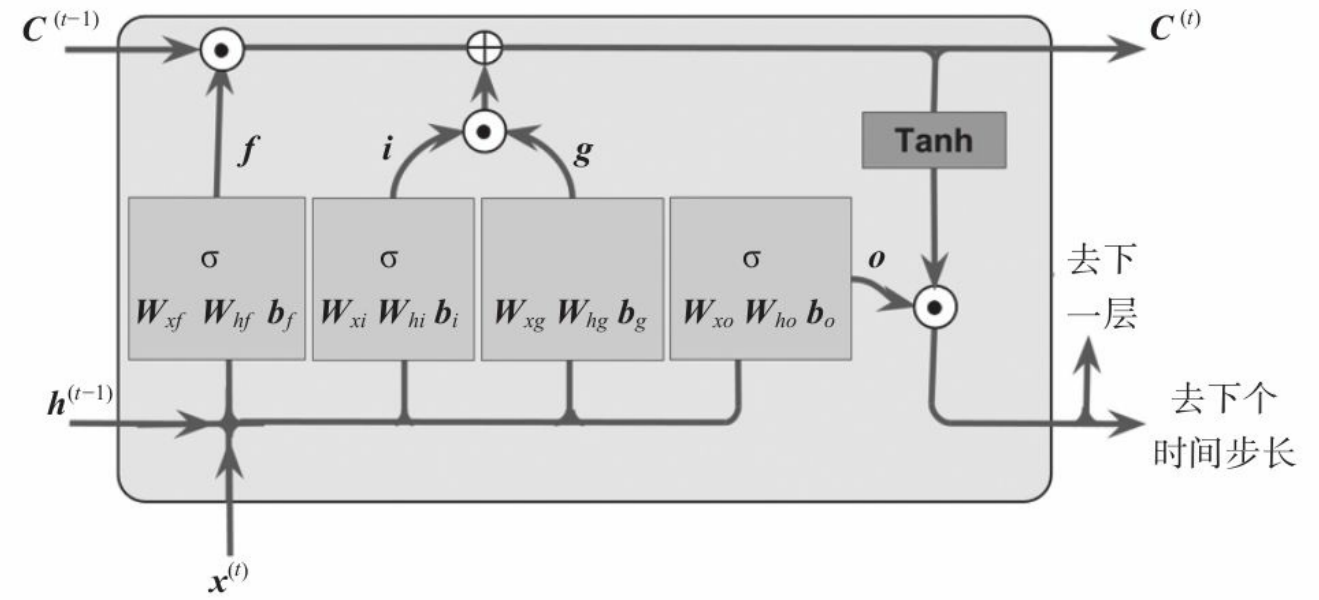
TBPTT把梯度限制在某个给定范围。尽管TBPTT能解决爆炸梯度问题，但截断限制了梯度可以有效回流和适当更新权重的步骤数。

另一方面，由霍克赖特和史密斯胡伯在1997年联合设计的LSTM，通过解决梯度消失问题已经在长期序列的建模中表现得更为成功。让我们更详细地讨论LSTM。

## 16.2.4 LSTM单元

LSTM最先被引入来解决梯度消失问题（S.霍克赖特，J.史密斯胡伯.《长短期记忆》.神经计算，1997，9（8）：1735-1780）。LSTM的构建模块是一个存储单元，它基本上代表了隐藏层。

每个存储单元都有一个理想的权重w=1的递归边缘，如前所述，目的是要解决梯度消失和爆炸的问题。与此递归边缘相关联的值称为信元状态。下图展示了现代LSTM单元的展开结构：




请注意前一个时间步的信元状态为$C^{(t-1)}$，对其进行修改以获得现在时间步的信元状态$C^{(t)}$，这不需要直接与任何权重因子相乘。

存储单元中的信息流是由将在这里描述的计算单元所控制。在前面的图中，⊙指元素级的乘积（元素乘），⊕意味着元素求和。此外，$x^{(t)}$表示t时的输入数据，$h^{(t-1)}$表示t-1时的隐藏单元。

四个框代表着激活函数，要么是S形函数（σ），要么是或双曲正切函数（tanh）和一组权重；这些框通过输入数据计算矩阵向量乘法来应用线性组合。这些具有S形激活函数的计算单元，其输出单元通过⊙，也被称为门。

LSTM单元有三种不同类型的门，分别为遗忘门、输入门和输出门：

·遗忘门（$f_t$）允许记忆单元在有限增长的情况下重置信元状态。事实上，遗忘门决定允许哪些信息通过，哪些信息被抑制。$f_t$计算如下：

$$f_t = \sigma(W_{xf}x^{(t)} + W_{hf}h^{(t-1)} + b_f)$$

注意，遗忘门并不是初始LSTM单元的组成部分，而是在原始模型出现几年后才被添加进来优化原始模型的（F.格尔斯，J.施米德胡贝，F.康明斯.《学习遗忘：用LSTM做连续预测》.神经计算，第12期，2000：2451-2471）。

·输入门（$f_t$）和输入节点（$g_t$）负责更新信元状态。计算如下：

$$i_t = \sigma(W_{xi}x^{(t)} + W_{hi}h^{(t-1)} + b_i)$$

$$g_t = tanh(W_{xg}x^{(t)} + W_{hg}h^{(t-1)} + b_g)$$

t时刻的信元状态计算如下：

$$C^{(t)} = (C^{(t-1)} \odot f_i) \oplus (i_t \odot g_t)$$

·输出门（$O_t$）决定如何更新隐藏单元的值：

$$o_t = \sigma(W_{xo}x^{(t)} + W_{ho}h^{(t-1)} + b_o)$$

鉴于此，计算当前时间步中的隐藏单元如下：

$$h^{(t)} = o_t \odot tanh(C^{(t)})$$

LSTM单元的结构及其底层计算初看起来可能很复杂。然而，好消息是TensorFlow已经在封装的函数中实现了LSTM所有的功能，使我们能轻松地定义LSTM单元。本章后面使用TensorFlow时将会看到LSTM的实际应用。

本节介绍了LSTM，它提供了一种长期依赖序列建模的基本方法。然而，重要的是要注意LSTM有许多在文献中描述的变种（拉法尔·乔泽夫维茨，沃吉切尔·扎雷姆巴，伊利亚·苏特斯威夫.《递归网络体系结构的经验探索》.ICML会议录，2015：2442-223）。

此外，值得注意的是2014年提出的一种更先进的方法，被称为门控递归单元（GRU）。GRU的体系结构比LSTM更简单，因而它们在完成像多音变化音乐建模这样的任务时，计算效率更高，可与LSTM相媲美。如果你有兴趣了解更多关于这些现代的RNN体系结构，请参阅由仲俊等在2014年发表的论文《门控递归神经网络序列建模经验评估》

（http://ARXIV.org/pdf/14123555v1.pdf）。

## 16.3 用TensorFlow实现多层RNN序列建模

介绍了RNN背后的基本理论后，我们已经准备好讨论更实用的部分，即用TensorFlow实现RNN。本章的其余部分将把RNN应用到两个常见的问题：

1.情感分析

2.语言建模

下面几页把这两个项目放在一起实现，这两个项目既迷人又涉及广泛。因此不会一下子提供所有的代码，而是将其分解成几个步骤来实现并详细讨论具体的代码。如果你喜欢宏观概览并在讨论之前立即研究所有的代码，那建议你先研究代码实现，你可以从下面的网站获得这些代码：

https://github.com/rasbt/python-machine-learning-book-2nd-edition/blob/master/code/ch16/ch16.ipynb

注意，在本章开始讨论编码之前，既然已经用了非常现代的TensorFlow，所以在代码实现时将使用TensorFlow Python API的contrib子模块，TensorFlow（1.3.0）的最新版本是2017年8月发布的。这些contrib函数、类及其在本章引用的文档，可能会在TensorFlow的未来版本中发生改变，甚至可能被集成到tf.nn子模块。因此，建议你关注TensorFlow的API文档（https://www.tensorflow.org/api_docs/python/）。

## 16.4 项目一：利用多层RNN对IMDb电影评论进行情感分析

你可能还记得第8章，情感分析涉及分析句子或文本文档所表达的想法。本节和下面的小节将用多对一的体系结构来实现多层RNN以用于情感分析。

下一节将实现多对多的RNN以便用于语言建模。虽然在特意选择的例子中简单地介绍了RNN的主要概念，但语言建模有着广泛而有趣的应用，如构建聊天机器人让计算机拥有直接与人交谈和互动的能力。

### 16.4.1 准备数据

在第8章的预处理步骤中，曾经创建过名为movie_data.csv的干净数据集，现在再次使用它。为此，首先导入必要的模块，然后将数据读入pandas的DataFrame，如下所示：

```
>>> import pyprind
>>> import pandas as pd
>>> from string import punctuation
>>> import re
>>> import numpy as np
>>>
>>> df = pd.read_csv('movie_data.csv', encoding='utf-8')
```

回想这个df数据帧，它有'review'和'sentiment'两列，其中'review'包含电影评论文本，而'sentiment'包含标签0或1。这些电影评论的文本成分是单词序列，因此，要建立RNN模型来处理每个序列中的单词，最后把整个序列分类为0或1。

为了准备输入到神经网络的数据，需要将其编码为数值。为此，首先要在整个数据集中找到独立的单词，这可以用Python中的集合来完成。然而，我发现用集合在这么大的数据集中寻找独立单词的效率并不高。更有效的方法是用软件包中的Counter函数。如果想了解更多有关Counter的内容，请参阅下述网站的文档：

https://docs.python.org/3/library/collections.html#collections.Counter

下面的代码将根据Counter类定义一个counts对象，该对象收集文本中每个独立单词出现的频率。请注意，在这个特定的应用中（与词袋模型相反），只对一组独立单词感兴趣，并不需要作为副产品创建词频统计。

然后，以字典的形式创建映射，将每个独立单词映射到数据集，得到唯一的整数。我们把该字典称为word_to_int，可以用它来将一篇评论的全文转换成数字列表。基于词频对独立词的排序，可以用任意顺序而不会影响其最终结果。执行以下的代码将文本转换成整数列表：

```
>>> ## Preprocessing the data:
>>> ## Separate words and
>>> ## count each word's occurrence
>>>
>>> from collections import Counter

>>> counts = Counter()
>>> pbar = pyprind.ProgBar(len(df['review']), \
...                        title='Counting words occurrences')
>>> for i,review in enumerate(df['review']):
...     text = ''.join([c if c not in punctuation else ' '+c+' ' \
...                     for c in review]).lower()
...     df.loc[i,'review'] = text
...     pbar.update()
...     counts.update(text.split())
>>>
>>> ## Create a mapping
>>> ## Map each unique word to an integer
>>> word_counts = sorted(counts, key=counts.get, reverse=True)
>>> print(word_counts[:5])
>>> word_to_int = {word: ii for ii, word in \
...                enumerate(word_counts, 1)}
>>>
>>>
>>> mapped_reviews = []
>>> pbar = pyprind.ProgBar(len(df['review']), \
...                        title='Map reviews to ints')
>>> for review in df['review']:

...     mapped_reviews.append([word_to_int[word] \
...                            for word in review.split()])
...     pbar.update()
```

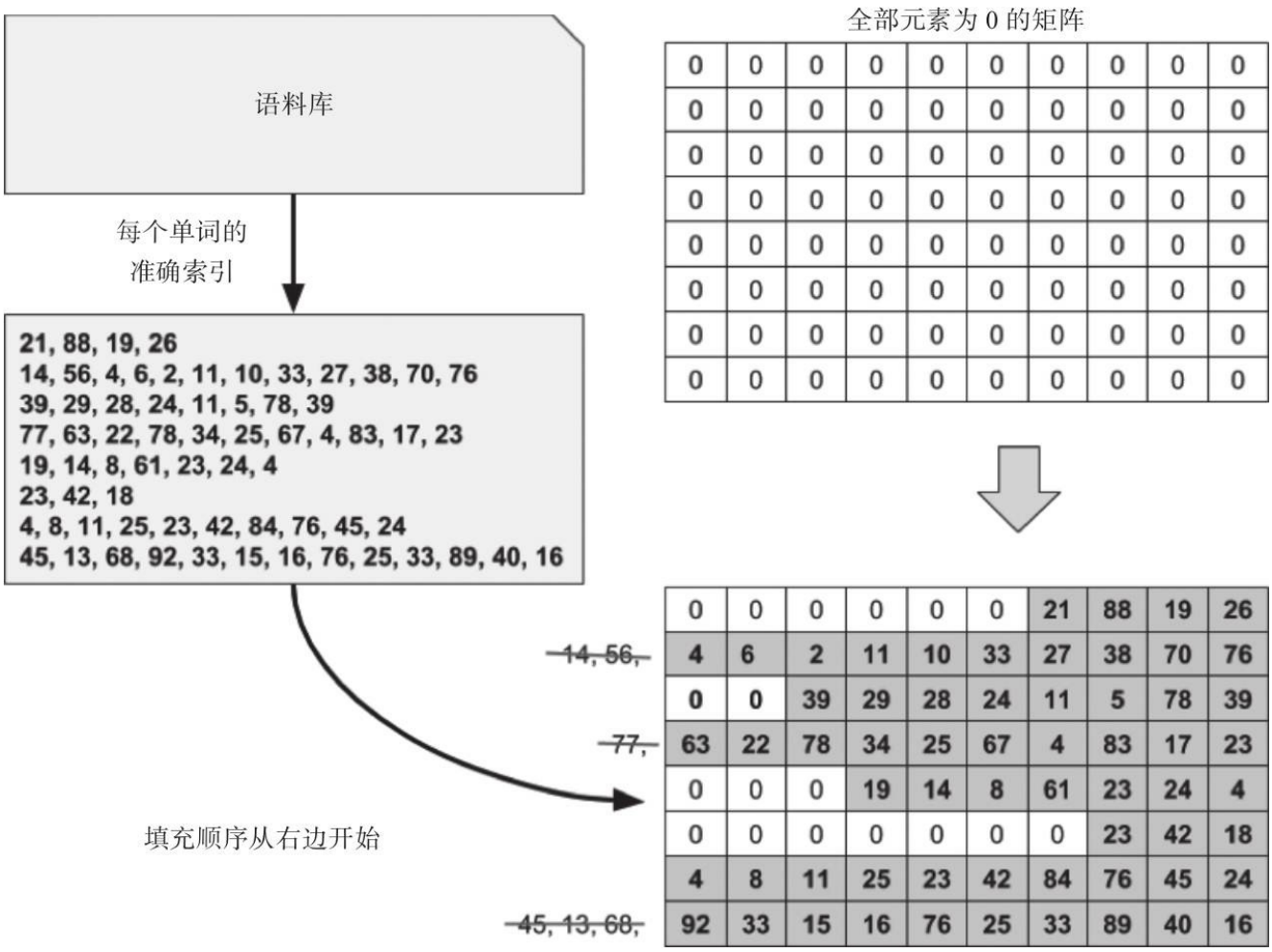
到目前为止，已经把单词序列转换成了整数序列。然而，仍然需要解决序列长度不同的问题。为了生成与RNN体系结构兼容的输入数据，需要确保所有序列的长度相同。

为此定义了参数sequence_length，其值设置为200。长度少于200个单词的序列将用零来填充。反之亦然，长度超过200个单词的序列将被截掉，只保留最后200个单词。可以用下面两步来实现预处理：

1.创建元素全为零的矩阵，其中每行对应一个长度为200的序列。

2.从矩阵的右边开始，根据序列中每个单词的索引来填充。如果序列长度为150，相应行的前50个元素将保持为0。

下图用一个只有8个词的小例子来说明这两个步骤，该例子中每个序列的长度为4、12、8、11、7、3、10和13。




请注意，事实上sequence_length是一个可以调整以优化性能的超参数。由于篇幅的限制，我们并没有进一步优化这个超参数，但是鼓励你尝试不同的序列长度，例如50、100、200、250和300。

以下的代码可以创建相同长度的序列：

```
>>> ## Define same-length sequences
>>> ## if sequence length < 200: left-pad with zeros
>>> ## if sequence length > 200: use the last 200 elements
>>>
>>> sequence_length = 200  ## (Known as T in our RNN formulas)
>>> sequences = np.zeros((len(mapped_reviews), sequence_length),
...                      dtype=int)
>>>
>>> for i, row in enumerate(mapped_reviews):
...     review_arr = np.array(row)
...     sequences[i, -len(row):] = review_arr[-sequence_length:]
```

在对数据集进行预处理之后，可以进一步将数据分割成单独的训练集和测试集。由于数据集已经洗牌，所以可以简单地将数据集的前半部分用于训练，后半部分用于测试，代码如下：

```
>>> X_train = sequences[:25000,:]
>>> y_train = df.loc[:25000, 'sentiment'].values
>>> X_test = sequences[25000:,:]
>>> y test = df.loc[25000:, 'sentiment'].values
```

现在，如果想为交叉验证而分割数据集，那么可以对数据集的下半部分进行进一步的分割，从而生成更小的测试集和用于超参数调优的验证集。

最后定义一个辅助函数，用于将给定的数据集（可以是训练集或测试集）分割成块并返回一个生成器来进一步迭代这些块（也就是小批量）：

```
>>> np.random.seed(123) # for reproducibility

>>> ## Define a function to generate mini-batches:
>>> def create_batch_generator(x, y=None, batch_size=64):
...     n_batches = len(x)//batch_size
...     x = x[:n_batches*batch_size]
...     if y is not None:
...         y = y[:n_batches*batch_size]
...     for ii in range(0, len(x), batch_size):
...         if y is not None:
...             yield x[ii:ii+batch_size], y[ii:ii+batch_size]
...         else:
...             yield x[ii:ii+batch_size]
```

正如在上面代码中所做的那样，使用生成器是解决内存受限问题的非常有用的技术。推荐使用上述方法将数据集分割成小批量来训练神经网络，而不是在训练过程中进行数据分割，创建数据子集并将它们保存在内存。

### 16.4.2 嵌入式

前面的数据准备过程生成了相同长度的序列。这些序列的元素对应于独立单词索引的整数。

这些单词索引可以用几种不同的方式转换成输入特征。一种简单的方法是用独热编码将索引转换成0和1的向量。然后，把每个单词映射到一个向量，该向量的大小是数据集中所有独立单词的数目。考虑到独立单词的数量（词汇量）可能达到20000这样的级别，同时这也将是输入特征的数量，在这样的特征上训练模型可能会导致维数灾难。此外，这些特征非常稀疏，因为除了某个特征以外，其他所有的特征都是零。

一个更优雅的方法是将每个单词映射到大小固定的向量上，并用实值元素（不一定是整数）。与独热编码向量相比，可以用有限大小的向量来表示无限个实数（理论上，可以从给定区间提取无限个实数，例如[-1，1]）。

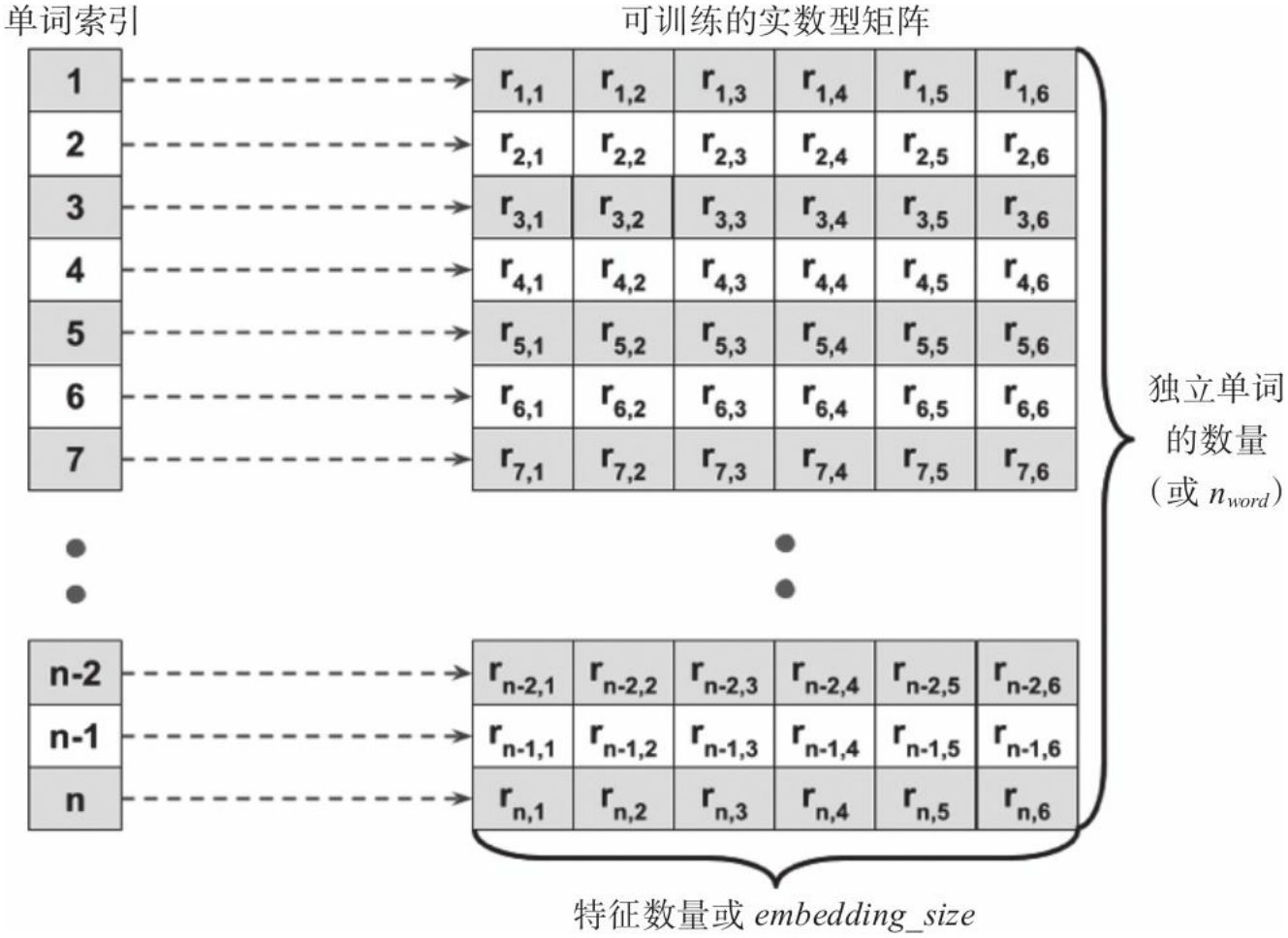
这就是所谓的嵌入式逻辑，这是一种特征学习技术，可以利用该技术自动学习代表数据集独立单词的那些显著特征。假如给定独立单词的数目为unique_words，可以选择嵌入向量的大小，使其远小于独立单词的数量（embedding_size<<unique_words），从而把全部词汇作为输入特征来表示。

与独热编码相比较，嵌入式的优点如下：

·减少特征空间的维数以减轻维数灾难的影响

·训练神经网络的嵌入层因而可提取显著特征

下面的示意图展示了如何将词汇索引映射到可训练的嵌入矩阵：




TensorFlow实现了一个高效的函数tf.nn.embedding_lookup，把对应于独立单词的每个整数映射到可训练矩阵的一行。例如，整数1映射到第一行，整数2映射到第二行，以此类推。然后，给定一个诸如<0，5，3，4，19，2…>这样的整数序列，需要为该序列的每个元素找到相应的行。

现在看看如何在实践中创建嵌入层。如果以tf_x作为输入层，其中对应的词汇索引以tf.int32类型输入，那么创建嵌入层可以分为以下两个步：

1.首先创建大小为[n_words×embedding_size]的矩阵作为张量变量，该变量被称为embedding，用[-1，1]之间的随机浮点数来初始化其元素：

```
embedding = tf.Variable(
                tf.random_uniform(
                    shape=(n_words, embedding_size),
                    minval=-1, maxval=1)
            )
```

2.然后用tf.nn.embedding_lookup函数在嵌入矩阵中查找与tf_x的每个元素相关联的行：

```
embed_x = tf.nn.embedding_lookup(embedding, tf_x)
```

正如在这些步骤中所观察到的，为了创建嵌入层，函数tf.nn.embedding_lookup需要两个参数：嵌入张量和查寻ID。

函数tf.nn.embedding_lookup有几个可选参数，以供调整嵌入层的行为，例如进行L2归一化。可以从下述官方网站阅读更多关于该函数的信息：

https://www.tensorflow.org/api_docs/python/tf/nn/embedding_lookup

### 16.4.3 构建一个RNN模型

现在已经做好了建立RNN模型的准备。下面将用以下方法实现SentimentRNN类：

·一个设置所有的模型参数，创建计算图，调用self.build方法建立多层RNN模型的构造器。

·build方法为输入数据、输入标签和隐藏层配置的淘汰概率声明三个占位符。在声明后，创建嵌入层，并用嵌入表示作为输入构建多层RNN。

·train方法用来创建TensorFlow会话以启动计算图，遍历小批量数据，迭代固定数量的次数以及最小化图中定义的成本函数。该方法在10次迭代后把模型存储起来以供阶段性检查。

·predict方法用来创建新会话，恢复在训练过程中最后一个检查点所保存的模型，以对测试数据进行预测。

下面的代码将看到这个类及其方法被分解成单独的代码段实现的细节。

## 16.4.4 情感RNN类构造器

首先从SentimentRNN类的类构造器开始，具体代码如下：

```
import tensorflow as tf

class SentimentRNN(object):
    def __init__(self, n_words, seq_len=200,
                 lstm_size=256, num_layers=1, batch_size=64,
                 learning_rate=0.0001, embed_size=200):
        self.n_words = n_words
        self.seq_len = seq_len
        self.lstm_size = lstm_size  ## number of hidden units
        self.num_layers = num_layers
        self.batch_size = batch_size
        self.learning_rate = learning_rate
        self.embed_size = embed_size

        self.g = tf.Graph()
        with self.g.as_default():
            tf.set_random_seed(123)
            self.build()
            self.saver = tf.train.Saver()
            self.init_op = tf.global_variables_initializer()
```

这里必须设置参数n_words使其等于独立单词的数目+1（加上1是因为当序列长度小于200时用0来填充），在创建嵌入层时使用该参数和超参数embed_size。同时，必须根据在前面预处理时创建的序列长度来设置变量seq_len。注意，lstm_size是这里用到的另一个超参数，它决定了每个RNN层中隐藏单元的数量。

### 16.4.5 build方法

接下来讨论SentimentRNN类的build方法。这是该序列中最长和最关键的方法，因此将对此进行非常详细的讨论。首先将代码完整地看一遍，以确保对整体有所认识，然后再对每个主要部分进行分析：

```
def build(self):
    ## Define the placeholders
    tf_x = tf.placeholder(tf.int32,
                shape=(self.batch_size, self.seq_len),
                name='tf_x')
    tf_y = tf.placeholder(tf.float32,
                shape=(self.batch_size),
                name='tf_y')
    tf_keepprob = tf.placeholder(tf.float32,
                name='tf_keepprob')

    ## Create the embedding layer
    embedding = tf.Variable(
                tf.random_uniform(
                    (self.n_words, self.embed_size),
                    minval=-1, maxval=1),
                name='embedding')
    embed_x = tf.nn.embedding_lookup(
                embedding, tf_x,
                name='embeded_x')

    ## Define LSTM cell and stack them together
    cells = tf.contrib.rnn.MultiRNNCell(
            [tf.contrib.rnn.DropoutWrapper(
               tf.contrib.rnn.BasicLSTMCell(self.lstm_size),
               output_keep_prob=tf_keepprob)
             for i in range(self.num_layers)])

    ## Define the initial state:
    self.initial_state = cells.zero_state(
             self.batch_size, tf.float32)
    print('  << initial state >> ', self.initial_state)

    lstm_outputs, self.final_state = tf.nn.dynamic_rnn(
             cells, embed_x,
             initial_state=self.initial_state)

    ## Note: lstm_outputs shape:
##  [batch_size, max_time, cells.output_size]
print('\n  << lstm_output   >> ', lstm_outputs)
print('\n  << final state   >> ', self.final_state)

logits = tf.layers.dense(
         inputs=lstm_outputs[:, -1],
```

```
            units=1, activation=None,
            name='logits')

    logits = tf.squeeze(logits, name='logits_squeezed')
    print ('\n  << logits        >> ', logits)

    y_proba = tf.nn.sigmoid(logits, name='probabilities')
    predictions = {
        'probabilities': y_proba,
        'labels' : tf.cast(tf.round(y_proba), tf.int32,
            name='labels')
    }
    print('\n  << predictions   >> ', predictions)

    ## Define the cost function
    cost = tf.reduce_mean(
            tf.nn.sigmoid_cross_entropy_with_logits(
            labels=tf_y, logits=logits),
            name='cost')

    ## Define the optimizer
    optimizer = tf.train.AdamOptimizer(self.learning_rate)
    train_op = optimizer.minimize(cost, name='train_op')
```

build方法首先创建输入数据所需要的三个占位符tf_x，tf_y和tf_keepprob。然后如前所述增加嵌入层并构建嵌入式表示embed_x。

接着用build方法构建带有LSTM单元的RNN网络。见以下三个步骤：

1.首先定义多层RNN单元。

2.其次定义这些单元的初态。

3.最后创建由RNN单元及其初态定义的RNN。

把这三步分解成三部分来详细阐述，由此可以深入研究如何用build方法来构建RNN网络。

### 第一步：定义多层RNN单元

想了解如何编码build方法来构建RNN网络，首先要定义多层RNN单元。

幸运的是TensorFlow有非常好的可用来定义LSTM单元的封装类BasicLSTMCell，它们可以通过调用MultiRNNCell封装类堆叠起来形成多层RNN。用淘汰堆叠RNN单元的过程有三个嵌套步骤，可以从内部把这三个嵌套步骤描述如下：

1.首先用f.contrib.rnn.BasicLSTMCell来创建RNN单元。

2.用tf.contrib.rnn.DropoutWrapper对RNN单元应用淘汰策略。

3.根据理想的RNN层数创建单元列表，然后把此列表传递给函数tf.contrib.rnn.MultiRNNCell。

在build方法的实现代码中，该列表是根据对Python列表的理解而创建的。注意，此列表对于每层只有一个单元。

可以从下述网站了解更多有关这些功能的信息：

·tf.contrib.rnn.BasicLSTMCell：https://www.tensorflow.org/api_docs/python

·tf.contrib.rnn.DropoutWrapper：https://www.tensorflow.org/api_docs/pythor

·tf.contrib.rnn.MultiRNNCell：https://www.tensorflow.org/api_docs/python/t

### 第二步：定义CNN单元的初态

创建RNN网络的第二步是定义RNN单元的初态。

你可能会想到LSTM的单元结构，LSTM单元有三种类型的输入：即输入数据x（t），前一个时间步隐藏单元的激活$h^{(t-1)}$，前一个时间步的单元状态$C^{(t-1)}$。

因此，在build方法的实现中，$x^{(t)}$是一个嵌入的embed_x数据张量。然而，在评估单元时，还需要指定单元先前的状态。因此，当开始处理新的输入序列时，要先将单元状态初始化为0状态，然后在每个时间步之后，需要更新存储单元的状态以用于下一个时间步。

一旦定义了多层RNN对象（实现的cells），就可以用cells.zero_state在build方法中定义它的初态。

### 第三步：用RNN单元及其初始化值创建RNN

用build方法创建RNN的第三步是使用tf.nn.dynamic_rnn函数组合所有的组件。

因此，函数tf.nn.dynamic_rnn整合嵌入数据、RNN单元及其初态，并根据LSTM单元所展现的体系结构为其创建管道。

函数tf.nn.dynamic_rnn返回的元组中包含了RNN单元的激活、输出outputs

及其终态state。输出为带形状的（batch_size，num_steps，lstm_size）三维张量。我们把outputs传递给全连接层以获得logits，并且存储终态以作为下个小批量数据的初态。

请在下述官方文档中阅读更多关于函数tf.nn.dynamic_rnn的信息：

https://www.tensorflow.org/api_docs/python/tf/nn/dynamic_rnn

最后，在用build方法建立了RNN网络组件后，可以像任何其他神经网络一样定义成本函数和优化方案。

## 16.4.6 train方法

SentimentRNN类的下一个方法是train。该方法的调用与第14章和第15章中创建的训练方法非常相似，除了有额外提供给网络张量的state以外。

下述代码展示了train方法的实现细节：

```
def train(self, X_train, y_train, num_epochs):
    with tf.Session(graph=self.g) as sess:
        sess.run(self.init_op)
        iteration = 1
        for epoch in range(num_epochs):
            state = sess.run(self.initial_state)

            for batch_x, batch_y in create_batch_generator(
                    X_train, y_train, self.batch_size):
                feed = {'tf_x:0': batch_x,
                        'tf_y:0': batch_y,
                        'tf_keepprob:0': 0.5,
                        self.initial_state : state}
                loss, _, state = sess.run(
                        ['cost:0', 'train_op',
                         self.final_state],
                         feed_dict=feed)

                if iteration % 20 == 0:
                    print("Epoch: %d/%d Iteration: %d "
                          "| Train loss: %.5f" % (
                           epoch + 1, num_epochs,
                           iteration, loss))

                iteration +=1
            if (epoch+1)%10 == 0:
                self.saver.save(sess,
                    "model/sentiment-%d.ckpt" % epoch)
```

在这种train方法的实现中，在每次迭代开始时，以RNN单元的0状态作为当前状态。通过将当前状态与数据batch_x及其标签batch_y一起在每个小批量数据上执行。在处理完一个小批量之后，调用tf.nn.dynamic_rnn函数，将状态更新为终态。更新后的状态将用于执行下一个小批量。反复进行该过程并在整个迭代过程中不断地更新当前的状态。

## 16.4.7 predict方法

SentimentRNN类中的最后一个方法是predict，它与train方法类似，需要不断地更新当前的状态，具体代码如下：

```
def predict(self, X_data, return_proba=False):
    preds = []
    with tf.Session(graph = self.g) as sess:
        self.saver.restore(
            sess, tf.train.latest_checkpoint('./model/'))
        test_state = sess.run(self.initial_state)
        for ii, batch_x in enumerate(
            create_batch_generator(
                X_data, None, batch_size=self.batch_size), 1):
            feed = {'tf_x:0' : batch_x,
                    'tf_keepprob:0' : 1.0,
                    self.initial_state : test_state}
            if return_proba:
                pred, test_state = sess.run(
                    ['probabilities:0', self.final_state],
                    feed_dict=feed)
            else:
                pred, test_state = sess.run(
                    ['labels:0', self.final_state],
                    feed_dict=feed)

            preds.append(pred)

    return np.concatenate(preds)
```

## 16.4.8 创建SentimentRNN类的实例

SentimentRNN类总共有四个部分，即类构造器、build、train和predict方法。

我们已经完成了编码和检查，现在准备创建一个情感RNN类的对象，其参数如下：

```
>>> n_words = max(list(word_to_int.values())) + 1
>>>
>>> rnn = SentimentRNN(n_words=n_words,
...                    seq_len=sequence_length,
...                    embed_size=256,
...                    lstm_size=128,
...                    num_layers=1,
...                    batch_size=100,
...                    learning_rate=0.001)
```

请注意，设置num_layers=1来使用单层RNN。尽管代码实现允许通过设置num_layers大于1来创建多层RNN。这里我们应该考虑数据集的规模，单层RNN可以更好地推广到未见过的数据，因为它不太可能过拟合训练数据。

### 16.4.9 训练与优化情感分析RNN模型

接下来，我们可以通过调用rnn.train函数来训练RNN模型。下面的代码用来自于X_train的输入和y_train存储的相应分类标签经过40次迭代来训练模型：

```
>>> rnn.train(X_train, y_train, num_epochs=40)
Epoch: 1/40 Iteration: 20 | Train loss: 0.70637
Epoch: 1/40 Iteration: 40 | Train loss: 0.60539
Epoch: 1/40 Iteration: 60 | Train loss: 0.66977
Epoch: 1/40 Iteration: 80 | Train loss: 0.51997
...
```

用TensorFlow的检查点系统保存训练过的模型，这部分曾经在第14章中讨论过。现在可以用训练好的模型来对测试集上的分类标签进行预测，详情如下：

```
>>> preds = rnn.predict(X_test)
>>> y_true = y_test[:len(preds)]
>>> print('Test Acc.: %.3f' % (
...     np.sum(preds == y_true) / len(y_true))
```

结果显示预测的准确率为86%。考虑到这是一个小规模的数据集，可以与第8章中从测试集上获得的预测准确率相比较。

可以通过调整超参数（如lstm_size，seq_len和embed_size）来进一步优化模型，以获得更好的泛化性能。然而，对于超参数调优，我们建议创建单独的验证数据集，这样就可以不必重复使用测试集进行评估，从而避免因测试数据的泄漏导致的偏差，这曾在第6章中讨论过。

此外，如果对测试集的预测概率而非分类标签的预测准确率感兴趣，那么可以如下设置return_proba=True：

```
>>> proba = rnn.predict(X_test, return_proba=True)
```

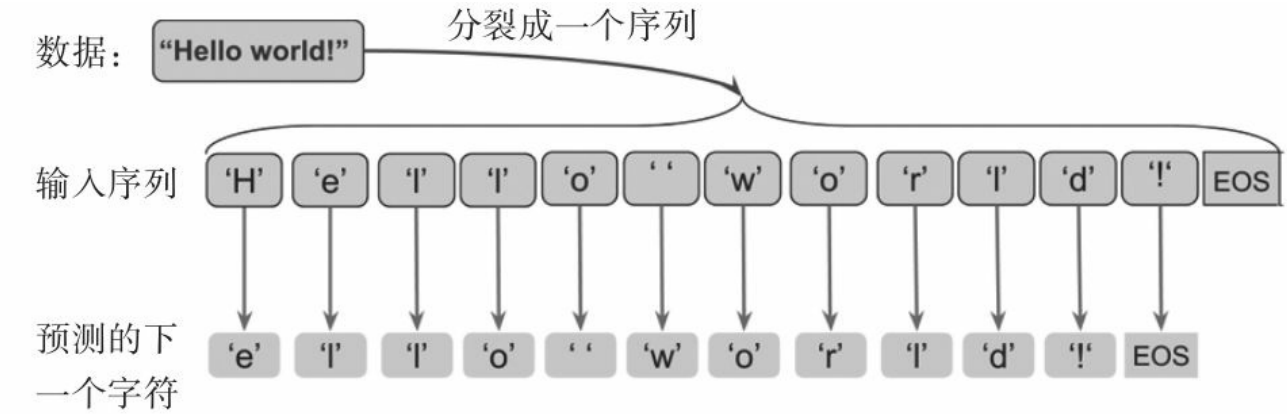
这是第一个用于情感分析的RNN模型。我们将进一步研究并建立用于字符序列语言建模的RNN，作为序列建模的另一个常用的应用。

## 16.5 项目二：用TensorFlow实现字符级RNN语言建模

语言建模是一个迷人的应用，它使机器能完成与人类语言相关的任务，如生成英语句子。苏特斯克、马滕斯和欣顿在这个方面做出了有趣的努力（苏特斯克、马滕斯和欣顿在2011年第28届《机器学习国际会议》（ICML-11）发表的论文《用递归神经网络生成文本》，https://pdfs.semanticscholar.org/93c2/0e38c85b69fc2d2eb314b3c1217913f

在要构建的模型中，输入为文本文档，目标是研发可以生成与输入文档相似新文本的模型。这样的输入例子可以是书或某种编程语言的计算机程序。

在字符级语言建模中，输入被分解为一系列字符，这些字符一次一个地输入到网络中。网络处理每个新字符，同时结合看过的字符的记忆来预测下一个字符。下图显示了字符级语言建模的示例：




可以把该实现分成三步：准备数据、建立RNN模型、预测下个字符以及采样生成新文本。

记得在本章的前几节中曾经提到过梯度爆炸问题。我们将有机会在该应用中体验如何用梯度裁剪技术来避免梯度爆炸。

## 16.5.1 准备数据

本节将为字符级语言建模准备数据。

古腾堡项目网站的网址是：https://www.gutenberg.org/，它提供数以千计的免费电子书，可以访问该网站以获得输入数据。例如，可以访问http://www.gutenberg.org/cache/epub/2265/pg2265.txt得到纯文本格式的威廉·莎士比亚的悲剧《哈姆雷特》。

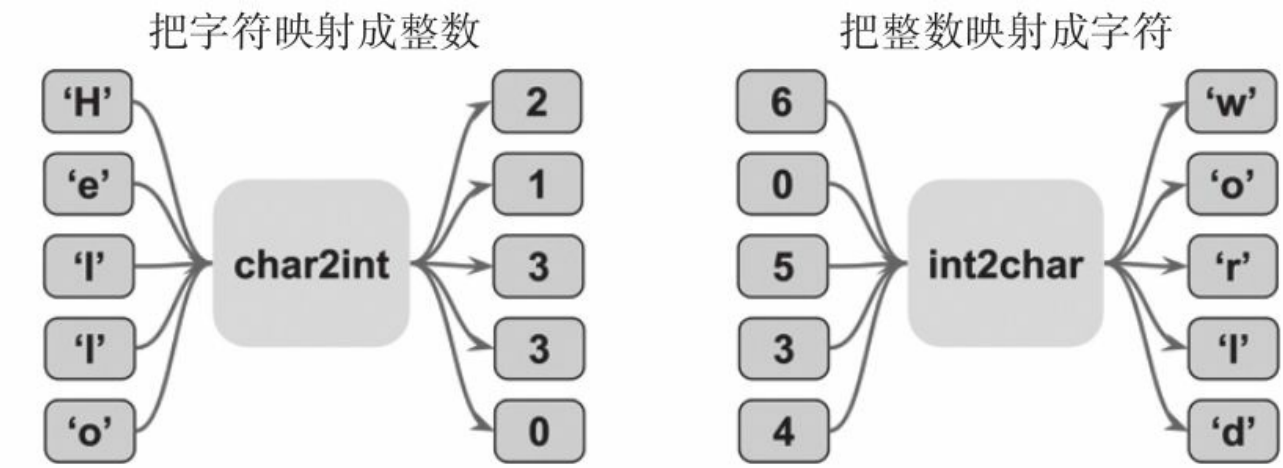
请注意，这个链接将直接把你带到下载页面。如果用的是macOS或Linux操作系统，则可以在终端上使用以下命令下载文件：

```
curl http://www.gutenberg.org/cache/epub/2265/pg2265.txt > pg2265.txt
```

如果将来该资源不可用，可以从本书代码库本章代码目录下获得该文本的副本：

https://github.com/rasbt/python-machine-learning-book-2nd-edition

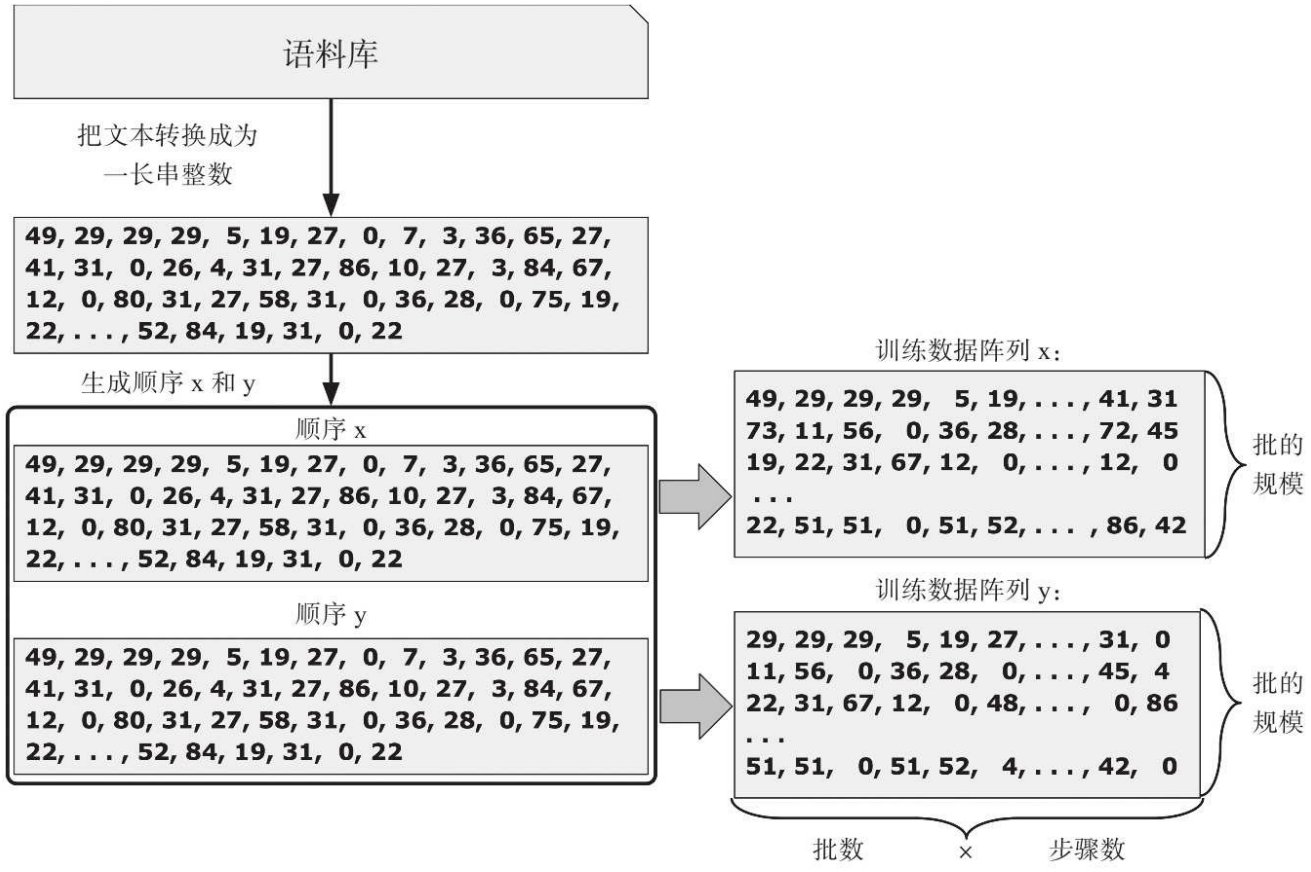
一旦有了一些数据，就可以把它作为纯文本读入Python会话。在下面的代码中，Python的变量chars代表该文本中所观察到的独立字符集。然后创建一个字典char2int，将每个字符映射为一个整数，也可以创建一个反向映射字典int2char，将整数映射为那些独立字符。用char2int字典将文本转换成NumPy的整数阵列。下图显示了将字符"Hello"和"world"转换成整数以及反向映射的例子：




该代码从下载的链接中读取文本，删除文本开始部分那些古腾堡项目的法律描述，然后基于文本来构造字典：

```
>>> import numpy as np
>>> ## Reading and processing text
>>> with open('pg2265.txt', 'r', encoding='utf-8') as f:
...     text=f.read()
>>> text = text[15858:]
>>> chars = set(text)
>>> char2int = {ch:i for i,ch in enumerate(chars)}
>>> int2char = dict(enumerate(chars))
>>> text_ints = np.array([char2int[ch] for ch in text],
...                      dtype=np.int32)
```

现在应该将数据重组成批量序列，这是准备数据的最重要步骤。正如所知道那样，目标是根据所观察到的字符序列来预测下一个字符。因此，将神经网络的输入（x）和输出（y）移动一个字符。下图显示了预处理的步骤，从文本语料库生成数据阵列x和y开始：

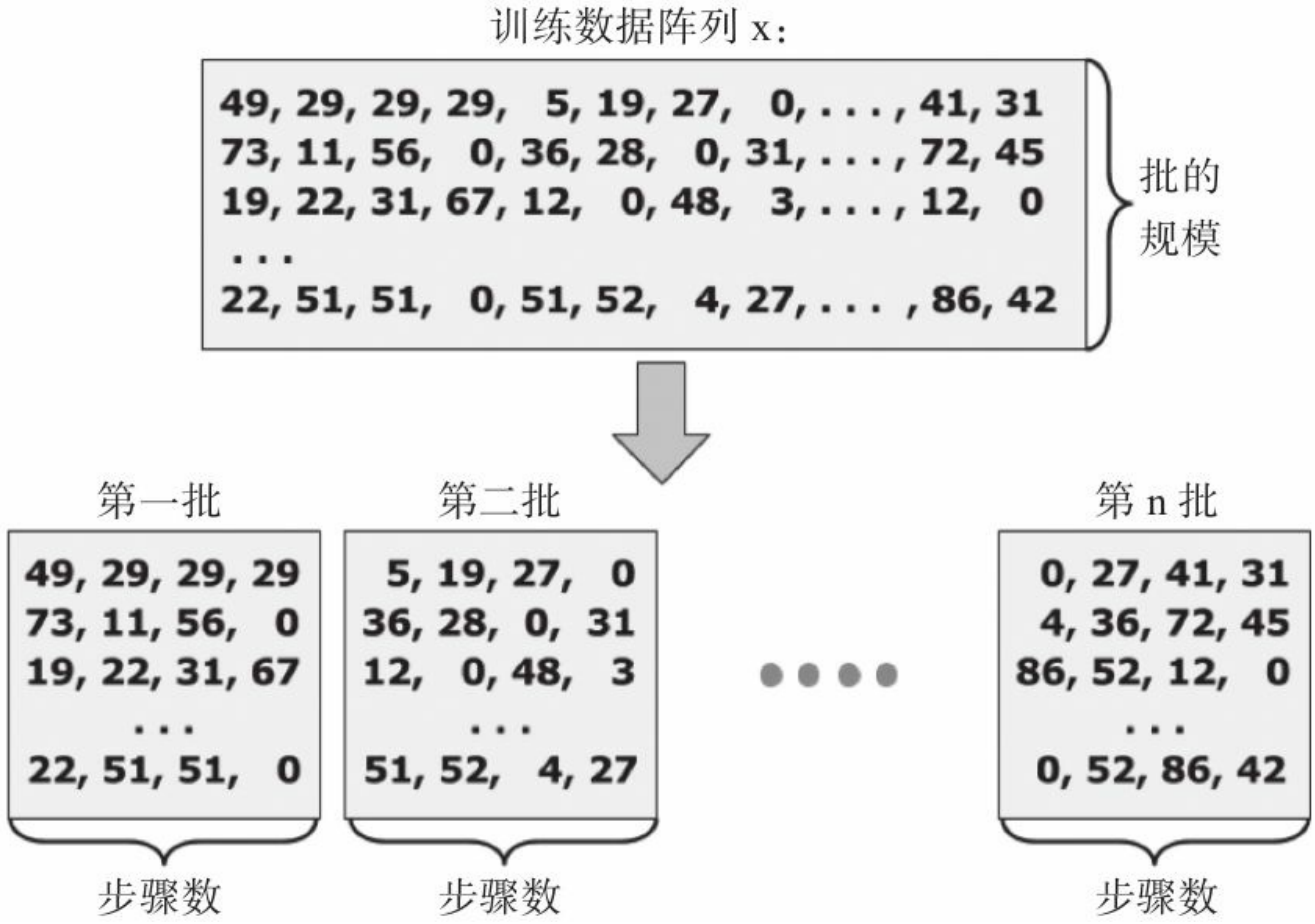



如图所示，训练阵列x和y具有相同的形状或尺寸，其中行数等于批的规模，列数为批数×步数。

假设输入阵列data包含与文本语料库中的字符对应的整数，下面的函数将生成x和y，与前图所示的结构一致：

```
>>> def reshape_data(sequence, batch_size, num_steps):
...     tot_batch_length = batch_size * num_steps
...     num_batches = int(len(sequence) / tot_batch_length)
...     if num_batches*tot_batch_length + 1 > len(sequence):
...         num_batches = num_batches - 1
...     ## Truncate the sequence at the end to get rid of
...     ## remaining charcaters that do not make a full batch
...     x = sequence[0: num_batches*tot_batch_length]
...     y = sequence[1: num_batches*tot_batch_length + 1]
...     ## Split x & y into a list batches of sequences:
...     x_batch_splits = np.split(x, batch_size)
...     y_batch_splits = np.split(y, batch_size)
...     ## Stack the batches together
...     ## batch_size x tot_batch_length
...     x = np.stack(x_batch_splits)
...     y = np.stack(y_batch_splits)
...
...     return x, y
```

下一步是将阵列x和y分成小批量，其中每行是一个长度等于步数的序列。拆分阵列数组x的过程见下图：




下面的代码定义了一个名为create_batch_generator的函数，如前图所示的那样拆分数据阵列x和y，并输出批处理生成器。稍后，将在训练网络时用该生成器迭代小批量：

```
>>> def create_batch_generator(data_x, data_y, num_steps):
...     batch_size, tot_batch_length = data_x.shape
...     num_batches = int(tot_batch_length/num_steps)
...     for b in range(num_batches):
...         yield (data_x[:, b*num_steps:(b+1)*num_steps],
...                data_y[:, b*num_steps:(b+1)*num_steps])
```

迄今已经完成了数据预处理，并且数据格式正确。下一节将实现字符级RNN语言建模。

## 16.5.2 构建字符级RNN语言模型

为了构建字符级神经网络，我们将实现一个被称为CharRNN的类，该类构造RNN图，以便在观察给定字符序列之后预测下一个字符。从分类角度看，类的数目是文本语料库中存在的独立字符的总数。CharRNN类有如下四个方法：

·构造器用于构造学习参数、创建计算图、调用build方法，构造基于采样模式与训练模式的图。

·build方法定义提供数据的占位符、用LSTM单元构建RNN、定义网络的输出、成本函数和优化器。

·train方法遍历小批量迭代并通过指定数量的迭代次数来训练网络。

·sample方法从给定字符串开始计算下一个字符出现的概率，并根据这些概率随机选择一个字符。反复进行这个过程，采样的字符将被串联在一起形成字符串。一旦该字符串达到指定长度，字符串将被返回。

把这四种方法分解成单独的代码段并分别进行解释。注意，该模型的RNN部分实现与项目一（使用多层RNN进行IMDb电影评论的情感分析）的RNN部分非常相似。因此这里将跳过建立RNN组件的描述。

### 16.5.3 构造器

在前面的情感分析实现中，相同的计算图被用于训练和预测模式中，与此相反，这次的训练与采样模式将使用不同的计算图。

因此，需要为构造器增加一个新的布尔类型参数，以确定正在构建的模型是用于训练模式还是采样模式。下面的代码显示了包含在类定义中的构造器的实现过程：

```
import tensorflow as tf
import os

class CharRNN(object):
    def __init__(self, num_classes, batch_size=64,
                 num_steps=100, lstm_size=128,
                 num_layers=1, learning_rate=0.001,
                 keep_prob=0.5, grad_clip=5,
                 sampling=False):
        self.num_classes = num_classes
        self.batch_size = batch_size
        self.num_steps = num_steps
        self.lstm_size = lstm_size
        self.num_layers = num_layers
        self.learning_rate = learning_rate
        self.keep_prob = keep_prob
        self.grad_clip = grad_clip

        self.g = tf.Graph()
        with self.g.as_default():
            tf.set_random_seed(123)

      self.build(sampling=sampling)

      self.saver = tf.train.Saver()

      self.init_op = tf.global_variables_initializer()
```

正如先前所计划的那样，布尔参数sampling用于确定CharRNN实例是在训练模式（sampling=False）还是在采样模式（sampling=True）中构建图。

除了参数sampling之外，还引入了一个被称为grad_clip的新参数，它用于简化梯度以避免前面提到的梯度爆炸问题。

然后，与前面的实现类似，构造器创建计算图，为了一致的输出而设置图级的随机种子，并通过调用build方法来构建图。

## 16.5.4 build方法

CharRNN类的下一个方法是build，它与项目一中（利用多层RNN进行IMDb电影评论的情感分析）的build方法非常相似，除了一些细微的差异以外。build方法首先根据模式定义batch_size和num_steps两个局部变量，具体如下：

$$\text{采样模式：}\begin{cases} batch_size = 1 \\ num_steps = 1 \end{cases}$$

$$\text{训练模式：}\begin{cases} batch_size = self.batch_size \\ num_steps = self.num_steps \end{cases}$$

回想在情感分析实现中，我们用嵌入层来创建数据集中独立词的显著表示。相反，我们在这里用独热编码方案，对于x和y都有depth=num_classes，其中num_classes实际上是文本语料库的字符总数。

建立多层RNN模型组件的过程与情感分析中用tf.nn.dynamic_rnn函数的实现过程完全相同。然而来源于tf.nn.dynamic_rnn函数的outputs是一个三维张量，其形状为batch_size、num_steps、lstm_size。接着该张量将被重新变换成二维张量，其形状为batch_size*num_steps，lstm_size，将其传递给tf.layers.dense函数，以形成全连接层并获得logits（净输入）。最后得到下一批字符的概率并定义成本函数。此外，这里用tf.clip_by_global_norm函数进行梯度简化，以避免梯度爆炸问题。

下面的代码展示了刚刚描述的新build方法的实现过程：

```
def build(self, sampling):
    if sampling == True:
        batch_size, num_steps = 1, 1
    else:
        batch_size = self.batch_size
        num_steps = self.num_steps

    tf_x = tf.placeholder(tf.int32,
                          shape=[batch_size, num_steps],
                          name='tf_x')
    tf_y = tf.placeholder(tf.int32,
                          shape=[batch_size, num_steps],
                          name='tf_y')
```

```
    tf_keepprob = tf.placeholder(tf.float32,
                          name='tf_keepprob')

    # One-hot encoding:
    x_onehot = tf.one_hot(tf_x, depth=self.num_classes)
    y_onehot = tf.one_hot(tf_y, depth=self.num_classes)

    ### Build the multi-layer RNN cells
    cells = tf.contrib.rnn.MultiRNNCell(
        [tf.contrib.rnn.DropoutWrapper(
            tf.contrib.rnn.BasicLSTMCell(self.lstm_size),
            output_keep_prob=tf_keepprob)
        for _ in range(self.num_layers)])

    ## Define the initial state
    self.initial_state = cells.zero_state(
                batch_size, tf.float32)

    ## Run each sequence step through the RNN
    lstm_outputs, self.final_state = tf.nn.dynamic_rnn(
                cells, x_onehot,
                initial_state=self.initial_state)

    print('  << lstm_outputs  >>', lstm_outputs)

    seq_output_reshaped = tf.reshape(
                lstm_outputs,
            shape=[-1, self.lstm_size],
            name='seq_output_reshaped')

logits = tf.layers.dense(
            inputs=seq_output_reshaped,
            units=self.num_classes,
            activation=None,
            name='logits')

proba = tf.nn.softmax(
            logits,
            name='probabilities')

y_reshaped = tf.reshape(
            y_onehot,
            shape=[-1, self.num_classes],
            name='y_reshaped')
cost = tf.reduce_mean(
            tf.nn.softmax_cross_entropy_with_logits(
                logits=logits,
                labels=y_reshaped),
            name='cost')

# Gradient clipping to avoid "exploding gradients"
tvars = tf.trainable_variables()
grads, _ = tf.clip_by_global_norm(
            tf.gradients(cost, tvars),
            self.grad_clip)
```

```
optimizer = tf.train.AdamOptimizer(self.learning_rate)
train_op = optimizer.apply_gradients(
            zip(grads, tvars),
            name='train_op')
```

### 16.5.5 train方法

CharRNN类的下一个方法是train，它与项目一（使用多层RNN进行IMDb电影评论的情感分析）中所描述的train方法非常类似。下面是train方法的代码：

```
def train(self, train_x, train_y,
          num_epochs, ckpt_dir='./model/'):
    ## Create the checkpoint directory
    ## if it does not exists
    if not os.path.exists(ckpt_dir):
        os.mkdir(ckpt_dir)

    with tf.Session(graph=self.g) as sess:
        sess.run(self.init_op)

        n_batches = int(train_x.shape[1]/self.num_steps)
        iterations = n_batches * num_epochs
        for epoch in range(num_epochs):

            # Train network
            new_state = sess.run(self.initial_state)
            loss = 0
            ## Mini-batch generator:
            bgen = create_batch_generator(
                    train_x, train_y, self.num_steps)
            for b, (batch_x, batch_y) in enumerate(bgen, 1):
                iteration = epoch*n_batches + b

                feed = {'tf_x:0': batch_x,
                        'tf_y:0': batch_y,
                        'tf_keepprob:0' : self.keep_prob,
                        self.initial_state : new_state}
                batch_cost, _, new_state = sess.run(
                        ['cost:0', 'train_op',
                            self.final_state],
                        feed_dict=feed)
                if iteration % 10 == 0:
                    print('Epoch %d/%d Iteration %d'
                          '| Training loss: %.4f' % (
                          epoch + 1, num_epochs,
                          iteration, batch_cost))

            ## Save the trained model
            self.saver.save(
                    sess, os.path.join(
                        ckpt_dir, 'language_modeling.ckpt'))
```

### 16.5.6 sample方法

CharRNN类中最后一个的方法是sample。sample方法的行为与在项目一（使用多层RNN进行IMDb电影评论的情感分析）所实现的predict方法类似。然而，差别在于这里我们根据观察到的序列observed_seq计算下一个字符的概率。然后，将这些概率传递给一个名为get_top_char的函数，根据所获得的概率随机选择一个字符。

最初，观察到的序列从作为参数提供的starter_seq开始。当根据预测的概率对新字符进行采样时，它们被添加到观察序列，然后用新的观察序列来预测下一个字符。

sample方法的代码实现如下：

```
def sample(self, output_length,
           ckpt_dir, starter_seq="The "):
    observed_seq = [ch for ch in starter_seq]
    with tf.Session(graph=self.g) as sess:
        self.saver.restore(
            sess,
            tf.train.latest_checkpoint(ckpt_dir))
        ## 1: run the model using the starter sequence
        new_state = sess.run(self.initial_state)
        for ch in starter_seq:
            x = np.zeros((1, 1))
            x[0, 0] = char2int[ch]
            feed = {'tf_x:0': x,
                    'tf_keepprob:0': 1.0,
                    self.initial_state: new_state}
            proba, new_state = sess.run(
                    ['probabilities:0', self.final_state],
                    feed_dict=feed)

        ch_id = get_top_char(proba, len(chars))
        observed_seq.append(int2char[ch_id])

        ## 2: run the model using the updated observed_seq
        for i in range(output_length):
            x[0,0] = ch_id
            feed = {'tf_x:0': x,
                    'tf_keepprob:0': 1.0,
                    self.initial_state: new_state}
            proba, new_state = sess.run(
                    ['probabilities:0', self.final_state],
                    feed_dict=feed)

            ch_id = get_top_char(proba, len(chars))
            observed_seq.append(int2char[ch_id])

    return ''.join(observed_seq)
```

这里，sample方法调用get_top_char函数来根据观察到的概率随机（ch_id）选择字符ID。

get_top_char函数首先对概率进行排序，然后将top_n的概率传递给函数numpy.random.choice以从头部概率中随机选择一个。get_top_char函数的实现过程如下：

```
def get_top_char(probas, char_size, top_n=5):
    p = np.squeeze(probas)
    p[np.argsort(p)[:-top_n]] = 0.0
    p = p / np.sum(p)
```

```
    ch_id = np.random.choice(char_size, 1, p=p)[0]
    return ch_id
```

注意，该函数应该在定义CharRNN类之前就定义好。我们已经按照这个顺序对概念进行了解释。通过浏览伴随本章的代码笔记本可以对函数定义的顺序有更好的宏观了解。

## 16.5.7 创建和训练CharRNN模型

现在准备好创建CharRNN类的实例来构建RNN模型，并用以下参数配置来训练：

```
>>> batch_size = 64
>>> num_steps = 100
>>> train_x, train_y = reshape_data(text_ints,
...                                 batch_size,
...                                 num_steps)
>>>
>>> rnn = CharRNN(num_classes=len(chars), batch_size=batch_size)
>>> rnn.train(train_x, train_y,
...           num_epochs=100,
...           ckpt_dir='./model-100/')
```

训练的模型将保存在一个名为./model-100/的目录中，这样就可以在以后重新加载来进行预测或继续训练。

## 16.5.8 处于取样状态的CharRNN模型

接下来，可以通过定义sampling=True，在采样模式下创建一个CharRNN类的新实例。调用sample方法将保存的模型加载到目录./model-100/，然后生成一个500个字符的序列：

```
>>> del rnn
>>>
>>> np.random.seed(123)
>>> rnn = CharRNN(len(chars), sampling=True)
>>> print(rnn.sample(ckpt_dir='./model-100/',
...                  output_length=500))
```

生成的文本如下：

```
   Ham. I woll thenke the Solde and as ither thes will, at a dis tantend
To maness of he and mering and the bus and,
The hiss a fit of ant ort time, and her wind of
A beart of his mine it a faulouthensers

   Hal. Whe that so my Larger,
Thin we selfe to mat tean the hims but
With was sore to beene tiue to ser is betit,
Was thin so a mangers and hill or and asthie

   Hor. This mest the senges of hation thee to hos the herr,
The sacke a my Lort worke. That his she lete,
And whise howers
```

从输出结果可以看到，有些英语单词大多被保留。同样重要的是要注意其来源是古英语文本，因此原文中的一些词可能不太熟悉。为了获得更好的结果，需要更多次迭代的模型训练。我们鼓励你采用更大的文档来重现这个结果，用更多的迭代来训练模型。

## 16.6 总结

我们希望你喜欢《Python机器学习》的最后一章，以及我们精彩的机器学习和深度学习之旅。本书已经涵盖了这个领域必须提供的基本主题，现在你应该已经有足够的准备将这些技术付诸行动来解决现实世界的问题。

我们从简要概述有监督学习、强化学习、无监督学习开始了我们的旅程。然后从第2章简单的单层神经网络开始，讨论了几种不同的可用于分类任务的学习算法。

我们继续在第3章讨论先进的分类算法，并且在第4章和第5章中学习了机器学习流水线的最重要的几个方面。

请记住，即使是最先进的算法也会受训练数据信息的限制。因此，在第6章中，我们了解了构建和评估预测模型的最佳实践，这是机器学习应用的另一个重要方面。

如果单个学习算法无法达到所期望的性能，有时可以创建专家组合来进行预测。第7章对此进行了探索。

第8章使用机器学习来分析现代社会中最常见和最有趣的数据形式之一，即由互联网社交媒体平台掌控的文本文档。

我们提醒自己，机器学习技术不仅限于离线的数据分析，接下来第9章讨论了如何将机器学习模型嵌入网络应用以便与外部世界共享。

在大多数情况下我们的聚焦点在分类算法上，这可能是机器学习最常见的应用。然而，这并不是我们旅程的终点！第10章探讨了回归分析预测连续值输出的几种算法。

机器学习的另外一个令人兴奋的子领域是聚类分析，它可以在数据中找到隐藏的结构，即使训练数据无法提供正确的答案供机器学习使用。第11章对此进行了讨论。

随后我们把注意力转移到整个机器学习领域中最令人兴奋的算法——人工神经网络。第12章用NumPy从头开始实现了多层感知器。

在第13章中，TensorFlow的能力变得很明显，这里用TensorFlow来帮助神经网络模型的构建，并充分利用GPU使多层神经网络的训练更为有效。

第14章深入研究了TensorFlow的工作原理，并讨论了TensorFlow的不同方面和工作机制，包括TensorFlow计算图中的变量和运算符、变量的范围、如何启动计算图以及处理节点的不同方法。

第15章深入讨论了卷积神经网络，由于其在图像识别任务中的绝佳表现，目前已经在计算机视觉中获得广泛的应用。

最后，第16章中我们学习了使用RNN的序列建模。虽然对深度学习的全面研究远远超出了本书的范围，但我们仍然希望能够激发你的兴趣，并不断地跟踪深度学习领域的最新进展。

如果正在考虑在机器学习方面发展自己的职业生涯，或者想跟上该领域的最新进展，我向你推荐下述机器学习领域的主要专家及其著作。

·杰弗里·希尔顿（http://www.cs.toronto.edu/~hinton/）

·吴恩达（http://www.andrewng.org/）

·烟·卢坤（http://yann.lecun.com）

·泽根·施密德琥珀（http://people.idsia.ch/~juergen/）

·约书华·斑鸠（http://www.iro.umontreal.ca/~bengioy/yoshua_en/）

当然，也请毫不犹豫地加入scikit-learn、TensorFlow和Keras社区的邮件群组，参与关于这些软件库和机器学习的有趣讨论。最后，可以从下述网址探索发现我们正在做些什么：

http://sebastianraschka.com

http://vahidmirjalili.com

如果你对这本书有任何疑问，或者需要了解一些关于机器学习的一般技巧，欢迎随时与我们联系。